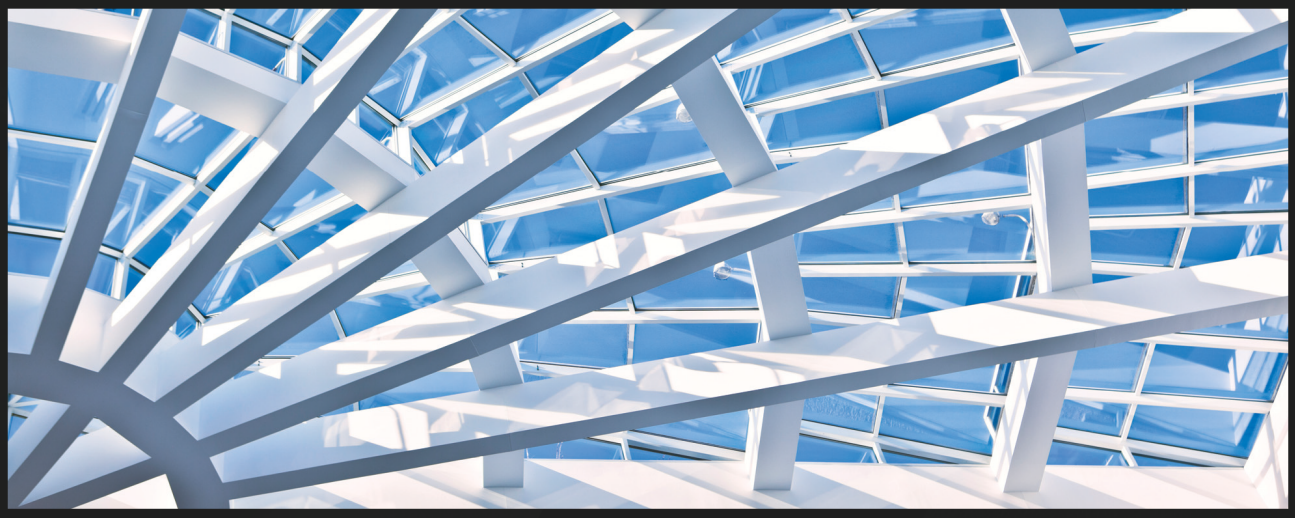


2ND EDITION

Rust Web Programming

A hands-on guide to developing, packaging, and deploying fully functional Rust web applications

MAXWELL FLITTON

Rust Web Programming

A hands-on guide to developing, packaging, and deploying fully functional Rust web applications

Maxwell Flitton

BIRMINGHAM—MUMBAI

More books: https://t.me/booksforits

Rust Web Programming

Copyright © 2023 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Pavan Ramchandani
Publishing Product Manager: Kushal Dave
Senior Editor: Mark D'Souza
Technical Editor: Joseph Aloocaran
Copy Editor: Safis Editing
Project Coordinator: Aishwarya Mohan
Proofreader: Safis Editing
Indexer: Subalakshmi Govindhan
Production Designer: Shyam Sundar Korumilli
Marketing Coordinator: Anamika Singh

First published: February 2021

Second edition: January 2023

Production reference: 1201023

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham
B3 2PB, UK.

ISBN 978-1-80323-469-4

www.packtpub.com

More books: https://t.me/booksforits

This book is dedicated to my newborn son, Henry, who came into this world while it was being written. I also have to say thank you to my wife, Mel, who has supported me throughout the writing of this book, and my mother, Allison Barson, who has always been on my side. I also have to say thank you to Caroline Morton, who has been an amazing business partner and co-software engineer. As a team, we have been able to build entire Kubernetes clusters with Rust microservices and React frontends. I also must acknowledge Charalampos Tsiligiannis, who has helped me and Caroline push our DevOps knowledge and system to another level.

I also have to say thank you again to the Oasis LMF team, who have supported a collaborative work environment. Ben Hayes has facilitated an engineering environment that embraces cutting-edge approaches, innovation, and experimentation. Because of this, I have been able to implement Rust into existing systems for financial loss calculations. Dickie Whitaker has pushed Oasis LMF to become a global influence and his vision and management style have enabled us to approach problems with the bigger picture in mind. My other work colleagues, Marco Tazzari, Sam Gamble, Hassan Chagani, Kamal Charles, Stephane Struzik, Matt Donovan, and Joh Carter, have all facilitated a great working environment.

– Maxwell Flitton

More books: https://t.me/booksforits

Contributors

About the author

Maxwell Flitton is a software engineer who works for the open source financial loss modeling foundation Oasis LMF. In 2011, Maxwell achieved his Bachelor of Science degree in nursing from the University of Lincoln, UK. While working 12-hour shifts in the A&E departments of hospitals, Maxwell obtained another degree in physics from the Open University in the UK, and then moved on to another milestone, getting a postgraduate diploma in physics and engineering in medicine from UCL in London. He's worked on numerous projects, such as medical simulation software for the German government and supervising computational medicine students at Imperial College London. He also has experience in financial tech and Monolith AI.

More books: https://t.me/booksforits

About the reviewers

Bas Zalmstra has worked on games at Abbey Games and robots at Smart Robotics. He is an avid C++ developer but has switched to full-time Rust development. With the power of Rust, programming has never been more fun. He is currently working at Prefix.dev, where he builds package management tools. In his spare time, he works on a Rust-inspired fully hot-reloadable programming language for game development called Mun, obviously built with Rust. Besides development, he was also a technical reviewer for the books *Hands-on Rust* and *Rust Brain Teasers* written by Herbert Wolverson.

Evgeni Pirianov is an experienced senior software engineer with deep expertise in backend technologies, Web3, and Blockchain. Evgeni graduated with a degree in engineering from Imperial College London and has worked for a few years developing non-linear solvers in C++. Ever since then, he has been at the forefront of architecting, designing, and implementing decentralized Web3 applications in the fields of DeFi and Metaverse. Some of the companies he has worked for are Dapper Labs and Kraken. Evgeni's passion for Rust is unsurpassed and he is a true believer in its bright future and wide range of applications.

Yage Hu is a software developer at Abbey Labs, where he uses Rust among various other technologies to tackle security problems. Though he previously worked at fantastic companies such as Uber, Amazon, and Meta, he enjoys cutting code in this currently three-person startup the most. Broadly, his technical interests are low-level systems programming and formal methods. He lives with his lovely wife, Zhiqi, and their scruffy companion, Wangcai, in California. They enjoy hiking, traveling, and building LEGOs.

More books: https://t.me/booksforits

More books: https://t.me/booksforits

Table of Contents

Preface xvii

Part 1: Getting Started with Rust Web Development

1

A Quick Introduction to Rust 3

Technical requirements 3
Why is Rust revolutionary? 4
Reviewing data types and variables in Rust 6
Using strings in Rust 8
Using integers and floats 10
Storing data in vectors and arrays 13
Mapping data with HashMaps 16
Handling results and errors 19
Controlling variable ownership 20
Copying variables 21
Moving variables 22
Immutable borrowing of variables 24
Mutable borrowing of variables 27
Scopes 28
Running through lifetimes 30
Building structs 33
Verifying with traits 38
Metaprogramming with macros 41
Summary 43
Questions 44
Answers 44
Further reading 45

2

Designing Your Web Application in Rust 47

Technical requirements 48
Managing a software project with Cargo 48
Building with Cargo 49
Shipping crates with Cargo 50
Documenting with Cargo 51

More books: https://t.me/booksforits

Interacting with Cargo 54
Structuring code 59
Building to-do structs 59
Managing structs with factories 65
Defining functionality with traits 67
Interacting with the environment 71
Reading and writing JSON files 72
Revisiting traits 75
Processing traits and structs 78
Summary 84
Questions 84
Answers 85

Part 2: Processing Data and Managing Displays

3

Handling HTTP Requests 89

Technical requirements 89
Introducing the Actix Web framework 90
Launching a basic Actix Web server 90
Understanding closures 92
Understanding asynchronous programming 97
Understanding async and await 101
Exploring async and await with web programming 108
Managing views using the Actix Web framework 112
Summary 116
Questions 117
Answers 117
Further reading 117

4

Processing HTTP Requests 119

Technical requirements 119
Getting to know the initial setup for fusing code 120
Passing parameters into views 122
Using macros for JSON serialization 129
Building our own serialization struct 131
Implementing the Serialize trait 133
Integrating serialization structs into our application code 136
Packaging our custom serialized struct to be returned to users 141
Extracting data from views 143
Extracting JSON from the body of a request 145
Extracting data from the header in requests 148
Simplifying header extraction with traits 150
Summary 153
Questions 154
Answers 154

More books: https://t.me/booksforits

5

Displaying Content in the Browser 155

Technical requirements 156
Serving HTML, CSS, and JavaScript using Rust 156
Serving basic HTML 156
Reading basic HTML from files 158
Serving basic HTML loaded from files 159
Adding JavaScript to an HTML file 160
Communicating with our server using JavaScript 160
Injecting JavaScript into HTML 163
Adding the delete endpoint 163
Adding a JavaScript loading function 166
Adding JavaScript tags in the HTML 166
Building a rendering JavaScript function 167
Building an API call JavaScript function 169
Building JavaScript functions for buttons 171
Injecting CSS into HTML 174
Adding CSS tags to HTML 174
Creating a base CSS 175
Creating CSS for the home page 178
Serving CSS and JavaScript from Rust 180
Inheriting components 182
Creating a React app 185
Making API calls in React 187
Creating custom components in React 192
Creating our ToDoItem component 193
Creating custom components in React 195
Constructing and managing custom components in our App component 197
Lifting CSS into React 200
Converting our React application into a desktop application 203
Summary 208
Questions 209
Answers 209
Further reading 209

Part 3: Data Persistence

6

Data Persistence with PostgreSQL 213

Technical requirements 214
Building our PostgreSQL database 214
Why we should use a proper database 214
Why use Docker? 214
How to use Docker to run a database 215
Running a database in Docker 217
Exploring routing and ports in Docker 219
Running Docker in the background with Bash scripts 223
Connecting to PostgreSQL with Diesel 225
Connecting our app to PostgreSQL 230
Creating our data models 232
Getting data from the database 235

More books: https://t.me/booksforits

Inserting into the database 237
Editing the database 239
Deleting data 240
Configuring our application 243
Building a database connection pool 245
Summary 249
Questions 249
Answers 250

7

Managing User Sessions 251

Technical requirements 252
Creating our user model 252
Creating a User data module 252
Creating a NewUser data model 253
Altering the to-do item data model 257
Updating the schema file 260
Creating and running migration scripts on the database 261
Authenticating our users 268
Managing user sessions 275
Cleaning up authentication requirements 278
Configuring expiration of auth tokens 284
Adding authentication into our frontend 287
Summary 294
Questions 295
Answers 295
Further reading 295
Appendix 296

8

Building RESTful Services 299

Technical requirements 300
What are RESTful services? 300
Mapping our layered system 300
Building a uniform interface 305
Implementing statelessness 309
Logging our server traffic 316
Caching 320
Code on demand 324
Summary 325
Questions 325
Answers 326

More books: https://t.me/booksforits

Part 4: Testing and Deployment

9

Testing Our Application Endpoints and Components 331

Technical requirements 332
Building our unit tests 332
Building JWT unit tests 336
Building a configuration for tests 337
Defining the requirements for JWT tests 338
Building basic function tests for JWT 340
Building tests for web requests 342
Writing tests in Postman 345
Writing ordered requests for tests 348
Creating a test for an HTTP request 353
Automating Postman tests with Newman 355
Building an entire automated testing pipeline 360
Summary 365
Questions 366
Answers 366
Further reading 367

10

Deploying Our Application on AWS 369

Technical requirements 370
Setting up our build environment 370
Setting up an AWS SSH key for an AWS EC2 instance 371
Setting up our AWS client 374
Setting up our Terraform build 377
Writing our Python application build script 382
Writing our Bash deployment script 383
Managing our software with Docker 387
Writing Docker image files 387
Building Docker images 388
Building an EC2 build server using Terraform 390
Orchestrating builds with Bash 391
Writing a Docker image file for the React frontend 392
Deploying images onto Docker Hub 395
Deploying our application on AWS 398
Running our application locally 399
Running our application on AWS 405
Writing our application build script 408
Summary 413
Further reading 414

More books: https://t.me/booksforits

11

Configuring HTTPS with NGINX on AWS 415

Technical requirements 416
What is HTTPS? 416
Binary protocol 416
Compressed headers 416
Persistent connections 417
Multiplex streaming 417
Implementing HTTPS locally with docker-compose 419
Attaching a URL to our deployed application on AWS 425
Attaching an elastic IP to our server 426
Registering a domain name 431
Enforcing HTTPS on our application on AWS 437
Getting certificates for our URL 438
Creating multiple EC2 instances 441
Creating a load balancer for our traffic 442
Creating security groups to lock down and secure traffic 445
Updating our Python deployment script for multiple EC2 instances 450
Attaching our URL to the load balancer 450
Summary 453
Further reading 454
Questions 454
Answers 455

Part 5: Making Our Projects Flexible

12

Recreating Our Application in Rocket 459

Technical requirements 460
What is Rocket? 460
Setting up our server 460
Plugging in our existing modules 463
Implementing Rocket traits 467
Plugging in our existing views 472
Accepting and returning JSON 472
Returning raw HTML 474
Returning status with JSON 475
Returning multiple statuses 476
Registering our views with Rocket 477
Plugging in our existing tests 480
Summary 481
Further reading 482
Questions 482
Answers 482

More books: https://t.me/booksforits

13

Best Practices for a Clean Web App Repository 483

Technical requirements 484
The general layout of a clean repository 484
Getting our configuration from environment variables 487
Setting up a local development database 491
Managing variables in Postman tests 495
Building distroless tiny server Docker images 498
Building a clean test pipeline 501
Building continuous integration with GitHub Actions 506
Summary 510
Further reading 511
Questions 511
Answers 511

Part 6: Exploring Protocol Programming and Async Concepts with Low-Level Network Applications

14

Exploring the Tokio Framework 515

Technical requirements 515
Exploring the Tokio framework for async programming 516
Working with workers 520
Exploring the actor model for async programming 523
Working with channels 526
Working with actors in Tokio 529
Summary 536
Further reading 537
Questions 537
Answers 537

15

Accepting TCP Traffic with Tokio 539

Technical requirements 540
Exploring TCP 540

More books: https://t.me/booksforits

Accepting TCP 541
Processing bytes 544
Passing TCP to an actor 547
Keeping track of orders with actors 550
Chaining communication between actors 555
Responding with TCP 557
Sending different commands via the client 559
Summary 561
Further reading 561
Questions 561
Answers 562

16

Building Protocols on Top of TCP 563

Technical requirements 563
Setting up our TCP client and server 564
Setting up our TCP server 564
Setting up our TCP client 566
Processing bytes using structs 567
Creating a message sender client 568
Processing messages in the server 569
Utilizing framing 571
Rewriting our client so that it supports framing 572
Rewriting our server so that it supports framing 574
Building an HTTP frame on top of TCP 576
Summary 579
Further reading 580
Questions 580
Answers 580

17

Implementing Actors and Async with the Hyper Framework 581

Technical requirements 582
Breaking down our project 582
Defining channel messages 584
Building our runner actor 586
Building our state actor 588
Handling HTTP requests using Hyper 593
Building an HTTP server using Hyper 596
Running our Hyper HTTP server 598
Summary 601
Further reading 601

More books: https://t.me/booksforits

18

Queuing Tasks with Redis 603

Technical requirements 604
Breaking down our project 604
Building the HTTP server 606
Building the polling worker 609
Getting our application running with Redis 612
Defining tasks for workers 613
Defining messages for the Redis queue 616
Integrating routing in the HTTP server 618
Running it all in Docker 621
Summary 626
Further reading 626

Index 627

Other Books You May Enjoy 638

More books: https://t.me/booksforits

More books: https://t.me/booksforits

Preface

Do you want to push your web applications to the limit for speed and low energy consumption but still have memory safety? Rust enables you to have memory safety without the garbage collection and a similar energy consumption as the C programming language. This means you can create high-performance and secure applications with relative ease.

This book will take you through each stage of web development resulting in you deploying advanced web applications built in Rust and packaged in distroless Docker, resulting in server images around the size of 50 MB on AWS using automated build and deployment pipes that we created.

You'll start with an introduction to programming in Rust so you can avoid the common pitfalls when migrating from a traditional dynamic programming language. The book will show you how to structure Rust code for a project that spans multiple pages and modules. Next, you'll explore the Actix Web framework and get a basic web server up and running. As you advance, you'll learn how to process JSON requests and display the data from the server via HTML, CSS, and JavaScript and even build a basic React application for our data. You'll also learn how to persist data and create RESTful services in Rust, where we log in and authenticate users and cache data in the frontend. Later, you'll build an automated build and deployment process for the app on AWS over two EC2 instances, where we load balance the HTTPS traffic from a registered domain to our application on those EC2 instances, which we will build using Terraform. You will also lock down the traffic directly to the EC2 instances by configuring security groups in Terraform. You'll then cover multi-layered builds to produce distroless Rust images. Finally, you'll cover advanced web concepts exploring async Rust, Tokio, Hyper, and TCP framing. With these tools, you will implement the actor model to enable you to implement advanced complex async real-time event processing systems, practicing by building a basic stock purchasing system. You'll finish the book by building your own queuing mechanism on Redis, where your own Rust home-built server and worker nodes consume the tasks on the queue and process these tasks.

Who this book is for

This book on web programming with Rust is for web developers who have programmed in traditional languages, such as Python, Ruby, JavaScript, and Java, and are looking to develop high-performance web applications with Rust. Although no prior experience with Rust is necessary, a solid understanding of web development principles and basic knowledge of HTML, CSS, and JavaScript is required if you want to get the most out of this book.

What this book covers

Chapter 1, *A Quick Introduction to Rust*, provides the basics of the Rust programming language.

Chapter 2, *Designing Your Web Application in Rust*, covers building and managing applications in Rust.

More books: https://t.me/booksforits

Chapter 3, *Handling HTTP Requests*, covers building a basic Rust server that handles HTTP requests using the Actix Web framework.

Chapter 4, *Processing HTTP Requests*, covers extracting and handling data from an incoming HTTP request.

Chapter 5, *Displaying Content in the Browser*, covers displaying data from the server and sending requests to the server from the browser with HTML, CSS, and JavaScript with React.

Chapter 6, *Data Persistence with PostgreSQL*, covers managing and structuring data in PostgreSQL and interacting with the database with our Rust web server.

Chapter 7, *Managing User Sessions*, covers authentication and managing of user sessions when making requests to the web server.

Chapter 8, *Building RESTful Services*, covers implementing RESTful concepts for Rust web servers.

Chapter 9, *Testing Our Application Endpoints and Components*, covers end-to-end testing pipelines and unit testing on Rust web servers using Postman.

Chapter 10, *Deploying Our Application on AWS*, covers building automated build and deployment pipelines to deploy on AWS using Docker and automating the infrastructure building with Terraform.

Chapter 11, *Configuring HTTPS with NGINX on AWS*, covers configuring HTTPS and routing to servers via load balancing on AWS with NGINX and routing traffic to different applications depending on endpoints in the URL.

Chapter 12, *Recreating Our Application in Rocket*, covers slotting our existing application into a Rocket web framework.

Chapter 13, *Best Practices for a Clean Web App Repository*, covers cleaning up a web application repository with multi-stage Docker builds for smaller images and init Docker containers to automate database migrations on deployment.

Chapter 14, *Exploring the Tokio Framework*, covers implementing basic async code using the Tokio framework to facilitate an async runtime.

Chapter 15, *Accepting TCP Traffic with Tokio*, covers sending, receiving, and processing TCP traffic.

Chapter 16, *Building Protocols on Top of TCP*, covers processing TCP byte streams into advanced data structures using structs and framing.

Chapter 17, *Implementing Actors and Async with the Hyper Framework*, covers building an async system using the actor framework that accepts HTTP requests via the Hyper framework.

Chapter 18, *Queuing Tasks with Redis*, covers accepting HTTP requests and packaging them as tasks to put on a Redis queue for a pool of workers to process.

More books: https://t.me/booksforits

To get the most out of this book

You will need to know some basic concepts around HTML and CSS. You will also need to have some basic understanding of JavaScript. However, the HTML, CSS, and JavaScript is only needed for displaying data in the browser. If you are just reading the book to build backend API servers in Rust, then knowledge of HTML, CSS, and JavaScript is not needed.

Some basic understanding of programming concepts such as functions and loops will also be needed as these will not be covered in the book.

Software/hardware covered in the book	**Operating system requirements**
Rust	Windows, macOS, or Linux (any)
Node (for JavaScript)	Windows, macOS, or Linux (any)
Python 3	Windows, macOS, or Linux (any)
Docker	Windows, macOS, or Linux (any)
docker-compose	Windows, macOS, or Linux (any)
Postman	Windows, macOS, or Linux (any)
Terraform	Windows, macOS, or Linux (any)

To take full advantage of the deployment on AWS chapters, you will need an AWS account, which might cost if you are not eligible for the Free Tier. However, the builds are automated with Terraform, so spinning up builds and tearing them down will be quick and easy, so you will not need to keep the infrastructure running while working through the book, keeping the costs to a minimum.

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to the copying and pasting of code.

By the of this book, you will have a solid foundation of building and deploying Rust servers. However, it must be noted that Rust is a powerful language. As this book focuses on web programming and deployment, there is scope for improvement in Rust programming after the book. Further reading on Rust is advised after the book to enable you to solve more complex problems.

Download the example code files

You can download the example code files for this book from GitHub at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition`. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing/`. Check them out!

More books: https://t.me/booksforits

Download the color images

We also provide a PDF file that has color images of the screenshots and diagrams used in this book. You can download it here: `https://packt.link/Z1lgk`.

Conventions used

There are a number of text conventions used throughout this book.

`Code in text`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. Here is an example: "This means that we must alter the existing schema for the to-do item table and add a user schema to the `src/schema.rs` file."

A block of code is set as follows:

```
table! {
    to_do (id) {
        id -> Int4,
        title -> Varchar,
        status -> Varchar,
        date -> Timestamp,
        user_id -> Int4,
    }
}
table! {
    users (id) {
        id -> Int4,
        username -> Varchar,
        email -> Varchar,
        password -> Varchar,
        unique_id -> Varchar,
    }
}
```

More books: https://t.me/booksforits

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
[default]
exten => s,1,Dial(Zap/1|30)
exten => s,2,Voicemail(u100)
exten => s,102,Voicemail(b100)
exten => i,1,Voicemail(s0)
```

Any command-line input or output is written as follows:

```
docker-compose up
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: "If we press the **Send** button in Postman another two times before the initial 30 seconds is up, we get the following printout:"

> **Tips or important notes**
> Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at `customercare@packtpub.com` and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit `www.packtpub.com/support/errata` and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `authors.packtpub.com`.

More books: https://t.me/booksforits

Share Your Thoughts

Once you've read *Rust Web Programming - Second Edition*, we'd love to hear your thoughts! Please click here to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

More books: https://t.me/booksforits

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere? Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



`https://packt.link/free-ebook/9781803234694`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

More books: https://t.me/booksforits

More books: https://t.me/booksforits

Part 1: Getting Started with Rust Web Development

Coding in Rust can be challenging. In this part, we cover the basics of Rust programming and how to build applications spanning multiple files in a project with dependencies. By the end of this part, you will be able to build applications in Rust, manage dependencies, navigate the Rust borrow checker, and manage data collections, structs, the basic design structure, and referencing structs.

This part includes the following chapters:

- *Chapter 1*, *A Quick Introduction to Rust*
- *Chapter 2*, *Designing Your Web Application in Rust*

More books: https://t.me/booksforits

More books: https://t.me/booksforits

1

A Quick Introduction to Rust

Rust is growing in popularity, but it has a steep learning curve. By covering the basic rules around Rust, as well as learning how to manipulate a range of data types and variables, we will be able to write simple programs in the same fashion as dynamically-typed languages using a similar number of lines of code.

The goal of this chapter is to cover the main differences between Rust and generic dynamic languages and to provide you with a quick understanding of how to utilize Rust.

In this chapter, we will cover the following topics:

- Why is Rust revolutionary?
- Reviewing data types and variables in Rust
- Controlling variable ownership
- Building structs
- Metaprogramming with macros

Once we have covered the main concepts in this chapter, you will be able to code basic programs in Rust that will run. You will also be able to debug your programs and understand the error messages that are thrown by the Rust compiler. As a result, you will have the foundations to be productive in Rust. You will also be able to move on to structuring Rust code over multiple files.

Technical requirements

For this chapter, we only need access to the internet as we will be using the online Rust playground to implement the code. The code examples provided can be run in the online Rust playground at `https://play.rust-lang.org/`.

For detailed instructions, please refer to the file found here: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter01`

More books: https://t.me/booksforits

Why is Rust revolutionary?

With programming, there is usually a trade-off between speed and resources and development speed and safety. Low-level languages such as C/C++ can give the developer fine-grained control over the computer with fast code execution and minimal resource consumption. However, this is not free. Manual memory management can introduce bugs and security vulnerabilities. A simple example of this is a buffer overflow attack. This occurs when the programmer does not allocate enough memory. For instance, if the buffer only has a size of 15 bytes, and 20 bytes are sent, then the excess 5 bytes might be written past the boundary. An attacker can exploit this by passing in more bytes than the buffer can handle. This can potentially overwrite areas that hold executable code with their own code. There are other ways to exploit a program that does not have correctly managed memory. On top of increased vulnerabilities, it takes more code and time to solve a problem in a low-level language. As a result of this, C++ web frameworks do not take up a large share of web development. Instead, it usually makes sense to go for high-level languages such as Python, Ruby, and JavaScript. Using such languages will generally result in the developer solving problems safely and quickly.

However, it must be noted that this memory safety comes at a cost. These high-level languages generally keep track of all the variables defined and their references to a memory address. When there are no more variables pointing to a memory address, the data in that memory address gets deleted. This process is called garbage collection and consumes extra resources and time as a program must be stopped to clean up the variables.

With Rust, memory safety is ensured without the costly garbage collection process. Rust ensures memory safety through a set of ownership rules checked at compile time with a borrow checker. These rules are the quirks mentioned in the following section. Because of this, Rust enables rapid, safe problem-solving with truly performant code, thus breaking the speed/safety trade-off.

> **Memory safety**
>
> Memory safety is the property of programs having memory pointers that always point to valid memory.

With more data processing, traffic, and complex tasks lifted into the web stack, Rust, with its growing number of web frameworks and libraries, has now become a viable choice for web development. This has led to some truly amazing results in the web space for Rust. In 2020, Shimul Chowdhury ran a series of tests against servers with the same specs but different languages and frameworks. The results can be seen in the following figure:

More books: https://t.me/booksforits

Benchmark Result

20 threads with 1000 concurrent request for 60 seconds, using https://github.com/wg/wrk.

	Flask	Falcon	Actix-web	Rocket	NestJS
Total Requests	11,222	14,802	95,333	1,02,441	30,913
Req/Sec	186.74	246.32	1586.27	1704.35	514.38
Total Read	6.84MB	10.46MB	105.95MB	162.43MB	8.82MB
Transfer/sec	116.53KB	178.21KB	1.76MB	1.24MB	150.21KB
Request Timeout	10,496	1955	3729	4184	4676

As expected Rust frameworks are way faster than others. But at my surprise Flask did pretty bad. NestJS also looks promising. We could potentially speed up more by using Fastify instead of Express.

Figure 1.1 – Results of different frameworks and languages by Shimul Chowdhury (found at https://www.shimul.dev/posts/06-04-2020-benchmarking-flask-falcon-actix-web-rocket-nestjs/)

In the preceding figure, we can see that there are some variations in the languages and frameworks. However, we must note that the Rust frameworks comprise Actix Web and Rocket. These Rust servers are in a completely different league when it comes to total requests handled and data transferred. Other languages, such as Golang, have come onto the scene, but the lack of garbage collection in Rust has managed to outshine Golang. This was demonstrated in Jesse Howarth's blog post *Why Discord is switching from Go to Rust*, where the following graph was published:

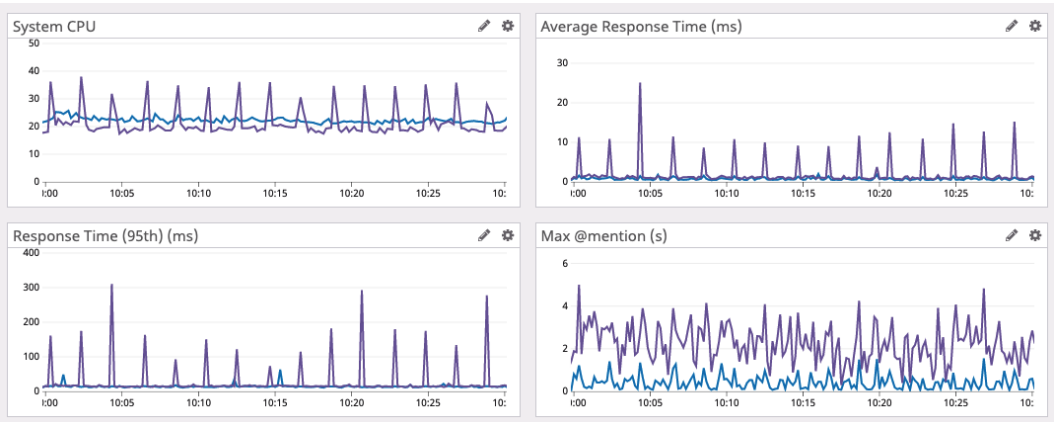


Figure 1.2 – Discord's findings => Golang is spikey and Rust is smooth (found at https://discord.com/blog/why-discord-is-switching-from-go-to-rust)

The garbage collection that Golang was implementing to keep the memory safe resulted in 2-minute spikes. This is not to say that we should use Rust for everything. It is best practice to use the right tool for the job. All these languages have different merits. What we have done in the preceding figure is merely display Rust's merits.

More books: https://t.me/booksforits

The lack of need for garbage collection is because Rust uses enforced rules to ensure memory safety using the borrow checker. Now that we have understood why we want to code in Rust, we can move on to reviewing data types in the next section.

Reviewing data types and variables in Rust

If you have coded in another language before, you will have used variables and handled different data types. However, Rust does have some quirks that can put off developers. This is especially true if the developer has come from a dynamic language, as these quirks mainly revolve around memory management and reference to variables. These can be intimidating initially, but when you get to understand them, you will learn to appreciate them. Some people might hear about these quirks and wonder why they should bother with the language at all. This is understandable, but these quirks are why Rust is such a paradigm-shifting language. Working with borrow checking and wrestling with concepts such as lifetimes and references gives us the high-level memory safety of a dynamic language such as Python. However, we can also get memory safe low-level resources such as those delivered by C and C++. This means that we do not have to worry about dangling pointers, buffer overflows, null pointers, segmentation faults, data races, and other issues when coding in Rust. Issues such as null pointers and data races can be hard to debug. With this in mind, the rules enforced are a good trade-off as we must learn about Rust's quirks to get the speed and control of non-memory safe languages, but we do not get the headaches these non-memory-safe languages introduce.

Before we do any web development, we need to run our first program. We can do this in the Rust playground at `https://play.rust-lang.org/`.

If you have never visited the Rust playground before, you will see the following layout once you are there:

```
fn main() {
    println!("hello world");
}
```

The preceding code will look like the following screenshot when it comes to using the online Rust playground:

More books: https://t.me/booksforits

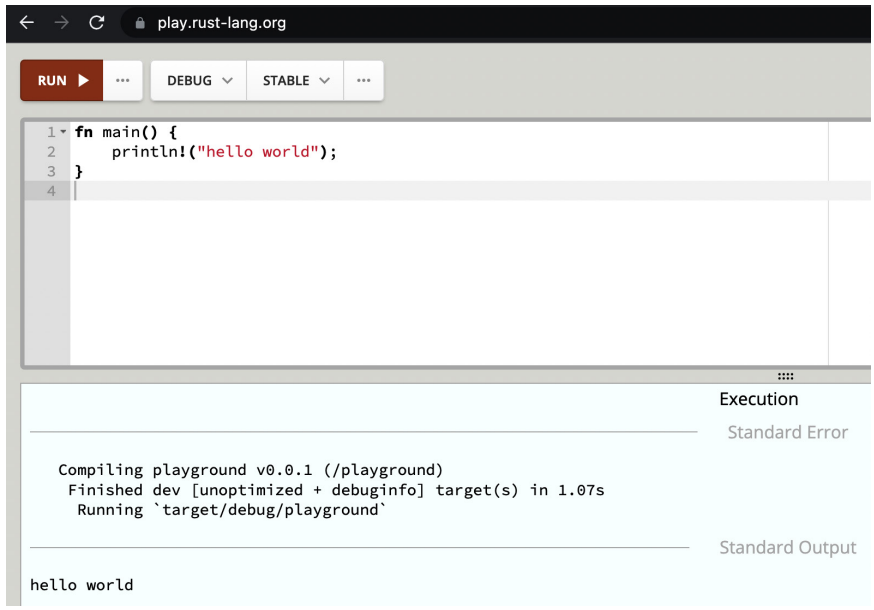


Figure 1.3 – View of the online Rust playground

In our `hello world` code, what we have is a `main` function, which is our entry point. This function fires when we run our program. All programs have entry points. If you have not heard of the concept before, due to coming from a dynamic language, the entry point is the script file that you point your interpreter at. For Python, a closer analogy would be the `main` block that runs if the file is directly run by the interpreter, denoted as follows:

```
if __name__ == "__main__":
    print("Hello, World!")
```

If you were to code in Python, you would probably see this used in a Flask application. Right now, we have not done anything new. This is a standard *Hello World* example with a little change in syntax; however, even with this example, the string that we are printing is not all that it seems. For instance, let us write our own function that accepts a string and prints it out with the following code:

```
fn print(message: str) {
    println!("{}", message);
}
fn main() {
    let message = "hello world";
    print(message);
}
```

More books: https://t.me/booksforits

This code should just work. We pass it into our function and print it. However, if we do print it, we get the following printout:

```
10 |     print(message);
   |           ^^^^^^^ doesn't have a size known at compile-
time
   |
   = help: the trait `Sized` is not implemented for `str`
   = note: all function arguments must have a statically known
size
```

This is not very straightforward, but it brings us to the first area we must understand if we are to code in Rust, and this is strings. Don't worry, strings are the quirkiest variable that you need to get your head around to write functional Rust code.

Using strings in Rust

Before we explore the error in the previous section, let us rectify it, so we know what to work toward. We can get the `print` function to work without any errors with the following code:

```
fn print(message: String) {
    println!("{}", message);
}
fn main() {
    let message = String::from("hello world");
    print(message);
}
```

What we did was create a `String` from `"hello world"` and passed the `String` into the `print` function. This time the compiler did not throw an error because we always know the size of a `String`, so we can keep the right amount of memory free for it. This may sound counterintuitive because strings are usually of different lengths. It would not be a very flexible programming language if we were only allowed to use the same length of letters for every string in our code. This is because strings are essentially pointers implemented as a vector of bytes, which, in Rust, is denoted as `Vec<u8>`. This holds a reference to the string content (`str`, also known as a string slice) in the heap memory, as seen in the following figure:

More books: https://t.me/booksforits

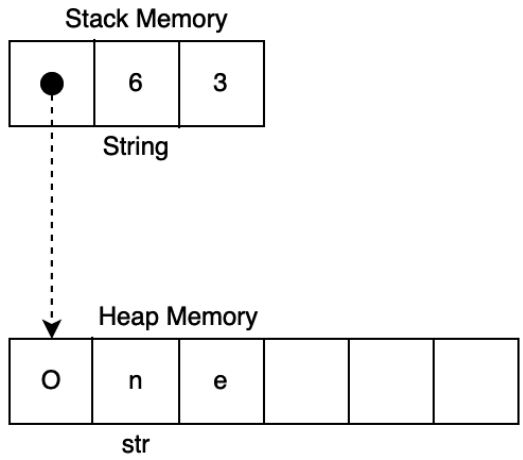


Figure 1.4 – A string's relationship to str "one"

We can see, in *Figure 1.4*, that a string is a vector of three numbers. One is the actual memory address of the `str` it is referencing. The second number is the size of the memory allocated, and the third is the length of the string content. Therefore, we can access string literals in our code without having to pass variables of various sizes around our code. We know that `String` has a set size and, therefore, can allocate this size in the `print` function. It also must be noted that `String` is on the stack memory while our string literal is on the heap memory. Considering that we know that `String` has a set size while our string literal varies, we can deduce that the stack memory is used for predictable memory sizes and is allocated ahead of time when the program runs. Our heap memory is dynamic, and therefore memory is allocated when it is needed. Now that we know the basics of strings, we can use the different ways in which they are created, as seen in the following code:

```
    let string_one = "hello world".to_owned();
    let string_two = "hello world".to_string();
    let string_three = string_two.clone();
```

We must note, however, that creating `string_three` is expensive as we must copy the underlying data in the heap, and heap operations are expensive. This is not a unique quirk of Rust. In our example, we are just experiencing what happens under the hood. For instance, if we alter strings in Python, we will have different outcomes:

```
# slower method
data = ["one", "two", "three", "four"]
string = ""
```

More books: https://t.me/booksforits

```
for i in data:
    string += i
# faster method
"".join(data)
```

Looping through and adding the strings is slower because Python must allocate new memory and copy the entire string to that new memory address. The `join` method is faster because Python can allocate the memory of all the data of the list and then copy over the strings in the array, meaning the string must only be copied once. This shows us that although high-level languages like Python may not force you to think about the memory allocation of strings, you will still end up paying the price if you don't acknowledge it.

We can also pass a string literal into the `print` function by borrowing it, as seen in the following code:

```
fn print(message: &str) {
    println!("{}", message);
}
fn main() {
    print(&"hello world");
}
```

The borrow is denoted by `&`. We will go into borrowing later in the chapter. For now, however, we can deduce that the borrow is only a fixed-size reference to a variable-sized string slice. If the borrow was a fixed size, we would not be able to pass it into the `print` function because we would not know the size. At this point, we can comfortably use strings in Rust productively. The next concept that we must understand before we start writing Rust programs is integers and floats.

Using integers and floats

In most high-level web programming languages, we merely assign a float or integer to a variable name and move on with the program. However, from what we have been exposed to in the previous section on strings, we now understand that we must worry about memory size when using strings in Rust. This is no different with integers and floats. We know that integers and floats have a range of sizes. Therefore, we must tell Rust what we are passing around our code. Rust supports signed integers, which are denoted by `i`, and unsigned integers, which are denoted by `u`. These integers consist of 8, 16, 32, 64, and 128 bits. Exploring the math behind numbers being represented in binary is not relevant for this book; however, we do need to understand the range of numbers that can be represented with several bits, as this will help us understand what the different types of floats and integers in Rust denote. Because binary is either a 0 or a 1, we can calculate the range of integers that can be represented by the bits by raising 2 to the power of the number of bits we have. For example, if we have an integer

More books: https://t.me/booksforits

that is represented by 8 bits, 2 to the power of 8 equates to 256. We must remember that 0 is also represented. Considering this, an integer of 8 bits has a range of 0 to 255. We can test this calculation with the following code:

```
let number: u8 = 256;
```

This is one higher than the range that we calculated. As a result, we should not be surprised to see the overflow error as follows:

```
the literal `256` does not fit into the type
`u8` whose range is `0..=255`
```

So, we can deduce that if we lower the unsigned integer to `255`, it will pass. However, let's say we change the unsigned integer into a signed integer with the following code:

```
let number: i8 = 255;
```

We will see that we get a helpful error message as follows:

```
the literal `255` does not fit into the type
`i8` whose range is `-128..=127`
```

With this helpful error message, we can see that a signed integer considers negative numbers, so the absolute value that a signed integer can take is roughly half. Therefore, we can increase the range by assigning the number as a 16-bit signed integer with the following code:

```
let number: i16 = 255;
```

This would work. However, let us add our 16-bit integer with our 8-bit integer using the following code:

```
let number = 255i16;
let number_two = 5i8;
let result = number + number_two;
```

The previous code might look a little different to you. All we have done in the preceding code is define the data type with a suffix instead. So, `number` has a value of `255` and a type of `i16`, and `number_two` has a value of `5` and a type of `i8`. If we run the previous code, we get the following error:

```
11 |     let result = number + number_two;
   |                          ^ no implementation for `i16 + i8`
   |
   = help: the trait `Add<i8>` is not implemented for `i16`
```

More books: https://t.me/booksforits

We will cover traits later in this chapter. For now, all we must understand is that we cannot add the two different integers. If they were both the same type, then we could. We can change the integer type through casting using `as`, as seen in the following line of code:

```
let result = number + number_two as i16;
```

This means that `number_two` is now a 16-bit integer, and `result` will be 260. However, we must be careful with casting because if we were to do it the wrong way, we could end up with a silent bug, which is unusual for Rust. If we cast `number` as `i8` instead of casting `number_two` as `i16`, then `result` would equate to 4, which does not make sense because 255 + 5 equals 260. This is because `i8` is smaller than `i16`. So, if we cast an `i16` integer as an `i8` integer, we are essentially chopping off some of the data, by just taking the lower bits of the number and disregarding the upper bits. Therefore, `number` ends up being -1 if we cast it to an `i8` integer. To be safer, we can use the `i8::from` function, as seen in the following code:

```
let result = i8::from(number) + number_two;
```

Running this will give us the following error:

```
let result = i8::from(number) + number_two;
|                  ^^^^^^^^ the trait `From<i16>` is not
                            implemented for `i8`
```

Again, we will go over traits later on in the chapter, but we can see in the preceding code that because the `From<i16>` trait is not implemented for an `i8` integer, we cannot cast an `i8` integer into an `i16` integer. With this understood, we are free to work with integers safely and productively. One last point about integer sizes in Rust is that they are not continuous. The supported sizes are shown in the following table:

Bits	Calculation	Size
8	2^8	256
16	2^16	65536
32	2^32	4294967296
64	2^64	1.8446744e+19
128	2^128	3.4028237e+38

Table 1.1 – Size of integer types

When it comes to floats, Rust accommodates `f32` and `f64` floating-point numbers. Both these floating-point types support negative and positive values. Declaring a floating-point variable requires the same syntax as integers, as seen in the following code:

```
let float: f32 = 2.6;
```

More books: https://t.me/booksforits

With this, we can comfortably work with integers and floats in our Rust code. However, we know as developers that just declaring floats and integers is not very useful. We want to be able to contain and loop through them. In the next section, we will do just that with vectors and arrays.

Storing data in vectors and arrays

In Rust, we can store our floats, integers, and strings in arrays and vectors. First, we will focus on arrays. Arrays are stored on stack memory. Knowing this, and remembering what we learned about strings, we can deduce that arrays are of a fixed size. This is because, as we remember, if the variable is stored on the stack, then the memory is allocated and loaded into the stack when the program starts. We can define an array of integers, loop through it, print each integer, and then access an integer by index with the following code:

```
fn main() {
    let int_array: [i32; 3] = [1, 2, 3];
    for i in int_array {
        println!("{}", i);
    }
    println!("{}", int_array[1]);
}
```

With the previous code, we define the type and size by wrapping them in square brackets. For instance, if we were going to create an array of floats with a length of 4, we would use `int_array: [f32; 4] = [1.1, 2.2, 3.3, 4.4]`. Running the preceding code will give you the following printout:

```
1
2
3
2
```

In the preceding printout, we see that the loop works and that we can access the second integer with square brackets. Although the memory size of the array is fixed, we can still change it. This is where mutability comes in. When we define a variable as mutable, this means that we can mutate it. In other words, we can alter the value of the variable after it has been defined if it is mutable. If you tried to update any of the variables in the code that we have written in this chapter, you will have realized that you can't. This is because all variables in Rust are immutable by default. We can make any variable in Rust mutable by putting a `mut` tag in front of the variable name. Going back to the fixed array, we cannot change the size of the array, meaning we cannot append/push new integers to it due to it being stored on stack memory. However, if we define a mutable array, we can update parts of it with other integers that are the same memory size. An example of this is the following code:

More books: https://t.me/booksforits

```
fn main() {
    let mut mutable_array: [i32; 3] = [1, 2, 0];
    mutable_array[2] = 3;
    println!("{:?}", mutable_array);
    println!("{}", mutable_array.len());
}
```

In the preceding code, we can see that the last integer in our array is updated to `3`. We then print out the full array and then print out the length. You may have also noted that the first print statement of the preceding code now employs `{:?}`. This calls the `Debug` trait. If `Debug` is implemented for the thing that we are trying to print, then the full representation of the thing we are printing is then displayed in the console. You can also see that we print out the result of the length of the array. Running this code will give the following printout:

```
[1, 2, 3]
3
```

With the preceding printout, we can confirm that the array is now updated. We can also access slices with our arrays. To demonstrate this, we can create an array of 100 zeros. We can then take a slice of this and print it out with the following code:

```
fn main() {
    let slice_array: [i32; 100] = [0; 100];
    println!("length: {}", slice_array.len());
    println!("slice: {:?}", &slice_array[5 .. 8]);
}
```

Running the preceding code will result in the following printout:

```
length: 100
slice: [0, 0, 0]
```

We are now able to be productive with arrays. Arrays can be useful for caching. For instance, if we know the amount that we need to store, then we can use arrays effectively. However, we have only managed to store one type of data in the array. If we tried to store strings and integers in the same array, we would have a problem. How would we define the type? This problem goes for all collections, such as vectors and **HashMaps**. There are multiple ways to do this, but the most straightforward is using enums. Enums are, well, enums. In dynamic languages such as Python, you may not have had to use them due to being able to pass any type anywhere you want. However, they are still available. Enum is short for enumerated type and basically defines a type with possible variants. In our case, we want our array to store strings and integers in the same collection. We can do this by initially defining our enum with the following code:

More books: https://t.me/booksforits

```
enum SomeValue {
    StringValue(String),
    IntValue(i32)
}
```

In the preceding code, we can see that we defined an enum with the name of `SomeValue`. We then denoted that `StringValue` holds the value of a string and that `IntValue` holds the value of an integer. We can then define an array with a length of `4`, consisting of 2 strings and 2 integers, with the following code:

```
    let multi_array: [SomeValue; 4] = [
        SomeValue::StringValue(String::from("one")),
        SomeValue::IntValue(2),
        SomeValue::StringValue(String::from("three")),
        SomeValue::IntValue(4)
    ];
```

In the preceding code, we can see that we wrap our strings and integers in our enum. Now, looping through and getting it out is going to be another task. For instance, there are things that we can do to an integer that we cannot do to a string and vice versa. Considering this, we are going to have to use a `match` statement when looping through the array, as seen in the following code:

```
    for i in multi_array {
        match i {
            SomeValue::StringValue(data) => {
                println!("The string is: {}", data);
            },
            SomeValue::IntValue(data) => {
                println!("The int is: {}", data);
            }
        }
    }
```

In the preceding code, we can see that if `i` is `SomeValue::StringValue`, we then assign the data wrapped in `SomeValue::StringValue` to the variable named `data`. We then pass `data` into the inner scope to be printed. We use the same approach with our integer. Even though we are merely printing to demonstrate the concept, we can do anything in these inner scopes to the `data` variable that the type allows us to. Running the preceding code gives the following printout:

```
The string is: one
The int is: 2
```

More books: https://t.me/booksforits

```
The string is: three
The int is: 4
```

Using enums to wrap data and `match` statements to handle them can be applied to HashMaps and vectors. Also, what we have covered with arrays can be applied to vectors. The only difference is that we do not have to define the length and that we can increase the size of the vector if needed. To demonstrate this, we can create a vector of strings and then add a string to the end with the following code:

```
    let mut string_vector: Vec<&str> = vec!["one", "two",
        "three"];
    println!("{:?}", string_vector);
    string_vector.push("four");
    println!("{:?}", string_vector);
```

In the preceding code, we can see that we use the `vec!` macro to create the vector of strings. You may have noticed with macros such as `vec!` and `println!` that we can vary the number of inputs. We will cover macros later in the chapter. Running the preceding code will result in the following printout:

```
["one", "two", "three"]
["one", "two", "three", "four"]
```

We can also create an empty vector with the `new` function from the `Vec` struct with `let _empty_vector: Vec<&str> = Vec::new();`. You may be wondering when to use vectors and when to use arrays. Vectors are more flexible. You may be tempted to reach for arrays for performance gains. At face value, this seems logical as it is stored in the stack. Accessing the stack is going to be quicker because the memory sizes can be computed at compile time, making the allocation and deallocation simpler compared to the heap. However, because it is on the stack it cannot outlive the scope that it is allocated. Moving a vector around would merely require moving a pointer around. However, moving an array requires copying the whole array. Therefore, copying fixed-size arrays is more expensive than moving a vector. If you have a small amount of data that you only need in a small scope and you know the size of the data, then reaching for an array does make sense. However, if you're going to be moving the data around, even if you know the size of the data, using vectors is a better choice. Now that we can be productive with basic collections, we can move on to a more advanced collection, a HashMap.

Mapping data with HashMaps

In some other languages, HashMaps are referred to as dictionaries. They have a key and a value. We can insert and get values using the key. Now that we have learned about handling collections, we can get a little more adventurous in this section. We can create a simple profile of a game character. In this character profile, we are going to have a name, age, and a list of items that they have. This means that

More books: https://t.me/booksforits

we need an enum that houses a string, an integer, and a vector that also houses strings. We will want to print out the complete HashMap to see whether our code is correct in one glance. To do this, we are going to implement the `Debug` trait for our enum, as seen in the following code:

```
#[derive(Debug)]
enum CharacterValue {
    Name(String),
    Age(i32),
    Items(Vec<String>)
}
```

In the preceding code, we can see that we have annotated our enum with the `derive` attribute. An attribute is metadata that can be applied to the `CharacterValue` enum in this case. The `derive` attribute tells the compiler to provide a basic implementation of a trait. So, in the preceding code, we are telling the compiler to apply the basic implementation of `Debug` to the `CharacterValue` enum. With this, we can then create a new HashMap that has keys pointing to the values we defined with the following code:

```
use std::collections::HashMap;
fn main() {
    let mut profile: HashMap<&str, CharacterValue> =
                     HashMap::new();
}
```

We stated that it is mutable because we are going to insert values with the following code:

```
profile.insert("name", CharacterValue::Name("Maxwell".to_
string()));
profile.insert("age", CharacterValue::Age(32));
profile.insert("items", CharacterValue::Items(vec![
    "laptop".to_string(),
    "book".to_string(),
    "coat".to_string()
]));
println!("{:?}", profile);
```

We can see that we have inserted all the data that we need. Running this would give us the following printout:

```
{"items": Items(["laptop", "book", "coat"]), "age": Age(32),
"name": Name("Maxwell")}
```

More books: https://t.me/booksforits

In the preceding output, we can see that our data is correct. Inserting it is one thing; however, we now must get it out again. We can do this with a `get` function. The `get` function returns an `Option` type. The `Option` type returns either `Some` or `None`. So, if we were to get `name` from our HashMap, we would need to do two matches, as seen in the following code:

```
    match profile.get("name") {
        Some(value_data) => {
            match value_data {
                CharacterValue::Name(name) => {
                    println!("the name is: {}", name);
                },
                _ => panic!("name should be a string")
            }
        },
        None => {
            println!("name is not present");
        }
    }
```

In the preceding code, we can check to see if there is a name in the keys. If there is not, then we just print out that it was not present. If the `name` key is present, we then move on to our second check, which prints out the name if it is `CharacterValue::Name`. However, there is something wrong if the `name` key is not housing `CharacterValue::Name`. So, we add only one more check in `match`, which is `_`. This is a catch meaning `anything else`. We are not interested in anything other than `CharacterValue::Name`. Therefore, the `_` catch maps to a `panic!` macro, which essentially throws an error. We could make this shorter. If we know that the `name` key is going to be in the HashMap, we can employ the `unwrap` function with the following code:

```
    match profile.get("name").unwrap() {
        CharacterValue::Name(name) => {
            println!("the name is: {}", name);
        },
        _ => panic!("name should be a string")
    }
```

The `unwrap` function directly exposes the result. However, if the result is `None`, then it will directly result in an error terminating the program, which would look like the following printout:

```
thread 'main' panicked at 'called `Option::unwrap()` on a
`None` value'
```

More books: https://t.me/booksforits

This might seem risky, but in practice, you will end up using the `unwrap` function a lot because you need direct access to the result, and you cannot carry on the program without it anyway. A prime example is connecting to a database. In a lot of web programming, if the database connection is not successful, then you cannot carry on with the API call. Therefore, it makes sense to just allow an error like most other web languages would. Now that we have been exposed to errors terminating the program, we might as well learn how to handle errors in the next section.

Handling results and errors

In the previous section, we learned that directly unwrapping `Option` resulting in `None` panics a thread. There is another outcome that can also throw an error if unsuccessfully unwrapped, and this is `Result`. The `Result` type can return either `Ok` or `Err`. To demonstrate this, we can create a basic function that returns a `Result` type based on a simple Boolean we pass into it with the following code:

```
fn error_check(check: bool) -> Result<i8, &'static str> {
    if check {
        Err("this is an error")
        }
    else {
        Ok(1)
    }
}
```

In the preceding code, we can see that we return `Result<i8, &'static str>`. This means that we return an integer if `Result` is `Ok`, or we return an integer if `Result` is `Err`. The `&'static str` variable is basically our error string. We can tell it's a reference because of `&`. The `'static` part means that the reference is valid for the entire lifetime of the running program. If this does not make sense now, do not worry, we will be covering lifetimes later in the chapter. Now that we have created our error-checking function, we can test to see what these outcomes look like with the following code:

```
fn main() {
    println!("{:?}", error_check(false));
    println!("{:?}", error_check(false).is_err());
    println!("{:?}", error_check(true));
    println!("{:?}", error_check(true).is_err());
}
```

Running the preceding code gives us the following printout:

```
Ok(1)
false
```

More books: https://t.me/booksforits

```
Err("this is an error")
true
```

In the preceding output, we can see that it returned exactly what we wanted. We can also note that we can run the `is_err()` function on the `Result` variable, resulting in `false` if returning `Ok` or `true` if returning `Err`. We can also directly unwrap but add extra tracing to the stack trace with the following `expect` function:

```
let result: i8 = error_check(true).expect("this has been
caught");
```

The preceding function will result in the following printout:

```
thread 'main' panicked at 'this has been caught: "this is an
error"'
```

Through the preceding example, we can see that we get the message from the `expect` function first, and then the error message returned in `Result`. With this understanding, we can throw, handle, and add extra tracing to errors. However, we are getting more exposed to lifetimes and borrow references as we move forward. Now is the time to address this by understanding variable ownership.

Controlling variable ownership

As we remember from the beginning of the chapter, Rust does not have a garbage collector. However, it has memory safety. It achieves this by having strict rules around variable ownership. These rules are enforced when Rust is being compiled. If you are coming from a dynamic language, then this can initially lead to frustration. This is known as *fighting the borrow checker*. Sadly, this unjustly gives Rust the false *steep learning curve* reputation, as when you are fighting the borrow checker without knowing what is going on, it can seem like an impossible task to get even the most basic programs written. However, if we take the time to learn the rules before we try and code anything too complex, the knowledge of the rules and the helpfulness of the compiler will make writing code in Rust fun and rewarding. Again, I take the time to remind you that Rust has been the most favorited language 7 years in a row. This is not because it's impossible to get anything done in it. The people who vote for Rust in these surveys understand the rules around ownership. Rust's compiling, checking, and enforcing of these rules protect against the following errors:

- **Use after frees**: This occurs when memory is accessed once it has been freed, which can cause crashes. It can also allow hackers to execute code via this memory address.
- **Dangling pointers**: This occurs when a reference points to a memory address that no longer houses the data that the pointer was referencing. Essentially, this pointer now points to null or random data.

More books: https://t.me/booksforits

- **Double frees**: This occurs when allocated memory is freed and then freed again. This can cause the program to crash and increases the risk of sensitive data being revealed. This also enables a hacker to execute arbitrary code.
- **Segmentation faults**: This occurs when the program tries to access the memory it's not allowed to access.
- **Buffer overrun**: An example of this error is reading off the end of an array. This can cause the program to crash.

To protect against these errors and thus achieve memory safety, Rust enforces the following rules:

- Values are owned by the variables assigned to them
- As soon as the variable moves out of the scope of where it was defined, it is then deallocated from the memory
- Values can be referenced and altered if we adhere to the rules for copying, moving, immutable borrowing, and mutable borrowing

Knowing the rules is one thing but, to practically work with the rules in Rust code, we need to understand copying, moving, and borrowing in more detail.

Copying variables

Copying occurs when a value is copied. Once it has been copied, the new variable owns the value, while the existing variable also owns its own value.

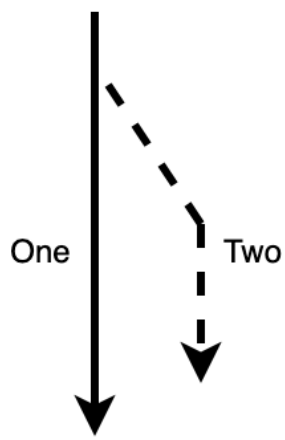


Figure 1.5 – Variable copy path

More books: https://t.me/booksforits

In *Figure 1.5*, we can see that the path of **One** is still solid, which denotes that it has not been interrupted and can be handled as if the copy did not happen. Path **Two** is merely a copy, and there is also no difference in the way in which it can be utilized as if it were self-defined. It must be noted that if the variable has a `copy` trait, then it will automatically be copied, as seen in the following code:

```
let one: i8 = 10;
let two: i8 = one + 5;
println!("{}", one);
println!("{}", two);
```

Running the preceding code will give us the following printout:

```
10
15
```

In the preceding example, we appreciate that the very fact that variables `one` and `two` can be printed indicates that `one` has been copied for `two` to utilize. To test this, we can test our example with strings using the following code:

```
let one = "one".to_string();
let two = one;
println!("{}", one);
println!("{}", two);
```

Running this code will result in the following error:

```
move occurs because `one` has type `String`, which does not
implement the `Copy` trait
```

Because strings do not implement the `Copy` trait, the code does not work, as `one` was moved to `two`. However, the code will run if we get rid of `println!("{}", one);`. This brings us to the next concept that we must understand: moving.

Moving variables

Moving refers to when the value is moved from one variable to another. However, unlike copying, the original variable no longer owns the value.

More books: https://t.me/booksforits

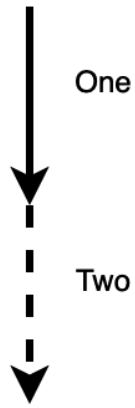


Figure 1.6 – Variable move path

From what we can see in *Figure 1.6*, `one` can no longer be accessed once it's moved to `two`. To really establish what is going on here and how strings are affected, we can set up some code designed to fail as follows:

```
let one: String = String::from("one");
let two: String = one + " two";
println!("{}", two);
println!("{}", one);
```

Running the preceding code gives the following error:

```
let one: String = String::from("one");
    --- move occurs because `one` has type
    `String`, which does not implement the
    `Copy` trait
let two: String = one + " two";
                  ------------ `one` moved due to usage in
operator
println!("{}", two);
println!("{}", one);
               ^^^ value borrowed here after move
```

As we can see, the compiler has been helpful here. It shows us where the string was moved to and where the value of that string is borrowed. So, we can make the code run instantly by merely removing the

More books: https://t.me/booksforits

`println!("{}", one);` line. However, we want to be able to use that `print` function at the bottom of the preceding code block. We should not have to constrain the functionality of the code due to the rules implemented by Rust. We can solve this by using the `to_owned` function with the following code:

```
let two: String = one.to_owned() + " two";
```

The `to_owned` function is available because strings implement the `ToOwned` trait. We will cover traits later in the chapter, so do not halt your reading if you do not know what this means yet. We could have used `clone` on the string. We must note that `to_owned` is a generalized implementation of `clone`. However, it does not really matter which approach we use. It is understandable to wonder why strings do not have the `Copy` trait. This is because the string is a pointer to a string literal. If we were to copy strings, we would have multiple unconstrained pointers to the same string literal data, which would be dangerous. Because of this, we can explore the move concept using strings. If we force our string outside of the scope with a function, we can see how this affects our move. This can be done with the following code:

```
fn print(value: String) {
    println!("{}", value);
}
fn main() {
    let one = "one".to_string();
    print(one);
    println!("{}", one);
}
```

If we run the preceding code, we will get an error stating that the `print` function moved the `one` value. As a result, the `println!("{}", one);` line borrows `one` after it is moved into the `print` function. The key part of this message is the word *borrow*. To understand what is going on, we need to explore the concept of immutable borrowing.

Immutable borrowing of variables

An immutable borrow occurs when a variable can be referenced by another variable without having to clone or copy it. This essentially solves our problem. If the borrowed variable falls out of scope, then it is not deallocated from the memory and the original reference to the value can still be used.

More books: https://t.me/booksforits

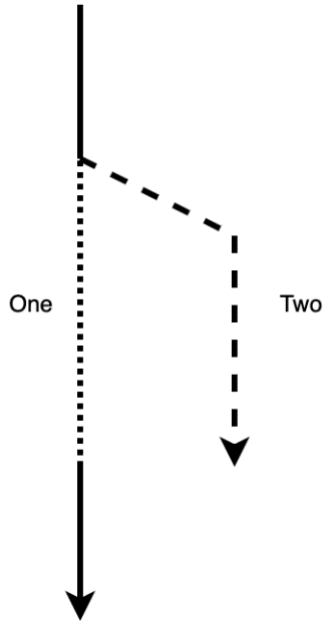


Figure 1.7 – Immutable borrow path

We can see in *Figure 1.7* that `two` borrows the value from `one`. It must be noted that when `one` is borrowed from, `one` is locked and cannot be accessed until the borrow is finished. To perform a borrow operation, we merely apply a prefix with `&`. This can be demonstrated with the following code:

```
fn print(value: &String) {
    println!("{}", value);
}
fn main() {
    let one = "one".to_string();
    print(&one);
    println!("{}", one);
}
```

In the preceding code, we can see that our immutable borrow enables us to pass a string into the `print` function and still print it afterward. This can be confirmed with the following printout:

```
one
one
```

More books: https://t.me/booksforits

From what we see in our code, the immutable borrow that we performed can be demonstrated in *Figure 1.8*.

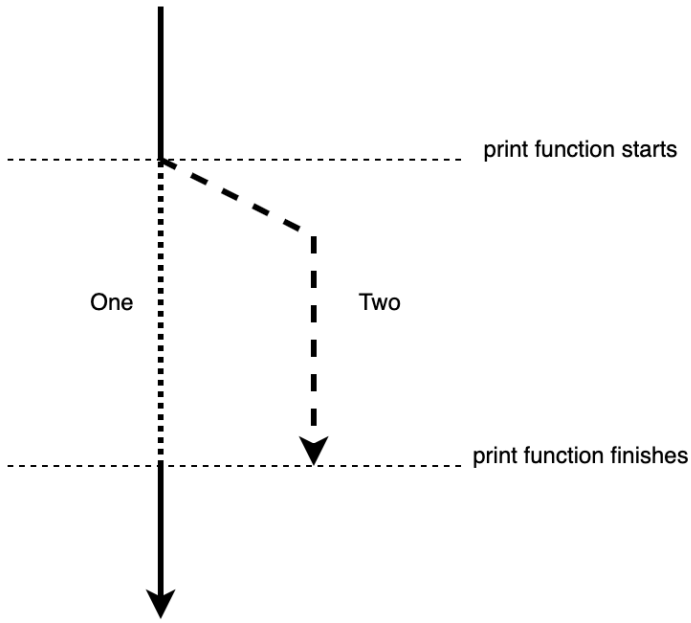


Figure 1.8 – Immutable borrow in relation to the print function

In the preceding figure, we can see that `one` is not available when the `print` function is running. We can demonstrate this with the following code:

```
fn print(value: &String, value_two: String) {
    println!("{}", value);
    println!("{}", value_two);
}
fn main() {
    let one = "one".to_string();
    print(&one, one);
    println!("{}", one);
}
```

If we run the preceding code, we will get the following error:

```
print(&one, one);
----- ----  ^^^ move out of `one` occurs here
```

More books: https://t.me/booksforits

```
|      |
|      borrow of `one` occurs here
borrow later used by call
```

We can see that we cannot utilize `one` even though it is utilized in the `print` function after `&one`. This is because the lifetime of `&one` is throughout the entire lifetime of the `print` function. Thus, we can conclude that *Figure 1.8* is correct. However, we can run one more experiment. We can change `value_one` to a borrow to see what happens with the following code:

```
fn print(value: &String, value_two: &String) {
    println!("{}", value);
    println!("{}", value_two);
}
fn main() {
    let one = "one".to_string();
    print(&one, &one);
    println!("{}", one);
}
```

In the preceding code, we can see that we do two immutable borrows of `one`, and the code runs. This highlights an important fact: we can make as many immutable borrows as we like. However, what happens if the borrow is mutable? To understand, we must explore mutable borrows.

Mutable borrowing of variables

A mutable borrow is essentially the same as an immutable borrow, except that the borrow is mutable. Therefore, we can change the borrowed value. To demonstrate this, we can create a `print` statement that will alter the borrowed value before printing it. We then print it in the `main` function to establish that the value has been changed with the following code:

```
fn print(value: &mut i8) {
     value += 1;
    println!("In function the value is: {}", value);
}
fn main() {
    let mut one: i8 = 5;
    print(&mut one);
    println!("In main the value is: {}", one);
}
```

More books: https://t.me/booksforits

Running the preceding code will give us the following printout:

```
In function the value is: 6
In main the value is: 6
```

The preceding output proves that `one` is `6` even after the lifetime of the mutable reference in the `print` function has expired. We can see that in the `print` function, we update the value of `one` using a `*` operator. This is called a dereference operator. This dereference operator exposes the underlying value so it can be operated. This all seems straightforward, but is it exactly like our immutable references? If we remember, we could have multiple immutable references. We can put this to the test with the following code:

```
fn print(value: &mut i8, value_two: &mut i8) {
     value += 1;
    println!("In function the value is: {}", value);
     value_two += 1;
}
fn main() {
    let mut one: i8 = 5;
    print(&mut one, &mut one);
    println!("In main the value is: {}", one);
}
```

In the preceding code, we can see that we make two mutable references and pass them through, just like in the previous section, but with immutable references. However, running it gives us the following error:

```
error[E0499]: cannot borrow `one` as mutable more than once at
a time
```

Through this example, we can confirm that we cannot have more than one mutable reference at a time. This prevents data races and has given Rust the *fearless concurrency* tag. With what we have covered here, we can now be productive when the compiler is combined with the borrow checker. However, we have touched on the concepts of scope and lifetimes. The use of them has been intuitive, but like the rules around borrowing, we need to dive into scopes and then lifetimes in more detail.

Scopes

To understand scopes, let us go back to how we declare variables. You will have noticed that when we declare a new variable, we use `let`. When we do, that variable is the only one that owns the resource. Therefore, if the value is moved or reassigned, then the initial variable no longer owns the value. When a variable is moved, it is essentially moved into another scope. Variables declared in an outer scope can be referenced in an inner scope, but a variable declared in an inner scope cannot be accessed in

More books: https://t.me/booksforits

the inner scope once the inner scope has expired. We can break down some code into scopes in the following diagram:

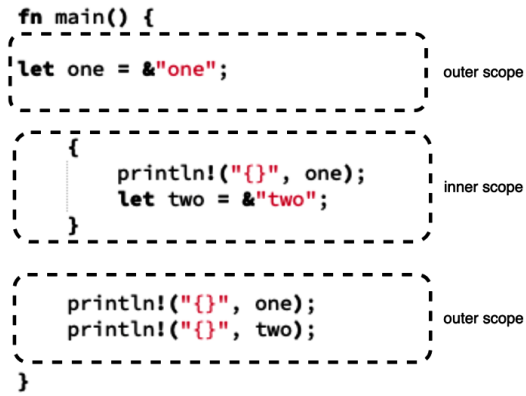


Figure 1.9 – Basic Rust code broken into scopes

Figure 1.9 shows us that we can create an inner scope by merely using curly brackets. Applying what we just learned about scopes to *Figure 1.9*, can you work out whether it will crash? If it will crash, how will it?

If you guessed that it would result in a compiler error, then you are correct. Running the code would result in the following error:

```
println!("{}", two);
                ^^^ not found in this scope
```

Because `one` is defined in the inner scope, we will not be able to reference it in the outer scope. We can solve this problem by declaring the variable in the outer scope but assigning the value in the inner scope with the following code:

```
fn main() {
    let one = &"one";
    let two: &str;
    {
        println!("{}", one);
        two = &"two";
    }
    println!("{}", one);
    println!("{}", two);
}
```

More books: https://t.me/booksforits

In the preceding code, we can see that we do not use `let` when assigning the value because we have already declared the variable in the outer scope. Running the preceding code gives us the following printout:

```
one
one
two
```

We also must remember that if we move a variable into a function, then the variable gets destroyed once the scope of the function finishes. We cannot access the variable after the execution of the function, even though we declared the variable before the execution of the function. This is because once the variable has been moved into the function, it is no longer in the original scope. It has been moved. And because it has been moved to that scope, it is then bound to the lifetime of the scope that it was moved into. This brings us to our next section: lifetimes.

Running through lifetimes

Understanding lifetimes will wrap up our exploration of borrowing rules and scopes. We can explore the effect of lifetimes with the following code:

```
fn main() {
    let one: &i8;
    {
        let two: i8 = 2;
        one = &two;
    } // ----------------------> two lifetime stops here
    println!("r: {}", one);
}
```

With the preceding code, we declare `one` before the inner scope starts. However, we assign it to have a reference of `two`. `two` only has the lifetime of the inner scope, so the lifetime dies before we try and print it out. This is established with the following error:

```
one = &two;     }    println!("r: {}", one);}
      ^^^^      -                        --- borrow later used
here
      |         |
      |         `two` dropped here while still borrowed
      borrowed value does not live long enough
```

`two` is dropped when the lifetime of `two` has finished. With this, we can state that the lifetimes of `one` and `two` are not equal.

More books: https://t.me/booksforits

While it is great that this is flagged when compiling, Rust does not stop here. This concept also applies to functions. Let's say that we build a function that references two integers, compares them, and returns the highest integer reference. The function is an isolated piece of code. In this function, we can denote the lifetimes of the two integers. This is done by using the `'` prefix, which is a lifetime notation. The names of the notations can be anything you come up with, but it is convention to use `a`, `b`, `c`, and so on. We can explore this by creating a simple function that takes in two integers and returns the highest one with the following code:

```
fn get_highest<'a>(first_number: &'a i8, second_number: &'a
    i8) -> &'a i8 {
    if first_number > second_number {
        first_number
        } else {
        second_number
    }
}
fn main() {
    let one: i8 = 1;
    let outcome: &i8;
    {
        let two: i8 = 2;
        let outcome: &i8 = get_highest(&one, &two);
    }
    println!("{}", outcome);
}
```

As we can see, the first and second lifetimes have the same notation of `a`. They both must be present for the duration of the function. We also must note that the function returns an `i8` integer with the lifetime of `a`. If we were to try and use lifetime notation on function parameters without a borrow, we would get some very confusing errors. In short, it is not possible to use lifetime notation without a borrow. This is because if we do not use a borrow, the value passed into the function is moved into the function. Therefore, its lifetime is the lifetime of the function. This seems straightforward; however, when we run it, we get the following error:

```
println!("{}", outcome);}
               ^^^^^^^ use of possibly-uninitialized `outcome`
```

The error occurs because all the lifetimes of the parameters passed into the function and the returned integer are the same. Therefore, the compiler does not know what could be returned. As a result, `two` could be returned. If `two` is returned, then the result of the function will not live long enough to

More books: https://t.me/booksforits

be printed. However, if `one` is returned, then it will. Therefore, there is a possibility of not having a value to print after the inner scope is executed. In a dynamic language, we would be able to run code that runs the risk of referencing variables that have not been initialized yet. However, with Rust, we can see that if there is a possibility of an error like this, it will not compile. In the short term, it might seem like Rust takes longer to code, but as the project progresses, this strictness will save a lot of time by preventing silent bugs. In conclusion of our error, there is no way of solving our problem with the exact function and main layout that we have. We would either have to move our printing of the outcome into the inner scope or clone the integers and pass them into the function.

We can create one more function to explore functions with different lifetime parameters. This time we will create a `filter` function. If the first number is lower than the second number, we will then return `0`. Otherwise, we will return the first number. This can be achieved with the following code:

```
fn filter<'a, 'b>(first_number: &'a i8, second_number: &'b
    i8) -> &'a i8 {
    if first_number < second_number {
        &0
    } else {
        first_number
    }
}
fn main() {
    let one: i8 = 1;
    let outcome: &i8;
    {
        let two: i8 = 2;
        outcome = filter(&one, &two);
    }
    println!("{}", outcome);
}
```

The preceding code works because we know the lifetimes are different. The first parameter has the same lifetime as the returned integer. If we were to implement `filter(&two, &one)` instead, we would get an error stating that the outcome does not live long enough to be printed. We have now covered all that we need to know for now to write productive code in Rust without the borrow checker getting in our way. We now need to move on to creating bigger building blocks for our programs so we can focus on tackling the complex problems we want to solve with code. We will start this with a versatile building block of programs: **structs**.

More books: https://t.me/booksforits

Building structs

In modern high-level dynamic languages, objects have been the bedrock for building big applications and solving complex problems, and for good reason. Objects enable us to encapsulate data, functionality, and behavior. In Rust, we do not have objects. However, we do have structs that can hold data in fields. We can then manage the functionality of these structs and group them together with traits. This is a powerful approach, and it gives us the benefits of objects without the high coupling, as highlighted in the following figure:

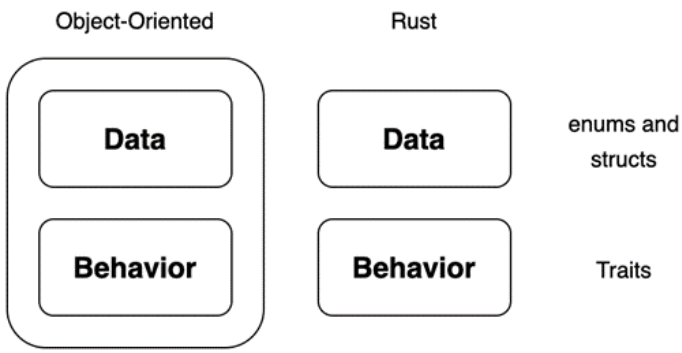


Figure 1.10 – Difference between Rust structs and objects

We will start with something basic by creating a `Human` struct with the following code:

```
#[derive(Debug)]
struct Human<'a> {
    name: &'a str,
    age: i8,
    current_thought: &'a str
}
```

In the preceding code, we can see that our string literal fields have the same lifetime as the struct itself. We have also applied the `Debug` trait to the `Human` struct, so we can print it out and see everything. We can then create the `Human` struct and print the struct out using the following code:

```
fn main() {
    let developer = Human{
        name: "Maxwell Flitton",
        age: 32,
        current_thought: "nothing"
```

More books: https://t.me/booksforits

```
    };
    println!("{:?}", developer);
    println!("{}", developer.name);
}
```

Running the preceding code would give us the following printout:

```
Human { name: "Maxwell Flitton", age: 32, current_thought:
    "nothing" }
Maxwell Flitton
```

We can see that our fields are what we expect. However, we can change our string slice fields to strings to get rid of lifetime parameters. We may also want to add another field where we can reference another `Human` struct under a `friend` field. However, we may also have no friends. We can account for this by creating an enum that is either a friend or not and assigning this to a `friend` field, as seen in the following code:

```
#[derive(Debug)]
enum Friend {
    HUMAN(Human),
    NIL
}
#[derive(Debug)]
struct Human {
    name: String,
    age: i8,
    current_thought: String,
    friend: Friend
}
```

We can then define the `Human` struct initially with no friends just to see if it works with the following code:

```
    let developer = Human{
        name: "Maxwell Flitton".to_string(),
        age: 32,
        current_thought: "nothing".to_string(),
        friend: Friend::NIL
    };
```

More books: https://t.me/booksforits

However, when we run the compiler, it does not work. I would like to think this is because the compiler cannot believe that I have no friends. But alas, it's to do with the compiler not knowing how much memory to allocate for this declaration. This is shown through the following error code:

```
enum Friend {    HUMAN(Human),    NIL}#[derive(Debug)]
^^^^^^^^^^^            ----- recursive without indirection
|
recursive type has infinite size
```

Because of the enum, theoretically, the memory needed to store this variable could be infinite. One `Human` struct could reference another `Human` struct as a `friend` field, which could in turn reference another `Human` struct, resulting in a potentially infinite number of `Human` structs being linked together through the `friend` field. We can solve this problem with pointers. Instead of storing all the data of a `Human` struct in the `friend` field, we store a memory address that we know has a maximum value because it's a standard integer. This memory address points to where another `Human` struct is stored in the memory. As a result, the program knows exactly how much memory to allocate when it crosses a `Human` struct, irrespective of whether the `Human` struct has a `friend` field or not. This can be achieved by using a `Box` struct, which is essentially a smart pointer for our enum, with the following code:

```
#[derive(Debug)]
enum Friend {
    HUMAN(Box<Human>),
    NIL
}
```

So, now our enum states whether the friend exists or not, and if so, it has a memory address if we need to extract information about this friend. We can achieve this with the following code:

```
fn main() {
    let another_developer = Human{
        name: "Caroline Morton".to_string(),
        age:30,
        current_thought: "I need to code!!".to_string(),
        friend: Friend::NIL
    };
    let developer = Human{
        name: "Maxwell Flitton".to_string(),
        age: 32,
        current_thought: "nothing".to_string(),
```

More books: https://t.me/booksforits

```
        friend: Friend::HUMAN(Box::new(another_developer))
    };
    match &developer.friend {
        Friend::HUMAN(data) => {
            println!("{}", data.name);
        },
        Friend::NIL => {}
    }
}
```

In the preceding code, we can see that we have created one `Human` struct, and then another `Human` struct with a reference to the first `Human` struct as a `friend` field. We then access the second `Human` struct's `friend` through the `friend` field. Remember, we must handle both possibilities as it could be a nil value.

While it is exciting that friends can be made, if we take a step back, we can see that there is a lot of code written for each human we create. This is not helpful if we must create a lot of humans in a program. We can reduce this by implementing some functionality for our struct. We will essentially create a constructor for the struct with extra functions, so we can add optional values if we want. We will also make the `thought` field optional. So, a basic struct with a constructor populating only the most essential fields can be achieved with the following code:

```
#[derive(Debug)]
struct Human {
    name: String,
    age: i8,
    current_thought: Option<String>,
    friend: Friend
}
impl Human {
    fn new(name: &str, age: i8) -> Human {
        return Human{
            name: name.to_string(),
            age: age,
            current_thought: None,
            friend: Friend::NIL
        }
    }
}
```

More books: https://t.me/booksforits

Therefore, creating a new human now only takes the following line of code:

```
let developer = Human::new("Maxwell Flitton", 32);
```

This will have the following field values:

- Name: `"Maxwell Flitton"`
- Age: `32`
- Current Thought: `None`
- Friend: `NIL`

We can add more functions in the implement block for adding friends and a current thought with the following code:

```
    fn with_thought(mut self, thought: &str) -> Human {
        self.current_thought = Some(thought.to_string());
        return self
    }
    fn with_friend(mut self, friend: Box<Human>) -> Human {
        self.friend = Friend::HUMAN(friend);
        return self
    }
```

In the preceding code, we can see that we pass in a mutable version of the struct that is calling these functions. These functions can be chained because they return the struct that called them. If we want to create a developer with a thought, we can do this with the following code:

```
let developer = Human::new("Maxwell Flitton", 32)
    .with_thought("I love Rust!");
```

We must note that a function that does not require `self` as a parameter can be called with `::`, while a function that does require `self` as a parameter can be called with a simple dot (`.`). If we want to create a developer with a friend, it can be done using the following code:

```
let developer_friend = Human::new("Caroline Morton", 30);

let developer = Human::new("Maxwell Flitton", 32)
    .with_thought("I love Rust!")
    .with_friend(Box::new(developer_friend));
Println!("{:?}", developer);
```

More books: https://t.me/booksforits

Running the code will result in the following parameters for `developer`:

```
Name: "Maxwell Flitton"
Age: 32
Current Thought: Some("I love Rust!")
Friend: HUMAN(Human { name: "Caroline Morton", age: 30,
    current_thought: None, friend: NIL })
```

We can see that structs combined with enums and functions that have been implemented with these structs can be powerful building blocks. We can define fields and functionality with only a small amount of code if we have defined our structs well. However, writing the same functionality for multiple structs can be time-consuming and result in a lot of repeated code. If you have worked with objects before, you may have utilized inheritance for that. Rust goes one better. It has traits, which we will explore in the next section.

Verifying with traits

We can see enums can empower structs so that they can handle multiple types. This can also be translated for any type of function or data structure. However, this can lead to a lot of repetition. Take, for instance, a `User` struct. Users have a core set of values, such as a username and password. However, they could also have extra functionality based on roles. With users, we must check roles before firing certain processes. We can wrap up structs with traits by creating a simple toy program that defines users and their roles with the following steps:

1. We can define our users with the following code:

```
struct AdminUser {
    username: String,
    password: String
}
struct User {
    username: String,
    password: String
}
```

We can see in the preceding code that the `User` and `AdminUser` structs have the same fields. For this exercise, we merely need two different structs to demonstrate the effect traits have on them. Now that our structs are defined, we can move on to our next step, which is creating the traits.

More books: https://t.me/booksforits

2. We will be implementing these traits in our structs. The total traits that we will have are, comprise create, edit, and delete. We will be using them to assign permissions to our users. We can create these three traits with the following code:

```
trait CanEdit {
    fn edit(&self) {
        println!("admin is editing");
    }
}
trait CanCreate {
    fn create(&self) {
        println!("admin is creating");
    }
}
trait CanDelete {
    fn delete(&self) {
        println!("admin is deleting");
    }
}
```

We can see that the functions for the traits only take in `self`. We cannot make any references to the fields in the functions to `self` as we do not know what structs will be implemented. However, we can override functions when we implement the trait to the struct if needed. If we are to return `self`, we will need to wrap it in a `Box` struct, as the compiler will not know the size of the struct being returned. We also must note that the signature of the function (input parameters and return values) must be the same as the original if we overwrite the function for a struct. Now that we have defined the traits, we can move on to the next step of implementing the traits to define roles for our user.

3. With our roles, we can make our admin have every permission and our user only the edit permission. This can be achieved with the following code:

```
impl CanDelete for AdminUser {}
impl CanCreate for AdminUser {}
impl CanEdit for AdminUser {}

impl CanEdit for User {
    fn edit(&self) {
        println!("A standard user {} is editing",
    self.username);
```

More books: https://t.me/booksforits

```
    }
}
```

From our previous step, we can remember that all the functions already worked for the admin by printing out that the admin is doing the action. Therefore, we do not have to do anything for the implementation of the traits for the admin. We can also see that we can implement multiple traits for a single struct. This adds a lot of flexibility. In our user implementation of the `CanEdit` trait, we have overwritten the `edit` function so that we can have the correct statement printed out. Now that we have implemented the traits, our `user` structs have permission in the code to enter scopes that require those traits. We can now build the functions for using these traits in the next step.

4. We could utilize the functions in the traits by directly running them in the `main` function on the structs that have implemented them. However, if we do this, we will not see their true power in this exercise. We may also want this standard functionality throughout our program in the future when we span multiple files. The following code shows how we create functions that utilize the traits:

```
fn create<T: CanCreate>(user: &T) -> () {
    user.create();
}
fn edit<T: CanEdit>(user: &T) -> () {
    user.edit();
}
fn delete<T: CanDelete>(user: &T) -> () {
    user.delete();
}
```

The preceding notation is fairly like the lifetime annotation. We use angle brackets before the input definitions to define the trait we want to accept at `T`. We then state that we will accept a borrowed struct that has implemented the trait as `&T`. This means that any struct that implements that specific trait can pass through the function. Because we know what the trait can do, we can then use the trait functions. However, because we do not know what struct is going to be passed through, we cannot utilize specific fields. But remember, we can overwrite a trait function to utilize struct fields when we implement the trait for the struct. This might seem rigid, but the process enforces good, isolated, decoupled coding that is safe. For instance, let's say we remove a function from a trait or remove a trait from a struct. The compiler would refuse to compile until all the effects of this change were complete. Thus, we can see that, especially for big systems, Rust is safe, and can save time by reducing the risk of silent bugs. Now that we have defined the functions, we can use them in the `main` function in the next step.

More books: https://t.me/booksforits

5. We can test to see whether all the traits work with the following code:

```
fn main() {
    let admin = AdminUser{
        username: "admin".to_string(),
        password: "password".to_string()
    };
    let user = User{
        username: "user".to_string(),
        password: "password".to_string()
    };
    create(&admin);
    edit(&admin);
    edit(&user);
    delete(&admin);
}
```

We can see that the functions that accept traits are used just like any other function.

Running the entire program will give us the following printout:

```
admin is creating
admin is editing
A standard user user is editing
admin is deleting
```

In our output, we can see that the overriding of the `edit` function for the `User` struct works.

We have now learned enough about traits to be productive with web development. Traits get even more powerful, and we will be using them for some key parts of our web programming. For instance, several web frameworks have traits that execute before the request is processed by the view/API endpoint. Implementing structs with these traits automatically loads the `view` function with the result of the `trait` function. This can be database connections, extraction of tokens from headers, or anything else we wish to work with. There is also one last concept that we need to tackle before we move on to the next chapter, and that is macros.

Metaprogramming with macros

Metaprogramming can generally be described as a way in which the program can manipulate itself based on certain instructions. Considering the strong typing Rust has, one of the simplest ways in

More books: https://t.me/booksforits

which we can meta program is by using generics. A classic example of demonstrating generics is through coordinates, as follows:

```
struct Coordinate <T> {
    x: T,
    y: T
}
fn main() {
    let one = Coordinate{x: 50, y: 50};
    let two = Coordinate{x: 500, y: 500};
    let three = Coordinate{x: 5.6, y: 5.6};
}
```

In the preceding snippet, we can see that the `Coordinate` struct managed to take in and handle three different types of numbers. We can add even more variance to the `Coordinate` struct so we can have two different types of numbers in one struct with the following code:

```
struct Coordinate <T, X> {
    x: T,
    y: X
}
fn main() {
    let one = Coordinate{x: 50, y: 500};
    let two = Coordinate{x: 5.6, y: 500};
    let three = Coordinate{x: 5.6, y: 50};
}
```

What is happening in the preceding code with generics is that the compiler is looking for all instances where the struct is used, creating structs with the types used when the compilation is running. Now that we have covered generics, we can move on to the main mechanism of metaprogramming: macros.

Macros enable us to abstract code. We've already been using macros in our print functions. The `!` notation at the end of the function denotes that this is a macro that's being called. Defining our own macros is a blend of defining a function and using a lifetime notation within a `match` statement in the function. To demonstrate this, we will define a macro that capitalizes a string with the following code:

```
macro_rules! capitalize {
    ($a: expr) => {
        let mut v: Vec<char> = $a.chars().collect();
        v[0] = v[0].to_uppercase().nth(0).unwrap();
```

More books: https://t.me/booksforits

```
        $a = v.into_iter().collect();
    }
}
fn main() {
    let mut x = String::from("test");
    capitalize!(x);
    println!("{}", x);
}
```

Instead of using the term `fn`, we use the `macro_rules!` definition. We then say that `$a` is the expression passed into the macro. We get the expression, convert it into a vector of chars, then make the first `char` uppercase, and then convert it back to a string. We must note that we don't return anything in the `capitalize` macro, and when we call the macro, we don't assign a variable to it. However, when we print the `x` variable at the end, we can see that it is capitalized. This does not behave like an ordinary function. We also must note that we didn't define a type, instead, we just said it was an expression; the macro still does checks via traits. Passing an integer into the macro creates the following error:

```
|     capitalize!(32);
|     ---------------- in this macro invocation
|
= help: the trait `std::iter::FromIterator<char>` is not
implemented for `{integer}`
```

Lifetimes, blocks, literals, paths, metaprogramming, and more, can also be passed instead of an expression. While it's important to have a brief understanding of what's under the hood of a basic macro for debugging and further reading, diving more into developing complex macros will not help us in developing web apps. We must remember that macros are a last resort and should be used sparingly. Errors thrown in macros can be hard to debug. In web development, a lot of the macros are already defined in third-party packages. Because of this, we do not need to write macros ourselves to get a web app up and running. Instead, we will mainly be using derive macros out of the box.

Summary

With Rust, we have seen that there are some traps when coming from a dynamic programming language background. However, with a little bit of knowledge of referencing and basic memory management, we can avoid common pitfalls and write safe, performant code quickly that can handle errors. By utilizing structs and traits, we can build objects that are analogous to classes in standard dynamic programming languages. On top of this, the traits enabled us to build mixin-like functionality. This

More books: https://t.me/booksforits

not only enables us to slot in functionality when it's useful to us but also perform checks on the structs through typing to ensure that the container or function is processing structs with certain attributes belonging to the trait that we can utilize in the code.

With our fully functioning structs, we bolted on even more functionality with macros and looked under the hood of basic macros by building our own `capitalize` function, giving us guidance for further reading and debugging. We also got to see a brief demonstration of how powerful macros combined with structs can be in web development with JSON serialization. With what we have learned in this chapter, we can now write basic Rust programs. Because we understand the concepts that the borrow checker enforces, we can debug the application that we code. Like other languages, there are limited real-world applications that we can do yet. However, we do have the essential foundation to build real-world applications spanning multiple files running on our own local computers.

We can now move on to the next chapter and investigate setting up a Rust environment on our own computers to structure files and code to enable us to build programs that can solve real-world problems.

Questions

1. What is the difference between `str` and `String`?
2. Why can't string slices be passed into a function (string slice meaning `str` as opposed to `&str`)?
3. How do we access the data belonging to a key in a HashMap?
4. When a function results in an error, can we handle other processes, or will the error crash the program instantly?
5. Why does Rust only allow one mutable borrow at a point in time?
6. When would we need to define two different lifetimes in a function?
7. How can structs link to the same struct via one of their fields?
8. How can we add extra functionality to a struct where the functionality can also be implemented by other structs?
9. How do we allow a container or function to accept different data structures?
10. What's the quickest way to add a trait, such as `Copy`, to a struct?

Answers

1. `String` is a fixed-size reference stored in the stack that points to string-type data on the heap. `str` is an immutable sequence of bytes stored somewhere in memory. Because the size of `str` is unknown, it can only be handled by a `&str` pointer.
2. Since we do not know the size of the string slice at compile time, we cannot allocate the correct amount of memory for it. Strings, on the other hand, have a fixed-size reference stored on the

More books: https://t.me/booksforits

stack that points to the string slice on the heap. Because we know this fixed size of the string reference, we can allocate the correct amount of memory and pass it through to a function.

3. We use the HashMap's `get` function. However, we must remember that the `get` function merely returns an `Option` struct. If we are confident that there is something there or we want the program to crash if nothing is found, we can directly unwrap it. However, if we don't want that, we can use a `match` statement and handle the `Some` and `None` output as we wish.
4. No, results must be unwrapped before exposing the error. A simple `match` statement can handle unwrapping the result and managing the error as we see fit.
5. Rust only allows one mutable borrow to prevent memory unsafety. In Goregaokar's blog, the example of an enum is used to illustrate this. If an enum supports two different data types (`String` and `i64`), if a mutable reference of the string variant of the enum is made, and then another reference is made, the mutable reference can change the data, and then the second reference would still be referencing the string variant of the enum. The second reference would then try to dereference the string variant of the enum, potentially causing a segmentation fault. Elaboration on this example and others is provided in the *Further reading* section.
6. We would need to define two different lifetimes when the result of a function relies on one of the lifetimes and the result of the function is needed outside of the scope of where it is called.
7. If a struct is referencing itself in one of its fields, the size could be infinite as it could continue to reference itself continuously. To prevent this, we can wrap the reference to the struct in the field in a `Box` struct.
8. We can slot extra functionality and freedom into a struct by using traits. Implementing a trait will give the struct the ability to use functions that belong to the trait. The trait's implementation also allows the struct to pass typing checks for that trait.
9. We allow a container or function to accept different data structures by declaring enums or traits in the type checking or by utilizing generics (see the *Further reading* section: *Mastering Rust* or *Hands-On Functional Programming in Rust (first chapter)*).
10. The quickest way to add a trait to a struct is by annotating the struct with a derive macro that has the copy and clone traits.

Further reading

- *Hands-On Functional Programming in Rust* (2018) by Andrew Johnson, Packt Publishing
- *Mastering Rust* (2019) by Rahul Sharma and Vesa Kaihlavirta, Packt Publishing
- *The Rust Programming Language* (2018): `https://doc.rust-lang.org/stable/book/`
- *The Problem With Single-threaded Shared Mutability* (2015) by Manish Goregaokar: `https://manishearth.github.io/blog/2015/05/17/the-problem-with-shared-mutability/`

More books: https://t.me/booksforits

More books: https://t.me/booksforits

2

Designing Your Web Application in Rust

We previously explored the syntax of Rust, enabling us to tackle memory management quirks and build data structures. However, as any experienced engineer will tell you, structuring code across multiple files and directories is an important aspect of building software.

In this chapter, we will build a basic command-line to-do program. We manage the dependencies needed to build our command-line program with Rust's **Cargo**. Our program will be structured in a scalable way where we build and manage our own modules, which will be imported into other areas of the program and utilized. We will learn these concepts by building a to-do application spanning multiple files that create, edit, and delete to-do applications. This application will save our multiple to-do application files locally, and we will be able to interact with our application using a command-line interface.

In this chapter, we will cover the following topics:

- Managing a software project with Cargo
- Structuring code
- Interacting with the environment

By the end of this chapter, you will be able to build applications in Rust that can be packaged and used. You will also be able to use third-party packages in your code. As a result, you will be able to build any command-line application that does not require a server or a graphical user interface if you understand the problem you are trying to solve and can break it down into logical chunks.

More books: https://t.me/booksforits

Technical requirements

As we move toward building web apps in Rust, we are going to have to start relying on third-party packages to do some of the heavy lifting for us. Rust manages dependencies through a package manager called Cargo. To use Cargo, we are going to have to install Rust on our computer from the following URL: `https://www.rust-lang.org/tools/install`.

This installation provides the programming language Rust and the dependency manager Cargo. You can find all the code files on GitHub:

`https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter02`

Managing a software project with Cargo

Before we start structuring our program with Cargo, we should build a basic single-file application. To do this, we initially must create a file called `hello_world.rs` in a local directory. The `.rs` extension denotes that the file is a Rust file. To be honest, it does not matter what the extension is. If there is viable Rust code written in that file, the compiler will compile and run it without any issues. However, having different extensions might confuse other developers and code editors and cause problems when importing code from other Rust files. So, it is best to use `.rs` when naming your Rust files. Inside our `hello_world.rs` file, we can have the following code:

```
fn main() {
    println!("hello world");
}
```

This is no different from our first code block in the previous chapter. Now that we have defined our entry point in our `hello_world.rs` file, we can compile the file with the following command:

```
rustc hello_world.rs
```

Once the compilation has finished, there will be a binary file in the same directory that can be run. If we compile it on Windows, we can run the binary with the following command:

```
.\hello_world.exe
```

If we compile it on Linux or macOS, we can run it with the following command:

```
./hello_world
```

Because we only built a simple `hello world` example, `hello world` will just be printed out. While this can be useful when building a simple application in one file, it is not recommended for managing programs spanning multiple files. It is not even recommended when relying on third-party

More books: https://t.me/booksforits

modules. This is where Cargo comes in. Cargo manages everything, including the running, testing, documentation, building/compiling, and third-party module dependencies, out of the box with a few simple commands. We will cover these commands throughout this chapter. From what we have seen when running our `hello world` example, we must compile the code before we can run it, so let's now move on to the next section where we build a basic application using Cargo.

Building with Cargo

Building with Cargo is straightforward. All we must do is navigate to the directory where we want to build our project and run the following command:

```
cargo new web_app
```

The preceding command builds a basic Cargo Rust project. If we explore this application, we'll see the following structure:

```
└── web_app
    ├── Cargo.toml
    └── src
        └── main.rs
```

We can see that there is only one Rust file, and this is the `main.rs` file that is housed in the `src` directory. If you open the `main.rs` file, you will see that this is the same as the file that we made in the previous section. It is an entry point with the default code printing out `hello world` to the console. The dependencies and metadata for our project are defined in the `Cargo.toml` file. If we want to run our program, we do not need to navigate to the `main.rs` file and run `rustc`. Instead, we can use Cargo and run it with the following command:

```
cargo run
```

When you do this, you will see the project compile and run with the following printout:

```
  Compiling web_app v0.1.0 (/Users/maxwellflitton/Documents/
   github/books/Rust-Web_Programming-two/chapter02/web_app)
    Finished dev [unoptimized + debuginfo] target(s) in 0.15s
     Running `target/debug/web_app`
hello world
```

Your printout will be slightly different because the base directory will be different. At the bottom, you will see `hello world`, which is what we expect. We can also see that the printout states that the compilation is unoptimized and that it is running in `target/debug/web_app`. We can navigate directly to the `target/debug/web_app` binary and run it just like we did in the previous section

More books: https://t.me/booksforits

as this is where the binary is stored. The `target` directory is where the files for compiling, running, and documenting our program reside. If we attach our code to a GitHub repository, we must make sure that the `target` directory is ignored by GitHub by putting it in the `.gitignore` file. Right now, we are running the unoptimized version. This means that it is slower but quicker to compile. This makes sense as when we are developing, we will be compiling multiple times. However, if we want to run the optimized version, we can use the following command:

```
cargo run --release
```

The preceding command gives us the following printout:

```
    Finished release [optimized] target(s) in 2.63s
     Running `target/release/web_app`
hello world
```

In the preceding output, we can see that our optimized binary is in the `target/release/web_app` path. Now that we have got our basic builds done, we can start to use Cargo to utilize third-party crates.

Shipping crates with Cargo

Third-party libraries are referred to as crates. Adding them and managing them with Cargo is straightforward. In this section, we will explore this process by utilizing the `rand` crate, available at `https://rust-random.github.io/rand/rand/index.html`. It must be noted that the documentation for this crate is clear and well structured with links to structs, traits, and modules. This is not a reflection of the rand crate itself. This is standard documentation for Rust that we will cover in the next section. To use this crate in our project, we open the `Cargo.toml` file and add the `rand` crate under the `[dependencies]` section, as follows:

```
[dependencies]
rand = "0.7.3"
```

Now that we've defined our dependency, we can use the `rand` crate to build a random number generator:

```
use rand::prelude::*;

fn generate_float(generator: &mut ThreadRng) -> f64 {
    let placeholder: f64 = generator.gen();
    return placeholder * 10.0
}
fn main() {
    let mut rng: ThreadRng = rand::thread_rng();
    let random_number = generate_float(&mut rng);
```

More books: https://t.me/booksforits

```
    println!("{}", random_number);
}
```

In the preceding code, we have defined a function called `generate_float`, which uses the crate to generate and return a float between `0` and `10`. Once we've done this, we print the number. The implementation of the `rand` crate is handled by the rand documentation. Our `use` statement imports the `rand` crate. When using the `rand` create for generating a float, the documentation tells us to import (`*`) from the `rand::prelude` module, which simplifies the importing of common items, as shown in the crate documentation at `https://rust-random.github.io/rand/rand/prelude/index.html`.

The `ThreadRng` struct is a random number generator that generates an `f64` value between `0` and `1`, which is elaborated on in the rand crate documentation at `https://rust-random.github.io/rand/rand/rngs/struct.ThreadRng.html`.

Now, we get to see the power of the documentation. With a few clicks on the introduction page of the rand documentation, we can dig into the declarations of the structs and functions used in the demonstration. Now that our code is built, we can run our program with the `cargo run` command. While Cargo is compiling, it pulls code from the `rand` crate and compiles that into the binary. We can also note that there is now a `cargo.lock` file. As we know that `cargo.toml` is for us to describe our own dependencies, `cargo.lock` is generated by Cargo and we should not edit it ourselves as it contains exact information about our dependencies. This seamless functionality combined with the easy-to-use documentation shows how Rust improves the development process through marginal gains via the development ecosystem as well as the quality of the language. However, all these gains from the documentation are not purely dependent on the third-party libraries; we can also autogenerate our own documentation.

Documenting with Cargo

Speed and safety are not the only benefits of picking a new language such as Rust to develop in. Over the years, the software engineering community keeps learning and growing. Simple things such as good documentation can make or break a project. To demonstrate this, we can define Markdown language within the Rust file with the following code:

```
/// This function generates a float number using a number
/// generator passed into the function.
///
/// # Arguments
/// * generator (&mut ThreadRng): the random number
/// generator to generate the random number
///
/// # Returns
```

More books: https://t.me/booksforits

```
/// (f64): random number between 0 -> 10
fn generate_float(generator: &mut ThreadRng) -> f64 {
    let placeholder: f64 = generator.gen();
    return placeholder * 10.0
}
```

In the preceding code, we've denoted the Markdown with the `///` markers. This does two things: it tells other developers who look at the code what the function does and renders Markdown in our autogeneration. Before we run the document command, we can define and document a basic user struct and a basic user trait to also show how these are documented:

```
/// This trait defines the struct to be a user.
trait IsUser {
    /// This function proclaims that the struct is a user.
    ///
    /// # Arguments
    /// None
    ///
    /// # Returns
    /// (bool) true if user, false if not
    fn is_user() -> bool {
        return true
    }
}
/// This struct defines a user
///
/// # Attributes
/// * name (String): the name of the user
/// * age (i8): the age of the user
struct User {
    name: String,
    age: i8
}
```

More books: https://t.me/booksforits

Now that we have documented a range of different structures, we can run the auto-documentation process with the following command:

```
cargo doc --open
```

We can see that the documentation is rendered in the same way as the rand crate:

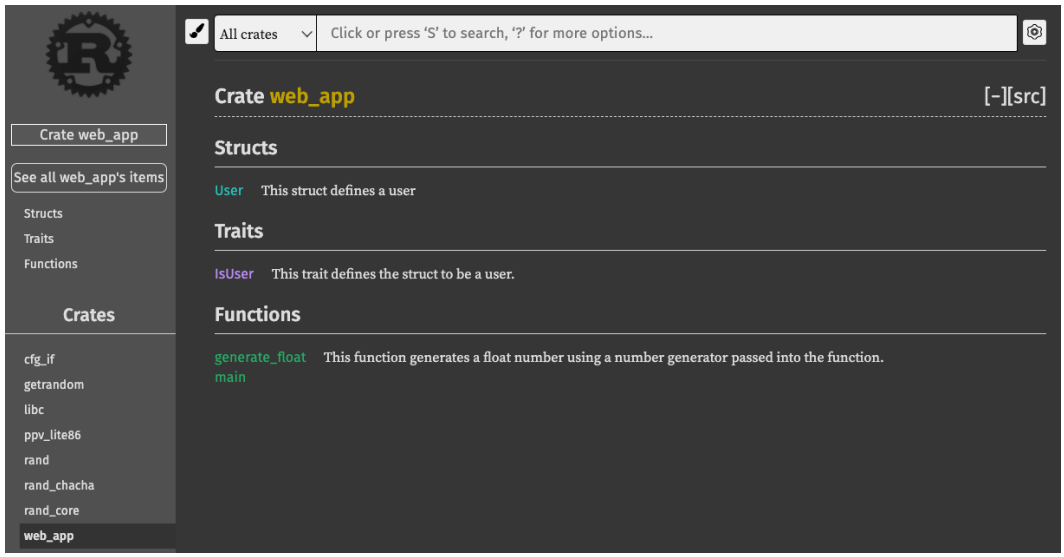


Figure 2.1 – Documentation view of the web app

In the preceding screenshot, we can see that **web_app** is a crate. We can also see that the documentation of the rand crate is involved (if we look at the bottom left of the screenshot, we can see the **rand** crate documentation just above our **web_app** crate documentation). If we click on the **User** struct, we can see the declaration of the struct, the Markdown that we wrote for the attributes, and the trait implications, as shown in the following figure:

More books: https://t.me/booksforits

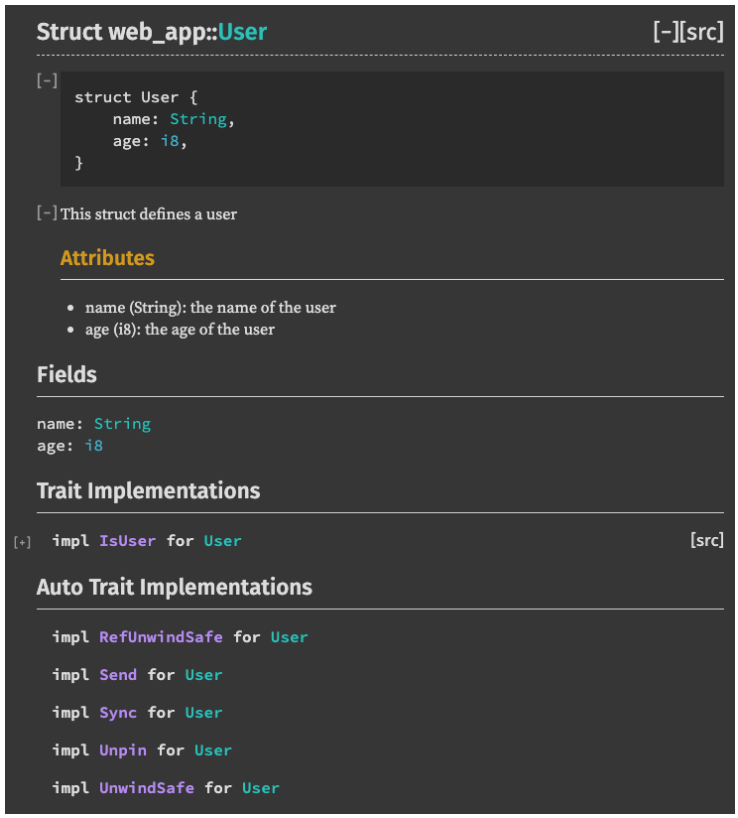


Figure 2.2 – Documentation on struct

It must be noted that in future sections of the book, we will not include Markdown in the code snippets to maintain readability. However, Markdown-documented code is provided in the book's GitHub repo. Now that we have a well-documented, running Cargo project, we need to be able to pass parameters into it to enable different configurations to run depending on the context.

Interacting with Cargo

Now that we have our program running and using third-party modules, we can start to interact with our Rust programs through command-line inputs. To enable our program to have some flexibility depending on the context, we need to be able to pass parameters into our program and keep track of the parameters in which the program is running. We can do this using the `std` (standard library) identifier:

```
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();
```

More books: https://t.me/booksforits

```
    println!("{:?}", args);
}
```

In the preceding code, we can see that we collect the arguments passed into the program into a vector and then print out the arguments in debug mode. Let us run the following command:

```
cargo run one two three
```

Running the preceding command gives the following printout:

```
["target/debug/interacting_with_cargo", "one", "two", "three"]
```

Here, we can see that our `args` vector has the arguments that we passed in. This is not surprising as many other languages also accept arguments passed into the program via the command line. We must note as well that the path to the binary is also included. Here, I also must highlight that I am using a different project named `interacting_with_cargo`, hence the `target/debug/interacting_with_cargo` path. We can also see from the command-line arguments that we are running in debug mode. Let us try to run a release version of our program with the following command:

```
cargo run --release one two three
```

We would receive the following printout:

```
["target/release/interacting_with_cargo", "one", "two",
"three"]
```

From the preceding output, we can see that `--release` is not in our vector. However, this does give us some extra functionality to play with. For instance, we might want to run different processes depending on the type of compilation. This can easily be done with the following code:

```
let args: Vec<String> = env::args().collect();
let path: &str = &args[0];

if path.contains("/debug/") {
    println!("Debug is running");
}
else if path.contains("/release/") {
    println!("release is running");
}
else {
    panic!("The setting is neither debug or release");
}
```

More books: https://t.me/booksforits

However, the preceding simple solution is patchy. The `path` that we extract is only consistent if we are running Cargo commands. While Cargo commands are great for building, compiling, and documenting, it does not make sense to carry all those files around in production. In fact, there are advantages to extracting the static binary, wrapping it in a Docker container completely by itself, and running the binary directly as this can reduce the size of the Docker image from 1.5 GB to 200 MB. So, while this might seem like a quick win, it can lead to breaking code when deploying our applications. Therefore, it is essential to put in the `panic` macro at the end to prevent this from reaching production and you not knowing about it.

So far, we have passed in some basic commands; however, this is not helpful or scalable. There would also be a lot of boilerplate code written for us to implement help guides for users. To scale our command-line interface, we can lean on the `clap` crate to handle arguments passed into the program, with the following dependency:

```
[dependencies]
clap = "3.0.14"
```

To flesh out our understanding of command-line interfaces, we can develop a toy application that merely takes in a few commands and prints them out. To do this, we must import what we need from the `clap` crate in the `main.rs` file with the following code:

```
use clap::{Arg, App};
```

Now, we can move on to defining our application:

1. Our application houses metadata about the application in the `main` function with the following code:

```
fn main() {
    let app = App::new("booking")
        .version("1.0")
        .about("Books in a user")
        .author("Maxwell Flitton");
. . .
```

   If we look at the documentation of `clap`, we can bind arguments directly to the `App` struct; however, this can get ugly and tightly bound. Instead, we will define them separately in the next step.

2. In our toy app, we are taking in a first name, last name, and age, which can be defined as follows:

```
    let first_name = Arg::new("first name")
        .long("f")
        .takes_value(true)
```

More books: https://t.me/booksforits

```
        .help("first name of user")
        .required(true);

    let last_name = Arg::new("last name")
        .long("l")
        .takes_value(true)
        .help("first name of user")
        .required(true);

    let age = Arg::new("age")
        .long("a")
        .takes_value(true)
        .help("age of the user")
        .required(true);
```

We can see that we can keep stacking the arguments. Right now, they are not bound to anything. Now, we can move on to binding them to our application and passing in the arguments in the next step.

3. Binding, getting, and parsing inputs can be achieved with the following code:

```
    let app = app.arg(first_name).arg(last_name).
arg(age);

    let matches = app.get_matches();

    let name = matches.value_of("first name")
        .expect("First name is required");
    let surname = matches.value_of("last name")
        .expect("Surname is required");
    let age: i8 = matches.value_of("age")
        .expect("Age is required").parse().unwrap();

    println!("{:?}", name);
    println!("{:?}", surname);
    println!("{:?}", age);
}
```

More books: https://t.me/booksforits

Now that we have a working example of how to pass command-line arguments, we can interact with our application to see how it displays by running the following command:

```
cargo run -- --help
```

The middle `--` before `--help` tells Cargo to pass all the arguments after `--` into `clap` as opposed to `cargo`. The preceding command will give us the following printout:

```
booking 1.0
Maxwell Flitton
Books in a user

USAGE:
    interacting_with_cargo --f <first name> --l <last name>
                           --a <age>

OPTIONS:
        --a <age>           age of the user
        --f <first name>    first name of user
    -h, --help              Print help information
        --l <last name>     first name of user
    -V, --version           Print version information
```

In the preceding output, we can see how to directly interact with our compiled binary file. We also have a nice help menu. To interact with Cargo, we need to run the following command:

```
cargo run -- --f first --l second --a 32
```

The preceding command will give the following printout:

```
"first"
"second"
32
```

We can see that the parsing works as we have two strings and an integer. The reason why crates such as `clap` are useful is that they are essentially self-documenting. Developers can look at the code and know what arguments are being accepted and view the metadata around them. Users can get help on the inputs by merely passing in the `help` parameter. This approach reduces the risk of the documentation becoming outdated as it is embedded in the code that executes it. If you accept command-line arguments, it is advised that you use a crate such as `clap` for this purpose. Now that we have explored structuring our command-line interface so it can scale, we can investigate structuring our code over multiple files to scale it in the next section.

More books: https://t.me/booksforits

Structuring code

We can now begin our journey of building a web application. In the rest of this chapter, we will not touch a web framework or build an HTTP listener. This will happen in the next chapter. However, we will construct a to-do module that will interact with a JSON file. It is going to be structured in such a way that it can be inserted into any web application that we build with minimal effort. This to-do module will enable us to create, update, and delete to-do items. We will then interact with this via the command line. The process here is to explore how to build well-structured code that will scale and be flexible. To gain an understanding of this, we will break down the building of this module into the following chunks:

1. Build structs for pending and done to-do items.
2. Build a factory that enables the structs to be built in the module with minimal clean input.
3. Build traits that enable a struct to delete, create, edit, and get to-do items.
4. Build a read-and-write-to-file module to store to-do items (we will replace this with a proper database in later chapters).
5. Build a `config` module that can alter the behavior of the application based on the variables in a `config` file.

Before we start tackling these steps, we need to get the application running. We can do this by navigating to the desired directory of where we want to house this application and start a new Cargo project called `todo_app`. Once this is done, we are going to put the logic that handles the management of to-do items in our `to_do` module. This can be achieved by creating a `to_do` directory and putting a `mod.rs` file at the base of this directory, as seen in the following layout:

```
├── main.rs
└── to_do
    ├── mod.rs
```

With this structure, we can start building out our `to_do` module starting with structs. Do not worry about the `to_do` file for now as this is covered in the first step, building structs for our done and pending to-do items.

Building to-do structs

Right now, we only have two structs for to-do items: ones that are waiting to be done and others that are already done. However, we might want to introduce other categories. For instance, we could add a backlog category, or an *on-hold* task for tasks that have been started but for one reason or another are blocked. To avoid mistakes and repetitive code, we can build a `Base` struct and have that be utilized by other structs. The `Base` struct houses common fields and functions. An alteration of the `Base` struct will propagate to all other to-do structs. We will also need to define the type of to-do item. We

More books: https://t.me/booksforits

could hardcode in strings for pending and done; however, this is not scalable and is also error prone. To avoid this, we will use an enum to classify and define the presentation of the type of to-do item. To achieve this, we need to create the following file structure for our module:

```
├── main.rs
└── to_do
    ├── enums.rs
    ├── mod.rs
    └── structs
        ├── base.rs
        ├── done.rs
        ├── mod.rs
        └── pending.rs
```

In the preceding code, we can notice that we have two `mod.rs` files. These files are essentially where we declare our files, and what we define in them to make them accessible to other files in the same directory. We can also allow the files to be accessed outside of the directory if we publicly declare them in the `mod.rs` file. Before we write any code, we can see in *Figure 2.3* how the data flows in our module:

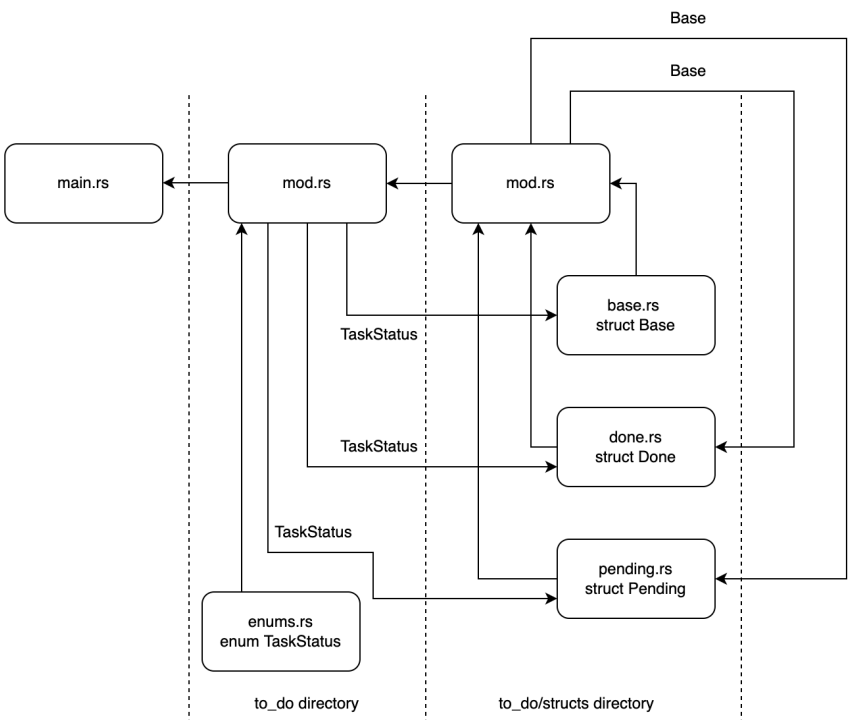


Figure 2.3 – The flow of data in our to-do module

More books: https://t.me/booksforits

We see that our `Base` struct is used by our other to-do structs. The other to-do structs would not be able to access the `Base` struct if we did not declare it. However, no file outside the `to_do/structs` directory is referencing the `Base` struct, therefore it does not have to be a public declaration.

Now that we understand the data flow for our module, we need to look back at *Figure 2.3* and work out what we need to work on first. We can see that our enums have no dependencies. In fact, our enum supplies all the structs. Therefore, we will start with our enum in the `/to_do/enums.rs` file. Our enum is defining the status of the task with the following code:

```
pub enum TaskStatus {
    DONE,
    PENDING
}
```

This will work in the code when it comes to defining the status of the task. However, if we want to write to a file or database, we are going to have to build a method to enable our enum to be represented in a string format. To do this, we can implement a `stringify` function for our `TaskStatus` enum with the following code:

```
impl TaskStatus {

    pub fn stringify(&self) -> String {
        match &self {
            &Self::DONE => {"DONE".to_string()},
            &Self::PENDING => {"PENDING".to_string()}
        }
    }
}
```

Calling this will enable us to print out the status of the to-do task in the console and write it in our JSON file.

> **Note**
>
> While the `stringify` function works, there is another way to convert the value of the enum to a string. To achieve the string conversion, we can implement the `Display` trait for `TaskStatus`. First, we must import the format module with the following code:

```
use std::fmt;
```

More books: https://t.me/booksforits

We can then implement the `Display` trait for the `TaskStatus` struct with the following code:

```
impl fmt::Display for TaskStatus {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match &self {
            &Self::DONE => {write!(f, "DONE")},
            &Self::PENDING => {write!(f, "PENDING")}
        }
    }
}
```

This trait implementation has the same logic as our `stringify` function. However, our trait is utilized when needed. So, we have the following code:

```
println!("{}", TaskStatus::DONE);
println!("{}", TaskStatus::PENDING);
let outcome = TaskStatus::DONE.to_string();
println!("{}", outcome);
```

This will result in the following printout:

```
DONE
PENDING
DONE
```

Here, we can see that when we pass `TaskStatus` into `println!`, the `Display` trait is automatically utilized.

We can now make our enum publicly available in the `/to_do/mod.rs` file with the following code:

```
pub mod enums;
```

We can now refer to *Figure 2.3* to see what we can build next, which is the `Base` struct. We can define the `Base` struct in the `/to_do/structs/base.rs` file with the following code:

```
use super::super::enums::TaskStatus;

pub struct Base {
    pub title: String,
    pub status: TaskStatus
}
```

More books: https://t.me/booksforits

From the import at the top of the file, we can access the `TaskStatus` enum using `super::super`. We know that the `TaskStatus` enum is in a higher directory. From this, we can deduce that `super` gives us access to what is declared in the `mod.rs` file of the current directory. So, using `super::super` in a file in the `/to_do/structs/` directory gives us access to what is defined in the `/to_do/mod.rs` file.

We can now declare our `Base` struct in our `/to_do/structs/mod.rs` file with the following code:

```
mod base;
```

We do not have to declare it as public because our `Base` struct is not accessed outside of the `/to_do/structs/` directory. Now, looking back at *Figure 2.3*, we can build our `Pending` and `Done` structs. This is when we use composition to utilize our `Base` struct in our `/to_do/structs/pending.rs` file with the following code:

```
use super::base::Base;
use super::super::enums::TaskStatus;

pub struct Pending {
    pub super_struct: Base
}
impl Pending {
    pub fn new(input_title: &str) -> Self {
        let base = Base{
            title: input_title.to_string(),
            status: TaskStatus::PENDING
        };
        return Pending{super_struct: base}
    }
}
```

Through the preceding code, we can see that our `super_struct` field houses our `Base` struct. We utilize our enum and define the status to be pending. This means that we only must pass the title into the constructor, and we have a struct with a title and a status of pending. Considering this, coding our `Done` struct should be straightforward in our `/to_do/structs/done.rs` file with the following code:

```
use super::base::Base;
use super::super::enums::TaskStatus;

pub struct Done {
```

More books: https://t.me/booksforits

```
    pub super_struct: Base
}

impl Done {
    pub fn new(input_title: &str) -> Self {
        let base = Base {
            title: input_title.to_string(),
            status: TaskStatus::DONE
        };
        return Done{super_struct: base}
    }
}
```

We can see that there is not much difference from the `Pending` struct definition apart from the `TaskStatus` enum having a `DONE` status. We can now make our structs available outside of the directory in the `/to_do/structs/mod.rs` file with the following code:

```
mod base;
pub mod done;
pub mod pending;
```

We can then make our structs accessible in the `main.rs` file by declaring these structs in the `/to_do/mod.rs` file with the following code:

```
pub mod structs;
pub mod enums;
```

We have now made a basic module and exposed it to the `main.rs` file. For now, we can write some basic code that will use our module to create a task that is pending and another that is completed. This can be done with the following code:

```
mod to_do;
use to_do::structs::done::Done;
use to_do::structs::pending::Pending;

fn main() {
    let done = Done::new("shopping");
    println!("{}", done.super_struct.title);
    println!("{}", done.super_struct.status.stringify());
```

More books: https://t.me/booksforits

```
    let pending = Pending::new("laundry");
    println!("{}", pending.super_struct.title);
    println!("{}", pending.super_struct.status.stringify()
    );
}
```

In the preceding code, we can see that we have declared our `to_do` module. We then imported our structs and created a `pending` and `done` struct. Running our code will give us the following printout:

```
shopping
DONE
laundry
PENDING
```

This stops the `main.rs` file from being overloaded with excessive code. If we were to stack more types of items that can be created, such as the on-hold or backlog items, the code in `main.rs` would balloon. This is where factories come in, which we will explore in the next step.

Managing structs with factories

A factory pattern is where we abstract the construction of structs in an entry point of the module. We can see how this would work with our module as follows:

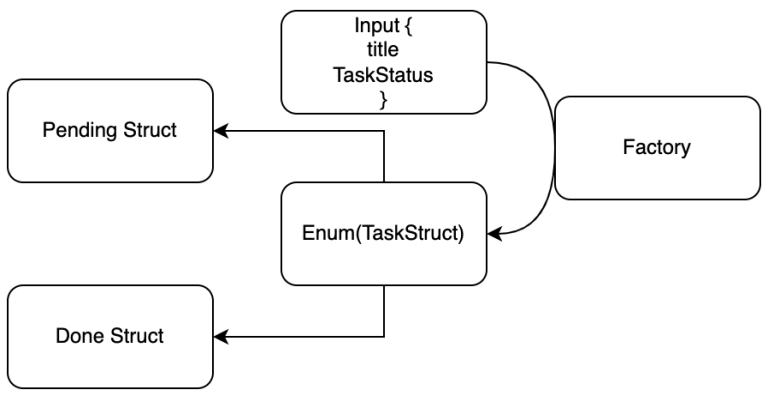


Figure 2.4 – Flow of to-do factory

What factories do is abstract the module by providing an interface. While we have enjoyed building our module, if another developer wanted to use it, a simple factory interface with good documentation would save them a lot of time. All they must do is pass in a few parameters and get the constructed structs out of the factory wrapped in an enum. If we change the internals of the module or it becomes

More books: https://t.me/booksforits

more complicated, this will not matter. If other modules use the interface, the changes would not break the rest of the code if we keep the interfaces consistent. We can build our factory by defining our factory function in the `/to_do/mod.rs` file with the following code:

```
pub mod structs;
pub mod enums;

use enums::TaskStatus;
use structs::done::Done;
use structs::pending::Pending;

pub enum ItemTypes {
    Pending(Pending),
    Done(Done)
}
pub fn to_do_factory(title: &str,
                     status: TaskStatus) -> ItemTypes {
    match status {
        TaskStatus::DONE => {
            ItemTypes::Done(Done::new(title))
        },
        TaskStatus::PENDING => {
            ItemTypes::Pending(Pending::new(title))
        }
    }
}
```

In the preceding code, we can see that we define an enum called `ItemTypes`, which packages the constructed task structs. Our `factory` function essentially accepts our inputted title and status. The factory then matches the inputted status. Once we have established what type of status was passed in, we build a task that matches the status and wraps it in the `ItemTypes` enum. This can grow and get more complicated, and our main file will be none the wiser. We can then implement this factory in our `main.rs` file with the following code:

```
mod to_do;
use to_do::to_do_factory;
use to_do::enums::TaskStatus;
use to_do::ItemTypes;
```

More books: https://t.me/booksforits

```
fn main() {
    let to_do_item = to_do_factory("washing",
                                   TaskStatus::DONE);
    match to_do_item {
        ItemTypes::Done(item) => {
            println!("{}", item.super_struct.status
                     .stringify());
            println!("{}", item.super_struct.title);
        },
        ItemTypes::Pending(item) => {
            println!("{}", item.super_struct.status
                     .stringify());
            println!("{}", item.super_struct.title);
        }
    }
}
```

In the preceding code, we can see that we pass into the factory the parameters we want to create for a to-do item, and then match the outcome to print the item's attributes. There is more code now introduced into the `main.rs` file. However, there is more code because we are unwrapping the returned enum to print out the attributes for demonstrative purposes. We will usually pass this wrapped enum into other modules to handle. To create the struct, we only need one line of code, which is the following:

```
let to_do_item = to_do_factory("washing", TaskStatus::DONE);
```

This means we can create a to-do item and pass it around with little hassle as it is wrapped in an enum. Other functions and modules must just accept the enum. We can see that this offers flexibility. However, as our code stands, we can do away with the `Base`, `Done`, and `Pending` structs and just have one struct that accepts the status and title. It would mean less code. However, it would also be less flexible. We are going to see how this is the case in our next step, where we add traits to our structs to lock down functionality and ensure safety.

Defining functionality with traits

Right now, our structs do not really do anything apart from holding the status and title of the task. However, our structs could have different functionality. Therefore, we have taken the extra trouble of defining individual structs. For instance, we are building a to-do application here. It is ultimately down to you how you structure your application, but it is not unreasonable to ensure that you cannot create a done task; otherwise, why are you adding it to your to-do list? This example might seem trivial. This book has used a to-do list to keep the problem that we are solving simple. Because of this,

More books: https://t.me/booksforits

we can focus on the technical aspects of developing web applications in Rust without spending time understanding the problem we are solving. However, we must acknowledge that in more demanding applications, such as a system that processes bank transactions, we would need to be strict on how to implement our logic and lock down the possibility of any undesired processes happening. We can do this in our to-do application by building individual traits for each process and assigning them to task structs that we want. To do this, we will need to create a `traits` directory in our `to_do` module and a file for each trait, which will take the following structure:

```
├── mod.rs
└── traits
    ├── create.rs
    ├── delete.rs
    ├── edit.rs
    ├── get.rs
    └── mod.rs
```

We can then publicly define all the traits in the `to_do/traits/mod.rs` file with the following code:

```
pub mod create;
pub mod delete;
pub mod edit;
pub mod get;
```

We must also publicly define our traits in our `to_do/mod.rs` file with the following code:

```
pub mod traits;
```

Now that we have all our trait files plumbed up in our module, we can start building our traits. We can start by defining our `Get` trait in the `to_do/traits/get.rs` file with the following code:

```
pub trait Get {
    fn get(&self, title: &str) {
        println!("{} is being fetched", title);
    }
}
```

This is simply a demonstration of how we apply traits; therefore, we will just print out what is happening for now. We must remember that we cannot reference fields from the `&self` parameter passed in because we can apply our trait to multiple structs; however, we can overwrite the `get` function for the trait that implements this. When it comes to the `Edit` trait, we can have two functions that change the status in the `to_do/traits/edit.rs` file with the following code:

More books: https://t.me/booksforits

```
pub trait Edit {
    fn set_to_done(&self, title: &str) {
        println!("{} is being set to done", title);
    }
    fn set_to_pending(&self, title: &str) {
        println!("{} is being set to pending", title);
    }
}
```

We can see a pattern here. So, for completeness, our `Create` trait takes the following form in the `to_do/traits/create.rs` file:

```
pub trait Create {
    fn create(&self, title: &str) {
        println!("{} is being created", title);
    }
}
```

Our `Delete` trait is defined in the `to_do/traits/delete.rs` file with the following code:

```
pub trait Delete {
    fn delete(&self, title: &str) {
        println!("{} is being deleted", title);
    }
}
```

We have now defined all the traits that we need. Thus, we can utilize them to define and lock down behavior in our to-do item structs. For our `Done` struct, we can import our traits into the `to_do/structs/done.rs` file with the following code:

```
use super::super::traits::get::Get;
use super::super::traits::delete::Delete;
use super::super::traits::edit::Edit;
```

We can then implement our `Done` struct in the same file after the definition of the `Done` struct with the following code:

```
impl Get for Done {}
impl Delete for Done {}
impl Edit for Done {}
```

More books: https://t.me/booksforits

Now, our `Done` struct can get, edit, and delete to-do items. Here, we can really see the power of traits as highlighted in *Chapter 1*, *A Quick Introduction to Rust*. We can stack on or remove traits easily. For instance, allowing done to-do items to be created would be achieved with a simple `impl Create for Done;`. Now that we have defined the traits that we want for our `Done` struct, we can move on to our `Pending` struct, importing what we need in the `to_do/structs/pending.rs` file with the following code:

```
use super::super::traits::get::Get;
use super::super::traits::edit::Edit;
use super::super::traits::create::Create;
```

Then, we can implement these traits after the definition of our `Pending` struct with the following code:

```
impl Get for Pending {}
impl Edit for Pending {}
impl Create for Pending {}
```

In the preceding code, we can see that our `Pending` struct can get, edit, and create but cannot delete. Implementing these traits also ties our `Pending` and `Done` structs together without compiling them. For instance, if we accepted a struct that implemented the `Edit` trait, it would accept both the `Pending` and `Done` structs. However, if we were to create a function that accepted structs that implemented the `Delete` trait, it would accept the `Done` struct but reject the `Pending` struct. This gives us a beautiful symphony of aggressive type-checking yet flexibility, which is truly a testament to Rust's design. Now that our structs have all the traits that we want, we can completely rewrite our `main.rs` file utilizing them. First, we import what we need with the following code:

```
mod to_do;

use to_do::to_do_factory;
use to_do::enums::TaskStatus;
use to_do::ItemTypes;

use crate::to_do::traits::get::Get;
use crate::to_do::traits::delete::Delete;
use crate::to_do::traits::edit::Edit;
```

The imports are important to note this time around. Although we have implemented our traits on the structs that we want, we will have to import the traits into the file that is using them. This can be a bit confusing. For example, calling the `get` function from the `Get` trait after a struct has been initialized would take the form of `item.get(&item.super_struct.title);`. The `get` function is tethered to the initialized struct. Intuitively, it makes sense not to need to import the trait. However,

More books: https://t.me/booksforits

your compiler or IDE will give you the unhelpful error that the function named `get` is not found in the struct if you do not import the trait. This is important as we will use traits from database crates and web frameworks in the future and we will need to import these traits for the package structs to be used. With our imports, we can then utilize our traits and factory in the `main` function with the following code:

```
fn main() {
    let to_do_items = to_do_factory("washing",
                                    TaskStatus::DONE);
    match to_do_items {
        ItemTypes::Done(item) => {
            item.get(&item.super_struct.title);
            item.delete(&item.super_struct.title);
        },
        ItemTypes::Pending(item) => {
            item.get(&item.super_struct.title);
            item.set_to_done(&item.super_struct.title);
        }
    }
}
```

Running the preceding code gives us the following printout:

```
washing is being fetched
washing is being deleted
```

What we have done here is build our own module, which contains an entry point. We've then imported it into the `main` function and run it. Now, the basic structure is built and working, but we need to get the module to interact with the environment by passing variables in and writing to a file to become useful.

Interacting with the environment

To interact with the environment, we must manage two things. First, we need to load, save, and edit the state of to-do items. Second, we also must accept user input to edit and display data. Our program can achieve this by running the following steps for each process:

1. Collect arguments from the user.
2. Define a command (`get`, `edit`, `delete`, and `create`) and define a to-do title from commands being passed into the application.

More books: https://t.me/booksforits

3. Load a JSON file that stores the to-do items from previous runs of the program.
4. Run a `get`, `edit`, `delete`, or `create` function based on the command passed into the program, saving the result of the state in a JSON file at the end.

We can start making this four-step process possible by initially loading our state with the `serde` crate.

Reading and writing JSON files

We are now at the stage where we are going to persist data in the form of a JSON file. We will upgrade this to a proper database in *Chapter 6*, *Data Persistence with PostgreSQL*. But for now, we are going to introduce our first dependency in our web application, which is `serde_json`. This `serde_json` crate handles the conversion of Rust data to JSON data and vice versa. We will use `serde_json` to process HTTP requests in the next chapter. We can install our crate in the `Cargo.toml` file with the following code:

```
[dependencies]
serde_json="1.0.59"
```

Seeing as we are going to be upgrading our storage option in the future, it makes sense to keep the operations around reading and writing to our JSON file separate from the rest of the application. We do not want a lot of debugging and refactoring when we pull it out for our database upgrade. We will also keep it simple as there are no schema or migrations that must be managed when reading and writing to a JSON file. Considering this, all we will need are `read` and `write` functions. As our module is small and simple, we can house our module in just one file next to the `main.rs` file. First, we need to import what we need in our `src/state.rs` file with the following code:

```
use std::fs::File;
use std::fs;
use std::io::Read;

use serde_json::Map;
use serde_json::value::Value;
use serde_json::json;
```

As we can see, we need the standard library and a series of structs to read the data as mapped out in *Figure 2.5*:

More books: https://t.me/booksforits

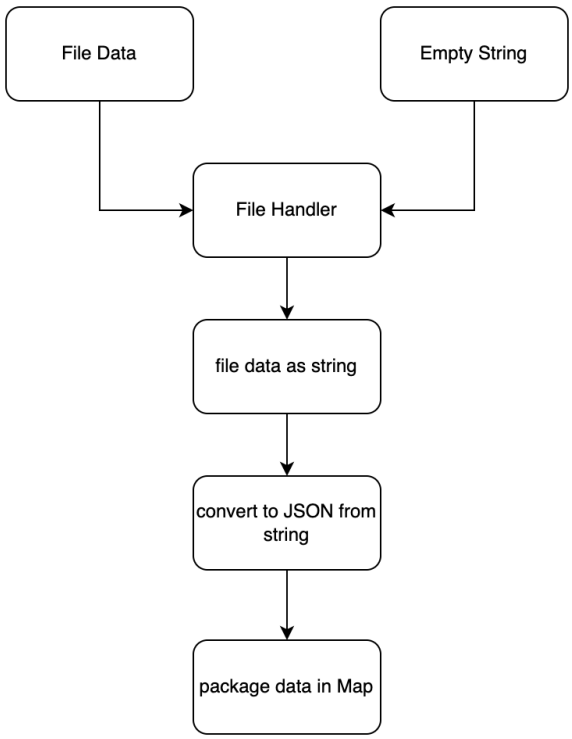


Figure 2.5 – Steps to read a JSON file

We can carry out the steps in *Figure 2.5* with the following code:

```
pub fn read_file(file_name: &str) -> Map<String, Value> {
    let mut file = File::open(file_name.to_string()).unwrap();

    let mut data = String::new();
    file.read_to_string(&mut data).unwrap();

    let json: Value = serde_json::from_str(&data).unwrap();
    let state: Map<String, Value> = json.as_object()
                                    .unwrap().clone();
    return state
}
```

In the preceding code, we can see that we directly unwrap the opening of the file. This is because there is no point in continuing the program if we cannot read the file, and as a result, we directly unwrap

More books: https://t.me/booksforits

the file read. We must also note that the string must be mutable as we are going to fill this string with JSON data. Additionally, we use the `serde_json` crate to process the JSON data and configure it into a map. We can now access our to-do items via this `Map` variable throughout the rest of the program. Now, we need to write our data, which can be done in the same file with the following code:

```
pub fn write_to_file(file_name: &str,
                     state: &mut Map<String, Value>) {
    let new_data = json!(state);
    fs::write(
          file_name.to_string(),
          new_data.to_string()
    ).expect("Unable to write file");
}
```

In the preceding code, we accept our `Map` variable and the path to the file. We then convert our `Map` variable into JSON using the `json!` macro from our `serde_json` crate. We then convert the JSON data to a string and then write it to our JSON file. Because of this, we now have functions to read and write to-do items to JSON files. We can now upgrade our `main.rs` file and build a simple command-line to-do application that reads and writes to-do items to a JSON file. We can interact with this using some basic arguments passed into the program with the following code:

```
mod state;

use std::env;
use state::{write_to_file, read_file};
use serde_json::value::Value;
use serde_json::{Map, json};

fn main() {
    let args: Vec<String> = env::args().collect();
    let status: &String = &args[1];
    let title: &String = &args[2];
    let mut state: Map<String, Value> =
    read_file("./state.json");
    println!("Before operation: {:?}", state);
    state.insert(title.to_string(), json!(status));
    println!("After operation: {:?}", state);
    write_to_file("./state.json", &mut state);
}
```

More books: https://t.me/booksforits

In the preceding code, we have done the following:

1. Collected the status and title from arguments passed into our program
2. Read the to-do items from the JSON file
3. Printed out the to-do items from the JSON file
4. Inserted our new to-do item
5. Printed out the new set of to-do items from memory
6. Written our new to-do items list to our JSON file

Our root path is going to be where the `Cargo.toml` file is, so we define an empty JSON file called `state.json` next to the `Cargo.toml` file. To interact with it, we can pass in the following command:

```
cargo run pending washing
```

The preceding command would result in the following printout:

```
Before operation: {}
After operation: {"washing": String("pending")}
```

In the preceding output, we can see that `washing` has been inserted. An inspection of our JSON file would also show that `washing` has been written to the file. You may have noticed that we have removed any mention of our `to_do` module, including all the structs and traits that we have built. We have not forgotten them. Instead, we are merely testing to see whether our interaction with the JSON file works before we try and fuse `to_do` with the `state` module. We will fuse the `to_do` and `state` modules by revising the traits implemented in our to-do structs in the next section.

Revisiting traits

Now that we have defined the module around managing the state of our to-do items in a JSON file, we have an idea of how our trait functions will process the data and interact with our JSON file. To start off, we can update our simplest trait, which is the `Get` trait in our `src/to_do/traits/get.rs` file. Here, we are merely getting the to-do item from our JSON map and printing it out. We can do this by simply passing the JSON map into our `get` function, getting the to-do item status from the map using the to-do item title from the state, and printing it out to the console with the following code:

```
use serde_json::Map;
use serde_json::value::Value;

pub trait Get {
    fn get(&self, title: &String, state: &Map<String, Value>) {
        let item: Option<&Value> = state.get(title);
```

More books: https://t.me/booksforits

```
        match item {
            Some(result) => {
                println!("\n\nItem: {}", title);
                println!("Status: {}\n\n", result);
            },
            None => println!("item: {} was not found",
                             title)
        }
    }
}
```

In the preceding code, we can see that we perform a `get` function on our JSON `Map`, matching the outcome and printing out what we extract. This means that any to-do item that implements the `Get` trait can now extract a to-do item from our state and print it out.

We can now move on to the next step in complexity, which is the `Create` trait in our `src/to_do/traits/create.rs` file. This is slightly more complex than our `Get` trait because we edit the state by inserting the new to-do item and then writing this updated state into our JSON. We can carry out these steps with the following code:

```
use serde_json::Map;
use serde_json::value::Value;
use serde_json::json;

use crate::state::write_to_file;

pub trait Create {
    fn create(&self, title: &String, status: &String,
              state: &mut Map<String, Value>) {
            state.insert(title.to_string(), json!(status));
            write_to_file("./state.json", state);
            println!("\n\n{} is being created\n\n", title);
    }
}
```

More books: https://t.me/booksforits

In the preceding code, we can see that we have used the `write_to_file` function from the `state` module to save our state to the JSON file. We can use the `create` function as a template for what we need to do when deleting a to-do item. Deleting is essentially the inverse of what we have done in our `create` function. You can try to write the `delete` function for the `Delete` trait in the `src/to_do/traits/delete.rs` file now before moving on if you want. Your function may look different; however, it should run along the same lines as the following code:

```
use serde_json::Map;
use serde_json::value::Value;

use crate::state::write_to_file;

pub trait Delete {
    fn delete(&self, title: &String,
        state: &mut Map<String, Value>) {
        state.remove(title);
        write_to_file("./state.json", state);
        println!("\n\n{} is being deleted\n\n", title);
    }
}
```

In the preceding code, we are merely using the `remove` function on our JSON `Map`, writing the updated state to our JSON file. We are near the end of the section. All we need to do now is build our `edit` function for our `Edit` trait in the `src/to_do/traits/edit.rs` file. We have two functions. One will set the to-do item status to `DONE`. The other function will set the to-do item status to `PENDING`. These will be achieved by updating the state and then writing the updated state to the JSON file. You can try and write this yourself before reading on. Hopefully, your code will look like the following code:

```
use serde_json::Map;
use serde_json::value::Value;
use serde_json::json;

use crate::state::write_to_file;
use super::super::enums::TaskStatus;
```

More books: https://t.me/booksforits

```
pub trait Edit {
    fn set_to_done(&self, title: &String,
        state: &mut Map<String, Value>) {
        state.insert(title.to_string(),
        json!(TaskStatus::DONE.stringify()));
        write_to_file("./state.json", state);
        println!("\n\n{} is being set to done\n\n", title);
    }
    fn set_to_pending(&self, title: &String,
        state: &mut Map<String, Value>) {
        state.insert(title.to_string(),
        json!(TaskStatus::PENDING.stringify()));
        write_to_file("./state.json", state);
        println!("\n\n{} is being set to pending\n\n", title);
    }
}
```

Our traits can now interact without the JSON file, carrying out the processes that we initially wanted them to do. There is nothing stopping us from utilizing these traits directly in the `main.rs` file as we did when we first defined these traits. However, this is not scalable. This is because we will essentially be building a web application with multiple views and API endpoints. Therefore, we will be interacting with these traits and storage processes in multiple different files. Therefore, we are going to have to come up with a way to interact with these traits in a standardized way without having to repeat code, which we will do in the next section.

Processing traits and structs

To enable our code to interact with a simple interface, enabling us to update with minimal pain and reduce repeated code and thus errors, we need a processes layer, as seen in *Figure 2.6*:

More books: https://t.me/booksforits

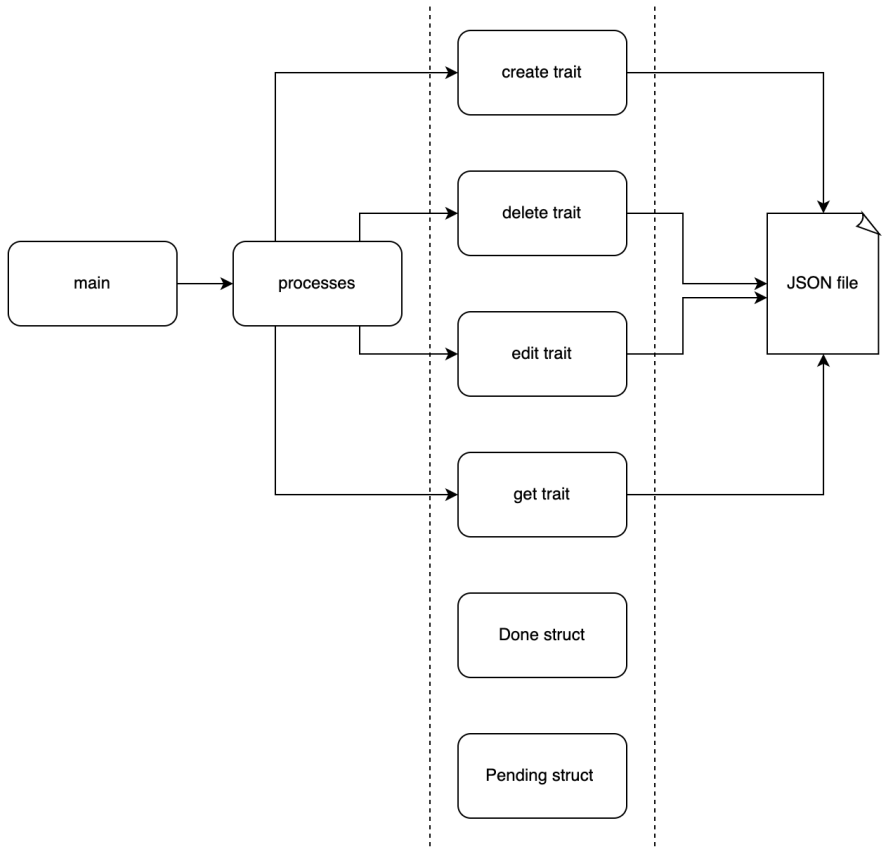


Figure 2.6 – Trait flow through a processes module

In *Figure 2.6*, we can see that our structs are bound to the traits in a loose fashion and that the data flow to and from the JSON goes through the traits. We can also see that we have one entry point in the `processes` module, which will then direct commands to the correct traits, which in turn will access the JSON file as and when needed. Seeing as we have defined our traits, all we need to do is build our `processes` module, connect it to the traits, and then connect it to our `main.rs` file. We will be building our entire `processes` module in a single `src/processes.rs` file. We are keeping it to a single file because we will be removing it when we cover databases. There is no need to take on too much technical debt if we know we are going to be removing it in the future. For now, we can start building our `processes` module by initially importing all the structs and traits that we need with the following code:

```
use serde_json::Map;
use serde_json::value::Value;
```

More books: https://t.me/booksforits

```
use super::to_do::ItemTypes;
use super::to_do::structs::done::Done;
use super::to_do::structs::pending::Pending;
use super::to_do::traits::get::Get;
use super::to_do::traits::create::Create;
use super::to_do::traits::delete::Delete;
use super::to_do::traits::edit::Edit;
```

We can now start to build non-public functions. We can start by processing our `Pending` structs. We know that we can either get, create, or edit pending to-do items, as seen in the following code:

```
fn process_pending(item: Pending, command: String,
                   state: &Map<String, Value>) {
    let mut state = state.clone();
    match command.as_str() {
    "get" => item.get(&item.super_struct.title, &state),
    "create" => item.create(&item.super_struct.title,
    &item.super_struct.status.stringify(), &mut state),
    "edit" => item.set_to_done(&item.super_struct.title,
                               &mut state),
    _ => println!("command: {} not supported", command)
    }
}
```

In the preceding code, we can see that we have ingested the `Pending` struct, command, and current state of our to-do items. We then match the command and execute the trait associated with that command. If there the command that is passed in is neither `get`, `create`, or `edit`, we do not support it, throwing an error that tells the user what command is not supported. This is scalable. For instance, if we allow a `Pending` struct to delete a to-do item from the JSON file, we merely must implement the `Delete` trait for the `Pending` struct and then add the `delete` command to our `process_pending` function. This would only take two lines of code in total, and this change would take effect throughout the application. This would also happen if we removed a command. We now have a flexible implementation of our `Pending` struct. With this in mind, you can choose to code our `process_done` function before reading on. If you have chosen to do so, hopefully, it will look like the following code:

```
fn process_done(item: Done, command: String,
                state: &Map<String, Value>) {
    let mut state = state.clone();
```

More books: https://t.me/booksforits

```
    match command.as_str() {
        "get" => item.get(&item.super_struct.title,
                          &state),
        "delete" => item.delete(&item.super_struct.title,
                                &mut state),
        "edit" =>
             item.set_to_pending(&item.super_struct.title,
                                 &mut state),
        _ => println!("command: {} not supported", command)
    }
}
```

We can now process both our structs. This is where the scalability of structs comes in when designing our module. Like the commands, we will want to stack our structs just like we did with our traits. This is where our entry point comes in, as seen in *Figure 2.7*:

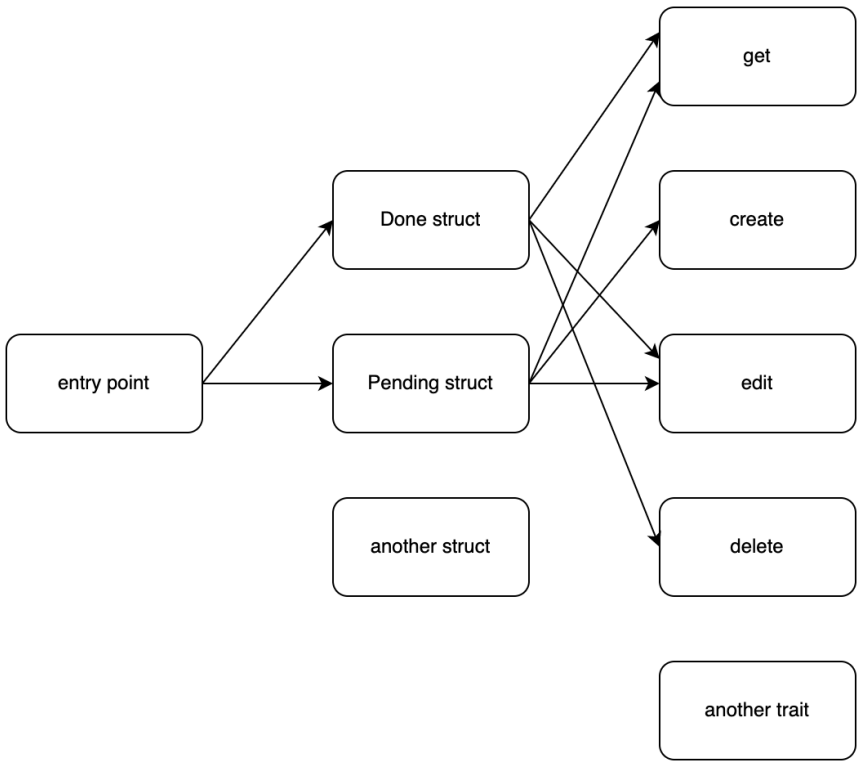


Figure 2.7 – Scalability of processes

More books: https://t.me/booksforits

We can see from *Figure 2.7* that we can scale the access to structs by increasing the routes by the entry point. To appreciate this more, we should define our entry point, which this time is a public function, with the following code:

```
pub fn process_input(item: ItemTypes, command: String,
                     state: &Map<String, Value>) {
    match item {
        ItemTypes::Pending(item) => process_pending(item,
                                    command, state),
        ItemTypes::Done(item) => process_done(item,
                                    command, state)
    }
}
```

In the preceding code, we can see that we route to the correct struct with the `ItemTypes` enum. Our module can process more structs by adding the new struct to the `ItemTypes` enum, writing a new function that processes that struct in the `processes` module, and then applying the desired traits to the struct. Our `processes` module is now fully completed, and we can rewrite our `main.rs` file to utilize it. First of all, we import what we need with the following code:

```
mod state;
mod to_do;
mod processes;

use std::env;

use serde_json::value::Value;
use serde_json::Map;

use state::read_file;
use to_do::to_do_factory;
use to_do::enums::TaskStatus;
use processes::process_input;
```

With these imports, we can see that we are going to be reading the data from the JSON file, using `to_do_factory` to create the structs from the input collected from the environment, and passing this into our `processes` module to update the JSON file. This is a good point in time to stop reading and try to code out this process by yourself. Remember, you must get the data from the JSON file and check to see whether the title of the to-do item is already stored in the JSON file. If we cannot find the

More books: https://t.me/booksforits

title in the data from the JSON file, we then know that it is going to be a pending status as we cannot create a done task. If you chose to do this, your code hopefully looks like the following:

```
fn main() {
    let args: Vec<String> = env::args().collect();
    let command: &String = &args[1];
    let title: &String = &args[2];

    let state: Map<String, Value> = read_file("./state.json");
    let status: String;

    match &state.get(*&title) {
        Some(result) => {
            status = result.to_string().replace('\"', "");
        }
        None=> {
            status = "pending".to_owned();
        }
    }
    let item = to_do_factory(title,
                             TaskStatus::from_string(
                             status.to_uppercase()));
    process_input(item, command.to_string(), &state);
}
```

Before we run anything, you might have realized that we created an instance of the `TaskStatus` using the `from_string` function. We have not built the `from_string` function yet. At this point you should be able to build it yourself in the `impl TaskStatus` block. If you have attempted to build the `from_string` function, it should look like the following code in the `src/to_do/enums.rs` file:

```
impl TaskStatus {
    . . .
    pub fn from_string(input_string: String) -> Self {
        match input_string.as_str() {
            "DONE" => TaskStatus::DONE,
            "PENDING" => TaskStatus::PENDING,
            _ => panic!("input {} not supported",
```

More books: https://t.me/booksforits

```
                            input_string)
            }
        }
}
```

If you have managed to utilize the interfaces that we have created to get our program running, then well done. We can see now that we orchestrate a range of processes in our `main` function with ease. We can interact with our program with the following command:

```
cargo run create washing
```

The preceding command creates a to-do item called `washing` in our JSON file with the status of pending. All our other traits are supported, and we can carry them out in the command line as well. We have now built a basic command application that stores to-do items in a JSON file. However, it is not just a basic command-line application. We have structured our modules so they are scalable and flexible.

Summary

What we have essentially done in this chapter is build a program that accepts some command-line inputs, interacts with a file, and edits it depending on the command and data from that file. The data is simple: a title and a status. We could have done this all in the `main` function with multiple `match` statements and `if`, `else if`, and `else` blocks. However, this is not scalable. Instead, we built structs that inherited other structs, which then implemented traits. We then packaged the construction of these structs into a factory, enabling other files to use all that functionality in a single line of code.

We then built a processing interface so the command input, state, and struct could be processed, enabling us to stack on extra functionality and change the flow of the process with a few lines of code. Our main function must only focus on collecting the command-line arguments and coordinating when to call the module interfaces. We have now explored and utilized how Rust manages modules, giving us the building blocks to build real-world programs that can solve problems and add features without being hurt by tech debt and a ballooning `main` function. Now that we can do this, we are ready to start building scalable web apps that can grow. In the next chapter, we will learn about the **Actix Web framework** to get a basic web server up and running.

Questions

1. What does the `--release` argument in Cargo do when added to a build and run?
2. How do we enable a file to be accessible within and outside the module?
3. What are the advantages of having traits with a single scope?
4. What steps would we have to take to add an `OnHold` to-do item that will only allow `Get` and `Edit` functionality?

More books: https://t.me/booksforits

5. What are the benefits of a factory function?
6. How do we effectively map a range of processes based on some processes?

Answers

1. In a build, the `--release` argument compiles the program in an optimized way as opposed to a debug compilation. In a run, the `--release` argument points to an optimized binary as opposed to the debug binary. An optimized binary takes longer to compile but will run at a faster pace.
2. To enable a file to be accessible to other files in a module, we must define the file as a module in the `mod.rs` file at the root of the module. We add `mod` before the definition to make it accessible outside the module.
3. Single-scope traits enable maximum flexibility when defining structs. A good example would be adding an `OnHold` to-do item. With this item, we might only allow it to have an edit trait, which we can do by implementing the single-scoped `Edit` trait. If we had one trait that did all the functions, this would not be possible.
4. Define a struct in its own file in structs that inherit from the base struct, which also implements the `Get` and `Edit` traits. Add a `hold` type to the enum in the factory file. Add another line in the `match` statement for the entry point in the processes that point to a new function processing the `OnHold` item.
5. The factory function standardizes the construction of structs. It also reduces the possibility of building one of a range of structs outside of the module with just one line of code. This stops other files from ballooning and does not require the developer to look around in the module to utilize it.
6. We use `match` statements that lead to other `match` statements. This enables us to code a tree-like effect and there is nothing stopping us from connecting branches later down the chain. This is demonstrated in *Figure 2.7*.

More books: https://t.me/booksforits

More books: https://t.me/booksforits

Part 2: Processing Data and Managing Displays

Now that we can build applications in Rust, we need to be able to handle HTTP requests. By the end of this part, you will know how to handle HTTP requests and route them. You will also be able to extract data from the request body and header. You will also structure the application so routing can scale and implement middleware to process and route HTTP requests before the view is loaded. Finally, you will understand how to display content in the browser by directly serving HTML, CSS, and JavaScript from the server. We will also explore a basic React application and wrap it in Electron to have a desktop application to talk to our Rust server. At this point, you will have learned everything to run a basic application without a proper database or authentication.

This part includes the following chapters:

- *Chapter 3*, *Handling HTTP Requests*
- *Chapter 4*, *Processing HTTP Requests*
- *Chapter 5*, *Displaying Content in the Browser*

More books: https://t.me/booksforits

More books: https://t.me/booksforits

3

Handling HTTP Requests

So far, we have structured our to-do module in a flexible, scalable, and reusable manner. However, this can only get us so far in terms of web programming. We want our to-do module to reach multiple people quickly without the user having to install Rust on their own computers. We can do this with a web framework. Rust has plenty to offer. Initially, we will build our main server in the **Actix Web** framework.

To achieve this, we will be building the views of the server in a modular fashion; we can slot our to-do module into our web application with minimal effort. It must be noted that the Actix Web framework defines views using `async` functions. Because of this, we will also cover asynchronous programming to get a better understanding of how the Actix Web framework works.

In this chapter, we will cover the following topics:

- Introducing the Actix Web framework
- Launching a basic Actix Web server
- Understanding closures
- Understanding asynchronous programming
- Understanding `async` and `await` with web programming
- Managing views using the Actix Web framework

Technical requirements

As we move toward building web apps in Rust, we are going to have to start relying on third-party packages to do some of the heavy lifting for us. Rust manages dependencies through a package manager called **Cargo**. To use Cargo, we are going to have to install Rust on our computer from the following URL: `https://www.rust-lang.org/tools/install`.

More books: https://t.me/booksforits

This installation delivers the Rust programming language and Cargo. You can find all the code files on GitHub at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter03`.

Introducing the Actix Web framework

At the time of writing, Actix Web is the most popular Rust web framework as can be seen from the activity on the GitHub page. You might be tempted to jump into another framework that looks more ergonomic, such as Rocket, or one that is faster and more lightweight, such as Hyper. We will be covering these frameworks later in this book over various different chapters; however, we must remember that we are trying to get our heads around web programming in Rust first. Considering that we are new to Rust and web programming, Actix Web is a great start. It is not too low-level that we will get caught up with just trying to get a server to handle a range of views, database connections, and authentication. It is also popular, stable, and has a lot of documentation. This will facilitate a pleasant programming experience when trying to go beyond the book and develop your own web application. It is advised that you get comfortable with Actix Web before moving on to other web frameworks. This is not to say that Actix Web is the best and that all other frameworks are terrible; it is just to facilitate a smooth learning and development experience. With this in mind, we can now move on to the first section, where we set up a basic web server.

Launching a basic Actix Web server

Building with Cargo is straightforward. All we need to do is navigate to a directory where we want to build our project and run the following command:

```
cargo new web_app
```

The preceding command builds a basic Cargo Rust project. When we explore this application, we get the following structure:

```
└── web_app
    ├── Cargo.toml
    └── src
        └── main.rs
```

We can now define our Actix Web dependency in our `Cargo.toml` file with the following code:

```
[dependencies]
actix-web = "4.0.1"
```

As a result of the preceding code, we can now move on to building the web application. For now, we will put it all in our `src/main.rs` file with the following code:

More books: https://t.me/booksforits

```
use actix_web::{web, App, HttpServer, Responder,
                HttpRequest};

async fn greet(req: HttpRequest) -> impl Responder {
    let name =
        req.match_info().get("name").unwrap_or("World");
    format!("Hello {}!", name)
}
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        App::new()
        .route("/", web::get().to(greet))
        .route("/{name}", web::get().to(greet))
        .route("/say/hello", web::get().to(||
                    async { "Hello Again!" }))

    })
    .bind("127.0.0.1:8080")?
    .run()
    .await
}
```

In the preceding code, we can see that we import the required structs and traits from the `actix_web` crate. We can see that we have used several different ways to define a view. We defined a view by building a function. This takes in an `HttpRequest` struct. It then gets `name` from the request and then returns a variable that can implement the `Responder` trait from the `actix_web` crate. The `Responder` trait converts our type into an HTTP response. We assign this `greet` function that we have created for our application server as the route view, with the `.route("/", web::get().to(greet))` command. We can also see that we can pass in the name from the URL to our `greet` function with the `.route("/{name}", web::get().to(greet))` command. Finally, we pass a closure into the final route. With our configuration, let's run the following command:

```
cargo run
```

We will get the following printout:

```
Finished dev [unoptimized + debuginfo] target(s) in 0.21s
 Running `target/debug/web_app`
```

More books: https://t.me/booksforits

We can see in the preceding output that, right now, there is no logging. This is expected, and we will configure logging later. Now that our server is running, for each of the following URL inputs, we should expect the corresponding outputs in the browser:

- **Input**: `http://127.0.0.1:8080/`
- **Output**: `Hello World!`
- **Input**: `http://127.0.0.1:8080/maxwell`
- **Output**: `Hello maxwell!`
- **Input**: `http://127.0.0.1:8080/say/hello`
- **Output**: `Hello Again!`

In the preceding code in the `src/main.rs` file, we can see that there is some new syntax that we have not come across before. We have decorated our `main` function with the `#[actix_web::main]` macro. This marks our `async main` function as the Actix Web system entry point. With this, we can see that our functions are `async` and that we are using closures to build our server. We will go through both concepts in the next couple of sections. In the next section, we will investigate closures to truly understand what is happening.

Understanding closures

Closures are, essentially, functions, but they are also anonymous, meaning that they do not have names. This means that closures can be passed around into functions and structs. However, before we delve into passing closures around, let us explore closures by defining a basic closure in a blank Rust program (you can use the Rust playground if you prefer) with the following code:

```
fn main() {
    let test_closure = |string_input| {
        println!("{}", string_input);
    };
    test_closure("test");
}
```

Running the preceding code will give us the following printout:

```
test
```

In the preceding output, we can see that our closure behaves like a function. However, instead of using curly brackets to define the inputs, we use pipes.

More books: https://t.me/booksforits

You might have noticed in the preceding closure that we have not defined the data type for the `string_input` parameter; however, the code still runs. This is different from a function that needs to have the parameter data types defined. This is because functions are part of an explicit interface that is exposed to users. A function could be called anywhere in the code if the code can access the function. Closures, on the other hand, have a short lifetime and are only relevant to the scope that they are in. Because of this, the compiler can infer the type being passed into the closure from the use of the closure in the scope. Because we are passing in `&str` when we call the closure, the compiler knows that the `string_input` type is `&str`. While this is convenient, we need to know that closures are not generic. This means that a closure has a concrete type. For instance, after defining our closure, let's try and run the following code:

```
    test_closure("test");
    test_closure(23);
```

We will get the following error:

```
7 |     test_closure(23);
  |                  ^^ expected `&str`, found integer
```

The error occurs because the first call to our closure tells the compiler that we are expecting `&str`, so the second call breaks the compilation process.

Scopes do not just affect closures. Closures adhere to the same scope rules that variables do. For instance, let's say we were going to try and run the following code:

```
fn main() {
    {
        let test_closure = |string_input| {
            println!("{}", string_input);
            };
    }
    test_closure("test");
}
```

It would refuse to compile because when we try and call the closure, it is not in the scope of the call. Considering this, you would be right to assume that other scope rules apply to closures. For instance, if we tried to run the following code, what do you think would happen?

```
fn main() {
    let another_str = "case";
    let test_closure = |string_input| {
        println!("{} {}", string_input, another_str);
```

More books: https://t.me/booksforits

```
    };
    test_closure("test");
}
```

If you thought that we would get the following output, you would be right:

```
test case
```

Unlike functions, closures can access variables in their own scope. So, to try and describe closures in a simplistic way that we can understand, they are kind of like dynamic variables in a scope that we call to perform a computation.

We can take ownership of the outside variables used in the closure by utilizing `move`, as seen with the following code:

```
let test_closure = move |string_input| {
    println!("{} {}", string_input, another_str);
};
```

Because `move` is utilized in the closure defined here, the `another_str` variable cannot be used after `test_closure` is declared because `test_closure` took ownership of `another_str`.

We can also pass closures into a function; however, it must be noted that we can also pass functions into other functions. We can achieve passing functions into other functions with the following code:

```
fn add_doubles(closure: fn(i32) -> i32,
               one: i32, two: i32) -> i32 {
    return closure(one) + closure(two)
}
fn main() {
    let closure = |int_input| {
        return int_input * 2
    };
    let outcome = add_doubles(closure, 2, 3);
    println!("{}", outcome);
}
```

In the preceding code, we can see that we define a closure that doubles an integer that is passed in and returned. We then pass this into our `add_doubles` function with the notation of `fn(i32)->i32`, which is known as a function pointer. When it comes to closures, we can implement one of the following traits:

- `Fn`: Immutably borrows variables

More books: https://t.me/booksforits

- `FnMut`: Mutably borrows variables
- `FnOnce`: Takes ownership of variables so it can only be called once

We can pass a closure that has one of the preceding traits implemented into our `add_doubles` function with the following code:

```
fn add_doubles(closure: Box<dyn Fn(i32) -> i32>,
               one: i32, two: i32) -> i32 {
    return closure(one) + closure(two)
}
fn main() {
    let one = 2;
    let closure = move |int_input| {
        return int_input * one
    };
    let outcome = add_doubles(Box::new(closure), 2, 3);
    println!("{}", outcome);
}
```

Here, we can see that the `closure` function parameter has the `Box<dyn Fn(i32) -> i32>` signature. This means that the `add_doubles` function is accepting closures that have implemented the `Fn` trait that accepted `i32`, and returned `i32`. The `Box` struct is a smart pointer where we have put the closure on the heap because we do not know the closure's size at compile time. You can also see that we have utilized `move` when defining the closure. This is because we are using the `one` variable, which is outside the closure. The `one` variable may not live long enough; therefore, the closure takes ownership of it because we used `move` when defining the closure.

With what we have covered about closures in mind, we can have another look at the `main` function in our server application with the following code:

```
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        App::new()
        .route("/", web::get().to(greet))
        .route("/{name}", web::get().to(greet))
        .route("/say/hello", web::get().to(
        || async { "Hello Again!" }))
    })
    .bind("127.0.0.1:8080")?
```

More books: https://t.me/booksforits

```
    .run()
    .await
}
```

In the preceding code, we can see that we are running our `HttpServer` after constructing it using the `HttpServer::new` function. Knowing what we know now, we can see that we have passed in a closure that returns the `App` struct. Based on what we know about closures, we can be more confident with what we do with this code. We can essentially do what we like within the closure if it returns the `App` struct. With this in mind, we can get some more information about the process with the following code:

```
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        println!("http server factory is firing");
        App::new()
        .route("/", web::get().to(greet))
        .route("/{name}", web::get().to(greet))
        .route("/say/hello", web::get().to(
               || async { "Hello Again!" }))
    })
    .bind("127.0.0.1:8080")?
    .workers(3)
    .run()
    .await
}
```

In the preceding code, we can see that we have added a `print` statement to tell us that the closure is firing. We also added another function called `workers`. This means we can define how many workers are being used to create our server. We also print out that the server factory is firing in our closure. Running the preceding code gives us the following printout:

```
    Finished dev [unoptimized + debuginfo] target(s) in
    2.45s
     Running `target/debug/web_app`
http server factory is firing
http server factory is firing
http server factory is firing
```

More books: https://t.me/booksforits

The preceding result tells us that the closure was fired three times. Altering the number of workers shows us that there is a direct relationship between this and the number of times the closure is fired. If the `workers` function is left out, then the closure is fired in relation to the number of cores your system has. We will explore how these workers fit into the server process in the next section.

Now that we understand the nuances around the building of the `App` struct, it is time to look at the main change in the structure of a program, asynchronous programming.

Understanding asynchronous programming

Up until this chapter, we have been writing code in a sequential manner. This is good enough for standard scripts. However, in web development, asynchronous programming is important, as there are multiple requests to servers, and API calls introduce idle time. In some other languages, such as Python, we can build web servers without touching any asynchronous concepts. While asynchronous concepts are utilized in these web frameworks, the implementation is defined under the hood. This is also true for the Rust framework Rocket. However, as we have seen, it is directly implemented in Actix Web.

When it comes to utilizing asynchronous code, there are two main concepts we must understand:

- **Processes**: A process is a program that is being executed. It has its own memory stack, registers for variables, and code.
- **Threads**: A thread is a lightweight process that is managed independently by a scheduler. However, it does share data, code, and the heap with other threads and the `main` program. However, threads do not share the stack.

This is demonstrated in the following classic diagram:

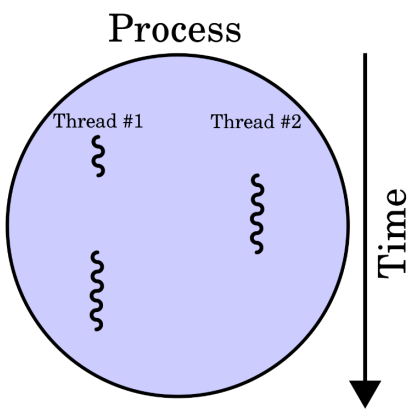


Figure 3.1 – Relationship between threads and processes [source: Cburnett (2007) (https://commons.wikimedia.org/wiki/File:Multithreaded_process.svg), CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0/deed.en)]

More books: https://t.me/booksforits

Now that we understand what threads are and what relation they have to our code on a high-level basis, we can play with a toy example to understand how to utilize threads in our code and see the effects of these threads firsthand. A classic example is to build a basic function that merely sleeps, blocking time. This can simulate a time-expensive function such as a network request. We can run it sequentially with the following code:

```
use std::{thread, time};

fn do_something(number: i8) -> i8 {
    println!("number {} is running", number);
    let two_seconds = time::Duration::new(2, 0);
    thread::sleep(two_seconds);
    return 2
}
fn main() {
    let now = time::Instant::now();
    let one: i8 = do_something(1);
    let two: i8 = do_something(2);
    let three: i8 = do_something(3);
    println!("time elapsed {:?}", now.elapsed());
    println!("result {}", one + two + three);
}
```

Running the preceding code will give us the following printout:

```
number 1 is running
number 2 is running
number 3 is running
time elapsed 6.0109845s
result 6
```

In the preceding output, we can see that our time-expensive functions run in the order that we expect them to. It also takes just over 6 seconds to run the entire program, which makes sense since we are running three expensive functions that sleep at 2 seconds each. Our expensive function also returns the value `2`. When we add the results of all three expensive functions together, we are going to get a result of the value `6`, which is what we have. We speed up our program to roughly 2 seconds for the entire program, by spinning up three threads at the same time and waiting for them to complete before moving on. Waiting for the threads to complete before moving on is called *joining*. So, before we start spinning off threads, we must import the `join` handler with the following code:

```
use std::thread::JoinHandle;
```

More books: https://t.me/booksforits

We can now spin up threads in our `main` function with the following code:

```
let now = time::Instant::now();

let thread_one: JoinHandle<i8> = thread::spawn(
    || do_something(1));
let thread_two: JoinHandle<i8> = thread::spawn(
    || do_something(2));
let thread_three: JoinHandle<i8> = thread::spawn(
    || do_something(3));

let result_one = thread_one.join();
let result_two = thread_two.join();
let result_three = thread_three.join();

println!("time elapsed {:?}", now.elapsed());

println!("result {}", result_one.unwrap() +
          result_two.unwrap() + result_three.unwrap());
```

Running the preceding code gives us the following printout:

```
number 1 is running
number 3 is running
number 2 is running
time elapsed 2.002991041s
result 6
```

As we can see, the whole process took just over 2 seconds to run. This is because all three threads are running concurrently. Note also that thread three is fired before thread two. Do not worry if you get a sequence of `1`, `2`, and `3`. Threads finish in an indeterminate order. The scheduling is deterministic; however, there are thousands of events happening under the hood that require the CPU to do something. As a result, the exact time slices that each thread gets are never the same. These tiny changes add up. Because of this, we cannot guarantee that the threads will finish in a determinate order.

Looking back at how we spin off threads, we can see that we pass a closure into our thread. If we try and just pass the `do_something` function through the thread, we get an error complaining that the compiler expected an `FnOnce<()>` closure and found `i8` instead. This is because a standard closure implements the `FnOnce<()>` public trait, whereas our `do_something` function simply returns `i8`. When `FnOnce<()>` is implemented, the closure can only be called once. This means

More books: https://t.me/booksforits

that when we create a thread, we can ensure that the closure can only be called once, and then when it returns, the thread ends. As our `do_something` function is the final line of the closure, `i8` is returned. However, it has to be noted that just because the `FnOnce<()>` trait is implemented, it does not mean that we cannot call it multiple times. This trait only gets called if the context requires it. This means that if we were to call the closure outside of the thread context, we could call it multiple times.

Note also that we directly unwrap our results. From what we know, we can deduce that the `join` function on the `JoinHandle` struct returns `Result`, which we also know can be `Err` or `Ok`. We know it is going to be okay to unwrap the result directly because we are merely sleeping and then returning an integer. We also printed out the results, which were indeed integers. However, our error is not what you would expect. The full `Result` type we get is `Result<i8, Box<dyn Any + Send>>`. We already know what `Box` is; however, `dyn Any + Send` seems new. `dyn` is a keyword that we use to indicate what type of trait is being used. `Any` and `Send` are two traits that must be implemented. The `Any` trait is for dynamic typing, meaning that the data type can be anything. The `Send` trait means that it is safe to be moved from one thread to another. The `Send` trait also means that it is safe to copy from one thread to another. So, what we are sending has implemented the `Copy` trait as what we are sending can be sent between threads. Now that we understand this, we can handle the results of the threads by merely matching the `Result` outcome, and then downcasting the error into a string to get the error message with the following code:

```
match thread_result {
    Ok(result) => {
        println!("the result for {} is {}",
                  result, name);
    }
    Err(result) => {
    if let Some(string) = result.downcast_ref::<String>() {
        println!("the error for {} is: {}", name, string);
    } else {
        println!("there error for {} does not have a
                  message", name);
        }
    }
}
```

The preceding code enables us to manage the results of threads gracefully. Now, there is nothing stopping you from logging failures of threads or spinning up new threads based on the outcomes of previous threads. Thus, we can see how powerful the `Result` struct is. There is more we can do with threads, such as give them names or pass data between them with channels. However, the focus of this book is web programming, not advanced concurrency design patterns and concepts. However, further reading on the subject is provided at the end of the chapter.

More books: https://t.me/booksforits

We now understand how to spin up threads in Rust, what they return, and how to handle them. With this information, we can move on to the next section about understanding the `async` and `await` syntax, as this is what will be used in our Actix Web server.

Understanding async and await

The `async` and `await` syntax manages the same concepts covered in the previous section; however, there are some nuances. Instead of simply spawning off threads, we create **futures** and then manipulate them as and when needed.

In computer science, a future is an unprocessed computation. This is where the result is not yet available, but when we call or wait, the future will be populated with the result of the computation. Another way of describing this is that a future is a way of expressing a value that is not yet ready. As a result, a future is not exactly a thread. In fact, threads can use futures to maximize their potential. For instance, let us say that we have several network connections. We could have an individual thread for each network connection. This is better than sequentially processing all connections, as a slow network connection would prevent other faster connections from being processed down the line until it itself is processed, resulting in a slower processing time overall. However, spinning up threads for every network connection is not free. Instead, we can have a future for each network connection. These network connections can be processed by a thread from a thread pool when the future is ready. Therefore, we can see why futures are used in web programming, as there are a lot of concurrent connections.

Futures can also be referred to as *promises*, *delays*, or *deferred*. To explore futures, we will create a new Cargo project and utilize the futures created in the `Cargo.toml` file:

```
[dependencies]
futures = "0.3.21"
```

With the preceding crate installed, we can import what we need in our `main.rs` using the following code:

```
use futures::executor::block_on;
use std::{thread, time};
```

We can define futures by merely using the `async` syntax. The `block_on` function will block the program until the future we defined has been executed. We can now define the `do_something` function with the following code:

```
async fn do_something(number: i8) -> i8 {
    println!("number {} is running", number);
    let two_seconds = time::Duration::new(2, 0);
    thread::sleep(two_seconds);
```

More books: https://t.me/booksforits

```
    return 2
}
```

The `do_something` function essentially does what the code says it does, which is print out what number it is, sleep for 2 seconds, and then return an integer. However, if we were to directly call it, we would not get `i8`. Instead, calling the `do_something` function directly will give us `Future<Output = i8>`. We can run our future and time it in the main function with the following code:

```
fn main() {
    let now = time::Instant::now();
    let future_one = do_something(1);
    let outcome = block_on(future_one);
    println!("time elapsed {:?}", now.elapsed());
    println!("Here is the outcome: {}", outcome);
}
```

Running the preceding code will give us the following printout:

```
number 1 is running
time elapsed 2.00018789s
Here is the outcome: 2
```

This is what is expected. However, let's see what happens if we enter an extra `sleep` function before we call the `block_on` function with the following code:

```
fn main() {
    let now = time::Instant::now();
    let future_one = do_something(1);
    let two_seconds = time::Duration::new(2, 0);
    thread::sleep(two_seconds);
    let outcome = block_on(future_one);
    println!("time elapsed {:?}", now.elapsed());
    println!("Here is the outcome: {}", outcome);
}
```

We will get the following printout:

```
number 1 is running
time elapsed 4.000269667s
Here is the outcome: 2
```

More books: https://t.me/booksforits

Thus, we can see that our future does not execute until we apply an executor using the `block_on` function.

This can be a bit laborious, as we may just want a future that we can execute later in the same function. We can do this with the `async/await` syntax. For instance, we can call the `do_something` function and block the code until it is finished using the `await` syntax inside the `main` function, with the following code:

```
let future_two = async {
    return do_something(2).await
};
let future_two = block_on(future_two);
println!("Here is the outcome: {:?}", future_two);
```

What the `async` block does is return a future. Inside this block, we call the `do_something` function blocking the `async` block until the `do_something` function is resolved, by using the `await` expression. We then apply the `block_on` function on the `future_two` future.

Looking at our preceding code block, this might seem a little excessive, as it can be done with just two lines of code that call the `do_something` function and pass it to the `block_on` function. In this case, it is excessive, but it can give us more flexibility on how we call futures. For instance, we can call the `do_something` function twice and add them together as a return with the following code:

```
let future_three = async {
    let outcome_one = do_something(2).await;
    let outcome_two = do_something(3).await;
    return outcome_one + outcome_two
};
let future_outcome = block_on(future_three);
println!("Here is the outcome: {:?}", future_outcome);
```

Adding the preceding code to our `main` function will give us the following printout:

```
number 2 is running
number 3 is running
Here is the outcome: 4
```

More books: https://t.me/booksforits

Whilst the preceding output is the result that we are expecting, we know that these futures will run sequentially and that the total time for this block of code will be just above 4 seconds. Maybe we can speed this up by using `join`. We have seen `join` speed up threads by running them at the same time. It does make sense that it will also work to speed up our futures. First, we must import the `join` macro with the following code:

```
use futures::join
```

We can now utilize `join` for our futures and time the implementation with the following code:

```
let future_four = async {
    let outcome_one = do_something(2);
    let outcome_two = do_something(3);
    let results = join!(outcome_one, outcome_two);
    return results.0 + results.1
};

let now = time::Instant::now();
let result = block_on(future_four);
println!("time elapsed {:?}", now.elapsed());
println!("here is the result: {:?}", result);
```

In the preceding code, we can see that the `join` macro returns a tuple of the results and that we unpack the tuple to give us the same result. However, if we do run the code, we can see that although we get the result that we want, our future execution does not speed up and is still stuck at just above 4 seconds. This is because a future is not being run using an `async` task. We will have to use `async` tasks to speed up the execution of our futures. We can achieve this by carrying out the following steps:

1. Create the futures needed.
2. Put them into a vector.
3. Loop through the vector, spinning off tasks for each future in the vector.
4. Join the `async` tasks and sum the vector.

This can be visually mapped out with the following figure:

More books: https://t.me/booksforits

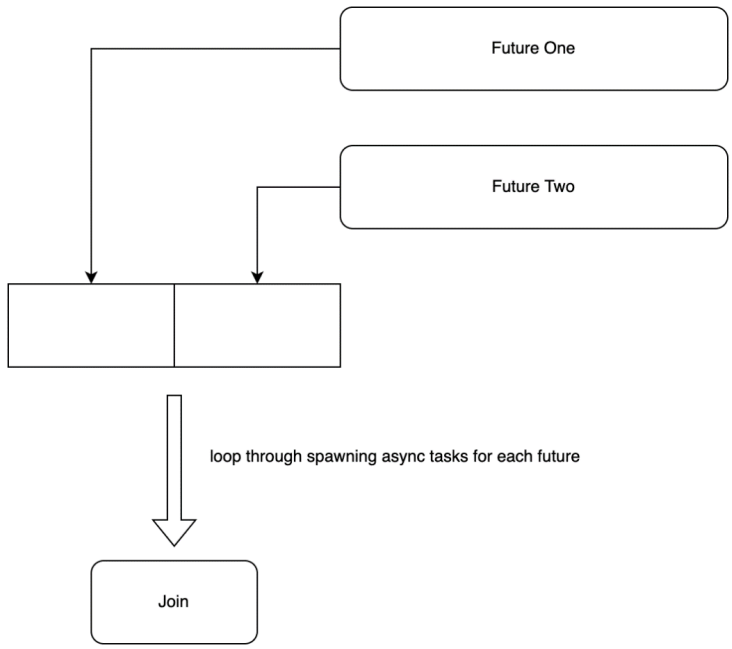


Figure 3.2 – The steps to running multiple futures at once

To join all our futures at the same time, we will have to use another crate to create our own asynchronous `join` function by using the `async_std` crate. We define this crate in the `Cargo.toml` file with the following code:

```
async-std = "1.11.0"
```

Now that we have the `async_std` crate, we can import what we need to carry out the approach laid out in *Figure 3.2*, by importing what we need at the top of the `main.rs` file with the following code:

```
use std::vec::Vec;
use async_std;
use futures::future::join_all;
```

In the `main` function, we can now define our future with the following code:

```
let async_outcome = async {
    // 1.
    let mut futures_vec = Vec::new();
    let future_four = do_something(4);
    let future_five = do_something(5);
```

More books: https://t.me/booksforits

```
    // 2.
    futures_vec.push(future_four);
    futures_vec.push(future_five);

    // 3.
    let handles = futures_vec.into_iter().map(
    async_std::task::spawn).collect::<Vec<_>>();

    // 4.
    let results = join_all(handles).await;
    return results.into_iter().sum::<i8>();
};
```

Here, we can see that we define our futures (*1*), and then we add them to our vector (*2*). We then loop through our futures in our vector using the `into_iter` function. We then spawn a thread on each future using `async_std::task::spawn`. This is similar to `std::task::spawn`. So, why bother with all this extra headache? We could just loop through the vector and spawn a thread for each task. The difference here is that the `async_std::task::spawn` function is spinning off an `async` task in the same thread. Therefore, we are concurrently running both futures in the same thread! We then join all the handles, `await` for these tasks to finish, and then return the sum of all these threads. Now that we have defined our `async_outcome` future, we can run and time it with the following code:

```
let now = time::Instant::now();
let result = block_on(async_outcome);
println!("time elapsed for join vec {:?}", now.elapsed());
println!("Here is the result: {:?}", result);
```

Running our additional code will give the following additional printout:

```
number 4 is running
number 5 is running
time elapsed for join vec 2.007713458s
Here is the result: 4
```

It's working! We have managed to get two `async` tasks running at the same time in the same thread, resulting in both futures being executed in just over 2 seconds!

More books: https://t.me/booksforits

As we can see, spawning threads and `async` tasks in Rust is straightforward. However, we must note that passing variables into threads and `async` tasks is not. Rust's borrowing mechanism ensures memory safety. We must go through extra steps when passing data into a thread. Further discussion on the general concepts behind sharing data between threads is not conducive to our web project. However, we can briefly signpost what types allow us to share data:

- `std::sync::Arc`: This type enables threads to reference outside data:

```
use std::sync::Arc;
use std::thread;

let names = Arc::new(vec!["dave", "chloe", "simon"]);
let reference_data = Arc::clone(&names);

let new_thread = thread::spawn(move || {
    println!("{}", reference_data[1]);
});
```

- `std::sync::Mutex`: This type enables threads to mutate outside data:

```
use std::sync::Mutex;
use std::thread;

let count = Mutex::new(0);

let new_thread = thread::spawn(move || {
     count.lock().unwrap() += 1;
});
```

  Inside the thread here, we dereference the result of the lock, unwrap it, and mutate it. It must be noted that the shared state can only be accessed once the lock is held.

We have now covered enough of async programming to return to our web programming. Concurrency is a subject that can be covered in an entire book, one of which is referenced in the *Further reading* section. For now, we must get back to exploring Rust in web development to see how our knowledge of Rust async programming affects how we understand the Actix Web server.

More books: https://t.me/booksforits

Exploring async and await with web programming

Knowing what we know about async programming, we can now see the `main` function in our web application in a different light, as follows:

```
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new( || {
        App::new()
        .route("/", web::get().to(greet))
        .route("/{name}", web::get().to(greet))
        .route("/say/hello", web::get().to(||
               async { "Hello Again!" }))
    })
    .bind("127.0.0.1:8080")?
    .workers(3)
    .run()
    .await
}
```

We know that our `greet` function is an `async` function and thus a future. We can also see that the closure we pass into the `/say/hello` view also utilizes the `async` syntax. We can also see that the `HttpServer::new` function utilized the `await` syntax in `async fn main()`. Therefore, we can deduce that our `HttpServer::new` function is an executor. However, if we were to remove the `#[actix_web::main]` macro, we would get the following error:

```
`main` function is not allowed to be `async`
```

This is because our `main` function, which is our entry point, would return a future as opposed to running our program. `#[actix_web::main]` is a runtime implementation and enables everything to be run on the current thread. The `#[actix_web::main]` macro marks the `async` function (which, in this case, is the `main` function) to be executed by the Actix system.

> **Note**
>
> At the risk of getting into the weeds here, the Actix crate runs concurrent computation based on the actor model. This is where an actor is a computation. Actors can send and receive messages to and from each other. Actors can alter their own state, but they can only affect other actors through messages, which removes the need for lock-based synchronization (the mutex we covered is lock-based). Further exploration of this model will not help us to develop basic web apps. However, the Actix crate does have good documentation on coding concurrent systems with Actix at `https://actix.rs/book/actix`.

More books: https://t.me/booksforits

We have covered a lot here. Do not feel stressed if you do not feel like you have retained all of it. We've briefly covered a range of topics around asynchronous programming. We do not need to understand it inside out to start building applications based on the Actix Web framework.

You may also feel like we have been excessive in what we have covered. For instance, we could have spun up a server and used the `async` syntax when needed to merely punch out views without really knowing what is going on. Not understanding what is going on but knowing where to put `async` would not have slowed us down when building our toy application. However, this whistle-stop tour is invaluable when it comes to debugging and designing applications. To establish this, we can look at an example in the wild. We can look at this smart *Stack Overflow* solution to running multiple servers in one file: `https://stackoverflow.com/questions/59642576/run-multiple-actix-app-on-different-ports`.

The code in the *Stack Overflow* solution involves basically running two servers at one runtime. First, they define the views with the following code:

```
use actix_web::{web, App, HttpServer, Responder};
use futures::future;

async fn utils_one() -> impl Responder {
    "Utils one reached\n"
}

async fn health() -> impl Responder {
    "All good\n"
}
```

Once the views are defined, the two servers are defined in the `main` function:

```
#[actix_rt::main]
async fn main() -> std::io::Result<()> {
    let s1 = HttpServer::new(move || {
            App::new().service(web::scope("/utils").route(
            "/one", web::get().to(utils_one)))
        })
        .bind("0.0.0.0:3006")?
        .run();
    let s2 = HttpServer::new(move || {
            App::new().service(web::resource(
```

More books: https://t.me/booksforits

```
            "/health").route(web::get().to(health)))
        })
        .bind("0.0.0.0:8080")?
        .run();
    future::try_join(s1, s2).await?;

    Ok(())
}
```

I have not added any notation to this code, but it should not intimidate you. We can confidently deduce that `s1` and `s2` are futures that the `run` function returns. We then join these two futures together and `await` for them to finish. There is also a slight difference between our code and the code in the *Stack Overflow* solution. Our solution utilizes `await?` and then returns `Ok` with the following code snippet:

```
    future::try_join(s1, s2).await?;
    Ok(())
}
```

This is because a `?` operator is essentially a `try` match. `join(s1, s2).await?` expands roughly to the following code:

```
match join(s1, s2).await {
    Ok(v) => v,
    Err(e) => return Err(e.into()),
}
```

Whereas `join(s1, s2).await.unwrap()` expands roughly to the following code:

```
match join(s1, s2).await {
    Ok(v) => v,
    Err(e) => panic!("unwrap resulted in {}", e),
}
```

Because of the `?` operator, the person providing the solution has to insert `Ok` at the end because the `main` function returns `Result`, and this was taken away by implementing the `?` operator.

Thus, in the wild solution, *Stack Overflow* has demonstrated the importance of covering async programming. We can look at code in the wild and work out what is going on and how the posters on *Stack Overflow* managed to achieve what they did. This can also mean that we can get creative ourselves. There is nothing stopping us from creating three servers and running them in the `main`

More books: https://t.me/booksforits

function. This is where Rust really shines. Taking the time to learn Rust gives us the ability to safely dive into low-level territory and have more fine-grain control over what we do. You will find this is true in any field of programming done with Rust.

There is one more concept that we should investigate before trying to build our application, and this is **runtimes**. We know that we must have the Actix Web macro to enable the `main` function to be a future. If we look at the **Tokio** crate, we can see that it is an asynchronous runtime for the Rust programming language by providing the building blocks needed to write network applications. The workings of Tokio are complex; however, if we look at the Tokio documentation on speeding up the runtime, we can add diagrams like the following one:

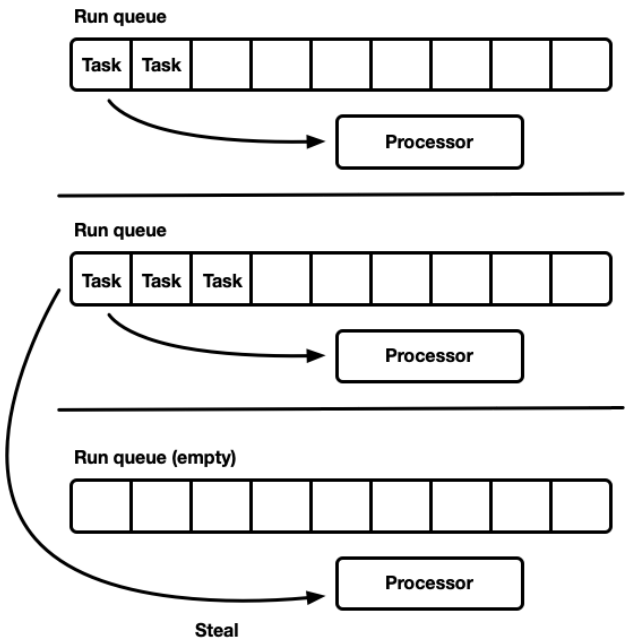


Figure 3.3 – Speeding up the Tokio runtime [source: Tokio Documentation (2019) (`https://tokio.rs/blog/2019-10-scheduler`)]

In the preceding figure, we can see that there are tasks queued up and processors processing them. We processed our tasks earlier, so this should look familiar. Considering this, it might not be too shocking to know that we can use Tokio instead of the Actix Web macro to run our server. To do this, we define our Tokio dependency in the `Cargo.toml` file with the following code:

```
tokio = { version = "1.17.0", features = ["full"] }
```

More books: https://t.me/booksforits

With the preceding code, we can now switch our macro in the `main.rs` file with the following code:

```
#[tokio::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new( || {
        App::new()
        .route("/", web::get().to(greet))
        .route("/{name}", web::get().to(greet))
    })
    .bind("127.0.0.1:8080")?
    .bind("127.0.0.1:8081")?
    .workers(3)
    .run()
    .await
}
```

Running the preceding code will give us the same outcome as running a server. There might be some inconsistencies when using Tokio instead of our Actix runtime macro. While this is an interesting result that demonstrates how we can confidently configure our server, we will use the Actix runtime macro for the rest of the book when it comes to developing the to-do application in Actix. We will revisit Tokio in *Chapter 14*, *Exploring the Tokio Framework*.

We have now covered enough of server configuration and how the server processes requests to be productive. We can now move on to defining our views and how they are handled in the next section.

Managing views using the Actix Web framework

So far, we have defined all our views in the `main.rs` file. This is fine for small projects; however, as our project grows, this will not scale well. Finding the right views can be hard, and updating them can lead to mistakes. It also makes it harder to remove modules from or insert them into your web application. Also, if we have all the views being defined on one page, this can lead to a lot of merge conflicts if a bigger team is working on the application, as they will all want to alter the same file if they are altering the definitions of views. Because of this, it is better to keep the logic of a set of views contained in a module. We can explore this by building a module that handles authentication. We will not be building the logic around authentication in this chapter, but it is a nice straightforward example to use when exploring how to manage the structure of a views module. Before we write any code, our web application should have the following file layout:

```
├── main.rs
└── views
    ├── auth
```

More books: https://t.me/booksforits

```
    |   ├── login.rs
    |   ├── logout.rs
    |   └── mod.rs
    ├── mod.rs
```

The code inside each file can be described as follows:

- `main.rs`: The entry point for the server where the server is defined
- `views/auth/login.rs`: The code defining the view for logging in
- `views/auth/logout.rs`: The code defining the view for logging out
- `views/auth/mod.rs`: The factory that defines the views for `auth`
- `views/mod.rs`: The factory that defines all the views for the whole app

First, let us start off our entry point with a basic web server with no extras in the `main.rs` file, with the following code:

```
use actix_web::{App, HttpServer};
mod views;

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        let app = App::new();
        return app
    })
        .bind("127.0.0.1:8000")?
        .run()
        .await
}
```

This preceding code is straightforward and there should be no surprises. We will alter the code later, and we can move on to defining the views. For this chapter, we just want to return a string saying what the view is. We will know that our application structure works. We can define our basic login view in the `views/auth/login.rs` file with the following code:

```
pub async fn login() -> String {
    format!("Login view")
}
```

More books: https://t.me/booksforits

Now, it will not be surprising that the logout view in the `views/auth/logout.rs` file takes the following form:

```
pub async fn logout() -> String {
    format!("Logout view")
}
```

Now that our views have been defined, all we need to do is define the factories in the `mod.rs` files to enable our server to serve them. Our factories give the data flow of our app, taking the following form:

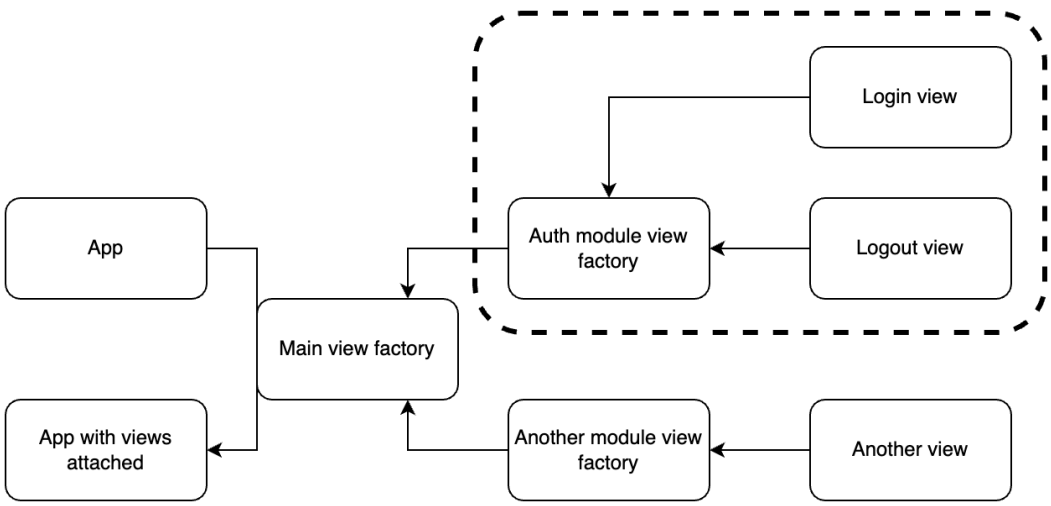


Figure 3.4 – The data flow of our application

We can see in *Figure 3.4* that chaining factories gives us a lot of flexibility. If we wanted to remove all the `auth` views from our application, we would be able to do this by simply removing one line of code in our main view factory. We can also reuse our modules. For instance, if we were to use the `auth` module on multiple servers, we could merely have a git submodule for the `auth` views module and use it on other servers. We can build our `auth` module factory view in the `views/auth/mod.rs` file with the following code:

```
mod login;
mod logout;

use actix_web::web::{ServiceConfig, get, scope};

pub fn auth_views_factory(app: &mut ServiceConfig) {
    app.service(scope("v1/auth").route("login",
```

More books: https://t.me/booksforits

```
                get().to(login::login)).route("logout",
                get().to(logout::logout))
    );
}
```

In the preceding code, we can see that we pass in a mutable reference of a `ServiceConfig` struct. This enables us to define things such as views on the server in different fields. The documentation on this struct states that it is to allow bigger applications to split up a configuration into different files. We then apply a service to the `ServiceConfig` struct. The service enables us to define a block of views that all get populated with the prefix defined in the scope. We also state that we are using `get` methods, for now, to make it easily accessible in the browser. We can now plug the `auth` views factory into the `main` views factory in the `views/mod.rs` file with the following code:

```
mod auth;
use auth::auth_views_factory;

use actix_web::web::ServiceConfig;

pub fn views_factory(app: &mut ServiceConfig) {
    auth_views_factory(app);
}
```

In the preceding code, we have been able to chop our entire views modules with just one line of code. We can also chain the modules as much as we like. For instance, if we wanted to have submodules within the `auth` views module, we could, and we merely feed the factories of those `auth` submodules into the `auth` factory. We can also define multiple services in a factory. Our `main.rs` file remains pretty much the same with the addition of a `configure` function, as seen with the following code:

```
use actix_web::{App, HttpServer};
mod views;

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        let app =
            App::new().configure(views::views_factory);
        return app
    })
        .bind("127.0.0.1:8000")?
```

More books: https://t.me/booksforits

```
            .run()
            .await
}
```

When we call the `configure` function on the `App` struct, we pass the views factory into the `configure` function, which will pass the `config` struct into our factory function for us. As the `configure` function returns `Self`, meaning the `App` struct, we can return the result at the end of the closure. We can now run our server, resulting in the following outcome:

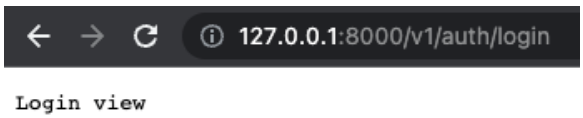


Figure 3.5 – The login view

We can see that our application with the expected prefix works! With this, we have covered all the basics to handle HTTP requests with confidence.

Summary

In this chapter, we covered the basics of threading, futures, and `async` functions. As a result, we were able to look at a multi-server solution in the wild and understand confidently what was going on. With this, we built on the concepts we learned in the previous chapter to build modules that define views. In addition, we chained factories to enable our views to be constructed on the fly and added to a server. With this chained factory mechanism, we can slot entire view modules in and out of a configuration when the server is being built.

We also built a utility struct that defines a path, standardizing the definition of a URL for a set of views. In future chapters, we will use this approach to build authentication, JSON serialization, and frontend modules. With what we've covered, we'll be able to build views that extract and return data from the user in a range of different ways in the next chapter. With this modular understanding, we have a strong foundation that enables us to build real-world web projects in Rust where logic is isolated and can be configured, and where code can be added in a manageable way.

In the next chapter, we will work on processing requests and responses. We will learn how to pass params, bodies, headers, and forms to views and process them by returning JSON. We will be using these new methods with the to-do module we built in the previous chapter to enable our interaction with to-do items through server views.

More books: https://t.me/booksforits

Questions

1. What parameter is passed into the `HttpServer::new` function and what does the parameter return?
2. How is a closure different from a function?
3. What is the difference between a process and a thread?
4. What is the difference between an `async` function and a normal one?
5. What is the difference between `await` and `join`?
6. What is the advantage of chaining factories?

Answers

1. A closure is passed into the `HttpServer::new` function. The `HttpServer::new` function has to return the `App` struct so that the `bind` and `run` functions can be acted on them after the `HttpServer::new` function has fired.
2. A closure can interact with variables outside of its scope.
3. A process is a program that is being executed with its own memory stack, registers, and variables, whereas a thread is a lightweight process that is managed independently but shares data with other threads and the main program.
4. A normal function executes as soon as it is called, whereas an `async` function is a promise and must be executed with a blocking function.
5. `await` blocks a program to wait for a future to be executed; however, the `join` function can run multiple threads or futures concurrently. `await` can also be executed on a `join` function.
6. Chaining factories gives us flexibility on how individual modules are constructed and orchestrated. A factory inside a module focuses on how the module is constructed, and the factory outside the module focuses on how the different modules are orchestrated.

Further reading

- *Hands-On Concurrency with Rust* (2018) by *Brian Troutwine*, *Packt Publishing*

More books: https://t.me/booksforits

More books: https://t.me/booksforits

4

Processing HTTP Requests

Up to this point, we have utilized the Actix Web framework to serve basic views. However, this can only get us so far when it comes to extracting data from the request and passing data back to the user. In this chapter, we will fuse code from *Chapter 2, Designing Your Web Application in Rust*, and *Chapter 3*, *Handling HTTP Requests*, to build server views that process to-do items. We will then explore **JSON serialization** for extracting data and returning it to make our views more user friendly. We will also extract data from the header with middleware before it hits the view. We will explore the concepts around data serialization and extracting data from requests by building out the create, edit, and delete to-do item endpoints for our to-do application.

In this chapter, we will cover the following topics:

- Getting to know the initial setup for fusing code
- Passing parameters into views
- Using macros for JSON serialization
- Extracting data from views

Once you have finished this chapter, you will be able to build a basic Rust server that can send and receive data in the URL, in the body using JSON, and in the header of the HTTP request. This is essentially a fully functioning API Rust server without a proper database for data storage, the ability to authenticate users, or the ability to display content in the browser. However, these concepts will be covered in the next three chapters. You are on the home run for having a fully working Rust server that is up and running. Let's get started!

Technical requirements

For this chapter, we need to download and install **Postman**. We will need Postman to make API requests to our server. You can download it from `https://www.postman.com/downloads/`.

More books: https://t.me/booksforits

We will also be building on the server code we created in the previous chapter, which can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter03/managing_views_using_the_actix_web_framework/web_app`.

You can find the full source code that will be used in this chapter here: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter04`.

Managing views code will be the basis of this chapter, and we will add features to this code base. We will be fusing this with the to-do module that we wrote in *Chapter 2*, *Designing Your Web Application in Rust*, which can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter02/processing_traits_and_structs`.

Getting to know the initial setup for fusing code

In this section, we will cover the initial setup of two fusing pieces of code we built in *Chapter 2*, *Designing Your Web Application in Rust*, and *Chapter 3*, *Handling HTTP Requests*. This fusion will give us the following structure:

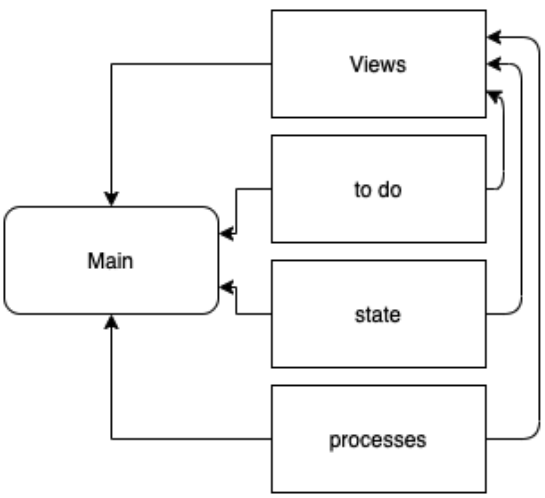


Figure 4.1 – Structure of our app and its modules

Here, we will register all the modules in the main file and then pull all these modules into the views to be used. We are essentially swapping the command-line interface from *Chapter 2*, *Designing Your Web Application in Rust*, with web views. Combining these modules gives us the following files in the code base:

```
├── main.rs
```

More books: https://t.me/booksforits

```
├── processes.rs
├── state.rs
```

We are then bolting our `to_do` module into the same directory of our `main.rs` file. If you built the `to_do` module when reading *Chapter 2*, *Designing Your Web Application in Rust*, your `to_do` module should have the following structure:

```
├── to_do
│   ├── enums.rs
│   ├── mod.rs
│   ├── structs
│   │   ├── base.rs
│   │   ├── done.rs
│   │   ├── mod.rs
│   │   └── pending.rs
│   └── traits
│       ├── create.rs
│       ├── delete.rs
│       ├── edit.rs
│       ├── get.rs
│       └── mod.rs
```

So now, our bolt on the `views` module from the previous chapter should contain the following:

```
└── views
    ├── auth
    │   ├── login.rs
    │   ├── logout.rs
    │   └── mod.rs
    ├── mod.rs
    ├── path.rs
```

The full structure, with all the code, can be found in the following GitHub repo:

`https://github/PacktPublishing/Rust-for-Web-Developers/tree/master/chapter04/getting_to_know_the_initial_setup`

Now that we have added modules from previous chapters to our project, we can bind them together in our program. To do this, we must create a new `src/main.rs` file. First, we must import the modules that we built and define a basic server using the following code:

More books: https://t.me/booksforits

```
use actix_web::{App, HttpServer};
mod views;
mod to_do;
mod state;
mod processes;

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        let app = App::new().configure(views::views_factory);
        return app
    })
        .bind("127.0.0.1:8000")?
        .run()
        .await
}
```

In the preceding code, we defined the modules and then our server. Because the server is utilizing `views_factory`, we will not have to alter this file for the rest of this chapter. Instead, we will chain our factory functions that will be called in the `views_factory` function.

At this point, we can sit back and appreciate the dividends of all the hard work that we did in the previous chapters. The isolation of principles and well-defined modules have enabled us to slot our logic from our command-line program into our server interface with minimal effort. Now, all we have to do is connect it to our `views` module, and pass parameters into those views. Before we move on to the next section, however, there is some minor housekeeping we must do to ensure that our server can run. First, our dependencies in the `Cargo.toml` file have the following requirements:

```
[dependencies]
actix-web = "4.0.1"
serde_json = "1.0.59"
```

We can then do a `cargo run` command, showing that our login and logout views work in our browser. With this done, we can start working on passing parameters into views.

Passing parameters into views

In this section, we will cover the initial setup of fusing two modules to create a to-do item and store it through a view. To do this, we will have to pass in the title of the to-do item into our `create` view that creates a to-do item. We can pass data into a view using the following routes:

More books: https://t.me/booksforits

- **URL**: Data and parameters can be passed inside the URL of the request. This is generally used in simple cases as it is easy to implement.
- **body**: Data can be stored under different fields in the request body. This is used for more complex data structures and bigger payloads.
- **header**: Data can be stored under different fields in the request header. This is used to store metadata about the request being sent. We also store the authentication data of the request in the header.

We will cover all these approaches throughout our project, but for now, we will pass our data using the URL method as this is the easiest introduction method. First, we will create the structure for our to-do views with the following layout:

```
└── views
    ├── auth
    │   ├── login.rs
    │   ├── logout.rs
    │   └── mod.rs
    ├── mod.rs
    └── to_do
        ├── create.rs
        └── mod.rs
```

We can see that we have put our `to-do` views in their own `views` module next to the `to_do` module. We will keep stacking our views in this way so that we can slot them in and out of our server and slot them into other projects if needed. For now, creating a to-do item will take the following form:

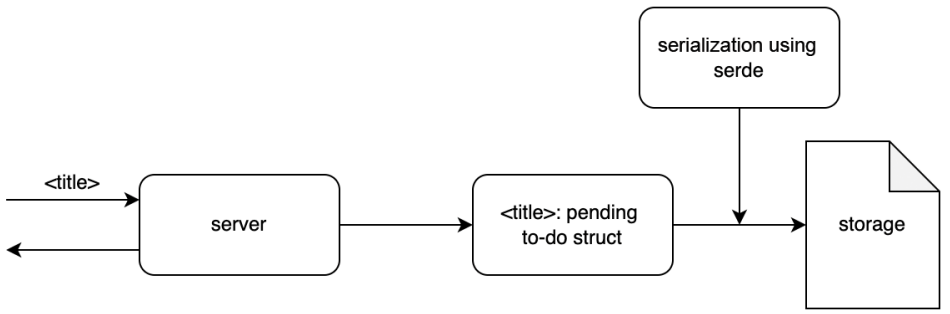


Figure 4.2 – Process of creating a to-do item

To carry out the process demonstrated in *Figure 4.2*, we need to perform the following steps:

1. Load the current state of the to-do item list.

More books: https://t.me/booksforits

2. Get the title of the new to-do item from the URL.
3. Pass the title and the status `pending` through `to_do_factory`.
4. Pass the result of the previous step along with the string `create` and the state into the process module interface.
5. Return a string to the user to signal that the process has finished.

We can carry out these steps defined previously in the `views/to_do/create.rs` file. First of all, we must import what we need with the following code:

```
use serde_json::value::Value;
use serde_json::Map;
use actix_web::HttpRequest;

use crate::to_do::{to_do_factory, enums::TaskStatus};
use crate::state::read_file;
use crate::processes::process_input;
```

We will use `serde_json::value::Value` and `serde_json::Map` to define what type of data we are reading from the `state.json` file and to extract the title from the URL using the `HttpRequest` struct. We will then import what we need from our other modules to enable us to create an item, read the state file, and process the input. Our view can be defined through the following code:

```
pub async fn create(req: HttpRequest) -> String {
    let state: Map<String, Value> = read_file(
        "./state.json"); // step 1

    let title: String = req.match_info().get("title"
    ).unwrap().to_string(); // step 2

    let item = to_do_factory(&title.as_str(),
                             TaskStatus::PENDING); // step 3
    process_input(item, "create".to_string(), &state);
    // step 4
    return format!("{} created", title) // step 5
}
```

We need to remember that this is an `async` function because it is a view that our server is processing. We can also see that our `title` is extracted from `HttpRequest` by using the `match_info` function. We must directly unwrap it because if there is no title in the URL, then we do not want

More books: https://t.me/booksforits

to continue with the process of making an item and then converting the extracted title to `String`. We then need to pass a reference of this to our `to_do_factory` to create an `ItemTypes enum`. We then pass our `enum` with a command and a reference to the current state of our application into our `process_input` function, which, as we remember, will go through a series of steps to work out how to handle the state based on the command and item type passed in. There is a lot going on here, but it must be noted none of the logic around how to process the item is in this view. This is called the separation of concerns of code orthogonality. Code orthogonality refers to the following mathematical concept:

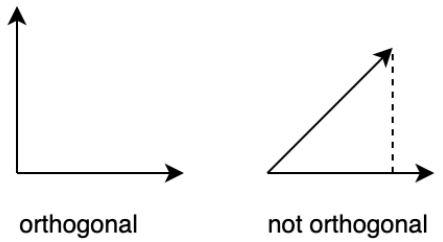


Figure 4.3 – Mathematical concept of orthogonality

We can see in *Figure 4.3* that if a vector is orthogonal to another vector then it is said to have no projection onto another vector. In physics, if the vectors are forces, these vectors do not have any effect on each other. Now, this cannot be completely true in programming: if we delete the code in our `processes` module, it will affect the `create` view as we must reference it. However, the logic of `processes` should not be defined in the `create` view. This is partly because we must use `processes` elsewhere, but this is not the entire story. When we look at the `create` view, we can see the logic around creating a pending item in relation to the rest of the application. This makes it easy for developers to know exactly what's going on. They are not getting lost in details that are not relevant to the five steps we specified earlier in this section to create a to-do item. If the developer wants to explore the logic around the saving of the item, they can investigate the file that defines this.

We must now make the views in our `to_do` module available to the outside. We can do this by creating a to-do views factory function in the `views/to_do/mod.rs` file with the following code:

```
mod create;

use actix_web::web::{ServiceConfig, get, scope};


pub fn to_do_views_factory(app: &mut ServiceConfig) {
    app.service(
```

More books: https://t.me/booksforits

```
        scope("v1/item")
        .route("create/{title}", get().to(create::create))
    );
}
```

In the preceding code, we can see that we do not make the `create` view public but we do use it in our factory to define the view. We also define the title being passed into the URL with the `/{title}` tag. Now that our item views are functional, we need to plug `to_do_views_factory` into our application in the `views/mod.rs` file with the following code:

```
mod auth;
mod to_do; // define the module

use auth::auth_views_factory;
use to_do::to_do_views_factory; // import the factory

use actix_web::web::ServiceConfig;


pub fn views_factory(app: &mut ServiceConfig) {
    auth_views_factory(app);
    to_do_views_factory(app); // pass the ServiceConfig
}
```

In the preceding code, we can see that we have defined the module, imported the factory, and then passed the application configuration. With this done, our application is ready to run and create to-do items. When our application is running, we can create the item with the following URL:

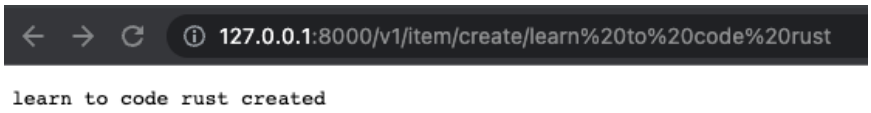


Figure 4.4 – View to create a to-do item

If we look at the console, we will see the following printout:

```
learn to code rust is being created
```

More books: https://t.me/booksforits

If we look at the `state.json` file in the root, we will get the following data:

```
{"learn to code rust":"PENDING"}
```

We can see that our process to create a to-do item worked! Our application takes in a title from the URL, creates a pending to-do item, and saves it in our JSON file. While this is a milestone, it must be noted that a JSON file is not the best solution for data storage. It will do for now, however, as we will configure a proper database in *Chapter 6*, *Data Persistence with PostgreSQL*. We can also see `%20` in the URL, which denotes a space. We can see that this space translates to the console printout and the saving of the data to the JSON file, and this space is also in the view displayed in the browser. What we have done is take in a to-do item title via the URL, print it out to the terminal, display it in the browser, and save it in a JSON file. We have essentially performed the basis for a web application as we can display data to a user and store it in a file.

The `GET` method works for us, but it is not the most appropriate method for creating a to-do item. `GET` methods can be cached, bookmarked, kept in the browser's history, and have restrictions in terms of their length. Bookmarking, storing them in browser history, or caching them doesn't just present security issues; it also increases the risk of the user accidentally making the same call again. Because of this, it is not a good idea to alter data with a `GET` request. To protect against this, we can use a `POST` request, which does not cache, does not end up in browser history, and cannot be bookmarked.

Because of the reasons we laid out, we will now turn our `create` view into a `POST` request. Remember our comments on code orthogonality. What defines how the routes of our views are processed is kept in our factory in the `views/to_do/mod.rs` file with the following code:

```
mod create;
use actix_web::web::{ServiceConfig, post, scope};

pub fn to_do_views_factory(app: &mut ServiceConfig) {
    app.service(
        scope("v1/item")
        .route("create", post().to(create::create))
    );
}
```

In the preceding code, we can see that we have merely changed the `get` to `post` in the import and the `route` definition. If we try and create a new to-do item using the previous approach, we get the following outcome:

More books: https://t.me/booksforits

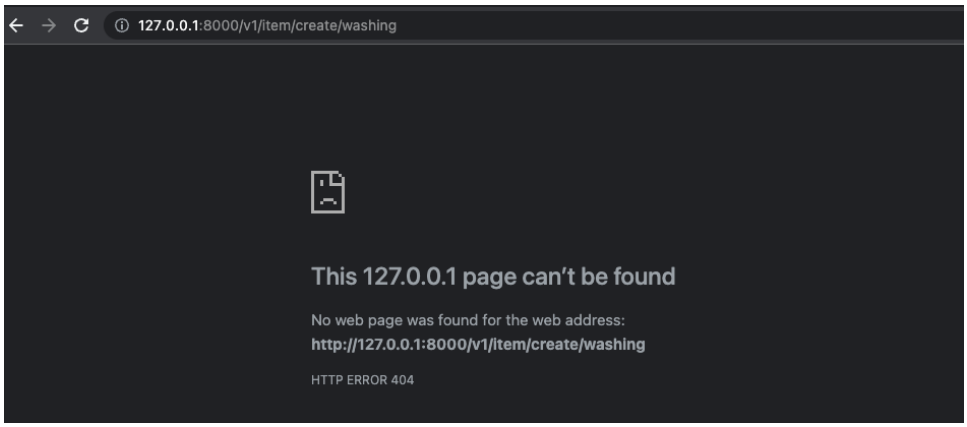


Figure 4.5 – Blocked method

In *Figure 4.5*, we can see that the page cannot be found. This might be confusing as the error is a 404-error stating that the page can't be found. The URL is still defined, but the `GET` method is no longer allowed for this URL. With this in mind, we can make a `POST` call using the following Postman configuration:

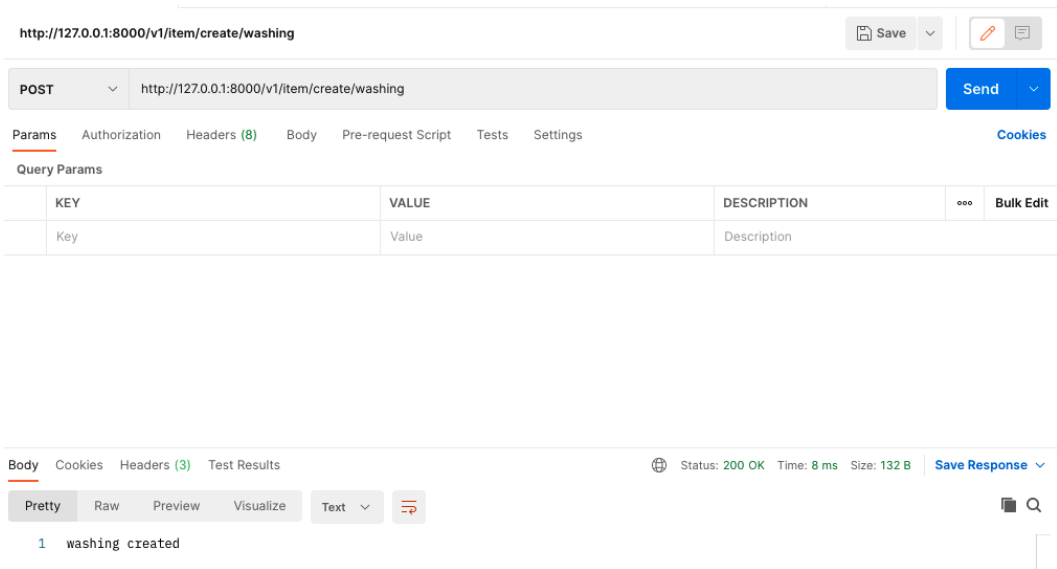


Figure 4.6 – Postman post method for creating an item

In *Figure 4.6*, we can see that our URL is still working with just a different method—the `POST` method. We can inspect our state file, finding the following data:

```
{"learn to code rust":"PENDING","washing":"PENDING"}
```

More books: https://t.me/booksforits

We can see that changing the allowed method for the `create` view did not affect the way in which we create or store to-do items. Looking back at *Figure 4.6*, we can also see that we get a status code of `200`, which is `OK`. This already tells us that the creation has happened. Because of this, we do not have to return anything as the status is `OK`.

Looking back at what we get when we try and send a `GET` request to our `create` view, we get a **not found** error. Can you think what this means? Is there anything stopping us from reusing the same URL but with a different method? Well, if the view cannot be found, then there should not be anything stopping us from using the same URL with a different method. This can be done in the `views/to_do/mod.rs` file with the following code:

```
app.service(
    scope("v1/item")
    .route("create/{title}", post().to(create::create))
    .route("create/{title}", get().to(create::create))
);
```

We can see that this works if we put our URL in our browser, which results in creating a pending to-do item. We also have the option to put a different function in our `get` route with the same URL if needed. This gives us flexibility on how to use and reuse our URLs. However, considering the differences that we covered between the `GET` and `POST` methods, it is sensible to just have a `POST` method for our `create` function.

We have now done all that we need to when it comes to creating our to-do item. However, in other views, we will have to return structured data to present the current state of our to-do items.

So far, we have passed data into our application using a URL, which is the most basic way in which we can pass data. However, we cannot pass structured data using a URL. For instance, if we wanted to send a hashmap or a list, a URL is just not able to house such structures. This is where we need to pass data to our application in the body of the request using JSON, which we will cover in the next section.

Using macros for JSON serialization

When it comes to serializing data and returning it to the client, we can achieve this quickly with minimal code using the JSON from the `Actix-web` crate. We can demonstrate this by creating a `GET` view that returns all our to-do items in the `views/to_do/get.rs` file:

```
use actix_web::{web, Responder};
use serde_json::value::Value;
use serde_json::Map;

use crate::state::read_file;
```

More books: https://t.me/booksforits

```
pub async fn get() -> impl Responder {
    let state: Map<String, Value> = read_file("./state.json");
    return web::Json(state);
}
```

Here, we can see that we are merely reading the JSON from the JSON file and then returning the values from this wrapped in the `web::Json` function. It might make sense to just return `Map<String, Value>` from the JSON file directly, as it is a `String` and `Value`. However, the type of `Map<String, Value>` does not implement the `Responder` trait. We could update the function to return the state directly with the following code:

```
pub async fn get() -> Map<String, Value>  {
    let state: Map<String, Value> = read_file("./state.json");
    return state;
}
```

However, this will not work because the `get().to()` function in the `views/to_do/mod.rs` file needs to accept a struct that has implemented the `Responder` trait. We can now plug in our `get` view in the `views/to_do/mod.rs` file with the following code:

```
mod create;
mod get; // import the get file

use actix_web::web::{ServiceConfig, post, get, scope};
// import get


pub fn to_do_views_factory(app: &mut ServiceConfig) {
    app.service(
        scope("v1/item")
        .route("create/{title}", post().to(create::create))
        .route("get", get().to(get::get)) // define view and
URL
    );
}
```

Running the URL `http://127.0.0.1:8000/item/get` gives us the following JSON data in the response body:

```
{
    "learn to code rust": "PENDING",
```

More books: https://t.me/booksforits

```
    "washing": "PENDING"
}
```

We now have some structured data that we can present to the frontend. While this essentially gets the job done, it is not too helpful. For instance, we would like to have two different lists for *pending* and *done*. We could also add timestamps telling users when the to-do item was created or edited. Simply returning the titles and the statuses of the to-do items will not enable us to scale complexity when needed.

Building our own serialization struct

To have more control over the type of data that we are going to return to the user, we are going to have to build our own serialization structs. Our serialization struct is going to present two lists, one for completed items and another for pending items. The lists will be populated with objects consisting of a title and a status. If we recall from *Chapter 2*, *Designing Your Web Application in Rust*, our `pending` and `Done` item structs are inherited via composition from a `Base` struct. Therefore, we must access the title and the status from the `Base` struct. However, our `Base` struct is not accessible to the public. We will have to make it accessible so that we can serialize the attributes for each to-do item:

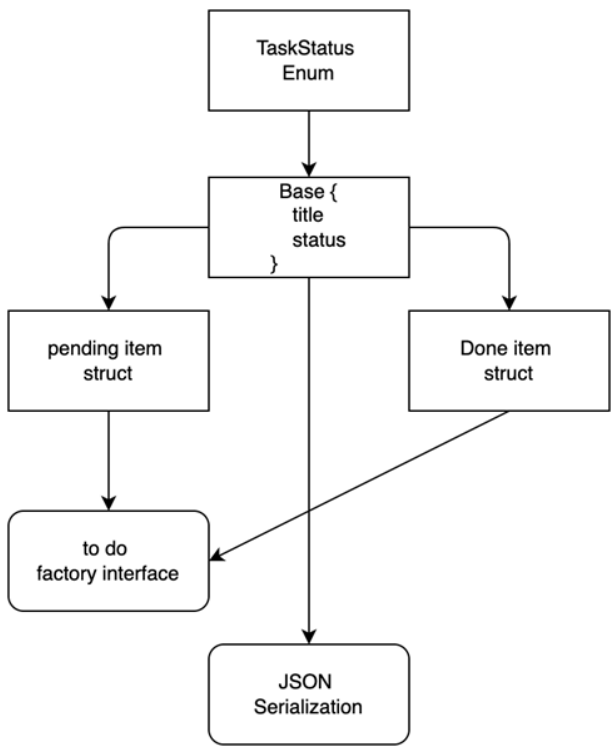


Figure 4.7 – Relationship that our to-do structs have with our interfaces

More books: https://t.me/booksforits

Looking at *Figure 4.7*, we can see that the `TaskStatus enum` is the root of the dependency. We need to be able to serialize this `enum` before we can serialize our to-do items. We can use the `serde` crate for this. In order to do this, we must update our dependencies in the `Cargo.toml` file:

```
[dependencies]
actix-web = "4.0.1"
serde_json = "1.0.59"
serde = { version = "1.0.136", features = ["derive"] }
```

We can see that we have added the `features = ["derive"]`. This will enable us to decorate our structs with `serde` traits. We can now look at how we defined our `enum` in the `src/to_do/enums.rs` file with the following code:

```
pub enum TaskStatus {
    DONE,
    PENDING
}

impl TaskStatus {

    pub fn stringify(&self) -> String {
        match &self {
            &Self::DONE => {return "DONE".to_string()},
            &Self::PENDING =>
                {return "PENDING".to_string()}
        }
    }
    pub fn from_string(input_string: String) -> Self {
        match input_string.as_str() {
            "DONE" => TaskStatus::DONE,
            "PENDING" => TaskStatus::PENDING,
            _ => panic!("input {} not supported",
                        input_string)
        }
    }
}
```

In the preceding code, we can see that we have two fields named `DONE` and `PENDING`; however, they are essentially their own types. How can we serialize this as a JSON value? There is a clue in the

More books: https://t.me/booksforits

`stringify` function. However, this is not the full picture. Remember, the return values of our server views need to implement traits. We can implement a `serde` trait for our `TaskStatus enum` by initially importing the traits that we need in the `src/to_do/enums.rs` file with the following code:

```
use serde::ser::{Serialize, Serializer, SerializeStruct};
```

We now have everything we need to implement the `Serialize` trait so we can customize how the structs that we write can be serialized in the following section.

Implementing the Serialize trait

`Serialize` is the trait that we will implement, and `Serializer` is a data formatter that can serialize any data format that is supported by `serde`. We can then implement a `Serialize` trait for our `TaskStatus enum` with the following code:

```
impl Serialize for TaskStatus {

    fn serialize<S>(&self, serializer: S) -> Result<S::Ok,
                    S::Error>
    where
        S: Serializer,
    {
        Ok(serializer.serialize_str(&self.stringify()
                                    .as_str())?)
    }
}
```

This is a standard approach that is defined in the `serde` documents. In the preceding code, we can see a `serialize` function has been defined. The `serialize` function gets called when serializing our `TaskStatus enum`. We also notice that the type of notation for `serializer` is `S`. We then use a `where` statement defining `S` as `Serializer`. This may seem counterintuitive, so we can take a step back from our application to explore it. The following code blocks are not needed to complete our application.

Let us define some basic structs as follows:

```
#[derive(Debug)]
struct TwoDposition {
    x: i32,
    y: i32
}
```

More books: https://t.me/booksforits

```
#[derive(Debug)]
struct ThreeDposition {
    x: i32,
    y: i32,
    z: i32
}
```

In the preceding code, we can see that we implement the `Debug` trait for both the `TwoDposition` and `ThreeDposition` structs. We can then define functions that print a debug statement for each `struct` with the following code:

```
fn print_two(s: &TwoDposition) {
    println!("{:?}", s);
}
fn print_three(s: &ThreeDposition) {
    println!("{:?}", s);
}
```

However, we can see that this does not scale well. We would be writing a function for everything that implements it. Instead, we can use a `where` statement so we can pass both of our structs into it as they implement the `Debug` trait. First, we must import the trait with the following code:

```
use core::fmt::Debug;
```

We can then define our flexible function with the following code:

```
fn print_debug<S>(s: &S)
where
    S: Debug {
    println!("{:?}", s);
}
```

What is happening here is that our function is generic in terms of the type of variable that we are passing into the function. We then take a reference to a value of the type `S`. This means that `S` can be any type if it implements the `Debug` trait. If we try and pass in a `struct` that does not implement the `Debug` trait, the compiler will refuse to compile. So, what is happening when we compile? Run the following code:

```
fn main() {
    let two = TwoDposition{x: 1, y: 2};
```

More books: https://t.me/booksforits

```
    let three = ThreeDposition{x: 1, y: 2, z: 3};
    print_debug(&two);
    print_debug(&three);
}
```

We will get the following printout:

```
TwoDposition { x: 1, y: 2 }
ThreeDposition { x: 1, y: 2, z: 3 }
```

The preceding output makes sense as this is the result of printing when invoking the debug trait. However, they are two different functions that are created when the compiler is compiling. Our compiler compiled the following two functions:

```
print_debug::<TwoDposition>(&two);
print_debug::<ThreeDposition>(&three);
```

This is not breaking what we know in terms of how Rust works; however, it does make our code more scalable. There are more advantages to using a `where` statement; for instance, we could specify what traits we need in an iterator with the following code:

```
fn debug_iter<I>(iter: I)
where
    I: Iterator
    I::Item: Debug
{
    for item in iter {
        println!("{:?}", iter);
    }
}
```

In the preceding code, we can see that we are accepting an iterator and that the items in the iterator need to implement the `Debug` trait. However, if we keep exploring the implementation of traits, we can lose focus of our main goal in this book: web programming in Rust.

With the knowledge of using the `where` statement to implement traits, we can look back at our implementation of the `Serialize` trait in the `TaskStatus` `enum`:

```
impl Serialize for TaskStatus {
    fn serialize<S>(&self, serializer: S) -> Result<S::Ok,
                                                    S::Error>
```

More books: https://t.me/booksforits

```
    where
        S: Serializer,
    {
        Ok(serializer.serialize_str(&self.stringify()
                                    .as_str())?)
    }
}
```

We can see that we merely call the `stringify` function and wrap it in an `Ok` result. We only want the status as a `String` as we are slotting it into a bigger body of data. If it was a struct with fields, then we could write the `serialize` function as follows:

```
fn serialize<S>(&self, serializer: S) -> Result<S::Ok,
S::Error>
where
    S: Serializer,
{
    let mut s = serializer.serialize_struct("TaskStatus",
                                            1)?;
    s.serialize_field("status", &self.stringify())?;
    s.end()
}
```

In the preceding code, our serializer is the struct called `"TaskStatus"` with the number of fields being one. We then attributed the result of the `stringify` function to the `status` field. Doing this essentially gives us the following structure:

```
#[derive(Serialize)]
struct TaskStatus {
    status: String
}
```

However, we will not utilize the `serialize_struct` approach for our current exercise as we will need the status to be inserted into a bigger body to be returned.

Integrating serialization structs into our application code

Now that we have enabled our `TaskStatus` `enum` to be serialized, we can look back at *Figure 4.7* and see that our `Base` struct is next to be serialized. We can also see that the `Base` struct is the key to JSON serialization, but it is currently not public, so we need to make it public. This can be done by

More books: https://t.me/booksforits

changing the declaration of the base module in the `to_do/structs/mod.rs` file from `mod base;` to `pub mod base;`. Now that the `Base` struct is directly available outside of the module, we can build our own `json_serialization` module in the `src` directory with the following structure:

```
├── main.rs
├── json_serialization
|   ├── mod.rs
|   └── to_do_items.rs
```

We will define what we will return to the viewer when the `get` view is called in the `src/json_serialization/to_do_items.rs` file with the following code:

```
use serde::Serialize;
use crate::to_do::ItemTypes;
use crate::to_do::structs::base::Base;

#[derive(Serialize)]
pub struct ToDoItems {
    pub pending_items: Vec<Base>,
    pub done_items: Vec<Base>,
    pub pending_item_count: i8,
    pub done_item_count: i8
}
```

In the preceding code, all we have done is define a standard public struct's parameters. We then used the `derive` macro to implement the `Serialize` trait. This enables the struct's attributes to be serialized to JSON with the name of the attribute as the key. For instance, if the `ToDoItems` struct had a `done_item_count` of one, then the JSON body would denote it as `"done_item_count": 1`. We can see that this is easier than the manual serialization that we did for our `TaskStatus` `enum` earlier. This is because the format of our fields is straightforward. If we do not need any extra logic during the serialization, decorating our `ToDoItems` with the `Serialize` trait is the easiest approach, which will result in fewer errors.

Now that serialization is defined, we must consider the processing of the data. It would not be scalable if we must sort the data and count it before calling the `struct`. This would add unnecessary code into the view that processes data for serialization as opposed to the logic belonging to the view in question. It would also enable duplicate code. There is only going to be one way we sort, count, and serialize the data. If other views are needed to return the list of items, then we would have to duplicate the code again.

More books: https://t.me/booksforits

Considering this, it makes sense to build a constructor for the struct where we ingest a vector of to-do items, sort them into the right attributes, and then count them. We can define the constructor with the following code:

```
impl ToDoItems {

    pub fn new(input_items: Vec<ItemTypes>) -> ToDoItems {
        . . . // code to be filled in
    }
}
```

In the preceding code, we can see that our constructor takes in a vector of to-do items that we will have loaded from our JSON file. Inside our constructor, we must carry out the following steps:

1. Sort the items into two vectors, one for pending items, and the other for complete items.

   We will merely loop through the vector of items appending to different vectors depending on the item type with the following code:

   ```
   let mut pending_array_buffer = Vec::new();
   let mut done_array_buffer = Vec::new();

   for item in input_items {
       match item {
           ItemTypes::Pending(packed) => pending_array_
   buffer.
               push(packed.super_struct),
           ItemTypes::Done(packed) => done_array_buffer.
   push(
               packed.super_struct)
       }
   }
   ```

2. Count the total number of pending and complete items.

   For the next step, we can then call the `len` function on each vector. The `len` function returns `usize`, which is a pointer-sized unsigned integer type. Because of this, we can cast it as `i8` with the following code:

   ```
   let done_count: i8 = done_array_buffer.len() as i8;
   let pending_count: i8 = pending_array_buffer.len() as i8;
   ```

More books: https://t.me/booksforits

3. We now have all the data that we need for constructing and returning the struct, which can be defined using the following code:

```
return ToDoItems{
    pending_items: pending_array_buffer,
    done_item_count: done_count,
    pending_item_count: pending_count,
    done_items: done_array_buffer
}
```

   Now our constructor is done.

We can now build our struct using this function. All we must do is plug it into our application so that we can pass it into our application. In the `json_serialization/mod.rs` file, we can make it public with the following code:

```
pub mod to_do_items;
```

We can now declare our module in the `src/main.rs` file with the following code:

```
mod json_serialization;
```

We also must ensure that our `base` module is public in the `src/to_do/structs/mod.rs` file. We are going to be serializing the | struct when returning data which can be achieved in the `src/to_do/structs/base.rs` file with the following code:

```
pub mod to_do_items;
use super::super::enums::TaskStatus;
use serde::Serialize;

#[derive(Serialize)]
pub struct Base {
    pub title: String,
    pub status: TaskStatus
}
```

To utilize our struct, we must define it in our `GET` view in our `views/to_do/get.rs` file and return it with the following code:

```
use actix_web::{web, Responder};
use serde_json::value::Value;
use serde_json::Map;
```

More books: https://t.me/booksforits

```
use crate::state::read_file;
use crate::to_do::{ItemTypes, to_do_factory,
enums::TaskStatus};
use crate::json_serialization::to_do_items::ToDoItems;

pub async fn get() -> impl Responder {
    let state: Map<String, Value> = read_file(
                                    "./state.json");
    let mut array_buffer = Vec::new();

    for (key, value) in state {
        let status = TaskStatus::from_string(
                          &value.as_str().unwrap())
                                    .to_string();
        let item: ItemTypes = to_do_factory(
                                &key, status);
        array_buffer.push(item);
    }
    let return_package: ToDoItems = ToDoItems::new(
                                     array_buffer);
    return web::Json(return_package);
}
```

The preceding code is another example of a moment where everything clicks together. We use our `read_file` interface to get the state from the JSON file. We can then loop through the map converting the item type into a string and feed it into our `to_do_factory` interface. Once we have the constructed item from the factory, we append it to a vector and feed that vector into our JSON serialization struct. After hitting the `get` view, we receive the following JSON body:

```
{
    "pending_items": [
        {
            "title": "learn to code rust",
            "status": "PENDING"
        },
        {
            "title": "washing",
```

More books: https://t.me/booksforits

```
            "status": "PENDING"
        }
    ],
    "done_items": [],
    "pending_item_count": 2,
    "done_item_count": 0
}
```

We now have a well-structured response that we can expand on and edit. The development of applications never stops so if you were going to continue to maintain this application you will be adding features to this return JSON body. We will soon move onto other views. However, before we do this, we must acknowledge that we will be returning the full list of items with counts every time we make an API call. Therefore, we must package this response in a function for every function; otherwise, we will be writing the same code that we wrote in the `get` view for every other view. In the next section, we will cover how we can package our to-do items so that they can be returned in multiple views.

Packaging our custom serialized struct to be returned to users

Now, our `GET` view returns an implementation of the `Responder` trait. This means that if our `ToDoItems` struct also implements this, it can be directly returned in a view. We can do this in our `json_serialization/to_do_items.rs` file. First, we must import the following structs and traits:

```
use serde::Serialize;
use std::vec::Vec;

use serde_json::value::Value;
use serde_json::Map;

use actix_web::{
    body::BoxBody, http::header::ContentType,
    HttpRequest, HttpResponse, Responder,
};

use crate::to_do::ItemTypes;
use crate::to_do::structs::base::Base;
use crate::state::read_file;
use crate::to_do::{to_do_factory, enums::TaskStatus};
```

More books: https://t.me/booksforits

We can see from the `actix_web` crate that we have imported a range of structs and traits that will enable us to build an HTTP response. We can now implement the `get` view code in a `get_state` function for the `ToDoItems` struct with the following code:

```
impl ToDoItems {

    pub fn new(input_items: Vec<ItemTypes>) -> ToDoItems {
        . . .
    }

    pub fn get_state() -> ToDoItems {
        let state: Map<String, Value> = read_file("./state.
            json");
        let mut array_buffer = Vec::new();

        for (key, value) in state {
            let status = TaskStatus::from_string(&value
                        .as_str().unwrap().to_string());
            let item = to_do_factory(&key, status);
            array_buffer.push(item);
        }
        return ToDoItems::new(array_buffer)
    }
}
```

The preceding code enables us to get all the to-do items from our JSON file with just one line of code. We must enable our `ToDoItems` struct to be returned in a view by implementing the `Responder` trait with the following code:

```
impl Responder for ToDoItems {
    type Body = BoxBody;

    fn respond_to(self, _req: &HttpRequest)
                            -> HttpResponse<Self::Body> {
        let body = serde_json::to_string(&self).unwrap();
        HttpResponse::Ok()
            .content_type(ContentType::json())
            .body(body)
```

More books: https://t.me/booksforits

```
    }
}
```

In the preceding code, what we have essentially done is serialize our `ToDoItems` struct using the `serde_json` crate, and then returned an HTTP response with the `ToDoItems` struct at the body. The `respond_to` function will be called when our `ToDoItems` struct is being returned in a view. Now this is where it gets really cool. We can rewrite our `views/to_do/get.rs` file with the following code:

```
use actix_web::Responder;
use crate::json_serialization::to_do_items::ToDoItems;

pub async fn get() -> impl Responder {
    return ToDoItems::get_state();
}
```

That is it! If we run our application now, we will get the same response as we had before. With this, we can see how traits can abstract code for our views. Now that we have created the `get` view, we must work on building other views that create, edit, and delete. To do this, we are going to move on to our next section, which is extracting data from our views.

Extracting data from views

In this section, we are going to explore extracting data from our HTTP requests from the header and body. We are then going to use these methods to edit, delete to-do items, and intercept requests before they are fully loaded with middleware. We will go one step at a time. For now, let us extract data from the body of the HTTP request to edit a to-do item. When it comes to accepting data in JSON format, we should do what we have been doing throughout the book, separating this code from the view. If we think about it, we just need to send in the item that we are editing. However, we can also use this same schema for deleting. We can define our schema in our `json_serialization/to_do_item.rs` file with the following code:

```
use serde::Deserialize;

#[derive(Deserialize)]
pub struct ToDoItem {
    pub title: String,
    pub status: String
}
```

More books: https://t.me/booksforits

In the preceding code, we have merely stated which type of data we need for each field as we cannot pass enums through JSON; only strings can be passed. The deserialization from JSON is enabled by decorating the `ToDoItem` struct with the `Deserialize` trait macro. We must remember to make the `ToDoItem` struct available to the rest of the application, so our `json_serialization/mod.rs` file should look like the following:

```
pub mod to_do_items;
pub mod to_do_item;
```

Now that our item extraction is done, we can move on to our `edit` view. In our `views/to_do/edit.rs` file we can import what we need with the following code:

```
use actix_web::{web, HttpResponse};
use serde_json::value::Value;
use serde_json::Map;

use crate::state::read_file;

use crate::to_do::{to_do_factory, enums::TaskStatus};
use crate::json_serialization::{to_do_item::ToDoItem,
                                to_do_items::ToDoItems};
use crate::processes::process_input;
```

In the preceding code, we can see that we need to import the standard serialization and web structs needed for a view. We also import the structs `ToDoItem` and `ToDoItems` for ingesting data and returning the entire state of the application. We can then import our `process_input` function that processes the input with a command. At this point, looking at the imports, can you think of the steps needed to perform our edit? Have a think before moving forward. The path is like what we did with a `get` view; however, we must update the state with the new updated item. We must also remember that our `process_input` function will edit the to-do item if the `edit` command is passed in.

After thinking it through, remember, there are many ways to solve a problem. If your steps solve the problem, then do not feel bad if it is different from the steps laid out. You may also produce a better solution. Our `edit` view involves the following steps:

1. Get the state of the entire application for the to-do items.
2. Check to see if the item is there, returning a `not found` response if it is not.
3. Pass the data through the `to_do_factory` factory to construct the existing data from the state to an item that we can manipulate.
4. Check that the status being put in is not the same as the existing status.

More books: https://t.me/booksforits

5. Pass the existing item into the `process_input` function with an `edit` command so it is saved to the JSON state file.
6. Get the state of the application and return it.

With these steps in mind, we can concrete our knowledge of extracting JSON from the body of the request and process it for editing in the next subsection.

Extracting JSON from the body of a request

Now that we have the imports done and the outline defined, we can define the outline of our view with the following code:

```
pub async fn edit(to_do_item: web::Json<ToDoItem>)
                                    -> HttpResponse {
    . . .
}
```

In the preceding code, we can see that our `ToDoItem` struct is wrapped in the `web::Json` struct. This means that the parameter `to_do_item` will be extracted from the body of the request, serialized, and constructed as the `ToDoItem` struct. So, inside our view, our `to_do_item` is a `ToDoItem` struct. Thus, inside our view, we can load our state with the following code:

```
let state: Map<String, Value> = read_file("./state.json");
```

We can then extract the item data from our state with the following code:

```
let status: TaskStatus;
match &state.get(&to_do_item.title) {
    Some(result) => {
        status = TaskStatus::new(result.as_str().unwrap());
    }
    None=> {
        return HttpResponse::NotFound().json(
            format!("{} not in state", &to_do_item.title))
    }
}
```

More books: https://t.me/booksforits

In the preceding code, we can see that we can construct the status from the data or return a `not found` HTTP response if it is not found. We then need to construct the item struct with the existing data using the following code:

```
let existing_item = to_do_factory(to_do_item.title.as_str(),
              status.clone());
```

In the preceding code, we can see why our factory is coming in handy. Now, we need to compare the new and existing status for the item. There is no point altering the status if the desired status is the same as the following code:

```
if &status.stringify() == &TaskStatus::from_string(
                          &to_do_item.status.as_str()
                          .to_string()).stringify() {
    return HttpResponse::Ok().json(ToDoItems::get_state())
}
```

Therefore, we need to check the current status and if it is the same as the desired status, we merely return an `Ok` HTTP response state. We do this because the frontend client might be out of sync. In the next chapter, we will be writing the frontend code and we will see that the items will be cached and rendered. If, let's say, another tab is open with our application or we have updated our to-do application on another device such as a phone, then the client making this request might be out of sync. We do not want to execute a command based on an out-of-sync frontend. We then need to process the input by making an edit and then returning the state with the following code:

```
process_input(existing_item, "edit".to_owned(), &state);
return HttpResponse::Ok().json(ToDoItems::get_state())
```

The preceding code should work but right now, it will not. This is because we need to clone our `TaskStatus` enum and our `TaskStatus` does not implement the `Clone` trait. This can be updated in our `src/to_do/enums.rs` file with the following code:

```
#[derive(Clone)]
pub enum TaskStatus {
    DONE,
    PENDING
}
```

We must then ensure that the `edit` view is available and defined in the `to-do` view factory. So, in the `src/views/to_do/mod.rs` file, our factory should look like the following:

```
mod create;
```

More books: https://t.me/booksforits

```
mod get;
mod edit;

use actix_web::web::{ServiceConfig, post, get, scope};


pub fn to_do_views_factory(app: &mut ServiceConfig) {
    app.service(
        scope("v1/item")
        .route("create/{title}", post().to(create::create))
        .route("get", get().to(get::get))
        .route("edit", post().to(edit::edit))
    );
}
```

We can see that our view factory is scaling nicely. We can also stand back and appreciate that all our views for to-do items are defined nicely in one isolated page meaning we can simply look at the preceding code and still know that we need a `delete` view. We can now run our application and make a request in Postman with the following configuration:

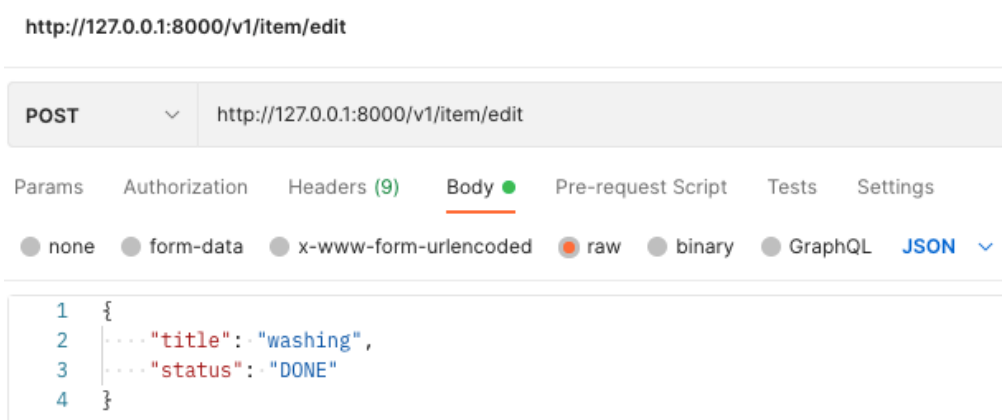


Figure 4.8 – Edit request with Postman

We can see in *Figure 4.8* that we are switching our washing task to `"DONE"`. We have put this data in the body as raw with the format to be JSON. If we make this call to the `edit` endpoint, we will get the following response:

```
{
    "pending_items": [
```

More books: https://t.me/booksforits

```
        {
            "title": "learn to code rust",
            "status": "PENDING"
        }
    ],
    "done_items": [
        {
            "title": "washing",
            "status": "DONE"
        }
    ],
    "pending_item_count": 1,
    "done_item_count": 1
}
```

In the preceding code, we can see that the done items list is now populated and that the counts have been altered. If we continue to make the same call, we will get the same response as we will be editing the `washing` item to `done` when it already has a done status. We will have to switch the status of `washing` back to `pending` or change the title in our call to get a different updated state. If we do not include `title` and `status` in the body of our call, then we will get a bad request response instantly, because the `ToDoItem` struct is expecting those two fields.

Now that we have locked down the process of receiving and returning JSON data in the URL parameters and body, we are nearly done. However, we have one more important method to cover that's used for data extraction – the header. Headers are used to store meta information such as security credentials.

Extracting data from the header in requests

If we needed to authorize a range of requests; it would not be scalable to put them in all our JSON structs. We also must acknowledge that the request body could be large, especially if the requester is being malicious. Therefore, it makes sense to access the security credentials before passing the request through to the view. This can be done by intercepting the request through what is commonly known as middleware. Once we've intercepted the request, we can access the security credentials, check them, and then process the view.

In the previous edition of this book, we manually developed our middleware for our authentication. However, this is not scalable in terms of code management and does not allow flexibility. However, it is important to cover manually configuring your own middleware to have a better understanding of how the server constructors work giving you the flexibility of processing requests. To intercept our requests, we need to add the `actix-service` crate. With this installation, our `Cargo.toml` file dependencies should look like the following definitions:

More books: https://t.me/booksforits

```
[dependencies]
actix-web = "4.0.1"
serde_json = "1.0.59"
serde = { version = "1.0.136", features = ["derive"] }
actix-service = "2.0.2"
```

Now, we can update our `src/main.rs` file. First, our imports should look like the following:

```
use actix_web::{App, HttpServer};
use actix_service::Service;

mod views;
mod to_do;
mod state;
mod processes;
mod json_serialization;
```

Now that all the imports are done, we can define our server construction with the following code:

```
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        let app = App::new()
            .wrap_fn(|req, srv|{
                println!("{:?}", req);
                let future = srv.call(req);
                async {
                    let result = future.await?;
                    Ok(result)
                }
        }).configure(views::views_factory);
        return app
    })
    .bind("127.0.0.1:8000")?
    .run()
    .await
}
```

More books: https://t.me/booksforits

In the preceding code, we can see that the `wrap_fn` enables us to interact with the request (`req`). The service routing (`srv`) can be called when we need it to pass the request. We must note that calling the routing service is a future that we then wait to finish in an `async` code block returning the result. This is middleware. We can manipulate our request, inspect it, and reroute or return it before calling the routing service to process the HTTP request in one of our views. For us, we are just printing out the debug of the request, which looks like the following printout:

```
ServiceRequest HTTP/1.1 GET:/v1/item/get
  headers:
    "accept-language": "en-GB,en-US;q=0.9,en;q=0.8"
    "accept": "text/html,application/xhtml+xml,application/xml;
    q=0.9,image/avif,image/webp,image/
        apng,*/*;q=0.8,application
    signed-exchange;v=b3;q=0.9"
    "sec-ch-ua-platform": "\"macOS\""
    "sec-fetch-site": "none"
    . . .
    "host": "127.0.0.1:8000"
    "connection": "keep-alive"
    "sec-fetch-user": "?1"
```

We can see that we have a lot of data to work with. But this is as far as we will go with our homemade middleware. We will now investigate extracting data from headers using traits.

Simplifying header extraction with traits

Before we do this, we will have to install the futures crate, adding the following to the dependencies section of the `Cargo.toml` file:

```
futures = "0.3.21"
```

We will now create an `src/jwt.rs` file to house our **JSON Web Token** (**JWT**). This is where we can store encrypted data about the user once the user has logged in. We can then send the JWT in the header with every HTTP request. In *Chapter 7*, *Managing User Sessions*, we will explore the security nuances of checking and managing these tokens. For now, we will simply build the header extraction process. We will start by importing the needed structs and traits in the `src/jwt.rs` file with the following code:

```
use actix_web::dev::Payload;
use actix_web::{Error, FromRequest, HttpRequest};
use futures::future::{Ready, ok};
```

More books: https://t.me/booksforits

The `Payload` struct houses the requests raw data stream. We then have the `FromRequest` trait which is what we are going to implement to extract the data before it hits the view. We then use the `Ready` and `ok` from futures to wrap the result of our data extraction to create a future that is immediately ready with a success value, which is the value from our header extraction. Now that we have imported what we need, we can define our JWT struct with the following code:

```
pub struct JwToken {
    pub message: String
}
```

For now, we are only going to have a message but, in the future, we will be adding fields like the ID of the user. With this struct, we can implement the `FromRequest` trait with the following code:

```
impl FromRequest for JwToken {
    type Error = Error;
    type Future = Ready<Result<JwToken, Error>>;

    fn from_request(req: &HttpRequest, _: &mut Payload)
                                          -> Self::Future {
    . . .
    }
}
```

We can deduce that the `from_request` function gets called before the view is loaded. We are extracting the header, which is why we have no interest in the payload. So, we mark the parameter with `_`. We need to define the `Future` type, which is a ready future housing a result that can either be our `JwToken` struct or an error. Inside the `from_request` function, we can extract the data from the header with the following code:

```
match req.headers().get("token") {
    Some(data) => {
        let token = JwToken{
            message: data.to_str().unwrap().to_string()
        };
        ok(token)
    },
    None => {
        let token = JwToken{
            message: String::from("nothing found")
        };
```

More books: https://t.me/booksforits

```
        ok(token)
    }
}
```

In the preceding code, we can see that for this chapter we just look for the `token` key and if it is there, we return the `JwToken` struct with the message. If not, we will return the `JwToken` struct with nothing found. As this chapter focuses on data, this is where we stop, but in *Chapter 7*, *Managing User Sessions*, we will revisit this function and explore concepts such as throwing errors and returning requests before they hit the view with unauthorized codes. For now, we must make our `JwToken` struct accessible by defining it in our `src/main.rs` file with the following line of code:

```
mod jwt;
```

Now that we have gone through the hassle of implementing a trait, using it is going to be compact and simple. Let us revisit our `edit` view in our `views/to_do/edit.rs` file, import our `JwToken` struct, add our `JwToken` struct to our parameters in the `edit` view, and print out the message as seen in the following code:

```
use crate::jwt::JwToken;

pub async fn edit(to_do_item: web::Json<ToDoItem>,
                  token: JwToken) -> HttpResponse {
    println!("here is the message in the token: {}",
             token.message);
    . . .
```

Clearly, we do not want to edit the rest of the view but as we can deduce from the preceding code, the `token` parameter is the constructed `JwToken` struct that has been extracted from the HTTP request and is ready for use just like the `ToDoItem` struct. If we make the same edit HTTP call now after running the server, we will see that the HTTP request is printed out, but we also get the following response:

```
here is the message in the token: nothing found
```

It looks like it is working, as we haven't added anything to the header yet. We can add a token into our header with the Postman setup defined in the following figure:

More books: https://t.me/booksforits

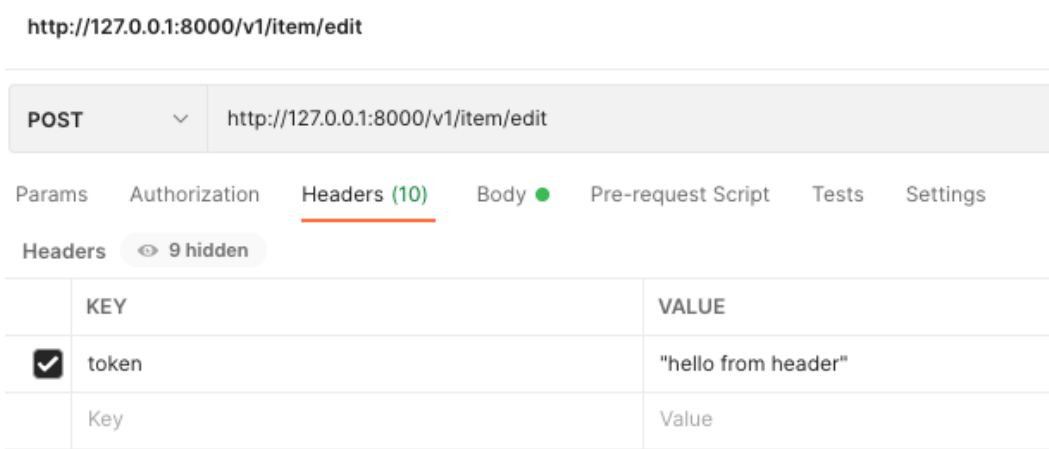


Figure 4.9 – Edit request in Postman with header

If we send the request again, we get the following printout:

```
here is the message in the token: "hello from header"
```

That's it! We can pass data through headers. What's more, adding and taking them away from views is as easy as defining them in the parameters and removing them.

Summary

In this chapter, we have put all of what we have learned in the previous chapters to good use. We fused the logic from the to-do item factory, which loads and saves to-do items from a JSON file, and looked at the to-do item process logic by using the basic views from `Actix-web`. With this, we have been able to see how the isolated modules click together. We will keep reaping the benefits of this approach in the next few chapters as we rip out the JSON file that loads and saves a database.

We also managed to utilize the `serde` crate to serialize complex data structures. This allows our users to get the full state update returned to them when they make an edit. We also built on our knowledge of futures, `async` blocks, and closures to intercept requests before they reached the view. Now, we can see that the power of Rust is enabling us to do some highly customizable things to our server, without us having to dig deep into the framework.

Thus, Rust has a strong future in web development. Despite its infancy, we can get things up and running with little to no code. With a few more lines of code and a closure, we are building our own middleware. Our JSON serialization structs were made possible with just one line of code, and the traits provided by `Actix` enabled us to merely define the parameter in the view function, thus enabling the view to automatically extract the data from the body and serialize it into the struct. This scalable,

More books: https://t.me/booksforits

powerful, and standardized way of passing data is more concise than many high-level languages. We can now fully interact with and inspect every part of the HTTP request.

Now that we are processing and returning well-structured data to the user, we can start displaying it in an interactive way for our user to point and click when editing, creating, and deleting to-do items.

In the next chapter, we will be serving HTML, CSS, and JavaScript from the `Actix-web` server. This will enable us to see and interact with to-do items via a graphical user interface, with the JavaScript making API calls to the endpoints we defined in this chapter.

Questions

1. What is the difference between a `GET` and `POST` request?
2. Why would we have middleware when we check credentials?
3. How do you enable a custom `struct` to be directly returned in a view?
4. How do you enact middleware for the server?
5. How do you enable a custom `struct` to serialize data into the view?

Answers

1. A `GET` request can be cached and there are limits to the types and amount of data that can be sent. A `POST` request has a body, which enables more data to be transferred. Also, it cannot be cached.
2. We use middleware to open the header and check the credentials before sending the request to the desired view. This gives us an opportunity to prevent the body from being loaded by returning an `auth` error before loading the view preventing the potentially malicious body.
3. For the struct to be directly returned, we will have to implement the `Responder` trait. During this implementation, we will have to define the `responded_to` function that accepts the HTTP request struct. The `responded_to` will be fired when the struct is returned.
4. In order to enact middleware, we enact the `wrap_fn` function on the `App` struct. In the `wrap_fn` function, we pass a closure that accepts the service request and routing structs.
5. We decorate the struct with the `#[derive(Deserialize)]` macro. Once we have done this, we define the parameter type to be wrapped in a JSON struct: `parameter: web::Json<ToDoItem>`.

More books: https://t.me/booksforits

5
Displaying Content in the Browser

We are now at the stage where we can build a web application that can manage a range of HTTP requests with different methods and data. This is useful, especially if we are building a server for microservices. However, we also want non-programmers to interact with our application to use it. To enable non-programmers to use our application, we must create a graphical user interface. However, it must be noted that this chapter does not contain much Rust. This is because other languages exist to render a graphical user interface. We will mainly use HTML, JavaScript, and CSS. These tools are mature and widely used for frontend web development. While I personally love Rust (otherwise I wouldn't be writing a book on it), we must use the right tool for the right job. At the time of writing this book, we can build a frontend application in Rust using the Yew framework. However, being able to fuse more mature tools into our Rust technical stack is a more valuable skill.

This chapter will cover the following topics:

- Serving HTML, CSS, and JavaScript using Rust
- Building a React application that connects to a Rust server
- Converting our React application into a desktop application to be installed on the computer

In the previous edition (*Rust Web Programming: A hands-on guide to developing fast and secure web apps with the Rust programming language*), we merely served frontend assets directly from Rust. However, due to feedback and revision, this does not scale well, leading to a lot of repetition. Raw HTML, CSS, and JavaScript served directly by Rust were also prone to errors due to the unstructured nature of using this approach, which is why, in this second edition, we will cover React as well as provide a brief introduction to serving frontend assets directly with Rust. By the end of this chapter, you will be able to write basic frontend graphical interfaces without any dependencies, as well as understand the trade-offs between low-dependency frontend solutions and full frontend frameworks such as React. Not only will you understand when to use them but you will also be able to implement both approaches as and when they are needed for your project. As a result, you will be able to pick the right tool for the right job and build an end-to-end product using Rust in the backend and JavaScript in the frontend.

More books: https://t.me/booksforits

Technical requirements

We will be building on the server code we created in the previous chapter, which can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter04/extracting_data_from_views/web_app`.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter05`.

We will also be using Node.js to run our React application. Node and npm can be installed by carrying out the steps laid out at `https://docs.npmjs.com/downloading-and-installing-node-js-and-npm`.

Serving HTML, CSS, and JavaScript using Rust

In the previous chapter, we returned all our data in the form of JSON. In this section, we are going to return HTML data for the user to see. In this HTML data, we will have buttons and forms that enable the user to interact with the API endpoints that we defined in the previous chapter to create, edit, and delete to-do items. To do this, we will need to structure our own `app` views module that takes the following structure:

```
views
├── app
│   ├── items.rs
│   └── mod.rs
```

Serving basic HTML

In our `items.rs` file, we will be defining the main view that displays the to-do items. However, before we do that, we should explore the simplest way in which we can return HTML in the `items.rs` file:

```
use actix_web::HttpResponse;

pub async fn items() -> HttpResponse {
    HttpResponse::Ok()
        .content_type("text/html; charset=utf-8")
        .body("<h1>Items</h1>")
}
```

Here, we simply return an `HttpResponse` struct that has an HTML content type and a body of `<h1>Items</h1>`. To pass `HttpResponse` into the app, we must define our factory in the `app/views/mod.rs` file, as follows:

More books: https://t.me/booksforits

```
use actix_web::web;
mod items;

pub fn app_views_factory(app: &mut web::ServiceConfig) {
    app.route("/", web::get().to(items::items));
}
```

Here, we can see that instead of building a service, we merely defined a `route` for our application. This is because this is the landing page. If we were going to define a `service` instead of a `route`, we would not be able to define the views for the service without a prefix.

Once we have defined `app_views_factory`, we can call it in our `views/mod.rs` file. However, first, we must define the app module at the top of the `views/mod.rs` file:

```
mod app;
```

Once we have defined the `app` module, we can call the app factory in the `views_factory` function within the same file:

```
app::app_views_factory(app);
```

Now that our HTML serving view is a part of our app, we can run it and call the home URL in our browser, giving us the following output:

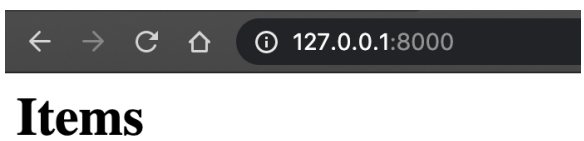


Figure 5.1 – First rendered HTML view

We can see that our HTML rendered! With what we saw in *Figure 5.1*, we can deduce that we can return a string at the body of the response with the following:

```
HttpResponse::Ok()
    .content_type("text/html; charset=utf-8")
    .body("<h1>Items</h1>")
```

This renders the HTML if the string is in HTML format. From this revelation, how do you think we can render HTML from HTML files that are served by our Rust server? Before moving on, think about this – this will exercise your problem-solving abilities.

More books: https://t.me/booksforits

Reading basic HTML from files

If we have an HTML file, we can render it by merely readying that HTML file to a string and inserting that string into the body of `HttpResponse`. Yes, it is that simple. To achieve this, we will build a content loader.

To build a basic content loader, start by building an HTML file reading function in the `views/app/content_loader.rs` file:

```
use std::fs;

pub fn read_file(file_path: &str) -> String {
    let data: String = fs::read_to_string(
        file_path).expect("Unable to read file");
    return data
}
```

All we must do here is return a string because this is all we need for the response body. Then, we must define the loader in the `views/app/mod.rs` file with the `mod content_loader;` line at the top of the file.

Now that we have a loading function, we need an HTML directory. This can be defined alongside the `src` directory called `templates`. Inside the `templates` directory, we can add an HTML file called `templates/main.html` with the following content:

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charSet="UTF-8"/>
        <meta name="viewport"
              content="width=device-width, initial-
                                           scale=1.0"/>
        <meta httpEquiv="X-UA-Compatible"
              content="ie=edge"/>
        <meta name="description"
              content="This is a simple to do app"/>
        <title>To Do App</title>
    </head>
    <body>
        <h1>To Do Items</h1>
```

More books: https://t.me/booksforits

```
    </body>
</html>
```

Here, we can see that our `body` tag has the same content that we presented previously – that is, `<h1>To Do Items</h1>`. Then, we have a `head` tag that defines a range of meta tags. We can see that we define `viewport`. This tells the browser how to handle the dimensions and scaling of the page content. Scaling is important because our application could be accessed by a range of different devices and screen sizes. With this viewport, we can set the width of the page to the same width as the device screen. Then, we can set the initial scale of the page that we access to `1.0`. Moving on to the `httpEquiv` tag, we set it to `X-UA-Compatible`, which means that we support older browsers. The final tag is simply a description of the page that can be used by search engines. Our `title` tag ensures that `to do app` is displayed on the browser tag. With that, we have our standard header title in our body.

Serving basic HTML loaded from files

Now that we have defined our HTML file, we must load and serve it. Going back to our `src/views/app/items.rs` file, we must load the HTML file and serve it with the following code:

```
use actix_web::HttpResponse;
use super::content_loader::read_file;


pub async fn items() -> HttpResponse {
    let html_data = read_file(
        "./templates/main.html");
    HttpResponse::Ok()
        .content_type("text/html; charset=utf-8")
        .body(html_data)
}
```

If we run our application, we will get the following output:

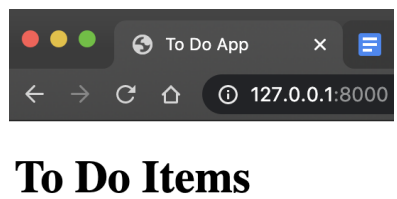


Figure 5.2 – View from loading the HTML page

More books: https://t.me/booksforits

In *Figure 5.2*, we can see that we have the same output as before. This is not surprising; however, we must notice that the tab in *Figure 5.2* now states **To Do App**, which means that the metadata in our HTML file is being loaded into the view. Nothing is stopping us from fully utilizing the HTML file. Now that our HTML file is being served, we can move on to our next ambition, which is adding functionality to our page.

Adding JavaScript to an HTML file

It would not be useful to the frontend user if they couldn't do anything to our state for to-do items. Before we amend this, we need to understand the layout of an HTML file by looking at the following figure:

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charSet="UTF-8"/>
        . . .
    </head>
    <style>
        {{CSS}}
    </style>
    <body>
        <h1>Done Items</h1>
        . . .
    </body>
    <script>
        {{JAVASCRIPT}}
    </script>
</html>
```

Figure 5.3 – General layout of an HTML file

Here, in *Figure 5.3*, we can see that we can define the meta tags in the header. However, we can also see that we can define style tags in the header. Within the style tags below the header, we can insert CSS into the style. Below the body, there is also a script section where we can inject JavaScript. This JavaScript runs in the browser and interacts with elements in the body. With this, we can see that serving an HTML file loaded with CSS and JavaScript provides a fully functioning frontend single-page app. With this, we can reflect on the introduction of this chapter. While I love Rust and feel a strong urge to tell you to write everything in it, this is just not a good idea for any language in software engineering. The ease with which we can serve functional frontend views with JavaScript now makes it the best choice for your frontend needs.

Communicating with our server using JavaScript

Now that we know where to insert the JavaScript into our HTML file, we can test our bearings. In the remainder of this section, we are going to create a button in the HTML body, fuse it to a JavaScript function, and then get the browser to print out an alert with an inputted message when that button

More books: https://t.me/booksforits

is pressed. This will do nothing to our backend application, but it will prove that our understanding of HTML files is correct. We can add the following code to our `templates/main.html` file:

```
<body>
    <h1>To Do Items</h1>
    <input type="text" id="name" placeholder="create to do
         item">
    <button id="create-button" value="Send">Create</button>
</body>
<script>
    let createButton = document.getElementById("create-
        button");
    createButton.addEventListener("click", postAlert);

    function postAlert() {
        let titleInput = document.getElementById("name");
        alert(titleInput.value);
        titleInput.value = null;
    }
</script>
```

In our body section, we can see that we define an `input` and a `button`. We give the `input` and `button` properties unique ID names. Then, we use the ID of `button` to add an event listener. After that, we bind our `postAlert` function to that event listener to be fired when our `button` is clicked. When we fire our `postAlert` function, we get `input` by using its ID and print out the value of `input` in an alert. Then, we set the value of `input` to `null` to enable the user to fill in another value to be processed. Serving our new `main.html` file, putting `testing` in `input` and then clicking the button will result in the following output:

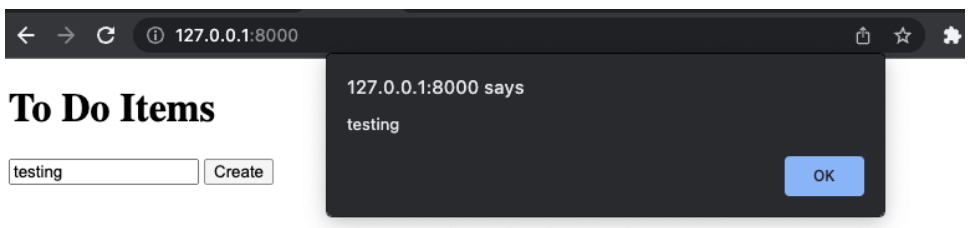


Figure 5.4 – The effect of clicking a button when connected to an alert in JavaScript

More books: https://t.me/booksforits

Our JavaScript does not have to stop with having elements interact in the body. We can also use JavaScript to perform API calls to our backend Rust application. However, we must stop and think before we rush off and write our entire application in our `main.html` file. If we did that, the `main.html` file would balloon into a massive file. It would be hard to debug. Also, this could lead to code duplication. What if we wanted to use the same JavaScript in other views? We would have to copy and paste it into another HTML file. This would not scale well and if we need to update a function, we could run the risk of forgetting to update some of the duplicated functions. This is where JavaScript frameworks such as React come in handy. We will explore React later in this chapter, but for now, we will complete our low-dependency frontend by coming up with a way in which we can separate our JavaScript from our HTML files.

It must be warned that we are essentially manually rewriting HTML on the fly with this JavaScript. People could describe this as a "hacky" solution. However, it is important to get to grips with our approach before exploring React to truly appreciate the benefits of different approaches. Before we move on to the next section, we do have to refactor our `create` view in the `src/views/to_do/create.rs` file. This is a good opportunity to revisit what we developed in the previous chapters. You must essentially convert the `create` view so that it returns our current state of the to-do items as opposed to a string. Once you have attempted this, the solution should look as follows:

```
use actix_web::HttpResponse;
use serde_json::Value;
use serde_json::Map;
use actix_web::HttpRequest;

use crate::to_do::{to_do_factory, enums::TaskStatus};
use crate::json_serialization::to_do_items::ToDoItems;
use crate::state::read_file;
use crate::processes::process_input;


pub async fn create(req: HttpRequest) -> HttpResponse {
    let state: Map<String, Value> =
        read_file("./state.json");

    let title: String = req.match_info().get("title"
    ).unwrap().to_string();

    let item = to_do_factory(&title.as_str(),
        TaskStatus::PENDING);
```

More books: https://t.me/booksforits

```
    process_input(item, "create".to_string(), &state);
    return HttpResponse::Ok().json(ToDoItems::get_state())
}
```

Now, all our to-do items are up to date and functioning. We can now move on to the next section, where we will be getting our frontend to make calls to our backend.

Injecting JavaScript into HTML

Once we have finished this section, we will have a not-so-pretty but fully functional main view where we can add, edit, and delete to-do items using JavaScript to make calls to our Rust server. However, as you may recall, we did not add a `delete` API endpoint. To inject JavaScript into our HTML, we will have to carry out the following steps:

1. Create a `delete` item API endpoint.
2. Add a `JavaScript loading` function and replace the JavaScript tag in the HTML data with the loaded JavaScript data in the main item Rust view.
3. Add a JavaScript tag in the HTML file and IDs to the HTML components so that we can reference the components in our JavaScript.
4. Build a `rendering` function for our to-do items in JavaScript and bind it to our HTML via IDs.
5. Build an `API call` function in JavaScript to talk to the backend.
6. Build the `get`, `delete`, `edit`, and `create` functions in JavaScript for our buttons to use.

Let's have a detailed look at this.

Adding the delete endpoint

Adding the `delete` API endpoint should be straightforward now. If you want to, it is advised to try and implement this view by yourself as you should be comfortable with this process by now:

1. If you are struggling, we can achieve this by importing the following third-party dependencies into the `views/to_do/delete.rs` file:

   ```
   use actix_web::{web, HttpResponse};
   use serde_json::value::Value;
   use serde_json::Map;
   ```

   These are not new, and you should be familiar with them and know where we need to utilize them.

2. Then, we must import our structs and functions with the following code:

   ```
   use crate::to_do::{to_do_factory, enums::TaskStatus};
   use crate::json_serialization::{to_do_item::ToDoItem,
   ```

More books: https://t.me/booksforits

```
    to_do_items::ToDoItems};
use crate::processes::process_input;
use crate::jwt::JwToken;
use crate::state::read_file;
```

Here, we can see that we are using our `to_do` module to construct our to-do items. With our `json_serialization` module, we can see that we are accepting `ToDoItem` and returning `ToDoItems`. Then, we execute the deletion of our item with the `process_input` function. We also do not want anyone who can visit our page to be deleting our items. Therefore, we need our `JwToken` struct. Finally, we read the state of our items with the `read_file` function.

3. Now that we have all that we need, we can define our `delete` view with the following code:

```
pub async fn delete(to_do_item: web::Json<ToDoItem>,
    token: JwToken) -> HttpResponse {
    . . .
}
```

Here, we can see that we have accepted `ToDoItem` as JSON and that we have attached `JwToken` for the view so that the user must be authorized to access it. At this point, we only have `JwToken` attaching a message; we will manage the authentication logic for `JwToken` in *Chapter 7*, *Managing User Sessions*.

4. Inside our `delete` view, we can get the state of our to-do items by reading our JSON file with the following code:

```
let state: Map<String, Value> = read_file("./state.
json");
```

5. Then, we can check if the item with this title is in the state. If it is not, then we return a not found HTTP response. If it is, we then pass on the status as we need the title and status to construct the item. We can achieve this checking and status extraction with the following code:

```
let status: TaskStatus;
match &state.get(&to_do_item.title) {
    Some(result) => {
        status = TaskStatus::from_string
                 (result.as_str().unwrap().to_
string()                 );
    }
    None=> {
        return HttpResponse::NotFound().json(
            format!("{} not in state",
```

More books: https://t.me/booksforits

```
                        &to_do_item.title))
    }
}
```

6. Now that we have the status and title of the to-do item, we can construct our item and pass it through our `process_input` function with a `delete` command. This will delete our item from the JSON file:

```
let existing_item = to_do_factory(to_do_item.title.as_
    str(),
    status.clone());
process_input(existing_item, "delete".
    to_owned(),
    &state);
```

7. Remember, we implemented the `Responder` trait for our `ToDoItems` struct, and our `ToDoItems::get_state()` function returns a `ToDoItems` struct populated with items from the JSON file. Therefore, we can have the following return statement from our `delete` view:

```
return HttpResponse::Ok().json(ToDoItems::get_state())
```

8. Now that our `delete` view has been defined, we can add it to our `src/views/to_do/mod.rs` file, resulting in our view factory that looks like this:

```
mod create;
mod get;
mod edit;
mod delete;

use actix_web::web::{ServiceConfig, post, get, scope};

pub fn to_do_views_factory(app: &mut ServiceConfig) {
    app.service(
        scope("v1/item")
        .route("create/{title}",
                post().to(create::create))
        .route("get", get().to(get::get))
        .route("edit", post().to(edit::edit))
        .route("delete", post().to(delete::delete))
    );
}
```

More books: https://t.me/booksforits

9. By quickly inspecting `to_do_views_factory`, we can see that we have all the views that we need to manage our to-do items. If we were to eject this module out of our application and insert it into another, we would instantly see what we were deleting and adding.

With our `delete` view fully integrated into our application, we can move on to the second step, which is building our JavaScript loading functionality.

Adding a JavaScript loading function

Now that all our endpoints are ready, we must revisit our main app view. In the previous section, we established that the JavaScript in the `<script>` section works even though it is all just part of one big string. To enable us to put our JavaScript into a separate file, our view will load the HTML file as a string that has a {{JAVASCRIPT}} tag in the `<script>` section of the HTML file. Then, we will load the JavaScript file as a string and replace the {{JAVASCRIPT}} tag with the string from the JavaScript file. Finally, we will return the full string in the body in the `views/app/items.rs` file:

```
pub async fn items() -> HttpResponse {
    let mut html_data = read_file(
        "./templates/main.html");
    let javascript_data = read_file(
        "./javascript/main.js");

    html_data = html_data.replace("{{JAVASCRIPT}}",
        &javascript_data);
    HttpResponse::Ok()
        .content_type("text/html; charset=utf-8")
        .body(html_data)
}
```

Adding JavaScript tags in the HTML

From our `items` function in the previous step, we can see that we need to build a new directory in the root called `JavaScript`. We must also create a file in it called `main.js`. With this change to the app view, we are also going to have to change the `templates/main.html` file by adding the following code:

```
<body>
    <h1>Done Items</h1>
    <div id="doneItems"></div>
    <h1>To Do Items</h1>
```

More books: https://t.me/booksforits

```
    <div id="pendingItems"></div>
    <input type="text" id="name" placeholder="create to do
     item">
    <button id="create-button" value="Send">Create</button>
</body>
<script>
    {{JAVASCRIPT}}
</script>
```

Recall that our endpoints return pending items and completed items. Because of this, we have defined both lists with their own titles. `div` with the ID of `"doneItems"` is where we will insert the done to-do items from an API call.

Then, we will insert our pending items from an API call in `div` with the ID of `"pendingItems"`. After that, we must define an input with text and a button. This will be for our user to create a new item.

Building a rendering JavaScript function

Now that our HTML has been defined, we are going to define the logic in our `javascript/main.js` file:

1. The first function that we are going to build renders all the to-do items on our main page. It must be noted that this is the most convoluted part of the code in the `javascript/main.js` file. We are essentially writing JavaScript code that writes HTML code. Later, in the *Creating a React app* section, we will replace the need to do this using the React framework. For now, we will build a render function that will create a list of items. Each item takes the following form in HTML:

```
<div>
    <div>
        <p>learn to code rust</p>

        <button id="edit-learn-to-code-rust">
            edit
        </button>

    </div>
</div>
```

More books: https://t.me/booksforits

We can see that the title of the to-do item is nested in a paragraph HTML tag. Then, we have a button. Recall that the `id` property of an HTML tag must be unique. Therefore, we construct this ID based on what the button is going to do on it and the title of the to-do item. This will enable us to bind functions performing API calls to these `id` properties using event listeners.

2. To build our render function, we are going to have to pass in the items to be rendered, the process type that we are going to perform (that is, `edit` or `delete`), the element ID of the section in the HTML where we are going to render these items, and the function that we are going to bind to each to-do item button. The outline of this function is defined in the following code:

```
function renderItems(items, processType,
    elementId, processFunction) {
 . . .
}
```

3. Inside our `renderItems` function, we can start by constructing HTML and looping through our to-do items with the following code:

```
let itemsMeta = [];
let placeholder = "<div>"

for (let i = 0; i < items.length; i++) {
    . . .
}
placeholder += "</div>"
document.getElementById(elementId).innerHTML =
    placeholder;
```

Here, we have defined an array that will collect metadata about the to-do items HTML that we generate for each to-do item. This is under the `itemsMeta` variable and will be used later in the `renderItems` function to bind `processFunction` to each to-do item button using event listeners. Then, we define the HTML that is housing all our to-do items for a process under the `placeholder` variable. Here, we start with a `div` tag. Then, we loop through the items, converting the data from each item into HTML, and then finish off the HTML with a closing `div` tag. After that, we insert the constructed HTML string known as `placeholder` into `innerHTML`. Where `innerHTML` is on the page is where we want to see our constructed to-do items.

4. Inside the loop, we must construct the individual to-do item HTML with the following code:

```
let title = items[i]["title"];
let placeholderId = processType +
"-" + title.replaceAll(" ", "-");
```

More books: https://t.me/booksforits

```
placeholder += "<div>" + title +
"<button " + 'id="' + placeholderId + '">'
+ processType +
'</button>' + "</div>";
itemsMeta.push({"id": placeholderId, "title": title});
```

Here, we extract the title of the item from the item that we are looping through. Then, we define our ID for the item that we are going to use to bind to an event listener. Note that we replace all empty spaces with a `-`. Now that we have defined the title and ID, we add a `div` with a title to our `placeholder` HTML string. We also add a `button` with a `placeholderId` and then finish it off with a `div`. We can see that our addition to the HTML string is finished with a `;`. Then, we add `placeholderId` and `title` to the `itemsMeta` array to be used later.

5. Next, we loop `itemsMeta`, creating event listeners with the following code:

```
    . . .
    placeholder += "</div>"
    document.getElementById(elementId).innerHTML
    = placeholder;
    for (let i = 0; i < itemsMeta.length; i++) {
        document.getElementById(
            itemsMeta[i]["id"]).addEventListener(
            "click", processFunction);
    }
}
```

Now, `processFunction` will fire if a button that we created next to a to-do item is clicked. Our function now renders the items, but we need to get them from our backend with an API call function. We will look at this now.

Building an API call JavaScript function

Now that we have our render function, we can look at our API call function:

1. First, we must define our API call function in the `javascript/main.js` file. This function takes in a URL, which is the endpoint of the API call. It also takes a method, which is a string of either `POST`, `GET`, or `PUT`. Then, we must define our request object:

```
function apiCall(url, method) {
    let xhr = new XMLHttpRequest();
    xhr.withCredentials = true;
```

More books: https://t.me/booksforits

2. Then, we must define the event listener inside the `apiCall` function that renders the to-do items with the JSON returned once the call has finished:

```
xhr.addEventListener('readystatechange', function() {
    if (this.readyState === this.DONE) {
        renderItems(JSON.parse(
        this.responseText)["pending_items"],
        "edit", "pendingItems", editItem);
        renderItems(JSON.parse(this.responseText)
            ["done_items"],
        "delete", "doneItems", deleteItem);
    }
});
```

Here, we can see that we are passing in the IDs that we defined in the `templates/main.html` file. We also pass in the response from the API call. We can also see that we pass in the `editItem` function, meaning that we are going to fire an `edit` function when a button alongside a pending item is clicked, turning the item into a done item. Considering this, if a button belonging to a done item is clicked, the `deleteItem` function is fired. For now, we will continue building the `apiCall` function.

3. After this, we must build the `editItem` and `deleteItem` functions. We also know that every time the `apiCall` function is called, the items are rendered.

Now that we have defined the event listener, we must prep the API call object with the method and the URL, define the headers, and then return the request object for us to send it whenever we need:

```
    xhr.open(method, url);
    xhr.setRequestHeader('content-type',
        'application/json');
    xhr.setRequestHeader('user-token', 'token');
    return xhr
}
```

Now, we can use our `apiCall` function to perform a call to the backend of our application and re-render the frontend with the new state of the items after our API call. With this, we can move on to the final step, where we will define our functions that perform create, get, delete, and edit functions on our to-do items.

More books: https://t.me/booksforits

Building JavaScript functions for buttons

Note that the header is just hardcoding the accepted token that is hardcoded in the backend. We will cover how to properly define auth headers in *Chapter 7*, *Managing User Sessions*. Now that our API call function has been defined, we can move on to the `editItem` function:

```
function editItem() {
    let title = this.id.replaceAll("-", " ")
        .replace("edit ", "");
    let call = apiCall("/v1/item/edit", "POST");

    let json = {
        "title": title,
        "status": "DONE"
    };
    call.send(JSON.stringify(json));
}
```

Here, we can see that the HTML section that the event listener belongs to can be accessed via `this`. We know that if we remove the `edit` word, and switch `-` with a space, it will convert the ID of the to-do item into the title of the to-do item. Then, we utilize the `apiCall` function to define our endpoint and method. Note that we have a space in the `"edit "` string in the `replace` function. We have this space because we must remove the space after the edit string as well. If we do not remove that space, it will be sent to the backend, causing an error since our application backend will not have the space next to the title of the item in our JSON file. Once our endpoint and API call method have been defined, we pass the title into a dictionary with the status as done. This is because we know that we are switching the pending item to done. Once this is done, we send the API call with the JSON body.

Now, we can use the same approach for the `deleteItem` function:

```
function deleteItem() {
    let title = this.id.replaceAll("-", " ")
        .replace("delete ", "");
    let call = apiCall("/v1/item/delete", "POST");

    let json = {
        "title": title,
        "status": "DONE"
    };
```

More books: https://t.me/booksforits

```
    call.send(JSON.stringify(json));
}
```

Again, there is a space in the `"delete "` string in the `replace` function. With that, our rendering process is fully processed. We have defined edit and delete functions and a render function. Now, we must load the items when the page is initially loaded without having to click any buttons. This can be done with a simple API call:

```
function getItems() {
    let call = apiCall("/v1/item/get", 'GET');
    call.send()
}

getItems();
```

Here, we can see that we just make an API call with a `GET` method and send it. Also, note that our `getItems` function is being called outside of the function. This will be fired once when the view has loaded.

It has been a long stint of coding; however, we are nearly there. We only need to define the functionality of the create text input and button. We can manage this with a simple event listener and API call for the `create` endpoint:

```
document.getElementById("create-button")
        .addEventListener("click", createItem);

function createItem() {
    let title = document.getElementById("name");
    let call = apiCall("/v1/item/create/" +
        title.value, "POST");
    call.send();
    document.getElementById("name").value = null;
}
```

We have also added the detail of setting the text input value to `null`. We set `input` to `null` so that the user can input another item to be created without having to delete the old item title that was just created. Hitting the main view for the app gives us the following output:

More books: https://t.me/booksforits

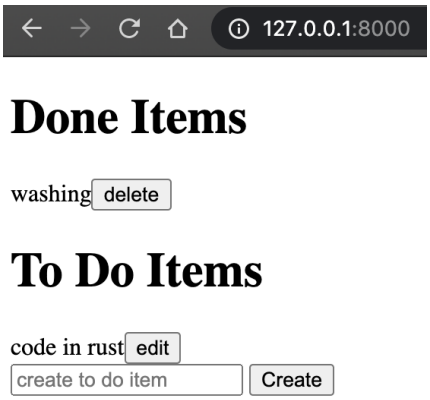


Figure 5.5 – Main page with rendered to-do items

Now, to see if our frontend works the way we want it to, we can perform the following steps:

1. Press the **delete** button next to the **washing** done item.
2. Type in `eat cereal for breakfast` and click **Create**.
3. Type in `eat ramen for breakfast` and click **Create**.
4. Click **edit** for the `eat ramen for breakfast` item.

These steps should yield the following result:

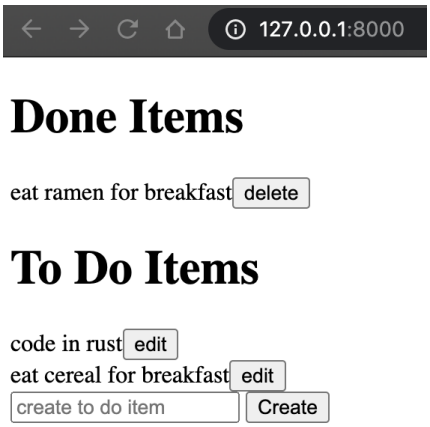


Figure 5.6 – Main page after completing the aforementioned steps

With that, we have a fully functioning web app. All the buttons work, and the lists are instantly updated. However, it does not look very pretty. There is no spacing, and everything is in black and white. To amend this, we need to integrate CSS into the HTML file, which we will do in the next section.

More books: https://t.me/booksforits

Injecting CSS into HTML

Injecting CSS takes the same approach as injecting JavaScript. We will have a CSS tag in the HTML file that will be replaced with the CSS from the file. To achieve this, we must carry out the following steps:

1. Add CSS tags to our HTML file.
2. Create a base CSS file for the whole app.
3. Create a CSS file for our main view.
4. Update our Rust crate to serve the CSS and JavaScript.

Let's have a closer look at this process.

Adding CSS tags to HTML

First, let's make some changes to our `templates/main.html` file:

```
 <style>
    {{BASE_CSS}}
    {{CSS}}
</style>
<body>
    <div class="mainContainer">
        <h1>Done Items</h1>
        <div id="doneItems"></div>
        <h1>To Do Items</h1>
        <div id="pendingItems"></div>
        <div class="inputContainer">
            <input type="text" id="name"
                   placeholder="create to do item">
            <div class="actionButton"
                 id="create-button"
                 value="Send">Create</div>
        </div>
    </div>
</body>
<script>
    {{JAVASCRIPT}}
</script>
```

More books: https://t.me/booksforits

Here, we can see that we have two CSS tags. The {{BASE_CSS}} tag is for base CSS, which will be consistent in multiple different views, such as the background color and column ratios, depending on the screen size. The {{BASE_CSS}} tag is for managing CSS classes for this view. Respectfully, the `css/base.css` and `css/main.css` files are made for our views. Also, note that we have put all the items in a `div` with a class called `mainContainer`. This will enable us to center all the items on the screen. We have also added some more classes so that the CSS can reference them, and changed the button for the create item from a `button` HTML tag to a `div` HTML tag. Once this is done, our `renderItems` function in the `javascript/main.js` file will have the following alteration for the loop of the items:

```
function renderItems(items, processType,
    elementId, processFunction) {
    . . .

    for (i = 0; i < items.length; i++) {
        . . .

        placeholder += '<div class="itemContainer">' +
            '<p>' + title + '</p>' +
            '<div class="actionButton" ' +
                  'id="' + placeholderId + '">'
            + processType + '</div>' + "</div>";
        itemsMeta.push({"id": placeholderId, "title":
        title});
    }
    . . .
}
```

With this considered, we can now define our base CSS in our `css/base.css` file.

Creating a base CSS

Now, we must define the style of the page and its components. A good place to start is by defining the body of the page in the `css/base.css` file. We can do a basic configuration of the body with the following code:

```
body {
    background-color: #92a8d1;
    font-family: Arial, Helvetica, sans-serif;
    height: 100vh;
}
```

More books: https://t.me/booksforits

The background color is a reference to a type of color. This reference might not seem like it makes sense just looking at it but there are color pickers online where you can see and pick a color and the reference code is supplied. Some code editors support this functionality but for some quick reference, simply Google `HTML color picker` and you will be spoilt for choice at the number of free online interactive tools that will be available. With the preceding configuration, the background for the entire page will have a code of `#92a8d1`, which is a navy-blue color. If we just had that, most of the page would have a white background. The navy-blue background would only be present where there is content. We set the height to `100vh`. `vh` is relative to 1% of the height of the viewport. With this, we can deduce that `100vh` means the styling we defined in the body occupies 100% of the viewport. Then, we define the font for all text unless overwritten to `Arial`, `Helvetica`, or `sans-serif`. We can see that we have defined multiple fonts in `font-family`. This does not mean that all of them are implemented or that there are different fonts for different levels of headers or HTML tags. Instead, this is a fallback mechanism. First, the browser will try and render `Arial`; if it is not supported by the browser, it will then try and render `Helvetica`, and if that fails too, it will try and render `sans-serif`.

With that, we have defined the general style for our body, but what about different screen sizes? For instance, if we were going to access our application on our phone, it should have different dimensions. We can see this in the following figure:

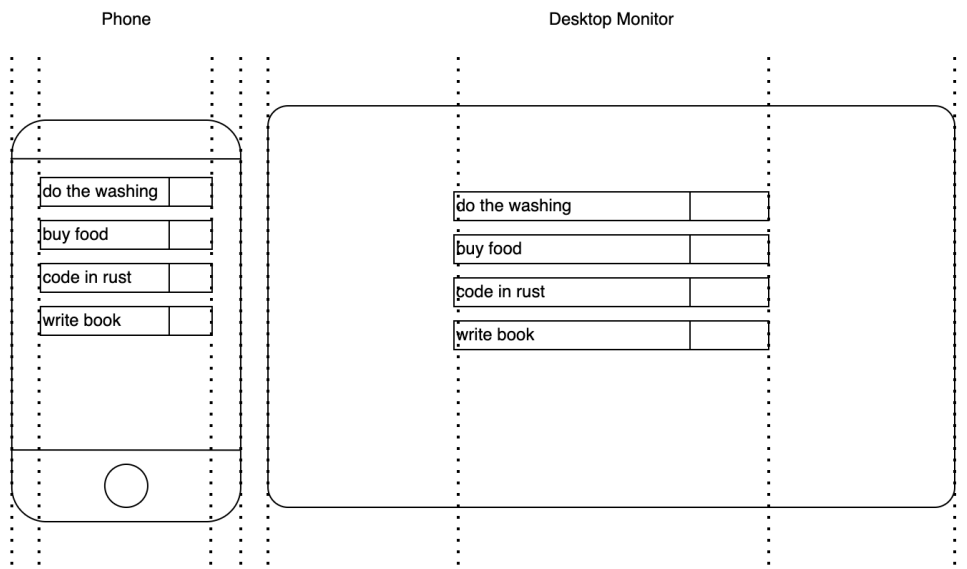


Figure 5.7 – Difference in margins between a phone and desktop monitor

Figure 5.7 shows the ratio of the margin to the space that is filled up by the to-do items list changes. With a phone, there is not much screen space, so most of the screen needs to be taken up by the to-do item; otherwise, we would not be able to read it. However, if we are using a widescreen desktop

More books: https://t.me/booksforits

monitor, we no longer need most of the screen for the to-do items. If the ratio was the same, the to-do items would be so stretched in the *X*-axis that it would be hard to read and frankly would not look good. This is where media queries come in. We can have different style conditions based on attributes such as the width and height of the window. We will start with the phone specification. So, if the width of the screen is up to 500 pixels, in our `css/base.css` file, we must define the following CSS configuration for our body:

```
@media(max-width: 500px) {
    body {
        padding: 1px;
        display: grid;
        grid-template-columns: 1fr;
    }
}
```

Here, we can see that the padding around the edge of the page and each element is just one pixel. We also have a grid display. This is where we can define columns and rows. However, we do not use it to its full extent. We just have one column. This means that our to-do items will take up most of the screen, as shown in the phone depiction in *Figure 5.7*. Even though we are not using a grid in this context, I have kept it in so that you can see the relationship this has to the other configurations for larger screens. If our screen gets a little bigger, we can split our page into three different vertical columns; however, the ratio of the width of the middle column to that of the columns on either side is 5:1. This is because our screen is still not very big, and we want our items to still take up most of the screen. We can adjust for this by adding another media query with different parameters:

```
@media(min-width: 501px) and (max-width: 550px) {
    body {
        padding: 1px;
        display: grid;
        grid-template-columns: 1fr 5fr 1fr;
    }
    .mainContainer {
        grid-column-start: 2;
    }
}
```

We can also see that, for our `mainContainer` CSS class where we house our to-do items, we will overwrite the `grid-column-start` attribute. If we don't do this, then `mainContainer` would be squeezed in the left margin at `1fr` width. Instead, we are starting and finishing in the middle at `5fr`. We can make `mainContainer` span across multiple columns with a `grid-column-finish` attribute.

More books: https://t.me/booksforits

If our screen gets larger, then we want to adjust the ratios even more as we do not want our items width to get out of control. To achieve this, we must define a 3 to 1 ratio for the middle column versus the 2 side columns, and then a 1 to 1 ratio when the screen width gets higher than `1001px`:

```
@media(min-width: 551px) and (max-width: 1000px) {
    body {
        padding: 1px;
        display: grid;
        grid-template-columns: 1fr 3fr 1fr;
    }
    .mainContainer {
        grid-column-start: 2;
    }
}
@media(min-width: 1001px) {
    body {
        padding: 1px;
        display: grid;
        grid-template-columns: 1fr 1fr 1fr;
    }
    .mainContainer {
        grid-column-start: 2;
    }
}
```

Now that we have defined our general CSS for all views, we can move on to our view-specific CSS in our `css/main.css` file.

Creating CSS for the home page

Now, we must break down our app components. We have a list of to-do items. Each item in the list will be a `div` that has a different background color:

```
.itemContainer {
    background: #034f84;
    margin: 0.3rem;
}
```

More books: https://t.me/booksforits

We can see that this class has a margin of 0.3. We are using `rem` because we want the margin to scale relative to the font size of the root element. We also want our item to slightly change color if our cursor hovers over it:

```
.itemContainer:hover {
    background: #034f99;
}
```

Inside an item container, the title of our item is denoted with a paragraph tag. We want to define the style of all the paragraphs in the item containers but not elsewhere. We can define the style of the paragraphs in the container using the following code:

```
.itemContainer p {
    color: white;
    display: inline-block;
    margin: 0.5rem;
    margin-right: 0.4rem;
    margin-left: 0.4rem;
}
```

`inline-block` allows the title to be displayed alongside `div`, which will be acting as the button for the item. The margin definitions merely stop the title from being right up against the edge of the item container. We also ensure that the paragraph color is white.

With our item title styled, the only item styling left is the action button, which is either `edit` or `delete`. This action button is going to float to the right with a different background color so that we know where to click. To do this, we must define our button style with a class, as outlined in the following code:

```
.actionButton {
    display: inline-block;
    float: right;
    background: #f7786b;
    border: none;
    padding: 0.5rem;
    padding-left: 2rem;
    padding-right: 2rem;
    color: white;
}
```

More books: https://t.me/booksforits

Here, we've defined the display, made it float to the right, and defined the background color and padding. With this, we can ensure the color changes on hover by running the following code:

```
.actionButton:hover {
    background: #f7686b;
    color: black;
}
```

Now that we have covered all the concepts, we must define the styles for the input container. This can be done by running the following code:

```
.inputContainer {
    background: #034f84;
    margin: 0.3rem;
    margin-top: 2rem;
}

.inputContainer input {
    display: inline-block;
    margin: 0.4rem;
}
```

We've done it! We have defined all the CSS, JavaScript, and HTML. Before we run the app, we need to load the data in the main view.

Serving CSS and JavaScript from Rust

We serve our CSS in the `views/app/items.rs` file. We do this by reading the HTML, JavaScript, base CSS, and main CSS files. Then, we replace our tags in the HTML data with the data from the other files:

```
pub async fn items() -> HttpResponse {
    let mut html_data = read_file(
        "./templates/main.html");
    let javascript_data: String = read_file(
        "./javascript/main.js");
    let css_data: String = read_file(
        "./css/main.css");
    let base_css_data: String = read_file(
        "./css/base.css");
```

More books: https://t.me/booksforits

```
    html_data = html_data.replace("{{JAVASCRIPT}}",
    &javascript_data);
    html_data = html_data.replace("{{CSS}}",
    &css_data);
    html_data = html_data.replace("{{BASE_CSS}}",
    &base_css_data);
    HttpResponse::Ok()
        .content_type("text/html; charset=utf-8")
        .body(html_data)
}
```

Now, when we spin up our server, we will have a fully running app with an intuitive frontend that will look like what's shown in the following screenshot:

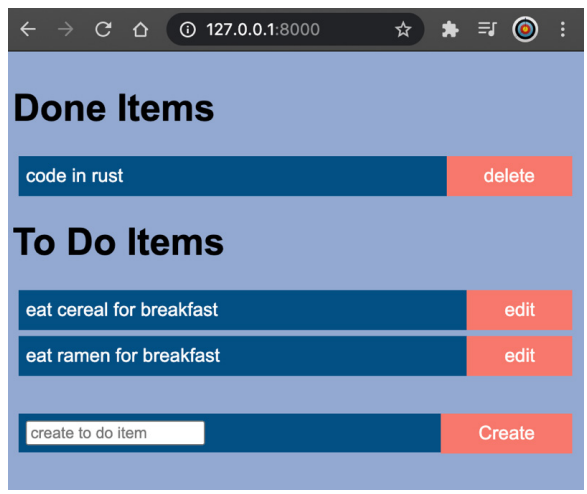


Figure 5.8 – Main page after CSS

Even though our app is functioning, and we have configured the base CSS and HTML, we may want to have reusable standalone HTML structures that have their own CSS. These structures can be injected into views as and when needed. What this does is gives us the ability to write a component once, and then import it into other HTML files. This, in turn, makes it easier to maintain and ensures consistency of the component in multiple views. For instance, if we create an information bar at the top of the view, we will want it to have the same styling in the rest of the views. Therefore, it makes sense to create an information bar once as a component and insert it into other views, as covered in the next section.

More books: https://t.me/booksforits

Inheriting components

Sometimes, we will want to build a component that can be injected into views. To do this, we are going to have to load both the CSS and HTML, and then insert them into the correct parts of the HTML.

To do this, we can create an `add_component` function that takes the name of the component, creates tags from the component name, and loads the HTML and CSS based on the component name. We will define this function in the `views/app/content_loader.rs` file:

```
pub fn add_component(component_tag: String,
    html_data: String) -> String {
    let css_tag: String = component_tag.to_uppercase() +
        "_CSS";
    let html_tag: String = component_tag.to_uppercase() +
        "_HTML";
    let css_path = String::from("./templates/components/")
        + &component_tag.to_lowercase() + ".css";
    let css_loaded = read_file(&css_path);

    let html_path = String::from("./templates/components/")
        + &component_tag.to_lowercase() + ".html";
    let html_loaded = read_file(&html_path);

    let html_data = html_data.replace(html_tag.as_str(),
        &html_loaded);
    let html_data = html_data.replace(css_tag.as_str(),
        &css_loaded);
    return html_data
}
```

Here, we use the `read_file` function that is defined in the same file. Then, we inject the component HTML and CSS into the view data. Note that we nested our components in a `templates/components/` directory. For this instance, we are inserting a `header` component, so our `add_component` function will try and load the `header.html` and `header.css` files when we pass the header into the `add_component` function. In our `templates/components/header.html` file, we must define the following HTML:

```
<div class="header">
    <p>complete tasks: </p><p id="completeNum"></p>
```

More books: https://t.me/booksforits

```
    <p>pending tasks: </p><p id="pendingNum"></p>
</div>
```

Here, we are merely displaying the counts for the number of completed and pending to-do items. In our `templates/components/header.css` file, we must define the following CSS:

```
.header {
    background: #034f84;
    margin-bottom: 0.3rem;
}
.header p {
    color: white;
    display: inline-block;
    margin: 0.5rem;
    margin-right: 0.4rem;
    margin-left: 0.4rem;
}
```

For our `add_component` function to insert our CSS and HTML into the right place, we must insert the `HEADER` tag into the `<style>` section of the `templates/main.html` file:

```
. . .
    <style>
        {{BASE_CSS}}
        {{CSS}}
        HEADER_CSS
    </style>
    <body>
        <div class="mainContainer">
            HEADER_HTML
            <h1>Done Items</h1>
. . .
```

Now that all of our HTML and CSS have been defined, we need to import the `add_component` function in our `views/app/items.rs` file:

```
use super::content_loader::add_component;
```

More books: https://t.me/booksforits

In the same file, we must add the header in the `items` view function, like so:

```
html_data = add_component(String::from("header"),
    html_data);
```

Now, we must alter the `apiCall` function in our `injecting_header/javascript/main.js` file to ensure that the header is updated with the to-do item counts:

```
document.getElementById("completeNum").innerHTML =
    JSON.parse(this.responseText)["done_item_count"];
document.getElementById("pendingNum").innerHTML =
    JSON.parse(this.responseText)["pending_item_count"];
```

Now that we have inserted our component, we get the following rendered view:

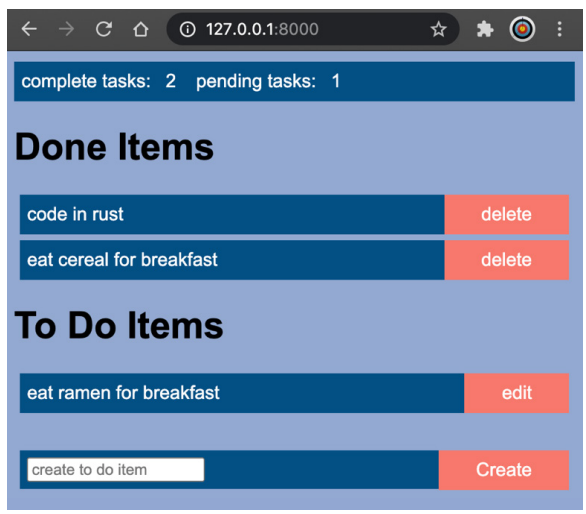


Figure 5.9 – Main page with header

As we can see, our header displays the data correctly. If we add the header tags to the view HTML file, and we call `add_component` in our view, we will get that header.

Right now, we have a fully working single-page application. However, this was not without difficulty. We can see that our frontend would start to spiral out of control if we started to add more features to the frontend. This is where frameworks such as React come in. With React, we can structure our code into proper components so that we can use them whenever we need to. In the next section, we will create a basic React application.

More books: https://t.me/booksforits

Creating a React app

React is a standalone application. Because of this, we will usually have our React application in its own GitHub repository. If you want to keep your Rust application and React application in the same GitHub repository, that is fine, but just make sure that they are in different directories in the root. Once we have navigated outside of the Rust web application, we can run the following command:

```
npx create-react-app front_end
```

This creates a React application in the `front_end` directory. If we look inside, we will see that there are a lot of files. Remember that this book is about web programming in Rust. Exploring everything about React is beyond the scope of this book. However, a book dedicated to React development is suggested in the *Further reading* section. For now, we will focus on the `front_end/package.json` file. Our `package.json` file is like our `Cargo.toml` file, where we define dependencies, scripts, and other metadata around the application that we are building. Inside our `package.json` file, we have the following scripts:

```
. . .
"scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
},
. . .
```

We can edit this if we want but as it stands, if we run the `npm start` command in the directory where our `package.json` file is, we will run the `react-scripts start` command. We will run our React application soon, but before this, we must edit our `front_end/src/App.js` file with the following code:

```
import React, { Component } from 'react';

class App extends Component {
  state = {
    "message": "To Do"
  }
  render() {
    return (
        <div className="App">
```

More books: https://t.me/booksforits

```
            <p>{this.state.message} application</p>
          </div>
      )
    }
}
export default App;
```

Before we break down this code, we must clarify something. If you go on the internet, you might come across articles stating that JavaScript is not a class-based object-oriented language. This book is not going to take a deep dive into JavaScript. Instead, this chapter is designed to give you enough knowledge to get a frontend up and running. Hopefully, this chapter is enough to facilitate further reading and kickstart your journey if you want to add a frontend to your Rust web application. For the sake of this chapter, we will just look at classes and objects that can support inheritance.

In the preceding code, we imported the `component` object from the `react` package. Then, we defined an `App` class that inherits the `component` class. The `App` class is the main part of our application, and we can treat the `front_end/src/App.js` file as our entry point for the frontend application. It is in the `App` class that we can define other routes if needed. We can also see that there is a `state` belonging to the `App` class. This is the overall memory of the application. We must call it `state`; every time the state is updated, the `render` function is executed, updating what the component renders to the frontend. This has abstracted a lot of what we were doing throughout the previous sections of this chapter when our state updated our homemade `render` function. We can see that our state can be referenced in the `render` function when returning. This is known as JSX, which allows us to write HTML elements directly in JavaScript without any extra methods. Now that the basic application has been defined, we can export it to make it available.

Let's navigate to the directory where the `package.json` file is placed and run the following command:

```
npm start
```

The React server will spin up and we will get the following view in our browser:

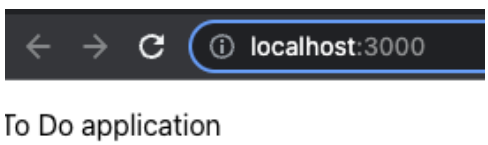


Figure 5.10 – First main view of our React application

Here, we can see that the message in our state has been passed into our `render` function and then displayed in our browser. Now that our React application is running, we can start loading data into our React application using API calls.

More books: https://t.me/booksforits

Making API calls in React

Now that the basic application is running, we can start performing an API call to our backend. For this, we will mainly focus on the `front_end/src/App.js` file. We can build up our application so that it can populate the frontend with items from our Rust application. First we must add the following to the dependencies of our `package.json` file:

```
"axios": "^0.26.1"
```

Then, we can run the following command:

```
npm install
```

This will install our extra dependency. Now, we can turn to our `front_end/src/App.js` file and import what we need with the following code:

```
import React, { Component } from 'react';
import axios from 'axios';
```

We are going to use `Component` for our `App` class inheritance and `axios` to perform API calls to our backend. Now, we can define our `App` class and update our state with the following code:

```
class App extends Component {

  state = {
      "pending_items": [],
      "done_items": [],
      "pending_items_count": 0,
      "done_items_count": 0
  }
}
export default App;
```

Here, we have the same structure as our homemade frontend. This is also the data that we return from our get items view in the Rust server. Now that we know what data we are going to work with, we can carry out the following steps:

1. Create a function inside our `App` class that gets the functions from the Rust server.
2. Ensure that this function is executed when the `App` class is mounted.
3. Create a function inside our `App` class that processes the items that are returned from the Rust server into HTML.

More books: https://t.me/booksforits

4. Create a function inside our App class that renders all the aforementioned components to the frontend once we are finished.
5. Enable our Rust server to receive calls from other sources.

Before we embark on these steps, we should note that the outline for our App class will take the following form:

```
class App extends Component {

  state = {
      . . .
  }
  // makes the API call
  getItems() {
      . . .
  }
  // ensures the API call is updated when mounted
  componentDidMount() {
      . . .
  }
  // convert items from API to HTML
  processItemValues(items) {
      . . .
  }
  // returns the HTML to be rendered
  render() {
    return (
        . . .
    )
  }
}
```

With this, we can start on the function that makes the API call:

1. Inside our App class, our getItems function takes the following layout:

```
  axios.get("http://127.0.0.1:8000/v1/item/get",
    {headers: {"token": "some_token"}})
    .then(response => {
```

More books: https://t.me/booksforits

```
        let pending_items = response.data["pending_items"]
        let done_items = response.data["done_items"]

        this.setState({
              . . .
          })
    });
```

Here, we define the URL. Then, we add our token to our header. For now, we will just hardcode a simple string because we are not setting up our user sessions in the Rust server yet; we will update this in *Chapter 7*, *Managing User Sessions*. Then, we close this. Because `axios.get` is a promise, we must use `.then`. The code inside the `.then` brackets is executed when the data is returned. Inside these brackets, we extract the data that we need and then execute the `this.setState` function. The `this.setState` function updates the state of the `App` class. However, executing `this.setState` also executes the `render` function of the `App` class, which will update the browser. Inside this `this.setState` function, we pass in the following code:

```
"pending_items": this.processItemValues(pending_items),
"done_items": this.processItemValues(done_items),
"pending_items_count": response.data["pending_item_
count"],
"done_items_count": response.data["done_item_count"]
```

With this, we have completed `getItems` and we can get items from the backend. Now that we have defined it, we must ensure that it gets executed, which we will do next.

2. Ensuring the `getItems` function is fired and thus the state is updated when the `App` class is loaded can be achieved with the following code:

```
componentDidMount() {
  this.getItems();
}
```

This is straightforward. `getItems` will execute immediately after our `App` component is mounted. We are essentially calling `this.setState` in the `componentDidMount` function. This triggers the extra rendering before the browser updates the screen. Even though `render` is called twice, the user will not see the intermediate state. This is one of the many functions that we inherit from the React `Component` class. Now that we load the data as soon as the page loads, we can move on to the next step: processing the loaded data.

More books: https://t.me/booksforits

3. For the `processItemValues` function inside our `App` class, we must take in an array of JSON objects that represent items and convert them into HTML, which can be achieved with the following code:

```
processItemValues(items) {
  let itemList = [];
  items.forEach((item, index)=>{
      itemList.push(
          <li key={index}>{item.title} {item.status}</li>
      )
  })
  return itemList
}
```

Here, we just loop through the items, converting them into `li` HTML elements and adding them to an empty array that is then returned once filled. Remember that we use the `processItemValue` function to process the data before it goes into the state in the `getItems` function. Now that we have all the HTML components in our state, we need to place them on the page with our `render` function.

4. For our `App` class, the `render` function only returns HTML components. We do not employ any extra logic in this. We can return the following:

```
<div className="App">
<h1>Done Items</h1>
<p>done item count: {this.state.done_items_count}</p>
{this.state.done_items}
<h1>Pending Items</h1>
<p>pending item count:
    {this.state.pending_items_count}</p>
{this.state.pending_items}
</div>
```

Here, we can see that our state is directly referenced. This is a lovely change from the manual string manipulation that we employed earlier in this chapter. Using React is a lot cleaner, reducing the risk of errors. On our frontend, the render process with the calls to the backend should work. However, our Rust server will block requests from the React application because it is from a different application. To fix this, we need to move on to the next step.

5. Right now, our Rust server will block our requests to the server. This is down to **Cross-Origin Resource Sharing** (**CORS**). We have not had any problems with this before because, by default, CORS allows requests from the same origin. When we were writing raw HTML and serving it from our Rust server, the requests were coming from the same origin. However, with the React

More books: https://t.me/booksforits

application, the requests are coming from a different origin. To rectify this, we need to install CORS as a dependency in our `Cargo.toml` file with the following code:

```
actix-cors = "0.6.1"
```

In our `src/main.rs` file, we must import CORS with the following code:

```
use actix_cors::Cors;
```

Now, we must define CORS policy before the definition of the server and wrap the CORS policy right after the views configuration with the following code:

```
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        let cors = Cors::default().allow_any_origin()
                                  .allow_any_method()
                                  .allow_any_header();
        let app = App::new()
            .wrap_fn(|req, srv|{
                println!("{}-{}", req.method(),
                          req.uri());
                let future = srv.call(req);
                async {
                    let result = future.await?;
                    Ok(result)
                }
        }).configure(views::views_factory).wrap(cors);
        return app
    })
    .bind("127.0.0.1:8000")?
    .run()
    .await
}
```

With this, our server is ready to accept requests from our React application.

Note

When we defined our CORS policy, we were expressive in that we wanted to allow all methods, headers, and origins. However, we can be more concise with the following CORS definition:

```
let cors = Cors::permissive();
```

More books: https://t.me/booksforits

Now, we can test our application to see if it is working. We can do this by running our Rust server with Cargo and running our React application in a different terminal. Once this is up and running, our React application should look like this when loaded:

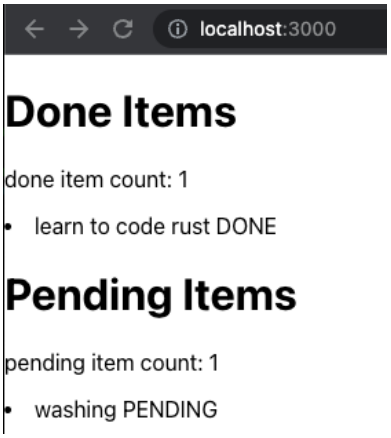


Figure 5.11 – The view of our React application when it first talks to our Rust server

With this, we can see that the call to our Rust application is now working as expected. However, all we are doing is listing the names and statuses of the to-do items. Where React shines is in building custom components. This means that we can build individual classes that have their own states and functions for each to-do item. We'll see this in the next section.

Creating custom components in React

When we look at our `App` class, we can see that it is useful to have a class that has a state and functions that can be utilized to manage how and when HTML is rendered to the browser. When it comes to individual to-do items, we could use a state and functions. This is because we have a button that gets attributes from the to-do item and calls the Rust server to either edit or delete it. In this section, we are going to build two components: a `ToDoItem` component in the `src/components/ToDoItem.js` file and a `CreateToDoItem` component in the `src/components/CreateToDoItem.js` file. Once we have built these, we can plug them into our `App` component as our `App` component will get the items' data and loop through these items, creating multiple `ToDoItem` components. There are a few steps that we need to process to achieve this, so this section will be split into the following subsections:

- Creating our `ToDoItem` component
- Creating our `CreateToDoItem` component
- Constructing and managing custom components in our `App` component

Let's get started.

More books: https://t.me/booksforits

Creating our ToDoItem component

We will start with the simpler `ToDoItem` component in the `src/components/ToDoItem.js` file. First, we must import the following:

```
import React, { Component } from 'react';
import axios from "axios";
```

This is nothing new. Now that we have imported what we need, we can focus on how we define `ToDoItem` with the following code:

```
class ToDoItem extends Component {
    state = {
        "title": this.props.title,
        "status": this.props.status,
        "button": this.processStatus(this.props.status)
    }
    processStatus(status) {
        . . .
    }
    inverseStatus(status) {
        . . .
    }
    sendRequest = () => {
        . . .
    }
    render() {
        return(
            . . .
        )
    }
}
export default ToDoItem;
```

Here, we populate the state with `this.props`, which is the parameter that's passed into the component when the component is constructed. Then, we have the following functions for our `ToDoItem` component:

- `processStatus`: This function converts the status of the to-do item, such as `PENDING`, into the message on the button, such as `edit`.

More books: https://t.me/booksforits

- `inverseStatus`: When we have a to-do item with a status of `PENDING` and we edit it, we want to convert it into a status of `DONE` so that it can be sent to the `edit` endpoint on the Rust server, which is the inverse. Therefore, this function creates an inverse of the status that's passed in.
- `sendRequest`: This function sends the request to the Rust server for either editing or deleting the to-do item. We can also see that our `sendRequest` function is an arrow function. The arrow syntax essentially binds the function to the component so that we can reference it in our `render` return statement, allowing the `sendRequest` function to be executed when a button that is bound to it is clicked.

Now that we know what our functions are supposed to do, we can define our status functions with the following code:

```
processStatus(status) {
    if (status === "PENDING") {
        return "edit"
    } else {
        return "delete"
    }
}
inverseStatus(status) {
    if (status === "PENDING") {
        return "DONE"
    } else {
        return "PENDING"
    }
}
```

This is straightforward and does not need much explanation. Now that our status processing functions are done, we can define our `sendRequest` function with the following code:

```
sendRequest = () => {
    axios.post("http://127.0.0.1:8000/v1/item/" +
                this.state.button,
        {
            "title": this.state.title,
            "status": this.inverseStatus(this.state.status)
        },
    {headers: {"token": "some_token"}})
```

More books: https://t.me/booksforits

```
        .then(response => {
            this.props.passBackResponse(response);
        });
}
```

Here, we use `this.state.button` to define part of the URL as the endpoint changes, depending on the button that we are pressing. We can also see that we execute the `this.props.passBackResponse` function. This is a function that we pass into the `ToDoItem` component. This is because we get the full state of our to-do items from the Rust server back after the edit or delete request. We will need to enable our `App` component to process the data that has been passed back. Here, we are getting a sneak peek of what we are going to be doing in the *Constructing and managing custom components in our App component* subsection. Our `App` component will have an unexecuted function under the `passBackResponse` parameter that it will pass through to our `ToDoItem` component. This function, under the `passBackResponse` parameter, will process the new to-do item's state and render it in the `App` component.

With that, we have configured all our functions. All that is left is to define the return of the `render` function, which takes the following form:

```
<div>
    <p>{this.state.title}</p>
    <button onClick={this.sendRequest}>
                    {this.state.button}</button>
</div>
```

Here, we can see that the title of the to-do item is rendered in the paragraph tag and that our button executes the `sendRequest` function when clicked. We have now finished this component and it is ready to be displayed in our application. However, before we do this, we need to build the component that creates to-do items in the next section.

Creating custom components in React

Our React application is functioning in terms of listing, editing, and deleting to-do items. However, we are not able to create any to-do items. This consists of an input and a `create` button so that we can put in a to-do item that we can then create by clicking the button. In our `src/components/CreateToDoItem.js` file, we need to import the following:

```
import React, { Component } from 'react';
import axios from "axios";
```

More books: https://t.me/booksforits

These are the standard imports to build our components. Once the imports have been defined, our `CreateToDoItem` component takes the following form:

```
class CreateToDoItem extends Component {

    state = {
        title: ""
    }
    createItem = () => {
        . . .
    }
    handleTitleChange = (e) => {
        . . .
    }
    render() {
        return (
            . . .
        )
    }
}
export default CreateToDoItem;
```

In the preceding code, we can see that our `CreateToDoItem` component has the following functions:

- `createItem`: This function sends a request to the Rust server to create the to-do item with the title in the state
- `handleTitleChange`: This function updates the state every time the input is updated

Before we explore these two functions, we will flip around the order in which we code these functions, and define the return of the `render` function with the following code:

```
<div className="inputContainer">
    <input type="text" id="name"
           placeholder="create to do item"
           value={this.state.title}
           onChange={this.handleTitleChange}/>
    <div className="actionButton"
         id="create-button"
```

More books: https://t.me/booksforits

```
          onClick={this.createItem}>Create</div>
</div>
```

Here, we can see that the value of the input is `this.state.title`. Also, when the input changes, we execute the `this.handleTitleChange` function. Now that we have covered the `render` function, there is nothing new to introduce. This is a good opportunity for you to look at the outline of our `CreateToDoItem` component again and try to define the `createItem` and `handleTitleChange` functions yourself. They take a similar form to the functions in the `ToDoItem` component.

Your attempt to define the `createItem` and `handleTitleChange` functions should look similar to the following:

```
createItem = () => {
    axios.post("http://127.0.0.1:8000/v1/item/create/" +
        this.state.title,
        {},
        {headers: {"token": "some_token"}})
        .then(response => {
            this.setState({"title": ""});
            this.props.passBackResponse(response);
        });
}

handleTitleChange = (e) => {
    this.setState({"title": e.target.value});
}
```

With that, we have defined both of our custom components. We are now ready to move on to the next subsection, where we will manage our custom components.

Constructing and managing custom components in our App component

While it was fun creating our custom components, they are not much use if we do not use them in our application. In this subsection, we will add some additional code to the `src/App.js` file to enable our custom components to be used. First, we must import our components with the following code:

```
import ToDoItem from "./components/ToDoItem";
import CreateToDoItem from "./components/CreateToDoItem";
```

More books: https://t.me/booksforits

Now that we have our components, we can move on to our first alteration. Our `App` component's `processItemValues` function can be defined with the following code:

```
processItemValues(items) {
  let itemList = [];
  items.forEach((item, _)=>{
      itemList.push(
          <ToDoItem key={item.title + item.status}
                    title={item.title}
                    status={item.status.status}
                    passBackResponse={
                    this.handleReturnedState}/>
      )
  })
  return itemList
}
```

Here, we can see that we loop through the data that we get from the Rust server but instead of passing the data into a generic HTML tag, we pass the parameters of the to-do item data into our own custom component, which is treated like an HTML tag. When it comes to handling our own response with the returned state, we can see that it is an arrow function that processes the data and sets the state with the following code:

```
handleReturnedState = (response) => {
  let pending_items = response.data["pending_items"]
  let done_items = response.data["done_items"]
  this.setState({
      "pending_items":
       this.processItemValues(pending_items),
      "done_items": this.processItemValues(done_items),
      "pending_items_count":
       response.data["pending_item_count"],
      "done_items_count": response.data["done_item_count"]
  })
}
```

This is very similar to our `getItems` function. You could do some refactoring here if you wanted to reduce the amount of duplicated code. However, to make this work, we must define the return statement for the `render` function with the following code:

More books: https://t.me/booksforits

```
<div className="App">
    <h1>Pending Items</h1>
    <p>done item count:
    {this.state.pending_items_count}</p>
    {this.state.pending_items}
    <h1>Done Items</h1>
    <p>done item count: {this.state.done_items_count}</p>
    {this.state.done_items}
    <CreateToDoItem
     passBackResponse={this.handleReturnedState} />
</div>
```

Here, we can see that there is not much change apart from adding the `createItem` component. Running our Rust server and our React application will give us the following view:

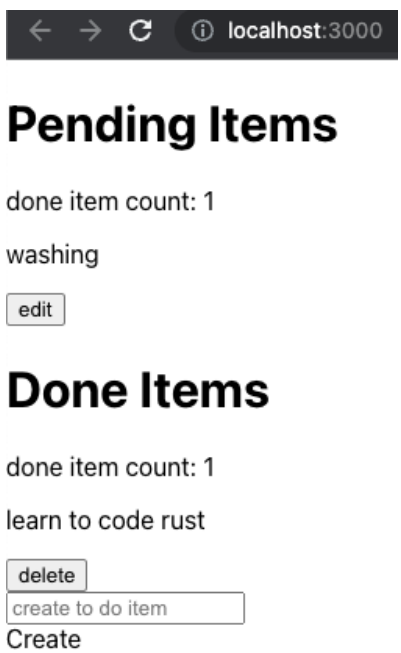


Figure 5.12 – The view of our React application with custom components

Figure 5.12 shows that our custom components are rendering. We can click on the buttons and, as a result, we will see that all our API calls work and our custom components work as they should. Now, all that is standing in our way is making our frontend look presentable, which we can do by lifting our CSS into the React application.

More books: https://t.me/booksforits

Lifting CSS into React

We are now on the final stretch of making our React application usable. We could split up our CSS into multiple different files. However, we are coming to the end of this chapter, and going over all the CSS again would unnecessarily fill up this chapter with loads of duplicate code. While our HTML and JavaScript are different, the CSS is the same. To get it running, we can copy all the CSS from the following files:

- `templates/components/header.css`
- `css/base.css`
- `css/main.css`

Copy the CSS files listed here into the `front_end/src/App.css` file. There is one change to the CSS, and this is where all the `.body` references should be replaced with `.App`, as shown in the following code snippet:

```
.App {
  background-color: #92a8d1;
  font-family: Arial, Helvetica, sans-serif;
  height: 100vh;
}
@media(min-width: 501px) and (max-width: 550px) {
  .App {
    padding: 1px;
    display: grid;
    grid-template-columns: 1fr 5fr 1fr;
  }
  .mainContainer {
    grid-column-start: 2;
  }
}
. . .
```

Now, we can import our CSS and use it in our app and components. We will also have to alter the return HTML in the `render` functions. We can work through all three files. For the `src/App.js` file, we must import the CSS with the following code:

```
import "./App.css";
```

More books: https://t.me/booksforits

Then, we must add a header and define our `div` tags with the correct classes with the following code for the return statement from our `render` function:

```
<div className="App">
    <div className="mainContainer">
        <div className="header">
            <p>complete tasks:
            {this.state.done_items_count}</p>
            <p>pending tasks:
            {this.state.pending_items_count}</p>
        </div>
        <h1>Pending Items</h1>
        {this.state.pending_items}
        <h1>Done Items</h1>
        {this.state.done_items}
        <CreateToDoItem passBackResponse=
       {this.handleReturnedState}/>
    </div>
</div>
```

In our `src/components/ToDoItem.js` file, we must import the CSS with the following code:

```
import "../App.css";
```

Then, we must change our `button` into a `div` and define the return statement for our `render` function with the following code:

```
<div className="itemContainer">
    <p>{this.state.title}</p>
    <div className="actionButton" onClick=
    {this.sendRequest}>
    {this.state.button}</div>
</div>
```

In our `src/components/CreateToDoItem.js` file, we must import the CSS with the following code:

```
import "../App.css";
```

More books: https://t.me/booksforits

Then, we must change our `button` into a `div` and define the return statement for our `render` function with the following code:

```
<div className="inputContainer">
    <input type="text" id="name"
           placeholder="create to do item"
           value={this.state.title}
           onChange={this.handleTitleChange}/>
    <div className="actionButton"
         id="create-button"
         onClick={this.createItem}>Create</div>
</div>
```

With this, we have lifted our CSS from our Rust web server into our React application. If we run the Rust server and React application, we'll get the output shown in the following figure:

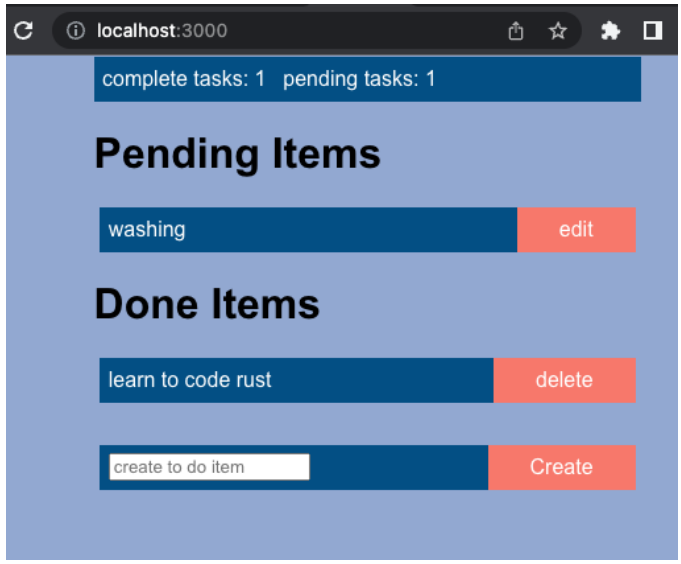


Figure 5.13 – The view of our React application with CSS added

And there we have it! Our React application is working. Getting our React application up and running takes more time, but we can see that we have more flexibility with React. We can also see that our React application is less error-prone because we are not having to manually manipulate strings. There is also one more advantage for us to build in React, and that is the existing infrastructure. In the next and final section, we will convert our React application into a compiled desktop application that runs in the applications of the computer by wrapping our React application in **Electron**.

More books: https://t.me/booksforits

Converting our React application into a desktop application

Converting our React application into a desktop application is not complex. We are going to use the Electron framework to do so. Electron is a powerful framework that converts our JavaScript, HTML, and CSS application into a desktop application that is compiled across platforms for macOS, Linux, and Windows. The Electron framework can also give us access to the computer's components via an API such as encrypted storage, notifications, power monitor, message ports, processes, shell, system preferences, and much more. Desktop applications such as Slack, Visual Studio Code, Twitch, Microsoft Teams, and many more are built into Electron. To convert our React application, we must start by updating the `package.json` file. First, we must update our metadata at the top of our `package.json` file with the following code:

```
{
  "name": "front_end",
  "version": "0.1.0",
  "private": true,
  "homepage": "./",
  "main": "public/electron.js",
  "description": "GUI Desktop Application for a simple To
                  Do App",
  "author": "Maxwell Flitton",
  "build": {
    "appId": "Packt"
  },
  "dependencies": {
    . . .
```

Most of this is general metadata. However, the `main` field is essential. This is where we will write our file that defines how the Electron application will run. Setting the `homepage` field to `"./"` also ensures that the asset paths are relative to the `index.html` file. Now that our metadata has been defined, we can add the following dependencies:

```
"webpack": "4.28.3",
"cross-env": "^7.0.3",
"electron-is-dev": "^2.0.0"
```

More books: https://t.me/booksforits

These dependencies help with building the Electron application. Once they have been added, we can redefine our scripts with the following code:

```
    . . .
"scripts": {
    "react-start": "react-scripts start",
    "react-build": "react-scripts build",
    "react-test": "react-scripts test",
    "react-eject": "react-scripts eject",
    "electron-build": "electron-builder",
    "build": "npm run react-build && npm run electron-
              build",
    "start": "concurrently \"cross-env BROWSER=none npm run
              react-start\" \"wait-on http://localhost:3000
              && electron .\""
},
```

Here, we have prefixed all our React scripts with `react`. This is to separate the React processes from our Electron processes. If we just want to run our React application in dev mode now, we must run the following command:

```
npm run react-start
```

We have also defined build commands and dev start commands for Electron. These will not work yet because we have not defined our Electron file. At the bottom of our `package.json` file, we must define our developer dependencies for building our Electron application:

```
    . . .
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  },
  "devDependencies": {
    "concurrently": "^7.1.0",
    "electron": "^18.0.1",
    "electron-builder": "^22.14.13",
    "wait-on": "^6.0.1"
```

More books: https://t.me/booksforits

```
  }
}
```

With that, we have defined all that we need in the `package.json` file. We need to install the new dependencies with the following command:

```
npm install
```

Now, we can start building our `front_end/public/electron.js` file so that we can build our Electron file. This is essentially boilerplate code, and you will probably see this file in other tutorials as this is the minimum to get an application running in Electron. First, we must import what we need with the following code:

```
const { app, BrowserWindow } = require("electron");
const path = require("path");
const isDev = require("electron-is-dev");
```

Then, we must define the function that creates our Desktop window with the following code:

```
function createWindow() {
    const mainWindow = new BrowserWindow({
        width: 800,
        height: 600,
        webPreferences: {
            nodeIntegration: true,
            enableRemoteModule: true,
            contextIsolation: false,
        },
    });
    mainWindow.loadURL(
        isDev
           ? "http://localhost:3000"
           : `file://${path.join(__dirname,
                                 "../build/index.html")}`
    );
    if (isDev) {
        mainWindow.webContents.openDevTools();
    }
}
```

More books: https://t.me/booksforits

Here, we essentially define the width and height of the window. Also, note that `nodeIntegration` and `enableRemoteModule` enable the renderer remote process (browser window) to run code on the main process. Then, we start loading the URL in the main window. If the run is in developer mode, we merely load `http://localhost:3000` as we have the React application running on localhost. If we build our application, then the assets and files that we coded get compiled and can be loaded through the `../build/index.html` file. We also state that if we are running in developer mode, we open the developer tools. We must execute the `createWindow` function when the window is ready with the following code:

```
app.whenReady().then(() => {
    createWindow();
    app.on("activate", function () {
        if (BrowserWindow.getAllWindows().length === 0){
           createWindow();
        }
    });
});
```

If the operating system is macOS, we must keep the program running, even if we close the window:

```
app.on("window-all-closed", function () {
    if (process.platform !== "darwin") app.quit();
});
```

Now, we must run the following command:

```
npm start
```

This runs the Electron application, giving us the following output:

More books: https://t.me/booksforits

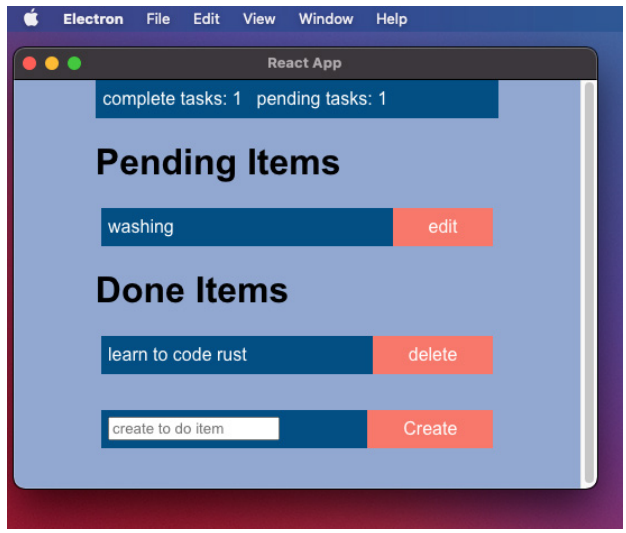


Figure 5.14 – Our React application running in Electron

In *Figure 5.13*, we can see that our application is running in a window on our desktop. We can also see that our application is accessible with the menu bar at the top of my screen. The application's logo is showing on my taskbar:

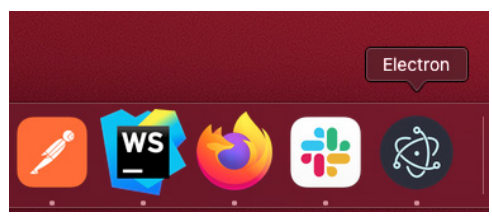


Figure 5.15 – Electron on my taskbar

The following command will compile our application in the `dist` folder, which, if clicked, will install the application onto your computer:

```
npm build
```

The following is an example of this in the applications area on my Mac when I used Electron to test out a GUI for the open source package I built called Camel for *OasisLMF*:

More books: https://t.me/booksforits

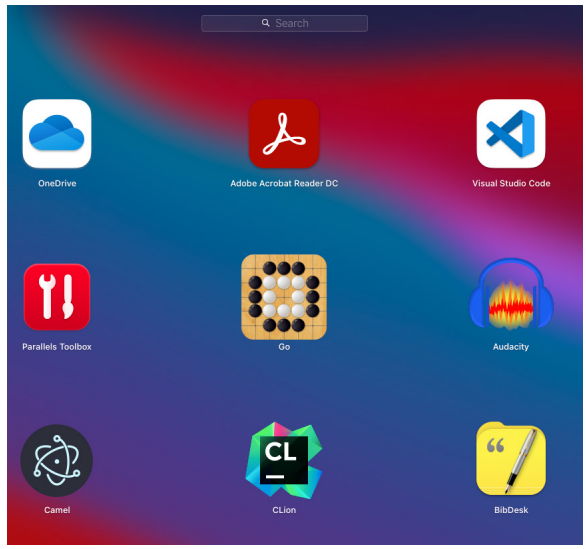


Figure 5.16 – Our Electron app in the applications area

Eventually, I will come up with a logo. However, this concludes this chapter on displaying content in the browser.

Summary

In this chapter, we have finally enabled our application to be used by a casual user as opposed to having to rely on a third-party application such as Postman. We defined our own app views module that housed read file and insert functions. This resulted in us building a process that loaded an HTML file, inserted data from JavaScript and CSS files into the view data, and then served that data.

This gave us a dynamic view that automatically updated when we edited, deleted, or created a to-do item. We also explored some basics around CSS and JavaScript to make API calls from the frontend and dynamically edit the HTML of certain sections of our view. We also managed the styling of the whole view based on the size of the window. Note that we did not rely on external crates. This is because we want to be able to understand how we can process our HTML data.

Then, we rebuilt the frontend in React. While this took longer and there were more moving parts, the code was more scalable and safer since we didn't have to manually manipulate strings to write HTML components. We can also see why we leaned into React as it fits nicely into Electron, giving us another way of delivering our application to our users.

While our app now works at face value, it is not scalable in terms of data storage. We do not have data filter processes. There are no checks on the data that we store, and we do not have multiple tables.

In the next chapter, we will build data models that interact with a **PostgreSQL** database that runs locally in Docker.

More books: https://t.me/booksforits

Questions

1. What is the simplest way to return HTML data to the user's browser?
2. What is the simplest (not scalable) way to return HTML, CSS, and JavaScript data to the user's browser?
3. How do we ensure that the background color and style standard of certain elements is consistent across all views of the app?
4. How do we update the HTML after an API call?
5. How do we enable a button to connect to our backend API?

Answers

1. We can serve HTML data by merely defining a string of HTML and putting it in the body of an `HttpResponse` struct while defining the content type as HTML. The `HttpResponse` struct is then returned to the user's browser.
2. The simplest way is to hardcode a full HTML string with the CSS hardcoded in the `<style>` section, and our JavaScript hardcoded in the `<script>` section. This string is then put in the body of an `HttpResponse` struct and returned to the user's browser.
3. We make a CSS file that defines the components that we want to be consistent throughout the app. Then, we put a tag in the `<style>` section of all our HTML files. Then, with each file, we load the base CSS file and replace the tag with the CSS data.
4. After the API call, we must wait for the status to be ready. Then, we get the HTML section we want to update using `getElementById`, serialize the response data, and then set the inner HTML of the element as the response data.
5. We give the button a unique ID. Then, we add an event listener, which is defined by the unique ID. In this event listener, we bind it to a function that gets the ID using `this`. In this function, we make an API call to the backend and then use the response to update the HTML of other parts of our view that display the data.

Further reading

To learn more about the topics that were covered in this chapter, take a look at the following resources:

- *React and React Native: A complete hands-on guide to modern web and mobile development with React.js*, by Adam Boduch and Roy Derks, Packt Publishing
- *Mastering React Test-Driven Development: Build rock-solid, well-tested web apps with React, Redux, and GraphQL*, by Daniel Irvine, Packt Publishing
- *Responsive Web Design with HTML5 and CSS: Develop future-proof responsive websites using the latest HTML5 and CSS techniques*, by Ben Frain, Packt Publishing
- Electron documentation: `https://www.electronjs.org/`

More books: https://t.me/booksforits

More books: https://t.me/booksforits

Part 3: Data Persistence

Now that our application handles HTTP requests and displays the content in the browser, we need to store and process data properly. In this part, you will learn how to manage databases locally in development with Docker and how to perform database migrations with SQL scripts. You will also learn how to map the database schema to the Rust application, querying and updating the database from Rust. After this part, you will be able to manage database connection pools, data models, and migrations; log in and authenticate requests with middleware; and cache data in the frontend, exploring RESTful concepts.

This part includes the following chapters:

- *Chapter 6*, *Data Persistence with PostgreSQL*
- *Chapter 7*, *Managing User Sessions*
- *Chapter 8*, *Building RESTful Services*

More books: https://t.me/booksforits

More books: https://t.me/booksforits

6
Data Persistence with PostgreSQL

By this point in the book, the frontend for our application has been defined, and our app is working at face value. However, we know that our app is reading and writing from a **JSON** file.

In this chapter, we will get rid of our JSON file and introduce a **PostgreSQL** database to store our data. We will do this by setting up a database development environment using Docker. We will also investigate how to monitor the Docker database container. We will then create migrations to build the schema for our database and then build data models in **Rust** to interact with the database. We will then refactor our app so that the `create`, `edit`, and `delete` endpoints interact with the database instead of the JSON file.

In this chapter, we will cover the following topics:

- Building our PostgreSQL database
- Connecting to PostgreSQL with Diesel
- Connecting our application to PostgreSQL
- Configuring our application
- Managing a database connection pool

By the end of this chapter, you will be able to manage an application that performs reading, writing, and deleting data in a PostgreSQL database with data models. If we make changes to the data models, we will be able to manage them with migrations. Once this is done, you will be able to optimize your database connections with pools and get the server to reject the HTTP request before it hits the view if the database connection cannot be made.

More books: https://t.me/booksforits

Technical requirements

In this chapter, we will be using **Docker** to define, run a PostgreSQL database, and run it. This will enable our app to interact with a database on our local machine. Docker can be installed by following the instructions at `https://docs.docker.com/engine/install/`.

We will also be using `docker-compose` on top of Docker to orchestrate our Docker containers. This can be installed by following the instructions at `https://docs.docker.com/compose/install/`.

The code files for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter06`.

Building our PostgreSQL database

Up to this point in the book, we have been using a JSON file to store our to-do items. This has served us well so far. In fact, there is no reason why we cannot use a JSON file throughout the rest of the book to complete the tasks. However, if you use a JSON file for production projects, you will come across some downsides.

Why we should use a proper database

If the reads and writes to our JSON file increase, we can face some concurrency issues and data corruption. There is also no checking on the type of data. Therefore, another developer can write a function that writes different data to the JSON file, and nothing will stand in the way.

There is also an issue with migrations. If we want to add a timestamp to the to-do items, this will only affect new to-do items that we insert into the JSON file. Therefore, some of our to-do items will have a timestamp, and others won't, which will introduce bugs into our app. Our JSON file also has limitations in terms of filtering.

Right now, all we do is read the whole data file, alter an item in the whole dataset, and write the whole dataset to the **JSON** file. This is not effective and will not scale well. It also inhibits us from linking these to-do items to another data model-like user. Plus, we can only search right now using the status. If we used a SQL database with a user table linked to a to-do item database, we would be able to filter to-do items based on the user, status, or title. We can even use a combination thereof. When it comes to running our database, we are going to use Docker. So why should we use Docker?

Why use Docker?

To understand why we would use Docker, we first need to understand what Docker is. Docker essentially has containers that work like virtual machines but in a more specific and granular way. Docker containers isolate a single application and all the application's dependencies. The application is run inside the Docker container. Docker containers can then communicate with each other. Because

More books: https://t.me/booksforits

Docker containers share a single common **operating system** (**OS**), they are compartmentalized from one another and from the OS at large, meaning that Docker applications use less memory compared to virtual machines. Because of Docker containers, we can be more portable with our applications. If the Docker container runs on your machine, it will run on another machine that also has Docker. We can also package our applications, meaning that extra packages specifically required for our application to run do not need to be installed separately, including dependencies on the OS level. As a result, Docker gives us great flexibility in web development as we can simulate servers and databases on our local machine.

How to use Docker to run a database

With all this in mind, it makes sense to go through the extra steps necessary to set up and run a SQL database. To do this, we are going to use Docker: a tool that helps us create and use containers. Containers themselves are Linux technology that package and isolate applications along with their entire runtime environment. Containers are technically isolated file systems, but to help visualize what we are doing in this chapter, you can think of them as mini lightweight virtual machines. These containers are made from images that can be downloaded from **Docker Hub**. We can insert our own code into these images before spinning up a container out of them, as seen in the following diagram:

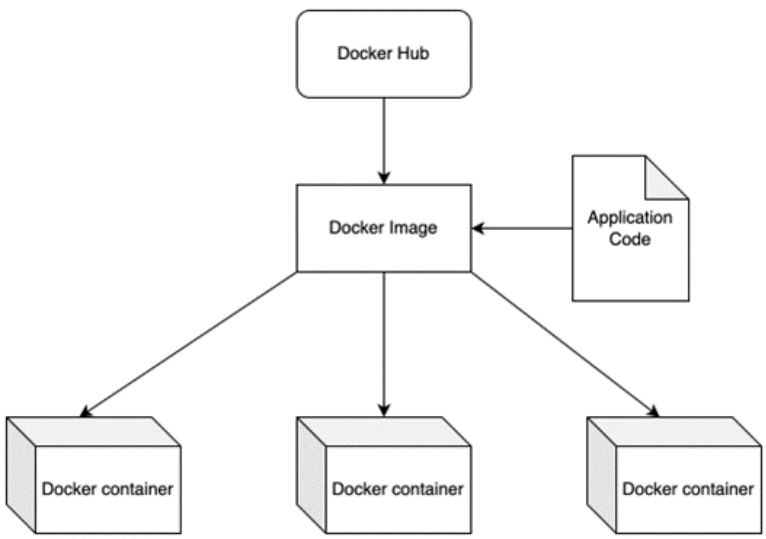


Figure 6.1 – Relationship between Docker images and containers

With Docker, we can download an image, such as a PostgreSQL database, and run it in our development environment. Because of Docker, we can spin up multiple databases and apps and then shut them down as and when we need them. First, we need to take stock of our containers by running the following command in the terminal:

```
docker container ls -a
```

More books: https://t.me/booksforits

If Docker is a fresh install, we get the following output:

```
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
```

As we can see, we have no containers. We also need to take stock of our images. This can be done by running the following terminal command:

```
docker image ls
```

The preceding command gives the following output:

```
REPOSITORY  TAG  IMAGE ID  CREATED  SIZE
```

Again, if Docker is a fresh install, then there will be no containers.

There are other ways in which we can create a database in Docker. For instance, we can create our own **Dockerfile**, where we define our OS and configurations. However, we have `docker-compose` installed. Using `docker-compose` will make the database definition straightforward. It will also enable us to add more containers and services. To define our PostgreSQL database, we code the following **YAML** code in a `docker-compose.yml` file in the root directory:

```
version: "3.7"

services:

  postgres:
    container_name: 'to-do-postgres'
    image: 'postgres:11.2'
    restart: always
    ports:
      - '5432:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'
      - 'POSTGRES_PASSWORD=password'
```

In the preceding code, at the top of the file, we have defined the version. Older versions, such as *2* or *1*, have different styles in which the file is laid out. The different versions also support different arguments. At the time of writing this book, version *3* is the latest version. The following URL covers the changes between each `docker-compose` version: `https://docs.docker.com/compose/compose-file/compose-versioning/`.

More books: https://t.me/booksforits

We then define our database service that is nested under the `postgres` tag. Tags such as `postgres` and `services` denote dictionaries, and lists are defined with `-` for each element. If we were to convert our `docker-compose` file to JSON, it would have the following structure:

```
{
  "version": "3.7",
  "services": {
    "postgres": {
      "container_name": "to-do-postgres",
      "image": "postgres:11.2",
      "restart": "always",
      "ports": [
        "5432:5432"
      ],
      "environment": [
        "POSTGRES_USER=username",
        "POSTGRES_DB=to_do",
        "POSTGRES_PASSWORD=password"
      ]
    }
  }
}
```

In the preceding code, we can see that our services are a dictionary of dictionaries denoting each service. Thus, we can deduce that we cannot have two tags with the same name, as we cannot have two dictionary keys that are the same. The previous code also tells us that we can keep stacking on service tags with their own parameters.

Running a database in Docker

With our database service, we have a name, so when we look at our containers, we know what each container is doing in relation to the service, such as a server or database. In terms of configuring the database and building it, we luckily pull the official `postgres` image. This image has everything configured for us, and Docker will pull it from the repository. The image is like a blueprint. We can spin up multiple containers with their own parameters from that one image that we pulled. We then define the restart policy as always. This means that containers' restart policies will trigger when the containers exit. We can also define it to only restart based on a failure or stopping.

More books: https://t.me/booksforits

It should be noted that Docker containers have their own ports that are not open to the machine. However, we can expose container ports and map the exposed port to an internal port inside the Docker container. Considering these features, we can define our ports.

However, in our example, we will keep our definition simple. We will state that we accept incoming traffic to the Docker container on port `5432` and route it through to the internal port `5432`. We will then define our environment variables, which are the username, the name of the database, and the password. While we are using generic, easy-to-remember passwords and usernames for this book, it is advised that you switch to more secure passwords and usernames if pushing to production. We can build a spin up for our system by navigating to the root directory where our `docker-compose` file is by running the following command:

```
docker-compose up
```

The preceding command will pull down the `postgres` image from the repository and start constructing the database. After a flurry of log messages, the terminal should come to rest with the following output:

```
LOG:  listening on IPv4 address "0.0.0.0", port 5432
LOG:  listening on IPv6 address "::", port 5432
LOG:  listening on Unix socket "/var/run/
postgresql/.s.PGSQL.5432"
LOG:  database system was shut down at 2022-04-23 17:36:45 UTC
LOG:  database system is ready to accept connections
```

As you can see, the date and time will vary. However, what we are told here is that our database is ready to accept connections. Yes, it is really that easy. Therefore, Docker adoption is unstoppable. Clicking *Ctrl* + *C* will stop our `docker-compose`; thus, shutting down our `postgres` container.

We now list all our containers with the following command:

```
docker container ls -a
```

The preceding command gives us the following output:

```
CONTAINER ID        IMAGE               COMMAND
c99f3528690f        postgres:11.2       "docker-entrypoint.s…"

CREATED             STATUS                          PORTS
4 hours ago         Exited (0) About a minute ago

NAMES
to-do-postgres
```

More books: https://t.me/booksforits

In the preceding output, we can see that all the parameters are there. The ports, however, are empty because we stopped our service.

Exploring routing and ports in Docker

If we were to start our service again and list our containers in another terminal, port `5432` would be under the `PORTS` tag. We must keep note of the `CONTAINER ID` reference to the Docker container as it will be unique and different/random for each container. We will need to reference these if we're accessing logs. When we run `docker-compose up`, we essentially use the following structure:

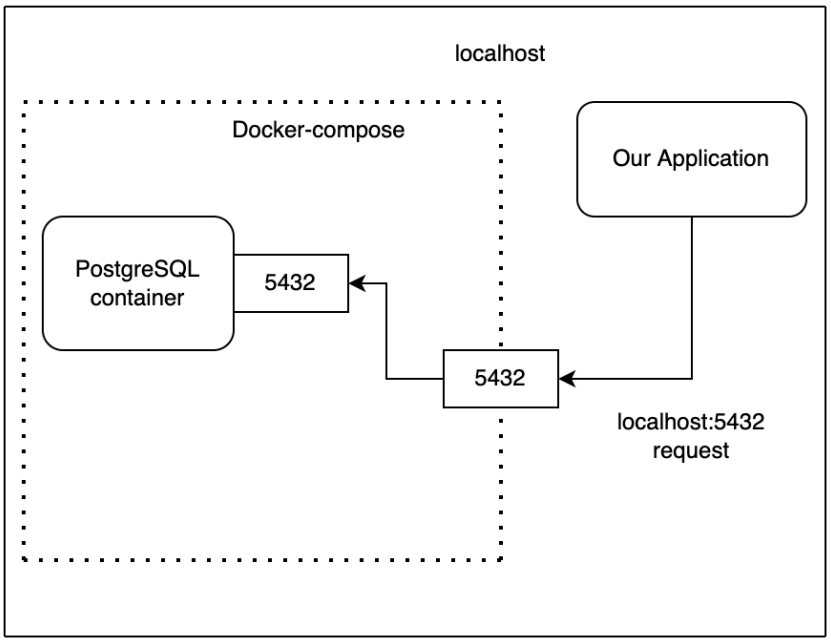


Figure 6.2 – docker-compose serving our database

In *Figure 6.2*, we can see that our `docker-compose` uses a unique project name to keep containers and networks in their namespace. It must be noted that our containers are running on the localhost. Therefore, if we want to make a call to a container managed by `docker-compose`, we will have to make a localhost request. However, we must make the call to the port that is open from `docker-compose`, and `docker-compose` will route it to the port that is defined in the `docker-compose.yml` file. For instance, we have two databases with the following `yml` file:

```
version: "3.7"

services:
```

More books: https://t.me/booksforits

```
  postgres:
    container_name: 'to-do-postgres'
    image: 'postgres:11.2'
    restart: always
    ports:
      - '5432:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'
      - 'POSTGRES_PASSWORD=password'
  postgres_two:
    container_name: 'to-do-postgres_two'
    image: 'postgres:11.2'
    restart: always
    ports:
      - '5433:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'
      - 'POSTGRES_PASSWORD=password'
```

In the preceding code, we can see that both of our databases accept traffic into their containers through port `5432`. However, there would be a clash, so one of the ports that we open with is port `5433`, which is routed to port `5432` in the second database container, which gives us the following layout:

More books: https://t.me/booksforits

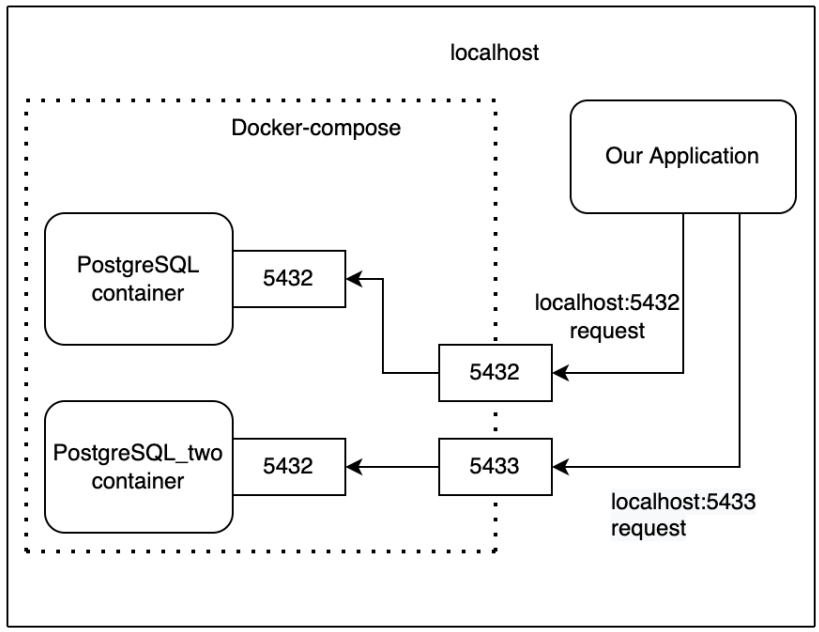


Figure 6.3 – docker-compose serving multiple databases

This routing gives us flexibility when running multiple containers. We are not going to run multiple databases for our to-do application, so we should delete our `postgres_two` service. Once we have deleted our `postgres_two` service, we can run `docker-compose` again and then list our containers with the following command:

```
docker image ls
```

The preceding command will now give us the following output:

```
REPOSITORY          TAG                 IMAGE ID
postgres            11.2                3eda284d1840

CREATED             SIZE
17 months ago       312MB
```

In the preceding output, we can see that our image has been pulled from the `postgres` repository. We also have a unique/random ID for the image and a date for when that image was created.

Now that we have a basic understanding of how to get our database up and running, we can run `docker-compose` in the background with the following command:

```
docker-compose up -d
```

More books: https://t.me/booksforits

The preceding command just tells us which containers have been spun up with the following output:

```
Starting to-do-postgres ... done
```

We can see our status when we list our containers with the following output:

```
STATUS              PORTS                     NAMES
Up About a minute   0.0.0.0:5432->5432/tcp   to-do-postgres
```

In the previous output, the other tags are the same, but we can also see that the `STATUS` tag tells us how long the container has been running, and which port it is occupying. Although `docker-compose` is running in the background, it does not mean we cannot see what is going on. We can access the logs of the container anytime by calling the `logs` command and referencing the ID of the container using the following command:

```
docker logs c99f3528690f
```

The preceding command should give the same output as our standard `docker-compose up` command. To stop `docker-compose`, we can run the `stop` command, shown as follows:

```
docker-compose stop
```

The preceding command will stop our containers in our `docker-compose`. It must be noted that this is different from the `down` command, shown as follows:

```
docker-compose down
```

The `down` command will also stop our containers. However, the `down` command will delete the container. If our database container is deleted, we will also lose all our data.

There is a configuration parameter called `volumes` that can prevent the deletion of our data when the container is removed; however, this is not essential for local development on our computers. In fact, you will be wanting to delete containers and images from your laptop regularly. *I did a purge on my laptop once of containers and images that I was no longer using, and this freed up 23GB!*

Docker containers on our local development machines should be treated as temporary. While Docker containers are more lightweight than standard virtual machines, they are not free. The idea behind Docker running on our local machines is that we can simulate what running our application would be like on a server. If it runs in Docker on our laptop, we can be certain that it will also run on our server, especially if the server is being managed by a production-ready Docker orchestration tool such as **Kubernetes**.

More books: https://t.me/booksforits

Running Docker in the background with Bash scripts

Docker can also help with consistent testing and development. We will want to be able to have the same results every time we run a test. We will also want to onboard other developers easily and enable them to tear down and spin up containers that will support development quickly and easily. I have personally seen development delayed when not supporting easy teardown and spin-up procedures. For instance, when working on a complex application, the code that we are adding and testing out might scar the database. Reverting back might not be possible, and deleting the database and starting again would be a pain, as reconstructing this data might take a long time. The developer may not even remember how they constructed the data in the first place. There are multiple ways to prevent this from happening and we will cover these in *Chapter 9*, *Testing Our Application Endpoints and Components*.

For now, we will build a script that spins up our database in the background, waits until the connection to our database is ready, and then tears down our database. This will give us the foundations to build pipelines, tests, and onboarding packages to start development. To do this, we will create a directory in the root directory of our Rust web application called `scripts`. We can then create a `scripts/wait_for_database.sh` file housing the following code:

```
#!/bin/bash

cd ..
docker-compose up -d

until pg_isready -h localhost -p 5432 -U username
do
  echo "Waiting for postgres"
  sleep 2;
done

echo "docker is now running"
docker-compose down
```

Using the preceding code, we move the current working directory of the script out of the `scripts` directory and into our root directory. We then start `docker-compose` in the background. Next, we loop, pinging port `5432` utilizing the `pq_isready` command to wait until our database is ready to accept connections.

More books: https://t.me/booksforits

Important note

The `pg_isready` Bash command might not be available on your computer. The `pg_isready` command usually comes with the installation of the PostgreSQL client. Alternatively, you can use the following Docker command instead of `pg_isready`:

```
until docker run -it postgres --add-host host.docker.internal:host-gateway docker.io/postgres:14-alpine -h localhost -U username pg_isready
```

What is happening here is that we are using the `postgres` Docker image to run our database check to ensure that our database is ready to accept connections.

Once our database is running, we print out to the console that our database is running and then tear down our `docker-compose`, destroying the database container. Running the command that runs the `wait_for_database.sh` Bash script will give the following output:

```
❯ sh wait_for_database.sh
[+] Running 0/0
 ⠿ Network web_app_default  Creating        0.2s
 ⠿ Container to-do-postgres      Started    1.5s
localhost:5432 - no response
Waiting for postgres
localhost:5432 - no response
Waiting for postgres
localhost:5432 - accepting connections
docker is now running
[+] Running 1/1
 ⠿ Container to-do-postgres  Removed        1.2s
 ⠿ Network web_app_default   Removed
```

From the preceding output, considering that we tell our loop to sleep for 2 seconds at every iteration of the loop, we can deduce that it took roughly 4 seconds for our newly spun-up database to accept connections. Thus, we can say that we have achieved basic competency in managing local databases with Docker.

In this section, we set up our environment. We also sufficiently understood the basics of Docker to build, monitor, shut down, and delete our database with just a few simple commands. Now, we can move on to the next section, where we'll interact with our database with Rust and the `diesel` crate.

More books: https://t.me/booksforits

Connecting to PostgreSQL with Diesel

Now that our database is running, we will build a connection to this database in this section. To do this, we will be using the `diesel` crate. The `diesel` crate enables us to get our Rust code to connect to databases. We are using the `diesel` crate instead of other crates because the `diesel` crate is the most established, having a lot of support and documentation. To do this, let us go through the following steps:

1. First, we will utilize the `diesel` crate. To do this, we can add the following dependencies in our `cargo.toml` file:

```
diesel = { version = "1.4.8", features = ["postgres",
                                           "chrono",
                                           "r2d2"] }
dotenv = "0.15.0"
chrono = "0.4.19"
```

   In the preceding code, we have included a `postgres` feature in our `diesel` crate. The `diesel` crate definition also has the `chrono` and `r2d2` features. The `chrono` feature enables our Rust code to utilize datetime structs. The `r2d2` feature enables us to perform connection pooling. We will cover connection pooling at the end of the chapter. We have also included the `dotenv` crate. This crate enables us to define variables in a `.env` file, which will then be passed through into our program. We will use this to pass in the database credentials and then into processes.

2. We now need to install the `diesel` client to run migrations to the database through our terminal as opposed to our app. We can do this with the following command:

```
cargo install diesel_cli --no-default-features
    --features postgres
```

3. We now need to define the environment `DATABASE_URL` URL. This will enable our client commands to connect to the database with the following command:

```
echo DATABASE_URL=postgres://username:password@localhost/
to_do
    > .env
```

   In the preceding URL, our username is denoted as username, and our password is denoted as password. Our database is running on our own computer, which is denoted as `localhost`, and our database is called `to_do`. This creates a `.env` file in the root file outputting the following contents:

```
DATABASE_URL=postgres://username:password@localhost/to_do
```

More books: https://t.me/booksforits

4. Now that our variables are defined, we can start to set up our database. We need to spin up our database container with `docker-compose` with our `docker-compose up` command. We then set up our database with the following command:

```
diesel setup
```

The preceding command then creates a `migrations` directory in the root with the following structure:

```
── migrations
│   └── 00000000000000_diesel_initial_setup
│       ├── down.sql
│       └── up.sql
```

The `up.sql` file is fired when the migration is upgraded, and the `down.sql` file is fired when the migration is downgraded.

5. Now, we need to create our migration to create our to-do items. This can be done by commanding our client to generate the migration with the following command:

```
diesel migration generate create_to_do_items
```

Once this is run, we should get the following printout in the console:

```
Creating migrations/2022-04-23-201747_create_to_do_items/
up.sql
Creating migrations/2022-04-23-201747_create_to_do_items/
down.sql
```

The preceding command gives us the following file structure in our migrations:

```
├── migrations
│   ├── 00000000000000_diesel_initial_setup
│   │   ├── down.sql
│   │   └── up.sql
│   └── 2022-04-23-201747_create_to_do_items
│       ├── down.sql
│       └── up.sql
```

At face value, it might seem unfortunate that with the `diesel` crate, we will have to create our own SQL files. However, this is forcing good practice. While it is easier to allow the crate to automatically write the SQL, this couples our application with the database. For instance, once I was working on refactoring a microservices system. The problem I had was that I was using a Python package to manage all my migrations for the database. However, I wanted to change the code of the server. You will not be surprised to hear that I was going to switch the server from Python to Rust. However, because the migrations were autogenerated by the Python library,

More books: https://t.me/booksforits

I had to construct helper Docker containers that, to this day, spin up when a new release is done and then perform a copy of the schema of the database to the Rust application when it is being built. This is messy. This also justified to me why we had to write all our SQL manually when I was an R&D software engineer in financial tech.

Databases are separate from our applications. Because of this, we should keep them isolated, so we do not see the writing of SQL manually as a hindrance. Embrace it as you are learning a good skill that will save you headaches in the future. We must remember not to force one tool into everything, it has to be the right tool for the right job. In our `create to-do items` migrations folder, we define our `to_do` table with the following SQL entries in our `up.sql` file:

```
CREATE TABLE to_do (
  id SERIAL PRIMARY KEY,
  title VARCHAR NOT NULL,
  status VARCHAR NOT NULL,
  date timestamp NOT NULL DEFAULT NOW()
)
```

In the preceding code, we have an `id` of the item that will be unique. We then have `title` and `status`. We have also added a `date` field that, by default, has the current time the to-do item is inserted into the table. These fields are wrapped in a `CREATE TABLE` command. In our `down.sql` file, we need to drop the table if we are downgrading the migration with the following SQL command:

```
DROP TABLE to_do
```

Now that we have written the SQL code for our `up.sql` and `down.sql` files, we can describe what our code does with the following diagram:

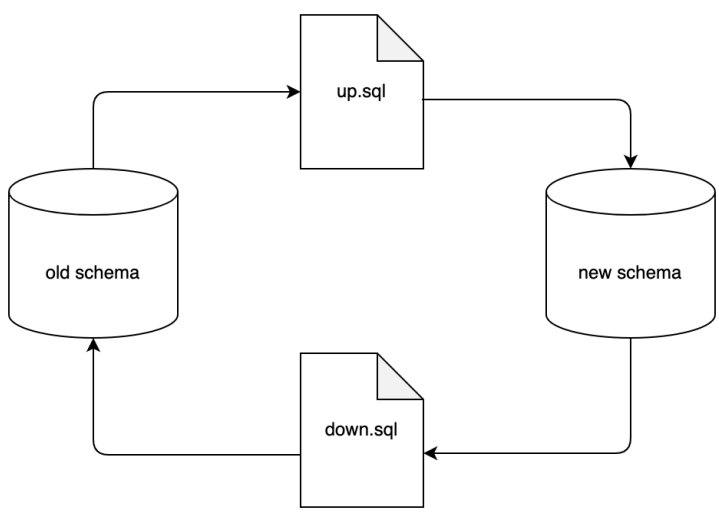


Figure 6.4 – The effect of the up.sql and down.sql scripts

More books: https://t.me/booksforits

6. Now that our migration is ready, we can run it with the following terminal command:

```
diesel migration run
```

The preceding command runs the migration creating the `to_do` table. Sometimes, we might introduce a different field type in the SQL. To rectify this, we can change SQL in our `up.sql` and `down.sql` files and run the following `redo` terminal command:

```
diesel migration redo
```

The preceding command will run the `down.sql` file and then run the `up.sql` file.

7. Next, we can run commands in our database Docker container to inspect that our database has the `to_do` table with the right fields that we defined. We can do this by running commands directly on our database Docker container. We can enter the container under the `username` username, while pointing to the `to_do` database using the following terminal command:

```
docker exec -it 5fdeda6cfe43 psql -U username to_do
```

It must be noted that, in the preceding command, my container ID is `5fdeda6cfe43`, but your container ID will be different. If you do not input the right container ID, you will not be running the right database commands on the right database. After running this command, we get a shell interface with the following prompt:

```
to_do=#
```

After the preceding prompt, when we type in `\c`, we will be connected to the database. This will usually be denoted with a statement saying that we are now connected to the `to_do` database and as the `username` user. Finally, typing `\d` will list the relations, giving the following table in the terminal:

```
Schema |             Name              |   Type   |  Owner
--------+-------------------------------+----------+--------
--
 public | __diesel_schema_migrations | table    |
username
 public | to_do                         | table    |
username
 public | to_do_id_seq                  | sequence |
username
```

From the preceding table, we can see that there is a migrations table to keep track of the migration version of the database.

8. We also have our `to_do` table and the sequence for the `to_do` item IDs. To inspect the schema, all we need to do is type in `\d+ to_do`, which gives us the following schema in the terminal:

```
Table "public.to_do"
```

More books: https://t.me/booksforits

```
 Column |            Type             | Collation |
--------+-----------------------------+-----------+
 id     | integer                     |           |
 title  | character varying           |           |
 status | character varying           |           |
 date   | timestamp without time zone |           |

| Nullable |              Default               |
Storage  |
+----------+------------------------------------+---------
-+
| not null | nextval('to_do_id_seq'::regclass) |
plain    |
| not null |                                    | extended
|
| not null |                                    | extended
|
| not null | now()                              |
plain    |
Indexes:
    "to_do_pkey" PRIMARY KEY, btree (id)
```

In the preceding table, we can see that our schema is exactly how we designed it. We get a little more information in the `date` column clarifying that we do not get the time the to-do item was created. Seeing as our migrations have worked, we should explore how migrations show up in the database in the next step.

9. We can inspect our migrations table by running the following SQL command:

```
SELECT * FROM  __diesel_schema_migrations;
```

The preceding command gives us the following table as output:

```
    version     |           run_on
----------------+----------------------------
 00000000000000 | 2022-04-25 23:10:07.774008
 20220424171018 | 2022-04-25 23:29:55.167845
```

As you can see, these migrations can be useful for debugging as sometimes we can forget to run a migration after updating a data model. To explore this further, we can revert our last migration with the following command outside the Docker container:

```
diesel migration revert
```

More books: https://t.me/booksforits

We then get the following printout informing us that the rollback has been performed:

```
Rolling back migration 2022-04-24-171018_create_to_do_
items
```

Now that our migration has been rolled back, our migrations table will look like the following printout:

```
    version     |           run_on
----------------+----------------------------
 00000000000000 | 2022-04-25 23:10:07.774008
```

In the preceding printout, we can see that our last migration was removed. Therefore, we can deduce that the migrations table is not a log. It only keeps track of migrations that are currently active. The tracking of the migrations will aid the `diesel` client when running a migration to run the right migration.

In this section, we have used the `diesel` client to connect to our database in a Docker container. We then defined the database URL in an environment file. Next, we initialized some migrations and created a table in our database. Even better, we directly connected with the Docker container, where we could run a range of commands to explore our database. Now that our database is fully interactive via our client in the terminal, we can start building our `to-do` item database models so that our Rust app can interact with our database.

Connecting our app to PostgreSQL

In the preceding section, we managed to connect to the PostgreSQL database using the terminal. However, we now need our app to manage the reading and writing to the database for our to-do items. In this section, we will connect our application to the database running in Docker. To connect, we must build a function that establishes a connection and then returns it. It must be stressed that there is a better way to manage the database connection and configuration, which we will cover at the end of the chapter. For now, we will implement the simplest database connection to get our application running. In the `src/database.rs` file, we define the function with the following code:

```
use diesel::prelude::*;
use diesel::pg::PgConnection;
use dotenv::dotenv;
use std::env;

pub fn establish_connection() -> PgConnection {
    dotenv().ok();

    let database_url = env::var("DATABASE_URL")
```

More books: https://t.me/booksforits

```
        .expect("DATABASE_URL must be set");
    PgConnection::establish(&database_url)
        .unwrap_or_else(
        |_| panic!("Error connecting to {}",
        database_url))
}
```

In the preceding code, first of all, you might notice the `use diesel::prelude::*;` import command. This command imports a range of connection, expression, query, serialization, and result structs. Once the required imports are done, we define the `connection` function. First of all, we need to ensure that the program will not throw an error if we fail to load the environment using the `dotenv().();` command.

Once this is done, we get our database URL from the environment variables and establish a connection using a reference to our database URL. We then `unwrap` the result, and we might panic displaying the database URL if we do not manage to do this, as we want to ensure that we are using the right URL with the right parameters. As the connection is the final statement in the function, this is what is returned.

Now that we have our own connection, we need to define the schema. This will map the variables of the `to-do` items to the data types. We can define our schema in the `src/schema.rs` file with the following code:

```
table! {
    to_do (id) {
        id -> Int4,
        title -> Varchar,
        status -> Varchar,
        date -> Timestamp,
    }
}
```

In the preceding code, we are using the `diesel` macro, `table!`, which specifies that a table exists. This map is straightforward, and we will use this schema in the future to reference columns and the table in database queries and inserts.

Now that we have built our database connection and defined a schema, we must declare them in our `src/main.rs` file with the following imports:

```
#[macro_use] extern crate diesel;
extern crate dotenv;

use actix_web::{App, HttpServer};
```

More books: https://t.me/booksforits

```
use actix_service::Service;
use actix_cors::Cors;

mod schema;
mod database;
mod views;
mod to_do;
mod state;
mod processes;
mod json_serialization;
mod jwt;
```

In the preceding code, our first import also enables procedural macros. If we do not use the `#[macro_use]` tag, then we will not be able to reference our schema in our other files. Our schema definition would also would not be able to use the table macro. We also import the `dotenv` crate. We keep the modules that we created in *Chapter 5*, *Displaying Content in the Browser*. We also define our schema and database modules. After doing this, we have all we need to start building our data models.

Creating our data models

We will use our data models to define parameters and behavior around the data from the database in Rust. They essentially act as a bridge between the database and the Rust app, as depicted in the following diagram:

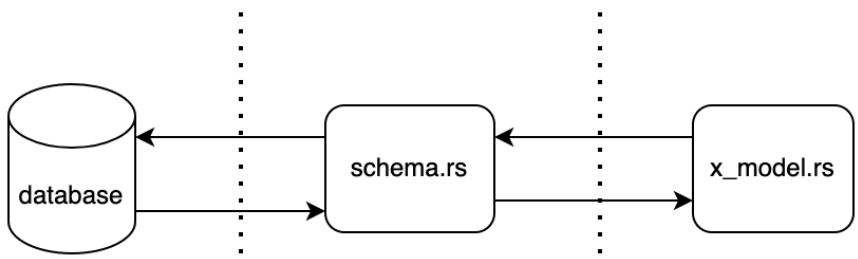


Figure 6.5 – The relationship between models, schemas, and databases

In this section, we will define the data models for to-do items. However, we need to enable our app to add more data models if needed. To do this, we carry out the following steps:

1. We define a new to-do item data model struct.
2. Then, we define a constructor function for the new to-do item structs.
3. And lastly, we define a to-do item data model struct.

More books: https://t.me/booksforits

Before we start writing any code, we define the following file structure in the `src` directory as follows:

```
├── models
|   ├── item
|   |   ├── item.rs
|   |   ├── mod.rs
|   |   └── new_item.rs
|   └── mod.rs
```

In the preceding structure, each data model has a directory in the `models` directory. Inside that directory, we have two files that define the model. One for a new insert and another for managing the data around the database. The new insert data model does not have an ID field.

There is no ID field because the database will assign an ID to the item; we do not define it before. However, when we interact with items in the database, we will get their ID, and we may want to filter by ID. Therefore, the existing data item model houses an ID field. We can define our new item data model in the `new_item.rs` file with the following code:

```
use crate::schema::to_do;
use chrono::{NaiveDateTime, Utc};

#[derive(Insertable)]
#[table_name="to_do"]
pub struct NewItem {
    pub title: String,
    pub status: String,
    pub date: NaiveDateTime
}
impl NewItem {
    pub fn new(title: String) -> NewItem {
        let now = Utc::now().naive_local();
        return NewItem{
            title,
            status: String::from("PENDING"),
            date: now
        }
    }
}
```

More books: https://t.me/booksforits

As you can see in the preceding code, we import our table definition because we are going to reference it. We then define our new item with `title` and `status`, which are to be strings. The `chrono` crate is used to define our `date` field as `NaiveDateTime`. We then use a `diesel` macro to define the table to belong to this struct at the `"to_do"` table. Do not be fooled by the fact that this definition uses quotation marks.

If we do not import our schema, the app will not compile because it will not understand the reference. We also add another `diesel` macro stating that we allow the data to be inserted into the database with the `Insertable` tag. As covered before, we are not going to add any more tags to this macro because we only want this struct to insert data.

We have also added a `new` function to enable us to define standard rules around creating a new struct. For instance, we are only going to be creating new items that are pending. This reduces the risk of a rogue status being created. If we want to expand later, the `new` function could accept a `status` input and run it through a match statement, throwing an error if the status is not one of the statuses that we are willing to accept. We also automatically state that the `date` field is the date that we create the item.

With this in mind, we can define our item data model in the `item.rs` file with the following code:

```
use crate::schema::to_do;
use chrono::NaiveDateTime;

#[derive(Queryable, Identifiable)]
#[table_name="to_do"]
pub struct Item {
    pub id: i32,
    pub title: String,
    pub status: String,
    pub date: NaiveDateTime
}
```

As you can see in the preceding code, the only difference between the `NewItem` and `Item` struct is that we do not have a constructor function; we have swapped the `Insertable` tag for `Queryable` and `Identifiable`, and we have added an `id` field to the struct. To make these available to the rest of the application, we define them in the `models/item/mod.rs` file with the following code:

```
pub mod new_item;
pub mod item;
```

Then, in the `models/mod.rs` file, we make the `item` module public to other modules and the `main.rs` file with the following code:

```
pub mod item;
```

More books: https://t.me/booksforits

Next, we define our model's module in the `main.rs` file with the following line of code:

```
mod models;
```

Now we can access our data models throughout the app. We have also locked down the behavior around reading and writing to the database. Next, we can move on to importing these data models and using them in our app.

Getting data from the database

When interacting with a database, it can take some time to get used to how we do this. Different **object-relational mappers** (**ORMs**) and languages have different quirks. While the underlying principles are the same, the syntax for these ORMs can vary greatly. Therefore, just clocking hours using an ORM will enable you to become more confident and solve more complex problems. We can start with the simplest mechanism, getting all the data from a table.

To explore this, we can get all the items from the `to_do` table and return them at the end of each view. We defined this mechanism in *Chapter 4*, *Processing HTTP Requests*. In terms of our application, you will recall that we have a `get_state` function that packages our to-do items for our frontend. This `get_state` function is housed in the `ToDoItems` struct in the `src/json_serialization/to_do_items.rs` file. Initially, we must import what we need with the following code:

```
use crate::diesel;
use diesel::prelude::*;

use crate::database::establish_connection;
use crate::models::item::item::Item;
use crate::schema::to_do;
```

In the preceding code, we have imported the `diesel` crate and macros, which enable us to build database queries. We then import our `establish_connection` function to make connections to the database and then import the schema and database model to make the query and handle the data. We can then refactor our `get_state` function with the following code:

```
pub fn get_state() -> ToDoItems {
    let connection = establish_connection();
    let mut array_buffer = Vec::new();

    let items = to_do::table
        .order(to_do::columns::id.asc())
        .load::<Item>(&connection).unwrap();
```

More books: https://t.me/booksforits

```
    for item in items {
        let status =
            TaskStatus::new(&item.status.as_str());
        let item = to_do_factory(&item.title, status);
        array_buffer.push(item);
    }
    return ToDoItems::new(array_buffer)
}
```

In the preceding code, we first establish the connection. Once the database connection is established, we then get our table and build a database query off it. The first part of the query defines the order. As we can see, our table can also pass references to columns which also have their own function.

We then define what struct will be used to load the data and pass in a reference to the connection. Because the macros defined the struct in the load, if we passed in the `NewItem` struct into the `load` function, we would get an error because the `Queryable` macro is not enabled for that struct.

It can also be noted that the `load` function has a prefix before the functions brackets where the parameters are passed in `load::<Item>(&connection)`. This prefix concept was covered in *Chapter 1*, *A Quick Introduction to Rust*, the *Verifying with traits* section, *step 4*, but for now, you can use the following code:

```
fn some_function<T: SomeType>(some_input: &T) -> () {
    . . .
}
```

The type that is passed into the prefix `<Item>` defines the type of input accepted. This means that when the compiler runs, a separate function for each type passed into our `load` function is compiled.

We then directly unwrap the `load` function resulting in a vector of items from the database. With the data from the database, we loop through constructing our item structs and appending them to our buffer. Once this is done, we construct the `ToDoItems` JSON schema from our buffer and return it. Now that we have enacted this change, all our views will return data directly from the database. If we run this, there will be no items on display. If we try and create any, they will not appear. However, although they are not being displayed, what we have done is get the data from the database and serialize it in the JSON structure that we want. This is the basis of returning data from a database and returning it to the requester in a standard way. This is the backbone of APIs built in Rust. Because we are no longer relying on reading from a JSON state file when getting the items from the database, we can remove the following imports from the `src/json_serialization/to_do_items.rs` file:

```
use crate::state::read_file;
```

More books: https://t.me/booksforits

```
use serde_json::value::Value;
use serde_json::Map;
```

We remove the preceding imports because we have not refactored any of the other endpoints. Resultantly, the `create` endpoint will fire correctly; however, the endpoint is just creating items in the JSON state file that `return_state` no longer reads. For us to enable creation again, we must refactor the `create` endpoint to insert a new item into the database.

Inserting into the database

In this section, we will build and create a view that creates a to-do item. If we remember the rules around our creating of to-do items, we do not want to create duplicate to-do items. This can be done with a unique constraint. However, for now, it is good to keep things simple. Instead, we will make a database fall with a filter based on the title that is passed into the view. We then check, and if no results are returned, we will insert a new `to-do` item into the database. We do this by refactoring the code in the `views/to_do/create.rs` file. First, we reconfigure the imports as seen in the following code:

```
use crate::diesel;
use diesel::prelude::*;

use actix_web::HttpResponse;
use actix_web::HttpRequest;

use crate::json_serialization::to_do_items::ToDoItems;

use crate::database::establish_connection;
use crate::models::item::new_item::NewItem;
use crate::models::item::item::Item;
use crate::schema::to_do;
```

In the preceding code, we import the necessary `diesel` imports to make a query as described in the previous section. We then import the `actix-web` structs needed for the view to process a request and define a result. We then import our database structs and functions to interact with the database. Now that we have everything, we can start working on our `create` view. Inside our `pub async fn create` function, we start by getting two references of the title of the `to-do` item from the request:

```
pub async fn create(req: HttpRequest) -> HttpResponse {
    let title: String = req.match_info().get("title"
    ).unwrap().to_string();
    . . .
}
```

More books: https://t.me/booksforits

Once we have extracted the title from the URL in the preceding code, we establish a database connection and make a database call to our table using that connection, as seen in the following code:

```
let connection = establish_connection();
let items = to_do::table
    .filter(to_do::columns::title.eq(&title.as_str()))
    .order(to_do::columns::id.asc())
    .load::<Item>(&connection)
    .unwrap();
```

As we can see in the preceding code, the query is pretty much the same as the query in the previous section. However, we have a `filter` section that refers to our `title` column that must be equal to our `title`. If the item being created is truly new, there will be no items created; therefore, the length of the result will be zero. Subsequently, if the length is zero, we should create a `NewItem` data model and then insert it into the database to return the state at the end of the function, as seen in the following code:

```
if items.len() == 0 {
    let new_post = NewItem::new(title);
    let _ =
     diesel::insert_into(to_do::table).values(&new_post)
     .execute(&connection);
}
return HttpResponse::Ok().json(ToDoItems::get_state())
```

We can see that Diesel has an `insert` function, which accepts the table, and references to the data model we built. Because we have switched to database storage, we can delete the following imports as they are not needed:

```
use serde_json::value::Value;
use serde_json::Map;
use crate::state::read_file;
use crate::processes::process_input;
use crate::to_do::{to_do_factory, enums::TaskStatus};
```

Now, using our app, we will be able to create to-do items and then see these items pop up on the frontend of our application. Therefore, we can see that our `create` and `get state` functions are working; they are engaging with our database. If you are having trouble, a common mistake is to forget to spin up our `docker-compose`. (*Note: Remember to do this, otherwise, the app will not be able to connect to the database as it is not running*). However, we cannot edit our to-do items status to `DONE`. We will have to edit our data on the database to do this.

More books: https://t.me/booksforits

Editing the database

When we edit our data, we are going to get the data model from the database and then edit the entry with a database call function from Diesel. To engage our `edit` function with the database, we can edit our view in the `views/to_do/edit.rs` file. We start by refactoring the imports, as you see in the following code:

```
use crate::diesel;
use diesel::prelude::*;
use actix_web::{web, HttpResponse};

use crate::json_serialization::{to_do_item::ToDoItem,
                                to_do_items::ToDoItems};
use crate::jwt::JwToken;
use crate::database::establish_connection;
use crate::schema::to_do;
```

As we can see in the preceding code, there is a pattern emerging. The pattern is that we import dependencies to handle authentication, a database connection, and a schema and then build a function that performs the operation that we want. We have covered the imports and the meanings behind them previously. In our `edit` view, we must only get one reference to the title this time, which is denoted in the following code:

```
pub async fn edit(to_do_item: web::Json<ToDoItem>,
    token: JwToken) -> HttpResponse {
    let connection = establish_connection();
    let results = to_do::table.filter(to_do::columns::title
        .eq(&to_do_item.title));

    let _ = diesel::update(results)
        .set(to_do::columns::status.eq("DONE"))
        .execute(&connection);
    return HttpResponse::Ok().json(ToDoItems::get_state())
}
```

In the preceding code, we can see that we perform a filter on our database without loading the data. This means that the `results` variable is an `UpdateStatement` struct. We then use this `UpdateStatement` struct to update our item to `DONE` in the database. With the fact that we are no longer using the JSON file, we can delete the following imports in the `views/to_do/edit.rs` file:

```
use crate::processes::process_input;
```

More books: https://t.me/booksforits

```
use crate::to_do::{to_do_factory, enums::TaskStatus};
use serde_json::value::Value;
use serde_json::Map;
use crate::state::read_file;
```

In the preceding code, we can see that we call the `update` function and fill it with the results that we got from the database. We then set the `status` column to `Done`, and then `execute` using the reference to the connection. Now we can use this to edit our `to-do` items so they shift to the done list. However, we cannot delete them. To do this, we are going to have to refactor our final endpoint to completely refactor our app to be connected to a database.

Deleting data

With deleting data, we will take the same approach that we took in the previous section when editing. We will get an item from the database, pass it through the Diesel `delete` function, and then return the state. Right now, we should be comfortable with this approach, so it is advised that you try and implement it by yourself in the `views/to_do/delete.rs` file. The code is given as follows for the imports:

```
use crate::diesel;
use diesel::prelude::*;

use actix_web::{web, HttpResponse};
use crate::database::establish_connection;
use crate::schema::to_do;

use crate::json_serialization::{to_do_item::ToDoItem,
                                to_do_items::ToDoItems};
use crate::jwt::JwToken;
use crate::models::item::item::Item;
```

In the preceding code, we rely on the Diesel crates and the prelude so we can use the Diesel macros. Without `prelude`, we would not be able to use the schema. We then import the Actix Web structs that are needed to return data to the client. We then import the crates that we have built to manage our to-do item data. For the `delete` function, the code is as follows:

```
pub async fn delete(to_do_item: web::Json<ToDoItem>,
                    token: JwToken) -> HttpResponse {
    let connection = establish_connection();
    let items = to_do::table
```

More books: https://t.me/booksforits

```
        .filter(to_do::columns::title.eq(
                &to_do_item.title.as_str()))
        .order(to_do::columns::id.asc())
        .load::<Item>(&connection)
        .unwrap();
    let _ = diesel::delete(&items[0]).execute(&connection);
    return HttpResponse::Ok().json(ToDoItems::get_state())
}
```

To conduct quality control, let us go through the following steps:

1. Enter `buy canoe` into the text input and click the **Create** button.
2. Enter `go dragon boat racing` into the text input and click the **Create** button.
3. Click the **edit** button on the **buy canoe** item. After doing this, we should have the following output in the frontend:

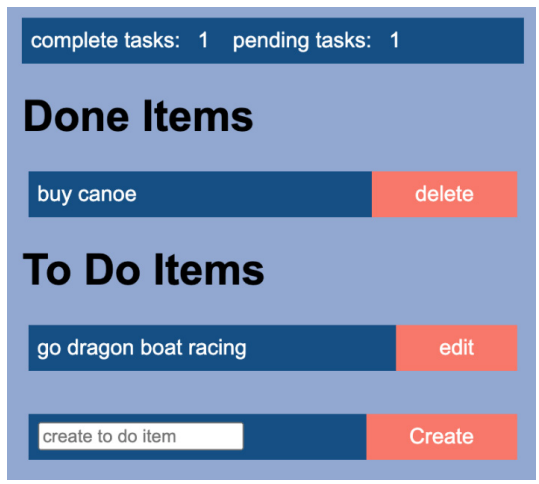


Figure 6.6 – Expected output

In the preceding figure, we have bought our canoe, but we have not gone dragon boat racing yet. And here we have it, our app is working seamlessly with our PostgreSQL database. We can create, edit, and delete our to-do items. Because of the structure that was defined in previous chapters, chopping out the JSON file mechanism for the database did not require a lot of work. The request for processing and returning of data was already in place. When we run our application now you may have realized that we have the following printout when compiling:

```
warning: function is never used: `read_file`
  --> src/state.rs:17:8
```

More books: https://t.me/booksforits

```
    |
17 | pub fn read_file(file_name: &str) -> Map<String, Value> {
. . .
warning: function is never used: `process_pending`
  --> src/processes.rs:18:4
    |
18 | fn process_pending(item: Pending, command: String,
                        state: &Map<String, Value>) {
```

What is happening, according to the preceding printout, is that we are no longer using our `src/state.rs` file or our `src/processes.rs` file. These files are no longer being used because our `src/database.rs` file is now managing our storage, deletion, and editing of persistent data with a real database. We are also processing our data from the database to a struct with our database model structs that have implemented the Diesel `Queryable` and `Identifiable` traits. Thus, we can delete the `src/state.rs` and `src/database.rs` files as they are no longer needed. We also do not need our `src/to_do/traits` module, so this can be removed too. Next, we can remove all the references to the traits. Removing the references essentially means removing traits from the `src/to_do/mod.rs` file and removing the import and implementation of these traits in our structs for the `to_do` module. For instance, to remove the traits from the `src/to_do/structs/pending.rs` file, we must merely remove the following imports:

```
use super::super::traits::get::Get;
use super::super::traits::edit::Edit;
use super::super::traits::create::Create;
```

We can then remove the following implementation of these traits:

```
impl Get for Pending {}
impl Edit for Pending {}
impl Create for Pending {}
```

We also must remove the traits in the `src/to_do/structs/done.rs` file. We now have an opportunity to appreciate the payoff of our well-structured isolated code. We can slot in different methods for persistent storage easily, as we can simply remove existing storage methods by deleting a couple of files and the traits directly. We then just removed the implementations of the traits. That was it. We did not have to dig into functions and alter lines of code. Because our reading functionality was in the traits, removing the implementations severed our application completely by just removing the implementations of the traits. Structuring code like this really pays off in the future. In the past, I have had to pull code out of one server into another when refactoring microservices or upgrading or switching a method, such as a storage option. Having isolated code makes refactoring a lot easier, quicker, and less error-prone.

More books: https://t.me/booksforits

Our application is now fully working with a database. However, we can improve the implementation of our database. Before we do this, however, we should refactor our configuration code in the next section.

Configuring our application

Right now, we are storing our database URL in a `.env` file. This is fine, but we can also use `yaml config` files. When developing web servers, they will be run in different environments. For instance, right now, we are running our Rust server on our local machine. However, later we will package the server into our own Docker image and deploy it on the cloud. With microservices infrastructure, we might use a server that we built in one cluster and slot it into a different cluster with different databases to connect to. Because of this, config files defining all inward and outward traffic become essential. When we deploy our server, we can make a request to file storage, such as AWS S3, to get the right config file for the server. It must be noted that environment variables can be preferred for deployment as they can be passed into containers. We will cover how to configure a web application using environment variables in *Chapter 13*, *Best Practices for a Clean Web App Repository*. For now, we will focus on using config files. We also need to enable our server to be flexible in terms of what config file is being loaded. For instance, we should not have to have a config file in a particular directory and with a particular name to be loaded into our server. We should keep our config files properly named for the right context to reduce the risk of the wrong config file being loaded into the server. Our path to the file can be passed when spinning up our server. Because we are using `yaml` files, we need to define the `serde_yaml` dependency on our `Cargo.toml` file as follows:

```
serde_yaml = "0.8.23"
```

Now that we can read `yaml` files, we can build our own configuration module, which loads to `yaml` file values into a HashMap. This can be done with one struct; therefore, we should put our config module into one file, which is the `src/config.rs` file. First, we import what we need with the following code:

```
use std::collections::HashMap;
use std::env;

use serde_yaml;
```

In the preceding code, we use `env` to capture environment variables passed into the program, `HashMap` to store the data from the config file, and the `serde_yaml` crate to process `yaml` values from the config file. We then define the struct that houses our config data with the following code:

```
pub struct Config {
    pub map: HashMap<String, serde_yaml::Value>
}
```

More books: https://t.me/booksforits

In the preceding code, we can see that our data key of the value is `String`, and the value belonging to those keys are `yaml` values. We then build a constructor for our struct that takes the last argument passed into the program, opens the file based on the path to the file passed into the program, and loads our `map` field with the data from the file with the following code:

```
impl Config {
    pub fn new() -> Config {
        let args: Vec<String> = env::args().collect();
        let file_path = &args[args.len() - 1];

        let file = std::fs::File::open(file_path).unwrap();
        let map: HashMap<String, serde_yaml::Value> =
            serde_yaml::
                 from_reader(file).unwrap();
        return Config{map}
    }
}
```

Now that the `config` struct is defined, we can define our `config` module in the `src/main.rs` file with the following line of code:

```
mod config;
```

We must then refactor our `src/database.rs` file to load from a `yaml` `config` file. Our refactored imports take the following form:

```
use diesel::prelude::*;
use diesel::pg::PgConnection;

use crate::config::Config;
```

We can see that all references to `env` have been removed, as this is now handled in our `config` module. We then load our file, get our `DB_URL` key, and directly unwrap the result of getting the variable associated with the `DB_URL` key, converting the `yaml` value to a string and directly unwrapping the result from that conversion. We directly unwrap the getting and conversion functions because if they fail, we will not be able to connect to the database anyway. If we cannot connect, we want to know about the error as soon as possible with a clear error message showing where this is happening. We can now shift our database URL into our `config.yml` file in the root of our Rust application with the following content:

```
DB_URL: postgres://username:password@localhost:5433/to_do
```

More books: https://t.me/booksforits

Next, we can run our application with the following command:

```
cargo run config.yml
```

The `config.yml` file is the path to the config file. If you run `docker-compose` and the frontend, you will see that the database URL from our config file is being loaded and our application is connected to the database. There is one problem with this database connection, however. Every time we execute the `establish_connection` function, we make a connection to our database. This will work; however, it is not optimal. In the next section, we will be more efficient with our database connections with database connection pooling.

Building a database connection pool

In this section, we will create a database connection pool. A database connection pool is a limited number of database connections. When our application needs a database connection, it will take the connection from the pool and place it back into the pool when the application no longer needs the connection. If there are no connections left in the pool, the application will wait until there is a connection available, as seen in the following diagram:

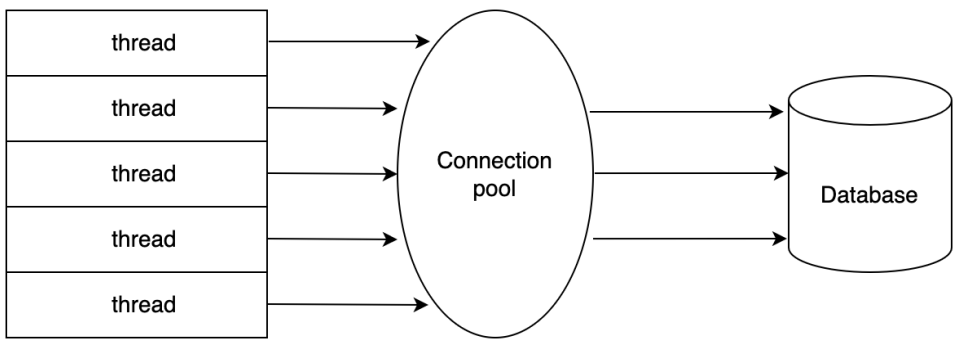


Figure 6.7 – Database connection pool with a limit of three connections

Before we refactor our database connection, we need to install the following dependency in our `Cargo.toml` file:

```
lazy_static = "1.4.0"
```

We then define our imports with the `src/database.rs` file with the following code:

```
use actix_web::dev::Payload;
use actix_web::error::ErrorServiceUnavailable;
use actix_web::{Error, FromRequest, HttpRequest};
```

More books: https://t.me/booksforits

```
use futures::future::{Ready, ok, err};

use lazy_static::lazy_static;
use diesel::{
   r2d2::{Pool, ConnectionManager, PooledConnection},
   pg::PgConnection,
};
use crate::config::Config;
```

From the imports we have defined in the preceding code, what do you think we will do when we write out our new database connection? At this point, it is a good time to stop and think about what you can do with these imports.

The first block of imports, if you remember, will be used to establish a database connection before the request hits the view. The second block of imports enables us to define our database connection pool. The final `Config` parameter is to get the database URL for the connection. Now that our imports are done, we can define the connection pool struct with the following code:

```
type PgPool = Pool<ConnectionManager<PgConnection>>;

pub struct DbConnection {
   pub db_connection: PgPool,
}
```

In the preceding code, we state that our `PgPool` struct is a connection manager that manages connections inside a pool. We then build our connection, which is a static reference, with the following code:

```
lazy_static! {
    pub static ref DBCONNECTION: DbConnection = {
        let connection_string =
            Config::new().map.get("DB_URL").unwrap()
            .as_str().unwrap().to_string();
      DbConnection {
          db_connection: PgPool::builder().max_size(8)
          .build(ConnectionManager::new(connection_string))
          .expect("failed to create db connection_pool")
       }
   };
}
```

More books: https://t.me/booksforits

In the preceding code, we get the URL from the config file and construct a connection pool that is returned and thus assigned to the `DBCONNECTION` static referenced variable. Because this is a static reference, the lifetime of our `DBCONNECTION` variable matches the lifetime of the server. We can now refactor our `establish_connection` function to take from the database connection pool with the following code:

```
pub fn establish_connection() ->
  PooledConnection<ConnectionManager<PgConnection>>{
    return DBCONNECTION.db_connection.get().unwrap()
}
```

In the preceding code, we can see that we are returning a `PooledConnection` struct. However, we do not want to call the `establish_connection` function every time we need it. We also want to reject the HTTP request before it hits the view if we cannot make the connection for whatever reason, as we do not want to load a view that we cannot process. Just like our `JWToken` struct, we can create a struct that creates a database connection and passes that database connection into the view. Our struct has one field, which is a pooled connection, as follows:

```
pub struct DB {
   pub connection:
PooledConnection<ConnectionManager<PgConnection>>
}
```

With this `DB` struct, we can implement the `FromRequest` trait like we did with our `JWToken` struct with the following code:

```
impl FromRequest for DB {
  type Error = Error;
  type Future = Ready<Result<DB, Error>>;

  fn from_request(_: &HttpRequest, _: &mut Payload) ->
      Self::Future{
      match DBCONNECTION.db_connection.get() {
         Ok(connection) => {
            return ok(DB{connection})
         },
         Err(_) => {
            return err(ErrorServiceUnavailable(
            "could not make connection to database"))
         }
```

More books: https://t.me/booksforits

```
        }
    }
}
```

Here we do not directly unwrap the getting the database connection. Instead, if there is an error when connecting, we return an error with a helpful message. If our connection is successful, we then return it. We can implement this in our views. To avoid repetitive code, we shall just use the `edit` view, but we should apply this approach to all our views. First, we define the following imports:

```
use crate::diesel;
use diesel::prelude::*;
use actix_web::{web, HttpResponse};

use crate::json_serialization::{to_do_item::ToDoItem,
                                to_do_items::ToDoItems};
use crate::jwt::JwToken;
use crate::schema::to_do;
use crate::database::DB;
```

In the preceding code, we can see that we have imported the `DB` struct. Our `edit` view should now look as follows:

```
pub async fn edit(to_do_item: web::Json<ToDoItem>,
    token: JwToken, db: DB) -> HttpResponse {
    let results = to_do::table.filter(to_do::columns::title
        .eq(&to_do_item.title));

    let _ = diesel::update(results)
        .set(to_do::columns::status.eq("DONE"))
        .execute(&db.connection);

    return HttpResponse::Ok().json(ToDoItems::get_state())
}
```

In the preceding code, we can see that we directly reference the `connection` field of our `DB` struct. The fact is that we can simply get a pooled connection into our view by passing the `DB` struct into the view like our authentication token.

More books: https://t.me/booksforits

Summary

In this chapter, we constructed a development environment where our app could interact with the database using Docker. Once we did this, we explored the listing of containers and images to inspect how our system is going. We then created migrations using the `diesel` crate. After this, we installed the `diesel` client and defined the database URL as an environment variable so that our Rust app and migrations could directly connect with the database container.

We then ran migrations and defined the SQL scripts that would fire when the migration ran, and in turn, ran them. Once all this was done, we inspected the database container again to see whether the migration had, in fact, been executed. We then defined the data models in Rust, and refactored our API endpoints, so they performed `get`, `edit`, `create`, and `delete` operations on the database in order to keep track of the to-do items.

What we have done here is upgraded our database storage system. We are one step closer to having a production-ready system as we are no longer relying on a JSON file to store our data. You now have the skills to perform database management tasks that enable you to manage changes, credentials/access, and schemas. We also performed all the basic operations on the database that are needed to run an app that creates, gets, updates, and deletes data. These skills are directly transferable to any other project you wish to undertake in Rust web projects. We then augmented our newly developed database skills by enabling our database connections to be limited by a connection pool. Finally, implementing our database connection structs was made easy by implementing the `FromRequest` trait so other developers can implement our connection just by passing the struct into the view as a parameter. The view is also protected if the database connection cannot be made.

In the next chapter, we will build on these skills to build a user authentication system so we can create users and check credentials when accessing the app. We will use a combination of database, extraction of data from headers, browser storage, and routing to ensure that the user must be logged in to access the to-do items.

Questions

1. What are the advantages of having a database over a JSON file?
2. How do you create a migration?
3. How do we check migrations?
4. If we wanted to create a user data model in Rust with a name and age, what should we do?
5. What is a connection pool, and why should we use it?

More books: https://t.me/booksforits

Answers

1. The database has advantages in terms of multiple reads and writes at the same time. The database also checks the data to see whether it is in the right format before inserting it and we can do advanced queries with linked tables.
2. We install the `diesel` client and define the database URL in the `.env` file. We then create migrations using the client and write the desired schema required for the migration. We then run the migration.
3. We use the container ID of the database to access the container. We then list the tables; if the table we desire is there, then this is a sign that the migration ran. We can also check the migration table in the database to see when it was last run.
4. We define a `NewUser` struct with the name as a string and age as an integer. We then create a `User` struct with the same field and an extra integer field, the ID.
5. A connection pool pools a limited number of connections that connect to the database. Our application then passes these connections to the threads that need them. This keeps the number of connections connecting to the database limited to avoid the database being overloaded.

More books: https://t.me/booksforits

7

Managing User Sessions

At this point, our app is manipulating data in a proper database through the clicking of buttons on the view. However, anyone who comes across our app can also edit the data. While our app is not the type of app that would require a lot of security, it is an important concept to understand and practice in general web development.

In this chapter, we will build a system that creates users. This system will also manage user sessions by requiring the user to log in before they can alter any to-do items through the frontend app.

In this chapter, we will cover the following topics:

- Creating user data models with relationships with other tables with unique constraints of certain fields via database migrations
- Authenticating our users
- Managing user sessions
- Cleaning up authentication requirements
- Configuring expiration of auth tokens
- Adding authentication into our frontend

After reading this chapter, you will be able to understand the basics of authenticating users on a web server. You will also be able to implement this authentication on the server side of our Rust application and store credentials in our React application in the frontend. The understanding of the concepts and practices covered in this chapter will also enable you to incorporate authentication in phone applications using React Native, and on a Rust server and desktop applications by wrapping our React application in **Electron**.

More books: https://t.me/booksforits

Technical requirements

In this chapter, we build on the code built in the previous chapter. This can be found at the following URL: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter06/building_a_database_connection_pool`.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter07`.

Creating our user model

Since we are managing user sessions in our app, we will need to store information about our users to check their credentials, before we allow our to-do items to be created, deleted, and edited. We will store our user data in a **PostgreSQL** database. While this is not essential, we will also link users in the database to to-do items. This will give us an understanding of how to alter an existing table and create links between tables. To create our user model, we are going to have to do the following:

1. Create a `User` data model.
2. Create a `NewUser` data model.
3. Alter the to-do item data model so that we can link it to a user model.
4. Update the schema file with the new table and altered fields.
5. Create and run migration scripts on the database.

In the following sections, we'll look at the preceding steps in detail.

Creating a User data module

Before we start, we will need to update the dependencies in the `Cargo.toml` file with the following dependencies:

```
[dependencies]
. . .
bcrypt = "0.13.0"
uuid = {version = "1.0.0", features = ["serde", "v4"]}
```

We will be using the `bcrypt` crate to hash and check passwords and the `uuid` crate to generate unique IDs for our user data models. As we covered in *Chapter 6*, *Data Persistence with PostgreSQL*, we will need to create two different structs for our user data model.

More books: https://t.me/booksforits

The new user will not have an `id` field because it does not exist in the database yet. This ID is created by the database when the new user is inserted into the table. We then have another struct that has all the same fields with the `id` field we added, since we might need to use this ID when we're interacting with existing users in the database. ID numbers can be useful for referencing other tables. They are short and we know that they are unique. We will be using a user ID to link the user to the to-do items. These data models can be housed in the following file structure in the `src/models.rs` directory:

```
└── user
    ├── mod.rs
    ├── new_user.rs
    └── user.rs
```

We will define the data model in our `new_user.rs` file. First, we must define the imports, as shown here:

```
use uuid::Uuid;
use diesel::Insertable;
use bcrypt::{DEFAULT_COST, hash};

use crate::schema::users;
```

It must be noted that we have not defined the users in the schema yet. We will get around to this after we have finished with all the data models. We will not be able to compile our code until we have defined our `users` schema. We will also import a unique ID crate because we are going to be creating a unique ID when we create a new user, and the `Insertable` trait from the `diesel` crate because we are going to be inserting the new user into our database. We then use the `hash` function from the `bcrypt` crate to hash the new password that we define for our new user. We can also see that we import the `DEFAULT_COST` constant from the `bcrypt` crate. The `DEFAULT_COST` is merely a constant that we will pass into the `hash` function. We will explore why this is the case in the next section when we cover hashing passwords. Now that we have defined our user data model module and imported what we need, we can move on to the next section to create the `NewUser` struct.

Creating a NewUser data model

We can define our data model with the following code:

```
#[derive(Insertable, Clone)]
#[table_name="users"]
pub struct NewUser {
    pub username: String,
    pub email: String,
```

More books: https://t.me/booksforits

```
    pub password: String,
    pub unique_id: String,
}
```

Here, we can see that we allowed our data model to be insertable. However, we are not allowing it to be queried. We want to ensure that when a user is retrieved from the database, their ID is present. We could move on to defining the general data model for users, but this is not secure. We need to ensure that our passwords are protected by hashing them. If you remember from *Chapter 2*, *Designing Your Web Application in Rust*, we utilized traits to allow certain to-do structs to perform actions. Some structs could create, while others could delete based on the traits that they implemented. We are locking down the functionality of our `NewUser` struct here by just implementing the `Insertable` trait. However, we will enable querying by implementing other traits for the `User` struct, as shown in the following figure:

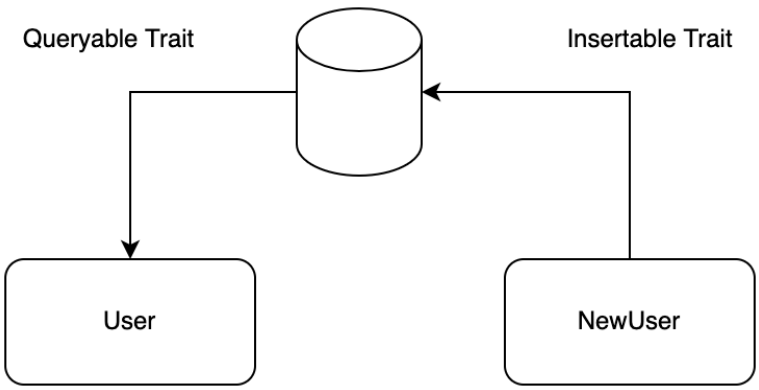


Figure 7.1 – Locking down data model structs with traits

Now that we have created the struct that inserts new users into the database, we can explore how to store our users' passwords in the database.

You may have wondered why you cannot recover forgotten passwords; you can only reset them. This is because the password is *hashed*. Hashing passwords is a common practice when it comes to storing them. This is where we use an algorithm to obfuscate a password so that it cannot be read. Once this is done, it cannot be reversed.

The hashed password is then stored in a database. To check the password, the input password is hashed and compared to the hashed password in the database. This allows us to see whether the input hashed password matches the hashed password stored in the database. This has a couple of advantages. First, it prevents employees who have access to your data from knowing your password. If there is a data leak, it also prevents the leaked data from directly exposing your password to whoever had the data.

More books: https://t.me/booksforits

Considering a lot of people use the same password for multiple things (even though they should not), you can only imagine the damage that may be caused to people using your app if you are not hashing passwords and there's a data breach. However, hashing gets more complicated than this. There is a concept called *salting* that ensures that when you hash the same password, it does not result in the same hash. It does this by adding an extra bit of data to the password before it is hashed. This is also where the `DEFAULT_COST` constant that we pass into the `hash` function comes in. Let's say we got hold of the data in the database and we want to write code that will guess the passwords we have in the data. If we have enough computing power, we could effectively guess the password. Therefore, we can pass in a cost parameter. As we increase the cost parameter, the amount of work in either CPU time or memory increases exponentially. Increasing the cost factor by one will increase the number of operations needed to compute the hash by 10,000 or even more.

Explaining password security in more detail is beyond the scope of this book. However, it must be stressed that password hashing is always a must when storing passwords. Luckily, there is a range of modules in all major languages that enable you to hash and check passwords with just a few lines of code. Rust is no different here.

To ensure that we can insert our new users into the database with hashed passwords, follow these steps:

1. First, we will have to ensure that the input password is hashed in our `NewUser` constructor, which is defined as follows:

```
impl NewUser {
    pub fn new(username: String,
        email: String, password: String) -> NewUser {
        let hashed_password: String = hash(
                password.as_str(), DEFAULT_COST
            ).unwrap();
        let uuid = Uuid::new_v4().to_string();
        return NewUser {
            username,
            email,
            password: hashed_password,
            unique_id: uuid
        }
    }
}
```

More books: https://t.me/booksforits

Here, we used the `hash` function from the `bcrypt` crate to hash our password, where we also passed in the `DEFAULT_COST` constant. We also created a unique ID using the `Uuid` crate and then constructed a new instance of the `NewUser` struct with those attributes. In our app, there is no real need for a unique ID. However, these can come in handy if you are communicating between multiple servers and databases.

2. Now that we have defined our `NewUser` data model, we can define our general user data model in the `user.rs` file with the following code. First, we must define the following imports:

```
extern crate bcrypt;

use diesel::{Queryable, Identifiable};
use bcrypt::verify;

use crate::schema::users;
```

Here, we can see that we are using the `verify` function and that we are also allowing the general user data model struct to be queryable and identifiable.

3. With the imports defined in the previous step, we can build our `User` struct. Remember, this is a struct that is going to be loaded from our database when we make database queries. Before you read further, this is a good time to try and build the `User` struct yourself, as it uses the same table as the `NewUser` struct but has an `id` field and is queried instead of inserted. Once you have built your `User` struct, it should look like the following code:

```
#[derive(Queryable, Clone, Identifiable)]
#[table_name="users"]
pub struct User {
    pub id: i32,
    pub username: String,
    pub email: String,
    pub password: String,
    pub unique_id: String
}
```

We can see that we just added the `id` field and derived the `Queryable` trait instead of the `Insertable` trait.

More books: https://t.me/booksforits

4. Now that our `User` struct has been defined, we can build a function that verifies whether an input password matches the password belonging to the user with the following code:

```
impl User {
    pub fn verify(&self, password: String) -> bool {
    verify(password.as_str(),
    &self.password).unwrap()
    }
}
```

5. Now that our models have been defined, we must remember to register them in the `models/user/mod.rs` file with the following code:

```
pub mod new_user;
pub mod user;
```

6. Furthermore, we can make these modules accessible to the app by adding the following line to the `models/mod.rs` file:

```
pub mod item;
pub mod user;
```

With that, our data models for the users have been defined. However, we still need to link them to our to-do items.

Altering the to-do item data model

To link data models to our to-do items, we must alter our to-do data models. There are multiple ways in which we can do this. For instance, we can add a `user_id` field to the item table that is just the `unique_id` field of the user table. When we are creating a new item, we then pass the unique ID of the user into the item constructor. This is easy to implement; however, it does have risks. Merely passing the unique ID of the user into the item does not enforce that the ID of the user is valid and in the database. There is nothing stopping us from inserting an ID of a deleted user into the item constructor and thus inserting an orphaned item into the database. This will then be hard to extract later, as we have no reference to the user ID that the orphaned item is associated with. We can also create a new table that references the user's ID with the item ID, as shown in the following figure:

More books: https://t.me/booksforits

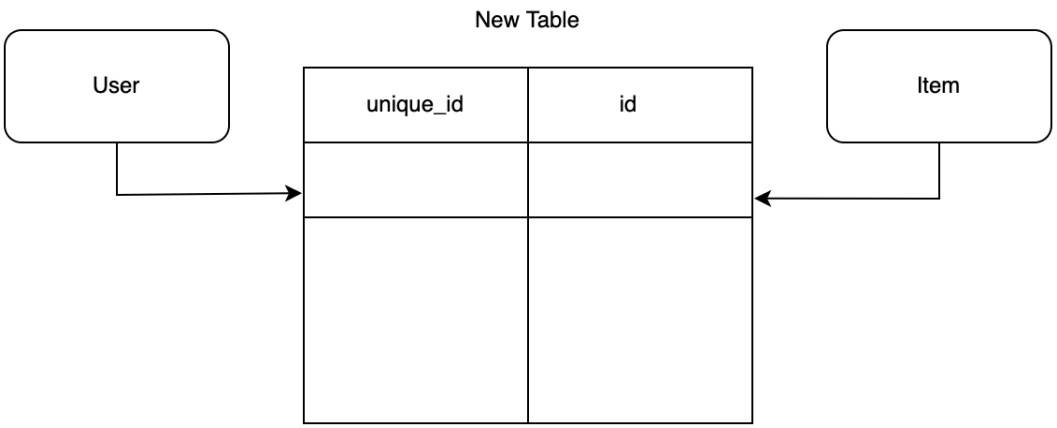


Figure 7.2 – A separate database table for logging item associations with users

The advantage of this is that it is easy to decouple the users from the items by merely dropping the table. However, it also does not have valid user ID enforcement or item ID enforcement when creating a new entry. We will also have to make two queries, one to the association table and then another to the item table to get the items from the user. As the previous two methods of attaching a user ID column to the items table or creating a bridge table holding an item ID and user unique ID are easy to implement, we will not explore them; you should be able to implement them yourself at this point. In the context of the to-do application, the previous two methods would be subpar, as they offer us no benefits yet introduce the risk of errors when inserting data into our database. This does not mean that the two previous methods should never be used. The data needs for each project are different. In our project, we will create a foreign key to link our users to items, as shown in the following figure:

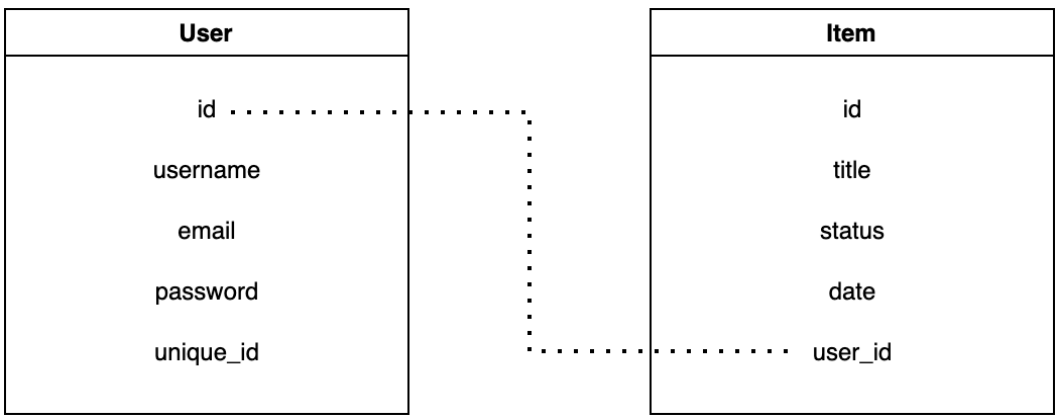


Figure 7.3 – Foreign key association between our user and items

More books: https://t.me/booksforits

This does not allow us to access the items associated with a user with one database call, but we are only allowed to insert items that have a reference to a legitimate user ID in the database. Foreign keys can also trigger cascade events where, if we delete a user, this will automatically delete all existing items associated with the user to prevent orphan items from being created. We create a foreign key by declaring the link to the table with a macro. In `models/item/item.rs`, we can achieve this by initially having the following imports:

```
use crate::schema::to_do;
use chrono::NaiveDateTime;
use super::super::user::user::User;
```

We can see that we must import the `User` struct, as we will be referencing it in the `belongs_to` macro to claim that our `Item` struct belongs to the `User` struct, as shown in the following code:

```
#[derive(Queryable, Identifiable, Associations)]
#[belongs_to(User)]
#[table_name="to_do"]
pub struct Item {
    pub id: i32,
    pub title: String,
    pub status: String,
    pub date: NaiveDateTime,
    pub user_id: i32,
}
```

Here, we can see that we imported the user data model struct, defined it with a `belongs_to` macro, and added a `user_id` field to link the struct. Note that the `belongs_to` macro will not be callable if we do not include the `Associations` macro.

One last thing we need to do is add the `user_id` field to the fields and constructor in the `models/item/new_item.rs` file. We need to do this so that we can link the new to-do item to the user creating the item. This can be achieved by using the following code:

```
use crate::schema::to_do;
use chrono::{NaiveDateTime, Utc};


#[derive(Insertable)]
#[table_name="to_do"]
pub struct NewItem {
```

More books: https://t.me/booksforits

```
    pub title: String,
    pub status: String,
    pub date: NaiveDateTime,
    pub user_id: i32,
}

impl NewItem {
    pub fn new(title: String, user_id: i32) -> NewItem {
        let now = Utc::now().naive_local();
        NewItem{
            title, status: String::from("PENDING"),
            date: now,
            user_id
        }
    }
}
```

So, taking stock of what we have done, all our data model structs have been altered, and we are able to use them as and when we need them in the app when interacting with the database. However, we have not updated our database, and we have not updated the bridge connecting the app to the database. We will do this next.

Updating the schema file

To make sure that the mapping from the data model struct to the database is up to date, we must update our schema with these changes. This means that we must alter the existing schema for the to-do item table and add a user schema to the `src/schema.rs` file. This is denoted by the following code:

```
table! {
    to_do (id) {
        id -> Int4,
        title -> Varchar,
        status -> Varchar,
        date -> Timestamp,
        user_id -> Int4,
    }
```

More books: https://t.me/booksforits

```
}
table! {
    users (id) {
        id -> Int4,
        username -> Varchar,
        email -> Varchar,
        password -> Varchar,
        unique_id -> Varchar,
    }
}
```

It must be noted that our fields in the schema file are defined in the same order as the Rust data models. This is important because, if we do not do this, the fields will be mismatched when we're connecting to the database. We might also realize that our schema is merely just defining the fields and their type; it is not covering the relationship between the to-do table and the user table.

We do not have to worry about this because when we create and run our own migrations, this schema file will be updated with the relationship. This leads us to create our own migrations to complete this schema file.

Creating and running migration scripts on the database

Running migrations has a similar process to what we covered in *Chapter 6*, *Data Persistence with PostgreSQL*, which covered how to install the Diesel client and connect to the database. First, we must run our database with the `docker-compose` command:

```
docker-compose up
```

We will need this running in the background when we run the migration. We can then create the migration scripts by running the following command:

```
diesel migration generate create_users
```

This creates a directory in the migrations, which includes `create_users` in the username of the directory. Inside this directory, we have two blank *SQL* files. Here, we will manually write our own SQL scripts for the migrations. Initially, you might find this unnecessary, as there are libraries in other languages that automatically generate these migrations, but there are some advantages to doing this.

More books: https://t.me/booksforits

irst, it keeps our hand in SQL, which is another handy tool. This enables us to think about solutions that utilize SQL in the day-to-day problems that we are trying to solve. It also gives us more fine-grained control of how migrations flow. For instance, in the migration that we are going to create, we are going to have to create the user table and then a base user so that when we alter the column in the `to_do` table, we can fill it with the ID of the placeholder user row. We carry this out in our `up.sql` file with the following table definition:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR NOT NULL UNIQUE,
    email VARCHAR NOT NULL UNIQUE,
    password VARCHAR NOT NULL,
    unique_id VARCHAR NOT NULL
);
```

This is straightforward. Note that the `email` and `username` fields are unique. This is because we do not want users with duplicate usernames and emails. It's good to put the constraint in at this level for several reasons. For instance, we could protect against this by doing a database call of the username and email and refusing to insert a new user if there is duplication.

However, there may be an error in the code, or someone might alter our code in the future. A new feature might be introduced that doesn't have this check, such as an edit feature. There might be a migration that alters rows or inserts new users. It is usually best practice, if you are writing your own SQL, to ensure that you use the `;` symbol to indicate that the operation has finished.

This SQL command is fired, and then the next command is fired afterward. Our next command in the `up.sql` file inserts a placeholder user row with the following command:

```
 INSERT INTO users (username, email, password, unique_id)
VALUES ('placeholder', 'placeholder email',
'placeholder password', 'placeholder unique id');
```

Now that we have created our user, we then alter our `to_do` table. We can do this with the following command, in the same file under the previous command we just wrote:

```
ALTER TABLE to_do ADD user_id integer default 1
CONSTRAINT user_id REFERENCES users NOT NULL;
```

More books: https://t.me/booksforits

With that, our `up.sql` migration has been defined. Now, we must define our `down.sql` migration. With the down migration, we basically must reverse what we did in the up migrations. This means dropping the `user_id` column in the `to_do` table and then dropping the user table entirely. This can be done with the following SQL code in the `down.sql` file:

```
ALTER TABLE to_do DROP COLUMN user_id;

DROP TABLE users
```

We must keep in mind that Docker must be running for the migration to influence the database. Once this migration is run, we can see that the following code has been added to the `src/schema.rs` file:

```
joinable!(to_do -> users (user_id));

allow_tables_to_appear_in_same_query!(
    to_do,
    users,
);
```

This enables our Rust data models to make queries concerning the relationship between users and to-do items. With this migration finished, we can run our app again. However, before we do that, there is just one slight alteration that we have to make in the `src/views/to_do/create.rs` file, where the constructor of the new item in the `create` view function adds the default user ID with the following line of code:

```
let new_post = NewItem::new(title, 1);
```

Running our app now will result in the same behavior we described in *Chapter 6*, *Data Persistence with PostgreSQL*, in that our app is running with the migrations that we have made. However, we also need to see whether our constructor for the new user works as we hash the password and generate the unique ID.

To do this, we need to build a create user endpoint. For this, we must define the schema, and then a view that inserts that new user into the database. We can create our schema in the `src/json_serialization/new_user.rs` file with the following code:

```
use serde::Deserialize;

#[derive(Deserialize)]
pub struct NewUserSchema {
    pub name: String,
```

More books: https://t.me/booksforits

```
    pub email: String,
    pub password: String
}
```

After this, we can declare the new user schema in our `src/json_serialization/mod.rs` file with `pub mod new_user;`. Once our schema has been defined, we can create our own user view module with the following file structure:

```
views
...
└── users
    ├── create.rs
    └── mod.rs
```

In our `users/create.rs` file, we need to build a create view function. First, import the following crates:

```
use crate::diesel;
use diesel::prelude::*;

use actix_web::{web, HttpResponse, Responder};
use actix_web::HttpResponseBuilder;

use crate::database::DB;
use crate::json_serialization::new_user::NewUserSchema;
use crate::models::user::new_user::NewUser;
use crate::schema::users;
```

Since we have been building our views multiple times now, none of these imports should be surprising. We import the `diesel` macros and crate to enable us to make calls to the database. We then import the `actix_web` traits and structs to enable data to flow in and out of the view. We then import our schemas and structs to structure the data that we are receiving and processing. Now that we've imported the correct crates, we must define the `create` view function with the following code:

```
pub async fn create(new_user: web::Json<NewUserSchema>,
                    db: DB) -> impl Responder {
    . . .
}
```

More books: https://t.me/booksforits

Here, we can see that we accept JSON data that is loaded into the `NewUserSchema` struct. We also establish a database connection from the connection pool with the `DB` struct. Inside our `create` view function, we extract the data that we need from the `NewUserSchema` struct to create a `NewUser` struct with the following code:

```
let new_user = NewUser::new(
    new_user.name.clone(),
    new_user.email.clone(),
    new_user.password.clone()
);
```

We must clone the fields to be passed into the `NewUser` constructor because strings do not implement the `Copy` trait, meaning we must do this manually. We then create our `insert` command for the database and execute it with the following code:

```
let insert_result = diesel::insert_into(users::table)
                            .values(&new_user)
                            .execute(&db.connection);
```

This returns a `Result` struct. However, we do not unwrap it directly. There could be a conflict. For instance, we could be trying to insert a new user with a username or email that is already in the database. However, we do not want this to just error out. This is an edge case that we will expect as we have implemented the unique username and email constraint ourselves. If there was a legitimate error that happens when the view is being executed, we need to know about it. Therefore, we must give response codes to the edge cases. Therefore, we match the result of the insert and return the appropriate response code with the following code:

```
match insert_result {
    Ok(_) => HttpResponse::Created(),
    Err(_) => HttpResponse::Conflict()
}
```

Here, we have established a database connection, extracted the fields from the JSON body, created a new `NewUser` struct, and then inserted it into the database. There is a slight difference here compared to the other views. In the return response, we are having to *await* and then *unwrap* it. This is because we are not returning a JSON body. Therefore, `HttpResponse::Ok()` is merely a builder struct.

More books: https://t.me/booksforits

Now that we have built our create view, we need to define our view factory in the `views/users/mod.rs` file, like so:

```
mod create;

use actix_web::web::{ServiceConfig, post, scope};


pub fn user_views_factory(app: &mut ServiceConfig) {
    app.service(
        scope("v1/user")
        .route("create", post().to(create::create))
    );
}
```

Again, since we have been building views regularly, none of this should come as a surprise to you. If it does, it is recommended that you read the *Managing views using the Actix Web framework* section in *Chapter 3*, *Handling HTTP Requests*, for clarity. Now, our main views factory in the `views/mod.rs` file should look like the following:

```
mod auth;
mod to_do;
mod app;
mod users;

use auth::auth_views_factory;
use to_do::to_do_views_factory;
use app::app_views_factory;
use users::user_views_factory;

use actix_web::web::ServiceConfig;

pub fn views_factory(app: &mut ServiceConfig) {
    auth_views_factory(app);
    to_do_views_factory(app);
    app_views_factory(app);
    user_views_factory(app);
}
```

More books: https://t.me/booksforits

Now that we have registered our user view, we can run our app and create our user with the following Postman call:

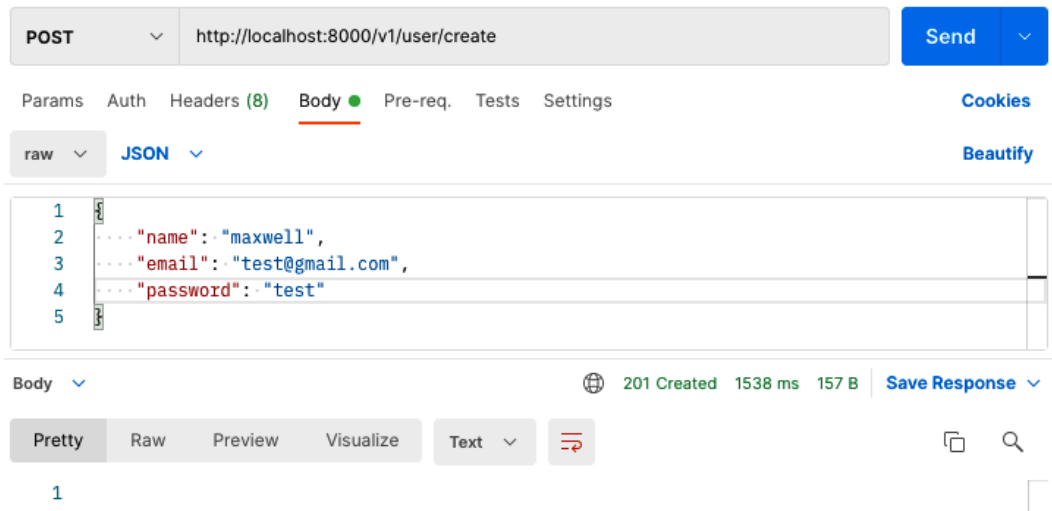


Figure 7.4 – Postman call to our create user endpoint

With this, we should get a `201` created response. If we call the exact same call again, we should get a `409` conflict. With this, we should expect that our new user has been created. With the steps covered in the *Connecting to PostgreSQL with Diesel* section in *Chapter 6*, *Data Persistence with PostgreSQL*, we can inspect the database in our Docker container, which gives us the following printout:

```
 id |     name      |        email
----+---------------+---------------------
  1 | placeholder | placeholder email
  2 | maxwell       | test@gmail.com

                            password
------------------------------------------------------------
 placeholder password
 $2b$12$jlfLwu4AHjrvTpZrB311Y.W0JulQ71WVy2g771xl50e5nS1UfqwQ.

              unique_id
--------------------------------------
 placeholder unique id
 543b7aa8-e563-43e0-8f62-55211960a604
```

More books: https://t.me/booksforits

Here, we can see the initial user that was created in our migration. However, we can also see the user we created via our view. Here, we have a hashed password and a unique ID. From this, we can see that we should never directly create our user; we should only create a user through the constructor function belonging to the `NewUser` struct.

In the context of our app, we do not really need a unique ID. However, in wider situations where multiple servers and databases are used, a unique ID can become useful. We also must note that our conflict response on the second one was correct; the third replica create user call, did not insert a replica user into the database.

With this, our app is running as normal, since there is now a user table with user models linked to the to-do items. Thus, we can create other data tables with relationships and structure migrations so that they can be seamlessly upgraded and downgraded. We have also covered how to verify and create passwords. However, we have not actually written any code that checks whether the user is passing the right credentials. In the next section, we will work on authenticating users and rejecting requests that do not contain the right credentials.

Authenticating our users

When it comes to authenticating our users, we have built a struct that extracts a message from the header of the HTTP request. We are now at the stage where we can make real use of this extraction by storing data about the user in the header. Right now, there is nothing stopping us from storing the username, ID, and password in the header of each HTTP request so that we can authenticate each one. However, this is a terrible practice. If someone intercepts the request or gets hold of the data stored in the browser to facilitate this, then the account is compromised and the hacker can do whatever they want. Instead, we are going to obfuscate the data, as shown in the following figure:

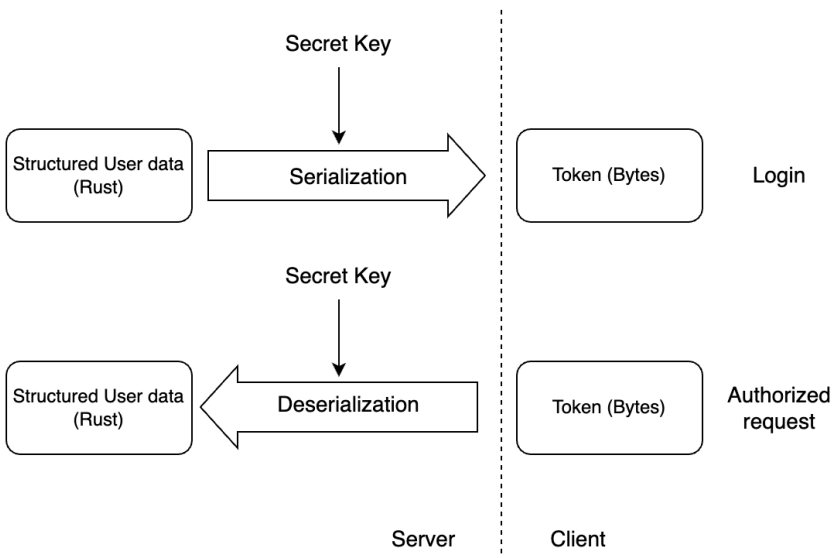


Figure 7.5 – Steps for authenticating requests

More books: https://t.me/booksforits

In *Figure 7.5*, we can see that we use a secret key to serialize the structured data that we have on the user into a token that is in bytes. We then give the token to the user to store in the browser. When the user wants to make an authorized request, the user must send the token in the header of the request. Our server then uses the secret key to deserialize the token back into structured data about the user. The algorithms used to do this process are standard hashing algorithms that are available to anyone. Therefore, we have a secret key that we define to keep the tokens out in the wild safe. For our application to carry out the processes laid out in *Figure 7.5*, we are going to have to rewrite most of our `src/jwt.rs` file, including the `JwToken` struct. Before we start, we need to update our `Cargo.toml` dependencies with the following code:

```
[dependencies]
. . .
chrono = {version = "0.4.19", features = ["serde"]}
. . .
jsonwebtoken = "8.1.0"
```

We can see that we have added the `serde` features to the `chrono` crate and added the `jsonwebtoken` crate. To rebuild the `JwToken` struct, we need to import the following in the `src/jwt.rs` file:

```
use actix_web::dev::Payload;
use actix_web::{Error, FromRequest, HttpRequest};
use actix_web::error::ErrorUnauthorized;
use futures::future::{Ready, ok, err};

use serde::{Deserialize, Serialize};
use jsonwebtoken::{encode, decode, Algorithm, Header,
                   EncodingKey, DecodingKey, Validation};

use chrono::{DateTime, Utc};
use chrono::serde::ts_seconds;
use crate::config::Config;
```

More books: https://t.me/booksforits

We can see that we import `actix_web` traits and structs to enable the processing of the requests and responses. We then import `futures` to enable us to handle the interception of the HTTP request before it hits the views. We then import `serde` and `jsonwebtoken` to enable the serialization and deserialization of data to and from the token. We then import the `chrono` crate because we want to log when these tokens are minted. We also need to have the key for the serialization, and we get this from the config file, which is why we import the `Config` struct. Now that we have imported all the traits and structs that we need, we can write our token struct with the following code:

```
#[derive(Debug, Serialize, Deserialize)]
pub struct JwToken {
    pub user_id: i32,
    #[serde(with = "ts_seconds")]
    pub minted: DateTime<Utc>
}
```

Here, we can see that we have the ID of the user, and we also have the date-time of when the token was created. We also decorate our `minted` field with a `serde` macro to state how we are going to serialize the `datetime` field. Now that we have the data that we need for the token, we can move on to defining the serialization functions with the following code:

```
impl JwToken {

    pub fn get_key() -> String {
        . . .
    }

    pub fn encode(self) -> String {
        . . .
    }

    pub fn new(user_id: i32) -> Self {
        . . .
    }

    pub fn from_token(token: String) -> Option<Self> {
        . . .
    }

}
```

More books: https://t.me/booksforits

We can explain what each one of the preceding functions does with the following bullet points:

- `get_key`: Gets the secret key for the serialization and deserialization from the `config.yml` file.
- `encode`: Encodes the data from the `JwToken` struct as a token
- `new`: Creates a new `JwToken` struct
- `from_token`: Creates a `JwToken` struct from a token. If there is a failure in the deserialization it returns a `None` as there can be failures in deserialization.

Once we have built the preceding functions, our `JwToken` struct will be able to handle tokens as and when we see fit. We flesh out the `get_key` function with the following code:

```
pub fn get_key() -> String {
    let config = Config::new();
    let key_str = config.map.get("SECRET_KEY")
                            .unwrap().as_str()
                            .unwrap();
    return key_str.to_owned()
}
```

Here, we can see that we load the key from the config file. Therefore, we need to add the key to the `config.yml` file, resulting in our file looking like this:

```
DB_URL: postgres://username:password@localhost:5433/to_do
SECRET_KEY: secret
```

If our server is in production, we should have a better secret key. However, for local development, this will work fine. Now that we are extracting the key from the config file, we can define our `encode` function with the following code:

```
pub fn encode(self) -> String {
    let key = EncodingKey::
              from_secret(JwToken::get_key().as_ref());
    let token = encode(&Header::default(), &self,
                       &key).unwrap();
    return token
}
```

More books: https://t.me/booksforits

Here, we can see that we have defined an encoding key using the secret key from the config file. We then use this key to encode the data from the `JwToken` struct into a token and return it. Now that we can encode our `JwToken` struct, we will need to create new `JwToken` structs when we need them, which can be achieved by the following `new` function:

```
pub fn new(user_id: i32) -> Self {
    let timestamp = Utc::now();
    return JwToken { user_id, minted: timestamp};
}
```

With the constructor, we know when our `JwToken` is minted. This can help us manage our user sessions if we want. For instance, if the token's age exceeds a threshold that we deem appropriate, we can force another login.

Now, all we have is the `from_token` function, where we extract the data from a token using the following code:

```
pub fn from_token(token: String) -> Option<Self> {
    let key = DecodingKey::from_secret(
                JwToken::get_key().as_ref()
              );
    let token_result = decode::<JwToken>(
                         &token, &key,
                         &Validation::new(Algorithm::HS256)
                         );
    match token_result {
        Ok(data) => {
            Some(data.claims)
        },
        Err(_) => {
            return None
        }
    }
}
```

More books: https://t.me/booksforits

Here, we define a decoding key and then use it to decode the token. We then return `JwToken` using `data.claims`. Now, our `JwToken` struct can be created, encoded into a token, and extracted from a token. Now, all we need to do is extract it from the header of an HTTP request before the view is loaded, using the following outline:

```
impl FromRequest for JwToken {
    type Error = Error;
    type Future = Ready<Result<JwToken, Error>>;

    fn from_request(req: &HttpRequest,
                    _: &mut Payload) -> Self::Future {
        . . .
    }
}
```

We have implemented the `FromRequest` trait multiple times now for the database connection and the previous implementation for the `JwToken` struct. Inside the `from_request` function, we extract the token from the header with the following code:

```
match req.headers().get("token") {
    Some(data) => {
        . . .
    },
    None => {
        let error = ErrorUnauthorized(
                    "token not in header under key 'token'"
                    );
        return err(error)
    }
}
```

If the token is not in the header, we directly return `ErrorUnauthorized`, avoiding the call to the view completely. If we manage to extract the token from the header, we can process it with the following code:

```
Some(data) => {
    let raw_token = data.to_str()
                        .unwrap()
                        .to_string();
```

More books: https://t.me/booksforits

```
    let token_result = JwToken::from_token(
                                raw_token
                            );
    match token_result {
        Some(token) => {
            return ok(token)
        },
        None => {
            let error = ErrorUnauthorized(
                                "token can't be decoded"
                        );
            return err(error)
        }
    }
},
```

Here, we convert the raw token extracted from the header to a string. We then deserialize the token and load it into the `JwToken` struct. However, if this fails due to a fake token being supplied, we return an `ErrorUnauthorized` error. Our authentication is now fully working; however, we will not be able to do anything because we do not have a valid token, as shown in the following figure:

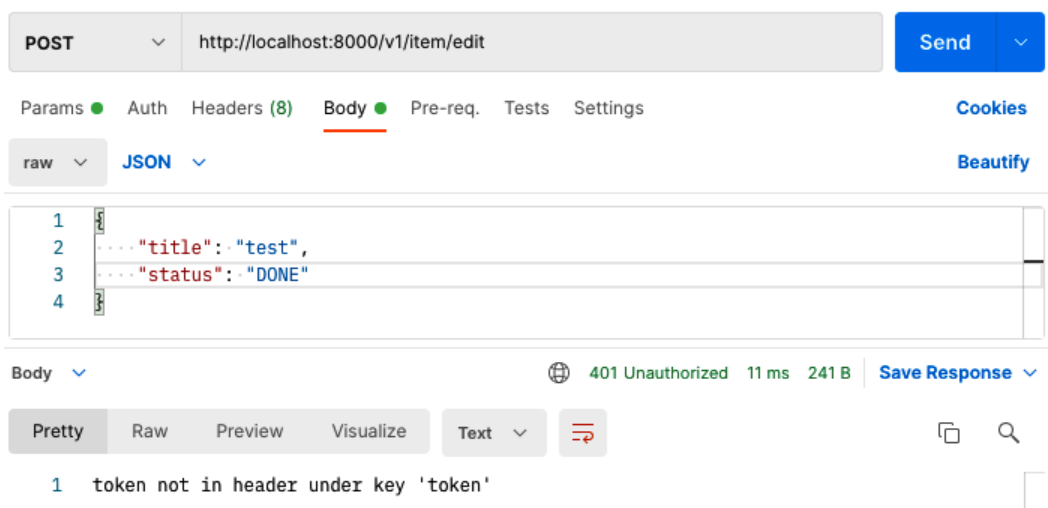


Figure 7.6 – Authentication blocking requests

In the next section, we will build login API endpoints to enable us to interact with our protected endpoints.

More books: https://t.me/booksforits

Managing user sessions

For our users, we are going to have to enable them to log in. This means that we must create an endpoint to check their credentials and then generate a JWT to be returned to the user in the frontend, via the header in the response. Our first step is to define a login schema in the `src/json_serialization/login.rs` file with the following code:

```
use serde::Deserialize;

#[derive(Deserialize)]
pub struct Login {
    pub username: String,
    pub password: String
}
```

We have to remember to register it in the `src/json_serialization/mod.rs` file with the `pub mod login;` line of code. Once we have done this, we can build our login endpoint. We can do this by editing the `src/views/auth/login.rs` file we created in the *Managing views using the Actix Web framework* section in *Chapter 3*, *Handling HTTP Requests*, which declares our basic login view. This just returns a string.

Now, we can start refactoring this view by defining the required imports, as shown in the following code:

```
use crate::diesel;
use diesel::prelude::*;
use actix_web::{web, HttpResponse, Responder};

use crate::database::DB;
use crate::models::user::user::User;
use crate::json_serialization::login::Login;
use crate::schema::users;
use crate::jwt::JwToken;
```

At this stage, we can glance at the imports and get a feel for what we are going to do. We are going to extract the username and password from the body. We are then going to connect to the database to check the user and password, and then use the `JwToken` struct to create the token that will be passed back to the user. We can initially lay out the outline of the view with the following code in the same file:

```
. . .
Use std::collections::HashMap;
```

More books: https://t.me/booksforits

```
pub async fn login(credentials: web::Json<Login>,
                   db: DB) -> impl HttpResponse {
    . . .
}
```

Here, we can see that we accept the login credentials from the body of the incoming request and prepare a database connection from the connection pool for the view. We can then extract the details we need from the request body and make a database call with the following code:

```
let password = credentials.password.clone();
let users = users::table
    .filter(users::columns::username.eq(
        credentials.username.clone())
    ).load::<User>(&db.connection).unwrap();
```

Now, we must check to see whether we got what we expected from the database call with the following code:

```
if users.len() == 0 {
    return HttpResponse::NotFound().await.unwrap()
} else if users.len() > 1 {
    return HttpResponse::Conflict().await.unwrap()
}
```

Here, we have done some early returns. If there are no users, then we return a `not found` response code. This is something we will expect from time to time. However, if there is more than one user with that username, we need to return a different code.

Due to the unique constraints shown, something is very wrong. A migration script in the future might undo these unique constraints, or the user query might be altered by accident. If this happens, we need to know that this has happened right away, since corrupted data that goes against our constraints can cause our application to behave in unexpected ways that can be hard to troubleshoot.

Now that we have checked that the right number of users have been retrieved, we can get the one and only user at index zero with confidence and check whether their password is passable, as follows:

```
match users[0].verify(password) {
    true => {
        let token = JwToken::new(users[0].id);
```

More books: https://t.me/booksforits

```
        let raw_token = token.encode();
        let mut body = HashMap::new();
      body.insert("token", raw_token);
        HttpResponse::Ok().json(body)
    },
    false => HttpResponse::Unauthorized()
}
```

Here, we can see that we used the `verify` function. If the password is a match, we then generate a token using the ID and return it to the user in the body. If the password is not correct, we return an unauthorized code instead.

In terms of our logout, we are going to take a far more lightweight approach. All we must do in our logout view is run two lines of JavaScript code. One is to remove the user token from the local storage and then revert the user to the main view. HTML can just host JavaScript that gets run as soon as you open it. Therefore, we can achieve this by putting the following code in the `src/views/auth/logout.rs` file:

```
use actix_web::HttpResponse;


pub async fn logout() -> HttpResponse {
    HttpResponse::Ok()
        .content_type("text/html; charset=utf-8")
        .body("<html>\
                <script>\
                    localStorage.removeItem('user-token'); \
                    window.location.replace(
                        document.location.origin);\
                </script>\
              </html>")
}
```

Since this view is already registered, we can run the app and make the call with Postman:

More books: https://t.me/booksforits

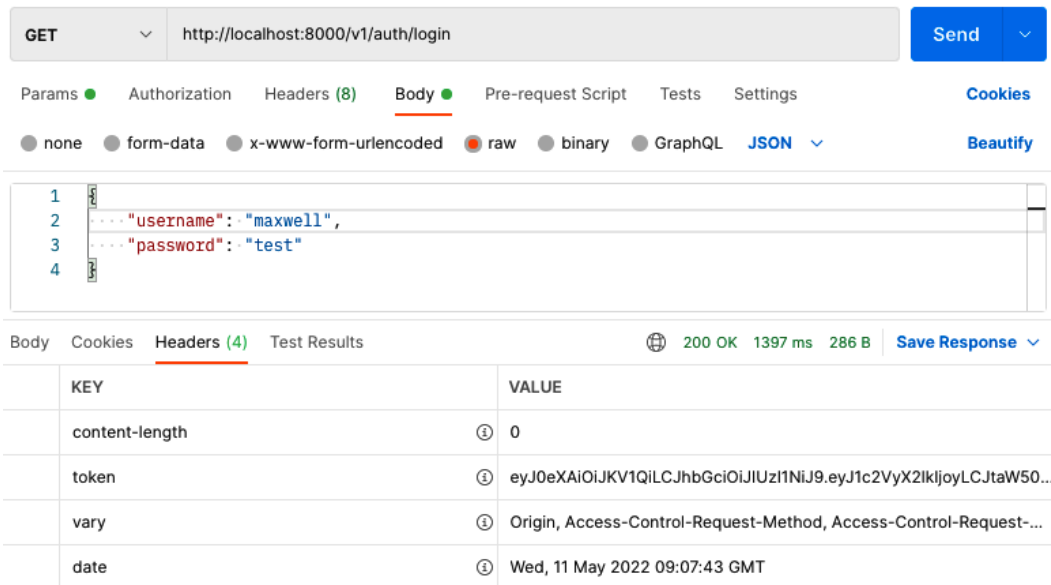


Figure 7.7 – Logging into our application using the login endpoint with Postman

Altering the username will give us a `404-response` code, whereas altering the password will give us a `401-response` code. If we have the correct username and password, we will get a `200-response` code and there will be a *token* in the response of the header, as shown in *Figure 7.7*. However, if we want to use our *token* in the response header, we will get a `token can't be decoded` message. In the next section, we are going to clean up our authentication requirements.

Cleaning up authentication requirements

In this section, we are going to clean up our Rust server in terms of authentication before we start configuring our frontend to handle these authentication processes. To keep the flow of the chapter engaging, we have not regularly carried out "housekeeping." Now, we are going to update our `to_do` views. We can start by updating the `create` view with authentication requirements. To do so, the function signature of our `create` view in the `src/views/to_do/create.rs` file should look like the following:

```
. . .
use crate::jwt::JwToken;
use crate::database::DB

pub async fn create(token: JwToken,
                    req: HttpRequest, db: DB) -> HttpResponse {
    . . .
```

More books: https://t.me/booksforits

We also must update the user ID when creating a new item with the ID from the token, using the following code:

```
if items.len() == 0 {
    let new_post = NewItem::new(title, token.user_id);
    let _ = diesel::
            insert_into(to_do::table).values(&new_post)
        .execute(&db.connection);
}
Return HttpResponse::Ok().json(
    ToDoItems::get_state(token.user_id)
)
```

With our `delete` view, we must ensure that we are deleting a to-do item that belongs to the user making the request. If we do not add a filter using the user ID, the deletion of the to-do item will be random. This filter can be added in our `src/views/to_do/delete.rs` file with the following code:

```
. . .
Use crate::database::DB;
. . .

pub async fn delete(to_do_item: web::Json<ToDoItem>,
                    token: JwToken, db: DB) -> HttpResponse {
    let items = to_do::table
        .filter(to_do::columns::title.eq(
                    &to_do_item.title.as_str())
                )
        .filter(to_do::columns::user_id.eq(&token.user_id))
        .order(to_do::columns::id.asc())
        .load::<Item>(&db.connection)
        .unwrap();
    let _ = diesel::delete(&items[0]).execute(&db.connection);
    return HttpResponse::Ok().json(ToDoItems::get_state(
        token.user_id
    ))
}
```

More books: https://t.me/booksforits

We can see that the `filter` functions can merely be chained when making a database query. Considering what we have done with our `delete` view, how do you think we will upgrade our authentication requirements for our `edit` in the `src/views/to_do/edit.rs` file? At this stage, I encourage you to try and update the `edit` view yourself, as the approach is like our `delete` view upgrade. Once you have done this, your `edit` view should look like the following code:

```
pub async fn edit(to_do_item: web::Json<ToDoItem>,
                  token: JwToken, db: DB) -> HttpResponse {
    let results = to_do::table.filter(to_do::columns::title
                              .eq(&to_do_item.title))
                              .filter(to_do::columns::user_
                                      id
                              .eq(&token.user_id));

    let _ = diesel::update(results)
        .set(to_do::columns::status.eq("DONE"))
        .execute(&db.connection);

    return HttpResponse::Ok().json(ToDoItems::get_state(
                                    token.user_id
    ))
}
```

Now that we have updated our specific views, we can now move on to the `get` view, which also has the `get_state` function that is applied to all other views. Our `get` view in the `src/views/to_do/get.rs` file now takes the following form:

```
use actix_web::Responder;
use crate::json_serialization::to_do_items::ToDoItems;
use crate::jwt::JwToken;

pub async fn get(token: JwToken) -> impl Responder {
    ToDoItems::get_state(token.user_id)
}
```

More books: https://t.me/booksforits

Now, everything in the preceding code should not be a surprise. We can see that we pass the user ID into the `ToDoItems::get_state` function. You must remember to fill in the user ID everywhere the `ToDoItems::get_state` function is implemented, which is all the to-do views. We can then redefine our `ToDoItems::get_state` function in the `src/json_serialization/to_do_items.rs` file with the following code:

```
. . .
use crate::database::DBCONNECTION;
. . .

impl ToDoItems {
    . . .

    pub fn get_state(user_id: i32) -> ToDoItems {
        let connection = DBCONNECTION.db_connection.get()
                         .unwrap();
        let items = to_do::table
                    .filter(to_do::columns::user_id.eq
                           (&user_id))
                    .order(to_do::columns::id.asc())
                    .load::<Item>(&connection)
                    .unwrap();
        let mut array_buffer = Vec::
                               with_capacity(items.len());
        for item in items {
            let status = TaskStatus::from_string(
            &item.status.as_str().to_string());
            let item = to_do_factory(&item.title, status);
            array_buffer.push(item);
        }
        return ToDoItems::new(array_buffer)
    }
}
```

More books: https://t.me/booksforits

Here, we can see that we have updated the database connection and the filter for the user ID. We have now updated our code to accommodate different users. There is one more change that we must make. Because we will be writing frontend code in our React application, we will try and keep the React coding as simple as possible, as React development is a book itself. To avoid over-complicating the frontend development of header extraction and `GET` posts using Axios, we will add a `Post` method to our login and return the token using the body. This is another good opportunity to try and solve this yourself, as we have covered all the concepts needed to pull this off.

If you have attempted to solve this problem yourself, it should look like the following. First, we define a response struct in the `src/json_serialization/login_response.rs` file with the following code:

```
use serde::Serialize;

#[derive(Serialize)]
pub struct LoginResponse {
    pub token: String
}
```

We remember to declare the preceding struct by putting in `pub mod login_response` in the `src/json_serialization/mod.rs` file. We now go to our `src/views/auth/login.rs` and have the following `return` statement in the `login` function:

```
match users[0].clone().verify(credentials.password.clone()) {
    true => {
        let user_id = users[0].clone().id;
        let token = JwToken::new(user_id);
        let raw_token = token.encode();
        let response = LoginResponse{token:
                                     raw_token.clone()};
        let body = serde_json::
                   to_string(&response).unwrap();
        HttpResponse::Ok().append_header(("token",
                            raw_token)).json(&body)
    },
    false => HttpResponse::Unauthorized().finish()
}
```

More books: https://t.me/booksforits

Note

You may have noticed that we made a slight change to our unauthorized to the following:

```
HttpResponse::Unauthorized().finish()
```

This is because we have switched our `return` type for the view function to an `HttpResponse` struct giving us the following function signature:

```
(credentials: web::Json<Login>, db: DB) -> HttpResponse
```

We had to make the switch because adding the `json` function to our response turns our response from `HttpResponseBuilder` to `HttpResponse`. Once the `json` function has been called, `HttpResponseBuilder` cannot be used. Going back to the unauthored response builder, we can deduce that the `finish` function converts `HttpResponseBuilder` to `HttpResponse`. We can also convert our `HttpResponseBuilder` to `HttpResponse` by using `await`, as shown in the following code:

```
HttpResponse::Unauthorized().await.unwrap()
```

Here, we can see that we return the token in the header and the body. This will give us flexibility and ease when writing the frontend code. However, it must be stressed that this is not the best practice. We are implementing the approach of passing the token back into the body and header to keep the frontend development section simple. We can then enable the `POST` method for our login view in our `src/views/auth/mod.rs` file with the following code:

```
mod login;
mod logout;
use actix_web::web::{ServiceConfig, get, post, scope};

pub fn auth_views_factory(app: &mut ServiceConfig) {
    app.service(
            scope("v1/auth")
            .route("login", get().to(login::login))
            .route("login", post().to(login::login))
            .route("logout", get().to(logout::logout))
    );
}
```

We can see that we have merely stacked a `get` function onto the same `login` view. Now, `POST` and `GET` are available for our login view. We can now move into the next section where we configure our authentication tokens so they can expire. We want our tokens to expire to increase our security. If a token is compromised and a bad actor gets hold of a token, they will be able to do whatever they want for as long as they want without ever having to log in. However, if our tokens expire, then the bad actor only has a limited time window before the token expires.

More books: https://t.me/booksforits

Configuring expiration of auth tokens

If we try and perform an API call on our now protected endpoints with a valid token obtained from logging in with the token in the header, we will get an unauthorized error. If we insert some `print` statements, we will get the following error when failing to decode the token:

```
missing required claim: exp
```

This implies that there is no field called `exp` in our `JwToken` struct. If we reference the `jsonwebtoken` documentation at `https://docs.rs/jsonwebtoken/latest/jsonwebtoken/fn.encode.html`, we can see that the `encode` instructions never mention `exp`:

```
use serde::{Deserialize, Serialize};
use jsonwebtoken::{encode, Algorithm, Header, EncodingKey};

#[derive(Debug, Serialize, Deserialize)]
struct Claims {
   sub: String,
   company: String
}

let my_claims = Claims {
    sub: "b@b.com".to_owned(),
    company: "ACME".to_owned()
};

// my_claims is a struct that implements Serialize
// This will create a JWT using HS256 as algorithm
let token = encode(&Header::default(), &my_claims,
&EncodingKey::from_secret("secret".as_ref())).unwrap();
```

Here, we can see that there is no mention of claims. However, what is happening is that when we try and deserialize our token, the `decode` function in the `jsonwebtoken` crate is automatically looking for the `exp` field to work out when the token is supposed to be expired. We are exploring this because the official documentation and slightly confusing error message could leave you wasting hours trying to figure out what is going on. With this in mind, we must go back to our `src/jwt.rs` file for some more rewriting, but this is the last time, I promise, and it is not an entire rewrite. First, we ensure that the following is imported alongside what is already in the `src/jwt.rs` file:

```
. . .
use jsonwebtoken::{encode, decode, Header,
```

More books: https://t.me/booksforits

```
                    EncodingKey, DecodingKey,
                    Validation};
use chrono::Utc;
. . .
```

We can then make sure our `JwToken` struct is written with the `exp` field with the following code:

```
#[derive(Debug, Serialize, Deserialize)]
pub struct JwToken {
    pub user_id: i32,
    pub exp: usize,
}
```

We now must rewrite the `new` constructor method for our `JwToken` struct. In the `new` function, we will have to define at what time the newly minted `JwToken` struct has expired. This must vary; as a developer, you might want to tweak the time taken to timeout. Remember that we must recompile every time we change the Rust code; therefore, it makes sense to have the timeout period defined in the config file. With the variance of timeout considered, our `new` function takes the following form:

```
pub fn new(user_id: i32) -> Self {
    let config = Config::new();
    let minutes = config.map.get("EXPIRE_MINUTES")
                            .unwrap().as_i64().unwrap();
    let expiration = Utc::now()
    .checked_add_signed(chrono::Duration::minutes(minutes))
                            .expect("valid timestamp")
                            .timestamp();
    return JwToken { user_id, exp: expiration as usize };
}
```

We can see that we define the number of minutes. We then convert our expiration as `usize` and then build our `JwToken` struct. Now that we have this, we need to be more specific with the type of error that we return, as it could be an error in the decoding of the token, or the token could be expired. We handle the different types of errors when decoding the token with the following code:

```
pub fn from_token(token: String) -> Result<Self, String> {
    let key = DecodingKey::
              from_secret(JwToken::get_key().as_ref());
    let token_result = decode::<JwToken>(&token.as_str(),
                               &key,&Validation::default());
```

More books: https://t.me/booksforits

```
    match token_result {
        Ok(data) => {
            return Ok(data.claims)
        },
        Err(error) => {
            let message = format!("{}", error);
            return Err(message)
        }
    }
}
```

Here, we can see that we have switched from returning `Option` to `Result`. We have switched to `Result` because we are returning the message that can be digested and processed in our `from_request` function in the `FromRequest` trait implementation. The rest of the code in the `from_request` function is the same. Where we make the change is checking the message if there is an error and returning a different message to the frontend with the following code:

```
fn from_request(req: &HttpRequest,
                _: &mut Payload) -> Self::Future {
    match req.headers().get("token") {
        Some(data) => {
            let raw_token = data.to_str()
                                .unwrap()
                                .to_string();
            let token_result = JwToken::
                        from_token(raw_token);

            match token_result {
                Ok(token) => {
                    return ok(token)
                },
                Err(message) => {
                    if message == "ExpiredSignature"
                                  .to_owned() {
                        return err(
                        ErrorUnauthorized("token expired"))
                    }
                    return err(
```

More books: https://t.me/booksforits

```
                    ErrorUnauthorized("token can't be
decoded"))
                }
            }
        },
        None => {
            return err(
            ErrorUnauthorized(
            "token not in header under key 'token'"))
        }
    }
}
```

With the nuanced error message, our frontend code can handle and adapt, as we can be more specific on how we handle the errors in the frontend. Being more specific in the frontend can aid the user, prompting them where they went wrong. However, when it comes to authentication, make sure you do not give too much away because this can also aid bad actors trying to obtain unauthorized access. We now have our login and logout endpoints running; we also have token authorization on the views that we need. However, this is not very useful if we want a standard user to interact with our application, as they are unlikely to use Postman. Therefore, we are going to have to incorporate our login/logout endpoints in the frontend in the next section.

Adding authentication into our frontend

We incorporate our login functionality. We must start off by building the login form in the `src/components/LoginForm.js` file. First, we import the following:

```
import React, {Component} from 'react';
import axios from 'axios';
import '../css/LoginForm.css';
```

The code for the imported CSS is provided in the *Appendix* of this chapter. We will not go through it here, as it is a lot of repetitive code. You can also download the CSS code from the GitHub repo. With these imports, we can build the framework for our login form with the following code:

```
class LoginForm extends Component {
    state = {
        username: "",
        password: "",
    }
```

More books: https://t.me/booksforits

```
    submitLogin = (e) => {
        . . .
    }
    handlePasswordChange = (e) => {
        this.setState({password: e.target.value})
    }
    handleUsernameChange = (e) => {
        this.setState({username: e.target.value})
    }
    render() {
        . . .
    }
}
export default LoginForm;
```

Here, we can see that we keep track of `username` and `password` that is constantly updating the state. Remember, when the state is updated, we execute the `render` function. This is powerful, as we can change whatever we want. For instance, if the length of `username` exceeds a certain length, we can change the color of the components or remove the button. We will not be making drastic changes ourselves, as this is out of the scope of this book. Now that we have defined our framework, we can state what our `render` function returns with the following code:

```
<form className="login" onSubmit={this.submitLogin}>
    <h1 className="login-title">Login</h1>
    <input type="text" className="login-input"
    placeholder="Username"
    autoFocus onChange={this.handleUsernameChange}
           value={this.state.username} />
    <input type="password" className="login-input"
    placeholder="Password"
    onChange={this.handlePasswordChange}
           value={this.state.password} />
    <input type="submit" value="Lets Go"
    className="login-button" />
</form>
```

More books: https://t.me/booksforits

Here, we can see that we have the `username` and `password` fields in the form that execute the `handleUsernameChange` and `handlePasswordChange` functions when there is a change. When we input `username` and `password`, we need to submit these fields to the backend via the `submitLogin` function, which we can define here:

```
submitLogin = (e) => {
    e.preventDefault();
    axios.post("http://localhost:8000/v1/auth/login",
        {"username": this.state.username,
         "password": this.state.password},
        {headers: {"Access-Control-Allow-Origin": "*"}}
        )
        .then(response => {
            this.setState({username: "", password: ""});
            this.props.handleLogin(response.data["token"]);
        })
        .catch(error => {
            alert(error);
            this.setState({password: "", firstName: ""});
        });
}
```

Here, we can see that we pass the response of the login API call to a function that we passed through using props. We will have to define this in the `src/app.js` file. If there is an error, we print this out in an alert to tell us what happened. Either way, we empty the `username` and `password` fields.

Now that we have defined our login form, we will need to show it when we need the user to log in. Once the user has logged in, we need to hide the login form. Before we can do this, we need to import our login form to the `src/app.js` file with the following code:

```
import LoginForm from "./components/LoginForm";
```

We now need to keep track of the login status. To do this, our `App` class's state needs to take the following form:

```
state = {
  "pending_items": [],
  "done_items": [],
  "pending_items_count": 0,
```

More books: https://t.me/booksforits

```
  "done_items_count": 0,
  "login_status": false,
}
```

We are keeping track of our items, but if `login_status` is `false`, we can show the login form. Once the user has logged in, we can set `login_status` to `true`, and as a result, we can hide the login form. Now that we are logging the login status, we can update the `App` class's `getItems` function:

```
getItems() {
  axios.get("http://127.0.0.1:8000/v1/item/get",
  {headers: {"token": localStorage.getItem("user-token")}})
  .then(response => {
      let pending_items = response.data["pending_items"]
      let done_items = response.data["done_items"]
      this.setState({
        "pending_items":
         this.processItemValues(pending_items),
        "done_items": this.processItemValues(done_items),
        "pending_items_count":
         response.data["pending_item_count"],
        "done_items_count":
         response.data["done_item_count"]
        })
  }).catch(error => {
      if (error.response.status === 401) {
        this.logout();
      }
  });
}
```

We can see that we get the token and put it in the header. If there is an error with unauthorized code, we execute the `logout` function of the `App` class. Our `logout` function takes the form defined here:

```
logout() {
  localStorage.removeItem("token");
  this.setState({"login_status": false});
}
```

More books: https://t.me/booksforits

We can see that we remove the token from the local storage and set our `login_status` to `false`. This `logout` function also needs to be executed if there is an error when trying to edit a to-do item, as we must remember that our token can expire so it can happen anywhere, and we must prompt another login. This means we must pass the `logout` function into the `ToDoItem` component with the following code:

```
processItemValues(items) {
  let itemList = [];
  items.forEach((item, _)=>{
      itemList.push(
          <ToDoItem key={item.title + item.status}
                    title={item.title}
                    status={item.status}
                    passBackResponse={this.handleReturnedState}
                    logout={this.logout}/>
      )
  })
  return itemList
}
```

Once we have passed our `logout` function into the `ToDoItem` component, we can update the API call to edit a to-do item in the `src/components/ToDoItem.js` file with the following code:

```
sendRequest = () => {
    axios.post("http://127.0.0.1:8000/v1/item/" +
                this.state.button,
        {
            "title": this.state.title,
            "status": this.inverseStatus(this.state.status)
        },
    {headers: {"token": localStorage.getItem(
         "user-token")}})
        .then(response => {
            this.props.passBackResponse(response);
        }).catch(error => {
            if (error.response.status === 401) {
                this.props.logout();
```

More books: https://t.me/booksforits

```
            }
    });
}
```

Here, we can see that we pass the token from local storage to the API call via the header. We then execute the `logout` function passed in via the props if we get an unauthorized status.

We now move back to the `src/app.js` file to wrap up the functionality of our application. Remember that our application needs to load data when we first access it. When our application initially loads, we must consider the token in the local storage with the following code:

```
componentDidMount() {
  let token = localStorage.getItem("user-token");

  if (token !== null) {
      this.setState({login_status: true});
      this.getItems();
  }
}
```

Now our application will only get the items from the backend when there is a token. We must only handle the login before we wrap up our application with the `render` function. You have seen how we are handling our token with local storage. At this point, you should be able to build the `handleLogin` function for the `App` class yourself. If you have attempted coding your own function, it should look like the following code:

```
handleLogin = (token) => {
  localStorage.setItem("user-token", token);
  this.setState({"login_status": true});
  this.getItems();
}
```

We are now at the stage of defining the `render` function for the `App` class. If our login status is `true`, we can show everything our application has to offer with the following code:

```
if (this.state.login_status === true) {
    return (
        <div className="App">
            <div className="mainContainer">
                <div className="header">
                    <p>complete tasks:
```

More books: https://t.me/booksforits

```
                        {this.state.done_items_count}</p>
                    <p>pending tasks:
                        {this.state.pending_items_count}</p>
                </div>
                <h1>Pending Items</h1>
                {this.state.pending_items}
                <h1>Done Items</h1>
                {this.state.done_items}
                <CreateToDoItem
                 passBackResponse={this.handleReturnedState}/>
            </div>
        </div>
    )
}
```

There is not too much new here. However, if our login status is not `true`, we can then just display the login form with the following code:

```
else {
    return (
        <div className="App">
            <div className="mainContainer">
                <LoginForm handleLogin={this.handleLogin}
                />
            </div>
        </div>
    )
}
```

As we can see, we have passed the `handleLogin` function into the `LoginForm` component. With this, we are ready to run the application. Our first view looks like the following:

More books: https://t.me/booksforits

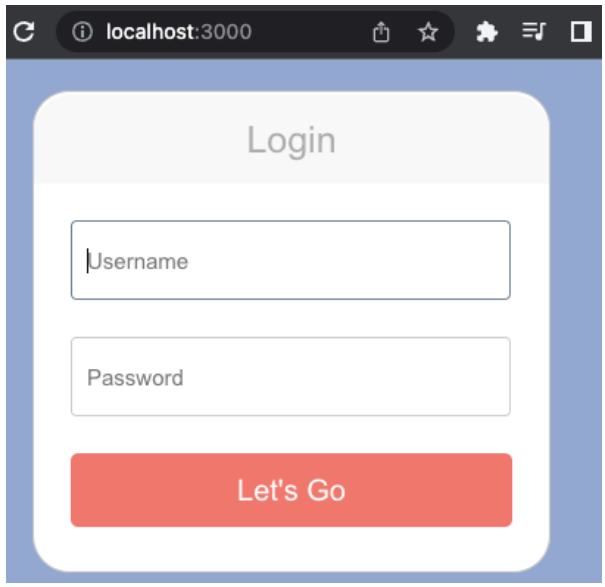


Figure 7.8 – Login and thus the loading view of our application

Once we enter the correct credentials, we will be able to access the application and interact with the to-do items. Our application is essentially working!

Summary

In this chapter, we built user data model structs and tied them to the to-do item data models in our migrations. We then got to dive a little deeper into our migrations by firing multiple steps in the SQL file to ensure our migration runs smoothly. We also explored how to add unique constraints to certain fields.

Once our data models were defined in the database, we hashed some passwords before storing them in our database with the stored user. We then created a JWT struct to enable our users to store their JWT in their browsers so that they can submit them when making an API call. We then explored how to redirect the URL in JavaScript and the HTML storage so that the frontend can work out whether the user even has credentials, before it entertains the notion of sending API calls to the items.

What we have done here is alter the database with migration so that our app can manage data models that handle more complexity. We then utilized frontend storage to enable our user to pass credentials. This is directly applicable to any other Rust web project you will embark on. Most web apps require some sort of authentication.

In the next chapter, we will explore **REST API** practices, where we will standardize interfaces, caching, and logging.

More books: https://t.me/booksforits

Questions

1. What are the advantages of defining unique constraints in SQL as opposed to server-side code?
2. What is the main advantage of a user having a JWT over storing a password?
3. How does a user store a JWT on the frontend?
4. How could a JWT be useful in the view once we have verified that it is passable?
5. What is the minimal approach to altering data in the frontend and redirecting it to another view when a user hits an endpoint?
6. Why is it useful to have a range of different response codes when logging in as a user, as opposed to just denoting that login is successful or unsuccessful?

Answers

1. Adding unique constraints directly on a database ensures that this standard is enforced, no matter whether data manipulation is done via migration or a server request. This also protects us from corrupting data if a new feature is added at another endpoint that forgets to enforce this standard, or if the code is altered in later alterations of the endpoints.
2. If an attacker manages to obtain a JWT, it does not mean that they have direct access to the user's password. Also, if the tokens get refreshed, then the access the attacker has to items has a limited timeframe.
3. The JWT can be stored in local HTML storage or cookies.
4. We can store multiple data points in the token when hashing it. Therefore, we can encrypt the user ID. With this, we can extract the user ID for operations related to the to-do item's creation, deletion, or edit.
5. We return an `HttpResponse` struct with HTML/text body that contains a string housing a couple of HTML tags. In between these tags are a couple of script tags. In between the script tags, we can have each of our JavaScript commands split by ;. We can then directly alter the HTML storage and window location.
6. There could be a range of reasons why data gets corrupted on a database, including alterations in the migrations. However, there could be an error that is not the fault of the user – for instance, a duplicate username for two different users. This is an error where our unique constraints have been violated. We need to know this has happened so that we can correct it.

Further reading

JWT standard: `https://tools.ietf.org/html/rfc7519`

More books: https://t.me/booksforits

Appendix

The CSS used for the login form:

```
body {
    background: #2d343d;
}

.login {
    margin: 20px auto;
    width: 300px;
    padding: 30px 25px;
    background: white;
    border: 1px solid #c4c4c4;
    border-radius: 25px;
}

h1.login-title {
    margin: -28px -25px 25px;
    padding: 15px 25px;
    line-height: 30px;
    font-size: 25px;
    font-weight: 300;
    color: #ADADAD;
    text-align:center;
    background: #f7f7f7;
    border-radius: 25px 25px 0px 0px;

}

.login-input {
    width: 285px;
    height: 50px;
    margin-bottom: 25px;
    padding-left:10px;
    font-size: 15px;
    background: #fff;
    border: 1px solid #ccc;
```

More books: https://t.me/booksforits

```
    border-radius: 4px;
}
.login-input:focus {
    border-color:#6e8095;
    outline: none;
}
.login-button {
    width: 100%;
    height: 50px;
    padding: 0;
    font-size: 20px;
    color: #fff;
    text-align: center;
    background: #f0776c;
    border: 0;
    border-radius: 5px;
    cursor: pointer;
    outline:0;
}

.login-lost
{
    text-align:center;
    margin-bottom:0px;
}

.login-lost a
{
    color:#666;
    text-decoration:none;
    font-size:13px;
}

.loggedInTitle {
    font-family: "Helvetica Neue";
    color: white;
}
```

More books: https://t.me/booksforits

More books: https://t.me/booksforits

8

Building RESTful Services

Our to-do application, written in Rust, technically works. However, there are some improvements that we need to make. In this chapter, we will apply these improvements as we explore the concepts of the **RESTful API** design.

In this chapter, we will finally reject unauthorized users before the request hits the view by assessing the layers of our system and refactoring how we handle requests throughout the request lifetime. We'll then use this authentication to enable individual users to have their own list of to-do items. Finally, we will log our requests so that we can troubleshoot our application and get a deeper look into how our application runs, caching data in the frontend to reduce API calls. We will also explore nice-to-have concepts such as executing code on command and creating a uniform interface to split the frontend URLs from the backend URLs.

In this chapter, we will cover the following topics:

- What are RESTful services?
- Mapping our layered system
- Building a uniform interface
- Implementing statelessness
- Logging our server traffic
- Caching
- Code on demand

By the end of this chapter, we will have refactored our Rust application to support the principles of RESTful APIs. This means that we are going to map out the layers of our Rust application, create uniform API endpoints, log requests in our application, and cache results in the frontend.

More books: https://t.me/booksforits

Technical requirements

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter08`.

What are RESTful services?

REST stands for **representational state transfer**. It is an architectural style for our **application programming interface** (**API**) to read (`GET`), update (`PUT`), create (`POST`), and delete (`DELETE`) our users and to-do items. The goal of a RESTful approach is to increase speed/performance, reliability, and the ability to grow by reusing components that can be managed and updated without affecting the system.

You may have noticed that before Rust, slow, high-level languages seemed to be a wise choice for web development. This is because they are quicker and safer to write. This is due to the main bottleneck for the speed of processing data in web development being the network connection speed. The RESTful design aims to improve the speed by economizing the system, such as reducing API calls, as opposed to just focusing on algorithm speed. With that in mind, in this section, we will be covering the following RESTful concepts:

- **Layered system**: This enables us to add extra functionality, such as authorization, without having to change the interface. For instance, if we must check the **JSON Web Token** (**JWT**) in every view, then this is a lot of repetitive code that is hard to maintain and is prone to error.
- **Uniform system**: This simplifies and decouples the architecture, enabling whole parts of the application to evolve independently without clashing.
- **Statelessness**: This ensures that our application does not directly save anything on the server. This has implications for microservices and cloud computing.
- **Logging**: This enables us to peek into our application and see how it runs, exposing undesirable behavior even if there are no errors displayed.
- **Caching**: This enables us to store data in the frontend to reduce the number of API calls to our backend API.
- **Code on demand**: This is where our backend server directly runs code on the frontend.

We'll look at the layered system concept in the next section.

Mapping our layered system

A layered system consists of layers with different units of functionality. It could be argued that these layers are different servers. This can be true in microservices and big systems. This can be the case when it comes to different layers of data. In big systems, it makes sense to have *hot data* that gets

More books: https://t.me/booksforits

accessed and updated regularly and *cold data* where it is rarely accessed. However, while it is easy to think of layers as on different servers, they can be on the same server. We can map our layers with the following diagram:

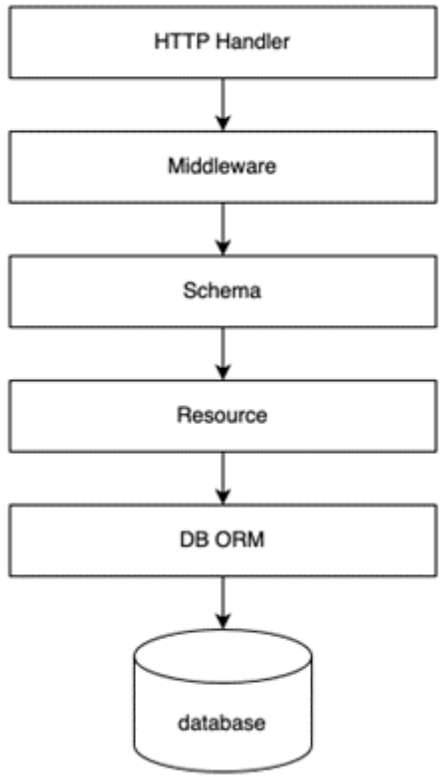


Figure 8.1 – The layers in our app

As you can see, our app follows this process:

1. First, our *HTTP Handler* accepts the call by listening to the port that we defined when creating the server.
2. Then, it goes through the *middleware*, which is defined by using the `wrap_fn` function on our app.
3. Once this is done, the URL of the request is mapped to the right view and the schemas we defined in our `src/json_serialization/` directory. These get passed into the resource (our views) defined in the `src/views` directory.

More books: https://t.me/booksforits

If we then want to update or get data from the database, we use the Diesel ORM to map these requests. At this stage, all our layers have been defined to manage the flow of data effectively, apart from our middleware. As pointed out in the previous chapter, *Chapter 7*, *Managing User Sessions*, we have implemented our middleware for authentication with the `JwToken` struct by implementing the `FromRequest` trait. With this, we can see that we can implement our middleware using either the `wrap_fn` or implementing the `FromRequest` trait. When do you think we should use the `wrap_fn` or `FromRequest` trait? Both have advantages and disadvantages. If we want to implement our middleware for specific individual views, then implementing the `FromRequest` trait is the best option. This is because we can slot a struct implementing the `FromRequest` trait into the view that we want. Authentication is a good use case for implementing the `FromRequest` trait because we want to pick and choose what endpoints require authentication. However, if we want to implement a blanket rule, we would be better off implementing the selection of views for authentication in the `wrap_fn` function. Implementing our middleware in the `wrap_fn` function means that it is implemented for every request.

An example of this could be that we are no longer supporting version one for all our endpoints. If we were going to do this, we would have to warn third-party users of our decision to no longer support version one of our API. Once our date has passed, we will have to give a helpful message that we are no longer supporting version one. Before we start working on our middleware layer, we must define the following imports at the top of our `main.rs` file:

```
use actix_web::{App, HttpServer, HttpResponse};
use actix_service::Service;
use futures::future::{ok, Either};
use actix_cors::Cors;
```

To ensure that we know that the incoming request is destined for a `v1` endpoint, we must define a flag that we can check later when deciding whether to process the request or reject it. We can do this by using the following code in our `main.rs` file:

```
.wrap_fn(|req, srv|{
    let passed: bool;

    if req.path().contains("/v1/") {
        passed = false;
    } else {
        passed = true;
    }
. . .
```

More books: https://t.me/booksforits

From the preceding code, we can see that we declare that there is a Boolean under the name of `passed`. If `v1` is not in the URL, then it is set to `true`. If `v1` is present in the URL, then `passed` is set to `false`.

Now that we have defined a flag, we can use it to dictate what happens to the request. Before we do this, we must take note of the last lines of `wrap_fn`, as denoted in this code block:

```
let future = srv.call(req);
async {
    let result = fut.await?;
    Ok(result)
}
```

We are waiting for the call to finish, then returning the result as the variable called `result`. With our blocking of the `v1` API called, we must check to see whether the request passes. If it does, we then run the preceding code. However, if the request fails, we must bypass this and define another future, which is just the response.

At face value, this can seem straightforward. Both will return the same thing, that is, a response. However, Rust will not compile. It will throw an error based on incompatible types. This is because `async` blocks behave like closures. This means that every `async` block is its own type. This can be frustrating, and due to this subtle detail, it can lead to developers burning hours trying to get the two futures to play with each other.

Luckily, there is an enum in the futures crate that solves this problem for us. The `Either` enum combines two different futures, streams, or sinks that have the same associated types into a single type. This enables us to match the `passed` flag, and fire and return the appropriate process with the following code:

```
    let end_result;
    if passed == true {
        end_result = Either::Left(srv.call(req))
    }
    else {
        let resp = HttpResponse::NotImplemented().body(
            "v1 API is no longer supported"
            );
        end_result = Either::Right(
            ok(req.into_response(resp)
                  .map_into_boxed_body())
        )
```

More books: https://t.me/booksforits

```
    }
    async move {
        let result = end_result.await?;
        Ok(result)
    }
}).configure(views::views_factory).wrap(cors);
```

From the preceding code, we can see that we assign `end_result` to be called as a view, or directly return it to an unauthorized response depending on the `passed` flag. We then return this at the end of `wrap_fn`. Knowing how to use the `Either` enum is a handy trick to have up your sleeve and will save you hours when you need your code to choose between two different futures.

To check to see whether we are blocking `v1`, we can call a simple `get` request as seen in the following figure:

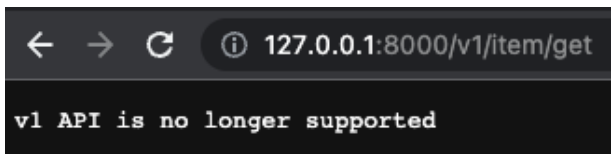


Figure 8.2 – The response to blocked v1

We can see that our API call is blocked with a helpful message. If we do the API call through Postman, we will see that we get a `501 Not Implemented` error as seen in the following figure:

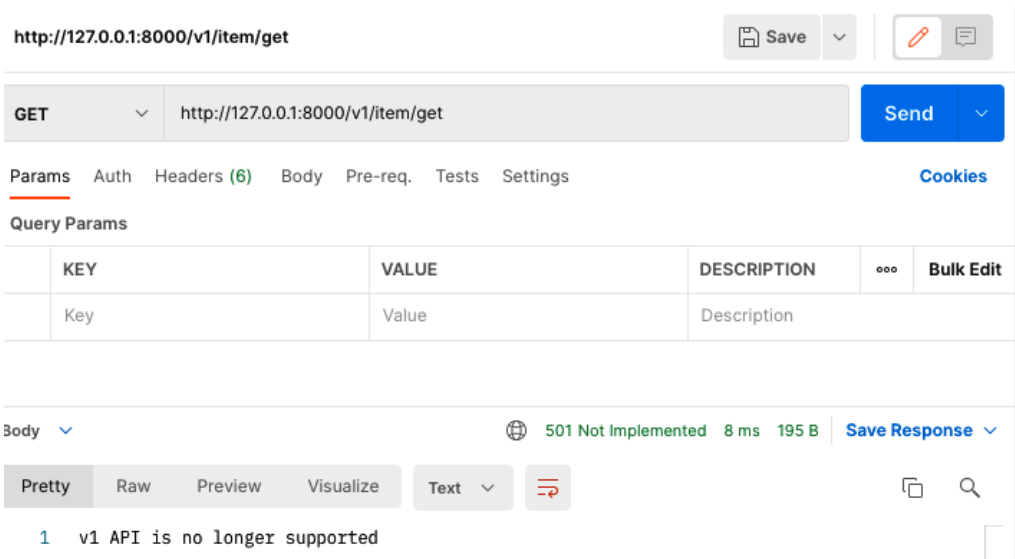


Figure 8.3 – Postman's response to blocked v1

More books: https://t.me/booksforits

We might want to add more resources for getting items in the future. This poses a potential problem because some views might start clashing with the app views. For instance, our to-do item API views only have the prefix `item`. Getting all the items requires the `v1/item/get` endpoint.

It could be reasonable to develop a view for the app that looks at a to-do item in detail for editing with the `v1/item/get/{id}` endpoint later. However, this increases the risk of clashes between the frontend app views and the backend API calls. To prevent this, we are going to have to ensure that our API has a uniform interface.

Building a uniform interface

Having a uniform interface means that our resources can be uniquely identifiable through a URL. This decouples the backend endpoints and frontend views, enabling our app to scale without clashes between the frontend views and backend endpoints. We have decoupled our backend from the frontend using the version tag. When a URL endpoint includes a version tag, such as `v1` or `v2`, we know that call is hitting the backend Rust server. When we are developing our Rust server, we might want to work on a newer version of our API calls. However, we do not want to allow users to access the version in development. To enable live users to access one version while we deploy another version on a test server, we will need to dynamically define the API version for the server. With the knowledge that you have acquired so far in this book, you could simply define the version number in the `config.yml` file and load it. However, we would have to read the `config.yml` config file for every request. Remember that when we set up the database connection pool we read the connection string from the `config.yml` file once, meaning that it is present for the entire lifetime of the program. We would like to define the version once and then refer to it for the lifecycle of the program. Intuitively, you might want to define the version in the `main` function in the `main.rs` file before we define the server and then access the definition of the version inside the `wrap_fn`, as seen in the following example:

```
let outcome = "test".to_owned();
HttpServer::new(|| {
    . . .
    let app = App::new()
        .wrap_fn(|req, srv|{
            println!("{}", outcome);
            . . .
        }
    . . .
```

More books: https://t.me/booksforits

However, if we try and compile the preceding code, it will fail as the lifetime of the `outcome` variable is not long enough. We can convert our `outcome` variable as a constant with the following code:

```
const OUTCOME: &str = "test";
HttpServer::new(|| {
    . . .
    let app = App::new()
        .wrap_fn(|req, srv|{
            println!("{}", outcome);
            . . .
        }
    . . .
```

The preceding code will run without any lifetime issues. However, if we were to load our version, we will have to read it from a file. In Rust, if we read from a file, we do not know what the size of the variable being read from the file is. Therefore, the variable we read from the file is going to be a string. The problem here is that allocating a string is not something that can be computed at compile time. Therefore, we are going to have to write the version directly into our `main.rs` file. We can do this by using a `build` file.

> **Note**
> We utilize `build` files in this problem to teach the concept of `build` files so you can use them if needed. There is nothing stopping you from hardcoding the constant in the code.

This is where a single Rust file runs before the Rust application is run. This `build` file will automatically run when we are compiling the main Rust application. We can define the dependencies needed to run our `build` file in the `build dependencies` section in the `Cargo.toml` file with the following code:

```
[package]
name = "web_app"
version = "0.1.0"
edition = "2021"
build = "build.rs"

[build-dependencies]
serde_yaml = "0.8.23"
serde = { version = "1.0.136", features = ["derive"] }
```

More books: https://t.me/booksforits

This means that our `build` Rust file is defined in the root of the application in the `build.rs` file. We will then define the dependencies needed for the build phase in the `[build-dependencies]` section. Now that our dependencies are defined, our `build.rs` file can take the following form:

```
use std::fs::File;
use std::io::Write;
use std::collections::HashMap;
use serde_yaml;


fn main() {
  let file =
      std::fs::File::open("./build_config.yml").unwrap();
  let map: HashMap<String, serde_yaml::Value> =
      serde_yaml::from_reader(file).unwrap();
  let version =
      map.get("ALLOWED_VERSION").unwrap().as_str()
          .unwrap();
  let mut f =
      File::create("./src/output_data.txt").unwrap();
  write!(f, "{}", version).unwrap();
}
```

Here we can see that we need to import what we need to read from a YAML file and write it to a standard text file. Then, we will open a `build_config.yml` file, which is in the root of the web application next to the `config.yml` file. We will then extract the `ALLOWED_VERSION` from the `build_config.yml` file and write it into a text file. Now that we have defined the build process and what is needed from the `build_config.yml` file, our `build_config.yml` file will have to take the following form:

```
ALLOWED_VERSION: v1
```

Now that we have everything defined for our build, we can introduce a `const` instance for our version through the file that we wrote to in our `build.rs` file. To do this, our `main.rs` file will need a few changes. First, we define the `const` with the following code:

```
const ALLOWED_VERSION: &'static str = include_str!(
    "./output_data.txt");
HttpServer::new(|| {
. . .
```

More books: https://t.me/booksforits

We then deem the request to pass if the version is allowed with the following code:

```
HttpServer::new(|| {
. . .
let app = App::new()
.wrap_fn(|req, srv|{
    let passed: bool;

    if *&req.path().contains(&format!("/{}/",
                              ALLOWED_VERSION)) {
        passed = true;
    } else {
        passed = false;
    }
```

Then, we define the error response and service call with the following code:

```
. . .
let end_result = match passed {
    true => {
        Either::Left(srv.call(req))
    },
    false => {
        let resp = HttpResponse::NotImplemented()
            .body(format!("only {} API is supported",
                ALLOWED_VERSION));
        Either::Right(
            ok(req.into_response(resp).map_into_boxed_body())
        )
    }
};
. . .
```

We are now ready to build and run our application with a specific version supported. If we run our application and make a `v2` request, we get the following response:

More books: https://t.me/booksforits

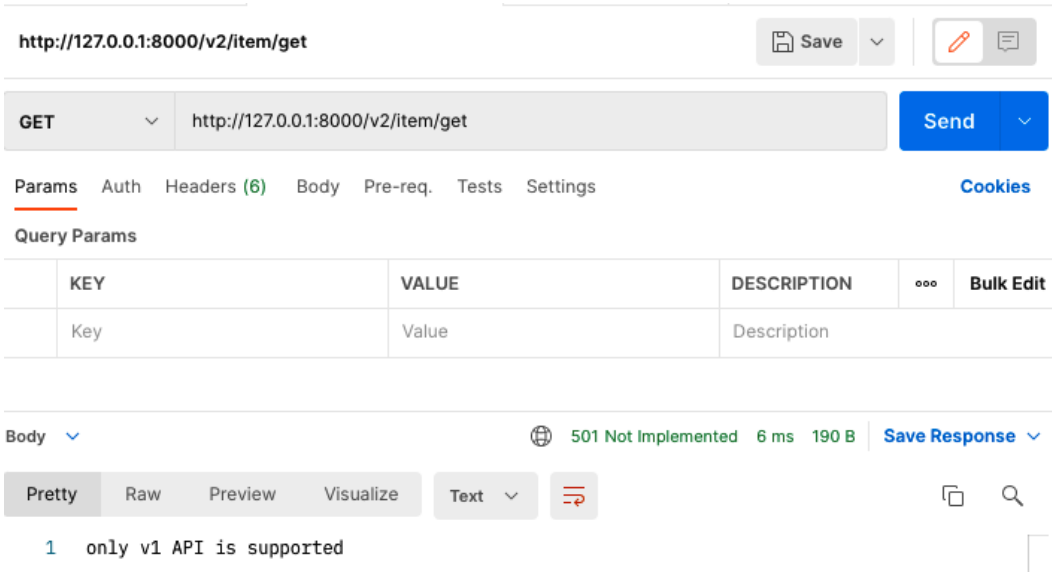


Figure 8.4 – Postman response to blocked v2

We can see that our version guard is now working. This also means that we must use the React application to access the frontend views or you can add a `v1` to the frontend API endpoints.

Now, if we run our app, we can see that our frontend works with the new endpoints. With this, we are one step closer to developing a RESTful API for our app. However, we still have some glaring shortcomings. Right now, we can create another user and log in under that user. In the next section, we'll explore how to manage our user state in a stateless fashion.

Implementing statelessness

Statelessness is where the server does not store any information about the client session. The advantages here are straightforward. It enables our application to scale more easily as we free up resources on the server side by storing session information on the client's side instead.

It also empowers us to be more flexible with our computing approach. For instance, let's say that our application has exploded in popularity. As a result, we may want to spin our app up on two computing instances or servers and have a load balancer direct traffic to both instances in a balanced manner. If information is stored on the server, the user will have an inconsistent experience.

They may update the state of their session on one computing instance, but then, when they make another request, they may hit another computing instance that has outdated data. Considering this, statelessness cannot just be achieved by storing everything in the client. If our database is not dependent on a computing instance of our app, we can also store our data on this database, as shown in the following diagram:

More books: https://t.me/booksforits

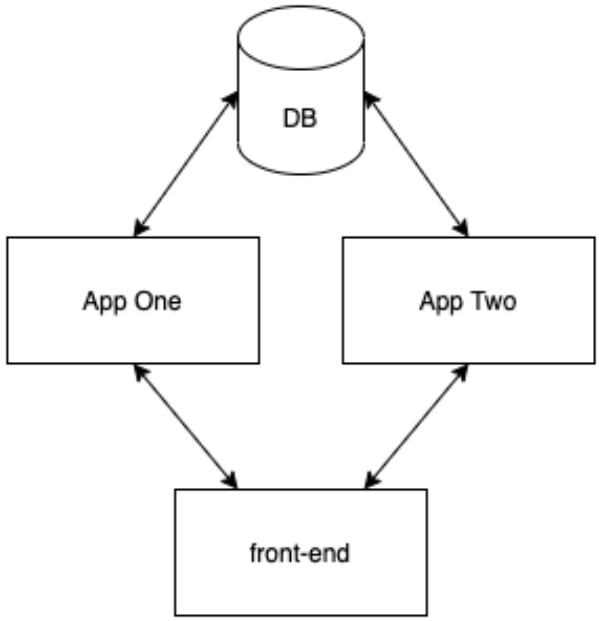


Figure 8.5 – Our stateless approach

As you can see, our app is already stateless. It stores the user ID in a JWT in the frontend and we store our user data models and to-do items in our PostgreSQL database. However, we might want to store Rust structs in our application. For instance, we could build a struct that counts the number of requests hitting the server. With reference to *Figure 8.5*, we cannot just save our structs locally on the server. Instead, we will store our structs in Redis, carrying out the processes in the following figure:

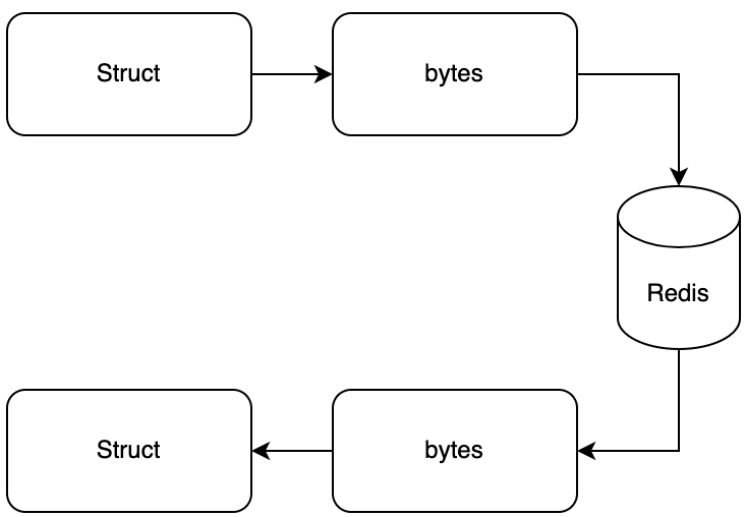


Figure 8.6 – Steps to saving structs in Redis

More books: https://t.me/booksforits

The difference between PostgreSQL and Redis

Redis is a database, but it is different from a PostgreSQL database. Redis is closer to a key-value store. Redis is also fast due to the data being in memory. While Redis is not as complete as PostgreSQL for managing tables and how they relate to each other, Redis does have advantages. Redis supports useful data structures such as lists, sets, hashes, queues, and channels. You can also set expiry times on the data that you insert into Redis. You also do not need to handle data migrations with Redis. This makes Redis an ideal database for caching data that you need quick access to, but you are not too concerned about persistence. For the channels and queues, Redis is also ideal for facilitating communications between subscribers and publishers.

We can achieve the process in *Figure 8.6* by carrying out the following steps:

1. Define Redis service for Docker.
2. Update Rust dependencies.
3. Update our config file for a Redis connection.
4. Build a counter struct that can be saved and loaded in a Redis database.
5. Implement the counter for each request.

Let's go over each step in detail:

1. When it comes to spinning up a Redis Docker service, we need to use a standard Redis container with standard ports. After we have implemented our Redis service, our `docker-compose.yml` file should have the current state:

```
version: "3.7"

services:

  postgres:
    container_name: 'to-do-postgres'
    image: 'postgres:11.2'
    restart: always
    ports:
      - '5433:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'
      - 'POSTGRES_PASSWORD=password'
  redis:
```

More books: https://t.me/booksforits

```
    container_name: 'to-do-redis'
    image: 'redis:5.0.5'
    ports:
      - '6379:6379'
```

We can see that we now have the Redis service and the database service running on the local machine. Now that Redis can be run, we need to update our dependencies in the next step.

2. Recalling *Figure 8.6*, we need to serialize the Rust struct into bytes before inserting it into Redis. With these steps in mind, we need the following dependencies in our `Cargo.toml` file:

```
[dependencies]
. . .
redis = "0.21.5"
```

We are using the `redis` crate to connect to the Redis database. Now that our dependencies are defined, we can start defining our config file in the next step.

3. When it comes to our `config.yml` file, we must add the URL for the Redis database connection. At this point in time of the book, our `config.yml` file should have the following form:

```
DB_URL: postgres://username:password@localhost:5433/to_do
SECRET_KEY: secret
EXPIRE_MINUTES: 120
REDIS_URL: redis://127.0.0.1/
```

We have not added the port number for our `REDIS_URL` parameter. This is because we are using the standard port in our Redis service, which is `6379`, so we do not have to define the port. We now have all the data ready to define a struct that can connect to Redis, which we will do in the next step.

4. We will define our `Counter` struct in the `src/counter.rs` file. First, we will have to import the following:

```
use serde::{Deserialize, Serialize};
use crate::config::Config;
```

5. We will be using the `Config` instance to get the Redis URL, and the `Deserialize` and `Serialize` traits to enable conversion to bytes. Our `Counter` struct takes the following form:

```
#[derive(Serialize, Deserialize, Debug)]
pub struct Counter {
    pub count: i32
}
```

More books: https://t.me/booksforits

6. Now that we have our `Counter` struct defined with all the traits, we need to define the functions that are required for our operations with the following code:

```
impl Counter {
    fn get_redis_url() -> String {
        . . .
    }
    pub fn save(self) {
        . . .
    }
    pub fn load() -> Counter {
        . . .
    }
}
```

7. With the preceding functions defined, we can load and save our `Counter` struct into the Redis database. When it comes to building our `get_redis_url` function, it should be no surprise that it can take the following form:

```
fn get_redis_url() -> String {
    let config = Config::new();
    config.map.get("REDIS_URL")
              .unwrap().as_str()
              .unwrap().to_owned()
}
```

8. Now that we have the Redis URL, we can save our `Counter` struct with the following code:

```
pub fn save(self) -> Result<(), redis::RedisError> {
    let serialized = serde_yaml::to_vec(&self).unwrap();

    let client = match redis::Client::open(
                    Counter::get_redis_url()) {
        Ok(client) => client,
        Err(error) => return Err(error)
    };
    let mut con = match client.get_connection() {
        Ok(con) => con,
        Err(error) => return Err(error)
```

More books: https://t.me/booksforits

```
    };
    match redis::cmd("SET").arg("COUNTER")
                           .arg(serialized)
                           .query::<Vec<u8>>(&mut con) {
        Ok(_) => Ok(()),
        Err(error) => Err(error)
    }
}
```

9. Here, we can see that we can serialize our `Counter` struct to a `Vec<u8>`. We will then define the client for the Redis and insert our serialized `Counter` struct under the key `"COUNTER"`. There are more features to Redis, however, you can utilize Redis for this chapter by thinking about Redis as being a big scalable in-memory hashmap. With the hashmap concept in mind, how do you think we could get the `Counter` struct from the Redis database? You probably guessed it; we use the `GET` command with the `"COUNTER"` key and then deserialize it with the following code:

```
pub fn load() -> Result<Counter, redis::RedisError> {
    let client = match redis::Client::open(
                    Counter::get_redis_url()){
        Ok(client) => client,
        Err(error) => return Err(error)
    };
    let mut con = match client.get_connection() {
        Ok(con) => con,
        Err(error) => return Err(error)
    };
    let byte_data: Vec<u8> = match redis::cmd("GET")
                                  .arg("COUNTER")
                                  .query(&mut con) {
        Ok(data) => data,
        Err(error) => return Err(error)
    };
    Ok(serde_yaml::from_slice(&byte_data).unwrap())
}
```

We have now defined our `Counter` struct. We have everything in-line to implement in our `main.rs` file in the next step.

More books: https://t.me/booksforits

10. When it comes to increasing the count by one every time a request comes in, we need to carry out the following code in the `main.rs` file:

```
. . .
mod counter;
. . .

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    . . .
    let site_counter = counter::Counter{count: 0};
    site_counter.save();

    HttpServer::new(|| {
        . . .
        let app = App::new()
            .wrap_fn(|req, srv|{
                let passed: bool;

                let mut site_counter = counter::
                                       Counter::load()
                                       .unwrap();
                site_counter.count += 1;
                println!("{:?}", &site_counter);
                site_counter.save();
                . . .
```

11. Here, we can see that we define the `counter` module. Before we spin up the server, we need to create the new `Counter` struct and insert it into Redis. We then get the `Counter` from Redis, increase the count, then save it for every request.

Now when we run our server, we can see that our counter is increasing every time we hit our server with a request. Our printout should look like the following:

```
Counter { count: 1 }
Counter { count: 2 }
Counter { count: 3 }
Counter { count: 4 }
```

More books: https://t.me/booksforits

Now that we have integrated another storage option, our app essentially functions the way we want it to. If we wanted to ship our application now, there is nothing really stopping us from configuring the build with Docker and deploying it on a server with a database and **NGINX**.

Concurrency issues

If two servers request the counter at the same time, there is a risk of missing a request. The counter example was explored to demonstrate how to store serialized structs in Redis. If you need to implement a simple counter in a Redis database and concurrency is a concern, it is suggested that you use the `INCR` command. The `INCR` command increases the number under the key you select by one in the Redis database, returning the new increased number as a result. Seeing as the counter is increased in the Redis database, we have reduced the risk of concurrency issues.

However, there are always things we can add. In the next section, we'll investigate logging requests.

Logging our server traffic

So far, our application does not log anything. This does not directly affect the running of the app. However, there are some advantages to logging. Logging enables us to debug our applications.

Right now, as we are developing locally, it may not seem like logging is really needed. However, in a production environment, there are many reasons why an application can fail, including Docker container orchestration issues. Logs that note what processes have happened can help us to pinpoint an error. We can also use logging to see when edge cases and errors arise for us to monitor the general health of our application. When it comes to logging, there are four types of logs that we can build:

- **Informational (info)**: This is general logging. If we want to track a general process and how it is progressing, we use this type of log. Examples of using this are starting and stopping the server and logging certain checkpoints that we want to monitor, such as HTTP requests.
- **Verbose**: This is information such as the type defined in the previous point. However, it is more granular to inform us of a more detailed flow of a process. This type of log is mainly used for debugging purposes and should generally be avoided when it comes to production settings.
- **Warning**: We use this type when we are logging a process that is failing and should not be ignored. However, we can use this instead of raising an error because we do not want the service to be interrupted or the user to be aware of the specific error. The logs themselves are for us to be alerted of the problem to allow us to then act. Problems such as calls to another server failing are appropriate for this category.

More books: https://t.me/booksforits

- **Error**: This is where the process is interrupted due to an error and we need to sort it out as quickly as possible. We also need to inform the user that the transaction did not go through. A good example of this is a failure to connect or insert data into a database. If this happens, there is no record of the transaction happening and it cannot be solved retroactively. However, it should be noted that the process can continue running.

If a warning comes up about the server failing to send an email, connect to another server to dispatch a product for shipping, and so on. Once we have sorted out the problem, we can retroactively make a database call to transactions in this timeframe and make the calls to the server with the right information.

In the worst case, there will be a delay. With the error type, we will not be able to make the database call as the server was interrupted by an error before the order was even entered in the database. Considering this, it is clear why error logging is highly critical, as the user needs to be informed that there is a problem and their transaction did not go through, prompting them to try again later.

We could consider the option of including enough information in the error logs to retroactively go back and update the database and complete the rest of the process when the issue is resolved, removing the need to inform the user. While this is tempting, we must consider two things. Log data is generally unstructured.

There is no quality control for what goes into a log. Therefore, once we have finally managed to manipulate the log data into the right format, there is still a chance that corrupt data could find its way into the database.

The second issue is that logs are not considered secure. They get copied and sent to other developers in a crisis and they can be plugged into other pipelines and websites, such as *Bugsnag*, to monitor logs. Considering the nature of logs, it is not good practice to have any identifiable information in a log.

Now that we have understood the uses of logging, we can start configuring our own logger. When it comes to logging, we are going to use the Actix-web logger. This gives us flexibility on what we log while having the underlying mechanics of logging configured and working well with our Actix server. To build our logger, we must define a new crate in our `Cargo.toml` file with the following code:

```
[dependencies]
. . .
env_logger = "0.9.0"
```

This enables us to configure a logger using environment variables. We can now focus on `main.rs` as this is where our logger is configured and used. First, we will import our logger with the following code:

```
use actix_web::{. . ., middleware::Logger};
```

More books: https://t.me/booksforits

With this import, we can define our logger in the `main` function with the following code:

```
. . .
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    . . .
    env_logger::init_from_env(env_logger::Env::new()
                              .default_filter_or("info"));
. . .
```

Here, we are stating that our logger is going to log information to the info stream. With the logger configured, we can then wrap our server with the logger using the following code:

```
. . .
        async move {
            let result = end_result.await?;
            Ok(result)
        }
}).configure(views::views_factory).wrap(cors)
    .wrap(Logger::new("%a %{User-Agent}i %r %s %D"));
return app
. . .
```

We can see in our logger that we passed in the `"&a %{User-Agent}I %r %s %D"` string. This string is interpreted by the logger, tell them what to log. The Actix logger can take the following inputs:

- `%%`: The percent sign
- `%a`: The remote IP address (IP address of proxy if using reverse proxy)
- `%t`: The time when the request was started to process
- `%P`: The process ID of the child that serviced the request
- `%r`: The first line of request
- `%s`: The response status code
- `%b`: The size of the response in bytes, including HTTP headers

More books: https://t.me/booksforits

- `%T`: The time taken to serve the request, in seconds with floating fraction in .06f format
- `%D`: The time taken to serve the request, in milliseconds
- `%{FOO}i`: `request.headers['FOO']`
- `%{FOO}o`: `response.headers['FOO']`
- `%{FOO}e`: `os.environ['FOO']`

With these inputs, we can work out that we are going to log the remote IP address, user-agent, endpoint of the request, and the time taken to process the request. We will do this for every request that our Rust server has. Starting our Rust server with the logging gives us the following output:

```
[2022-05-25T17:22:32Z INFO  actix_server::builder] Starting 8
workers
[2022-05-25T17:22:32Z INFO  actix_server::server] Actix runtime
found; starting in Actix runtime
```

Here, we can see that the time of the starting of the server with the number of workers is automatically logged. Then, if we start our frontend, we should be prompted to log in as the token should have expired by now. A full standard request log should look like the following output:

```
[2022-05-25T17:14:56Z INFO  actix_web::middleware::logger]
127.0.0.1 Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/101.0.4951.64 Safari/537.36
GET /v1/item/get HTTP/1.1 401 9.466000
```

We can see the time, the fact that it is an `INFO`-level log, and what logger is logging it. We can also see my IP address, which is local as I am running my application locally, my computer/browser details, and the API call with a `401` response code. If we trim everything from the request log apart from the method, API endpoint, response code, and response time, our login prompt will look like the following:

```
GET /v1/item/get HTTP/1.1 401 9.466000
OPTIONS /v1/auth/login HTTP/1.1 200 0.254000
POST /v1/auth/login HTTP/1.1 200 1405.585000
OPTIONS /v1/item/get HTTP/1.1 200 0.082000
GET /v1/item/get HTTP/1.1 200 15.470000
```

More books: https://t.me/booksforits

We can see that I fail in getting items, with an unauthorized response. I then log in and get an `OK` response from the login. Here, we can see an `OPTIONS` and `POST` method. The `OPTIONS` method is for our CORS, which is why the `OPTIONS` calls processing time is a fraction of other API requests. We can see that we then get our items that are then rendered to the page. However, we can see what happens in the following logs when we refresh the page:

```
OPTIONS /v1/item/get HTTP/1.1 200 0.251000
OPTIONS /v1/item/get HTTP/1.1 200 0.367000
GET /v1/item/get HTTP/1.1 200 88.317000
GET /v1/item/get HTTP/1.1 200 9.523000
```

We can see that there are two `GET` requests for the items. However, we have not altered the to-do items in the database. This is not an error, but it is wasteful. To optimize this, we can utilize the REST constraint of caching in the following section.

Caching

Caching is where we store data in the frontend to be reused. This enables us to reduce the number of API calls to the backend and reduce latency. Because the benefits are so clear, it can be tempting to cache everything. However, there are some things to consider.

Concurrency is a clear issue. The data could be outdated, leading to confusion and data corruption when sending the wrong information to the backend. There are also security concerns. If one user logs out and another user logs in on the same computer, there is a risk that the second user will be able to access the first user's items. With this, there must be a couple of checks in place. The correct user needs to be logged in and the data needs to be timestamped so that if the cached data is accessed past a certain period, a `GET` request is made to refresh the data.

Our application is fairly locked down. We cannot access anything unless we are logged in. The main process that we could cache in our application is the `GET` items call. All other calls that edit the state of the item list in the backend return the updated items. Considering this, our caching mechanism looks like the following:

More books: https://t.me/booksforits

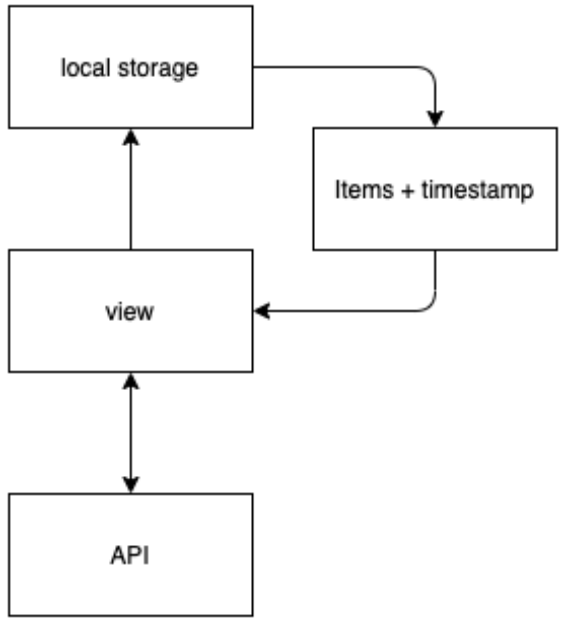


Figure 8.7 – Our caching approach

The loop in the diagram can be executed as many times as we want when refreshing the page. However, this might not be a good idea. If a user logs onto our application on their phone when they are in the kitchen to update the list, then the user will have a problem when they go back to their computer to do some work, refreshing the page on the computer and updating the list. This caching system would expose the user to out-of-date data that will be sent to the backend. We can reduce the risk of this happening by referencing the time stamp. When the timestamp is older than the cutoff, we will then make another API call to refresh the data when the user refreshes the page.

When it comes to our caching logic, it will all be implemented in the `front_end/src/App.js` file under the `getItems` function. Our `getItems` function will take the following layout:

```
getItems() {
  . . .
  if (difference <= 120) {
      . . .
  }
  else {
      axios.get("http://127.0.0.1:8000/v1/item/get",
          {headers: {"token": localStorage
                      .getItem("token")}}).then(response => {
```

More books: https://t.me/booksforits

```
                . . .
                })
            }).catch(error => {
            if (error.response.status === 401) {
                this.logout();
            }
        });
    }
}
```

Here, we state that the time difference between the last cached items and the current time must be less than 120 seconds, which is 2 minutes. If the time difference is below 2 minutes, we will get our data from the cache. However, if the time difference is above 2 minutes, then we make a request to our API backend. If we get an unauthorized response, we will then log out. First, in this `getItems` function, we get the date that the items were cached and calculate the difference between then and now, using the following code:

```
let cachedData = Date.parse(localStorage
                            .getItem("item-cache-date"));
let now = new Date();
let difference = Math.round((now - cachedData) / (1000));
```

If our time difference is 2 minutes, we will get our data from the local storage and update our state with that data with the following code:

```
let pendingItems =
    JSON.parse(localStorage.getItem("item-cache-data-
pending"));
let doneItems =
    JSON.parse(localStorage.getItem("item-cache-data-
                                    done"));
let pendingItemsCount = pendingItems.length;
let doneItemsCount = doneItems.length;
this.setState({
  "pending_items": this.processItemValues(pendingItems),
  "done_items": this.processItemValues(doneItems),
  "pending_items_count": pendingItemsCount,
  "done_items_count": doneItemsCount
})
```

More books: https://t.me/booksforits

Here, we must parse the data from the local storage because the local storage only handles string data. Because local storage only handles strings, we must stringify the data that we are inserting into the local storage when we make the API request with the following code:

```
let pending_items = response.data["pending_items"]
let done_items = response.data["done_items"]

localStorage.setItem("item-cache-date", new Date());
localStorage.setItem("item-cache-data-pending",
                     JSON.stringify(pending_items));
localStorage.setItem("item-cache-data-done",
                     JSON.stringify(done_items));

this.setState({
  "pending_items": this.processItemValues(pending_items),
  "done_items": this.processItemValues(done_items),
  "pending_items_count":
      response.data["pending_item_count"],
  "done_items_count": response.data["done_item_count"]
})
```

If we run our application, we only make one API call. If we refresh our application before the 2 minutes are up, we can see that there are no new API calls despite our frontend rendering all the items from the cache. However, if we create, edit, or delete an item, and then refresh the page before the 2 minutes is up, we will see that our view will revert to the previous out-of-date state. This is because the created, edited, and deleted items also return to their previous state but they are not stored in the local storage. This can be handled by updating our `handleReturnedState` function with the following code:

```
handleReturnedState = (response) => {
  let pending_items = response.data["pending_items"]
  let done_items = response.data["done_items"]

  localStorage.setItem("item-cache-date", new Date());
  localStorage.setItem("item-cache-data-pending",
                       JSON.stringify(pending_items));
  localStorage.setItem("item-cache-data-done",
                       JSON.stringify(done_items));
```

More books: https://t.me/booksforits

```
  this.setState({
     "pending_items":this.processItemValues(pending_items),
     "done_items": this.processItemValues(done_items),
     "pending_items_count":response
         .data["pending_item_count"],
      "done_items_count": response.data["done_item_count"]
  })
}
```

Here, we have it. We have managed to cache our data and reuse it to prevent our backend API from being hit excessively. This can be applied to other frontend processes too. For instance, a customer basket could be cached and used when the user checks out.

This takes our simple website one step closer to being a web app. However, we must acknowledge that as we use caching more, the complexity of the frontend increases. For our application, this is where the caching stops. Right now, there are no more alterations needed on our applications for the rest of the hour. However, there is one more concept that we should briefly cover, which is code on demand.

Code on demand

Code on demand is where the backend server directly executes code on the frontend. This constraint is optional and not widely used. However, it can be useful as it gives the backend server the right to decide as and when code is executed on the frontend. We have already been doing this; in our logout view, we directly execute JavaScript on the frontend by simply returning it in a string. This is done in the `src/views/auth/logout.rs` file. We must remember that we have now added to-do items to our local storage. If we do not remove these items from our local storage when logging out, somebody else would be able to access our to-do items if they manage to log in to their own account on the same computer within 2 minutes. While this is highly unlikely, we might as well be safe. Remember that our logout view in the `src/views/auth/logout.rs` file takes the following form:

```
pub async fn logout() -> HttpResponse {
    HttpResponse::Ok()
        .content_type("text/html; charset=utf-8")
        .body(. . .)
}
```

Inside the body of our response, we have the following content:

```
"<html>\
<script>\
    localStorage.removeItem('user-token'); \
```

More books: https://t.me/booksforits

```
    localStorage.removeItem('item-cache-date'); \
    localStorage.removeItem('item-cache-data-pending'); \
    localStorage.removeItem('item-cache-data-done'); \
    window.location.replace(
        document.location.origin);\
</script>\
</html>"
```

With this, we have not only removed the user token, but we have also removed all items and dates. With this, our data is safe once we have logged out.

Summary

In this chapter, we have gone through the different aspects of RESTful design and implemented them into our application. We have assessed the layers of our application, enabling us to refactor the middleware to enable two different futures to be processed depending on the outcome. This doesn't just stop at authorizing requests. Based on the parameters of the request, we could implement middleware to redirect requests to other servers, or directly respond with a code-on-demand response that makes some changes to the frontend and then makes another API call. This approach gives us another tool, custom logic with multiple future outcomes in the middleware before the view is hit.

We then refactored our path struct to make the interface uniform, preventing clashes between frontend and backend views.

We then explored the different levels of logging and logged all our requests to highlight silent yet undesirable behavior. After refactoring our frontend to rectify this, we then used our logging to assess whether our caching mechanism was working correctly when caching to-do items into the frontend to prevent excessive API calls. Now, our application is passable. We can always make improvements; however, we are not at the stage where if we were to deploy our application onto a server, we would be able to monitor it, check the logs when something is going wrong, manage multiple users with their own to-do lists, and reject unauthorized requests before they even hit the view. We also have caching, and our application is stateless, accessing and writing data on a PostgreSQL and Redis database.

In the next chapter, we will be writing unit tests for our Rust structs and functional tests for our API endpoints, as well as cleaning the code up ready for deployment.

Questions

1. Why can we not simply code multiple futures into the middleware and merely call and return the one that is right considering request parameters and authorization outcomes, but must wrap them in an enum instead?

More books: https://t.me/booksforits

2. How do we add a new version of views but still support the old views in case our API is serving mobile apps and third parties that might not update instantly?
3. Why is the stateless constraint becoming more important in the era of elastic cloud computing?
4. How could we enable another service to be incorporated utilizing the properties of the JWT?
5. A warning log message hides the fact that an error has happened from the user but still alerts us to fix it. Why do we ever bother telling the user that an error has occurred and to try again with an error log?
6. What are the advantages of logging all requests?
7. Why do we sometimes have to use `async move`?

Answers

1. Rust's strong typing system will complain. This is because `async` blocks behave like closures meaning that every `async` block is its own type. Pointing to multiple futures is like pointing to multiple types, and thus it will look like we are returning multiple different types.
2. We add a new module in the views directory with the new views. These have the same endpoints and views with new parameters that are needed. We can then add a version parameter in the factory function. These new views will have the same endpoints with `v2` in them. This enables users to use the new and old API endpoints. We then notify users when the old version will no longer be supported, giving them time to update. At a specific time, we will move our version in the build to `v2`, cutting all requests that make `v1` calls and responding with a helpful message that `v1` is no longer supported. For this transition to work, we will have to update the allowed versions in the `build` config to a list of supported versions.
3. With orchestration tools, microservices, and elastic computing instances on demand, spinning up and shutting down elastic computing instances due to demand is becoming more common practice. If we store data on the instance itself, when the user makes another API call, there is no guarantee that the user will hit the same instance, getting inconsistent data read and writes.
4. The JWT token enables us to store the user ID. If the second service has the same secret key, we can merely pass requests to the other service with the JWT in the header. The other service does not have to have the login views or access to the user database and can still function.
5. When an error happens that prevents us from retroactively going back and sorting out the issue, then we must raise an error instead of a warning. A classic example of an error is not being able to write to a database. A good example of a warning is another service not responding. When the other service is up and running, we can do a database call and call the service to finish off the process.

More books: https://t.me/booksforits

6. In production, it is needed to assess the state of a server when troubleshooting. For instance, if a user is not experiencing an update, we can quickly check the logs to see if the server is in fact receiving the request or if there is an error with the caching in the frontend. We can also use it to see if our app is behaving the way we expect it to.
7. There could be a possibility that the lifetime of the variable that we are referencing in the `async` block might not live long enough to see the end of the `async` block. To resolve this, we can shift the ownership of the variable to the block with an `async move` block.

More books: https://t.me/booksforits

More books: https://t.me/booksforits

Part 4: Testing and Deployment

When an application is built, we need to deploy it on a server so others can use it. We also need to test it to ensure it works to our expectations before we deploy it. In this part, we will cover unit and end-to-end testing using tools such as Postman. We'll build our own build and testing pipelines to automate the testing, building, and deployment processes. We'll cover how HTTP requests route to servers and what the HTTPS protocol is so we can implement it on AWS. We will also route traffic to our frontend and backend with NGINX, balance traffic between two individual servers on AWS, and lock down traffic to these servers and load balancer with AWS security groups. We will automate the AWS infrastructure using Terraform.

This part includes the following chapters:

- *Chapter 9*, *Testing Our Application Endpoints and Components*
- *Chapter 10*, *Deploying Our Application on AWS*
- *Chapter 11*, *Configuring HTTPS with NGINX on AWS*

More books: https://t.me/booksforits

More books: https://t.me/booksforits

9
Testing Our Application Endpoints and Components

Our to-do Rust application now fully works. We are happy with our first version as it manages authentication, different users, and their to-do lists, and logs our processes for inspection. However, a web developer's job is never done.

While we have now come to the end of adding features to our application, we know that the journey does not stop here. In future iterations beyond this book, we may want to add teams, new statuses, multiple lists per user, and so on. However, as we add these features, we must ensure that our old application's behavior stays the same unless we actively change it. This is done by building tests.

In this chapter, we'll build tests that check our existing behavior, laying down traps that will throw errors that report to us if the app's behavior changes without us actively changing it. This prevents us from breaking the application and pushing it to a server after adding a new feature or altering the code.

In this chapter, we will cover the following topics:

- Building our unit tests
- Building JWT unit tests
- Writing functional API tests in Postman
- Automating Postman tests with Newman
- Building an entire automated testing pipeline

By the end of this chapter, we will understand how to build unit tests in Rust, inspecting our structs in detail with a range of edge cases. If our structs behave in a way we do not expect, our unit tests will report it to us.

More books: https://t.me/booksforits

Technical requirements

In this chapter, we'll build on the code built in *Chapter 8*, *Building RESTful Services*. This can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter08/caching`.

Node and NPM are also needed for installing and running the automated API tests, which can be found at `https://nodejs.org/en/download/`.

We will also be running a part of the automated testing pipeline in Python. Python can be downloaded and installed at `https://www.python.org/downloads/`.

You can find the full source code used in this chapter here: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter09`.

Building our unit tests

In this section, we will explore the concept of unit tests and how to build unit test modules that contain tests as functions. Here, we are not going to achieve 100% unit test coverage for our application. There are places in our application that can be covered by our functional tests, such as API endpoints and JSON serialization. However, unit tests are still important in some parts of our application.

Unit tests enable us to look at some of our processes in more detail. As we saw with our logging in *Chapter 8*, *Building RESTful Services*, a functional test might work the way we want it to end-to-end, but there might be edge cases and behaviors that we do not want. This was seen in the previous chapter, where we saw our application make two `GET` calls when one was enough.

In our unit tests, we will break down the processes one by one, mock certain parameters, and test the outcomes. These tests are fully isolated. The advantage of this is that we get to test a range of parameters quickly, without having to run a full process each time. This also helps us pinpoint exactly where the application is failing and with what configuration. Unit testing is also useful for test-driven development, where we build the components of a feature bit by bit, running the unit tests and altering the components as and when the test outcomes require.

In big, complex systems, this saves a lot of time as you do not have to spin up the app and run the full system to spot a typo or failure to account for an edge case.

However, before we get too excited, we must acknowledge that unit testing is a tool, not a lifestyle, and there are some fallbacks to using it. The tests are only as good as their mocks. If we do not mock realistic interactions, then a unit test could pass but the application could fail. Unit tests are important, but they also must be accompanied by functional tests.

Rust is still a new language, so at this point, unit testing support is not as advanced as with other languages such as Python or Java. For instance, with Python, we can mock any object from any file

More books: https://t.me/booksforits

with ease at any point in the test. With these mocks, we can define outcomes and monitor interactions. While Rust does not have these mocks so readily available, this does not mean we cannot unit test.

A bad craftsman always blames their tools. The craftsmanship behind successful unit testing is constructing our code in such a way that individual code pieces won't depend on each other, giving the pieces will have as much autonomy as possible. Because of this lack of dependency, testing can easily be performed without the necessity of having a complex mocking system.

First, we can test our to-do structs. As you'll remember, we have the `done` and `pending` structs, which inherit a `base` struct. We can start by unit testing the struct that has no dependencies and then move down to other structs that have dependencies. In our `src/to_do/structs/base.rs` file, we can define our unit tests for the `base` struct at the bottom of the file with the following code:

```
#[cfg(test)]
mod base_tests {

    use super::Base;
    use super::TaskStatus;

    #[test]
    fn new() {
        let expected_title = String::from("test title");
        let expected_status = TaskStatus::DONE;
        let new_base_struct = Base{
            title: expected_title.clone(),
            status: TaskStatus::DONE
        };
        assert_eq!(expected_title,
                   new_base_struct.title);
        assert_eq!(expected_status,
                   new_base_struct.status);
    }
}
```

In the preceding code, we merely create a struct and assess the fields of that struct, ensuring that they are what we expect them to be. We can see that we created our `test` module, which is annotated with a `#[cfg(test)]` attribute. The `#[cfg(test)]` attribute is a conditional check where the code is only active if we run `cargo test`. If we do not run `cargo test`, the code annotated with `#[cfg(test)]` is not compiled.

More books: https://t.me/booksforits

nside the module, we will import the `Base` struct from the file outside of the `base_tests` module, which is still in the file. In the Rust world, it is typical to import what we are testing using `super`. There is a well-established standard to have the testing code right under the code that is being tested in the same file. We will then test the `Base::new` function by decorating our `new` function with a `#[test]` attribute.

This is the first time we have covered attributes. An **attribute** is simply metadata applied to modules and functions. This metadata aids the compiler by giving it information. In this case, it is telling the compiler that this module is a test module and that the function is an individual test.

However, if we run the preceding code, it would not work. This is because the `Eq` trait is not implemented in the `TaskStatus` enum, meaning that we cannot execute the following line of code:

```
assert_eq!(expected_status, new_base_struct.status);
```

This also means that we cannot use the == operator between two `TaskStatus` enums. Therefore, before we try and run our test, we will have to implement the `Eq` trait on the `TaskStatus` enum in the `src/to_do/structs/enums.rs` file with the following code:

```
#[derive(Clone, Eq, Debug)]
pub enum TaskStatus {
    DONE,
    PENDING
}
```

We can see that we have implemented the `Eq` and `Debug` traits, which are needed for the `assert_eq!` macro. However, our test still will not run because we have not defined the rules around equating two `TaskStatus` enums. We could implement the `PartialEq` trait by simply adding the `PartialEq` trait to our derive annotation. However, we should explore how to write our own custom logic. To define the equating rules, we implement the `eq` function under the `PartialEq` trait with the following code:

```
impl PartialEq for TaskStatus {
    fn eq(&self, other: &Self) -> bool {
        match self {
            TaskStatus::DONE => {
                match other {
                    &TaskStatus::DONE => return true,
                    &TaskStatus::PENDING => false
                }
            },
            TaskStatus::PENDING => {
```

More books: https://t.me/booksforits

```
                match other {
                    &TaskStatus::DONE => return false,
                    &TaskStatus::PENDING => true
                }
            }
        }
    }
}
```

Here, we can see that we manage to confirm if the `TaskStatus` enum is equal to the other `TaskStatus` enum being compared using two match statements. It seems more intuitive to use the `==` operator in the `eq` function; however, using the `==` operator calls the `eq` function resulting in an infinite loop. The code will still compile if you use the `==` operator in the `eq` function but if you run it you will get the following unhelpful error:

```
fatal runtime error: stack overflow
```

We have now essentially created a new `base` struct and then checked to see if the fields are what we expected. To run this, run the `cargo test` functionality, pointing it to the file we want to test, which is denoted by the following command:

```
cargo test to_do::structs::base
```

We will get the following output:

```
running 1 test
test to_do::structs::base::base_tests::new ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.00s
```

We can see that our test was run and that it passed. Now, we'll move on to writing tests for the rest of the module, which are the `Done` and `Pending` structs. Now is the time to see if you can write a basic unit test in the `src/to_do/structs/done.rs` file. If you have attempted to write a unit test for the `Done` struct in the `src/to_do/structs/done.rs` file, your code should look like the following code:

```
#[cfg(test)]
mod done_tests {

    use super::Done;
```

More books: https://t.me/booksforits

```
    use super::TaskStatus;

    #[test]
    fn new() {
        let new_base_struct = Done::new("test title");
        assert_eq!(String::from("test title"),
                   new_base_struct.super_struct.title);
        assert_eq!(TaskStatus::DONE,
                   new_base_struct.super_struct.status);
    }
}
```

We can run both tests with the following command:

```
cargo test
```

This gives the following output:

```
running 2 tests
test to_do::structs::base::base_tests::new ... ok
test to_do::structs::done::done_tests::new ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0
measured; 0 filtered out; finished in 0.00s
```

Running `cargo test` runs all the tests across all Rust files. We can see that all our tests have now run and passed.

Now that we have done some basic testing, let's look at the other modules that we can test. Our JSON serialization and views can be tested in our functional tests with **Postman**. Our database models do not have any advanced functionality that we have purposefully defined.

Building JWT unit tests

All our models do is read and write to the database. This has been shown to work. The only module left that we'll unit test is the `auth` module. Here, we have some logic that has multiple outcomes based on the inputs. We also must do some mocking as some of the functions accept `actix_web` structs, which have certain fields and functions. Luckily for us, `actix_web` has a test module that enables us to mock requests.

More books: https://t.me/booksforits

Building a configuration for tests

Before we start building our unit tests for the JWT, we must remember that there is a dependency on the `config` file to get the secret key. Unit tests must be isolated. They should not need to have the correct parameters passed into them to work. They should work every time in isolation. Because of this, we are going to have to build a `new` function for our `Config` struct in our `src/config.rs` file. The outline for the coding tests will look like the following code:

```
impl Config {

    // existing function reading from file
    #[cfg(not(test))]
    pub fn new() -> Config {
        . . .
    }

    // new function for testing
    #[cfg(test)]
    pub fn new() -> Config {
        . . .
    }
}
```

The preceding outline shows that there are two `new` functions. Our new `new` function gets compiled if tests are being run, and the old `new` function gets compiled if the server is running as normal. Our test `new` function has the standard values hardcoded in with the following code:

```
let mut map = HashMap::new();
map.insert(String::from("DB_URL"),
           serde_yaml::from_str(
           "postgres://username:password@localhost:5433/
           to_do").unwrap());
map.insert(String::from("SECRET_KEY"),
           serde_yaml::from_str("secret").unwrap());
map.insert(String::from("EXPIRE_MINUTES"),
           serde_yaml::from_str("120").unwrap());
map.insert(String::from("REDIS_URL"),
```

More books: https://t.me/booksforits

```
            serde_yaml::from_str("redis://127.0.0.1/")
            .unwrap());
return Config {map}
```

These default functions are the same as our development `config` file; however, we know that these variables are going to be consistent. We do not need to pass in anything when running the tests and we do not run the risk of reading another file. Now that our tests have been configured, we can define the requirements, including the configuration for our JWT tests.

Defining the requirements for JWT tests

Now that we have secured our `Config` struct for tests, we can go to our `src/jwt.rs` file and define the imports for our tests with the following code:

```
#[cfg(test)]
mod jwt_tests {
    use std::str::FromStr;

    use super::{JwToken, Config};
    use actix_web::{HttpRequest, HttpResponse,
                    test::TestRequest, web, App};
    use actix_web::http::header::{HeaderValue,
                                  HeaderName, ContentType};
    use actix_web::test::{init_service, call_service};
    use actix_web;
    use serde_json::json;
    use serde::{Deserialize, Serialize};

    #[derive(Debug, Serialize, Deserialize)]
    pub struct ResponseFromTest {
        pub user_id: i32,
        pub exp_minutes: i32
    }
    . . .
}
```

More books: https://t.me/booksforits

With the preceding code, we can import a range of `actix_web` structs and functions to enable us to create fake HTTP requests and send them to a fake application to test how the `JwToken` struct works during the HTTP request process. We will also define a `ResponseFromTest` struct that can be processed to and from JSON to extract the user ID from the HTTP request as the `JwToken` struct houses the user ID. The `ResponseFromTest` struct is the HTTP response we are expecting to have so we are closely mocking the response object.

Now that we have imported all that we need, we can define the outline of our tests with the following code:

```
#[cfg(test)]
mod jwt_tests {
    . . .
    #[test]
    fn get_key() {
        . . .
    }
    #[test]
    fn get_exp() {
        . . .
    }
    #[test]
    fn decode_incorrect_token() {
        . . .
    }
    #[test]
    fn encode_decode() {
        . . .
    }
    async fn test_handler(token: JwToken,
                          _: HttpRequest) -> HttpResponse {
        . . .
    }
    #[actix_web::test]
    async fn test_no_token_request() {
        . . .
    }
    #[actix_web::test]
    async fn test_passing_token_request() {
```

More books: https://t.me/booksforits

```
        . . .
    }

    #[actix_web::test]
    async fn test_false_token_request() {
        . . .
    }
}
```

Here, we can see that we test the getting of the key and the encoding and decoding of the token. They are native functions to the `JwToken` struct and, with what we have covered previously, you should be able to write them yourself. The other functions are decorated with `#[actix_web::test]`. This means that we are going to create fake HTTP requests to test how our `JwToken` implements the `FromRequest` trait. Now, there is nothing stopping us from writing the tests, which we will cover in the next section.

Building basic function tests for JWT

We will start with the most basic test, getting the key, which takes the following form:

```
#[test]
fn get_key() {
    assert_eq!(String::from("secret"), JwToken::get_key());
}
```

We must remember that `"secret"` is the hardcoded key defined in the `Config::new` function for test implementations. If the test `Config::new` function works, the aforementioned test will work. Getting the expiry can also be important. Because we directly rely on the expiration minutes to be extracted from `config`, the following test will ensure that we are returning 120 minutes:

```
#[test]
fn get_exp() {
    let config = Config::new();
    let minutes = config.map.get("EXPIRE_MINUTES")
                        .unwrap().as_i64().unwrap();
    assert_eq!(120, minutes);
}
```

More books: https://t.me/booksforits

We can now move on to test how invalid tokens are handled with the following test:

```
#[test]
fn decode_incorrect_token() {
    let encoded_token: String =
        String::from("invalid_token");
    match JwToken::from_token(encoded_token) {
        Err(message) => assert_eq!("InvalidToken",
                                   message),
        _ => panic!(
            "Incorrect token should not be able to be
             encoded"
             )
    }
}
```

Here, we pass in an `"invalid_token"` string that should fail the decoding process because it is clearly not a valid token. We will then match the outcome. If the outcome is an error, we will then assert that the message is that the error is a result of an invalid token. If there is any other output apart from an error, then we throw an error failing the test because we expect the decode to fail.

Now that we have written two tests for our `JwToken` struct functions, this is a good time for you to attempt to write the test for encoding and decoding a token. If you have attempted to write the encoding and decoding test, it should look like the following code:

```
#[test]
fn encode_decode() {
    let test_token = JwToken::new(5);
    let encoded_token = test_token.encode();
    let new_token =
        JwToken::from_token(encoded_token).unwrap();
    assert_eq!(5, new_token.user_id);
}
```

The preceding test essentially boils down the login and authenticated request process around the token. We create a new token with a user ID, encode the token, and then decode the token testing to see if the data we passed into the token is the same as we get out when we decode it. If we don't, then the test will fail.

More books: https://t.me/booksforits

Now that we have finished testing the functions for the `JwToken` struct, we can move on to testing how the `JwToken` struct implements the `FromRequest` trait. Before we do this, we must define a basic view function that will merely handle the authentication of `JwToken` and then returns the user ID from the token with the following code:

```
async fn test_handler(token: JwToken,
                      _: HttpRequest) -> HttpResponse {
    return HttpResponse::Ok().json(json!({"user_id":
                                          token.user_id,
                                         "exp_minutes":
                                          60}))
}
```

This is nothing new, in fact, this outline is also how we define our views in our application. With our basic tests defined, we can move on to building tests for web requests.

Building tests for web requests

We can now test our test view to see how it handles a request with no token in the header with the following code:

```
#[actix_web::test]
async fn test_no_token_request() {
    let app = init_service(App::new().route("/", web::get()
                                .to(test_handler))).await;
    let req = TestRequest::default()
        .insert_header(ContentType::plaintext())
        .to_request();
    let resp = call_service(&app, req).await;
    assert_eq!("401", resp.status().as_str());
}
```

In the preceding code, we can see that we can create a fake server and attach our `test_handler` test view to it. We can then create a fake request that does not have any token in the header. We will then call the server with the fake request, then assert that the response code of the request is unauthorized. We can now create a test that inserts a valid token with the following code:

```
#[actix_web::test]
async fn test_passing_token_request() {
    let test_token = JwToken::new(5);
```

More books: https://t.me/booksforits

```
    let encoded_token = test_token.encode();

    let app = init_service(App::new().route("/", web::get()
                                .to(test_handler))).await;
    let mut req = TestRequest::default()
        .insert_header(ContentType::plaintext())
        .to_request();
    let header_name = HeaderName::from_str("token")
                                                .unwrap();
    let header_value = HeaderValue::from_str(encoded_token
                                                .as_str())
                                                .unwrap();
    req.headers_mut().insert(header_name, header_value);

    let resp: ResponseFromTest = actix_web::test::
        call_and_read_body_json(&app, req).await;
    assert_eq!(5, resp.user_id);
}
```

Here, we can see that we create a valid token. We can create our fake server and attach our `test_handler` function to that fake server. We will then create a request that can be mutated. Then, we will insert the token into the header and call the fake server with the fake request, using the `call_and_read_body_json` function. It must be noted that when we call the `call_and_read_body_json` function, we declare that the type returned under the `resp` variable name to be `ResponseFromTest`. We then assert that the user ID is from the request response.

Now that we have seen how to create a fake HTTP request with a header, this is a good opportunity for you to try and build the test that makes a request with a fake token that cannot be decoded. If you have attempted this, it should look like the following code:

```
#[actix_web::test]
async fn test_false_token_request() {
    let app = init_service(App::new().route("/", web::get()
                .to(test_handler))).await;
    let mut req = TestRequest::default()
        .insert_header(ContentType::plaintext())
        .to_request();
    let header_name = HeaderName::from_str("token")
        .unwrap();
```

More books: https://t.me/booksforits

```
    let header_value = HeaderValue::from_str("test")
        .unwrap();
    req.headers_mut().insert(header_name, header_value);
    let resp = call_service(&app, req).await;
    assert_eq!("401", resp.status().as_str());
}
```

Looking at the following code, we can see that we inserted a false token into the header using the approach laid out in the passing token request test with the unauthorized assertion used in the test with no token provided. If we run all the tests now, we should get the following printout:

```
running 9 tests
test to_do::structs::base::base_tests::new ... ok
test to_do::structs::done::done_tests::new ... ok
test to_do::structs::pending::pending_tests::new ... ok
test jwt::jwt_tests::get_key ... ok
test jwt::jwt_tests::decode_incorrect_token ... ok
test jwt::jwt_tests::encode_decode ... ok
test jwt::jwt_tests::test_no_token_request ... ok
test jwt::jwt_tests::test_false_token_request ... ok
test jwt::jwt_tests::test_passing_token_request ... ok

test result: ok. 9 passed; 0 failed; 0 ignored;
0 measured; 0 filtered out; finished in 0.00s
```

From the preceding output, our `jwt` and `to_do` modules are now fully unit-tested. Considering that Rust is still a new language, we have managed to painlessly unit test our code because we structured our code in a modular fashion.

The `tests` crate that `actix_web` provided enabled us to test edge cases quickly and easily. In this section, we tested how our functions processed requests with missing tokens, false tokens, and correct tokens. We have seen first-hand how Rust enables us to run unit tests on our code.

Everything is configured with `cargo`. We do not have to set up paths, install extra modules, or configure environment variables. All we must do is define modules with the `test` attribute and run the `cargo test` command. However, we must remember that our views and JSON serialization code are not unit-tested. This is where we switch to Postman to test our API endpoints.

More books: https://t.me/booksforits

Writing tests in Postman

In this section, we will be implementing functional integration tests using Postman to test our API endpoints. This will test our JSON processing and database access. To do this, we will follow these steps:

1. We are going to have to create a test user for our Postman tests. We can do this with the JSON body shown as follows:

```
{
    "name": "maxwell",
    "email": "maxwellflitton@gmail.com",
    "password": "test"
}
```

2. We need to add a `POST` request to the `http://127.0.0.1:8000/v1/user/create` URL. Once we have done this, we can use our login endpoint for our Postman tests. Now that we have created our test user, we must get the token from the response header of the `POST` request to the `http://127.0.0.1:8000/v1/auth/login` URL with the JSON request body:

```
{
    "username": "maxwell",
    "password": "test"
}
```

This gives us the following Postman layout:

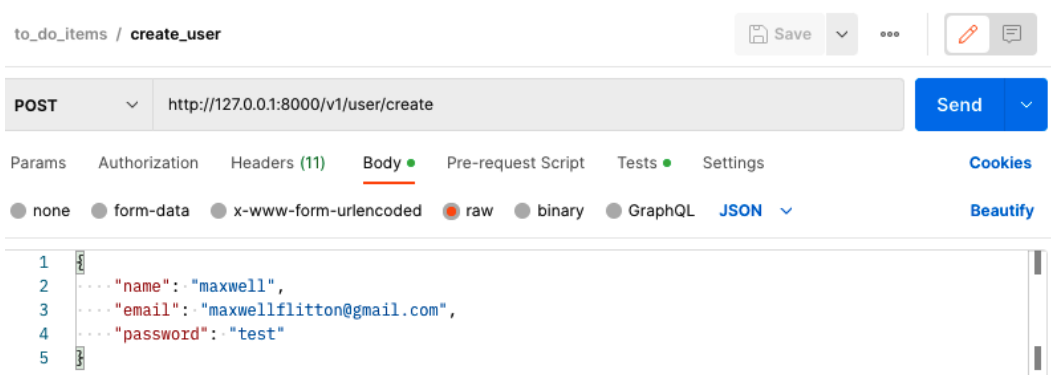


Figure 9.1 – Creating a new use Postman request

With this token, we have all the information needed to create our Postman collection. Postman is a collection of API requests. In this collection, we can bunch all our to-do item API calls together using the user token as authentication. The result of the call is as follows:

More books: https://t.me/booksforits

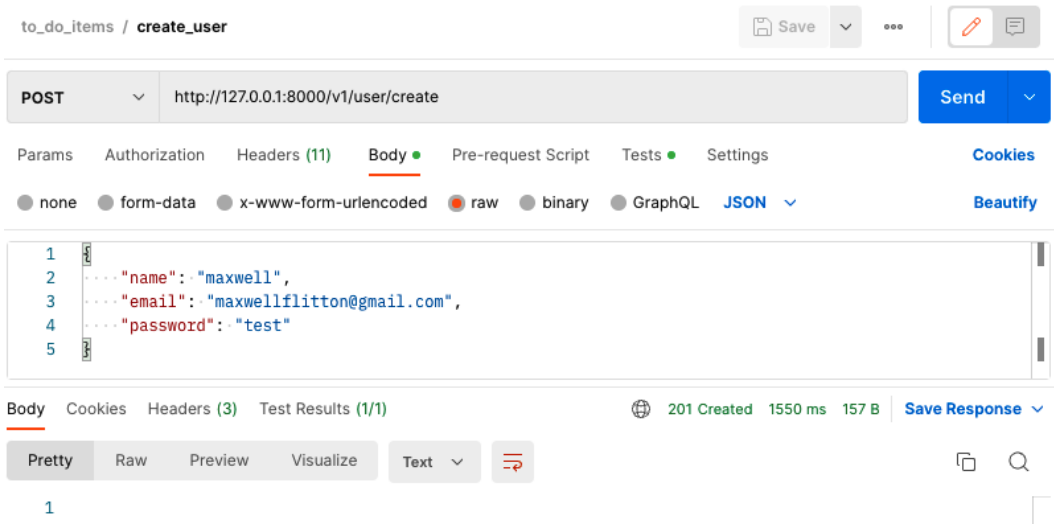


Figure 9.2 – Creating new use Postman response

3. We can create our collection with the following Postman button, that is, + **New Collection**:

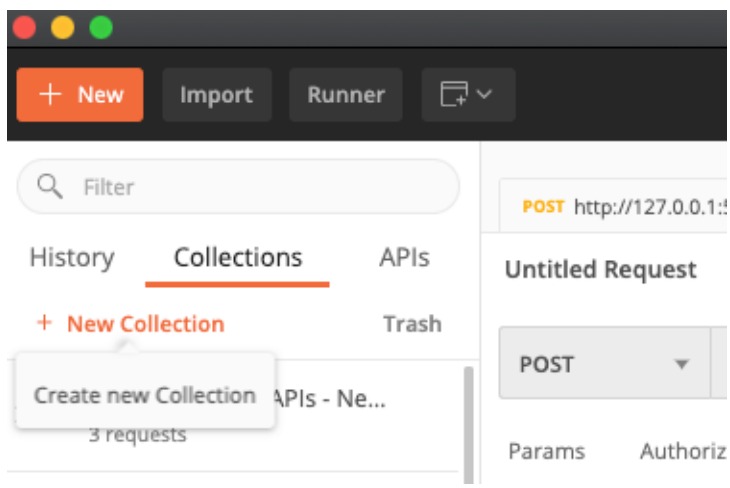


Figure 9.3 – Creating new Postman collection

More books: https://t.me/booksforits

4. Once we have clicked this, we must make sure that our user token is defined for the collection, as all to-do item API calls need the token. This can be done by using the **Authorization** configuration for our API calls, as seen in the following screenshot:

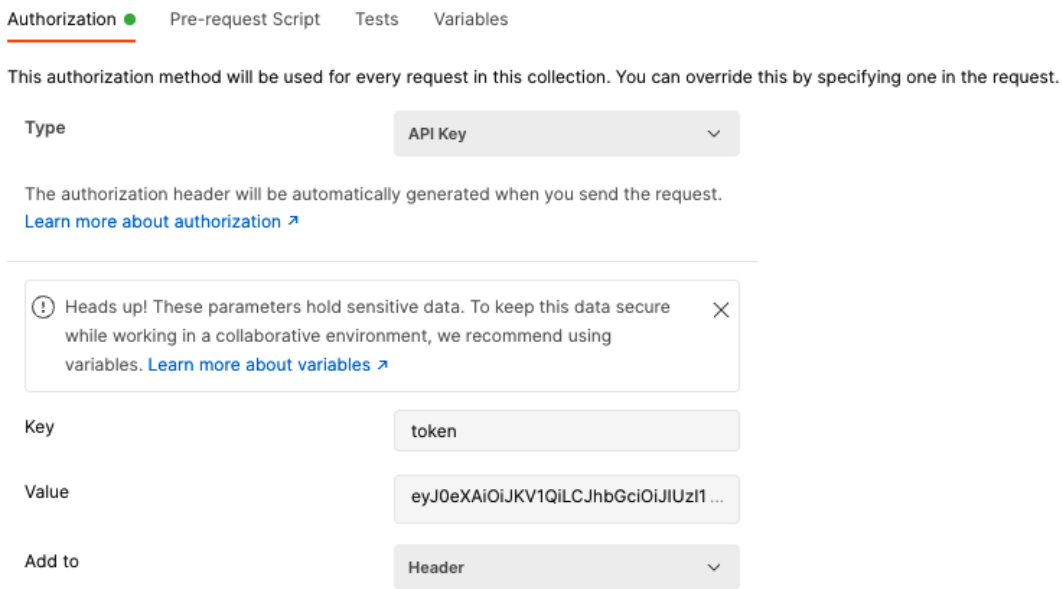


Figure 9.4 – Defining AUTH credentials in a new Postman collection

We can see that we have merely copied and pasted our token into the value with **token** as the key, which will be inserted into the header of the requests. This should now be passed in all our requests in the collection. This collection is now stored on the left-hand side navigation bar under the **Collections** tab.

5. We have now configured our collection and can now add requests under the collection by clicking the grayed-out **Add Request** button shown in this screenshot:

More books: https://t.me/booksforits

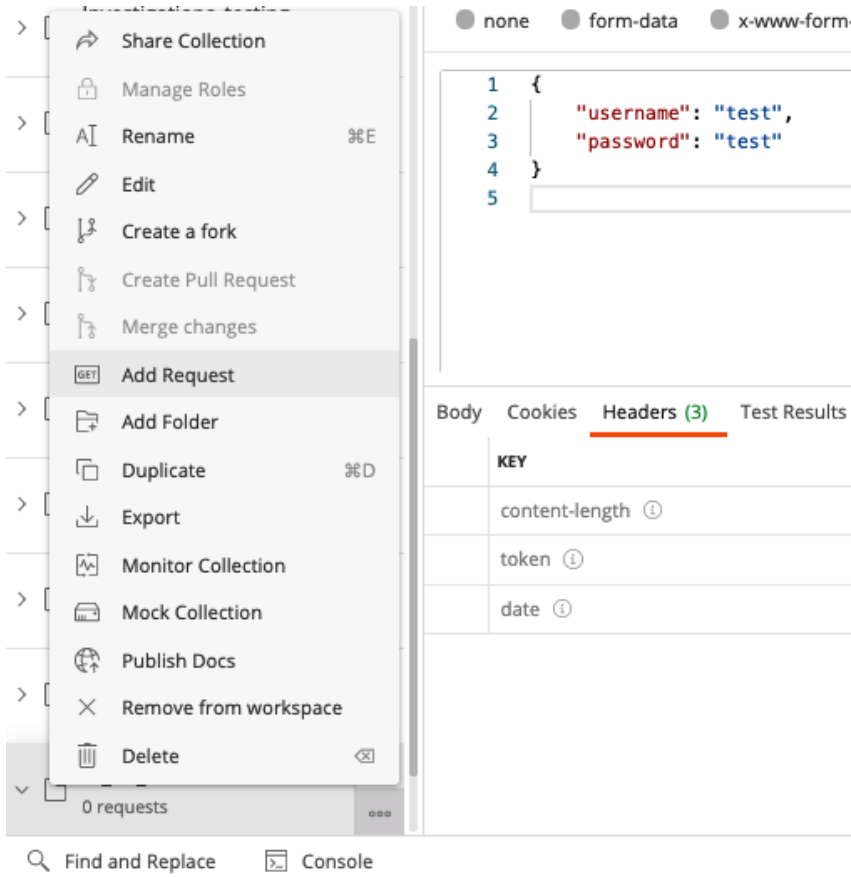


Figure 9.5 – Creating a new request for our Postman collection

Now, we must think about our approach to testing the flow of testing as this has to be self-contained.

Writing ordered requests for tests

Our requests will take the following order:

1. **Create**: Create a to-do item, then check the return to see if it is stored correctly.
2. **Create**: Create another to-do item, checking the return to see if the previous one is stored and that the process can handle two items.
3. **Create**: Create another to-do item with the same title as one of the other items, checking the response to ensure that our application is not storing duplicate to-do items with the same title.
4. **Edit**: Edit an item, checking the response to see if the edited item has been changed to *done* and that it is stored in the correct list.

More books: https://t.me/booksforits

5. **Edit**: Edit the second item to see if the *edit* effect is permanent and that the *done* list supports both items.
6. **Edit**: Edit an item that is not present in the application to see if the application handles this correctly.
7. **Delete**: Delete one to-do item to see if the response no longer returns the deleted to-do item, meaning that it is no longer stored in the database.
8. **Delete**: Delete the final to-do item, checking the response to see if there are no items left, showing that the *delete* action is permanent.

We need to run the preceding tests for them to work as they rely on the previous action being correct. When we create a request for the collection, we must be clear about what the request is doing, which step it is on, and what type of request it is. For instance, creating our first *create* test will look like the following:

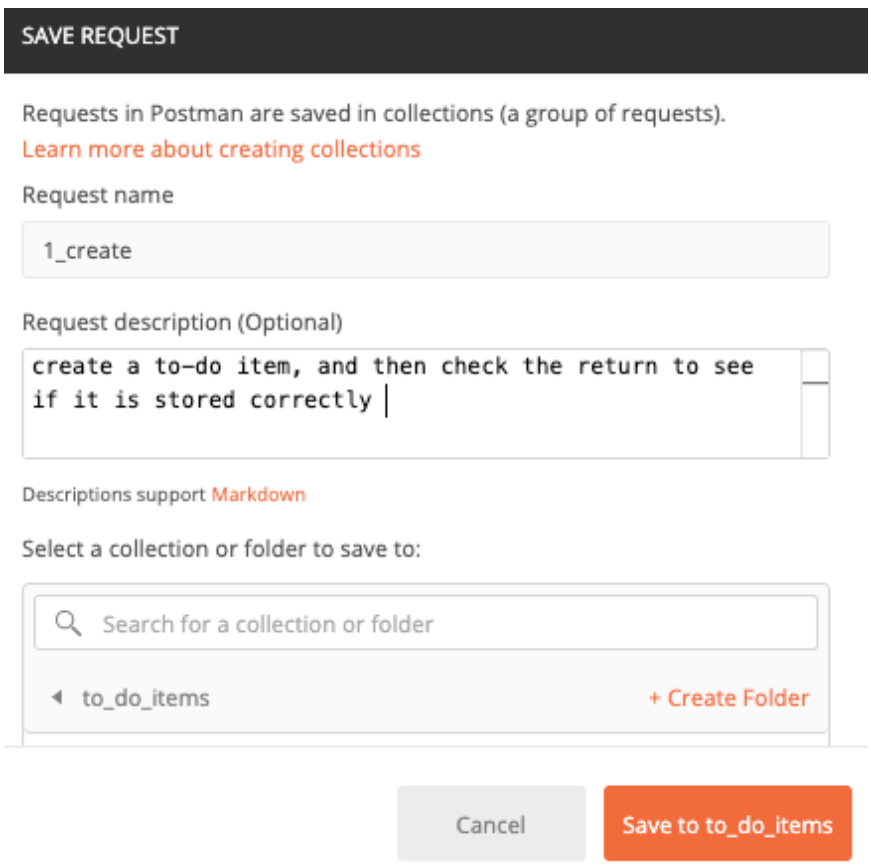


Figure 9.6 – Creating our first Postman create request

More books: https://t.me/booksforits

As we can see, the step is appended with the type by an underscore. We then put the description of the test from the list in the **Request description (Optional)** field. When defining the request, you may realize that the API key is not in the header of the request.

This is because it is in the hidden autogenerated headers of the request. Our first request must be a `POST` request with the `http://127.0.0.1:8000/v1/item/create/washing` URL.

This creates the to-do item *washing*. However, before we click the **Send** button, we must move over to the **Tests** tab in our Postman request, just to the left of the **Settings** tab, to write our tests as seen in the following screenshot:

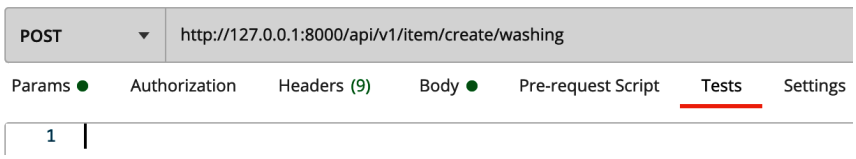


Figure 9.7 – Accessing the tests script in Postman

Our tests must be written in JavaScript. However, we can get access to Postman's `test` library by typing `pm` into the test script. First, at the top of the test script, we need to process the request, which is done with this code:

```
var result = pm.response.json()
```

With the preceding line, we can access the response JSON throughout the test script. To comprehensively test our request, we need to follow these steps:

1. First, we need to check the basic content of the response. Our first test is to check to see if the response is `200`. This can be done with the following code:

   ```
   pm.test("response is ok", function () {
       pm.response.to.have.status(200);
   });
   ```

   Here, we define the test description. Then, the function that the test runs is defined.

2. Then, we check the length of data in the response. After the preceding test, we will define our test to check if the pending item has a length of one via the following code:

   ```
   pm.test("returns one pending item", function(){
       if (result["pending_items"].length !== 1){
           throw new Error(
           "returns the wrong number of pending items");
       }
   })
   ```

More books: https://t.me/booksforits

In the preceding code, we do a simple check of the length and throw an error if the length is not one as we only expect one pending item in the `pending_items` list.

3. Then, we inspect the title and status of the pending item in the following code:

```
pm.test("Pending item has the correct title", function(){
    if (result["pending_items"][0]["title"] !==
        "washing"){
        throw new Error(
        "title of the pending item is not 'washing'");
    }
})
pm.test("Pending item has the correct status",
         function()
    {
        if (result["pending_items"][0]["status"] !==
            "PENDING"){
            throw new Error(
            "status of the pending item is not
                'pending'");
    }
})
```

In the preceding code, we throw an error if the status or title does not match what we want. Now we have satisfied our tests for the pending items, we can move on to the tests for the done items.

4. Seeing as our done items should be zero, the tests have the following definition:

```
pm.test("returns zero done items", function(){
    if (result["done_items"].length !== 0){
        throw new Error(
        "returns the wrong number of done items");
    }
})
```

In the preceding code, we are merely ensuring that the `done_items` array has a length of zero.

5. Now, we must check the counts of our done and pending items. This is done in the following code:

```
pm.test("checking pending item count", function(){
    if (result["pending_item_count"] !== 1){
        throw new Error(
```

More books: https://t.me/booksforits

```
            "pending_item_count needs to be one");
    }
})
pm.test("checking done item count", function(){
    if (result["done_item_count"] !== 0){
        throw new Error(
        "done_item_count needs to be zero");
    }
})
```

Now that our tests are built, we can make the request by clicking the **SEND** button in Postman to get the following output for the tests:

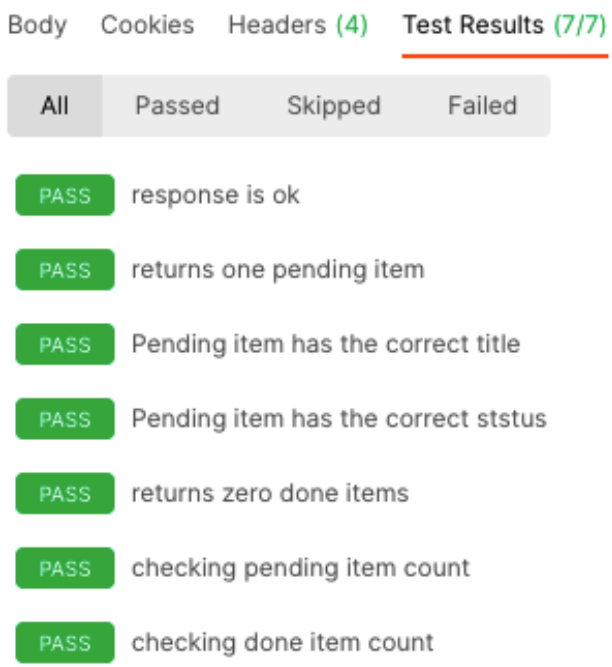


Figure 9.8 – Postman tests output

We can see that our test descriptions and the status of the test are highlighted. If you get an error the status will be red with a **FAIL**. Now that our first create test has been done, we can create our second create test.

More books: https://t.me/booksforits

Creating a test for an HTTP request

We can then create the `2_create` test with this URL: `http://127.0.0.1:8000/v1/item/create/cooking`. This is a good opportunity to try and build the test yourself with the testing methods that we have explored in the previous step. If you have attempted to build the tests, they should look like the following code:

```
var result = pm.response.json()

pm.test("response is ok", function () {
    pm.response.to.have.status(200);
});
pm.test("returns two pending item", function(){
    if (result["pending_items"].length !== 2){
        throw new Error(
        "returns the wrong number of pending items");
    }
})
pm.test("Pending item has the correct title", function(){
    if (result["pending_items"][0]["title"] !== "washing"){
        throw new Error(
        "title of the pending item is not 'washing'");
    }
})
pm.test("Pending item has the correct status", function(){
    if (result["pending_items"][0]["status"] !==
        "PENDING"){
        throw new Error(
        "status of the pending item is not 'pending'");
    }
})
pm.test("Pending item has the correct title", function(){
    if (result["pending_items"][1]["title"] !== "cooking"){
        throw new Error(
        "title of the pending item is not 'cooking'");
    }
})
pm.test("Pending item has the correct status", function(){
```

More books: https://t.me/booksforits

```
    if (result["pending_items"][1]["status"] !==
        "PENDING"){
        throw new Error(
        "status of the pending item is not 'pending'");
    }
})
pm.test("returns zero done items", function(){
    if (result["done_items"].length !== 0){
        throw new Error(
        "returns the wrong number of done items");
    }
})
pm.test("checking pending item count", function(){
    if (result["pending_item_count"].length === 1){
        throw new Error(
        "pending_item_count needs to be one");
    }
})
pm.test("checking done item count", function(){
    if (result["done_item_count"].length === 0){
        throw new Error(
        "done_item_count needs to be zero");
    }
})
```

We can see that we have added a couple of extra tests on the second pending item. The preceding tests also directly apply to the `3_create` test as a duplicate creation will be the same as we will be using the same URL as `2_create`.

The preceding tests require a fair amount of repetition in these tests, slightly altering the length of arrays, item counts, and attributes within these arrays. This is a good opportunity to practice basic Postman tests. If you need to cross-reference your tests with mine, you can assess them in the JSON file at the following URL: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/blob/main/chapter09/building_test_pipeline/web_app/scripts/to_do_items.postman_collection.json`.

More books: https://t.me/booksforits

In this section, we have put in a series of steps for Postman to test when an API call is made. This is not just useful for our application. Postman can hit any API on the internet it has access to. Therefore, you can use Postman tests to monitor live servers and third-party APIs.

Now, running all these tests can be arduous if it must be done manually every time. We can automate the running and checking of all the tests in this collection using **Newman**. If we automate these collections, we can run tests at certain times every day on live servers and third-party APIs we rely on, alerting us to when our servers or the third-party API breaks.

Newman will give us a good foundation for further development in this area. In the next section, we'll export the collection and run all the API tests in the exported collection in sequence using Newman.

Automating Postman tests with Newman

To automate the series of tests, in this section, we will export our to-do item Postman collection in the correct sequence. But first, we must export the collection as a JSON file. This can be done by clicking on our collection in Postman on the left-hand navigation bar and clicking the grayed-out **Export** button, as seen in the following screenshot:

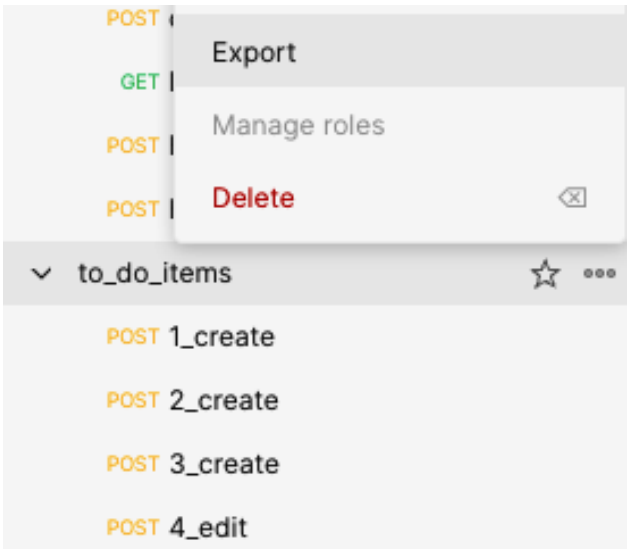


Figure 9.9 – Exporting our Postman collection

More books: https://t.me/booksforits

Now that we have exported the collection, we can quickly inspect it to see how the file is structured. The following code defines the header of the suite of tests:

```
"info": {
    "_postman_id": "bab28260-c096-49b9-81e6-b56fc5f60e9d",
    "name": "to_do_items",
    "schema": "https://schema.getpostman.com
    /json/collection/v2.1.0/collection.json",
    "_exporter_id": "3356974"
},
```

The preceding code tells Postman what schema is needed to run the tests. If the code is imported into Postman, the ID and name will be visible. The file then goes on to define the individual tests via the code given as follows:

```
"item": [
    {
        "name": "1_create",
        "event": [
            {
                "listen": "test",
                "script": {
                    "exec": [
                        "var result = pm.response.json()",
                        . . .
                    ],
                    "type": "text/javascript"
                }
            }
        ],
        "request": {
            "method": "POST",
            "header": [
                {
```

More books: https://t.me/booksforits

```
                    "key": "token",
                    "value": "eyJhbGciOiJIUzI1NiJ9
                    .eyJ1c2VyX2lkIjo2fQ.
                    uVo7u877IT2GEMpB_gxVtxhMAYAJD8
                    W_XiUoNvR7_iM",
                    "type": "text",
                    "disabled": true
                }
            ],
            "url": {
                "raw": "http://127.0.0.1:8000/
                v1/item/create/washing",
                "protocol": "http",
                "host": ["127", "0", "0", "1"],
                "port": "8000",
                "path": ["v1", "item", "create", "washing"]
            },
            "description": "create a to-do item,
            and then check the
            return to see if it is stored correctly "
        },
        "response": []
    },
```

From the preceding code, we can see that our tests, method, URL, headers, and more are all defined in an array. A quick inspection of the `item` array will show that the tests will be executed in the order that we want.

Now, we can simply run it with Newman. We can install Newman with the following command:

```
npm install -g newman
```

More books: https://t.me/booksforits

Note

It must be noted that the preceding command is a global install, which can sometimes have issues. To avoid this, you can setup a `package.json` file with the following contents:

```
{
  "name": "newman testing",
  "description": "",
  "version": "0.1.0",
  "scripts": {
    "test": "newman run to_do_items.
             postman_collection.json"
  },
  "dependencies": {
    "newman": "5.3.2"
  }
}
```

With this `package.json`, we have defined the test command and the Newman dependency. We can install our dependencies locally with the following command:

```
npm install
```

This then installs all we need under the `node_modules` directory. Instead of running the Newman test command directly, we can use the test command defined in `package.json` with the following command:

```
npm run test
```

Now that we have installed Newman, we can run the collection of tests against the exported collection JSON file with this command:

```
newman run to_do_items.postman_collection.json
```

More books: https://t.me/booksforits

The preceding command runs all the tests and gives us a status report. Each description is printed out and the status is also denoted by the side of the test. The following is a typical printout of an API test being assessed:

```
→ 1_create
    POST http://127.0.0.1:8000/v1/item/create/washing
    [200 OK, 226B, 115ms]
    ✓ response is ok
    ✓ returns one pending item
    ✓ Pending item has the correct title
    ✓ Pending item has the correct status
    ✓ returns zero done items
    ✓ checking pending item count
    ✓ checking done item count
```

The preceding output gives us the name, method, URL, and response. Here, all of them passed. If one did not, then the test description would sport a *cross* instead of a *tick*. We also get the following summary:

	executed	failed
iterations	1	0
requests	8	0
test-scripts	16	0
prerequest-scripts	8	0
assertions	64	0
total run duration: 288ms		
total data received: 1.07kB (approx)		
average response time: 19ms [min: 11ms, max: 63ms, s.d.: 16ms]		

Figure 9.10 – Newman summary

We can see that all our tests passed. With this, we have managed to automate our functional testing, enabling us to test a full workflow with minimal effort. However, what we have done is not maintainable. For instance, our token will expire, meaning that if we run tests later in the month, they will fail. In the next section, we will build an entire automated pipeline that will build our server, update our token, and run our tests.

More books: https://t.me/booksforits

Building an entire automated testing pipeline

When it comes to development and testing, we need an environment that can be torn down and recreated easily. There is nothing worse than building up data in a database on your local machine to be able to develop further features using that data. However, the database container might be deleted by accident, or you may write some code that corrupts the data. Then, you must spend a lot of time recreating the data before you can get back to where you were. If the system is complex and there is missing documentation, you might forget the steps needed to recreate your data. If you are not comfortable with destroying your local database and starting again when developing and testing, there is something wrong and it is only a matter of time before you get caught out. In this section, we are going to create a single Bash script that carries out the following actions:

1. Starts database Docker containers in the background.
2. Compiles the Rust server.
3. Runs unit tests.
4. Starts running the Rust server.
5. Runs migrations to the database running in Docker.
6. Makes an HTTP request to create a user.
7. Makes an HTTP request to log in and get a token.
8. Updates the Newman JSON file with the token from the login.
9. Runs the Newman tests.
10. Removes the files produced in this whole process.
11. Stops the Rust server from running.
12. Stops and destroy the Docker containers that were running for the whole process.

There are a lot of steps laid out in the preceding list. Glancing at this list, it would seem intuitive to break the code blocks that we are going to explore into steps; however, we are going to run nearly all the steps in one Bash script. A lot of the preceding steps outlined can be achieved in one line of Bash code each. It would be excessive to break the code down into steps. Now that we have all the steps needed, we can set up our testing infrastructure. First, we need to set up a `scripts` directory alongside the `src` directory in the root of `web_app`. Inside the `scripts` directory, we then need to have a `run_test_pipeline.sh` script that will run the main testing process. We also need to put our Newman JSON `config` file in the `scripts` directory.

More books: https://t.me/booksforits

We will use `bash` to orchestrate the entire testing pipeline, which is the best tool for orchestrating testing tasks. In our `srcipts/run_test_pipeline.sh` script, we will start out with the following code:

```
#!/bin/bash

# move to directory of the project
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
cd ..
```

In the preceding code, we told the computer that the code block is a Bash script with the `#!/bin/bash` shebang line. Bash scripts run from the current working directory of where the Bash script it called from. We can call the script from multiple directories so we need to ensure that we get the directory of where the script is housed, which is the `scripts` directory, assign that to a variable called `SCRIPTPATH`, move to that directory, and then move out one with the `cd..` command to be in the main directory where the Docker, config, and Cargo files are. We can then spin up our Docker containers in the background with the `-d` flag and loop until the database is accepting connections with the following code:

```
# spin up docker and hold script until accepting connections
docker-compose up -d
until pg_isready -h localhost -p 5433 -U username
do
  echo "Waiting for postgres"
  sleep 2;
done
```

Now that our Docker containers are running, we can now move on to building our Rust server. First, we can compile the Rust server and run our unit tests with the following code:

```
cargo build
cargo test
```

Once the unit tests have been run, we can then run our server in the background with the following code:

```
# run server in background
cargo run config.yml &
SERVER_PID=$!
sleep 5
```

More books: https://t.me/booksforits

With `&` at the end of the command, the `cargo run config.yml` runs in the background. We then get the process ID of the `cargo run config.yml` command and assign it to the variable `SERVER_PID`. We then sleep for 5 seconds to be sure that the server is ready to accept connections. Before we make any API calls to our server, we must run our migrations to the database with the following code:

```
diesel migration run
```

We then move back into our `scripts` directory and make an API call to our server that creates a user:

```
# create the user
curl --location --request POST 'http://localhost:8000/v1/user/
create' \
--header 'Content-Type: application/json' \
--data-raw '{
    "name": "maxwell",
    "email": "maxwellflitton@gmail.com",
    "password": "test"
}'
```

If you are wondering how to use `curl` to make HTTP requests in Bash, you can autogenerate them using your Postman tool. On the right-hand side of the Postman tool, you can see a **Code** button, as shown in the following screenshot:

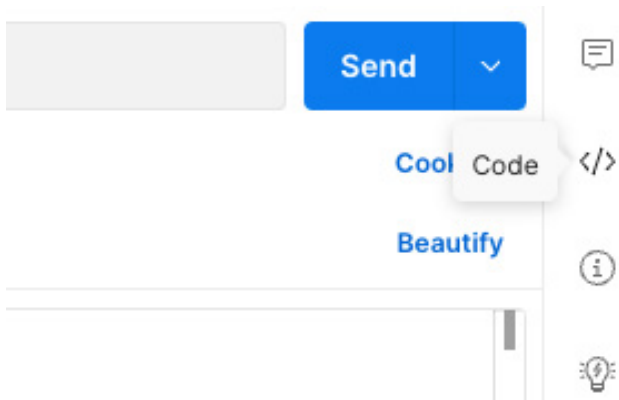


Figure 9.11 – Code generation tool

More books: https://t.me/booksforits

Once you have clicked on the code tag, there is a drop-down menu where you can select from a range of languages. Once you have selected the language you want, your API call will be displayed in a code snippet for your chosen language, which you can then copy and paste.

Now that we have created our user, we can log in and store the token in the `fresh_token.json` file with the following code; however, it must be noted that `curl` first needs to be installed:

```
# login getting a fresh token
echo $(curl --location --request GET 'http://localhost:8000/v1/
auth/login' \
--header 'Content-Type: application/json' \
--data-raw '{
    "username": "maxwell",
    "password": "test"
}') > ./fresh_token.json
```

What is happening here is that we can wrap the result of the API call into a variable with `$(...)`. We then echo this and write it to the file using `echo $(...) > ./fresh_token.json`. We can then insert the fresh token into the Newman data and run the Newman API tests with the following code:

```
TOKEN=$(jq '.token' fresh_token.json)
jq '.auth.apikey[0].value = '"$TOKEN"''
to_do_items.postman_collection.json > test_newman.json

newman run test_newman.json
```

Our testing is now done. We can clean up the files created when running the tests, destroy the Docker containers, and stop our server running with the following code:

```
rm ./test_newman.json
rm ./fresh_token.json

# shut down rust server
kill $SERVER_PID

cd ..
docker-compose down
```

More books: https://t.me/booksforits

> **Note**
>
> `curl` and `jq` both need to be installed before we can run the Bash script. If you are using Linux, you might need to run the following command:
>
> ```
> sudo chmod +x ./run_test_pipeline.sh
> ```

We can then run our testing script with the following command:

```
sh run_test_pipeline.sh
```

Showing the whole printout would just needlessly fill up the book. However, we can see the end of the test printout in the following screenshot:

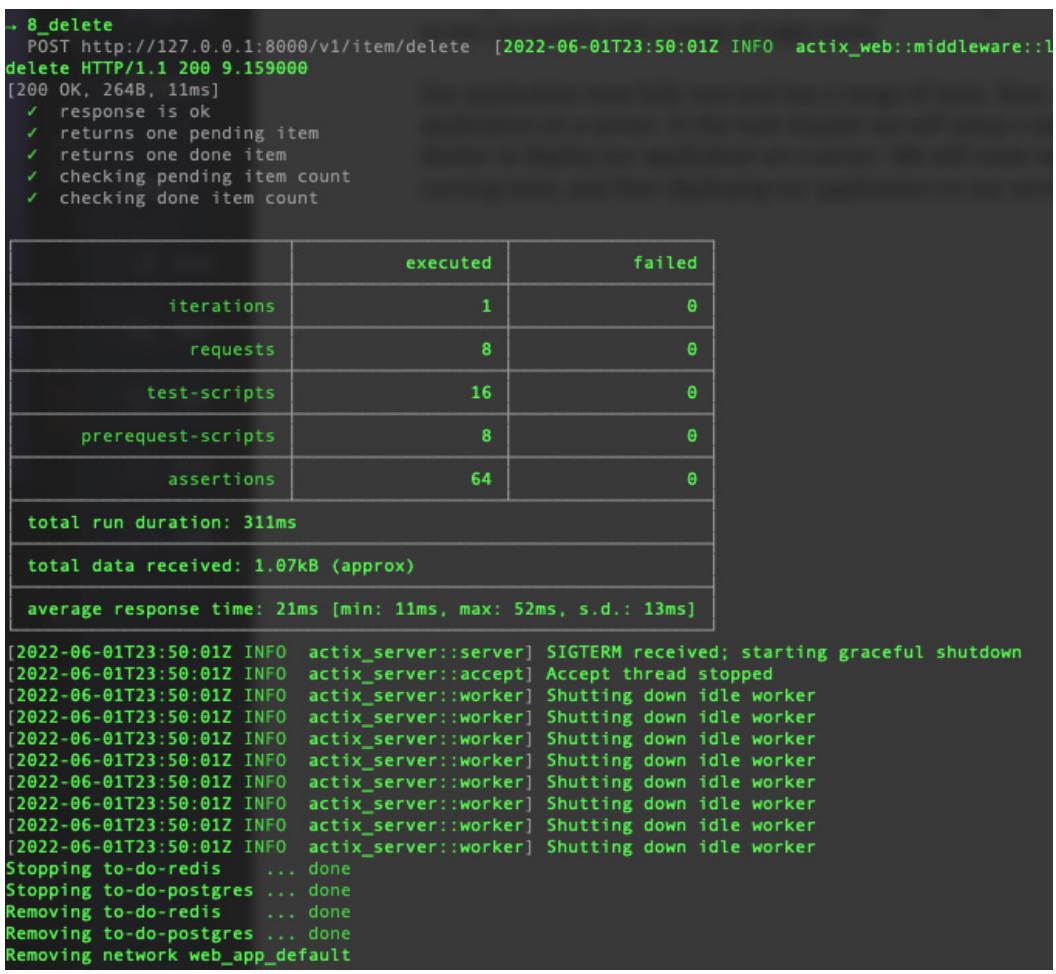


Figure 9.12 – The testing pipeline output

More books: https://t.me/booksforits

Here, the printout makes it clear that the Newman tests have run and passed. After the tests were completed, the server was shut down and the Docker containers that were supporting the server were stopped and removed. If you want to write this log to a `txt` file, you can do so with the following command:

```
sh run_test_pipeline.sh > full_log.txt
```

There you have it! A fully working test pipeline that automates the setting up, testing, and clean-up of our server. Because we have written it in a simple Bash test pipeline, we could integrate these steps in automation pipelines such as Travis, Jenkins, or GitHub Actions. These pipeline tools fire automatically when a `pull` request and merges are performed.

Summary

In this chapter, we went through the workflows and components of our application, breaking them down so we could pick the right tools for the right part. We used unit testing so we could inspect several edge cases quickly to see how each function and struct interacted with others.

We also directly inspected our custom structs with unit tests. We then used the `actix_web` test structs to mock requests to see how the functions that use the structs and process the requests work. However, when we came to the main API views module, we switched to Postman.

This is because our API endpoints were simple. They created, edited, and deleted to-do items. We could directly assess this process by making API calls and inspecting the responses. Out of the box we managed to assess the JSON processing for accepting and returning data. We were also able to assess the querying, writing, and updating of the data in the database with these Postman tests.

Postman enabled us to test a range of processes quickly and efficiently. We even sped up this testing process by automating it via Newman. However, it must be noted that this approach is not a one-size-fits-all approach. If the API view functions become more complex, with more moving parts, such as communicating with another API or service, then the Newman approach would have to be redesigned. Environment variables that trigger mocking such processes would have to be considered so we can quickly test a range of edge cases.

Mocking objects will be needed if the system grows as the dependencies of our structs will grow. This is where we create a fake struct or function and define the output for a test. To do this, we will need an external crate such as `mockall`. The documentation on this crate is covered in the *Further reading* section of this chapter.

Our application now fully runs and has a range of tests. Now, all we have left is to deploy our application on a server.

In the next chapter, we will set up a server on **Amazon Web Services** (**AWS**), utilizing *Docker* to deploy our application on a server. We will cover the process of setting up the AWS configuration, running tests, and then deploying our application on our server if the tests pass.

More books: https://t.me/booksforits

Questions

1. Why do we bother with unit tests if we can just manually play with the application?
2. What is the difference between unit tests and functional tests?
3. What are the advantages of unit tests?
4. What are the disadvantages of unit tests?
5. What are the advantages of functional tests?
6. What are the disadvantages of functional tests?
7. What is a sensible approach to building unit tests?

Answers

1. When it comes to manual testing, you may forget to run a certain procedure. Running tests standardizes our standards and enables us to integrate them into continuous integration tools to ensure new code will not break the server as continuous integration can block new code merges if the code fails.
2. Unit tests isolate individual components such as functions and structs. These functions and structs are then assessed with a range of fake inputs to assess how the component interacts with different inputs. Functional tests assess the system, hitting API endpoints, and checking the response.
3. Unit tests are lightweight and do not need an entire system to run. They can test a whole set of edge cases quickly. Unit tests can also isolate exactly where the error is.
4. Unit tests are essentially isolated tests with made-up inputs. If the type of input is changed in the system but not updated in the unit test, then this test will essentially pass when it should fail. Unit tests also do not assess how the system runs.
5. Functional tests ensure that the entire infrastructure works together as it should. For instance, there could be an issue with how we configure and connect to the database. With unit tests, such problems can be missed. Also, although mocking ensures isolated tests, unit test mocks might be out of date. This means that a mocked function might return data that the updated version does not. As a result, the unit test will pass but the functional tests will not as they test everything.
6. Functional tests need to have the infrastructure to run like a database. There also must be a setup and teardown function. For instance, a functional test will affect the data stored in the database. At the end of the test, the database needs to be wiped before running the test again. This can increase the complications and can require "glue" code between different operations.
7. We start off with testing structs and functions that do not have any dependencies. Once these have been tested, we know that we are comfortable with them. We then move on to the functions and structs that have the dependencies we previously tested. Using this approach, we know that the current test we are writing does not fail due to a dependency.

More books: https://t.me/booksforits

Further reading

- Mockall documentation: `https://docs.rs/mockall/0.9.0/mockall/`
- Github Actions documentation: `https://github.com/features/actions`
- Travis documentation: `https://docs.travis-ci.com/user/for-beginners/`
- Circle CI documentation: `https://circleci.com/docs/`
- Jenkins documentation: `https://www.jenkins.io/doc/`

More books: https://t.me/booksforits

More books: https://t.me/booksforits

10
Deploying Our Application on AWS

In a lot of tutorials and educational materials, deployment is rarely covered. This is because there are a lot of moving parts, and the process can be fairly brittle. It may be more convenient to refer to other resources when mentioning deployment.

In this chapter, we will cover enough to automate deployment on a server on **Amazon Web Services** (**AWS**) and then build and connect to a database from there. It must be stressed that deployment and cloud computing are big topics—there are whole books written on them.

In this chapter, we will get to a point where we can deploy and run our application for others to use. Learning how to deploy applications on a server is the final step. This is where you will turn the application that you have been developing into a practical reality that can be used by others all over the world.

In this chapter, we will cover the following topics:

- Setting up our build environment
- Managing our software with Docker
- Deploying our application on AWS

By the end of this chapter, you will be able to wrap your code up in Docker images and deploy it on a server instance on AWS so that it can be accessed by other users. You will also be able to configure infrastructure using **Terraform**, which is a tool used to define cloud computing infrastructure such as servers and databases as code. Once you have finished this chapter, you will have a few build scripts that create a build and deployment server using Terraform, pass the data from that Terraform build into config files, **SSH** into these servers, and run a series of commands resulting in database migrations and spinning up the Docker containers on our server.

More books: https://t.me/booksforits

Technical requirements

In this chapter, we'll build on the code built in *Chapter 9*, *Testing Our Application Endpoints and Components*. This can be found at the following URL: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter09/building_test_pipeline`.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter10`.

This chapter also has the following requirements:

- We will be using Terraform to automate the building of our servers. Because of this, we will need to install Terraform using the following URL: `https://learn.hashicorp.com/tutorials/terraform/install-cli`.
- When we are using Terraform, it will be making calls to the AWS infrastructure. We will need AWS authentication, which will be done using the AWS client. We will be using the AWS client in this chapter, which can be installed using the following URL: `https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html`.
- You will also need a Docker Hub account so that we can package and deploy our application. This can be found at `https://hub.docker.com/`.
- Since you will be deploying the application on a server, you will need to sign up for an AWS account. This can be done at the following URL: `https://aws.amazon.com/`.

Setting up our build environment

So far, we have been running our application with the `cargo run` command. This has been working well, but you might have noticed that our application is not very fast. In fact, it is relatively slow when we try to log in to the application. This seems to be counterintuitive as we are learning Rust to develop faster applications.

So far, it does not look very fast. This is because we are not running an optimized version of our application. We can do this by adding the `--release` tag. As a result, we run our optimized application using the following command:

```
cargo run --release config.yml
```

Here, we notice that the compilation takes a lot longer. Running this every time we alter the code, and during a development process, is not ideal; hence, we have been building and running in debug mode using the `cargo run` command. However, now that our optimized application is running, we can see that the login process is a lot faster. While we can run the server locally, our aim is to deploy our application on a server. To run our application on a server, we are going to have to build

More books: https://t.me/booksforits

our application in a Docker image. To ensure that our Docker image build runs smoothly, we are going to use an online computing unit on AWS. Before we run our build process, we need to carry out the following steps:

1. Set up an AWS **Secure Shell** (**SSH**) key for **Elastic Compute Cloud** (**EC2**)
2. Set up the AWS client for our local computer
3. Write a Terraform script that builds our build server
4. Write a Python script that manages the build
5. Write a Bash script that orchestrates the build on the server

Once we have done the aforementioned steps, we will be able to run an automated pipeline that creates a build EC2 server on AWS and then build our Rust application. First, let us get started by creating an SSH key for our server.

Setting up an AWS SSH key for an AWS EC2 instance

If we want to run commands on a server, we are going to have to connect to the server using the SSH protocol over the **Hypertext Transfer Protocol** (**HTTP**). However, we cannot have anyone accessing our server as it will not be secure. To stop anyone from connecting to our server and running any commands they want, we will only allow users to connect to our server if they have an SSH key. To create our key, we need to log in to the AWS console and navigate to our EC2 dashboard, which can be accessed via the search box, as seen in the following screenshot:

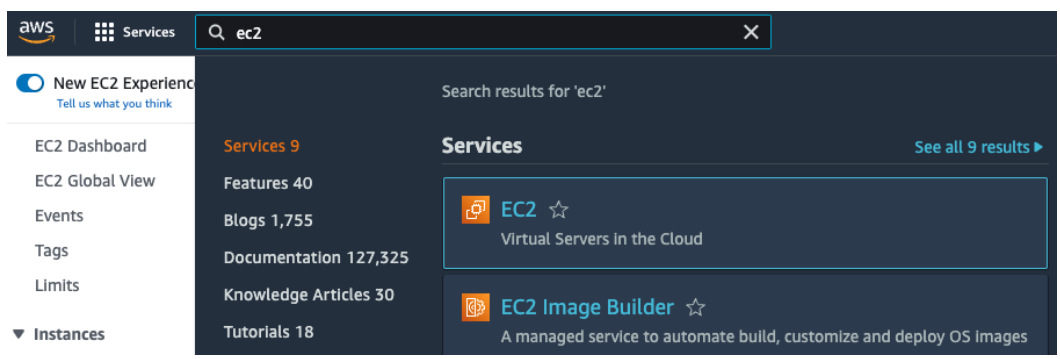


Figure 10.1 – Navigating to the EC2 dashboard with the search box

> **Note**
> It must be noted that AWS might not look like the screenshots in this chapter as AWS keeps changing the UI. However, the fundamental concepts will be the same.

More books: https://t.me/booksforits

Once we have navigated to the EC2 dashboard, we can navigate to **Key Pairs** in the **Network & Security** section of the panel on the left side of the view, as seen in the following screenshot:

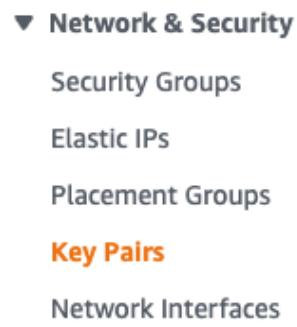


Figure 10.2 – Navigating to Key Pairs

Once we have navigated to the **Key Pairs** section, there will be a list of key pairs that you already own. If you have built EC2 instances before, you might already see some listed. If you have never built an EC2 instance before, then you will not have anything listed. On the top right of the screen, you can create a key by clicking on the **Create key pair** button shown in the following screenshot:

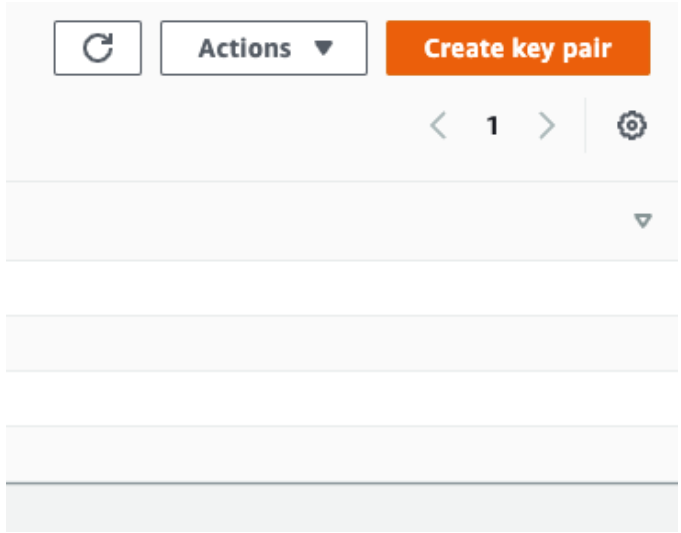


Figure 10.3 – Button to allow the creation of a key pair

Once we have clicked on this button, we will see the following form:

More books: https://t.me/booksforits

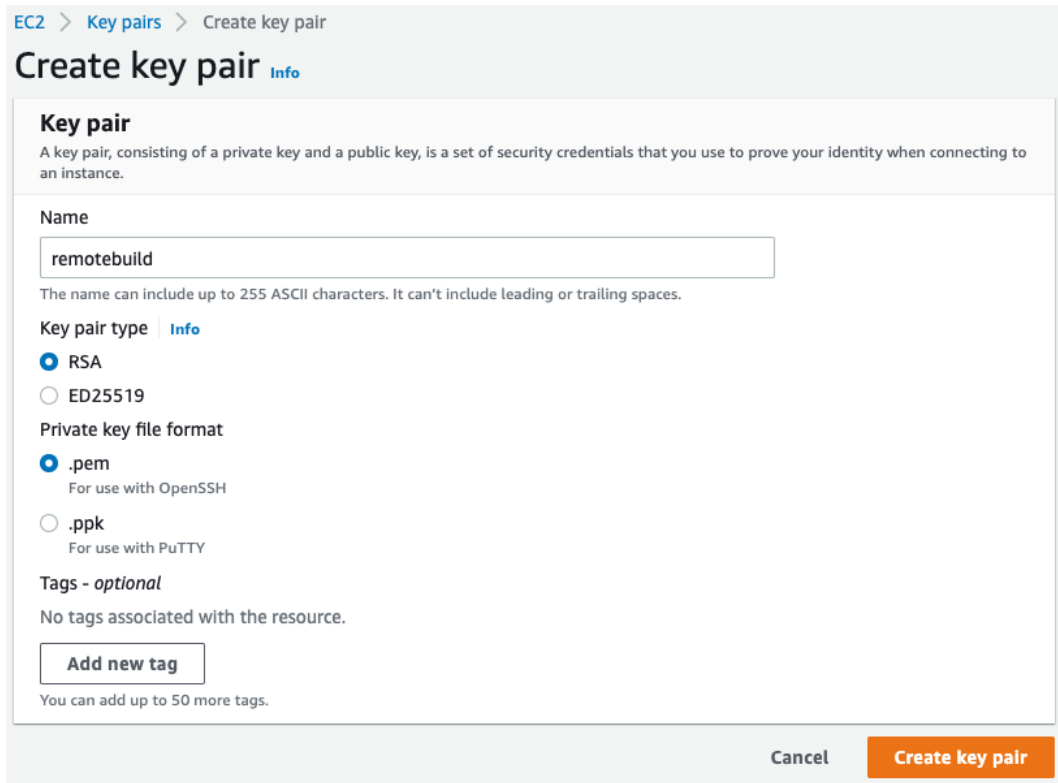


Figure 10.4 – Create key pair form

In the preceding screenshot, we can see that we have named our `remotebuild` key; we also state that our key has the `.pem` format and is of the **RSA** type. Once we click on the **Create key pair** button, you will download the key. We now must store our key. This can be done by storing it in our home directory in the `.ssh` directory. Inside the `.ssh` directory, we can create a `keys` directory with the following command:

```
mkdir "$HOME"/.ssh/keys/
```

The `"$HOME"` environment variable is always available in the Bash shell, and it denotes the home directory for the user. This means that other users who log in to the computer under a different username cannot access the SSH keys we downloaded if we store them in the directory that we have just created. We can now navigate to where our key has been downloaded and copy it to our `keys` directory with the following command:

```
cp ./remotebuild.pem "$HOME"/.ssh/keys/
```

More books: https://t.me/booksforits

We then must change the permissions for the key so that only the owner of the file can read the file with the 600 code for us to use the key for SSH purposes with the following command:

```
chmod 600 "$HOME"/.ssh/keys/remotebuild.pem
```

We now have an SSH key stored in our `.ssh/keys` directory where we can access this key with the correct permissions in order to access the servers that we create. Now that we have this, we need programmatic access to our AWS services by setting up our AWS client.

Setting up our AWS client

We will be using Terraform to automate our server build. In order to do this, Terraform will make calls to the AWS infrastructure. This means that we must have the AWS client installed on our local machine. Before we configure our client, we need to get some programmatic user keys. If we do not obtain programmatic access, our code will not be authorized to build infrastructure on our AWS account. The first step of obtaining the user keys is navigating to the **IAM** section via the search box, as seen in the following screenshot:

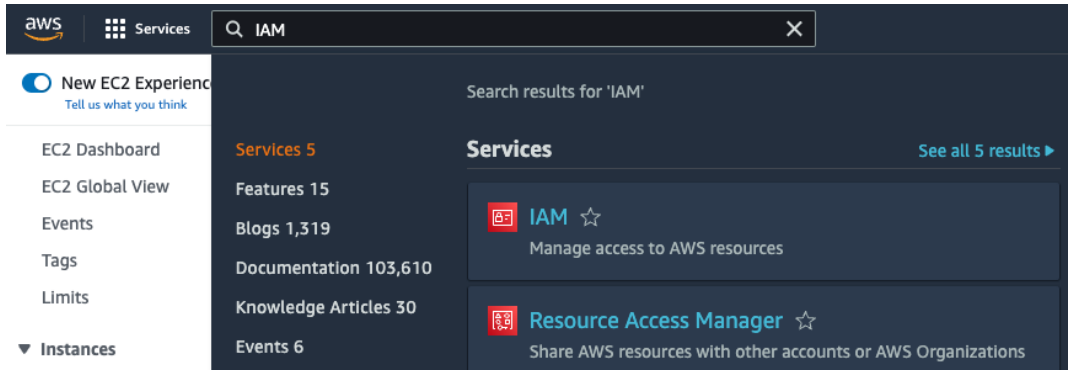


Figure 10.5 – Navigating to the IAM section

Once we have navigated to the **IAM** section, we will get the following layout:

More books: https://t.me/booksforits

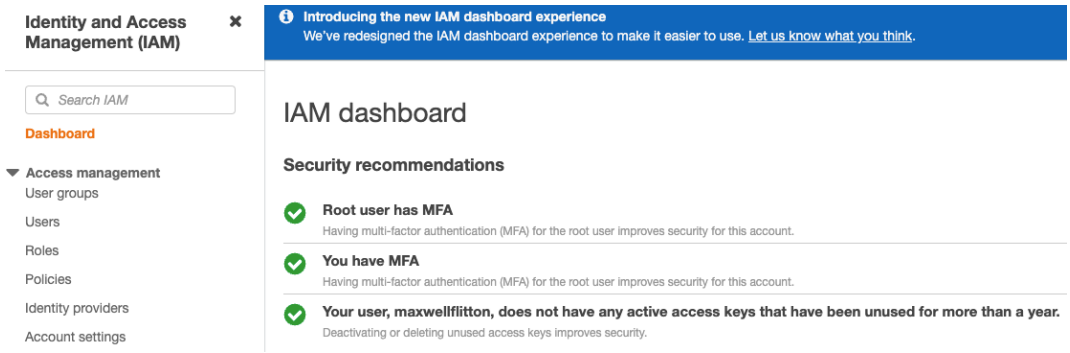


Figure 10.6 – View of the IAM section

We can see in the preceding screenshot that my user has **multi-factor authentication** (**MFA**), and I keep my access key rotated. If you are new to AWS, this checklist might not be satisfactory, and it is suggested that you follow the security recommendations that AWS gives you. We now must access the **Users** option on the left-hand side of the view and create a new user by clicking the **Add users** button in the top-right corner of the screen, as seen in the following screenshot:

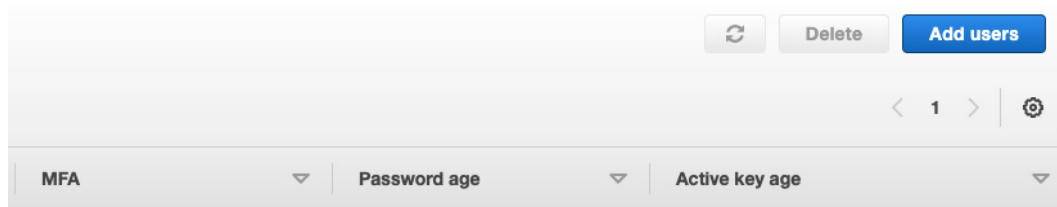


Figure 10.7 – Creating users

We can then create a user. It does not matter what name you call the user, but you must ensure that they have programmatic access by checking the **Access key - Programmatic access** option, as seen in the following screenshot:

More books: https://t.me/booksforits

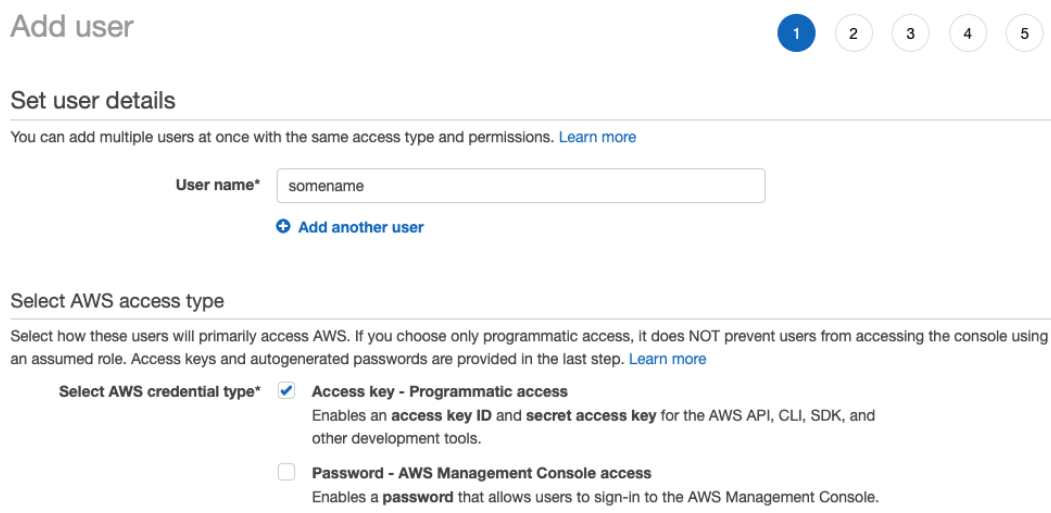


Figure 10.8 – First stage of creating a user

Once we have highlighted the programmatic access and defined the username, we can move on to the permissions, as seen in the following screenshot:

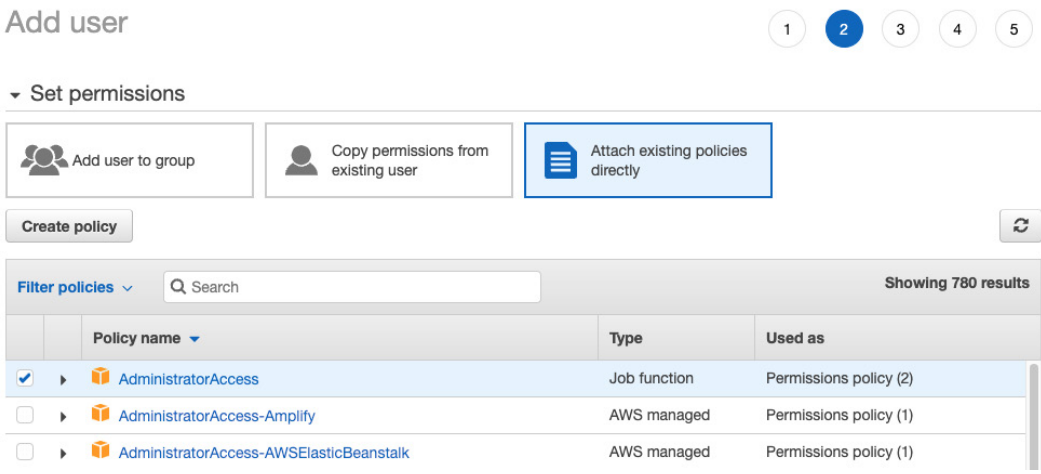


Figure 10.9 – Defining permissions

More books: https://t.me/booksforits

We can see that we have given the user `AdministratorAccess` permissions, which will enable us to create and destroy servers and databases. The rest of the steps in the user creation process are trivial, and you can get through these by just clicking **Next**. Once the user is created, you will be exposed to an access key and a secret access key. It is important to note these down in a secure location such as a password manager because you will never be able to see your secret access key again on the AWS site, and we will need them when configuring our AWS client on our local computer. Now that we have the user keys handy, we can configure our AWS client with the following command:

```
aws configure
```

The AWS client will then prompt you to enter the user keys as and when they are required. Once this is done, your AWS client is configured, and we can use AWS features programmatically on our local computer. Now, we are ready to start creating servers using Terraform in the next section.

Setting up our Terraform build

When it comes to building infrastructure on AWS, we can simply point and click in the EC2 dashboard. However, this is not desirable. If you are anything like me, when I point and click a series of configuration settings, I forget what I have done unless I document it, and let's be honest, documenting what you've clicked is not something you are going to look forward to. Even if you are a better person than me and you document it, when you change the configuration, there is a chance that you will not go back to update the documentation of that change. Pointing and clicking is also time-consuming. If we wanted to create some infrastructure and then destroy it a week later, and then recreate it a month after that, we would be reluctant to touch it if we had to point and click, and our server bills would be higher. This is where **infrastructure as code** (**IaC**) comes in. We will have to do some pointing and clicking, as we did in the previous sections. We cannot do any programmatic access without pointing and clicking to set up programmatic access.

Now that we have programmatic access, we can build out our `build` directory, which should be next to our `web_app` and `front_end` directories. In the `build` directory, we can define our infrastructure for the build server in the `build/main.tf` file. It must be noted that the `.tf` extension is the standard extension for Terraform files. First, we define which version of Terraform is being used with the following code:

```
terraform {
  required_version = ">= 1.1.3"
}
```

More books: https://t.me/booksforits

Now that we have defined the Terraform version, we can declare that we are using the AWS module. The Terraform registry has modules for a range of platforms, including Google Cloud and Microsoft Azure. Anyone can build modules and abstract infrastructure to be downloaded on the Terraform registry. Our AWS module usage declaration takes the following form:

```
provider "aws" {
    version = ">= 2.28.1"
    region = "eu-west-2"
}
```

You might want to pick a different region if another region suits you. A list of the available regions on AWS for EC2 can be found via the following link: `https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html`.

I'm merely using `"eu-west-2"` for demonstration purposes. We can now build our EC2 instance with the following code:

```
resource "aws_instance" "build_server" {
    ami = "ami-0fdbd8587b1cf431e"
    instance_type = "t2.medium"
    key_name = "remotebuild"
    user_data = file("server_build.sh")
    tags = {
      Name = "to-do build server"
    }
}
```

`resource` declares that we are defining a resource to be built, and `aws_instance` states that we are using the EC2 instance template in the AWS module. A list of the available AWS Terraform modules and their documentation can be found via the following link: `https://registry.terraform.io/providers/hashicorp/aws/latest/docs`.

`build_server` is what we are calling it. We can refer to `build_server` anywhere else in the Terraform script, and Terraform will work out the order in which resources need to be built to make sure all references are accounted for. We can see that we have referenced the `"remotebuild"` key that we defined in the previous section. We can create multiple EC2 instances that can be accessed by one key if we want. We also declare the name so that when we look at our EC2 instances, we know what the server is for. We must also note that `user_data` is the Bash script that will be run on the new EC2 server once it has been built. The `ami` section is a reference to the type of operating system and version being used. Do not directly copy my **Amazon Machine Image** (**AMI**) ID in the example

More books: https://t.me/booksforits

unless you are using the same region, as AMI IDs can vary depending on the region. If you want to find out the AMI ID, go to your EC2 dashboard and click on **Launch an instance**, which will result in the following window:

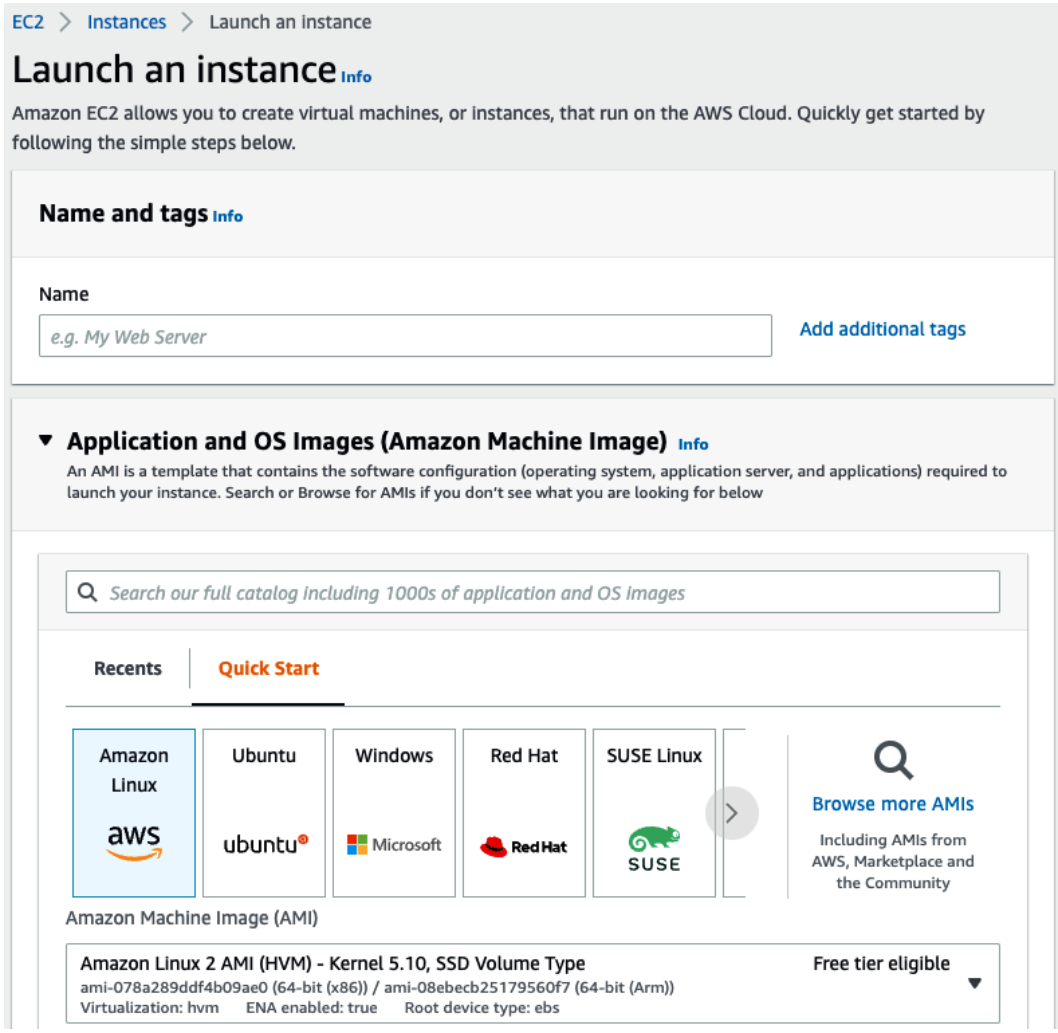


Figure 10.10 – Launching an instance

Here, we can see that we are picking **Amazon Linux**. You must select this; otherwise, your build scripts will not work. If we zoom in, we can see that the AMI ID is visible, as seen here:

More books: https://t.me/booksforits

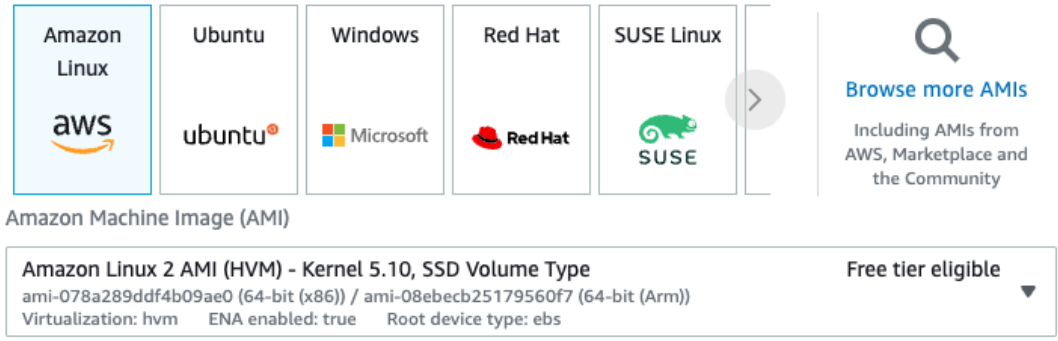


Figure 10.11 – Accessing AMI ID for the server

This will be the AMI ID for the Amazon Linux operating system in the region that you want to launch. You can also see nothing is stopping you from using other operating systems in other Terraform projects. If you have built EC2 instances in the past, you will know that the IP address is random unless we attach an elastic IP to the EC2 instance. We will have to produce an output of the IP of our EC2 instance so that we can connect to it. Our output is defined with the following code:

```
output "ec2_global_ips" {
  value = ["${aws_instance.build_server.*.public_ip}"]
}
```

Here, we can see that we reference our build server using `aws_instance.build_server`. Further reading on Terraform outputs is provided in the *Further reading* section. At this point, our Terraform build is nearly done. We must remember that we need to build the `server_build.sh` script that is going to be run on the EC2 instance once the EC2 instance has been built. In our `/build/server_build.sh` file, we can install the basic requirements needed for our server with the following code:

```
#!/bin/bash

sudo yum update -y
sudo yum install git -y
sudo yum install cmake -y
sudo yum install tree -y
sudo yum install vim -y
sudo yum install tmux -y
sudo yum install make automake gcc gcc-c++ kernel-devel -y
```

More books: https://t.me/booksforits

With the preceding packages, we will be able to navigate around the server looking at file trees with `tree`; we will also be able to perform `git` operations, open files, and edit them using `vim`, and have multiple panels open through one terminal if we need to with `tmux`. The other packages enable us to compile our Rust code. We must also note that we have appended each install with a `-y` tag. This is to tell the computer to bypass input prompts and put in default answers. This means that we can run this script in the background without any problems. We now must install the PostgreSQL drivers with the following code:

```
sudo amazon-linux-extras install postgresql10 vim epel -y
sudo yum install -y postgresql-server postgresql-devel -y
```

We nearly have everything that we need. In the next section, we will be using Docker to build and package our applications. This can be done with the following code:

```
sudo amazon-linux-extras install docker
sudo service docker start
sudo usermod -a -G docker ec2-user
```

Here, we can see that we install Docker, start the Docker service, and then register our user with the Docker service so that we do not have to use `sudo` with every Docker command. Seeing as we have installed Docker, we might as well install `docker-compose` for completeness. This can be done at the end of our script with the following code:

```
sudo curl -L "https://github.com/docker/compose/releases
/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)"
-o /usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose
```

The `curl` command downloads `docker-compose`. We then make the `docker-compose` executable with the `chmod` command. The build script is almost finished. However, running all the commands that we have defined in the build script will take a while. There is a chance that we can SSH into our server before the build script has finished. Therefore, we should write a `FINISHED` string to a file to inform other processes that the server has all the packages that have been installed. We can write our flag with the following code:

```
echo "FINISHED" > home/ec2-user/output.txt
```

We have now built out the infrastructure that is going to build our applications for deployment. In the next section, we are going to build scripts that orchestrate the building of our application using the build server that we have configured.

More books: https://t.me/booksforits

Writing our Python application build script

We could technically try to fit our application-building script in the same scripts that we wrote in the previous section. However, it is desired to keep our scripts separate. For instance, if we were going to use a **continuous integration** (**CI**) solution such as GitHub Actions, Travis CI, or CircleCI, we would not need the Terraform build process because we could build on their systems automatically. In this section, we will build a script that orchestrates the entire build process in Python because it is an easy scripting language that can handle JSON data that we get from the Terraform output. In our `build/run_build.py` script, we start by importing everything that we need with the following code:

```
from subprocess import Popen
from pathlib import Path
import json
import time

DIRECTORY_PATH = Path(__file__).resolve().parent
```

We now have the absolute path for the `build` directory, we can run Bash commands through the `Popen` class, and we can load JSON. Now that we have everything imported, we can run our Terraform commands with the following code:

```
init_process = Popen(f"cd {DIRECTORY_PATH} && terraform init",
                     shell=True)
init_process.wait()
apply_process = Popen(f"cd {DIRECTORY_PATH} && terraform
apply",
                      shell=True)
apply_process.wait()
```

In each of the commands, we go to the directory path and then perform the Terraform command. We then wait for the command to finish before moving on to the next command. While Python is a slow, easy, and not strongly typed language, here it adds a lot of power. We can run multiple Bash commands at the same time and then wait for them later if needed. We can also pull data out of processes and manipulate this data and feed it into another command easily. The two Terraform commands that we carry out in the preceding snippet are `init` and `apply`. `init` sets up the Terraform state to record what is going on and downloads the modules that we need, which in this case is AWS. The `apply` command runs the Terraform build, which will build our EC2 server.

Once our EC2 instance is built, we can get the output of Terraform and write it to a JSON file, and then load the data from that JSON file with the following code:

```
produce_output = Popen(f"cd {DIRECTORY_PATH} && terraform
```

More books: https://t.me/booksforits

```
                          output -json > {DIRECTORY_PATH}/
                          output.json", shell=True)
produce_output.wait()

with open(f"{DIRECTORY_PATH}/output.json", "r") as file:
    data = json.loads(file.read())

server_ip = data["ec2_global_ips"]["value"][0][0]
```

Now that we have the server IP, we can SSH into this server and get it to do our builds. However, there could be some concurrency issues. There is a small-time window where the Terraform build finishes, but the server is not ready yet to accept connections. Therefore, we just need the script to wait for a small period before continuing with the following code:

```
print("waiting for server to be built")
time.sleep(5)
print("attempting to enter server")
```

Once this is done, we need to pass our server IP into another Bash script that manages the build and then destroy the server afterward with the following code:

```
build_process = Popen(f"cd {DIRECTORY_PATH} &&
                      sh ./run_build.sh {server_ip}",
                      shell=True)
build_process.wait()
destroy_process = Popen(f"cd {DIRECTORY_PATH} &&
                        terraform destroy", shell=True)
destroy_process.wait()
```

We have now built the orchestration of a build and the Terraform script that defines the infrastructure for the build. We now must build the final build script that will run the build commands on the server.

Writing our Bash deployment script

Our Bash deployment script must take in the IP address of the build server, SSH into the server, and run a series of commands on the server. It will also have to copy our code onto the server to be built. We can see from the preceding code that we can build our build Bash script in the `/build/run_build.sh` file. First, we start with the standard boilerplate code:

```
#!/usr/bin/env bash
```

More books: https://t.me/booksforits

```
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
```

With this boilerplate code, we have stated that this is a Bash script and that the rest of the Bash code will run in the `build` directory. We then upload the code from the Rust application with the following code:

```
rm -rf ../web_app/target/

scp -i "~/.ssh/keys/remotebuild.pem" -r
../web_app ec2-user@$1:/home/ec2-user/web_app
```

Here, we can see that we remove the `target` directory. As we remember, the `target` directory is built when we build our Rust application; we do not need to upload build files from the local build. We then copy our Rust code using the `scp` command. We access the first argument passed into the script, which is `$1`. Remember that we pass in the IP address, so `$1` is the IP address. We then SSH into our server and run commands on this server with the following code:

```
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  until [ -f ./output.txt ]
  do
      sleep 2
  done
  echo "File found"
  curl https://sh.rustup.rs -sSf | bash -s -- -y
  source ~/.cargo/env
  cd web_app
  cargo build --release
EOF
```

Here, we see that we loop sleeping for 2 seconds on each iteration until the `output.txt` file is present. Once the `output.txt` file is present, we know that the build script from the Terraform is complete and we can start our build. We signal this by echoing `"File found"` to the console. We then install Rust, load our `cargo` commands into our shell with the `source` command, move into the `web_app` directory, and build our Rust application.

More books: https://t.me/booksforits

Note

We can get rid of the Python dependency if needed by using the `jq` command in our `run_build.sh` script with the following code insertion:

```
. . .
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

terraform init
terraform apply
terraform output -json > ./output.json

IP_ADDRESS=$(jq --raw-output '.ec2_global_ips.value[0][0]'
output.json)
echo $IP_ADDRESS

echo "waiting for server to be built"
sleep 5
echo "attempting to enter server"

rm -rf ../web_app/target/
scp -i "~/.ssh/keys/remotebuild.pem" -r
../web_app ec2-user@$IP_ADDRESS:/home/ec2-user/web_app
. . .
```

It must be noted that the references to the `$1` variable are replaced with `$IP_ADDRESS`.

To run our pipeline without the Python dependency, we merely need to run the following command:

```
Sh run_build.sh
```

However, we will be relying on our Python script later on in the chapter.

Now that our build pipeline is built, we can run it with the following command:

```
python3 run_build.py
```

More books: https://t.me/booksforits

> **WARNING**
>
> Terraform can sometimes be temperamental; if it fails, you may need to run it again. Sometimes, I've had to perform a Terraform run up to three times before it fully works. Every action is stored by Terraform, so do not worry—running the Python script again will not result in duplicate servers.

When running this command, you will be prompted to write `yes` at three different points of the process. The first time is to approve the building of the server with Terraform, the second time is to add the IP address of the server to the list of known hosts to approve the SSH connection, and the last time is to approve the destruction of the server. It usually makes sense to show the printouts in this book; however, the printout here is long and would probably take up multiple pages. Also, the printouts are obvious. Terraform openly states what is being built, the copying and building are also verbose, and the destruction of the server with Terraform is also printed out. It will be very clear what is going on when you run this build pipeline. You might also notice that there is a range of Terraform files that have been created. These files keep track of the state of our resources that have been built on the AWS platform. If you delete these state files, you have no way of knowing what is built, and duplicates will be spun up. It will also prevent Terraform from cleaning up. At the place where I work, at the time of writing this, we use Terraform to build massive data models for calculating the risk of financial loss over geographical locations. The data being processed can go over terabytes per chunk. We use Terraform to spin up a range of powerful computers, run data through it (which can take days), and then shut it down when it is finished. Multiple people need to monitor this process, so our Terraform state is housed in a **Simple Storage Service** (**S3**) bucket so that anyone on the project can shut it down if needed. An example of this configuration takes the following form in the `state.tf` file:

```
terraform {
    backend "s3" {
    bucket = "some_bucket"
    key    = "some/ptaht/terraform.tfstate"
    region = "eu-west-1"
  }
}
```

It must be noted that your account needs to have access to the bucket defined.

Now, it is time to build our frontend application. Looking and what we have just done, all we need to do is add to the `/build/run_build.sh` file the steps to upload our frontend code to the build server and build the frontend application. At this point, you should be able to code this yourself. Right now, it would be a good use of your time to stop reading and attempt to build it. If you have attempted it yourself, it should look like the code shown here:

```
rm -rf ../front_end/node_modules/
```

More books: https://t.me/booksforits

```
scp -i "~/.ssh/keys/remotebuild.pem" -r
../front_end ec2-user@$1:/home/ec2-user/front_end

ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  curl -o- https://raw.githubusercontent.com/nvm-sh
  /nvm/v0.37.2/install.sh | bash
  . ~/.nvm/nvm.sh
  nvm install --lts
  cd front_end
  npm install
EOF
```

Here, we remove the node modules, copy the code to the server, install node, and then run the `install` command for our application. Now that our build pipeline is fully working, we can move on to wrapping our software in Docker so that we can package the software in Docker and deploy it.

Managing our software with Docker

So far, we have been using Docker to manage our PostgreSQL and Redis databases. When it comes to running our frontend and Rust server, we have merely been running it directly on our local computer. However, when it comes to running our applications on remote servers, it is simpler and easier to distribute. Before we get on to deploying our Docker images on servers, we need to build and run them locally, which starts with writing our Docker image file.

Writing Docker image files

Before we proceed, it must be noted that the approach carried out here is the simplest, least optimized way to build a Rust server Docker image, because we are juggling a lot of new concepts. We cover an optimized way of building Rust server Docker images in *Chapter 13*, *Best Practices for a Clean Web App Repository*. When it comes to building a Docker image, we need a Dockerfile. This is where we define the steps needed to build our image. In our `web_app/Dockerfile` file, we basically borrow a base image and then run our commands on top of this image for our application to work. We can define the base image and the requirements to run our database interactions with the following code:

```
FROM rust:1.61

RUN apt-get update -yqq && apt-get install -yqq cmake g++
RUN cargo install diesel_cli --no-default-features --features
postgres
```

More books: https://t.me/booksforits

Here in our Docker build, we are starting with the official `rust` image. We then update `apt` so that we can download all the available packages. We then install `g++` and the `diesel` client so that our database operations will work. We then copy the code and config files from our Rust application and define our work directory with the following code:

```
COPY . .
WORKDIR .
```

Now, we have everything to build our Rust application with the following code:

```
RUN cargo clean
RUN cargo build --release
```

Now our build is done, we move the static binary from the `target` directory into our home directory, remove excessive code such as the `target` and `src` directories, and allow the static binary file to be executable with the following code:

```
RUN cp ./target/release/web_app ./web_app
RUN rm -rf ./target
RUN rm -rf ./src
RUN chmod +x ./web_app
```

Now everything is done, we can expose the port at which the web server will be running to make the server exposed by the container and execute the command that gets run when the Docker image is spun up with the following code:

```
EXPOSE 8000

CMD ["./web_app", "config.yml"]
```

Now that our Docker image file is written, we can move on to building Docker images.

Building Docker images

Before we can run this, however, we need to remove our build script. There is only one thing left our Docker image build needs. When copying over the code into the Docker image, we know that the `target` directory has a lot of code and files that we do not need in our image. We can avoid copying over the `target` directory by having the following code in the `.dockerignore` file:

```
target
```

More books: https://t.me/booksforits

If we try to compile our application with the build script, Docker will just throw itself into an infinite file loop and then time out. This means that our `ALLOWED_VERSION` variable in our `main.rs` file takes the following form:

```
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    const ALLOWED_VERSION: &'static str = "v1";
    . . .
```

And we then must comment out our `build` dependency in our `Cargo.toml` file with the following code and remove the `build.rs` file entirely:

```
[package]
name = "web_app"
version = "0.1.0"
edition = "2021"
# build = "build.rs"
```

We are now ready to build our image; we navigate to where the Dockerfile is and run the following command:

```
docker build . -t rust_app
```

This command executes the build defined by the Dockerfile in the current directory. The image is tagged `rust_app`. We can list our images with the following command:

```
docker image ls
```

This will give us the following printout:

```
REPOSITORY   TAG       IMAGE ID       CREATED         SIZE
rust_app     latest    c2e465545e30   2 hours ago     2.69GB
. . .
```

We can then test to see if our application is properly built; we just run the following command:

```
docker run rust_app
```

This directly runs our Rust app. Our application should crash instantly with the following error:

```
thread 'main' panicked at 'called `Result::unwrap()`
on an `Err` value: Connection refused (os error 111)',
```

More books: https://t.me/booksforits

```
src/counter.rs:24:47 note: run with `RUST_BACKTRACE=1`
environment variable to display a backtrace
```

We can see that the error is not down to our build, but a connection issue with Redis from our counter file. This is reassuring, and it will work when we run our Rust application with our two databases.

There is an approach in Docker where you can do multi-layered builds. This is where we start off with the `rust` base image, build our application, and then move our build into another Docker image with no dependencies. The result is that our server image is usually merely 100 MB as opposed to multiple GB. However, our application has a lot of dependencies, and this multi-layered build approach will result in multiple driver errors. We explore building tiny images in *Chapter 13*, *Best Practices for a Clean Web App Repository*.

Building an EC2 build server using Terraform

We have now built our Rust Docker image locally. We can now build it on our build server. Before we do this, we are going to have to increase the size of the hard drive on our builder server; otherwise, the image will refuse to build due to lack of space. This can be done in our `/build/main.tf` file by adding a root block device, as seen in the following code snippet:

```
resource "aws_instance" "build_server" {
    ami = "ami-0fdbd8587b1cf431e"
    instance_type = "t2.medium"
    key_name = "remotebuild"
    user_data = file("server_build.sh")
    tags = {
      Name = "to-do build server"
    }
    # root disk
    root_block_device {
      volume_size           = "150"
      volume_type           = "gp2"
      delete_on_termination = true
    }
}
```

`gp2` is the version of SSD that AWS supports we are using. `150` is the number of GB that we are connecting to the server. This will be enough memory to build our Docker images, leaving us only to build the pipeline that constructs our Docker images.

More books: https://t.me/booksforits

Orchestrating builds with Bash

At this point, we are also going to optimize our build Bash script in our `/build/run_build.sh` file. First, we do not remove the `target` directory; instead, we are selective with what we upload onto our server with the following code:

```
#!/usr/bin/env bash

SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 "mkdir web_
app"

scp -i "~/.ssh/keys/remotebuild.pem" -r
../web_app/src ec2-user@$1:/home/ec2-user/web_app/src
scp -i "~/.ssh/keys/remotebuild.pem"
../web_app/Cargo.toml ec2-user@$1:/home/ec2-user/web_app/Cargo.
toml
scp -i "~/.ssh/keys/remotebuild.pem"
../web_app/config.yml ec2-user@$1:/home/ec2-user/web_app/
config.yml
scp -i "~/.ssh/keys/remotebuild.pem"
../web_app/Dockerfile ec2-user@$1:/home/ec2-user/web_app/
Dockerfile
```

Here, we can see that we make the `web_app` directory, and then upload the files and directories that we need to build our Rust Docker image. We then need to connect to the server to install Rust with the following code:

```
echo "installing Rust"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  curl https://sh.rustup.rs -sSf | bash -s -- -y
  until [ -f ./output.txt ]
  do
      sleep 2
  done
  echo "File found"
EOF
echo "Rust has been installed"
```

More books: https://t.me/booksforits

Here, we can see that we install Rust before we block the script until the server is ready with everything installed. This means that we are running the installation of Rust at the same time the rest of the server is being built, saving time. We then exit the connection to our build server. Finally, we connect to the server again and build our Docker image with the following code:

```
echo "building Rust Docker image"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  cd web_app
  docker build . -t rust_app
EOF
echo "Docker image built"
```

We have then built the pipeline that builds our Rust Docker image on our build server. This seems like a lot of steps, and you would be right. We can build our image locally with a different target operating system and chip architecture. Exploring it here would disjoint the flow of what we are trying to achieve, but further information on compiling with different targets will be provided in the *Further reading* section.

Writing a Docker image file for the React frontend

For now, we are going to package our frontend application in Docker. In our `front_end/Dockerfile` file, we inherit the node base image, copy the code, and define the working directory with the following code:

```
FROM node:17.0.0

WORKDIR .
COPY . ./
```

We then install a `serve` package to serve the web app build files, the modules needed to build the application, and build the React application with the following code:

```
RUN npm install -g serve
RUN npm install
RUN npm run react-build
```

We then expose the port and server to our application with the following code:

```
EXPOSE 4000
CMD ["serve", "-s", "build", "-l", "4000"]
```

More books: https://t.me/booksforits

We can then build our Docker image with the following command:

```
docker build . -t front_end
```

We can then run the recently built image with the following command:

```
docker run -p 80:4000 front_end
```

This routes the container's external port (`80`) to the locally exposed port `4000`. When our image is running, we get the following printout:

```
INFO: Accepting connections at http://localhost:4000
```

This shows that our image is running in a container. We will be able to access our frontend container by merely accessing our localhost, which is port `80`, as seen in the following screenshot:

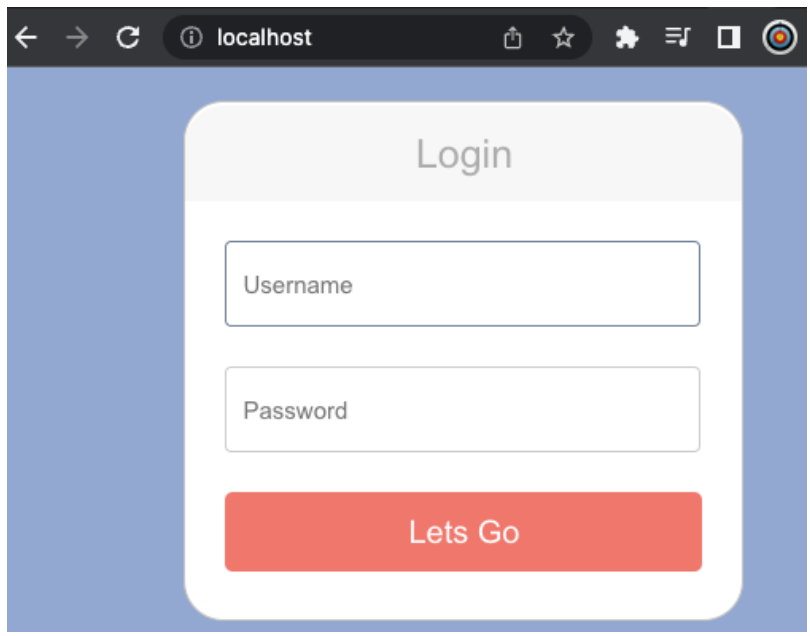


Figure 10.12 – Accessing our frontend container using localhost

We will not be able to do anything with it, however, because our Rust server is not running. We can now lift the steps that we have carried out to build our frontend into our `/build/run_build.sh` script to have our build pipeline construct our frontend image as well. This is a good opportunity for you to try to add the step yourself. We will have to install node and then carry out the frontend build steps on the build server.

More books: https://t.me/booksforits

If you have had an attempt at incorporating our React build in our pipeline, it should look like the following code:

```
echo "copying React app"
rm -rf ../front_end/node_modules/
scp -i "~/.ssh/keys/remotebuild.pem" -r
../front_end ec2-user@$1:/home/ec2-user/front_end
echo "React app copied"

echo "installing node on build server"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  curl -o- https://raw.githubusercontent.com/nvm-sh/nvm
  /v0.37.2/install.sh | bash
  . ~/.nvm/nvm.sh
  nvm install --lts
EOF
echo "node installed"

echo "building front-end on server"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  cd front_end
  docker build . -t front_end
EOF
echo "front-end Docker image has been built"
```

We can be more optimal in our implementation; however, the preceding code is the simplest application. First, we copy over the code we need to the build server. We then connect to our build server to install node. After installing node, we connect to the server again to move into the React application directory and build our Docker image.

Our build pipeline is now working. Just think of what we have achieved here—we have built a pipeline that constructs a build server; we then copied our code onto the build server, constructed Docker images, and then destroyed the server after the build was done. Even though this pipeline is not perfect, we have explored some powerful tools that will enable you to automate tasks and lift a lot of the code that we have covered in this subsection in other CI pipeline tools. However, right now, we are just building the Docker images and then destroying them with the server. In the next section, we will deploy our images on **Docker Hub**.

More books: https://t.me/booksforits

Deploying images onto Docker Hub

Before we can pull our images from Docker Hub, we will have to push our images to Docker Hub. Before we can push our images to Docker Hub, we will have to create a Docker Hub repo. Registering our image on Docker Hub is straightforward. After logging in, we click on the **Create Repository** button in the top-right corner, as shown in the following screenshot:

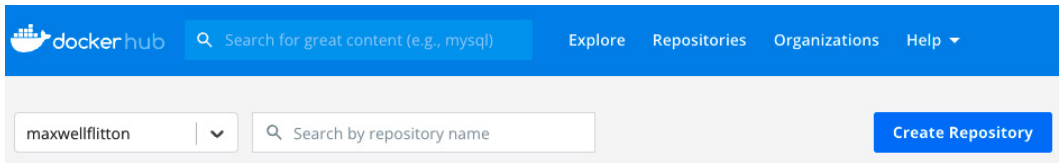


Figure 10.13 – Creating a new repository on Docker Hub

Once we have clicked on this, we define the repository with the following configuration:

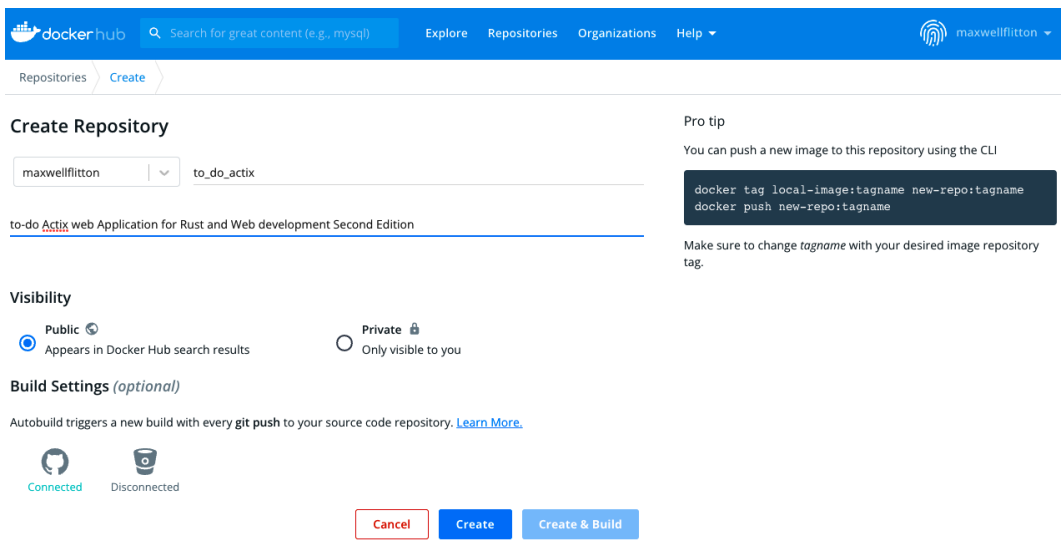


Figure 10.14 – Defining a new Docker repository

We can see that there is an option for connecting our repository with GitHub by clicking on the *GitHub* button seen in the preceding screenshot. The **Connected** GitHub status in the preceding screenshot simply means that my GitHub is connected to my Docker Hub account. This means that every time a successful pull request gets completed, the image is rebuilt with the code, and then it is sent to the repository. This can be helpful if you are building a fully automated pipeline. However, for this book, we will not connect our GitHub repository. We will push it onto our build server. You will also need to create a Docker Hub repository for our frontend if you are deploying the React application.

More books: https://t.me/booksforits

Now that we have defined our Docker Hub repositories, we need to add a Docker login in our build pipeline. This means that we must pass our Docker Hub password and username into our Python build script. Our Python script can then pass the Docker Hub credentials into the build Bash script. This build Bash script will then log in to Docker on the build server so that we can push our images to our Docker Hub. In our `/build/run_build.py` file, we define the arguments passed into the Python script with the following code:

```
. . .
import argparse
. . .
parser = argparse.ArgumentParser(
                    description='Run the build'
                  )
parser.add_argument('--u', action='store',
                    help='docker username',
                    type=str, required=True)
parser.add_argument('--p', action='store',
                    help='docker password',
                    type=str, required=True)
args = parser.parse_args()
```

We can see that we have set `required` to `True`, which means that the Python script will not run unless both parameters are supplied. If we supply a `-h` parameter in the Python script, the parameters that we have defined in the preceding code will be printed out with help information. Now that we have ingested the Docker credentials, we can then pass them into our build Bash script in the `/build/run_build.py` file with the following adaptation to our code:

```
. . .
print("attempting to enter server")

build_process = Popen(f"cd {DIRECTORY_PATH} && sh ./run_build.
sh
    {server_ip} {args.u} {args.p}", shell=True)
build_process.wait()

destroy_process = Popen(f"cd {DIRECTORY_PATH} && terraform
destroy",
    shell=True)
. . .
```

More books: https://t.me/booksforits

Here, we can see that the Docker username is accessed using the `args.u` attribute and the Docker password is accessed using the `args.p` attribute. Now that we are passing Docker credentials into our build Bash script, we need to use these credentials to push our images. In our `/build/run_build.sh` file, we should log in after Docker is installed on our build server with the following code:

```
. . .
echo "Rust has been installed"

echo "logging in to Docker"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  echo $3 | docker login --username $2 --password-stdin
EOF
echo "logged in to Docker"

echo "building Rust Docker image"
. . .
```

In the preceding code, we use `–password-stdin` to pipe our password into the Docker login. `stdin` ensures that the password is not stored in the logs, making it a little bit more secure. We can then build, tag, and then push our Rust application with the update in the following code:

```
echo "building Rust Docker image"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  cd web_app
  docker build . -t rust_app
  docker tag rust_app:latest maxwellflitton/to_do_actix:latest
  docker push maxwellflitton/to_do_actix:latest
EOF
echo "Docker image built"
```

Here, we build the Rust image as we always do. We then tag the Rust app image as the latest release and then push it to the Docker repository. We also must push our frontend application to Docker Hub. At this point, this is a good chance for you to write the code that pushes the frontend image. If you did attempt to push the frontend image, it should look like the following code snippet:

```
echo "building front-end on server"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  cd front_end
  docker build . -t front_end
```

More books: https://t.me/booksforits

```
  docker tag front_end:latest maxwellflitton/to_do_react:latest
  docker push maxwellflitton/to_do_react:latest
EOF
echo "front-end Docker image has been built"
```

We have now coded all that we need to build both of our images and push them to our Docker repositories. We can run our Python build script with the following command:

```
python3 run_build.py --u some_username --p some_password
```

Again, the printout is lengthy, but we can check our Docker Hub repository to see if our image was pushed. As we can see in the following screenshot, the Docker Hub repository states when the image has been pushed:

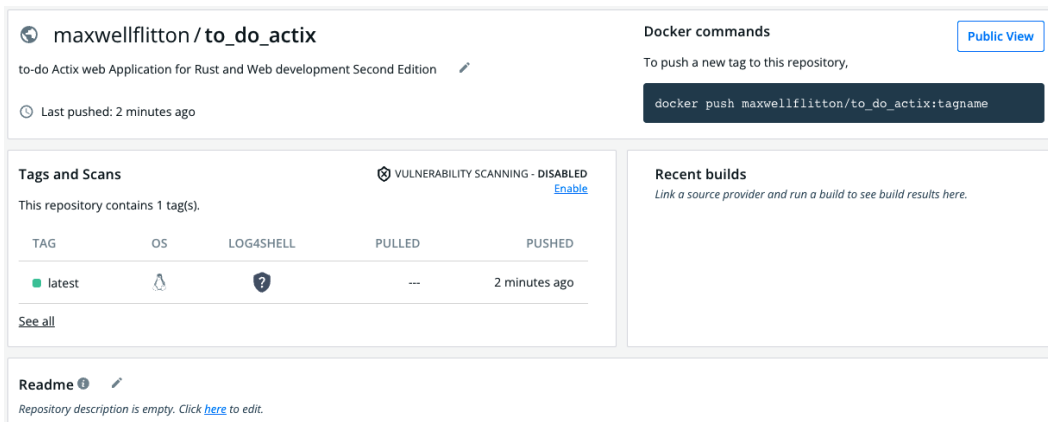


Figure 10.15 – View of pushed Docker repository

We now have our images pushed onto our Docker Hub repositories! This means that we can pull them onto any computer that we need, just as we did when pulling the PostgreSQL image in the `docker-compose` file. We should now pull our images onto a server so that other people can access and use our application. In the next section, we will deploy our application for external use.

Deploying our application on AWS

Even though we have packaged our Rust application in Docker, we have not run our Rust application in a Docker container. Before we run our Rust application on a server on AWS, we should run our Rust application locally. This will help us understand how a simple deployment works without having to build servers.

More books: https://t.me/booksforits

Running our application locally

When it comes to running our application locally, we will be using `docker-compose` with the following layout:

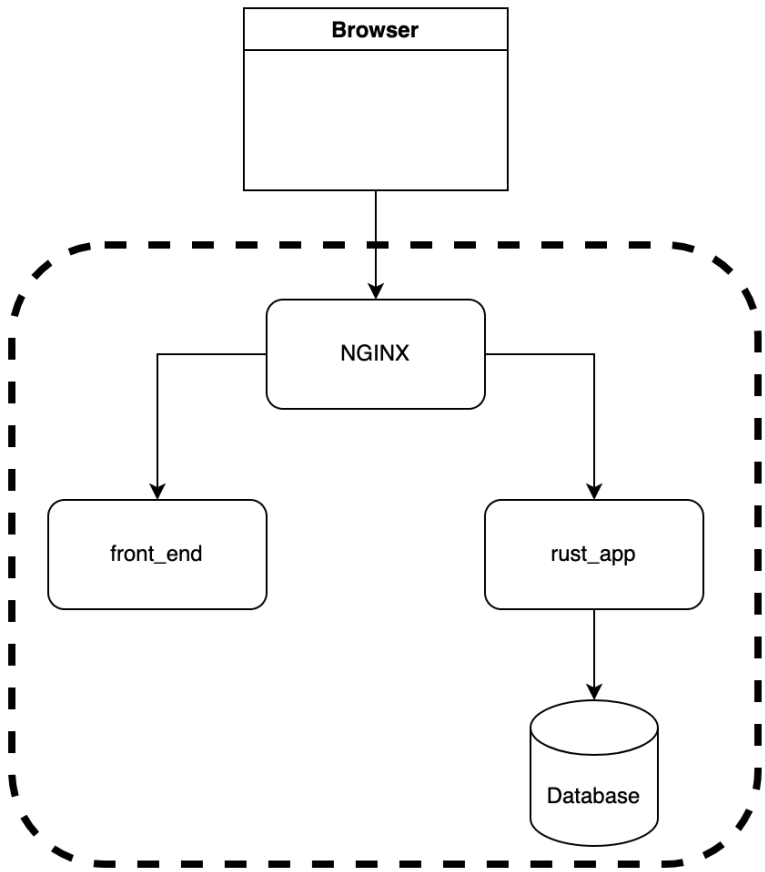


Figure 10.16 – Structure for local deployment

Here, we can see that the NGINX container takes in traffic from outside of the `docker-compose` network and directs the traffic to the appropriate container. Now that we understand our structure, we can define our `docker-compose` file. First, we need to make a directory called `deployment` next to our `build`, `front_end`, and `web_app` directories. Our general layout for our `docker-compose.yml` file in our `deployment` directory takes the following form:

```
services:
  nginx:
    . . .
```

More books: https://t.me/booksforits

```
  postgres_production:
    . . .
  redis_production:
      . . .
  rust_app:
    . . .
  front_end:
    . . .
```

We can start with our NGINX service. Now that we know the outside port is `80`, it makes sense that our NGINX container listens to the outside port of `80` with the following code:

```
  nginx:
    container_name: 'nginx-rust'
    image: "nginx:latest"
    ports:
      - "80:80"
    links:
      - rust_app
      - front_end
    volumes:
      - ./nginx_config.conf:/etc/nginx/nginx.conf
```

We can see that we get the latest NGINX image. We also have links to the `front_end` and `rust_app` containers because we will be passing HTTP requests to these containers. It also must be noted that we have a volume. This is where we share a volume outside of the container with a directory inside the container. So, this volume definition means that our `deploy/nginx_config.conf` file can be accessed in the NGINX container in the `etc/nginx/nginx.conf` directory. With this volume, we can configure NGINX routing rules in our `deploy/nginx_config.conf` file.

First, we set the number of worker processes to `auto`. We can manually define the number of worker processes if needed. `auto` detects the number of CPU cores available and sets the number to that with the following code:

```
worker_processes  auto;
error_log  /var/log/nginx/error.log warn;
```

More books: https://t.me/booksforits

The `error_log` directive defines the logging to a particular file. We not only define the file, but we also state the minimum level needed to write to the log file, which is `warning`. By default, the logging level needed to write to the file is `error`. We can now move on to defining contexts in our `deploy/nginx_config.conf` file. In the `events` context, we define the maximum number of connections that a worker can entertain at a time. This is achieved with the following code:

```
events {
    worker_connections  512;
}
```

The number of workers that we have defined is the default number that NGINX sets. Now that this is done, we can move on to our `http` context. Here, we define the `server` context. Inside this, we instruct the server to listen to port `80`, which is the port that listens to outside traffic, with the following code:

```
http {
    server {
        listen 80;
        location /v1 {
            proxy_pass http://rust_app:8000/v1;
        }
        location / {
            proxy_pass http://front_end:4000/;
        }
    }
}
```

Here, we can see that if the URL has `/v1/` at the start of the endpoint, we then pass it through the Rust server. It must be noted that we pass `v1` forward to the Rust server. If we did not pass `v1` forward to the Rust server, `v1` will be missing from the URL when it hits the Rust server. If the URL does not contain `v1` in the URL, then we forward it to our `front_end` container. Our NGINX container is now ready to manage traffic in our `docker-compose` system. Before we move on to our frontend and backend services, we need to define the Redis and PostgreSQL databases. There is nothing new here, so at this point, you can try to define them yourself. If you have, then your code should look like this:

```
  postgres_production:
    container_name: 'to-do-postgres-production'
    image: 'postgres:latest'
    restart: always
    ports:
```

More books: https://t.me/booksforits

```
      - '5433:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'
      - 'POSTGRES_PASSWORD=password'
    expose:
      - 5433
  redis_production:
      container_name: 'to-do-redis'
      image: 'redis:5.0.5'
      ports:
        - '6379:6379'
      volumes:
       - ./data/redis:/tmp
```

The preceding database definitions are the same as what we have used before in local development. With these databases, we can define our Rust application with the following code:

```
  rust_app:
    container_name: rust_app
    build: "../web_app"
    restart: always
    ports:
      - "8000:8000"
    links:
      - postgres_production
      - redis_production
    expose:
      - 8000
    volumes:
      - ./rust_config.yml:/config.yml
```

Here, we can see that we are not defining an image. Instead of declaring the image, we point to a build. The `build` tag is where we point to a **Dockerfile** that is going to be built for the Rust application server. We do this because our laptop might have a different chip architecture from the build server. If we directly pull our image from Docker Hub, it might not run. Also, we are developing our application and running it locally. If we make changes in the Rust application, it will be directly run in our local system without us having to push and pull from Docker Hub every time we make a change to the Rust

More books: https://t.me/booksforits

code and want to run it in a local system. We also define the links. This is because calling localhost inside a container will be looking for services inside the container. Instead, we must reference the link when calling the service. We can also see that we have our own config file in the volumes that we update. With the defined links, our `deploy/rust_config.yml` file takes the following form:

```
DB_URL: postgres://username:password@postgres_production/to_do
SECRET_KEY: secret
EXPIRE_MINUTES: 120
REDIS_URL: redis://redis_production/
```

Here, we can see that we reference the name of the service defined in the `docker-compose` system instead of the URL. We also must change the address for our Rust application in our `web_app/src/main.rs` file to zeros, as seen here:

```
})
.bind("0.0.0.0:8000")?
.run()
```

We then must remove our config file in our Docker build in our `web_app/Dockerfile` file with the following line of code:

```
RUN rm config.yml
```

If we do not do this, then our Rust application will not connect with the NGINX container. Now everything is defined for our Rust server, we can move on to defining the frontend application in our `docker-compose.yml` file with the following code:

```
  front_end:
    container_name: front_end
    image: "maxwellflitton/to_do_react:latest"
    restart: always
    ports:
      - "4000:4000"
    expose:
      - 4000
```

Here, we see that we reference our image in Docker Hub and expose the ports. Now that our local system is defined, we can run our system and interact with it by running the following command:

```
docker-compose up
```

More books: https://t.me/booksforits

Everything will build and run automatically. Before we can interact with our system, we need to run our `diesel` migrations in our Rust application build with the following command:

```
diesel migration run
```

We then need to create a user with the following `curl` command:

```
curl --location --request POST 'http://localhost/v1/user/
create' \
--header 'Content-Type: application/json' \
--data-raw '{
    "name": "maxwell",
    "email": "maxwellflitton@gmail.com",
    "password": "test"
}'
```

We now have everything in place to interact with our application. We can see that localhost with no reference to ports works with the following screenshot:

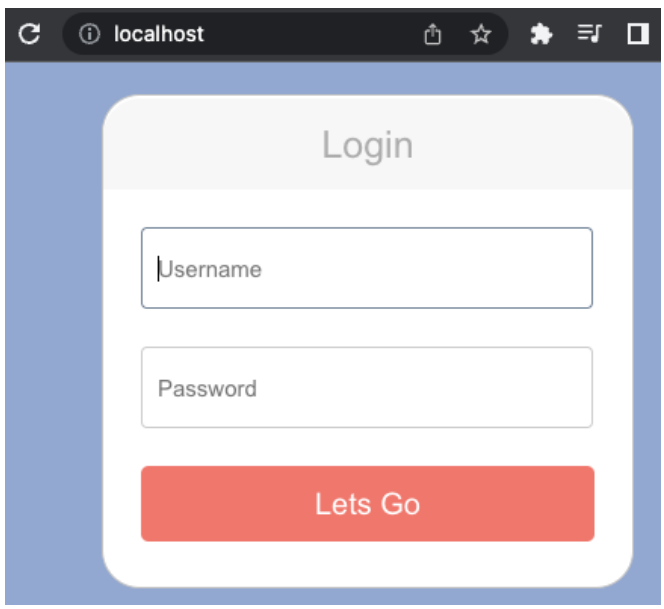


Figure 10.17 – Accessing our docker-compose system through the browser

If your NGINX is working, you should be able to log in and interact with the to-do application as before. We are now able to deploy our system on AWS so that other people can access and use our to-do application in the next section.

More books: https://t.me/booksforits

Running our application on AWS

We can deploy our application on AWS by carrying out the following steps:

1. Building a server
2. Running our `docker-compose` system on that server
3. Running database migrations on that server
4. Creating a user

Once we have carried out those steps, we will be able to access the application on the remote server. However, before we do this, we are going to have to alter the React application. Right now, our React application makes API calls to the localhost via `127.0.0.1`. When we are using a remote server, this will not work as we will have to make calls to the server to which we have deployed our application. To do this, we can extract where our API calls are made in the React application and update the root of the URL for the API call with the following code:

```
axios.get(window.location.href + "/v1/item/get",
```

What is happening here is that `window.location.href` returns the current location, which will be the IP of the server our application is deployed on, or localhost if we are developing it locally on our computer. The following files have API calls that need to be updated:

- `src/components/LoginForm.js`
- `src/components/CreateToDoitem.js`
- `src/components/ToDoitem.js`
- `src/App.js`

Once we have updated these files, we will be able to run another build in the `build` directory by running the following command:

```
python3 run_build.py --u some_username --p some_password
```

Once our build has been done, both our images will be updated. We can now move to our `deployment` directory and flesh it out with the following files:

- `main.tf`: This should be the same as the `main.tf` file in the `build` directory, except for the server having a different tag
- `run_build.py`: This should be the same as the `run_build.py` file in the `build` directory, except for the *destroy server* process at the end of the `run_build.py` script
- `server_build.sh`: This should be the same as the `server_build.sh` script in the `build` directory as we want our server to have the same environment as when our images were built

More books: https://t.me/booksforits

- `deployment-compose.yml`: This should be the same as the `docker-compose.yml` file in the `deployment` directory, except that `rust_app` service has an image tag instead of a build tag and the image tag should have the image of `maxwellflitton/to_do_actix:latest`
- `.env`: This should be the same as the `.env` file in the `web_app` directory, and we will need it to perform database migrations

We are now ready to code our `run_build.sh` file that will enable us to deploy our application, run migrations, and create a user. First, we start off with some standard boilerplate code to ensure that we are in the right directory, as follows:

```
#!/usr/bin/env bash

SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
```

We then copy the files needed to spin up our `docker-compose` system and perform database migrations with the following code:

```
scp -i "~/.ssh/keys/remotebuild.pem"
./deployment-compose.yml ec2-user@$1:/home/ec2-user/docker-
compose.yml
scp -i "~/.ssh/keys/remotebuild.pem"
./rust_config.yml ec2-user@$1:/home/ec2-user/rust_config.yml
scp -i "~/.ssh/keys/remotebuild.pem"
./.env ec2-user@$1:/home/ec2-user/.env
scp -i "~/.ssh/keys/remotebuild.pem"
./nginx_config.conf ec2-user@$1:/home/ec2-user/nginx_config.
conf
scp -i "~/.ssh/keys/remotebuild.pem" -r
../web_app/migrations ec2-user@$1:/home/ec2-user/migrations
```

None of this should be a surprise as we needed all the preceding files to run our `docker-compose` system. We then install Rust and wait for the server build to be done with the following code:

```
echo "installing Rust"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  curl https://sh.rustup.rs -sSf | bash -s -- -y
  until [ -f ./output.txt ]
  do
```

More books: https://t.me/booksforits

```
      sleep 2
  done
  echo "File found"
EOF
echo "Rust has been installed"
```

Again, this is nothing new that we have not seen. We then install the `diesel` client with the following code:

```
echo "installing diesel"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  cargo install diesel_cli --no-default-features --features
postgres
EOF
echo "diesel has been installed"
```

We then log in to Docker, spin up our `docker-compose` system, run our migrations, and then make a user with the following code:

```
echo "building system"
ssh -i "~/.ssh/keys/remotebuild.pem" -t ec2-user@$1 << EOF
  echo $3 | docker login --username $2 --password-stdin
  docker-compose up -d
  sleep 2
  diesel migration run
  curl --location --request POST 'http://localhost/v1/user/
create' \
  --header 'Content-Type: application/json' \
  --data-raw '{
      "name": "maxwell",
      "email": "maxwellflitton@gmail.com",
      "password": "test"
  }'
EOF
echo "system has been built"
```

More books: https://t.me/booksforits

With this, our system is deployed! We will be able to access our application on our server by putting the IP of the server that is in the `output.json` file into the browser. We will be able to log in and use our to-do application just like when we were running our system on our local computer, as seen on in the following screenshot:

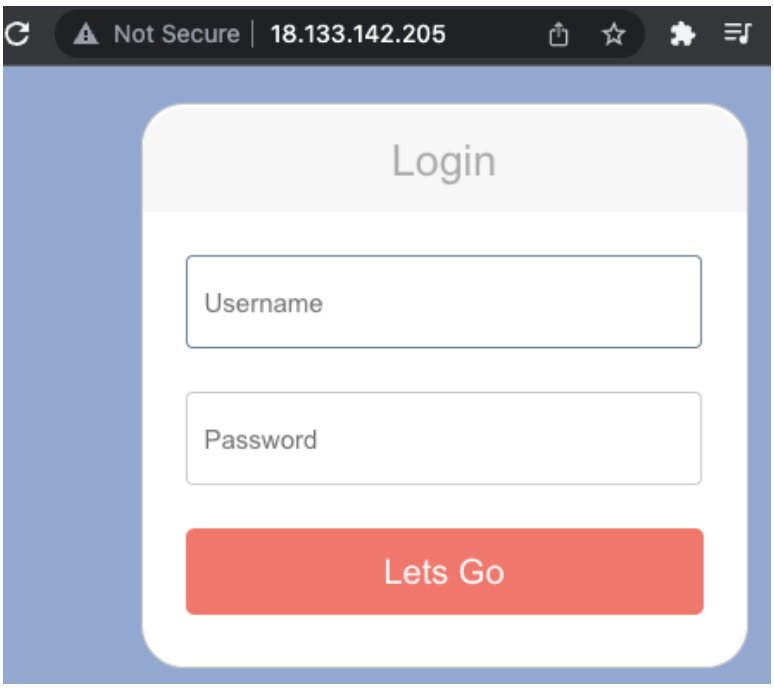


Figure 10.18 – Our application on our deployment server

As we can see, the connection is not secure and our browser is giving us a warning because we are not implementing the HTTPS protocol. This is because our connection is not encrypted. We will cover how to encrypt our connection in the next chapter.

Writing our application build script

Right now, our application is running a database locally on the EC2 instance. This has a few problems. Firstly, it means that the EC2 is stateful. If we tear down the instance, we will lose all our data.

More books: https://t.me/booksforits

Secondly, if we wipe the containers on the instance, we could also lose all our data. Data vulnerability is not the only issue here. Let's say that our traffic drastically increases, and we need more computing instances to manage it. This can be done by using NGINX as a load balancer between two instances, as shown in the following diagram:

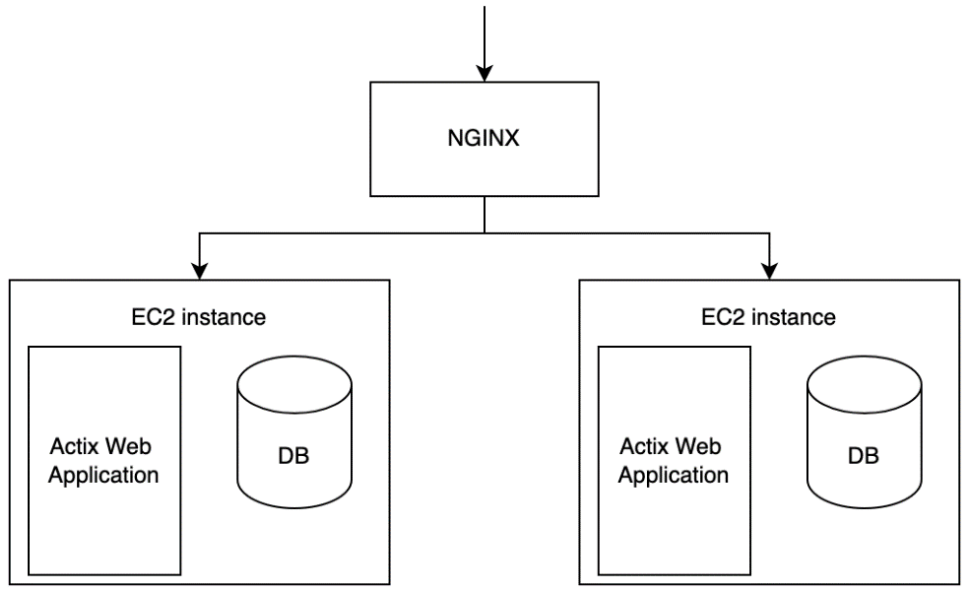


Figure 10.19 – Doubling our EC2 instances for our system

As you can see, the problem here is accessing random data. If user one creates an item, and this request hits the instance on the left, it is stored in the database on the left. However, user one can then make a `GET` request, which hits the instance on the right side. The second request will not be able to access the item that was created in the first request. The user would be accessing random states depending on which instance the request hit.

This can be solved by deleting the database from our `docker-compose` file and creating a database outside it, as shown in this diagram:

More books: https://t.me/booksforits

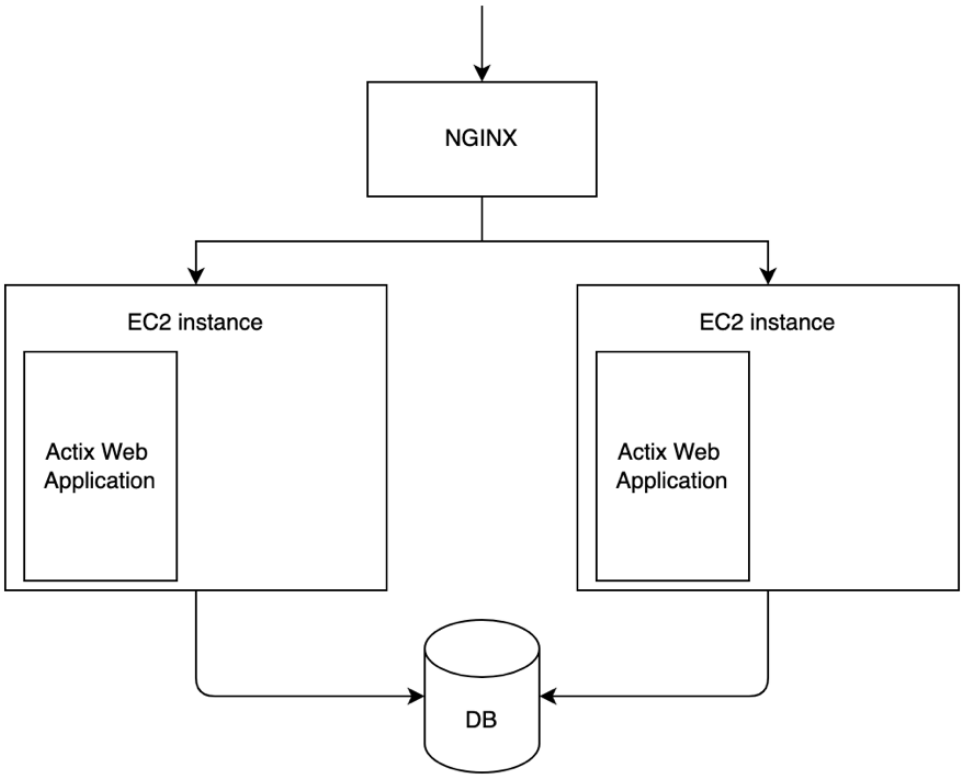


Figure 10.20 – Our new improved system

Now, we have a single point of truth for our data, and our EC2 instances are stateless, meaning we have the freedom to create and delete instances as and when we need to.

When it comes to adding an AWS database to our deployment, we are going to have to build our database in Terraform, and then pass the information about the constructed database to our `deployment/.env` file for database migrations and our `deployment/rust_config.yml` file for our Rust server to access. First, we must add the database definition to our `deployment/main.tf` file with the following code:

```
resource "aws_db_instance" "main_db" {
  instance_class         = "db.t3.micro"
  allocated_storage      = 5
  engine                 = "postgres"
  username               = var.db_username
```

More books: https://t.me/booksforits

```
  password               = var.db_password
  db_name                = "to_do"
  publicly_accessible    = true
  skip_final_snapshot    = true
  tags = {
      Name = "to-do production database"
    }
}
```

The fields defined in the preceding code are straightforward apart from the `allocated_storage` field, which is the number of GB allocated to the database. We can also see that we use variables with the `var` variable. This means that we must pass a password and username into our Terraform build when running. We need to define our input variables in the `deployment/variables.tf` file with the following code:

```
variable "db_password" {
    description = "The password for the database"
    default = "password"
}
variable "db_username" {
    description = "The username for the database"
    default = "username"
}
```

These variables have defaults, so we do not need to pass in a variable. However, if we want to pass in a variable, we can do this with the following layout:

```
-var"db_password=some_password" -var"db_username=some_username"
```

We now must pass these parameters into the needed files and the Terraform build. This is where Python starts to shine. We will be reading and writing YAML files, so we will have to install the YAML Python package with the following command:

```
pip install pyyaml
```

We then import this package in our `deployment/run_build.py` file at the top of the script with the following code:

```
import yaml
```

More books: https://t.me/booksforits

We then load database parameters from a JSON file called `database.json` and create our `vars` command string with the following code:

```
with open("./database.json") as json_file:
    db_data = json.load(json_file)

params = f' -var="db_password={db_data["password"]}"
           -var="db_username={db_data["user"]}"'
```

You can come up with any parameters you want for your `deployment/database.json` file; I have recently been playing with GitHub Copilot, which is an AI pair programmer that auto-fills code, and this gave me the following parameters:

```
{
    "user": "Santiago",
    "password": "1234567890",
    "host": "localhost",
    "port": "5432",
    "database": "test"
}
```

I do not know who `Santiago` is, but the Copilot AI clearly thinks that `Santiago` is the right user, so I am going to use it. Going back to our `deployment/run_build.py` file, we must pass our parameters to the Terraform `apply` command by updating the following code:

```
apply_process = Popen(f"cd {DIRECTORY_PATH} && terraform apply"
+ params, shell=True)
```

After the processes that run the Terraform build have finished, we then store the output of that build in a JSON file. We then create our own database URL and write this URL to a text file with the following code:

```
. . .
produce_output.wait()

with open(f"{DIRECTORY_PATH}/output.json", "r") as file:
    data = json.loads(file.read())

database_url = f"postgresql://{db_data['user']}:{db_
data['password']}
    @{data['db_endpoint']['value'][0]}/to_do"
```

More books: https://t.me/booksforits

```
with open("./database.txt", "w") as text_file:
    text_file.write("DATABASE_URL=" + database_url)
```

The only thing we need to do now is update our Rust application config data with the following code:

```
with open("./rust_config.yml") as yaml_file:
    config = yaml.load(yaml_file, Loader=yaml.FullLoader)

config["DB_URL"] = database_url

with open("./rust_config.yml", "w") as yaml_file:
    yaml.dump(config, yaml_file, default_flow_style=False)
```

There is only one change left in our pipeline, and this is in our `deployment/run_build.sh` file. Instead of copying our local `.env` file to our deployment server, we copy our `deployment/database.txt` file with the following code:

```
scp -i "~/.ssh/keys/remotebuild.pem"
./database.txt ec2-user@$1:/home/ec2-user/.env
```

Running our deployment again will deploy our server and connect it to the AWS database that we have created. Again, these build scripts can be brittle. Sometimes, a connection can be refused when copying one of the files to the deployment server, which can result in the breaking of the entire pipeline. Because we have coded all the steps ourselves and understand each step, if there is a break, it will not take us much to manually sort out the break or try to run the Python build script again.

Summary

We have finally come to the end of our journey. We have created our own Docker image, packaging our Rust application. We then ran this on our local computer with the protection of an NGINX container. We then deployed it onto a Docker Hub account, enabling us to use it to deploy onto an AWS server that we set up.

It must be noted that we have gone through the lengthy steps of configuring containers and accessing our server via SSH. This has enabled us to apply this process to other platforms as our general approach was not AWS-centric. We merely used AWS to set up the server. However, if we set up a server on another provider, we would still be able to install Docker on the server, deploy our image onto it, and run it with NGINX and a connection to a database.

More books: https://t.me/booksforits

There are a few more things we can do as a developer's work is never done. However, we have covered and achieved the core basics of building a Rust web application from scratch and deploying it in an automated fashion.

Considering this, there is little holding back developers from building web applications in Rust. Frontend frameworks can be added to improve the frontend functionality, and extra modules can be added to our application to increase its functionality and API endpoints. We now have a solid base to build a range of applications and read further on topics to enable us to develop our skills and knowledge of web development in Rust.

We are at an exciting time with Rust and web development, and hopefully, after getting to this point, you feel empowered to push Rust forward in the field of web development. In the next chapter, we will be encrypting our web traffic to our application using HTTPS.

Further reading

- Rust compiling to different targets documentation: `https://doc.rust-lang.org/rustc/targets/index.html`
- GitHub Actions documentation: `https://github.com/features/actions`
- Travis CI documentation: `https://docs.travis-ci.com/user/for-beginners/`
- CircleCI documentation: `https://circleci.com/docs/`
- Jenkins documentation: `https://www.jenkins.io/doc/`
- Terraform output documentation: `https://developer.hashicorp.com/terraform/language/values/outputs`
- *AWS Certified Developer - Associate Guide*, *Second Edition*, *V. Tankariya* and *B. Parmar* (2019), *Packt Publishing*, *Chapter 5*, *Getting Started with Elastic Compute Cloud (EC2)*, page 165
- *AWS Certified Developer - Associate Guide*, *Second Edition*, *V. Tankariya* and *B. Parmar* (2019), *Packt Publishing*, *Chapter 10*, *AWS Relational Database Service (RDS)*, page 333
- *AWS Certified Developer - Associate Guide*, *Second Edition*, *V. Tankariya* and *B. Parmar* (2019), *Packt Publishing*, *Chapter 21*, *Getting Started with AWS CodeDeploy*, page 657
- *Mastering Kubernetes*, *G. Sayfan* (2020), *Packt Publishing*
- *Getting Started with Terraform*, *K. Shirinkin* (2017), *Packt Publishing*
- *Nginx HTTP Server*, *Fourth Edition*, *M. Fjordvald* and *C. Nedelcu* (2018), *Packt Publishing*

More books: https://t.me/booksforits

11
Configuring HTTPS with NGINX on AWS

When it comes to deploying our applications, a lot of tutorials and books go through simple deployments and smooth over the concept of encrypting traffic to and from the server using HTTPS. However, HTTPS is essential and usually the biggest hurdle that a developer must overcome to get their website or API out into the world. While this book's title is *Rust Web Programming*, it is essential to dedicate a chapter to truly understanding how HTTPS works so that you can implement HTTPS locally and then on the **Amazon Web Services** (**AWS**) cloud. This chapter will cover the following topics:

- What is HTTPS?
- Implementing HTTPS locally with `docker-compose`
- Attaching a URL to our deployed application on AWS
- Enforcing HTTPS on our application on AWS

By the end of this chapter, you will be able to build infrastructure in Terraform code that can encrypt traffic and lock down unwanted traffic to our **Elastic Compute Cloud** (**EC2**) instances so that they can only accept traffic from the load balancer, which will be explained later in the chapter. What is best is that most of this is automated, and seeing as we are using Docker to deploy our applications, you will be able to transfer this skill to any web project that you want to deploy in the future. While this is not the best implementation as there are entire books dedicated to cloud computing, you will be able to implement a solid, safe deployment that will continue to serve users even if an EC2 instance is out of service as the load balancer can route to other instances. It will also be able to scale if traffic demand increases.

More books: https://t.me/booksforits

Technical requirements

In this chapter, we'll build on the code built in *Chapter 10*, *Deploying Our Application on AWS*. This can be found at the following URL: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter10/deploying_our_application_on_aws`.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter11`.

What is HTTPS?

So far, our frontend and backend applications have been running through HTTP. However, this is not secure and has some drawbacks. To secure the traffic between our browser and our NGINX server, we are going to have to ensure that our application is using the HTTP/2 protocol. The HTTP/2 protocol has the following differences from the standard HTTP/1 protocol:

- Binary protocol
- Compressed headers
- Persistent connections
- Multiplex streaming

We can go through the preceding laid-out points and discuss the differences.

Binary protocol

HTTP uses a text-based protocol whereas HTTP/2 uses a binary protocol. A binary protocol uses bytes to transmit data as opposed to human-readable characters, which are encoded using the **American Standard Code for Information Interchange** (**ASCII**). Using bytes reduces the number of possible errors and the size needed to transfer data. It will also enable us to encrypt our data stream, which is the basis of HTTPS.

Compressed headers

HTTP/2 compresses headers when sending requests. Compressing headers has similar benefits to the binary protocol, which results in a lower size of data needed to transfer the same request. The HTTP/2 protocol uses the HPACK format.

More books: https://t.me/booksforits

Persistent connections

When using HTTP, our browser must make a request every time a resource is needed. For instance, we could get our NGINX server to serve an HTML file. This will result in one request to get the HTML file. Inside the HTML file, there could be a reference to a CSS file, and this would result in another request to the NGINX server. It is also not uncommon to have a reference to a JavaScript file in the HTML file. This would result in another request. Therefore, to load a standard web page, our browser requires up to three requests. This does not scale well when we are running a server with multiple users. With HTTP/2, we can have persistent connections. This means that we can make three requests for the HTML, CSS, and JavaScript files in one connection.

Multiplex streaming

Making requests using HTTP/1 means that we must send our requests sequentially. This means that we make one request, wait for this request to be resolved, and then send another request. With HTTP/2, we use multiplex streaming, which means that we can send multiple requests at the same time and resolve them when the responses are returned. Combining multiplex streaming with a persistent connection results in a faster loading time. Any reader who used to use the internet in the 1990s will remember having to wait a long time for a simple page to be loaded. Granted—the internet connection back then was not as fast, but it also was a result of making multiple HTTP sequential requests with different connections to load multiple pictures, HTML, and CSS.

Now that we have explored the difference between HTTP and HTTP/2, we can explore HTTPS, which is built on top of HTTP/2. Before we go forward, however, it must be stated that security is a field in itself. Learning the high-level concepts around HTTPS is enough to make us understand the importance of what we are implementing and why we take certain steps. However, it does not make us security experts.

Before we explore the steps of HTTPS, we need to understand what a middleman attack is because this attack inspires the steps of HTTPS. A middleman attack is exactly what it sounds like: a malicious eavesdropper that can intercept communication packets between the user and the server. This also means that the eavesdropper can also obtain encryption if they are passed over a network. Simply googling "middleman attacks" will result in loads of tutorials and software that can be downloaded to implement such attacks. There are more caveats to security that are beyond the scope of this book, but to conclude, if you are hosting a website that you expect users to connect and log in to, there is no excuse to not use HTTPS.

When it comes to HTTPS, there is a range of steps that need to take place. First, before any requests are made to the server, the owner of the server and domain must get a certificate from a trusted central authority. There are not many trusted central authorities around. What these authorities do is get some identification of the person who owns the domain and some proof that the person applying for the certificate owns the domain. This might sound like a headache, but a lot of URL providers such as AWS have streamlined the process using information, such as payment details, to send to the central trusted authority in the backend when you are pointing and clicking to buy the domain. We will

More books: https://t.me/booksforits

have to fill in some extra forms, but if you have a working AWS account, this will not be too taxing. These central authorities are limited because anyone with a computer can create a digital certificate. For instance, if we were intercepting traffic between a server and a user, we could generate our own digital certificates with a key and forward them to the user. Because of this, mainstream browsers only recognize certificates that have been issued by a small number of recognized authorities. If the browser gets a certificate that is not recognized, these browsers will give a warning, such as in the following Chrome example:

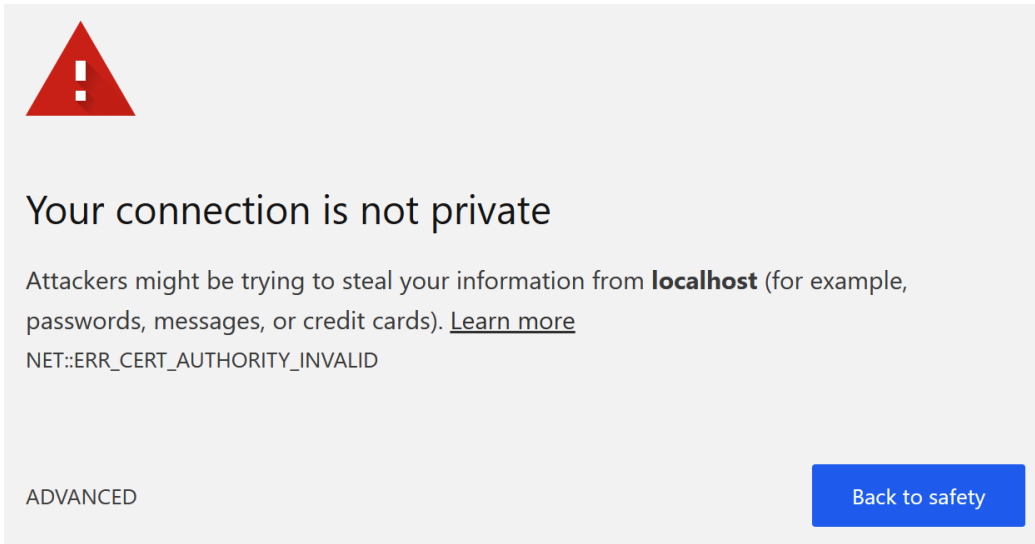


Figure 11.1 – Chrome blocking access due to unrecognized certificate

It is not advised to click on **ADVANCED** to proceed.

Once the owner of the domain and server has the certificate from the central authority, the user can make requests. Before any meaningful data to the application is exchanged, the server sends over the certificate with the public key to the user. The user then creates a session key and encrypts this session key and the public key of the certificate, which is then sent back to the server. The server can then decrypt the key using the private key that was not sent over the network. Therefore, even if the eavesdropper manages to intercept the messages and get the session key, it is encrypted, so they cannot use it. Both the server and client can check the validity of mutually encrypted messages. We can use the session key to send encrypted messages to and from the server and the user, as seen in the following diagram:

More books: https://t.me/booksforits

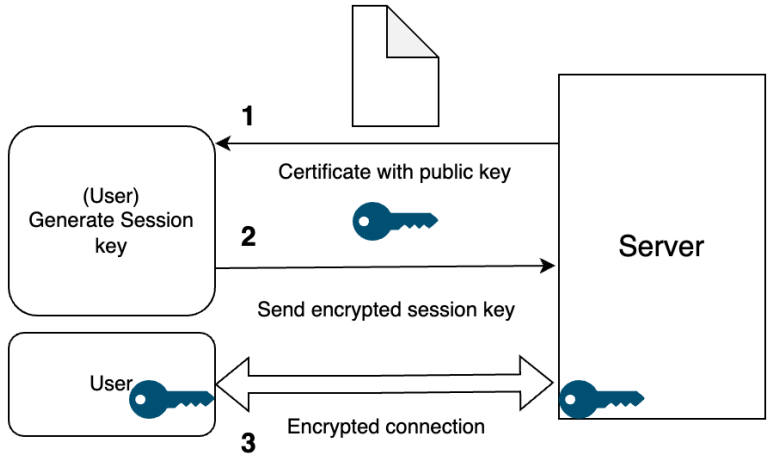


Figure 11.2 – Steps required when making an HTTPS connection

Do not worry—there are packages and tools that will help us manage the HTTPS process; we will not have to implement our own protocol. You will understand why we must carry out certain steps and how to troubleshoot problems when they arise. In the next section, we will implement a basic HTTPS protocol with NGINX locally.

Implementing HTTPS locally with docker-compose

When it comes to implementing HTTPS, most of the work is going to be achieved through NGINX. Although we have worked a little with NGINX, the NGINX configuration is a powerful tool. You can implement conditional logic, pull variables and data from the request and act on them, redirect traffic, and much more. In this chapter, we are going to do enough to implement HTTPS, but it is advised if you have time to read up on the fundamentals of NGINX configurations; reading material is provided in the *Further reading* section. For our `deployment/nginx_config.yml` file, we need to have the following layout:

```
worker_processes  auto;
error_log  /var/log/nginx/error.log warn;

events {
    worker_connections  512;
}
http {
    server {
        . . .
```

More books: https://t.me/booksforits

```
    }

    server {
        . . .
    }
}
```

Here, we can see that we have two `server` scopes in our `http` scope. This is because we need to enforce HTTPS. We must remember that our outside port is `80`. However, if we want to carry out an HTTPS connection, we instead need to connect to port `443`, which is the standard for HTTPS. Typing `https://` into the browser will target port `443`, and typing `http://` into the browser will target port `80`. If we allow port `80` to be open, loads of users will access our site in an unsecured way because some people will type `http://` into the browser. Hackers would also spread the HTTP links around because they want as many people as possible to not use a secure network. However, if we block port `80`, people who put `http://` into the browser will just get blocked from accessing the website. It is unlikely that the average user is going to understand the differences in ports, look at what they typed in, and correct it. Instead, they are just going to think that the site is down. Therefore, we are going to have to listen to both ports `443` and `80`. However, when a request is made to port `80`, we going to redirect the request to port `443` instead. Our first server scope can redirect with the following code:

```
server {
    listen 80;
    return 301 https://$host$request_uri;
}
```

Here, we can see that we listen to port `80`, and then return the same request that was sent but with the HTTPS protocol, which means that it will hit our `443` port. We can also see that we reference `$host` and `$request_uri` variables. These variables are standard variables in NGINX that are automatically populated. We can define our own variables with the following line of code:

```
set $name 'Maxwell';
```

However, we want our NGINX instance to work on our server and localhost, so using the standard variables is the best choice here. Now that we have defined our rerouting rule, we can move on to the next server scope; we listen to port `443` with the following code:

```
server {
    listen 443 ssl http2;

    ssl_certificate /etc/nginx/ssl/self.crt;
```

More books: https://t.me/booksforits

```
    ssl_certificate_key /etc/nginx/ssl/self.key;

    location /v1 {
        proxy_pass http://rust_app:8000/v1;
    }
    location / {
        proxy_pass http://front_end:4000/;
    }
}
```

Looking at the preceding code, it must be noted that we are handling and directing traffic for both the frontend and backend applications in the same NGINX instance. Alongside the port definition, we also declare that we are using the `ssl` and `http2` NGINX modules. This is not surprising as HTTPS is essentially SSL on top of HTTP/2. We then define where our server certificate is in the NGINX container. We will add these later in the `docker-compose` volume. We can also see that we pass our HTTPS request to the appropriate application via HTTP. If we try to change these proxies to the HTTPS protocol, then we would get a bad gateway error. This is because the handshake between NGINX and our services would fail. It is not essential because we must remember that the ports that the frontend and backend applications expose are not available to anyone outside of localhost. Yes—on our local machine we can access them, but this is because they are running on our local machine. If we were to deploy our application server, it will look like this:

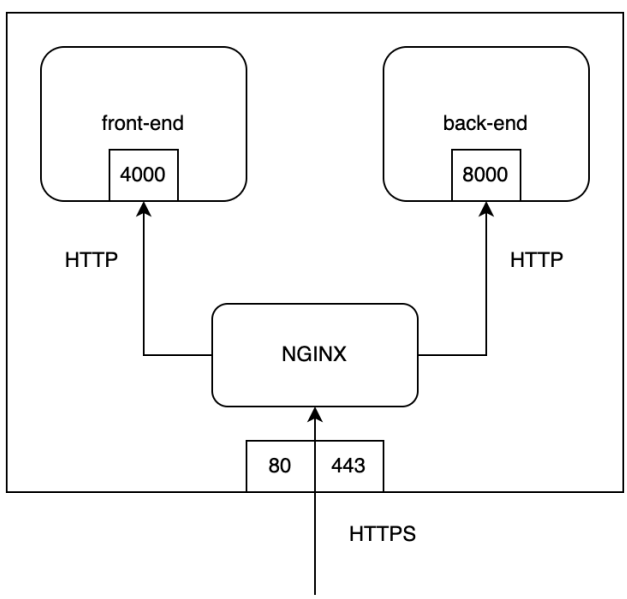


Figure 11.3 – Traffic flow if deployed on a server

More books: https://t.me/booksforits

Our NGINX configuration is not optimum. There are settings that can be tweaked in terms of ciphers, caching, and managing timeouts. However, this is enough to get an HTTPS protocol working. If you get to the point that you need to optimize the caching and encryption methods of your NGINX configuration, it is suggested that you seek further education materials on DevOps and NGINX.

Now that we have defined our NGINX configuration, we must define our certificates.

> **Note**
>
> To define our own certificates, we must install the `openssl` package by following the steps using the next links:
>
> **Linux**:
>
> `https://fedingo.com/how-to-install-openssl-in-ubuntu/`
>
> **Windows**:
>
> `https://linuxhint.com/install-openssl-windows/`
>
> **Mac**:
>
> `https://yasar-yy.medium.com/installing-openssl-library-on-macos-catalina-6777a2e238a6`

This can be done with the following command:

```
openssl req -x509 -days 10 -nodes -newkey rsa:2048
-keyout ./self.key -out ./self.crt
```

This creates a key with `x509`, which is the international telecommunication union standard. We state that the certificate will expire in 10 days and that the key and certificate will have the name `self`. They can be called anything; however, for us, it makes sense to call the certificate `self` because it is a self-issued certificate. The command shown in the previous code snippet will push several prompts. It does not matter what you say to these prompts as we will just be making localhost requests with them, meaning that they will never go anywhere outside of our local computer. We can now stash the key and certificate anywhere within the `deployment` directory if you can reference the key and certificate placed in the `docker-compose.yml` file. In our `docker-compose.yml` file, our NGINX service now takes the following form:

```
nginx:
container_name: 'nginx-rust'
image: "nginx:latest"
ports:
  - "80:80"
  - 443:443
```

More books: https://t.me/booksforits

```
links:
  - rust_app
  - front_end
volumes:
  - ./nginx_config.conf:/etc/nginx/nginx.conf
  - ./nginx_configs/ssl/self.crt:/etc/nginx/ssl/self.crt
  - ./nginx_configs/ssl/self.key:/etc/nginx/ssl/self.key
```

Here, we can see that I chose to store the key and certificate inside a directory called `nginx_configs/ssl/`. This is because I have added several simple NGINX configs into the GitHub repo under the `nginx_configs` directory if you want some easy quick references on handling variables, conditional logic, and serving HTML files directly from NGINX. While where you get your key and certificate from may vary, it is important that you put the key and the certificate inside the `etc/nginx/ssl/` directory inside the NGINX container.

We are now at the point where you can test our application to see if the local HTTPS is working. If you spin up your `docker-compose` instance and then go to the `https://localhost` URL in your browser, you should get a warning that it is not secure, and you will not be able to instantly connect to the frontend. This is reassuring because we are not a central authority, so our browser will not recognize our certificate. There is a multitude of browsers, and we would waste a lot of space in this book describing how to get past this for every browser. Considering browsers are free to download, we can get around the blocking of our application in Chrome by going to the `flags` URL, as seen here:

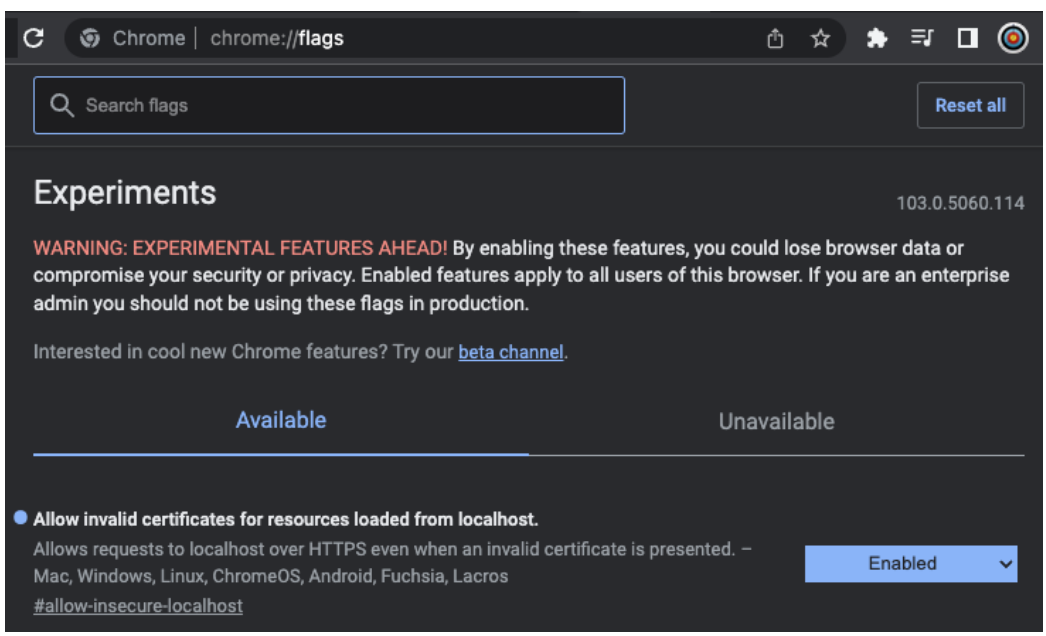


Figure 11.4 – Allowing our application certificate to pass in Chrome

More books: https://t.me/booksforits

Here, we can see that I have allowed invalid certificates from localhost. Now that our invalid certificate is enabled in our browser, we can access our application, as seen here:

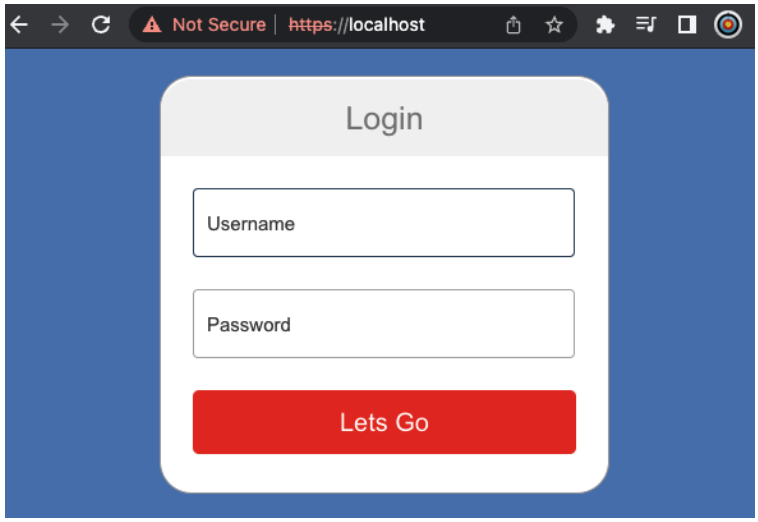


Figure 11.5 – Accessing our application through HTTPS

Here, we are using the HTTPS protocol; however, as we can see in the preceding screenshot, Chrome is complaining stating that it is not secure. We can inspect the reason why by clicking on the **Not Secure** statement, giving the following view:

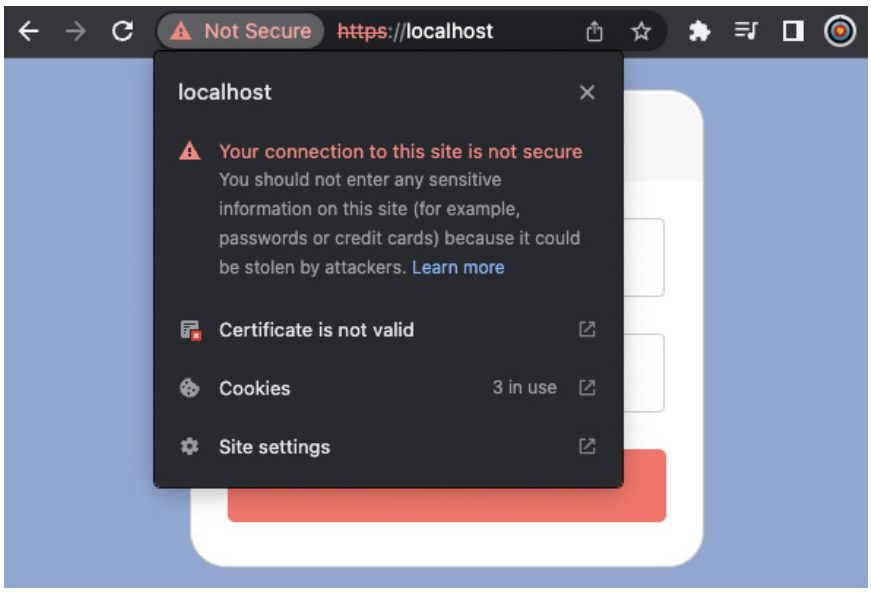


Figure 11.6 – Explaining why the connection is not secure

More books: https://t.me/booksforits

Here, we can see that our certificate is not valid. We expected the certificate to not be valid because we issued it, which makes it not officially recognized. However, our HTTPS connection is working! This is interesting to see how HTTPS works; however, it is not useful with a self-signed certificate running on our localhost. If we want to utilize HTTPS, we are going to have to apply it to our application on AWS. There are a couple of steps we need to carry out before we implement HTTPS on AWS. In the next section, we will assign a URL to our application.

Attaching a URL to our deployed application on AWS

In the previous chapter, we managed to deploy our to-do application onto a server on AWS and access this application directly by putting the IP address of the server into our browser. When it comes to registering our URL, you will be exposed to multiple acronyms. To feel comfortable when navigating AWS routing, it makes sense to be familiar with URL acronyms by reading the following diagram:

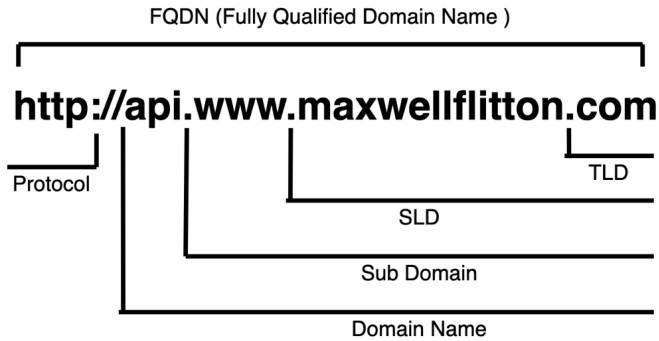


Figure 11.7 – Anatomy of a URL

When we are associating a URL with our application, we are going to be configuring a **Domain Name System** (**DNS**). A DNS is a system that translates a user-friendly URL to an IP address. For a DNS system to work, we will need the following components:

- **Domain registrar**: An organization such as AWS, Google Cloud, Azure, GoDaddy, and so on that will register a domain if it receives payment for the domain and personal details of the person responsible for the domain. This organization will also handle reports of abuse if the URL is used in illegal activity.
- **DNS record**: A registered URL can have multiple DNS records. A DNS record essentially defines a routing rule for the URL. For instance, a simple DNS record will forward the URL to an IP address of a server.
- **Zone file**: A container for DNS records (in our case, the zone file will be managed by AWS).

More books: https://t.me/booksforits

The DNS records and registrars are essential for our URL to work. Even though we can directly connect to IP addresses, there are a couple of middlemen if we want to connect to a URL, as laid out here:

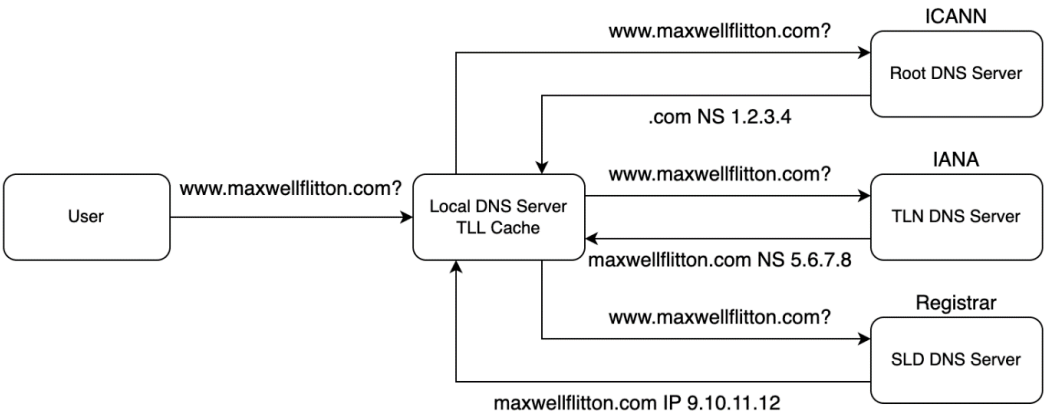


Figure 11.8 – Steps required to connect a URL to an IP address

As we can see from the preceding diagram, if we want to connect to a server, we send the URL to the local DNS server. This server then makes three calls in sequential order from top to bottom. By the end of the three requests, the local DNS server will have the IP address related to the URL. We can see that the registrar is responsible for part of the mapping. This is where our DNS records are configured. If we remove our DNS records, then the URL is no longer available on the internet. We do not have to make the calls laid out in *Figure 11.8* every time we enter a URL. Our browsers and the local DNS server will cache the URL to the IP address mapped to reduce the number of calls to the other three servers. There is a problem, however; when we have been building our production server, you might have realized that the IP address has changed every time we tear down and spin up a production server. There's nothing wrong going on here; when we create an EC2 instance, we must take a server that is available. A cloud provider such as AWS cannot hold the server aside just for us unless we want to pay for it. In the next section, we will keep our IP consistent with elastic IP addresses.

Attaching an elastic IP to our server

Elastic IP addresses are essentially fixed IP addresses that we keep. We can then attach these elastic IP addresses to any one EC2 instance at one point in time as we see fit. This is very helpful when it comes to routing. We can set up the routing of a URL to an elastic IP and then switch the allocation of the elastic IP to the server that we need. This means that we can deploy a new application to another server, test it, and then switch our elastic IP to the new deployment server without having to touch the routing for our URL.

We will not be creating an elastic IP every time we spin up a production server. Because of this, it is OK to point and click in the AWS console to create and attach the elastic IP address. Before we do

More books: https://t.me/booksforits

this, however, we need to deploy our production server with the previous NGINX config file that does not have HTTPS defined, but instead has the following form:

```
worker_processes  auto;
error_log  /var/log/nginx/error.log warn;

events {
    worker_connections  512;
}
http {
    server {
        listen 80;

        location /v1 {
            proxy_pass http://rust_app:8000/v1;
        }
        location / {
            proxy_pass http://front_end:4000/;
        }
    }
}
```

This should make sense to you now, as the NGINX config merely listens to HTTP requests through the outside port `80` and then passes them through to our applications. We also must remove our reference to our self-signed certificates because we will not need them, and we will also not be uploading those certificates to our server. Considering our lack of reference to our certificates, our `docker-compose` instance in our `deployment` directory should have the following NGINX definition:

```
nginx:
  container_name: 'nginx-rust'
  image: "nginx:latest"
  ports:
    - "80:80"
  links:
    - rust_app
    - front_end
  volumes:
    - ./nginx_config.conf:/etc/nginx/nginx.conf
```

More books: https://t.me/booksforits

We are now ready to deploy our build on our production server. Remember—we can do this using the `deployment/run_build.py` Python script that we set up in the previous chapter. Once the server is built, we know that there is an EC2 instance live with the `"to-do production server"` tag. We are now ready to allocate an elastic IP address to our EC2 instance.

To allocate elastic IPs, we first need to navigate to the EC2 service by searching for EC2 in the search bar in the AWS dashboard at the top and clicking on the service, resulting in the following view:

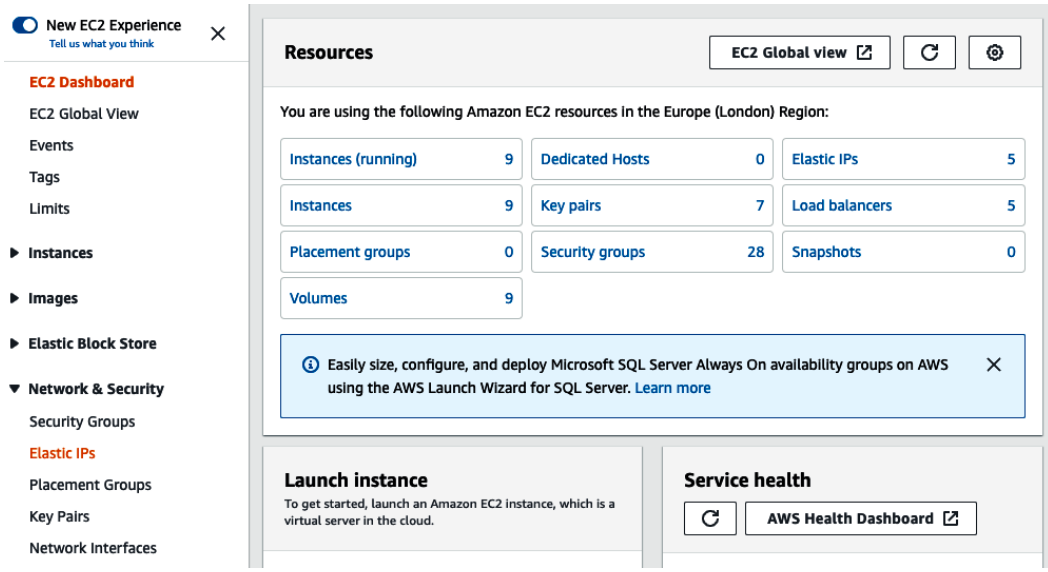


Figure 11.9 – EC2 dashboard view

We can see that **Elastic IPs** can be accessed on the right of the **Resources** panel and on the left of the screen. Once we are in the elastic IP address dashboard, we will have a list of elastic IP addresses that you have. On the top right of the screen, there will be an orange button labeled **Allocate Elastic IP address**. If you click this button, you will get the following creation form:

More books: https://t.me/booksforits

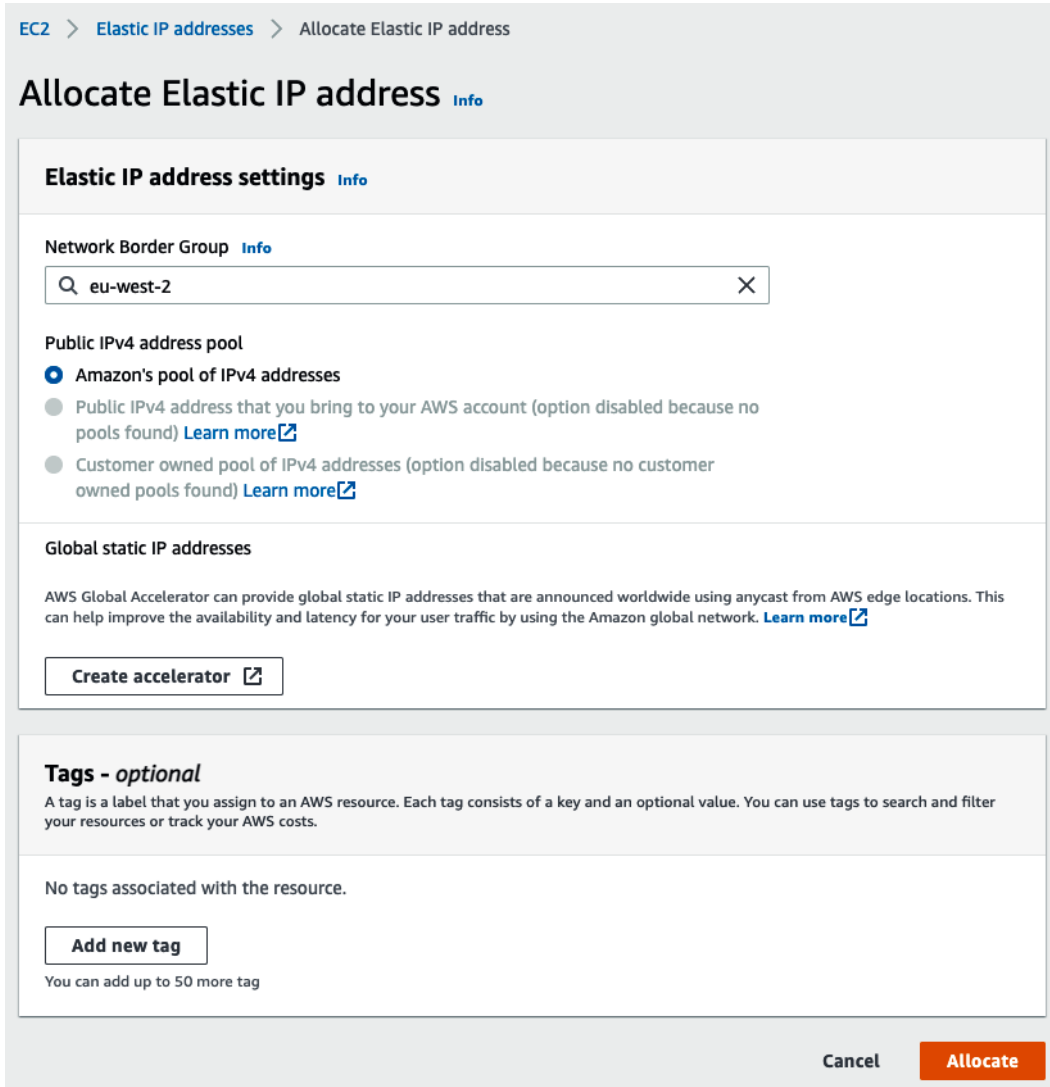


Figure 11.10 – Allocating (creating) an elastic IP address

What we are doing here is grabbing an elastic IP address from a pool of IP addresses for the area you are working on. There is a limit of five elastic IP addresses per account. If you think this is not enough for you, you will need to get more creative with your networking infrastructure. You can also investigate creating sub-accounts for the main account. As with clean code, there are benefits to having clean accounts that only work on one project at a time. This will help you keep track of costs, and shutting down all infrastructure for a project will be clean as you can be sure that everything for that project has been cleared by deleting the account.

More books: https://t.me/booksforits

Moving on to our allocation of the elastic IP to our EC2 server, we can allocate our elastic IP by highlighting the desired elastic IP address in the elastic IP dashboard and clicking on the **Actions** button at the top right of the dashboard, as seen here:

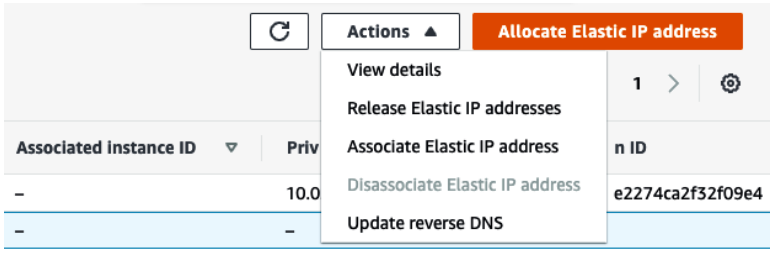


Figure 11.11 – Actions on our elastic IP

Under **Actions**, we must click on the **Associate Elastic IP address** option, giving us the following display:

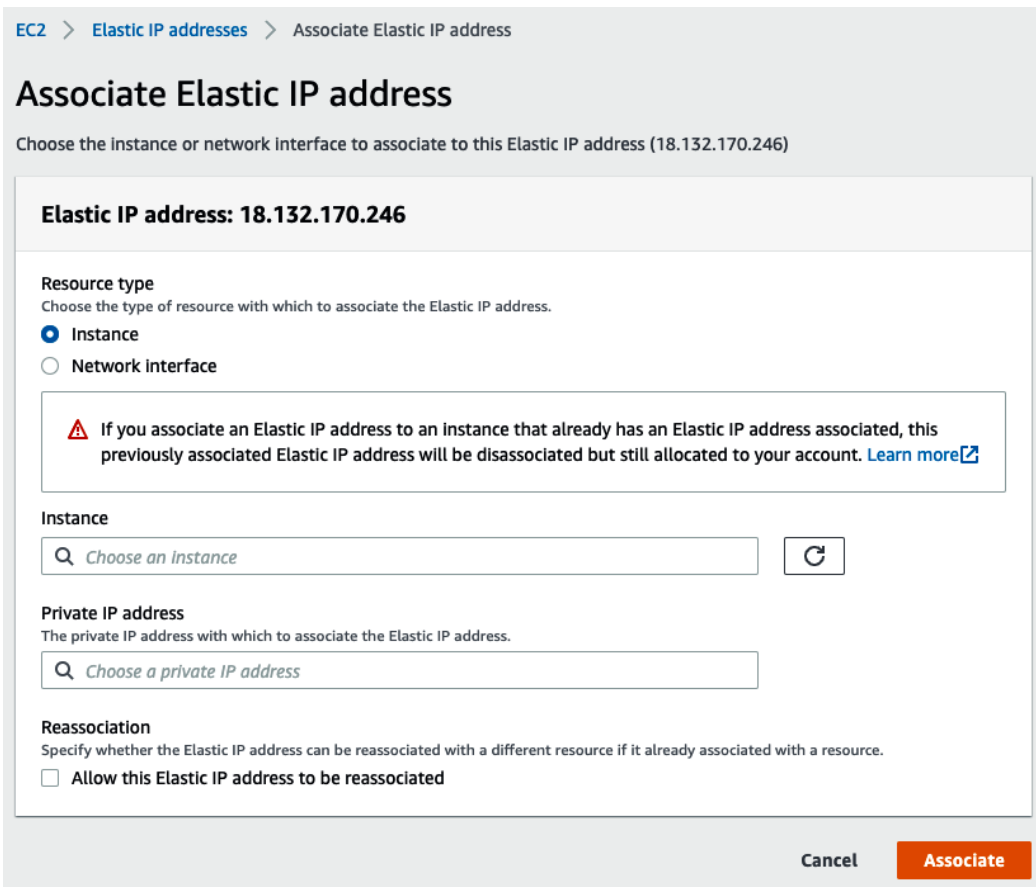


Figure 11.12 – Associating an elastic IP address

More books: https://t.me/booksforits

The **Instance** option will provide a drop-down menu of the EC2 instances that we have running. Luckily, we have the helpful `"to-do production server"` tag that we defined in Terraform. However, if we do not have a tag, we can still choose an instance from the drop-down menu. We can then click the **Associate** button. Once this is done, we can go to our elastic IP address in our browser, and we should be able to access our to-do application, as seen here:

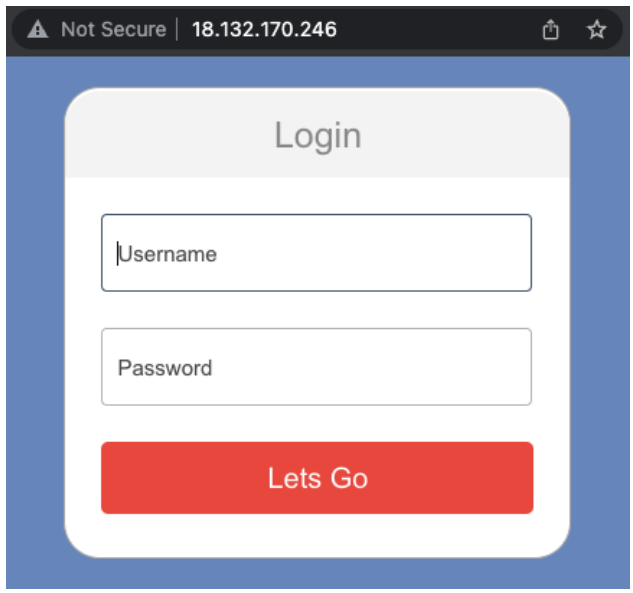


Figure 11.13 – Accessing our to-do application using a static IP

And here we have it—our application can be accessed with an elastic IP!! We can now spin up a new server if we want, test it, and redirect our elastic IP to the new server if we are happy, providing seamless updates without our users knowing. However, getting users to type in the raw IP address is not desirable. In the next section, we will register a domain and connect it to our elastic IP address.

Registering a domain name

When it comes to registering a domain, it can all be handled in AWS with the Route 53 service. First, we navigate to Route 53, which is the service that handles routing and URL registration. On the left side of the **Route 53** dashboard web page, we can click on the **Registered domains** section, as seen in the following screenshot:

More books: https://t.me/booksforits

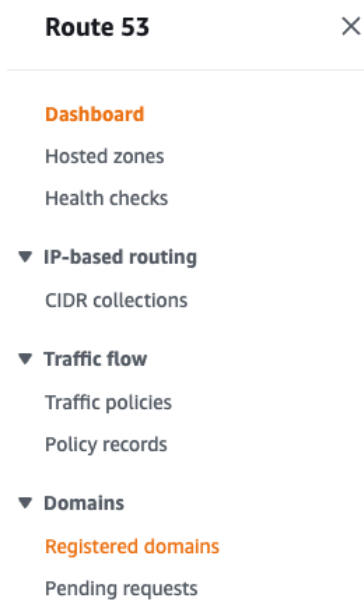


Figure 11.14 – Navigation to registered domains in the Route 53 dashboard

We will then see a list of registered domains that we already own and the option to register a domain, as shown in the following screenshot:

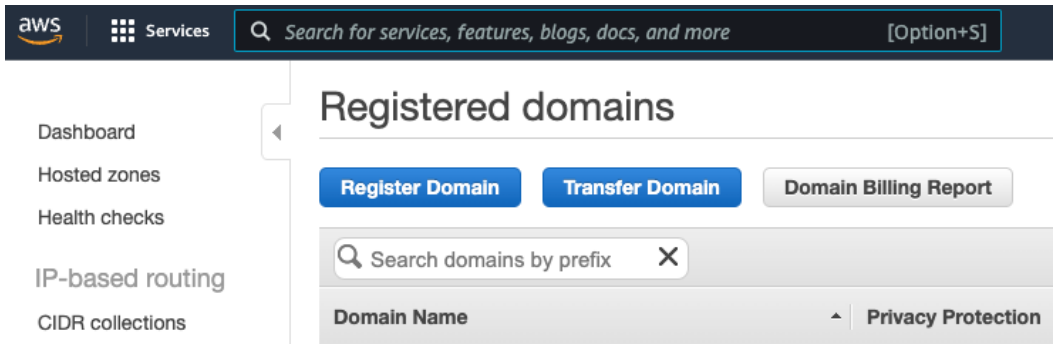


Figure 11.15 – Registered domains view

More books: https://t.me/booksforits

If you click on the **Register Domain** button, you will be led through a straightforward series of forms to register your domain. Screenshotting these steps would be excessive. The forms ask you for the domain you want to register. They will then tell you if the domain and other domains such as this are available. Domains cost about $12 a year on average at the time of writing this book. Once you have selected your domain and clicked on the checkout, you will be passed through a series of personal information forms. These forms include a contact address and whether the domain is for personal use or for a company. Screenshotting these forms would result in excessive pages with little educational advantage as these forms are personal and easy to fill in. It is recommended that you select the validation through DNS because this is automated.

Once your domain is registered, you can go to the **Hosted zones** section in the main Route 53 dashboard. Here, we will see a list of hosted zones for each URL that you own. A hosted zone is essentially a collection of DNS records. If we click on a hosted zone for a URL, there will be two DNS records: NS and SOA (NS—name server; SOA—start of authority). These records should not be deleted, and if they are, somebody else who knows what these records are could hijack your URL by implementing those records themselves. DNS records are essentially records on how to route traffic for a domain name. Each DNS record has the following attributes:

- **Domain/subdomain name**: The name of the URL the record belongs to
- **Record type**: The type of record (A, AAAA, CNAME, or NS)
- **Value**: Target IP address
- **Routing policy**: How Route 53 responds to queries
- **TLL (Time to Live)**: The amount of time the record is cached in the DNS resolvers in the client so that we do not have to query the Route 53 servers too often, with the trade-off of reducing traffic to DNS servers versus time for updates to roll onto clients

The attributes just defined are self-explanatory, apart from the record type. There are advanced record types that we can build in Route 53. However, the following record types are needed for us to route our domain to our IP:

- **A**: The simplest record type. Type A merely routes traffic from the URL to an IPv4 IP address.
- **AAAA**: Routes traffic from the URL to an IPv6 address.
- **CNAME**: Maps a hostname to another hostname (target must be an A or AAAA record type).
- **NS**: Name server for the hosted zone (controls how the traffic is routed).

More books: https://t.me/booksforits

For us, we are going to create an DNS A record by clicking on the **Create record** button, as seen in the following screenshot:

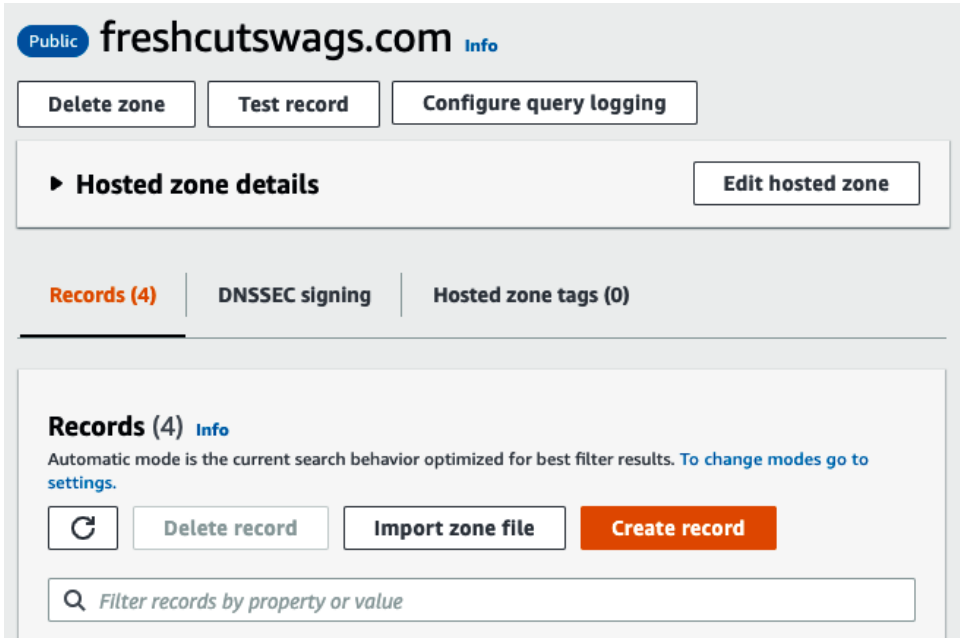


Figure 11.16 – Creating a DNS record

Once we have clicked on this, we get the following layout:

More books: https://t.me/booksforits

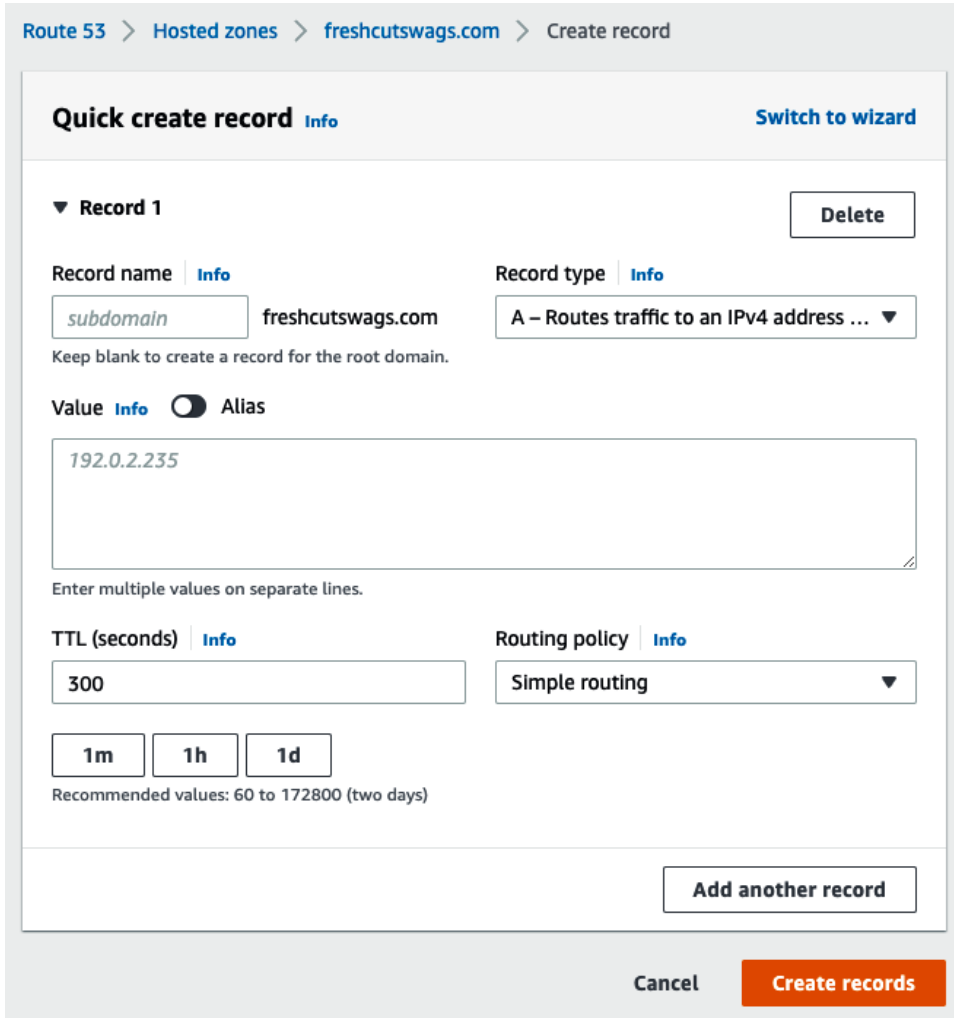


Figure 11.17 – Creating a DNS record form

We can see that record type `A` is the default. We can also see that we can add a subdomain. This gives us some flexibility. For instance, if we wanted, the `api.freshcutswags.com` URL could point to a different IP address from `freshcutswags.com`. For now, we are going to just leave the subdomain empty. We are then going to put the elastic IP address that we set up in the previous section in the **Value** section and click **Create records**. We are then going to create the exact same DNS record but with the subdomain of `www`. Once this is done, we should have two `A` records. We can then check where our URL is mapping the traffic using the `www.digwebinterface.com` website. Here, we can enter URLs, and the website will tell us where the URLs are being mapped. We can see here that our URLs are both mapping to the correct elastic IP:

More books: https://t.me/booksforits

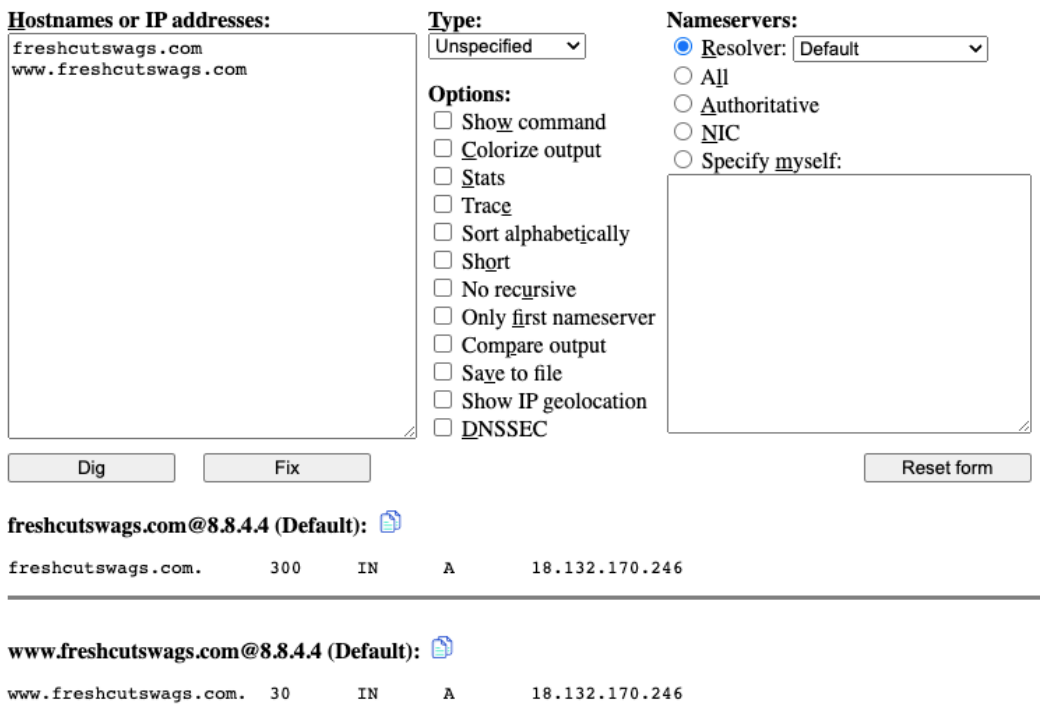


Figure 11.18 – Inspecting our URL mapping

With the mapping result confirmed, as seen in the preceding screenshot, we can visit our URL and expect to see the following:

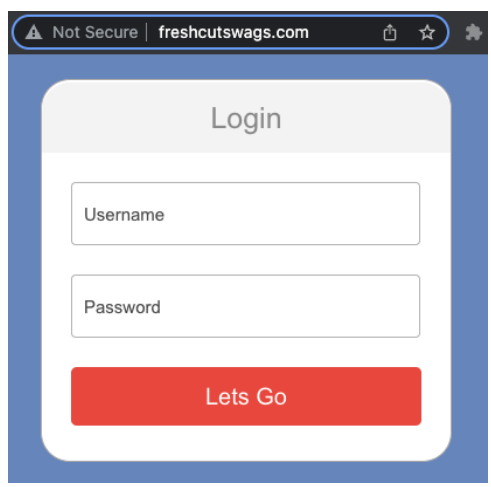


Figure 11.19 – Accessing our application through the registered URL

More books: https://t.me/booksforits

We can see that our URL is now working. However, the connection is not secure. In the next section, we will enforce an HTTPS protocol for our application and lock it down, as right now, even though we can access our application through the URL, there is nothing stopping us from accessing the IP of the server directly.

Enforcing HTTPS on our application on AWS

Right now, our application kind of works, but it is a nightmare in terms of security. By the end of this section, we will not have the most secure application, as further reading of a networking and DevOps textbook is suggested to achieve gold-standard security. However, we will have configured security groups, locked down our EC2 instances so that they cannot be directly accessed by outsiders, and enforced encrypted traffic through a load balancer that will then direct traffic to our EC2 instances. The result of our efforts will be the following system:

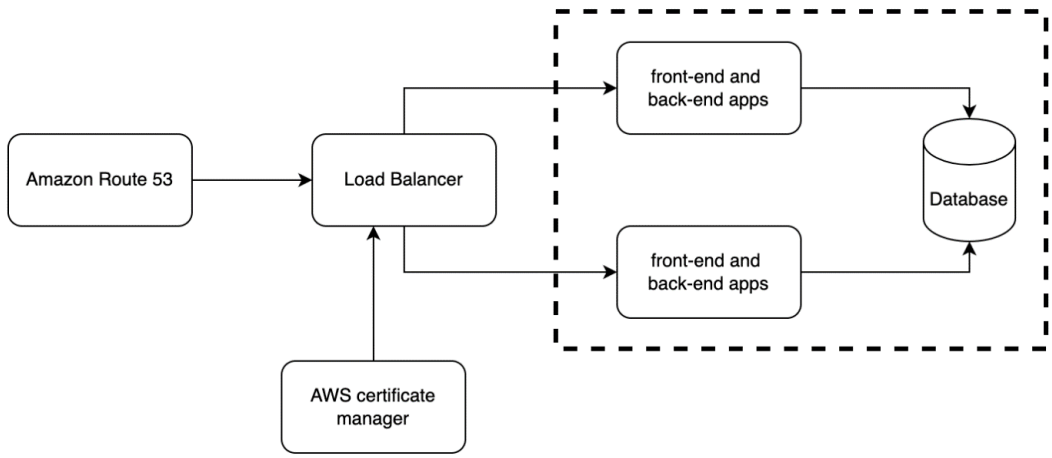


Figure 11.20 – Layout of our desired system to achieve HTTPS

To achieve the system shown in *Figure 11.20*, we need to carry out the following steps:

1. Get certificates approved for our URL and variations.
2. Create multiple EC2 instances to distribute traffic and ensure that the service survives outages.
3. Create a load balancer to handle incoming traffic.
4. Create security groups.
5. Update our Python build script to support multiple EC2 instances.
6. Attach our URL to the load balancer using the Route 53 wizard.

More books: https://t.me/booksforits

In the previous section on attaching a URL to our application on AWS, we did a lot of pointing and clicking. As said in the previous chapter, pointing and clicking should be avoided if possible as it is not repeatable and we as humans forget what we did. Sadly, with URL approvals pointing and clicking is the best option. In this section, only the first and sixth steps will require pointing and clicking. The rest will be achieved with Terraform and Python. We are going to be making some big changes to our Terraform config, so it is advised that you run a `terraform destroy` command before altering your Terraform config. Before we can do any of the coding, however, we are going to have to get our certificates for our URL.

Getting certificates for our URL

Because we brought our URL through Route 53, which is handled by AWS, and our servers are running in AWS, the certification and implementation of those certificates is a straightforward process. We need to navigate to Certificate Manager by typing `certificate manager` into the services search bar and clicking on it. Once there, we are displayed with a page that only has one orange button labeled **Request a certificate**. Click on this, and we will be transported to the following page:

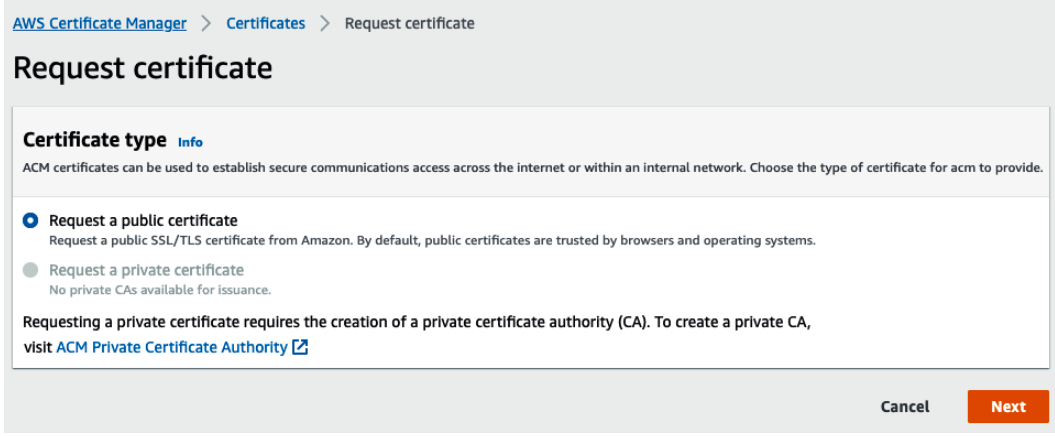


Figure 11.21 – Certificate Manager: start of journey

We are going to want our certificate to be public facing; therefore, we are happy with the default selection, and we click **Next**. We are then presented with the following form:

More books: https://t.me/booksforits

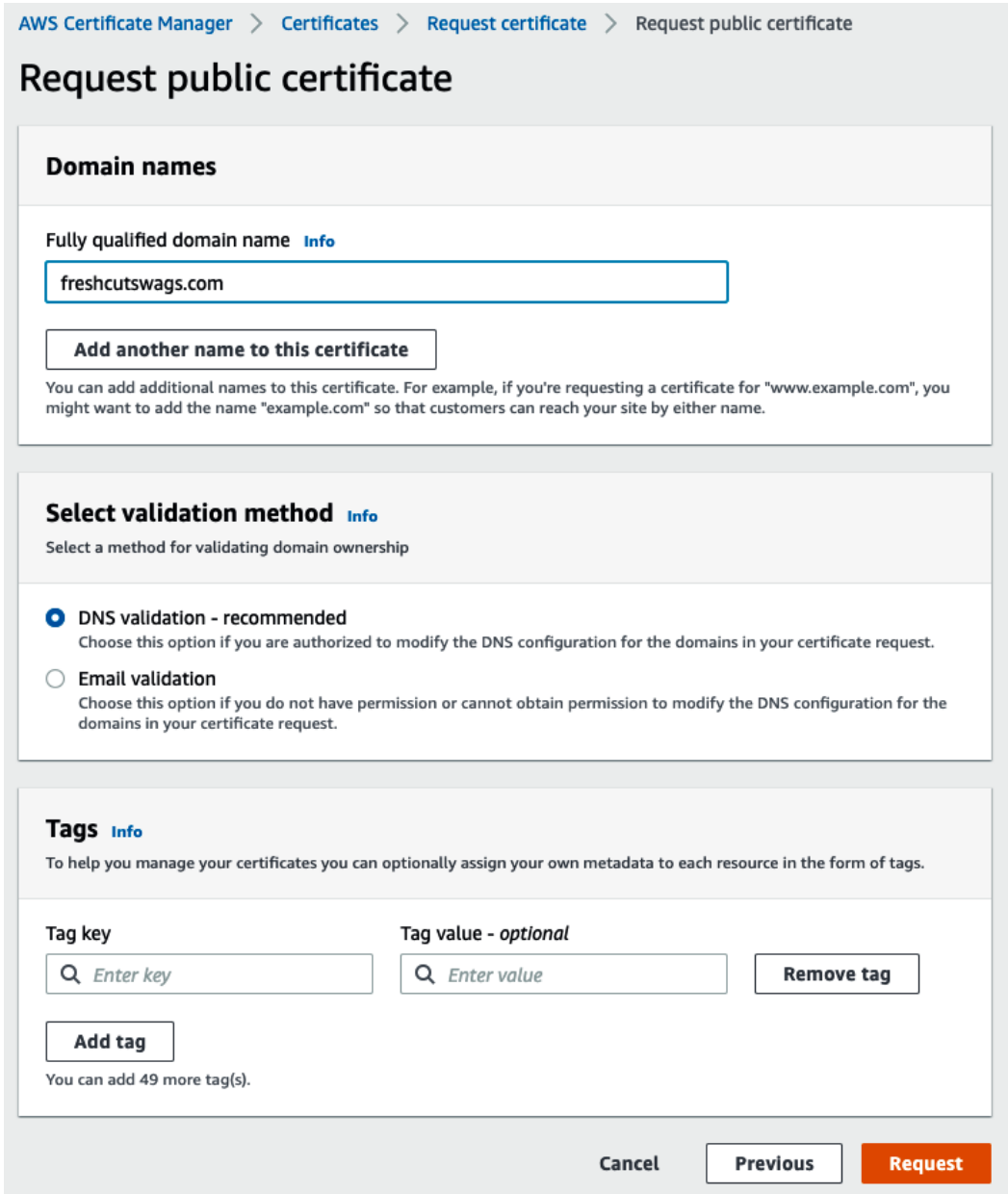


Figure 11.22 – Defining certificate request

More books: https://t.me/booksforits

Here, we type in the URL we want to associate with the certificate. We could add another, but we will do a separate certificate for our URL that has prefixes as we want to explore how to attach multiple certificates in Terraform. We can also see that DNS validation is already highlighted, and this is recommended as we have our servers on AWS, meaning we will have to take no more action to get the certificate issued. We can then click on the button labeled **Request**, and we will be redirected to a page with the list of certificates. I find that the new certificate request is not present nearly every time I have done this. My guess is that there is a delay. Do not worry—merely refresh the page, and you will see the pending certificate request listed. Click on this listing, and you will be directed to a detailed view of this certificate request. In the middle of the screen on the right, you need to click the **Create records in Route 53** button, as seen here:

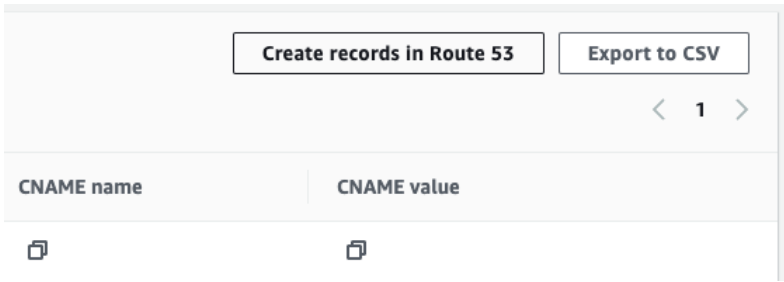


Figure 11.23 – Creating records for DNS confirmation

Follow the prompts after clicking the button, and the CNAME record will be created. If you do not do this, then the pending status for the ticket will go on indefinitely because the cloud provider needs the route to issue the certificate, considering we selected DNS validation. After a few minutes, the certificate should be issued. Once this is done, carry out the same steps for the prefix wildcard. Once this is done, your certificate lists should look something like the following:

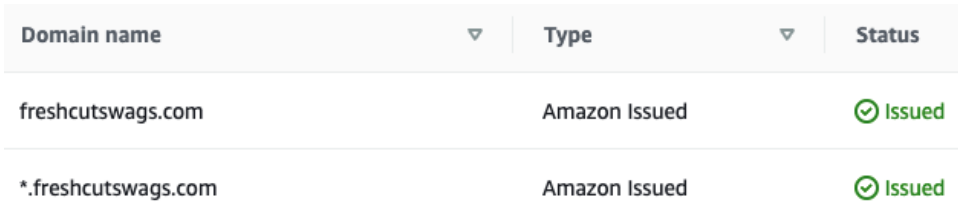


Figure 11.24 – Issued certificates

In the preceding screenshot, we can see that I have two certificates: one for if a user directly types in the URL with no prefix, and the wildcard that covers all prefixes. We are ready to use these certificates, but before we do this, we are going to have to carry out some other steps. Before we define rules around the traffic, we are going to have to build the infrastructure where the traffic is going. In the next section, we will build two EC2 instances.

More books: https://t.me/booksforits

Creating multiple EC2 instances

We will be using a load balancer. Because of this, we need a minimum of two EC2 instances. This means that if one EC2 instance is down, we can still use the other EC2 instance. We can also scale our application. For instance, if everyone in the world suddenly realized that they needed a to-do app to sort out their lives, there is nothing stopping us from increasing the number of EC2 instances to distribute the traffic across. We can increase our EC2 instances to two by going into our `deployment/main.tf` file and having the following definition of our EC2 instances:

```
resource "aws_instance" "production_server" {
    ami = "ami-0fdbd8587b1cf431e"
    instance_type = "t2.medium"
    count = 2
    key_name = "remotebuild"
    user_data = file("server_build.sh")
    tags = {
      Name = "to-do prod ${count.index}"
    }
    # root disk
    root_block_device {
      volume_size = "20"
      volume_type = "gp2"
      delete_on_termination = true
    }
}
```

Here, we can see that we have added a `count` parameter and defined it as `2`. We have also altered the tag. We can also see that we access the number of EC2 instances being created with `index`. The index starts at zero and increases by one every time a resource is made. Now that we have two instances, we must update the outputs at the bottom of the `deployment/main.tf` file with the following code:

```
output "ec2_global_ips" {
  value = ["${aws_instance.production_server.*.public_ip}"]
}
output "db_endpoint" {
  value = "${aws_db_instance.main_db.*.endpoint}"
}
output "public_dns" {
  value =
```

More books: https://t.me/booksforits

```
      ["${aws_instance.production_server.*.public_dns}"]
}
output "instance_id" {
    value = ["${aws_instance.production_server.*.id}"]
}
```

Here, we can see that apart from the database endpoint, all the other outputs have been changed to lists. This is because they all reference our multiple EC2 instances. Now that we have our EC2 instances defined, we can route traffic to our instances with a load balancer.

Creating a load balancer for our traffic

There is a range of different load balancers that we can pick from. We have already discussed NGINX, which is a popular load balancer. For this chapter, we are going to use the application load balancer to route the traffic to our EC2 instances and implement the HTTPS protocol. There are multiple features that load balancers can offer, and they protect against **distributed denial-of-service** (**DDoS**) attacks whereby the attacker will try to overload the server with excessive requests. We will create our load balancer in the `deployment/load_balancer.tf` file. First, we collect the data that we need with the following code:

```
data "aws_subnet_ids" "subnet" {
    vpc_id = aws_default_vpc.default.id
}
data "aws_acm_certificate" "issued_certificate" {
    domain   = "*.freshcutswags.com"
    statuses = ["ISSUED"]
}
data "aws_acm_certificate" "raw_cert" {
    domain   = "freshcutswags.com"
    statuses = ["ISSUED"]
}
```

We can see that instead of `resource`, we used the `data` declaration. This is where we make queries to AWS for specific types of data to be used in the rest of the Terraform script. We get the **virtual private cloud** (**VPC**) ID. In Terraform, we can define and build a VPC, but throughout this book, we have been using the default VPC. We can get the default VPC ID for our load balancer. We then get the data for the certificates that we have defined in the previous section with the preceding code.

More books: https://t.me/booksforits

We now must define a target for our load balancer. This is done in the form of a target group where we can bunch a group of instances together for the load balancer to target with the following code:

```
resource "aws_lb_target_group" "target-group" {
    health_check {
        interval = 10
        path = "/"
        protocol = "HTTP"
        timeout = 5
        healthy_threshold = 5
        unhealthy_threshold = 2
    }

    name = "ToDoAppLbTg"
    port = 80
    protocol = "HTTP"
    target_type = "instance"
    vpc_id   = aws_default_vpc.default.id
}
```

Here, we can see that we define the parameters for a health check. The parameters for the health check are self-explanatory. The health check will alert a service targeting the health status of the target group. We would not want to route traffic to a target group that is down. We then define the protocol and port of the traffic, the type of resource in the target group, and the ID of the VPC. Now that our target group is defined, we can attach our EC2 instances to it with the following code:

```
resource "aws_lb_target_group_attachment" "ec2_attach" {
    count = length(aws_instance.production_server)
    target_group_arn = aws_lb_target_group.target-group.arn
    target_id =
        aws_instance.production_server[count.index].id
}
```

We can see that we get the IDs of the EC2 servers for the target ID. With this, our EC2 instances can be targeted by the load balancer. Now that we have a target, we can create our load balancer with the following code:

```
resource "aws_lb" "application-lb" {
    name = "ToDoApplicationLb"
```

More books: https://t.me/booksforits

```
    internal = false
    ip_address_type = "ipv4"
    load_balancer_type = "application"
    security_groups = ["${aws_security_group.
        alb-security-group.id}"]
        subnets = data.aws_subnet_ids.subnet.ids

    tags = {
        name = "todo load balancer"
    }
}
```

We can see that the parameters in the load balancer definition are straightforward. However, you may have noticed the security group definition. We are referencing a security group even though we have not defined any security groups. If you do not know what a security group is, do not worry—we will cover and build all the security groups we need in the next section. Before we do that, however, we might as well define listening and routing rules for the load balancer. First, we can define the HTTP listener for port `80`. If you remember the first section of this chapter when getting HTTPS working on our localhost, what do you think we need to do for the HTTP traffic? You don't have to know the specific Terraform code, but what is the general behavior that we want to facilitate? With this in mind, we can achieve that behavior with the following code:

```
resource "aws_lb_listener" "http-listener" {
    load_balancer_arn = aws_lb.application-lb.arn
    port = 80
    protocol = "HTTP"

    default_action {
        type = "redirect"

        redirect {
            port        = "443"
            protocol    = "HTTPS"
            status_code = "HTTP_301"
        }
    }
}
```

More books: https://t.me/booksforits

That's right! We receive HTTPS traffic from port `80` and we then redirect it to port `443` with the HTTPS protocol. We can see that we have attached this listener using the **Amazon Resource Name** (**ARN**) of the load balancer that we created. We can now define our HTTPS listener with the following code:

```
resource "aws_lb_listener" "https-listener" {
    load_balancer_arn = aws_lb.application-lb.arn
    port = 443
    protocol = "HTTPS"
    certificate_arn = data.aws_acm_certificate.
                      issued_certificate.arn
    default_action {
        target_group_arn = aws_lb_target_group.target-
            group.arn
        type = "forward"
    }
}
```

Here, we can see that we accept HTTPS traffic and then forward HTTP traffic to our target group that we defined using the ARN of the target group. We can also see that we have attached one of the certificates to the listener. However, this does not cover all our URL combinations. Remember—we have another certificate that we want to attach. We can attach our second certificate with the following code:

```
resource "aws_lb_listener_certificate" "extra_certificate" {
  listener_arn = "${aws_lb_listener.https-listener.arn}"
  certificate_arn =
      "${data.aws_acm_certificate.raw_cert.arn}"
}
```

This connection should be easy to understand. Here, we merely reference the ARN of the HTTPS listener and the ARN of the certificate that we want to attach. We have now defined everything we need for the resources of the load balancer. However, what of the traffic? We have defined the load balancer, EC2 instances, and the routing for HTTPS for the load balancer. However, what is stopping someone just directly connecting to the EC2 instance, bypassing the load balancer and HTTPS completely? This is where security groups come in. In the next section, we will lock down the traffic by creating security groups so that users cannot bypass our load balancer.

Creating security groups to lock down and secure traffic

Security groups are essentially firewalls. We can define the traffic to and from a resource that has a security group implemented. The rules of the traffic can be fine-grained. A single security group can

More books: https://t.me/booksforits

have multiple rules defining the origin (even specific IP addresses if needed) and the protocol of the traffic. When it comes to our security groups, we are going to need two. One will accept HTTP and HTTPS traffic from all IPs anywhere in the world. This is going to be for our load balancer because we want our application to be available to everyone. The other security group will be implemented by our EC2 instances; this one blocks all HTTP traffic apart from the first security group. We will also enable SSH inbound traffic because we need to SSH into the servers to deploy the applications, giving us the following traffic layout:

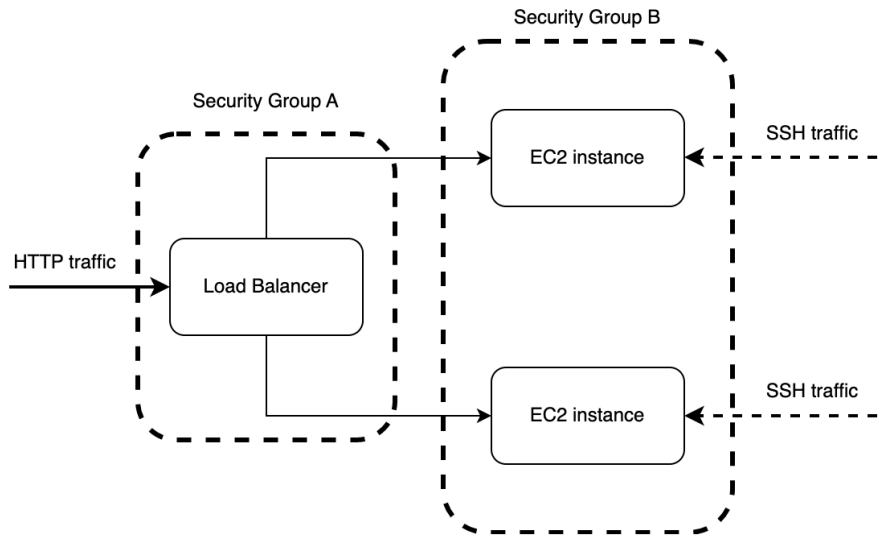


Figure 11.25 – Security groups for our deployment system

This is where you must be careful with online tutorials. There is no shortage of *YouTube* videos and *Medium* articles that will get a load balancer up and running with some pointing and clicking. However, they leave the EC2 instances exposed and do not bother exploring security groups. Even with this section, we are going to leave the database exposed. I am doing this because it is a good question to be asked in the *Questions* section. However, I'm highlighting it here because you need to be warned that it is exposed. The way to lock down the database will be covered in the *Answers* section of this chapter. When it comes to our security groups, we can define them in the `deployment/security_groups.tf` file. We can start with the load balancer security group with the following code:

```
resource "aws_security_group" "alb-security-group" {
    name = "to-do-LB"
    description = "the security group for the
                   application load balancer"

    ingress {
```

More books: https://t.me/booksforits

```
        . . .
    }
    ingress {
        . . .
    }
    egress {
        . . .
    }
    tags = {
        name: "to-do-alb-sg"
    }
}
```

Here, we have two inbound rules under the `ingress` tag and one outbound rule under the `egress` tag. Our first inbound rule is to allow HTTP data from anywhere with the following code:

```
ingress {
    description = "http access"
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
```

The `cidr` blocks of all zeros means *from anywhere*.

Our second inbound rule is HTTPS traffic from anywhere. How do you think this will be defined? It can be defined with the following code:

```
ingress {
    description = "https access"
    from_port = 443
    to_port = 443
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
```

More books: https://t.me/booksforits

Now for our outbound rule. We must allow all traffic and protocols out of the load balancer as they are coming from our resources. This can be achieved with the following code:

```
egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}
```

It must be noted that `from_port` and `to_port` are zero, meaning that we allow outgoing traffic from all ports. We also have set `protocol` to `-1`, meaning we are allowing all protocols as outgoing traffic. We have now defined the security group for the load balancer. We can now move on to defining our security group for the EC2 instances with the following code:

```
resource "aws_security_group" "webserver-security-group" {
    name = "to-do-App"
    description = "the security group for the web server"

    ingress {
        . . .
    }
    ingress {
        . . .
    }
    egress {
        from_port = 0
        to_port = 0
        protocol = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
    tags = {
        name: "to-do-webserver-sg"
    }
}
```

More books: https://t.me/booksforits

The outbound rules are going to be the same as the load balancer because we want to return data to anywhere that is available to request it. When it comes to our HTTP inbound rules, we only want to accept traffic from the load balancer with the following code:

```
ingress {
    description = "http access"
    from_port = 80
    to_port = 80
    protocol = "tcp"
    security_groups = ["${aws_security_group.
                          alb-security-group.id}"]
}
```

Here, we can see that instead of defining the `cidr` blocks, we rely on the security group of the load balancer. Now that all the user traffic is defined, we only need to define the SSH traffic for deployment with the following code:

```
ingress {
    description = "SSH access"
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
```

Here, we access SSH through port `22`. This will enable us to SSH into our servers and deploy our applications. Nearly everything is done. We only must attach our EC2 instances to our EC2 security group, which can be done with the following code:

```
resource "aws_network_interface_sg_attachment"
    "sg_attachment_webserver" {
         count = length(aws_instance.production_server)
             security_group_id = aws_security_group.
                 webserver-security-group.id
  network_interface_id = aws_instance.
                         production_server[count.index].
                         primary_network_interface_id
}
```

More books: https://t.me/booksforits

Our Terraform scripts are now complete and will be able to spin up multiple EC2 instances, a database, and a load balancer while locking down traffic. This is a good template for other projects if you want to get a basic web app with HTTPS and access to a database off the ground with restricted traffic.

Now that we have two different EC2 instances, we are going to have to change our deployment scripts so that both have applications installed in the next section.

Updating our Python deployment script for multiple EC2 instances

There are ways in which we can optimize the deployment process, such as running multiple processes at the same time. This will speed up your deployment the more EC2 instances that we have. However, we must remember that this is a book on Rust and web programming, with some deployment chapters included so that you can use what you create. We could write an entire book on optimizing and improving our deployment pipeline. When it comes to supporting multiple EC2 instances in our `deployment/run_build.py` file, at the end of the file, we merely loop through the list of global IPs from the output with the following code:

```
for server_ip in data["ec2_global_ips"]["value"][0]:
    print("waiting for server to be built")
    time.sleep(5)
    print("attempting to enter server")

    build_process = Popen(f"cd {DIRECTORY_PATH} && sh
        ./run_build.sh
                            {server_ip} {args.u} {args.p}",
                            shell=True)
    build_process.wait()
```

This is it. Multiple servers are now supported. Here, we can see the effectiveness of separating the logic behind managing the data around the deployment in the Python file and the individual Bash script for deploying the applications on the individual server. Keeping things isolated keeps technical debt down, enabling easy refactoring. Now, all our code infrastructure is done! We can run this Python script and deploy our build onto AWS. Everything is nearly done; all we must do is connect our URL to our load balancer in the next section.

Attaching our URL to the load balancer

This is the home run. We are finally near the end. I appreciate you sticking with me on this chapter as it is not as exciting as coding in Rust. However, it is important if you want to use your Rust server. To connect our URL to the load balancer, we must navigate to the hosted zone for your URL. Once

More books: https://t.me/booksforits

there, click on the **Create record** button. When in the **Create record** display, if you are not using the wizard, click on the **Switch to wizard** link at the top right of the **Create record** display to get the following wizard view:

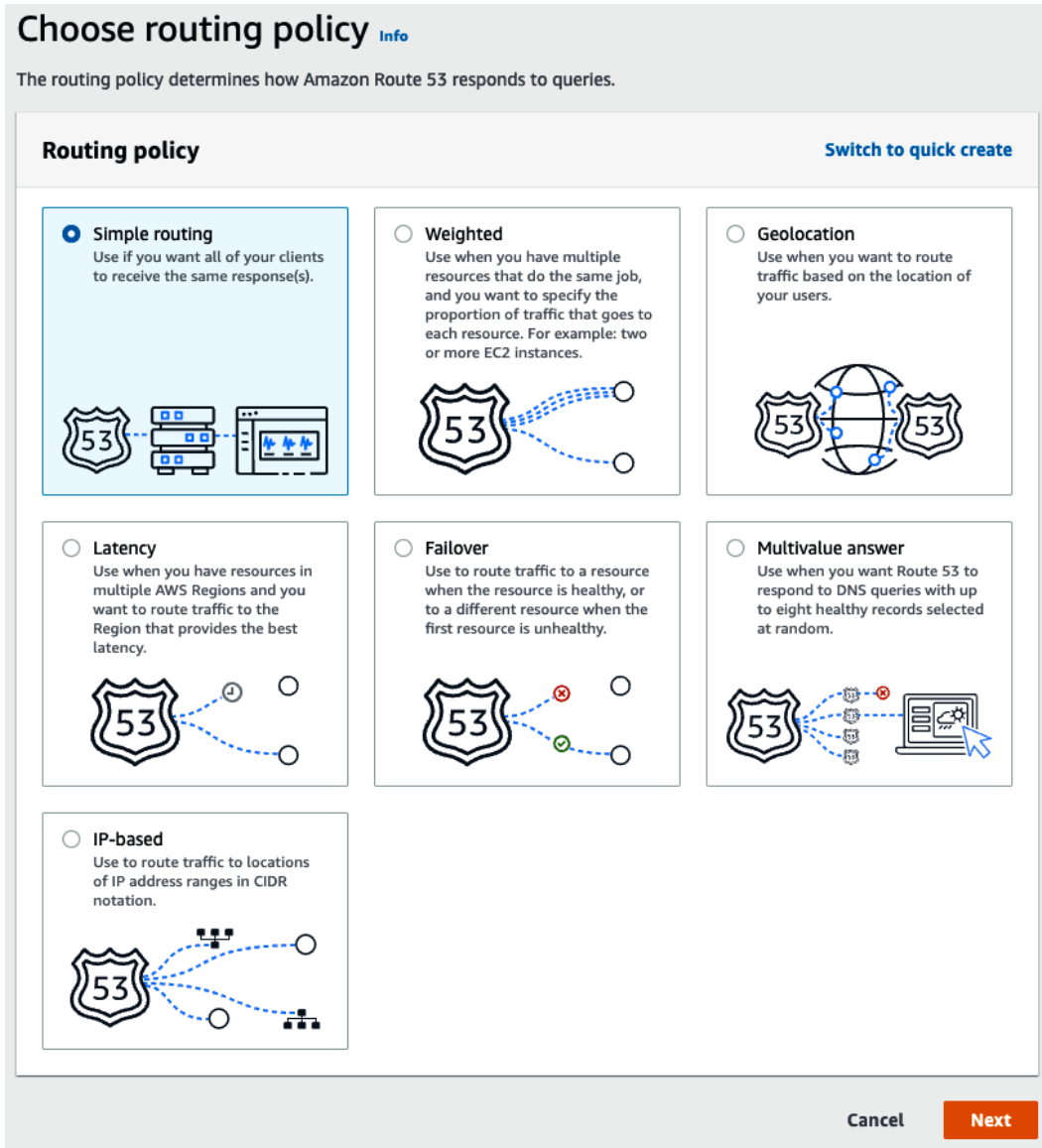


Figure 11.26 – Create record wizard view

More books: https://t.me/booksforits

Here, we can see a range of fancy ways to route traffic. However, we are going to merely select **Simple routing** as we just need to pass traffic to the load balancer, which is doing the distribution of traffic between EC2 instances. Selecting **Simple routing** gives us the following form to fill in:

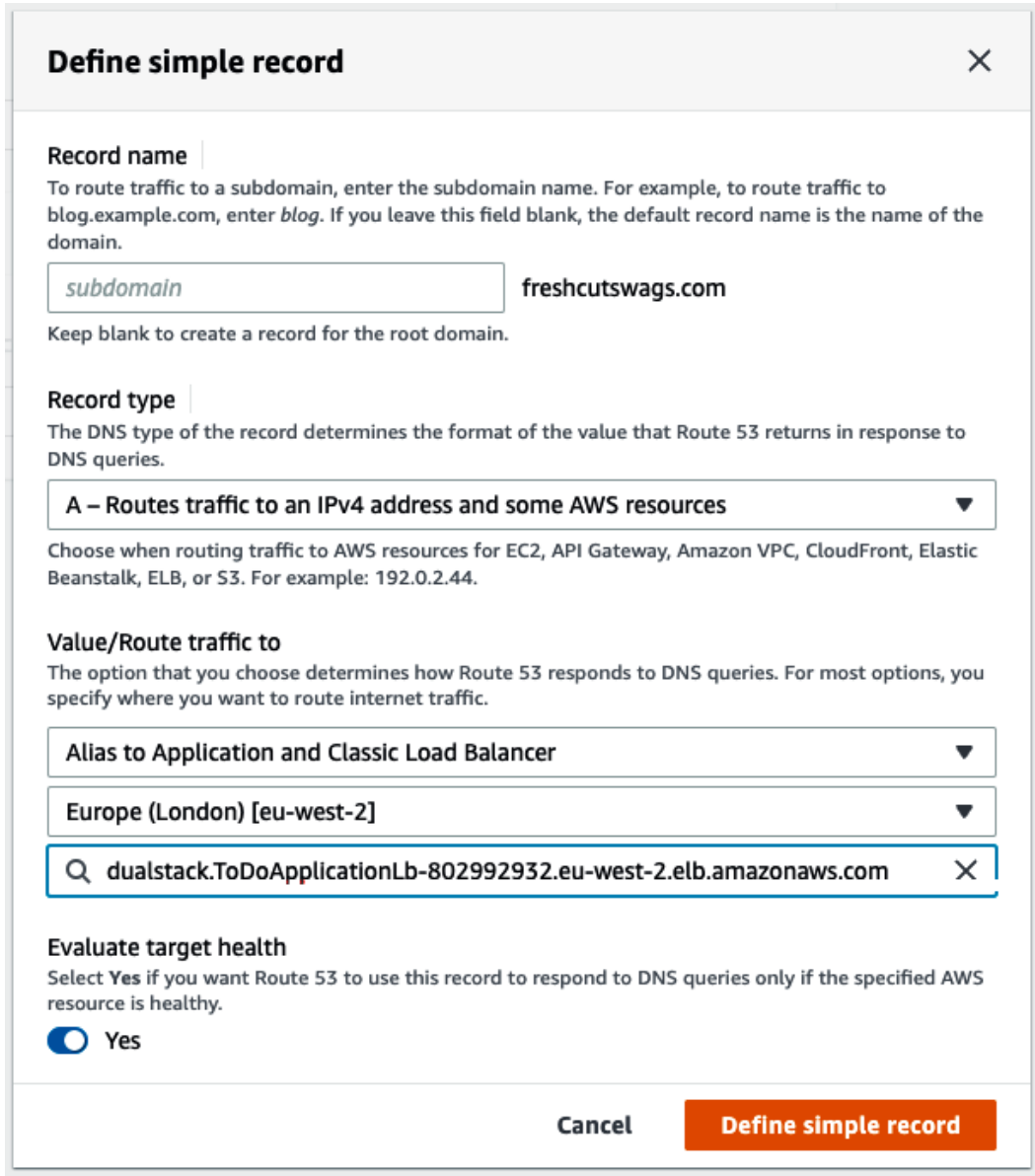


Figure 11.27 – Define simple record wizard view

More books: https://t.me/booksforits

Here, we can see that I have selected **Alias to Application and Classic Load Balancer** to route traffic to. I have then selected the location where the load balancer is. This has given me a dropdown where I can see my `ToDoApplicationLb` load balancer to select. When you click the **Define simple record** button, you are then navigated to a list of records that are to be created. We carry out the creation wizard process one more time to account for all prefixes with the wildcard, and then confirm our creation of records. With this, our HTTPS now works with our application, as seen here:

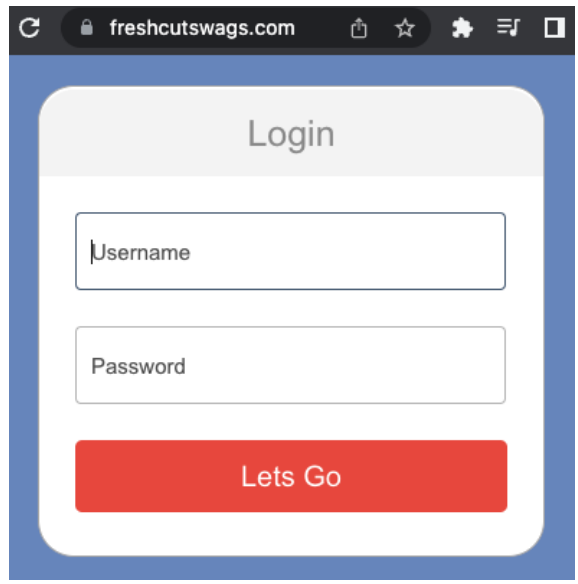


Figure 11.28 – HTTPS working with our application on the internet

With this, our chapter is complete. If you try to access one of our EC2 instances directly through an IP address, you will be blocked. Therefore, we cannot directly access our EC2 instances but can access them through HTTPS through our URL. If you give users any variation of your URL, even an HTTP link to the URL, your users will happily use your application with the HTTPS protocol.

Summary

For now, we have done all we need to do to deploy a robust and secure application on AWS with locked-down traffic and HTTPS enforced. We have covered a lot to get here, and the skillset that you have gained in this chapter can be applied to pretty much any other project you want to deploy on AWS if you can package it in Docker. You now understand the advantages of HTTPS and the steps needed to not only achieve the HTTPS protocol but also to map your URL to the IP address of a server or a load balancer. What is more, we automated the attachment of certificates that we created using Certificate Manager to our load balancer in Terraform using the powerful data query resource that Terraform has to offer. Finally, this all came together when we managed to access our application

More books: https://t.me/booksforits

using HTTPS and only HTTPS. Not only have we developed some practical skills that will become useful in many future projects, but we have also explored the nature of how HTTPS and DNS work, giving us a deeper understanding and appreciation of how the internet generally works when we type in a URL to the browser.

In the next chapter, we will explore the Rocket framework. Due to how we have built our Rust modules in our Actix web application, we will be able to lift modules directly from the Actix web application and slot them into the Rocket application. Considering what we have done in this chapter, we will also be able to wrap our Rocket application in Docker and slot it into the build pipeline here by merely changing one line of code in the deployment `docker-compose` file. In the next chapter, you will see firsthand that when everything is well structured and isolated, changing features and frameworks is not going to be a headache and is, in fact, fairly joyful.

Further reading

- Application load balancers documentation: `https://docs.aws.amazon.com/AmazonECS/latest/developerguide/load-balancer-types.html`
- Security group documentation: `https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html`

Questions

1. Once we remember that our system still has the database exposed to the rest of the world, how would we lock it down?
2. Why should we use an elastic IP in some cases?
3. How could we automate the association of an existing elastic IP to an EC2 instance?
4. How do we utilize security groups to lock down traffic between a URL and EC2 instances?
5. Let's say that the traffic for our application greatly increased and our two instances cannot handle the pressure. What could we do to enable our system to handle the increase?
6. What is the basic premise of a DNS record?
7. What is a URL hosted zone?

More books: https://t.me/booksforits

Answers

1. We would create a security group for the database that only accepts traffic to and from the EC2 instance security group. If we were having to make a migration, we can SSH into an EC2 instance and use it as a proxy to connect to a database. You can also do this with database viewing software such as DataGrip. Sometimes you can have an EC2 instance that is just there for a user to use as a proxy to access the database. This is known as a bastion server.
2. When we destroy and create EC2 instances, the IP address of the EC2 instance will change. We can use an elastic IP to ensure that the IP address remains consistent, which can be helpful for automation pipelines as we can continue to point to that IP address. However, if we are using a load balancer, we do not need to use an elastic IP address as we will point our URL to the load balancer.
3. We can automate the association of an elastic IP to an EC2 instance using Terraform and its powerful data resource. This means we get the data of the existing elastic IP with a data query, and then attach this IP to an EC2 instance that we are creating.
4. Ensure that the security group for the EC2 instances in the target group that the load balancer is attached to can only accept HTTP traffic from the load balancer.
5. Considering that everything in our system is automated and piped into each other, all we must do is increase the count in our EC2 instance definition from 2 to 3. This will increase the number of EC2 instances handling the traffic from 2 to 3.
6. A DNS record is essentially a routing rule that will tell us how to route traffic from the URL to a server IP, URL namespace, or AWS service.
7. A URL hosted zone is a collection of DNS records for a URL.

More books: https://t.me/booksforits

More books: https://t.me/booksforits

Part 5: Making Our Projects Flexible

We now have a fully working application deployed on AWS with HTTPS and a database. However, there are a lot of moving parts to get there. In this part, we transfer the application that we have built over the book into a Rocket application to see how structuring our code well will give us flexibility when choosing a web framework. We then cover practices on how to keep our web application repository clean and flexible with build/deployment pipelines and automated migration Docker containers that fire once to apply database migrations and then die. We also cover multi-stage builds and how to build distroless server Docker images that are roughly 50 MB.

This part includes the following chapters:

- *Chapter 12*, *Recreating Our Application in Rocket*
- *Chapter 13*, *Best Practices for a Clean Web App Repository*

More books: https://t.me/booksforits

More books: https://t.me/booksforits

12
Recreating Our Application in Rocket

At this point, we have built a fully functioning to-do application with the **Actix Web framework**. In this chapter, we will go through the core concepts so that there will be nothing holding us back if we decide to completely recreate the to-do application in **Rocket**. This framework might appeal to some developers because it does not need as much boilerplate code.

In this chapter, we will fully utilize our isolated modular code to completely recreate our application in one chapter by copying over and plugging in our existing modules, views, database connection configuration, and test pipeline. Even if you are not interested in building web applications in Rocket, I still suggest you still complete this chapter because you will get to experience why it is important to perform well-decoupled testing and write well-structured code, as good tests and structure will enable you to switch web frameworks will little effort.

In this chapter, we will cover the following topics:

- What is Rocket?
- Setting up our server
- Plugging in our existing modules
- Returning status with JSON
- Returning multiple statuses
- Registering our views with Rocket
- Plugging in our existing tests

More books: https://t.me/booksforits

By the end of this chapter, you will have a fully working to-do application in Rocket with minimal coding. Not only will you understand the basics of configuring and running a Rocket server but you will also be able to port over modules, views, and tests from other code bases that have used Actix Web and plug them into your Rocket server, and vice versa. Not only is this a valuable skill but it also concretes the need for good-quality, isolated code. You will see firsthand how and why you should structure your code the way we have.

Technical requirements

In this chapter, we'll build on the code built in *Chapter 11*, *Configuring HTTPS with NGINX on AWS*. This can be found at the following URL: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter11/running_https_on_aws`.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter12`.

What is Rocket?

Rocket is a Rust web framework, like Actix Web. It's newer than Actix Web and has a lower user base at the time of writing. In the previous edition of this book, Rocket was running on nightly Rust, meaning that the releases were not stable. However, now, Rocket is running on stable Rust.

The framework does have some advantages, depending on your style of coding. Rocket is simpler to write, since it implements boilerplate code itself, so the developer does not have to write boilerplate themselves. Rocket also supports JSON parsing, forms, and type checking out of the box, which can all be implemented with just a few lines of code. Features such as logging are already implemented as soon as you start a Rocket server. If you want to just get an application off the ground with little effort, then Rocket is a good framework. However, it is not as established as Actix Web, meaning that as you get more advanced, you might find yourself envying some of the features and implementations Actix Web has. However, in all my years of web development, I have never come across a problem that has been severely held back by the choice of framework. It will mainly come down to preference. To really feel the difference, it makes sense to take Rocket for a spin. In the next section, we will create a basic server.

Setting up our server

When it comes to setting up a basic server in Rocket, we are going to start with everything defined in the `main.rs` file. First, start a new Cargo project and then define the Rocket dependency in the `Cargo.toml` file with the following code:

```
[dependencies]
rocket = "0.5.0-rc.2"
```

More books: https://t.me/booksforits

This is all we need for now in terms of dependencies. Now, we can move to our `src/main.rs` file to define the application. Initially, we need to import the Rocket crate and the macros associated with the Rocket crate with the following code:

```
#[macro_use] extern crate rocket;
```

We can now define a basic *hello world* view with the following code:

```
#[get("/")]
fn index() -> &'static str {
    "Hello, world!"
}
```

With the preceding code, we can deduce that the macro before the function defines the method and URL endpoint. The function is the logic that is executed when the call is made to the view, and whatever the function returns is what is returned to the user. To get a feel for how powerful the URL macro is, we can create two more views – one saying `hello` and the other saying `goodbye`:

```
#[get("/hello/<name>/<age>")]
fn hello(name: String, age: u8) -> String {
    format!("Hello, {} year old named {}!", age, name)
}
#[get("/bye/<name>/<age>")]
fn bye(name: String, age: u8) -> String {
    format!("Goodbye, {} year old named {}!", age, name)
}
```

Here, we can see that we can pass parameters from the URL into the function. Again, this code is clear and straightforward. There is nothing left for us to do, apart from attaching these views to the server and starting it with the following code:

```
#[launch]
fn rocket() -> _ {
    rocket::build().mount("/", routes![index, hello, bye])
}
```

Here, we can see that we must decorate the `main` function with a macro from Rocket, and we are attaching the views that we defined with no prefix. We can then run the `cargo run` command to launch the server. Once we have done the `run` command, we get the following output:



```
Configured for debug.
   >> address: 127.0.0.1
```

More books: https://t.me/booksforits

```
    >> port: 8000
    >> workers: 8
    >> ident: Rocket
    >> limits: bytes = 8KiB, data-form = 2MiB, file = 1MiB, form
=
    32KiB, json = 1MiB, msgpack = 1MiB, string = 8KiB
    >> temp dir: /var/folders/l7/
q2pdx7lj0l72s0lsf3kc34fh0000gn/T/
    >> http/2: true
    >> keep-alive: 5s
    >> tls: disabled
    >> shutdown: ctrlc = true, force = true, signals =
[SIGTERM],
    grace = 2s, mercy = 3s
    >> log level: normal
    >> cli colors: true
🚩 Routes:
    >> (index) GET /
    >> (bye) GET /bye/<name>/<age>
    >> (hello) GET /hello/<name>/<age>
📡 Fairings:
    >> Shield (liftoff, response, singleton)
🛡 Shield:
    >> X-Content-Type-Options: nosniff
    >> X-Frame-Options: SAMEORIGIN
    >> Permissions-Policy: interest-cohort=()
🚀 Rocket has launched from http://127.0.0.1:8000
```

Here, we can see that the logging is comprehensive. It defines the port, address, and configuration of the server. It then defines the routes that have been attached, as well as fairings. With the preceding logging, we can see that the server is healthy, and we have the routes that we expected. Here, we can see that logging comes out of the box. We do not have to define anything, unlike in Actix Web. We also get a note stating what views are mounted and the URL that the server is listening on.

More books: https://t.me/booksforits

We can now call our *hello* view in the browser, which gives us the following output:

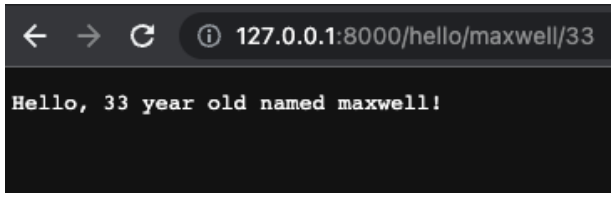


Figure 12.1 – The result of calling our hello view

Calling this view also gives us the following log:

```
GET /hello/maxwell/33 text/html:
   >> Matched: (hello) GET /hello/<name>/<age>
   >> Outcome: Success
   >> Response succeeded.
```

From looking at the logs, it seems that we cannot ask any more of it. We now have a basic server up and running; however, this does not have all the functionality that we had in our previous application, built in Actix Web. Recoding all the features that we had would result in an excessively long chapter. In the next section, we will take advantage of our modular code and slot all our functionality into our Rocket application.

Plugging in our existing modules

Throughout the book, we have been building isolated modules in their own files or directories that only concern themselves with one process. For instance, the database file only focuses on creating and managing database connections. The to-do module only focuses on constructing to-do items, and the JSON serialization module is entirely concerned with serializing data structures to and from JSON. With all this in mind, we will see how easily these modules can be copied into our application and used. Once we have done this, you will get to appreciate firsthand why isolated modules are important.

First, we must define our dependencies in the `Cargo.toml` file with the following code:

```
[dependencies]
rocket = {version = "0.5.0-rc.2", features = ["json"]}
bcrypt = "0.13.0"
serde_json = "1.0.59"
serde_yaml = "0.8.23"
chrono = {version = "0.4.19", features = ["serde"]}
serde = { version = "1.0.136", features = ["derive"] }
uuid = {version = "1.0.0", features = ["serde", "v4"]}
```

More books: https://t.me/booksforits

```
diesel = { version = "1.4.8", features = ["postgres",
                                "chrono", "r2d2"] }
lazy_static = "1.4.0"
```

These are the crates that we have used in our previous modules. We can now copy over our old modules from the Actix Web application in the `web_app` directory to our Rocket application, with the following Bash commands:

```
cp -r ./web_app/src/json_serialization ./rocket_app/src/json_
serialization

cp -r ./web_app/src/to_do ./rocket_app/src/to_do
cp -r ./web_app/src/models ./rocket_app/src/models
cp web_app/src/config.rs rocket_app/src/config.rs
cp web_app/config.yml rocket_app/config.yml
cp web_app/src/schema.rs rocket_app/src/schema.rs
cp ./web_app/src/database.rs ./rocket_app/src/database.rs
cp -r ./web_app/migrations ./rocket_app/migrations
cp ./web_app/docker-compose.yml ./rocket_app/docker-compose.yml
cp ./web_app/.env ./rocket_app/.env
```

Everything is close to working; however, we do have some references to the Actix web framework. These can be deleted by deleting the trait implications. As we can see in the following diagram, isolated modules can be directly referenced, and the advanced integration can be implemented using traits:

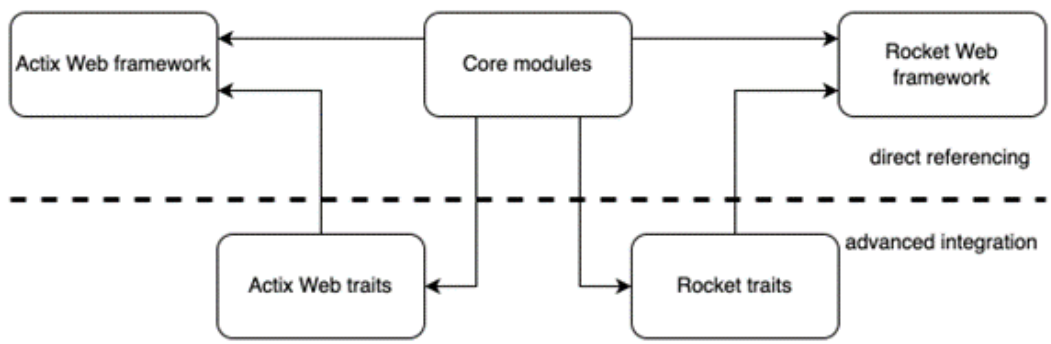


Figure 12.2 – How our modules can interact with different frameworks

More books: https://t.me/booksforits

Once we have deleted the Actix Web trait implementations in the `src/database.rs` and `src/json_serialization/to_do_items.rs` files, we can define and import our modules in the `main.rs` file. The top of the `main.rs` file should look like the following:

```
#[macro_use] extern crate rocket;
#[macro_use] extern crate diesel;
use diesel::prelude::*;

use rocket::serde::json::Json;

mod schema;
mod database;
mod json_serialization;
mod models;
mod to_do;
mod config;

use crate::models::item::item::Item;
use crate::json_serialization::to_do_items::ToDoItems;
use crate::models::item::new_item::NewItem;
use database::DBCONNECTION;
```

With the modules imported, we can recreate the `create` view with the following code:

```
#[post("/create/<title>")]
fn item_create(title: String) -> Json<ToDoItems> {
    let db = DBCONNECTION.db_connection.get().unwrap();
    let items = schema::to_do::table
        .filter(schema::to_do::columns::title.eq(&title.as_
str()))
        .order(schema::to_do::columns::id.asc())
        .load::<Item>(&db)
        .unwrap();

    if items.len() == 0 {
        let new_post = NewItem::new(title, 1);
        let _ = diesel::insert_into(schema::to_do::table)
```

More books: https://t.me/booksforits

```
                        .values(&new_post)
                        .execute(&db);
    }
    return Json(ToDoItems::get_state(1));
}
```

We can see from the preceding code that it is like our Actix Web implementation because we are using our existing modules. The only difference is that we pass the `ToDoItems` struct into the `Json` function from the Rocket crate. We have not implemented authentication yet, so we are just passing the user ID value of `1` for now into all the operations that require a user ID.

Now that our `create` view is done, we can attach it to our server with the following code:

```
#[launch]
fn rocket() -> _ {
    rocket::build().mount("/", routes![index, hello, bye])
                   .mount("/v1/item", routes![item_create])
}
```

We can see that we do not have to build our own configuration functions. We can just line up the views in the array associated with the prefix, and the macro decorating the view functions defines the rest of the URL. We can now run our Rocket server with the following command:

```
cargo run config.yml
```

We must remember to spin up our `docker-compose` so the database is accessible and run migrations on the database using the `diesel` client. We can then create our first to-do item with a `post` request, using the following URL:

```
http://127.0.0.1:8000/v1/item/create/coding
```

After making the `post` request, we will get the following response body:

```
{
    "pending_items": [
        {
            "title": "coding",
            "status": "PENDING"
        }
    ],
    "done_items": [],
```

More books: https://t.me/booksforits

```
    "pending_item_count": 1,
    "done_item_count": 0
}
```

And there you have it! Our application is functioning, and we did not have to recode our entire code base. I know I am repeating myself throughout this book, but the importance of well-structured, isolated code cannot be overstated. What we have done here is useful when refactoring systems. For instance, I have worked on microservices systems where we have had to rip functionality out of one server because the scope of it was getting too big and create another one. As you have seen here, isolated modules make such tasks a dream that can be completed in record time with minimal effort.

Now that we have integrated our existing modules in a basic sense, we can move on to advanced integration by implementing Rocket traits for our modules.

Implementing Rocket traits

Most of the logic that we have defined in our modules that we copied over can be directly referenced in our code. However, we do have to utilize the database connection and the JWT struct that had Actix Web trait implementations. If we are to copy over our views, we are going to have to implement Rocket traits for the database connection and JWT authentication because we pass them into our view functions in the Actix Web application.

Before we implement the Rocket traits, we must copy over the JWT file with the following command:

```
cp web_app/src/jwt.rs rocket_app/src/jwt.rs
```

We then must declare the following dependency in the `Cargo.toml` file with the following code:

```
jsonwebtoken = "8.1.0"
```

We can now move on to the `src/jwt.rs` file for our Rocket trait implementation. First, we must import the following traits and structs at the top of the file with the following code:

```
use rocket::http::Status;
use rocket::request::{self, Outcome, Request, FromRequest};
```

The core logic of the implementation of `FromRequest` will be the same because we are concerned with the decoding and authentication of our token. However, there will be some minor differences because we are implementing a trait from the Rocket framework, as opposed to an Actix Web crate. The main difference is that we must build our own `enum` that defines the possible outcomes with the following code:

```
#[derive(Debug)]
pub enum JwTokenError {
```

More books: https://t.me/booksforits

```
    Missing,
    Invalid,
    Expired
}
```

We have picked the different possibilities here because the token might not be in the header, so it would be missing. Alternatively, the token might not be one of ours, so it could be invalid. And remember, we have a timestamp to force an expiry time. If the token has expired, it will have an expired status.

The next step is merely implementing the `FromRequest` trait. We do not have to touch our `JwToken` struct because the code is isolated and only concerned with the encoding and decoding of tokens. The outline of our trait implementation can be defined with the following code:

```
#[rocket::async_trait]
impl<'r> FromRequest<'r> for JwToken {
    type Error = JwTokenError;

    async fn from_request(req: &'r Request<'_>)
                          -> Outcome<Self, Self::Error> {
        . . .
    }
}
```

Here, we can see that we have decorated the implementation of the trait with an `async` trait macro. This is because requests happen in an async fashion. We also must define lifetime notation. This is because we must declare that the lifetime of the request will be the same as that of the trait implementation. We can see this with the `request` parameter in the `from_request` function. We can now lift the logic from our old Actix Web implementation into our `from_request` function with a few changes in the types we return. The lifted code should end up looking like the following:

```
match req.headers().get_one("token") {

    Some(data) => {
        let raw_token = data.to_string();
        let token_result = JwToken::from_token(raw_token);

        match token_result {
            Ok(token) => {
                return Outcome::Success(token)
            },
```

More books: https://t.me/booksforits

```
            Err(message) => {
                if message == "ExpiredSignature".to_owned() {
                    return
Outcome::Failure((Status::BadRequest,

JwTokenError::Expired))
                }
                return Outcome::Failure((Status::BadRequest,
                    JwTokenError::Invalid))
            }
        }
    },
    None => {
        return Outcome::Failure((Status::BadRequest,
                                 JwTokenError::Missing))
    }
}
```

We can see that we have wrapped our returns in the Rocket `Outcome`, which is not too surprising. We have also included our `enum` when decoding or accessing the token from the header has failed.

Our `JwToken` struct can now be plugged into our Rocket application, but we do have to remember to remove the old Actix implementation and all references to the Actix Web framework. We also must declare our `jwt` module in the `main.rs` file with the following code:

```
mod jwt;
```

Our next step is to implement the `FromRequest` trait for our database connection. At this point, it is a good idea for you to try and implement the `FromRequest` trait for the database connection by yourself. There is nothing new that you must know to achieve this.

If you have tried to implement the `FromRequest` trait for the database connection yourself, then it should be like the following steps.

First, we must import the required Rocket structs and traits in the `src/database.rs` file with the following code:

```
use rocket::http::Status;
use rocket::request::{self, Outcome, Request, FromRequest};
```

More books: https://t.me/booksforits

We then must define the outcomes. We either get the connection or we do not, so there is only one possible error for our `enum`, which takes the following form:

```
#[derive(Debug)]
pub enum DBError {
    Unavailable
}
```

We then implement the `FromRequest` trait for the database connection with the following code:

```
#[rocket::async_trait]
impl<'r> FromRequest<'r> for DB {
    type Error = DBError;

    async fn from_request(_: &'r Request<'_>)
                          -> Outcome<Self, Self::Error> {
      match DBCONNECTION.db_connection.get() {
         Ok(connection) => {
            return Outcome::Success(DB{connection})
         },
         Err(_) => {
            return Outcome::Failure((Status::BadRequest,
                                     DBError::Unavailable))
         }
      }
    }
}
```

The preceding code should not be too much of a surprise; we have merely fused the existing logic of getting a database connection with the implementation of a `FromRequest` trait that was laid out in the `JwToken` implementation.

More books: https://t.me/booksforits

Note

You may have noticed that we have annotated our `FromRequest` implementation with `[rocket::async_trait]`. We use this because, at the time of writing, the stabilization for the `async` feature in Rust does not include support for `async` functions in traits. If we try and implement an `async` function in a trait without the annotation, we will get the following error:

```
trait fns cannot be declared `async`
```

The `[rocket::async_trait]` annotation enables us to define `async` functions in a trait implementation. There are reasons why we cannot simply desugar the `async` function and have the following function signature:

```
async fn from_request(_: &'r Request<'_>)
                      -> Pin<Box<dyn Future<Output
                      = Outcome<Self, Self::Error>>
                      + Send + '_>> {
```

However, it will not work because we cannot return the `impl` trait in trait functions because this is not supported. For in-depth reading on why `async` functions in traits are hard, please visit the following blog post: `https://smallcultfollowing.com/babysteps/blog/2019/10/26/async-fn-in-traits-are-hard/`.

We can now implement our database connection in the `create` view in the `main.rs` file. Again, this is a good opportunity for you to try and implement the database connection using the `FromRequest` trait yourself.

If you have attempted to utilize the Rocket `FromRequest` trait in the `create` view, your code should look as follows:

```
#[post("/create/<title>")]
fn item_create(title: String, db: DB) -> Json<ToDoItems> {
    let items = schema::to_do::table
        .filter(schema::to_do::columns::title.eq(&title.as_
            str()))
        .order(schema::to_do::columns::id.asc())
        .load::<Item>(&db.connection)
        .unwrap();

    if items.len() == 0 {
```

More books: https://t.me/booksforits

```
        let new_post = NewItem::new(title, 1);
        let _ = diesel::insert_into(schema::to_do::table)
            .values(&new_post)
            .execute(&db.connection);
    }
    return Json(ToDoItems::get_state(1));
}
```

If we run our application again and then hit the `create` endpoint, we will see that our implementation works! This is a revelation that our views with a few alterations can be copied and pasted over to our Rocket application from our Actix Web application. In the next section, we will integrate our existing views into our Rocket web application.

Plugging in our existing views

When it comes to our views, they are also isolated, and we can copy our views over to the Rocket application with a few minor changes to recycle the views that we built for our Actix Web application. We can copy the views with the following command:

```
cp -r web_app/src/views rocket_app/src/views
```

With this copy, it goes without saying now that we must go through and scrub the views of any mentions of the Actix Web framework, as we are not using it. Once we have cleaned our views of any mention of Actix Web, we can refactor our existing code so that it works with the Rocket framework. We will start with our `login` view, as this takes in a JSON body and returns JSON in the following subsection.

Accepting and returning JSON

Before we change our view, we need to make sure that we have imported all we need in the `src/views/auth/login.rs` file with the following code:

```
use crate::diesel;
use diesel::prelude::*;
use rocket::serde::json::Json;

use crate::database::DB;
use crate::models::user::user::User;
use crate::json_serialization::{login::Login,
                login_response::LoginResponse};
```

More books: https://t.me/booksforits

```
use crate::schema::users;
use crate::jwt::JwToken;
```

We can see that there is not much that has changed apart from the `Json` struct coming from the Rocket crate. Implementing those Rocket traits really helped us sever a link in our code to the Actix framework and connect to the Rocket framework, without having to change how the structs that are implementing those traits are imported or used. With this in mind, the following outline of our login view should not be a shock:

```
#[post("/login", data = "<credentials>", format = "json")]
pub async fn login<'a>(credentials: Json<Login>, db: DB) ->
                                          Json<LoginResponse> {
    . . .
}
```

We can see that we reference our incoming JSON body and database connection in the same way as we did before in the Actix login view. The major difference is the macro that highlights what the data is and what format that incoming data takes. Inside the `login` view, we have the following logic:

```
let username: String = credentials.username.clone();
let password: String = credentials.password.clone();

let users = users::table
    .filter(users::columns::username.eq(username.as_str()))
    .load::<User>(&db.connection).unwrap();

match users[0].clone().verify(password) {
    true => {
        let user_id = users[0].clone().id;
        let token = JwToken::new(user_id);
        let raw_token = token.encode();
        let body = LoginResponse{token: raw_token.clone()};
        return Json(body)
    },
    false => panic!("unauthorised")
}
```

More books: https://t.me/booksforits

We can see in the code that the only difference is instead of returning multiple different codes, we merely throw an error. This approach is not optimal. In previous builds, the Rocket framework used to implement a straightforward response builder, like in Actix. However, at the time of writing, Rocket has implemented a lot of breaking changes in its recent releases. Standard response builders simply do not work now, and convoluted implementations of traits are needed just to return a response with a code, body, and values in the header. Documentation and examples of this are also limited at the time of writing. Further reading on constructing more advanced responses is supplied in the *Further reading* section.

Now that our `login` view is defined, we can move on to our `logout` view that returns raw HTML.

Returning raw HTML

If you recall, our logout mechanism returns raw HTML that runs JavaScript in a browser to remove our token. When it comes to Rocket, returning raw HTML is simple. In our `src/views/auth/logout.rs` file, the entire code takes the following form:

```
use rocket::response::content::RawHtml;

#[get("/logout")]
pub async fn logout() -> RawHtml<&'static str> {
        return RawHtml("<html>\
                <script>\
                    localStorage.removeItem('user-token'); \
                    window.location.replace(
                        document.location.origin);\
                </script>\
               </html>")
}
```

We can see that it is returning a string, just like in the previous Actix Web view, but this string is wrapped in a `RawHtml` struct. We can now start updating our to-do action views so that our users can manipulate to-do items, as discussed in the next section.

More books: https://t.me/booksforits

Returning status with JSON

So far, we have returned JSON and raw HTML. However, remember that our to-do applications return JSON with different statuses. To explore this concept, we can revisit our `create` view in the `src/views/to_do/create.rs` file, where we must return a created status with a JSON body. First, all our imports are the same as they were before, apart from the status and JSON structs from the Rocket framework with the following code:

```
use rocket::serde::json::Json;
use rocket::response::status::Created;
```

With these imports, we can define the outline of our `create` view function with the following code:

```
#[post("/create/<title>")]
pub async fn create<'a>(token: JwToken, title: String, db: DB)
                                    -> Created<Json<ToDoItems>> {
    . . .
}
```

We can see that our return value is the `Created` struct, containing the `Json` struct, which in turn contains the `ToDoItems` struct. We can also see that our JWT authentication is implemented in a view the same way because, again, we are implementing the Rocket traits. Our database logic is the same as the previous view, as seen with the following code:

```
let items = to_do::table
    .filter(to_do::columns::title.eq(&title.as_str()))
    .order(to_do::columns::id.asc())
    .load::<Item>(&db.connection)
    .unwrap();
if items.len() == 0 {
    let new_post = NewItem::new(title, token.user_id);
    let _ = diesel::insert_into(to_do::table).values(&new_post)
        .execute(&db.connection);
}
```

If the task is already not present in the database, we will insert our new to-do item. Once we have done this, we get the state of our system and return it with the following code:

```
let body = Json(ToDoItems::get_state(token.user_id));
return Created::new("").body(body)
```

More books: https://t.me/booksforits

The empty string is the location. This can be left blank with no consequences. We then attach our body with the `body` function of the status. This is all that is needed to get our `create` view running as we want it.

When it comes to the other views for our to-do tasks, they will all be some variation of what we have done for the `create` view. All to-do views need to take the following steps:

1. Authenticate using JWT.
2. Connect to a database.
3. Take in data from the JSON body and/or user data from the JWT.
4. Make some manipulation of the data in the database (apart from the `GET` view).
5. Return the state of the database for the user.

After seeing what we have done with the `create` view, you should be able to work through all the other views to make them compatible with the Rocket framework. We have covered everything needed to make these changes. Spelling out these changes in the book will lead to needless repetitive steps being carried out, excessively bloating it. These changes are available on the book's GitHub repository.

Once the to-do item views have been carried out, we can move on to the final view that's needed, the creation of a user, where we must return different statuses depending on the outcome.

Returning multiple statuses

When it comes to creating a user, we merely return a created status code or a conflict status code and nothing else. We do not need to return data because the person who has just created the user already knows the user details. In Rocket, we can return multiple different status codes with no body. We can explore this concept in the `src/views/to_do/create.rs` file, but first, we must ensure that we import the following:

```
use crate::diesel;
use diesel::prelude::*;

use rocket::serde::json::Json;
use rocket::http::Status;

use crate::database::DB;
use crate::json_serialization::new_user::NewUserSchema;
use crate::models::user::new_user::NewUser;
use crate::schema::users;
```

More books: https://t.me/booksforits

Now that we have everything we need, we can define the outline of the view with the following code:

```
#[post("/create", data = "<new_user>", format = "json")]
pub async fn create_user(new_user: Json<NewUserSchema>, db: DB)
    -> Status {
    . . .
}
```

Here, we can see that there is nothing new, apart from returning a single `Status` struct. Our database logic takes the following form:

```
let name: String = new_user.name.clone();
let email: String = new_user.email.clone();
let password: String = new_user.password.clone();
let new_user = NewUser::new(name, email, password);
let insert_result = diesel::insert_into(users::table)
            .values(&new_user).execute(&db.connection);
```

And we return a status out of two possible ones with the following code:

```
match insert_result {
    Ok(_) => Status::Created,
    Err(_) => Status::Conflict
}
```

Our views are complete. We can now move on to the next section to register our views with the Rocket application.

Registering our views with Rocket

Before we move on to the `src/main.rs` file, we must ensure that our view functions are available to the `src/main.rs`. This means going through all the `mod.rs` files in each view module and declaring the functions that define these views as public. We can then move on to the `src/main.rs` file and ensure that the following is imported:

```
#[macro_use] extern crate rocket;
#[macro_use] extern crate diesel;

use rocket::http::Header;
```

More books: https://t.me/booksforits

```
use rocket::{Request, Response};
use rocket::fairing::{Fairing, Info, Kind};
```

The `macro_use` declarations should not be a surprise; however, we import the Rocket structs to define our CORS policy. With these crates imported, we now must ensure that the following modules have been declared:

```
mod schema;
mod database;
mod json_serialization;
mod models;
mod to_do;
mod config;
mod jwt;
mod views;
```

These modules should all look familiar to you. We then must import our views with the following code:

```
use views::auth::{login::login, logout::logout};
use views::to_do::{create::create, delete::delete,
                   edit::edit, get::get};
use views::users::create::create_user;
```

We now have everything we need imported. Before declaring our views on the server, we need to define our CORS policy. This is achieved by declaring a struct with no fields. We then implement the `Fairing` trait for this struct, allowing traffic. Fairings essentially define middleware. Further information on fairings is provided in the *Further reading* section. Our CORS policy can be defined with the following code:

```
pub struct CORS;

#[rocket::async_trait]
impl Fairing for CORS {

    fn info(&self) -> Info {
        Info {
            name: "Add CORS headers to responses",
            kind: Kind::Response
        }
    }
```

More books: https://t.me/booksforits

```
    async fn on_response<'r>(&self, _request: &'r Request<'_>,
                                response: &mut Response<'r>) {
        response.set_header(Header::new(
                            "Access-Control-Allow-Origin", "*"));
        response.set_header(Header::new(
                           "Access-Control-Allow-Methods",
                           "POST, GET, PATCH, OPTIONS"));
        response.set_header(Header::new(
                           "Access-Control-Allow-Headers", "*"));
        response.set_header(Header::new(
                           "Access-Control-Allow-Credentials",
                           "true"));
    }
}
```

By this point in the book, we are now familiar with the concept of CORS and how to implement a Rocket trait. The preceding code needs no elaboration.

We now have everything needed to mount our views to the server with the following code:

```
#[launch]
fn rocket() -> _ {
    rocket::build().mount("/", routes![index, hello, bye])
                   .mount("/v1/item/", routes![create, delete,
                                               edit, get])
                   .mount("/v1/auth/", routes![login, logout])
                   .mount("/v1/user/", routes![create_user])
                   .attach(CORS)
                   .manage(CORS)
}
```

Again, there is no explanation needed. You may have noticed that we have started to simply show code with little to no explanation. This is good, as we have become familiar with the building blocks that we are using. Do not worry – we have come to the end of building the main Rocket application, as it will run and do everything we need. We could manually test this. However, this would take time and is error-prone.

Remember, we built our tests in Newman using Postman! In the next section, we will test all our endpoints with a few commands using the existing testing pipeline.

More books: https://t.me/booksforits

Plugging in our existing tests

Because we have used Newman in our testing pipeline, we do not have to worry about high coupling with our choice of web framework. First, we need to copy over our tests in the `scripts` directory with the following command:

```
cp -r web_app/scripts rocket_app/scripts
```

However, before running it, we must add a `GET` method for our `login` view with the following outline:

```
#[get("/login", data = "<credentials>", format = "json")]
pub async fn login_get<'a>(credentials: Json<Login>, db: DB)
                                    -> Json<LoginResponse> {
    // same logic as in the login view
}
```

We then need to import this view into the `src/main.rs` file and declare it in the `auth` mounting for our server. We are now ready to run our full tests with the following command:

```
sh scripts/run_test_pipeline.sh
```

This will run our full pipeline and give the following results:

	executed	failed
iterations	1	0
requests	8	0
test-scripts	16	0
prerequest-scripts	8	0
assertions	64	3
total run duration: 307ms		
total data received: 1.07kB (approx)		
average response time: 20ms [min: 11ms, max: 53ms, s.d.: 13ms]		

Figure 12.3 – Results from our full test pipeline

More books: https://t.me/booksforits

We can see that out of 64 checks, only 3 have failed. If we scroll further down, we can see that the errors occur only because we are returning different response codes for the `create` view, as seen here:

```
  #  failure                  detail

1.  AssertionError           response is ok
                             expected response to have status
                             code 200 but got 201
                             at assertion:0 in test-script
                             inside "1_create"

2.  AssertionError           response is ok
                             expected response to have status
                             code 200 but got 201
                             at assertion:0 in test-script
                             inside "2_create"

3.  AssertionError           response is ok
                             expected response to have status
                             code 200 but got 201
                             at assertion:0 in test-script
                             inside "3_create"
```

Everything else, in terms of the logging in, authentication, migrations, and state of the data in the database between every step, has behaved just like we expected it to.

Summary

In this chapter, we have gone through the main concepts needed to replicate our to-do application. We built and ran a Rocket server. We then defined routes and established a database connection for our server. After that, we explored middleware and built authentication and data processing, using guards for our views. With this, we created a view that utilized everything we have covered in this book.

What we gained here was a deeper appreciation for the modular code that we have built throughout this book. Even though some of the concepts we revisited had not been touched since the start of this book, these modules were isolated, did one thing, and did what their labels proposed. Because of this, they can easily be copied over and utilized in a completely different framework. Our test pipeline also came in handy, instantly confirming that our Rocket application behaves in the same way our Actix Web application does. With this in mind, our Rocket application could be seamlessly integrated into our build and deployment pipelines instead of our Actix Web application.

More books: https://t.me/booksforits

In the next chapter, we will cover the best practices for building a web application, resulting in a clean web application repository. Here, you will not only learn how to structure a web application repository in terms of testing and configuration but also how to package a web application in Docker as a distroless distribution, resulting in tiny Docker images that are roughly 50 MB.

Further reading

- Rocket documentation: `https://rocket.rs/`
- Fairings documentation: `https://rocket.rs/v0.4/guide/fairings/`
- *Rust Web Development with Rocket: A practical guide to starting your journey in Rust web development using the Rocket framework*, *Karuna Murti* (2022), *Packt Publishing*

Questions

1. We have witnessed firsthand the importance of isolating processes such as our test pipeline. Looking at the test pipeline, is there a dependency that could be removed to further decouple the pipeline so that it isn't even dependent on us testing a Rust server?
2. How do we attach and detach all functionality in our modules with frameworks such as Actix and Rocket?
3. How would we deploy our Rocket server on AWS?

Answers

1. Right now, our test pipeline relies on Diesel to do the migrations. We could simply build our own SQL scripts, housed in directories that define our version with a version table in the database. This would completely decouple our testing pipeline from the server that we are testing. If the server has the same endpoints and access to a PostgreSQL database, it can be tested using our pipeline, no matter what language the server is coded in.
2. If the module is simple with a good interface, we can merely copy it over and import it where we want to use it. If the module relies on advanced functionality with the framework, we must delete the trait implementations for the framework and implement traits for the new one.
3. We need to note that our test pipeline ran the Rocket server without any alterations. This is because we are using the same config and using Cargo to build and run the application. We would merely just have to point our build to the Rocket application and copy over our Dockerfile to the Rocket application. Our build process would then build the Rocket application in Docker and deploy it to Docker Hub. Our deployment process would then pull the image from Docker Hub and deploy it. We know our endpoints are the same and behave in the same way, so integration should be painless.

More books: https://t.me/booksforits

13

Best Practices for a Clean Web App Repository

Throughout this book, we have been building out our application piece by piece and adding automation scripts and tools to help us with testing and deploying our application. However, although this path is useful for learning tools and concepts, the structure of our projects in previous chapters has not been optimal for running a project for production.

In this chapter, we will create a new repository, lift our Rust code into that repository, and then structure the code for clean database migrations, tests, and optimized Docker builds for our application so that it can be deployed smoothly.

In this chapter, we will cover the following topics:

- The general layout of a clean repository
- Getting our configuration from environment variables
- Setting up a local development database
- Managing variables in Postman tests
- Building distroless tiny server Docker images
- Building a clean test pipeline
- Building continuous integration with GitHub Actions

By the end of this chapter, you will be able to structure a repository with scripts, Docker builds, and tests that will make development smooth and easy to add new features. You will also be able to build **distroless** Docker images for the application, making them secure and dropping the size of our server images from 1.5 GB to 45 MB!

More books: https://t.me/booksforits

Technical requirements

In this chapter, we will be referencing parts of the code defined in *Chapter 9*, *Testing Our Application Endpoints and Components*. This can be found at the following URL: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter09`.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter13`.

The general layout of a clean repository

When it comes to a clean layout, we must have directories in the repository that have a single focus, just like our isolated code, which is modularized. In the clean approach taken in this chapter, our repository will have the following layout:

```
├── Cargo.toml
├── README.md
├── .dockerignore
├── .gitignore
├── .github
│   . . .
├── builds
│   . . .
├── database
│   . . .
├── docker-compose.yml
├── scripts
│   . . .
├── src
│   . . .
└── tests
    . . .
```

These files and directories have the following responsibilities:

- `Cargo.toml`: Defines the requirements for the Rust build.
- `README.md`: Renders on a GitHub page when visited, telling the reader what the project is about and how to interact with the project.
- `.dockerignore`: Tells the Docker build what to ignore when copying directories and files into the Docker image.

More books: https://t.me/booksforits

- `.gitignore`: Tells git what to ignore when committing code to the git repository.
- `.github`: A directory that houses GitHub Actions workflows.
- `builds`: A directory that houses different builds for Docker, depending on the chip architecture.
- `database`: A directory that houses all scripts and Docker builds required to handle database migrations.
- `docker-compose.yml`: Defines the containers needed to run a development build.
- `scripts`: A directory that houses all the Bash scripts needed to run dev servers or tests.
- `src`: A directory that houses all the Rust code to build the server.
- `tests`: A directory that houses a `docker-compose` configuration and a Postman collection to enable a fully integrated test. We must remember that unit tests are coded within the `src` directory and are conditionally compiled when the `test` command in Cargo is executed. In standard and release builds, the unit tests are excluded.

Now that we know what our repository structure is like, we can add some rules and files to ensure that our builds and git commits behave in exactly the right way. It is good to do this at the very start of a project to avoid accidentally adding unwanted code to the git history or Docker builds.

First, we will start with the `.gitignore` file, which has the following rules defined:

```
/target/

Cargo.lock

# These are backup files generated by rustfmt
**/*.rs.bk

# jetbrains
.idea

# mac
.DS_Store
```

Here, we can see that we avoid anything in the `target` directory, which gets filled up with a lot of files when performing Rust builds and tests. These files add nothing to the project's development and will balloon the size of your project very quickly. If you like to use JetBrains or are using a Mac, I have added `.idea` and `.DS_Store` as these files can sneak into repositories; they are not required for running any of the web application's code.

More books: https://t.me/booksforits

Now, let's look at our `.dockerignore` file, which has the following rules:

```
./tests
./target
./scripts
./database
.github
```

These rules should make sense. We do not want to add our build files, scripts, database migrations, or GitHub workflows to our Docker builds.

We have now defined all our rules for the repository. Before we move on to the next section, we might as well define the general layout of our application. Here, we can lift the source Rust code from the existing to-do application into our new repository with the following command:

```
cp -r web_app/src ./clean_web_app/src
```

If you created an `src` directory in the clean app repository before running the preceding command, you must delete the `src` directory in the clean app repository; otherwise, you will end up with two `src` directories, where the copied `src` is inside the existing `src`. Our `Cargo.tml` file has the same dependencies as our existing web application; however, we can change its name with the following code:

```
[package]
name = "clean_app"
version = "0.1.0"
edition = "2021"
```

Let's check if lifting our code works with the following `test` command:

```
cargo test
```

This should give us the following output:

```
running 9 tests
test to_do::structs::base::base_tests::new ... ok
test to_do::structs::done::done_tests::new ... ok
test to_do::structs::pending::pending_tests::new ... ok
test jwt::jwt_tests::get_key ... ok
test jwt::jwt_tests::decode_incorrect_token ... ok
test jwt::jwt_tests::encode_decode ... ok
test jwt::jwt_tests::test_no_token_request ... ok
```

More books: https://t.me/booksforits

```
test jwt::jwt_tests::test_false_token_request ... ok
test jwt::jwt_tests::test_passing_token_request ... ok
```

This output shows that our code compiles and that our `Cargo.tml` file is properly defined. Now that we have confirmed that our unit tests have passed, we have some assurance that our code is working. However, how we define our configuration will give us some hurdles when we are deploying applications to the cloud. In the next section, we will smooth out our deployments by using environment variables to configure our web applications.

Getting our configuration from environment variables

So far, we have been loading configuration variables from YML files. This has a few issues. First, we must move these files around where our deployment is. Also, files do not work efficiently with orchestration tools such as Kubernetes. Kubernetes uses ConfigMaps, which essentially define environment variables for each container they are running. Environment variables also work well with tools such as Secret Manager and AWS credentials. We can also directly overwrite the environment variables in `docker-compose`. With all these advantages in mind, we will switch our configuration values from files to environment variables. To map where we have implemented configuration variables from a file, all we must do is delete our `src/config.rs` file and the module declaration of that `config` module in the `main.rs` file. Then, we can run the `cargo test` command again to get the following output:

```
--> src/database.rs:12:12
   |
12 | use crate::config::Config;
   |            ^^^^^^ could not find `config` in the crate
root

error[E0432]: unresolved import `crate::config`
 --> src/jwt.rs:9:12
  |
9 | use crate::config::Config;
  |            ^^^^^^ could not find `config` in the crate root

error[E0432]: unresolved import `crate::config`
 --> src/counter.rs:4:12
  |
4 | use crate::config::Config;
  |            ^^^^^^ could not find `config` in the crate root
```

More books: https://t.me/booksforits

Here, we used `config` in the `jwt`, `database`, and `counter` modules. This makes sense because we must connect to external structures when using these modules. To fix the breaking imports, all we must do is replace the config references with environment variable references. To demonstrate this, we can use the `src/counter.rs` file. First, we must delete the following lines of code:

```
...
use crate::config::Config;
...

let config = Config::new();
let redis_url = config.map.get("REDIS_URL")
                          .unwrap().as_str()
                          .unwrap().to_owned();
...
```

Then, we must replace the preceding lines of code with the following code:

```
...
use std::env;
...

let redis_url = env::var("REDIS_URL").unwrap();
...
```

We can follow this format for the `JWT` and `database` modules as well. In the `JWT` module, there is one variable that is not a string and has to be converted into an integer, which is `expire minutes`. This can be done with the following line of code:

```
let minutes = env::var("EXPIRE_MINUTES").unwrap()
                                        .parse::<i64>()
                                        .unwrap();
```

If we run the `cargo test` command now, we will get the following output:

```
running 9 tests
test jwt::jwt_tests::encode_decode ... FAILED
test jwt::jwt_tests::get_key ... FAILED
test jwt::jwt_tests::decode_incorrect_token ... FAILED
test to_do::structs::pending::pending_tests::new ... ok
test to_do::structs::base::base_tests::new ... ok
```

More books: https://t.me/booksforits

```
test to_do::structs::done::done_tests::new ... ok
test jwt::jwt_tests::test_passing_token_request ... FAILED
test jwt::jwt_tests::test_no_token_request ... ok
test jwt::jwt_tests::test_false_token_request ... FAILED
```

Our tests ran, so we know that compiling the code worked. However, some of the JWT tests are failing. If we scroll further down the log, we will see the following error:

```
---- jwt::jwt_tests::encode_decode stdout ----
thread 'jwt::jwt_tests::encode_decode' panicked at
'called `Result::unwrap()` on an `Err` value: NotPresent',
src/jwt.rs:52:50
```

This is telling us that our environment variables are not present. Considering that this is failing in the `JWT` module, we can be sure that it will also fail in the `database` and `counter` modules. Therefore, before we run or test our application, we need to define these environment variables. We can build a test pipeline for our application with environment variables by building a `scripts/run_unit_tests.sh` script with the following code:

```
#!/usr/bin/env bash

# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
cd ..

export SECRET_KEY="secret"
export EXPIRE_MINUTES=60
cargo test
```

Here, we navigate to the root directory, export the environment variables, and then run the `test` command. Running the preceding script results in all the unit tests passing.

More books: https://t.me/booksforits

Note

While I like putting as much as possible in the Bash script as it acts like documentation for other developers to see all the moving parts, there are other approaches. For instance, you may find that the approach outlined previously is clunky as this approach deviates from running the standard `cargo test` command. Other approaches include the following:

- Manually injecting variables into the test

- Using the `dotenv` crate to load environment variables from a file (`https://github.com/dotenv-rs/dotenv`)

- Having sensible defaults for environment variables

How would you create the script that runs the dev server? This would be a good time for you to try and write the script yourself. If you have attempted writing the script yourself, your `scripts/run_dev_server.sh` script should look something like the following code:

```
#!/usr/bin/env bash

# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
cd ..

export SECRET_KEY="secret"
export EXPIRE_MINUTES=60
export DB_URL="postgres://username:password@localhost:5433/
to_do"
export REDIS_URL="redis://127.0.0.1/"
cargo run
```

However, if we try and run the preceding script, it will crash because we cannot connect to the Redis database. We need to define our dev services in the `docker-compose.yml` file with the following code:

```
version: "3.7"

services:
  postgres:
    container_name: 'to-do-postgres'
    image: 'postgres:11.2'
    restart: always
```

More books: https://t.me/booksforits

```
    ports:
      - '5433:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'
      - 'POSTGRES_PASSWORD=password'
  redis:
      container_name: 'to-do-redis'
      image: 'redis:5.0.5'
      ports:
        - '6379:6379'
```

Now that our dev services have been defined, we can spin up our `docker-compose` and run our `run_dev_server.sh` script, resulting in our dev server running. However, if we try and perform any requests, the server will crash. This is because we have not performed migrations on the database. In the next section, we will perform migrations on our dev database.

Setting up a local development database

When it comes to migrations, there is the advantage of decoupling the programming language that we are using from the migrations. In the past, I have had to switch servers from one language to another and simply wished that the migration's implementation was not coupled with the language. This is also a deployment issue. For instance, in Kubernetes, deploying a new server or an update might require a migration to be run. Ideally, you want to run the migration automatically through what we call *init Pods*. This is a container that is spun up and executed before the main server is deployed. This init Pod can perform a database migration command. However, if the init Pod requires something such as Rust to be present to execute the migration, this can greatly increase the size of the init pod. Therefore, I built an open source Bash tool that is only dependent on the `psql` and `wget` libraries. It can create new migrations and roll the database up and down versions. However, it must be stressed that this tool is not for every use. To quote the documentation of the migrations tool I wrote (`https://github.com/yellow-bird-consult/build_tools/tree/develop#use-yb-database-migrations-if-you-have-the-following`), you should choose to use the migrations tool for projects if you have the following:

- **A light throughput of migrations**: Migrations are not timestamped; they are simply numbered. The design of the tool is simple to keep track of what's going on. Light applications in microservices are an ideal environment.
- **Well-tested code**: There are no guardrails. If there is an error in part of your SQL script, your database will be scarred with a partly run migration. You should have testing regimes with Docker databases before implementing migrations on a live production database.

More books: https://t.me/booksforits

- **You plan on writing your own SQL**: Because this tool is completely decoupled from any programming language, you have to write your own SQL scripts for each migration. This is not as daunting as you might think and gives you more control.
- **You want complete control**: SQL migrations and the simple implementation are essentially defined in a single Bash script. This simple implementation gives you 100% control. Nothing is stopping you from opening up your database in a GUI and directly altering the version number or manually running particular sections of the migration.

Now that we know what we are getting ourselves in for, we can navigate to the `database` directory and install the migration tool with the following command:

```
wget -O - https://raw.githubusercontent.com/yellow-bird-consult
/build_tools/develop/scripts/install.sh | bash
```

This installs a couple of Bash scripts in your home directory. You may have to refresh your terminal to get the command alias. Not all operating systems will support the command alias. If your command alias does work, we can create a new set of migrations by using the following command:

```
yb db init
```

However, if the alias does not work, you can run all your commands through the Bash script as each Bash script is 100% self-contained. All we must do is pass the same arguments in with the following command:

```
bash ~/yb_tools/database.sh db init
```

With the `init` command, we get the following structure:

```
├── database_management
│   └── 1
│       ├── down.sql
│       └── up.sql
```

This is the same as the Diesel migrations tool but with just plain numbers. We have two migrations from our to-do application, so we can create them with the following commands:

```
cp -r database_management/1 database_management/2
```

Once we have done this, we can create our migration files. The `database_management/1/up.sql` file creates the `to_do` table with the following code:

```
CREATE TABLE to_do (
  id SERIAL PRIMARY KEY,
```

More books: https://t.me/booksforits

```
  title VARCHAR NOT NULL,
  status VARCHAR NOT NULL,
  date timestamp NOT NULL DEFAULT NOW()
)
```

The `database_management/1/down.sql` file drops the `to_do` table with the following code:

```
DROP TABLE to_do
```

The `database_management/2/up.sql` file creates the `user` table and links all existing items to a placeholder user with the following code:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR NOT NULL,
    email VARCHAR NOT NULL,
    password VARCHAR NOT NULL,
    unique_id VARCHAR NOT NULL,
    UNIQUE (email),
    UNIQUE (username)
);

INSERT INTO users (username, email, password, unique_id)
VALUES ('placeholder', 'placeholder email',
'placeholder password', 'placeholder unique id');

ALTER TABLE to_do ADD user_id integer default 1
CONSTRAINT user_id REFERENCES users NOT NULL;
```

The `database_management/2/down.sql` file drops the `users` table with the following code:

```
ALTER TABLE to_do DROP COLUMN user_id;
DROP TABLE users
```

More books: https://t.me/booksforits

Our migrations are now ready. However, we need to connect to our database to get information and perform migrations. We can spin up our `docker-compose` to get the dev database up and running. Once this is done, we must define our database URL in the environment variables. The migration tool looks for the URL in environment variables. However, if there is a `.env` file in the current working directory, the migration tool will also load all the variables in this file. In our `database_management/.env` file, we can define the database URL with the following code:

```
DB_URL="postgres://username:password@localhost:5433/to_do"
```

Now that our database is running and we have our URL defined, we can get what migration level the database is currently at with the following command:

```
# with alias
yb db get
# without alias
bash ~/yb_tools/database.sh db get
```

Right now, we should get a `-1`. This means that there is no migrations versioning table at all on the database. If there is, but no migrations have been performed on the database, the version will be `0`. If there are any migrations, then the response will be the migration number that it is currently at. We can use the following `db` commands when using the build tool to perform commands on the database:

- `set`: Creates a migrations version table if there is not one
- `up`: Goes up one migration version by applying the `up.sql` script
- `down`: Goes down one migration version by applying the `down.sql` script
- `new`: Creates a new migration folder if you are on the latest version
- `rollup`: Creates a new migrations version table if there is not one and then loops up all the versions in the `database_management` directory, starting from the current version of the database

We will run the `rollup` command with the following command:

```
# with alias
yb db rollup
# without alias
bash ~/yb_tools/database.sh db rollup
```

This will perform migrations on the database. If you run the `get` command, you will see that the version of the database is now `2`. Our database is now ready to be queried by our application.

More books: https://t.me/booksforits

> **Note**
>
> Migrations can also be achieved using the `sqlx-cli` crate, which can be found at the following link: `https://crates.io/crates/sqlx-cli`.
>
> However, Cargo is needed to install `sqlx-cli`, which will complicate the creation of init Pods for executing these migrations.

Instead of randomly making requests, in the next section, we will refine our Postman tests so that we can run a series of requests and check that our application runs in the way that we want it to.

Managing variables in Postman tests

In *Chapter 9*, *Testing Our Application Endpoints and Components*, we built a Postman collection. However, it was a bit ropey as we had to rely on Python to load the new token into the Newman collection. While this was important to use as using Python as glue code between processes is a useful skill, our old version of readying a Newman collection with Python is not the cleanest approach. At the start of our collection, we will add two new requests. The first one will create a user with the following parameters:

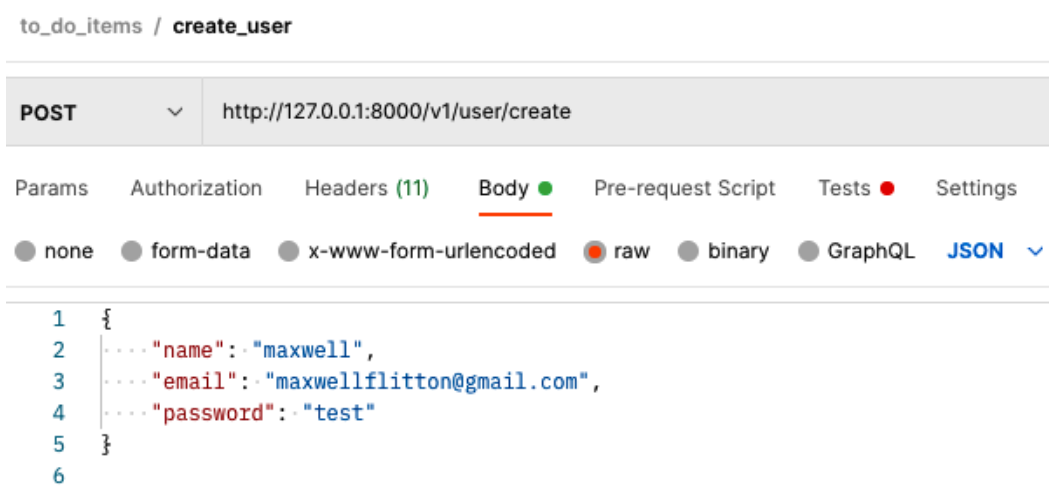


Figure 13.1 – Create user Postman request

With the create user request, we get the following JavaScript in the **Tests** tab in Postman:

```
pm.test("response is created", function () {
    pm.response.to.have.status(201);
});
```

More books: https://t.me/booksforits

With this, the first request of our collection will create the user and throw an error if the request was not successful. Then, we can create the second request for our collection, which consists of logging in, with the following parameters:

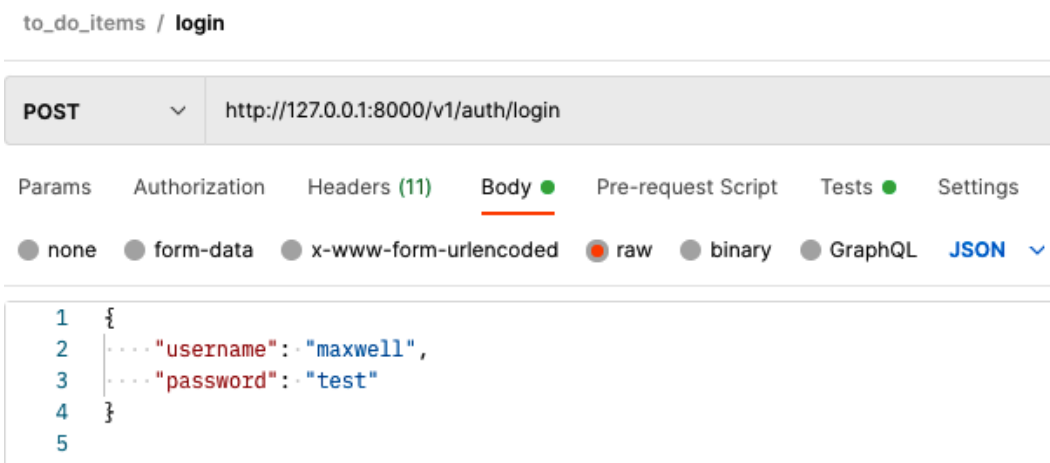


Figure 13.2 – Login Postman request

With this request, we must check the response and set a collection variable as the token that we just got from the login by running the following JavaScript in the **Tests** tab in Postman:

```
var result = pm.response.json()

pm.test("response is ok", function () {
    pm.response.to.have.status(200);
});

pm.test("response returns token", function () {
    pm.collectionVariables.set("login_token", result["token"]);
})
```

Once we have set our collection variable, we will be able to reference our token throughout the rest of the collection. To do this, we must update the authorization for the entire collection so that our new token value will propagate through all our requests. To access the authorization settings, click on the header of a `create` request to get the following:

More books: https://t.me/booksforits

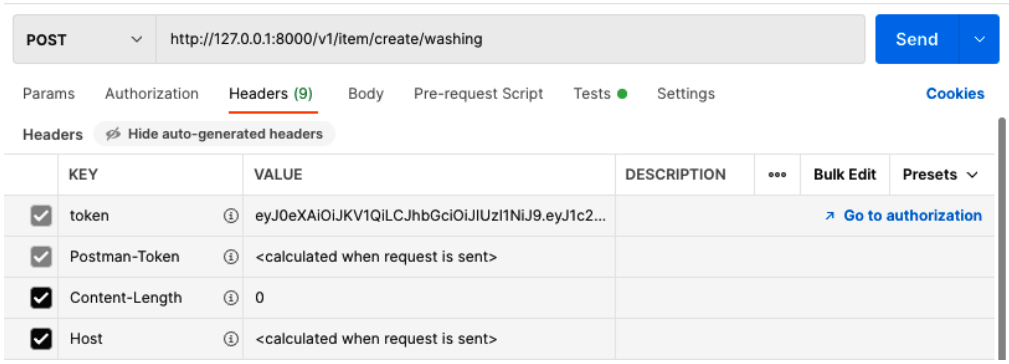


Figure 13.3 – Header of the request

On the right-hand side of the previous screenshot, we can see that there is a **Go to authorization** button. If we click on this, we get the following:

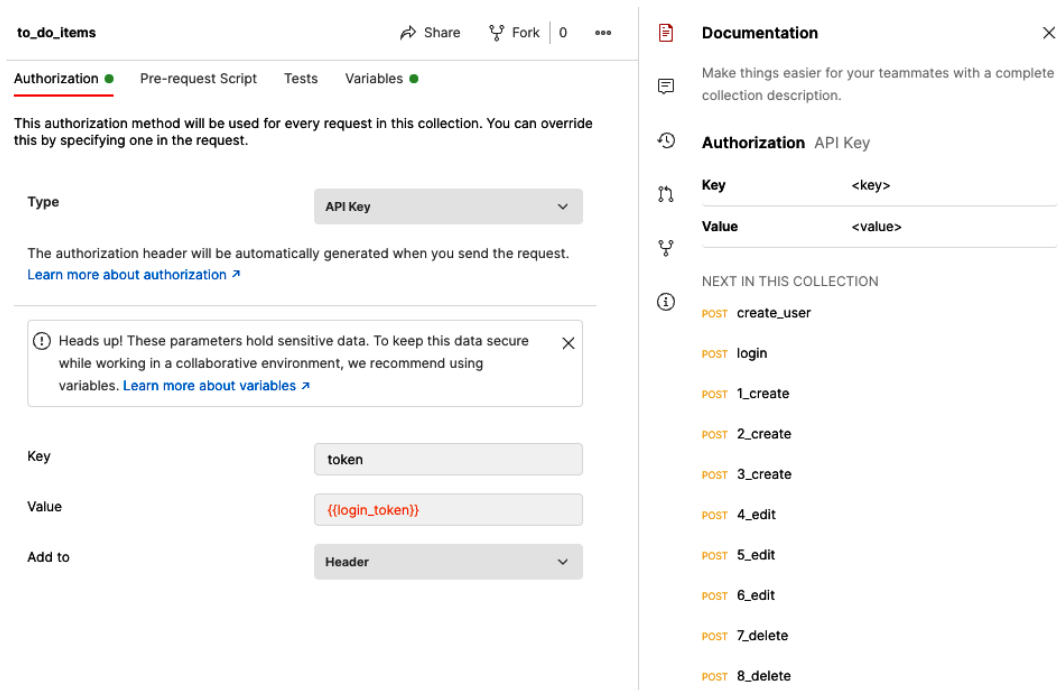


Figure 13.4 – Configuring authorization

We can see that the value has been changed to `{{login_token}}`. If we save this and then export the collection JSON file to the `tests` directory in our repository, the value that belongs to `{{login_token}}` will be inserted into the collection JSON file.

More books: https://t.me/booksforits

We now have a Postman collection that updates itself with a fresh token after the login request without having to rely on Python to glue processes together. This is much cleaner; however, we want to ensure that the rest of our testing pipeline mimics as much of a production setting as possible. In the next section, we will build Docker images that contain our application that are a fraction of the size of what our server images were in previous chapters.

Building distroless tiny server Docker images

In previous chapters, our server Docker images were roughly around 1.5 GB. This is pretty big and not ideal when we want to distribute our Rust images on servers or to other developers. Note that there is a shell that we can access in the Docker container when the image is running. This is useful in development but not great in production because if anyone manages to gain access to the Docker container, they will be able to look around and run commands in the Docker container. If the permissions on the server are not locked down, the hacker could even start running commands on the cluster that you have. I have seen cryptojacking happen through this method, where a hacker spun up a load of mining Pods at the expense of the owner of the AWS account.

We are going to solve these problems by using distroless images. These distroless images are tiny in size and do not have shells. So, if someone manages to gain access to our server, they will not be able to do anything because there are no shells. We will be able to drop the size of our image from 1.5 GB to 45 MB! This is something we want. However, before we start building our distroless images, we must know that distroless images have close to nothing on them. This means that if we compile our application and stuff it into a distroless image, it will not work. For instance, if we make a connection to a database, we need the `libpq` library in our distroless image. As the distroless image does not contain the library, the image will not be able to run because our static binary will not be able to locate the `libpq` library.

We know that our 1.5 GB image runs because it contains everything and the kitchen sink. We can use our 1.5 GB to inspect what dependencies the static binary has in the image. We can do this by moving to our `deployment` directory where we wrote code to deploy our application on AWS and spinning up `docker-compose` there. Once this is running, we can inspect our containers with the following command:

```
docker container ls
```

This will give us the following output:

```
CONTAINER ID   IMAGE                  . . .
0ae94ab0bbc5   nginx:latest           . . .
6b49526250e3   deployment_rust_app.   . . .
9f4dcdc8a455   redis:5.0.5            . . .
```

More books: https://t.me/booksforits

Your IDs will be different, but we will use these IDs to SSH into our Rust app by using the following command:

```
docker exec -it 6b49526250e3 /bin/bash
```

This opens an interactive shell so that we can navigate the Docker container. Here, we must remember that the static binary – that is, the Rust server – is called `web_app` and that this is in the root directory, so we do not need to go anywhere within the container. We can list the dependencies by using the following command:

```
ldd web_app
```

This will give us the following output:

```
linux-vdso.so.1 (0x0000ffffb8a9d000)
libpq.so.5 => /usr/lib/aarch64-linux-gnu/libpq.so.5
libgcc_s.so.1 => /lib/aarch64-linux-gnu/libgcc_s.so.1
libpthread.so.0 => /lib/aarch64-linux-gnu/libpthread.so.0
libm.so.6 => /lib/aarch64-linux-gnu/libm.so.6
. . .
```

There are 29 dependencies in total. On the left of the list, there is the name of the library. On the right of the list, there is the path to where the library is. We can see that the database `libpq` library is needed alongside other libraries. Your paths may look different. This is because I am running this image on a MacBook M1, which has an ARM chip architecture. If you do not have this, then you will have `x86_64-linux-gnu` in your path as opposed to `aarch64-linux-gnu`. This is fine – we will supply both Docker files in the GitHub repository online.

In our Docker build, we must copy these libraries into our distroless image. In our `clean_web_app/builds` directory, we must create two files: `aarch64_build` and `x86_64_build`. Both these files are essentially the same Dockerfiles but with different references to libraries. At the time of writing, I wish that there was a smarter way to achieve builds with different chips in one Docker file; however, Docker builds are terrible at passing variables throughout the build as each step is isolated, and conditional logic is limited at best. It is easier to just have two different files. Also, if the builds change in the future, then the two different chip builds are decoupled. In our `clean_web_app/builds/arch_build` file, we must get the Rust image, install the database library, copy over the code of the application to be compiled, and define what type of build we are doing:

```
FROM rust:1.62.1 as build

RUN apt-get update
RUN apt-get install libpq5 -y
```

More books: https://t.me/booksforits

```
WORKDIR /app
COPY . .

ARG ENV="PRODUCTION"
RUN echo "$ENV"
```

We can see that the environment is set to `"PRODUCTION"` by default. If there is an accident and the environment is not defined, it should be `"PRODUCTION"` by default. Accidentally taking longer to compile on a test build is much better than accidentally deploying a non-production server into production. Then, we compile using the release flag if it is production and switch the static binary into the release directory if it is not compiled using the release flag:

```
RUN if [ "$ENV" = "PRODUCTION" ] ; then cargo build --release ;
\
else cargo build ; fi
RUN if [ "$ENV" = "PRODUCTION" ] ; then echo "no need to copy"
; \
else mkdir /app/target/release/ && cp /app/target/debug/clean_
app \
/app/target/release/clean_app ; fi
```

At this point, our application has been compiled. Everything we have covered is independent of what type of chip we are using, so the `x86_64_build` file will contain the same code that we have just laid out in the `aarch64_build` file. For both build files, we can also get our distroless image with the following code:

```
FROM gcr.io/distroless/cc-debian10
```

Now, this is where the build scripts differ. In the ARM chip build, we must copy the libraries needed from the previous Rust image into our distroless image, like so:

```
COPY --chown=1001:1001 --from=build \
/usr/lib/aarch64-linux-gnu/libpq.so.5 \
/lib/aarch64-linux-gnu/libpq.so.5
. . .
COPY --chown=1001:1001 --from=build \
/lib/aarch64-linux-gnu/libcom_err.so.2 \
/lib/aarch64-linux-gnu/libcom_err.so.2
```

More books: https://t.me/booksforits

Including them all would simply provide needless bloat for the book and again, these files are available in this book's GitHub repository. What we must note, however, is that the directory for the first part of each copy is the directory listed when we explored the Docker image of our large application. The second part is the same path; however, if there is a `/usr/lib/` at the start of the path, it is shortened to `/lib/`. There is no shell or users in the distroless image.

Once all the libraries have been copied over, we must copy the static binary of our web application into the root of our image, expose the port, and define the entry point, which is the static binary, with the following code:

```
COPY --from=build /app/target/release/clean_app \
/usr/local/bin/clean_app

EXPOSE 8000
ENTRYPOINT ["clean_app"]
```

With this, our distroless image is done. Right now, both builds are stored away, and we will get them out depending on the chip type in a bash script to be built.

> **Note**
> We don't have to build our distroless application manually. Instead, we can use Apko via the following link: `https://github.com/chainguard-dev/apko`.

You can copy your chosen build to the root directory of the repository under the `Dockerfile` filename. Then, run the following command:

```
docker build . -t clean_app
```

When you list your Docker images, you will see that this image is 46.5 MB! This is a massive reduction from 1.5 GB. In the next section, we will include these build files in a test pipeline.

Building a clean test pipeline

When it comes to testing our application, we want to package it in the Docker image that we wish to deploy onto the servers, run migrations on the database as we would on the servers, and run a series of Postman requests and tests to mimic a user making a series of requests. This can be orchestrated with one Bash script in the `scripts/run_full_release_test.sh` file. First, we must find out what chip we are running on with the following code:

```
#!/usr/bin/env bash
```

More books: https://t.me/booksforits

```
# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

if [ "$(uname -m)" = "arm64" ]
then
    cp ../builds/aarch64_build ../Dockerfile
else
    cp ../builds/x86_64_build ../Dockerfile
fi
```

Here, we pull the correct build depending on the type of chip. Depending on the computer you are using, this might be different. I am using a Mac M1, so when I call the `uname -m` command in the terminal, I get an `arm64` output. If you are not using an arch or ARM chip, you do not need the conditional logic. Instead, you just need to pull the `x86_64_build` file. Then, we must move to the `tests` directory and build our `docker-compose` with the following code:

```
cd ../tests

# build the images and network
docker-compose build --no-cache
docker-compose up -d

# wait until rust server is running
sleep 5
```

We are now ready to run our tests and clean up the images with the following code:

```
# run the api tests
newman run to_do_items.postman_collection.json

# destroy the container and image
docker-compose down
docker image rm test_server
docker image rm init_test_db
docker image rm test_postgres
rm ../Dockerfile
```

More books: https://t.me/booksforits

Before we run this, however, we need to build our `docker-compose` in our `tests` directory. Our `tests/docker-compose.yml` file has the following outline:

```
version: "3.7"

services:
    test_server:
      . . .
    test_postgres:
      . . .
    test_redis:
      . . .
    init_test_db:
        . . .
```

First, we will focus on the test server. Seeing as we are running a test, we need to point to the build, pass a `NOT PRODUCTION` argument into the build, define the environment variables for the server to utilize, and then wait for the Redis database to be operational before spinning it up. We can do this with the following code:

```
test_server:
  container_name: test_server
  image: test_auth_server
  build:
    context: ../
    args:
      ENV: "NOT_PRODUCTION"
  restart: always
  environment:
    - 'DB_URL=postgres://username:password@test_postgres:54
         32/to_do'
    - 'SECRET_KEY=secret'
    - 'EXPIRE_MINUTES=60'
    - 'REDIS_URL=redis://test_redis/'
  depends_on:
      test_redis:
        condition: service_started
  ports:
```

More books: https://t.me/booksforits

```
    - "8000:8000"
  expose:
    - 8000
```

As we can see, `docker-compose` is a powerful tool. A few tags can result in some complex orchestration. Then, we can move to our database and Redis containers with the following code:

```
test_postgres:
  container_name: 'test_postgres'
  image: 'postgres'
  restart: always
  ports:
    - '5433:5432'
  environment:
    - 'POSTGRES_USER=username'
    - 'POSTGRES_DB=to_do'
    - 'POSTGRES_PASSWORD=password'

test_redis:
  container_name: 'test_redis'
  image: 'redis:5.0.5'
  ports:
    - '6379:6379'
```

These databases are nothing new. However, in the last service, we create an init container that spins up briefly just to run the migrations on the server:

```
init_test_db:
    container_name: init_test_db
    image: init_test_db
    build:
      context: ../database
    environment:
      - 'DB_URL=postgres://username:password@test_postgres:
            5432/to_do'
    depends_on:
        test_postgres:
```

More books: https://t.me/booksforits

```
          condition: service_started
    restart: on-failure
```

As we can see, there must be a Docker build in the `database` directory for our init container to make a database migration before closing. This means that our init container must have `psql` installed, our migrations tool, and the `rollup` command as the entry point. Initially, we install what we need in our `database/Dockerfile` file with the following code:

```
FROM postgres

RUN apt-get update \
  && apt-get install -y wget \
  && wget -O - https://raw.githubusercontent.com/\
  yellow-bird-consult/build_tools/develop/scripts/\
  install.sh | bash \
  && cp ~/yb_tools/database.sh ./database.sh
```

Here, we can see that we get the `psql` library from the `postgres` Docker image. Then, we install `wget` and use this to install our migrations build tool. Finally, we copy the `database.sh` Bash script from the home directory into the root directory of the image so that we do not have to worry about aliases. Once we have configured our installments, we must copy the migrations SQL files from the current directory into the root directory of the image and define the migration command as the entry point:

```
WORKDIR .
ADD . .

CMD ["bash", "./database.sh", "db", "rollup"]
```

This will work fine; however, we do have to define a `database/.dockerignore` file with the following content to avoid the environment variable being passed into the image:

```
.env
```

If we do not stop this environment variable from being copied into the image, then whatever variables we pass into the init container through `docker-compose` could get overwritten.

We now have everything we need in place, so all we must do is run our `scripts/run_full_release.sh` script. This will produce a lengthy printout of building the images, spinning up `docker-compose`, and running the API tests via Newman. The last output should look like this:

More books: https://t.me/booksforits

	executed	failed
iterations	1	0
requests	10	0
test-scripts	20	0
prerequest-scripts	12	0
assertions	67	0
total run duration: 3.2s		
total data received: 1.21kB (approx)		
average response time: 302ms [min: 6ms, max: 1559ms, s.d.: 589ms]		

Figure 13.5 – Result of a full test run

We can see that all the tests ran and passed. Our distroless build works and our init container for the database also makes the migrations. Nothing is stopping us from putting this infrastructure on AWS, with the difference of pointing to images on Docker Hub as opposed to local builds. Considering how small our distroless server is, pulling the image from Docker Hub and spinning it up will be very quick.

We've now got all the ingredients to build continuous integration for our GitHub repository to ensure that tests are run when we create pull requests. In the next and final section, we will configure continuous integration through **GitHub Actions**.

Building continuous integration with GitHub Actions

When it comes to ensuring that code quality is maintained, it can be handy to have a continuous integration pipeline that will run every time a pull request is done. We can do this with GitHub Actions. It must be noted that with GitHub Actions, you get several free minutes every month; then, you must pay for the minutes you go over. So, be careful and keep an eye on how much time you're spending using GitHub Actions.

GitHub Actions gives us flexibility when it comes to implementing tasks. We can run workflows when a pull request is merged or made and when an issue is created and much more. We can also be selective about the type of branches we use. In this example, we will merely focus on a pull request on any branch to run unit tests and then full integration tests.

More books: https://t.me/booksforits

To build a workflow called `tests`, we need to create a file called `.github/workflows/run-tests.yml`. In this file, we will define the general outline of the unit and integration tests with the following code:

```
name: run tests

on: [pull_request]

jobs:
  run-unit-tests:
    . . .
  run-integration-test:
    . . .
```

Here, we have defined the name of the workflow and the conditions that the workflow is triggered on pull requests for all branches. Then, we define two jobs – one to run unit tests and the other to run integration tests.

Each job has steps. We can also define dependencies for our steps. We can define our unit test job with the following code:

```
run-unit-tests:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v2
      - uses: actions-rs/toolchain@v1
        with:
          toolchain: stable
      - name: run the unit test
        run: |
          export SECRET_KEY="secret"
          export EXPIRE_MINUTES=60
          cargo test
```

Here, we used the `checkout` action. If we do not use the `checkout` action, we will not be able to access any of the files in the GitHub repository. Then, we export the environment variables that are needed for the unit tests to run, and then we run the unit tests using Cargo. Also, note that we define a timeout. Defining a timeout is important just in case something ends up in a loop and you do not burn all your minutes in one job.

More books: https://t.me/booksforits

Now, let's move on to our integration test job:

```
run-integration-test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v2
      - name: create environment build and run newman
        run: |
          cd tests
          cp ../builds/server_build ../Dockerfile
          docker-compose build --no-cache
          docker-compose up -d
          sleep 5
      - uses: actions/checkout@master
      - uses: matt-ball/newman-action@master
        with:
          collection:
              ./tests/cerberus.postman_collection.json
```

Here, we move into the `tests` directory, get the server build Docker file, spin up `docker-compose`, and then use the `newman` action to run the Newman tests. If we make a pull request, the actions will be shown on the pull request. If we click on the GitHub Actions button, we can access the status and results, as shown in the following screenshot:

More books: https://t.me/booksforits

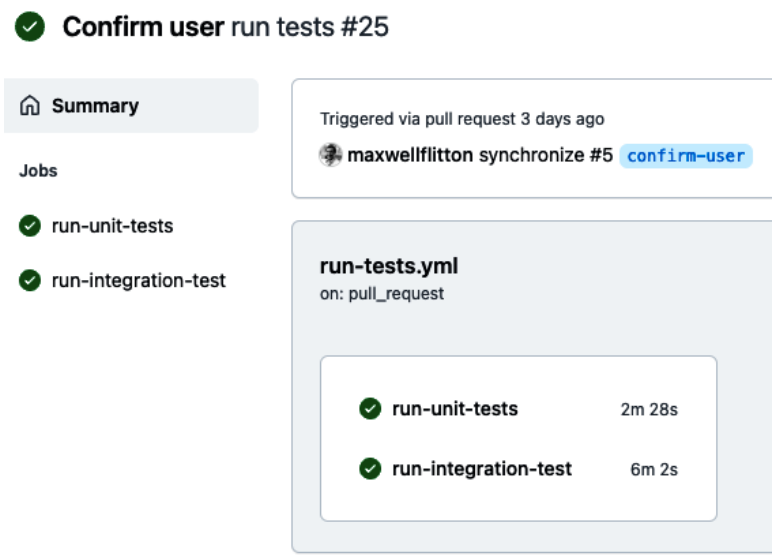


Figure 13.6 – GitHub Actions options

Then, we can click on the test to see the steps of the job, as shown in the following screenshot:

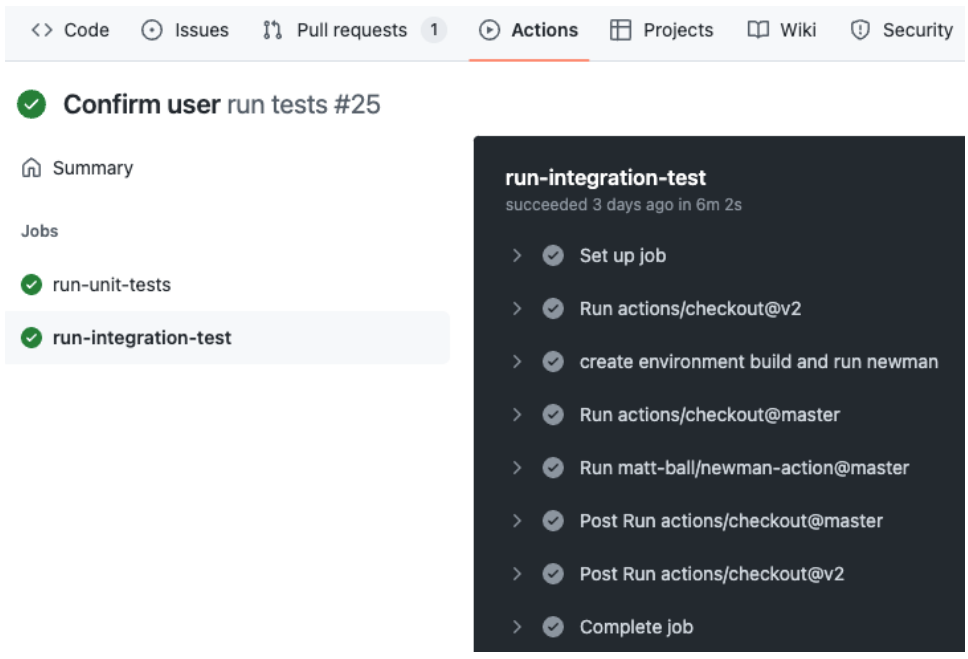


Figure 13.7 – GitHub Actions job view

More books: https://t.me/booksforits

Now, if we click on a step in the job, it will expand. We will see that our Newman tests work:



Figure 13.8 – Newman step result

As we can see, our continuous integration works! We have now come to the end of this chapter as our repository is clean and functional.

Summary

We have finally made it to the end of structuring a web application in Rust and building the infrastructure around the application to make ongoing development of new features safe and easy to integrate. We have structured our repository into one that's clean and easy to use where directories have individual purposes. Like in well-structured code, our well-structured repository can enable us to slot tests and scripts in and out of the repository easily. Then, we used pure bash to manage migrations for our database without any code dependencies so that we can use our migrations on any application, regardless of the

More books: https://t.me/booksforits

language being used. Then, we built init containers to automate database migrations, which will work even when deployed on a server or cluster. We also refined the Docker builds for our server, making them more secure and reducing the size from 1.5 GB to 45 MB. After, we integrated our builds and tests into an automated pipeline that is fired when new code is merged into the GitHub repository.

This brings a natural end to building a web application and deploying it on a server. In the following chapters, we will dive deeper into web programming with Rust, looking at lower-level frameworks so that we can build custom protocols over TCP sockets. This will enable you to build lower-level applications for web servers or even local processes. In the next chapter, we will explore the Tokio framework, a building block of async programs such as TCP servers.

Further reading

- Database migrations documentation and repository: `https://github.com/yellowbird-consult/build_tools`
- GitHub Actions documentation: `https://docs.github.com/en/actions/guides`

Questions

1. The bash migrations tool uses incremental single-digit integers to denote migrations. What is the big downside to this?
2. Why are distroless servers more secure?
3. How did we remove the need for Python when running our Newman tests that required a fresh token?
4. What are the advantages of using environment variables for configuration values?

Answers

1. Using incremental single-digit integers exposes the migrations to clashes. So, if one developer writes migrations on one branch while another developer writes migrations on a different branch, there will be a conflict of migrations when they both merge. GitHub should pick this up, but it's important to keep the traffic of migrations low, plan out database alterations properly, and keep the services using the migrations small. If this is a concern for you, however, please use a different migrations tool that is heavier but has more guardrails.
2. Distroless servers do not have shells. This means that if a hacker manages to access our server container, they cannot run any commands or inspect the contents of the container.
3. In the login request, we get the token that is returned from the server in the test script and assign it to a collection variable that can be accessed by other requests, removing the reliance on Python.

More books: https://t.me/booksforits

4. Environment variables are simply easier to implement when deploying our application to the cloud. For instance, Kubernetes's ConfigMaps use environment variables to pass variables into Docker containers. It is also easier to implement services such as Secrets Manager on AWS by using environment variables.

More books: https://t.me/booksforits

Part 6: Exploring Protocol Programming and Async Concepts with Low-Level Network Applications

Web programming has evolved to more than just simple applications that interact with databases. In this part, we cover more advanced concepts with async Rust by covering the basics of async Rust, Tokio, and Hyper. With Tokio and Hyper, we leverage async Rust and the actor model to implement async designs such as passing messages between actors in different threads, queuing tasks in Redis to be consumed by multiple workers, and processing byte streams with Tokio framing and TCP ports. By the end of this part, you will be able to implement more complex event-processing solutions on your server to handle more complex problems. You will also have practical knowledge of how to implement async Rust, which is an up-and-coming field.

This part includes the following chapters:

- *Chapter 14*, *Exploring the Tokio Framework*
- *Chapter 15*, *Accepting TCP Traffic with Tokio*
- *Chapter 16*, *Building Protocols on Top of TCP*
- *Chapter 17*, *Implementing Actors and Async with the Hyper Framework*
- *Chapter 18*, *Queuing Tasks with Redis*

More books: https://t.me/booksforits

More books: https://t.me/booksforits

14

Exploring the Tokio Framework

So far in this book, we have been building web apps using frameworks and packaging them in Docker to be deployed on a server. Building standard servers is useful and will enable us to solve a range of problems. However, there will come a point in your web development career where a standard REST API server will not be the best solution. It is useful to reach for another tool to implement more custom solutions.

In this chapter, we will explore the **Tokio** framework to enable async programming. We will then use the Tokio runtime to build safe custom async systems, by sending messages to async code blocks using channels. These messages can even be sent over different threads. We will then facilitate async solutions to complex problems by implementing the actor model.

In this chapter, we will cover the following topics:

- Exploring the Tokio framework for async programming
- Working with workers
- Exploring the actor model for async programming
- Working with channels
- Working with actors in Tokio

By the end of this chapter, you will be able to create async programs that solve complex problems using the actor model. Your async program will not need any external infrastructure such as a database, and the implementation of our async program will be safe and isolated because you will be able to pass data around your system and threads using channels. You will be able to understand and implement the building blocks of highly concurrent async programs and network applications.

Technical requirements

In this chapter, no previous code is needed.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter14`.

More books: https://t.me/booksforits

Exploring the Tokio framework for async programming

Before we explore what Tokio is and how it works, we should try to execute some async code in normal Rust. Throughout this chapter, we will be building a basic simulation using Tokio. Therefore, the Tokio code that we will be writing is in the `simulation` directory as its own Cargo project. Seeing as we are running `async` functions in our Rust server code to process views, we can see if we can execute a basic `async` function in our `main` function in the `main.rs` file with the following code:

```
async fn hello() {
    println!("Hello, world!");
}
fn main() {
    hello();
}
```

This looks simple enough; however, if we try to run our `main` function, we get the following output:

```
warning: unused implementer of `Future` that must be used
 --> src/main.rs:9:5
  |
9 |     hello();
  |     ^^^^^^^^
  |
  = note: `#[warn(unused_must_use)]` on by default
  = note: futures do nothing unless you `.await` or poll them
```

Here, we are reminded that our `async` function is a **future**. This means that it is a promise that something will be executed in the future. As a result, we have not been able to see the `Hello, world!` message because we did not wait for the `hello` function to execute. However, if we implement `await` on our `hello` function, we will get the following error:

```
8 | fn main() {
  |    ---- this is not `async`
9 |     hello().await;
  |             ^^^^^^ only allowed inside `async`
                       functions and blocks
```

The `main` function is not `async`. If we try to turn our `main` function into an `async` function, we get a very clear error message that the `main` function is not allowed to be `async`.

More books: https://t.me/booksforits

We could implement our own structs that implement a `Future` trait and then create our own poll method. However, creating our own futures from scratch would be excessive for the context of this book, as this book is not dedicated to async Rust. Luckily, the Tokio framework comes to the rescue by turning our `main` runtime function into an `async` runtime function. To run our `hello` function, we first need to add Tokio to our `Cargo.toml` file with the following code:

```
[dependencies]
tokio = { version = "1", features = ["full"] }
```

We then import the following Tokio macro and `Error` struct in our `main.rs` file to enable an async runtime:

```
use tokio::main;
use std::error::Error;
```

We can then apply our Tokio macro to make our `main` function async and execute our `hello` function with the following code:

```
#[main]
async fn main() -> Result<(), Box<dyn Error>> {
    let outcome = hello().await;
    Ok(outcome)
}
```

Here, we can see that we are either returning a `Result` with an empty tuple, which is the same as `None` or `Void` in other languages, or an error. If we return an error, we return a struct that implements the `Error` trait that is on heap memory due to the `Box` notation. When we run our program now, we will get the `hello world` message. From this, we can deduce that our program is blocked until the `hello` function has executed. However, the preceding code requires us to return something, and there is a bit of boilerplate code in the definition. If we drop all imports, we can have the following simpler `main` function:

```
#[tokio::main]
async fn main() {
    hello().await;
    println!("program has run");
}
```

More books: https://t.me/booksforits

Here, we can see that we are not having to bother with `return` definitions, and we can do whatever we want in the last statement of the `main` function. We still block the thread on `await`, as we can see with the following printout:

```
Hello, world!
program has run
```

Now that we have got basic `async` functions running in Tokio, we can do an experiment. We can run multiple `async` functions and wait for them with the following code:

```
async fn hello(input_int: i32) -> i32 {
    println!("Hello, world! {}", input_int);
    return input_int
}
#[tokio::main]
async fn main() {
    let one = hello(1);
    let two = hello(2);
    let three = hello(3);

    let one = one.await;
    let three = three.await;
    let two = two.await;
    println!("{} {} {}", one, two, three);
}
```

Running this code will give the following output:

```
Hello, world! 1
Hello, world! 3
Hello, world! 2
1 2 3
```

Here, we can see that the operations are executed in the order `main` is executed. This means that when we use `await` to wait on a future to complete, the runtime is blocked until the future has completed. Even though the `two` future was defined before the `three` future, the `three` future executes before the `two` future because the `three` future was awaited before the `two` future.

More books: https://t.me/booksforits

So what? What's the big deal? If our `async` functions are blocking the runtime, then why don't we just define normal functions? We can do one last experiment to get to know Tokio: the standard sleep test. First, we need to import the following:

```
use std::time::Instant;
use std::{thread, time};
```

We then redefine our `hello` function to sleep for 5 seconds before printing out to the terminal with the following code:

```
async fn hello(input_int: i32) -> i32 {
    let five_seconds = time::Duration::from_secs(5);
    tokio::time::sleep(five_seconds).await;
    println!("Hello, world! {}", input_int);
    input_int
}
```

We then spawn Tokio tasks when waiting for our futures to execute in our hello function. We spawn a Tokio task using `tokio::spawn`. A Tokio task is a light weight, non-blocking unit of execution. While Tokio tasks are like OS threads, they are not managed by the OS scheduler but by the Tokio runtime instead. The Tokio task spawned is run on a thread pool. The spawned task could correspond to a thread, but it could also not. It is up to the Tokio runtime. We spawn our tasks with the following code:

```
#[tokio::main]
async fn main() {
    let now = Instant::now();

    let one = tokio::spawn({
        hello(1)
    });
    let two = tokio::spawn({
        hello(2)
    });
    let three = tokio::spawn({
        hello(3)
    });
    one.await;
    two.await;
    three.await;
```

More books: https://t.me/booksforits

```
    let elapsed = now.elapsed();
    println!("Elapsed: {:.2?}", elapsed);
}
```

If our futures block the entire runtime, then the time elapsed will be 15 seconds. However, running our program will give us the following output:

```
Hello, world! 2
Hello, world! 3
Hello, world! 1
Elapsed: 5.00s
```

Here, we can see that there is some asynchronous order in which the futures can be executed. However, the total time is 5 seconds because they are running concurrently. Again, at face value, this does not seem too impressive. However, this is where it gets exciting. The super smart people building Tokio keep track of threading using polling. This is where the Tokio runtime keeps checking to see if a future has executed by polling a bit of memory. Because of polling, checking up on threads does not take a lot of resources. Because checking up on threads does not take a lot of resources, Tokio can literally keep millions of tasks open. If there is an `await` instance in the thread and it is blocking the runtime of that thread, Tokio will merely switch over to another thread and start executing the new thread. A future running in the background would occasionally poll the task executor end query if there is a result or not. As a result, Tokio is powerful, and therefore we used Tokio as a runtime for our Actix Web server when exploring Actix in *Chapter 3*, *Handling HTTP Requests*. Getting to know Tokio will enable us to build our own networking applications at a lower level.

While spinning off multiple threads using Tokio is exciting, we must explore the trade-offs of working with multiple workers.

Working with workers

When it comes to defining workers, we can augment the Tokio runtime macro with the following code:

```
#[tokio::main(flavor = "multi_thread", worker_threads = 4)]
async fn main() {
    ...
}
```

Here, we can see that we state that the runtime is multithreaded, and we have four worker threads. Workers are essentially processes that run in constant loops. Each worker consumes tasks through a channel, putting them in a queue. The worker then works through the tasks, executing them in the order received. If a worker has finished all the tasks, it will search other queues belonging to other workers, stealing tasks if they are available, as seen here:

More books: https://t.me/booksforits

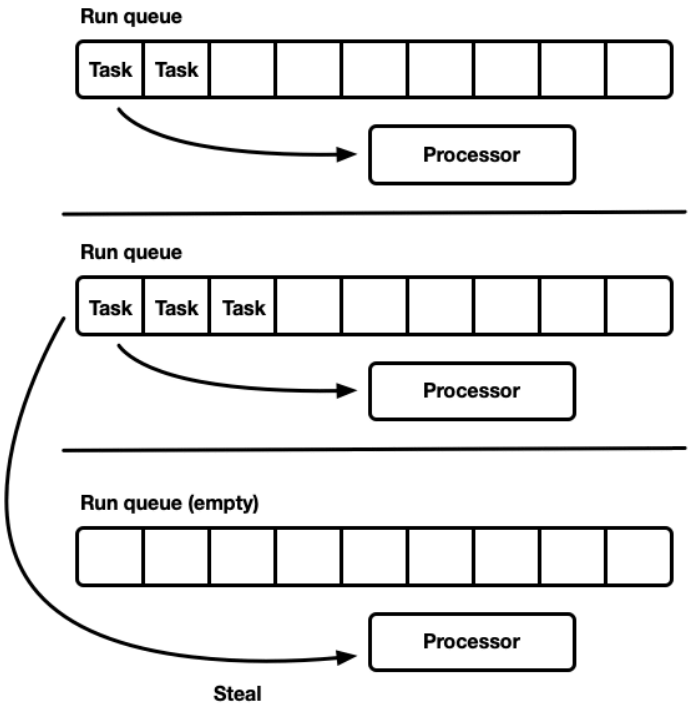


Figure 14.1 – Worker event loops (Work stealing runtime. By Carl Lerche – License MIT: https://tokio.rs/blog/2019-10-scheduler#the-next-generation-tokio-scheduler)

Now that we know we can alter the number of workers, we can test how the number of worker threads affects how our runtime. First, we must change our `hello` function to sleep for only 1 second at a time. We then loop through a range of numbers where we spawn a task for each iteration of that range, pushing the handle of the spawned task to a vector. We then await all the futures in the vector with the following code:

```
#[tokio::main(flavor = "multi_thread", worker_threads = 4)]
async fn main() {
    let now = Instant::now();
    let mut buffer = Vec::new();

    for i in 0..20 {
        let handle = tokio::spawn(async move {
            hello(i).await
        });
        buffer.push(handle);
```

More books: https://t.me/booksforits

```
    }
    for i in buffer {
        i.await;
    }
    let elapsed = now.elapsed();
    println!("Elapsed: {:.2?}", elapsed);
}
```

We could keep running the program with a different number of worker threads to see how the number of threads affects the time taken. To save you time, this has been done, providing the graph shown in *Figure 14.2*:

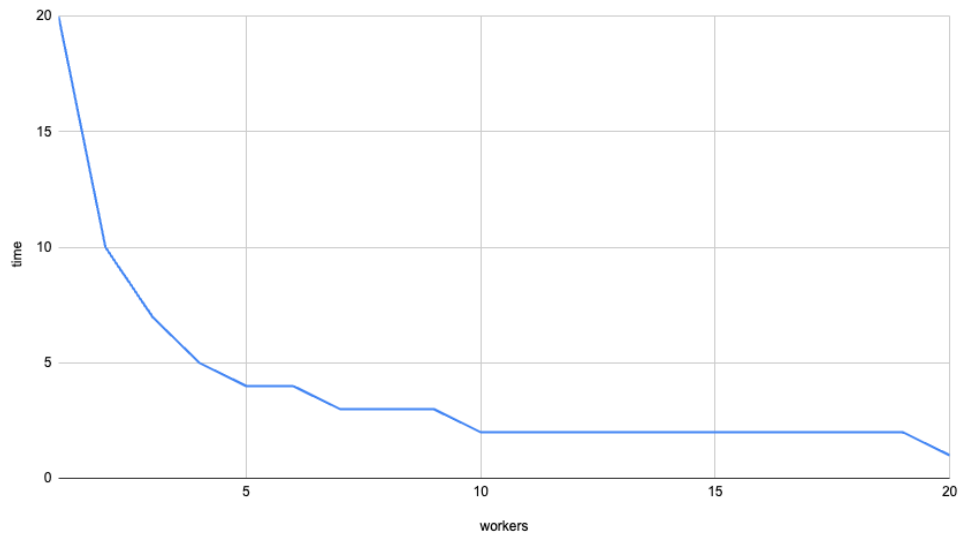


Figure 14.2 – Time versus the number of workers

We can see that the first four workers have a big effect on the overall time of the program. However, the returns sharply diminish as we increase the number of workers to more than four. This might be different for you. I got this graph because my computer that is running our program has four cores. It is advised that we have one worker thread per core.

We have now managed to speed up our program using Tokio worker threads. We have also gotten a deeper appreciation for the trade-off of worker threads and how they process tasks. However, what advantage is there to understanding this? We could just use a higher-level framework like Actix Web or Rocket to concurrently handle API endpoints. Yes, higher-level frameworks are useful and do solve a problem, but these are just one solution to the problems. In the next section, we will cover the actor model as this is an asynchronous solution that can run in a program or a web server depending on your needs.

More books: https://t.me/booksforits

Exploring the actor model for async programming

If you have coded complex solutions to complex problems before in an object-oriented fashion, you will be familiar with objects, attributes, and class inheritance. If you are not familiar, do not worry—we are not going to implement them in this chapter. However, it is advised that you read up on the concepts of object-oriented programming to gain a full appreciation for the actor model.

For objects, we have a range of processes and an encapsulated state around those processes, which can be attributes of the object. Objects are useful for compartmentalizing logic and state around a concept or process. Some people merely see objects as a tool to reduce repeated code; however, objects can be used as interfaces between modules or can be used to orchestrate processes. Objects are the cornerstone of many complex systems. However, when it comes to asynchronous programming, objects can get messy—for instance, when we have two objects referencing data from another object or in general when we have shared common resources between objects, modifying them simultaneously, as seen in the following diagram:

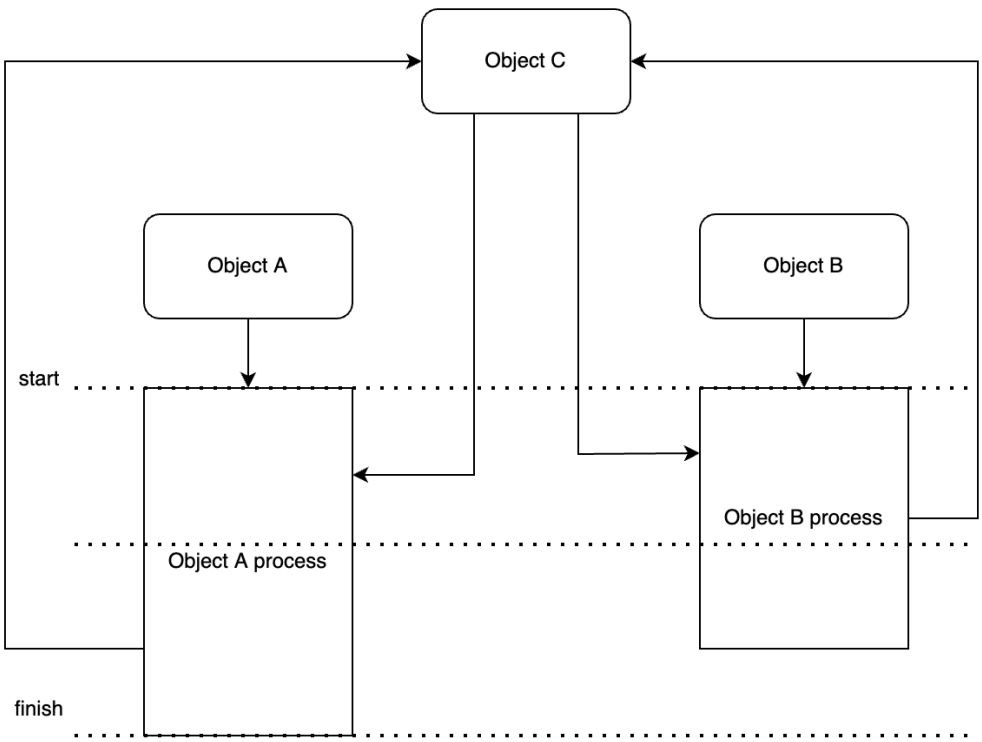


Figure 14.3 – Objects and async

In this diagram, we can imagine a system where *Object C* is keeping track of investments. We have a rule coded into *Object C* that we do not spend over £50. However, *Object A* and *Object B* are both performing processes that invest in different stocks. For a range of reasons, from network latency to

More books: https://t.me/booksforits

maybe different price calculation strategies, the processes for deciding to invest in a stock can vary. Even though both *Object A* and *Object B* get approved by getting the total amount already invested from *Object C*, *Object B* places the order first due to *Object B*'s process finishing first. So, when *Object A* finishes its process and places an order, it could tip our total investment over £50. This is a data race problem. We can potentially spend over our investment budget because two competing investment executions are operating too close to each other. When we consider that we could be keeping track of potentially hundreds of positions, it will become certain that data races will end up breaking the rules of our strategy.

To solve the data race problem, we could implement a database, or try to keep an eye on all the threads running to implement locks in the form of Mutexes, RwLocks, and so on. However, the database solution requires more memory, infrastructure, and code to handle the data. The data is also persisted when the program has been shut down, which adds to the complexity of the management. The locking system and keeping track of common shared resources also introduce potential bugs and excessive complexity for the simple problem that we have defined. This is where actors come in. Actors are essentially units of computation with their own state. However, it must be noted that actors are not free; they require heavier RAM usage. Also, actors can die, which will also kill their state, so database backups can be helpful to persist the state if and when actors die. Actors only communicate through channels using messages. These messages are queued and then processed in the order the messages were sent. This gives us the following system dynamic:

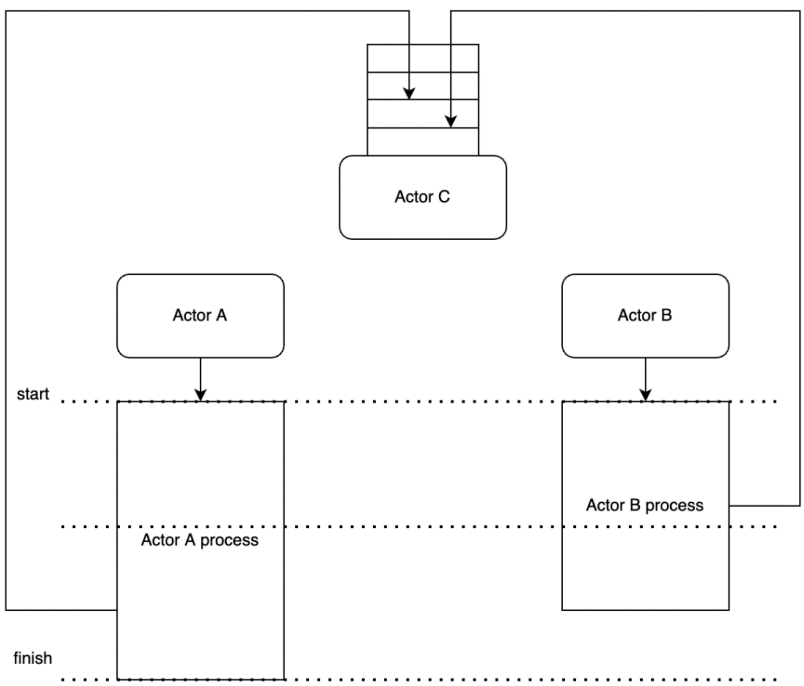


Figure 14.4 – Actors and async

More books: https://t.me/booksforits

Here, we can see that *Actor C* accepts messages and places an order if we have not exceeded our investment budget. We can have hundreds of actors sending buy-and-sell messages to *Actor C* and still be confident that the budget would not be exceeded. Our system does not get increasingly complicated as we add more actors. There is nothing stopping us from getting our *Actor C* to send messages to other actors to stop or increase the number of buy-or-sell positions, depending on the current state of our investment budget.

To build more complex systems, you must be familiar with the following terms:

- **Actor**: A unit of work that can contain state and modifies that state by processing messages it receives. You never have reference to an actor state or actor directly.
- **Actor reference**: A handle to an actor. This allows you to send the actor messages without knowing its implementation type or location on the network. All I need to know is that if we send a message to an actor, we get *X* back. We do not need to know if the actor is local somewhere else.
- **Actor system**: A collection of actors that exist inside a single process and communicate via in-memory message passing.
- **Cluster**: A collection of networked actor systems whose actors communicate via TCP message passing.

What makes this exciting is that we can achieve asynchronism without having to rely on a database or on keeping track of threads. We can run it all in our Tokio runtime in one static binary! This is very powerful. However, there is nothing stopping us from using the actor model on a microservices-cluster scale in the form of nanoservices. At the time of writing this book, nano services are lightly written about and used by some companies such as Netflix. However, with what we have explored with Rust and distroless containers, we can deploy Rust servers into a cluster at 50 MB a server. With what we have covered so far in this book, there is nothing stopping you from pushing the boundaries and building nano services, and implementing the actor model using nano services.

The actor model is being used in a lot of IoT devices and real-time event systems. Some applications that make use of the actor model are listed as follows:

- Event-driven applications (chat, workflow, CRM)
- Finance (pricing, fraud detection, algorithmic trading)
- Gaming (multi-player)
- Analytics and monitoring
- Marketing automation
- Systems integration
- IoT (healthcare, transportation, security)

More books: https://t.me/booksforits

OK—we may never truly understand Hollywood actors, but at least we are familiar with computational actors. With the high-level concepts out of the way, we can move on to constructing the building blocks of an actor system: actors. Before we work with actors, we need to get actors to communicate with each other. This is achieved using channels in Tokio.

Working with channels

We can experiment with channels by rewriting our `main.rs` file. First, we need to import channels and implement the `main` trait for the `main` function with the following code:

```
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    ...
}
```

Here, we can see that we import the `mpsc` module. **MPSC** stands for **multiple producers, single consumer channel**. The name does what it says on the tin. We are going to be able to have an actor receive messages from multiple actors with `mpsc`. We can now create our channel and spawn a thread that sends multiple messages to a receiver down the channel we created in our `main` function with the following code:

```
let (tx, mut rx) = mpsc::channel(1);

tokio::spawn(async move {
    for i in 0..10 {
        if let Err(e) = tx.send(i).await {
            println!("send error: {:?}", e);
            break;
        }
        println!("sent: {}", i);
    }
});
```

More books: https://t.me/booksforits

Here, we can see that the creation of a channel returns a tuple, which we unpack to transmit (`tx`) and receive (`rx`). We then loop through a range of integers from 0 to 10, sending them down the channel we created. If there is an error, we print it out and return an empty tuple, breaking the loop. In the following loop, we print out what we sent down the channel. We can then receive the messages with the code shown next:

```
while let Some(i) = rx.recv().await {
    println!("got: {}", i);
}
```

It must be noted that split halves of channels implement the `Iterator` trait, enabling us to read messages from these channels in a `while` loop. If we run our code now, we get the following output:

```
sent: 0
got: 0
sent: 1
got: 1
sent: 2
got: 2
...
```

The printout continues to `9`, but we get the idea of what is going on. We are not sending all our messages and then processing them. Both code blocks are executing side by side, sending and receiving messages. If we make one of the code blocks sleep between iterations, we will still get the same printout.

While getting the basis of async message sending and receiving is a nice step in the right direction, it is not useful. Right now, we are just sending a number. If we want to be able to implement actors that communicate with each other, we need to be able to send more comprehensive messages over the channel. To work out how to do this, we can inspect the `channel` source code, which reveals the following:

```
pub fn channel<T>(buffer: usize) -> (Sender<T>, Receiver<T>) {
    assert!(buffer > 0, "mpsc bounded channel requires
            buffer > 0");
    let semaphore = (semaphore::Semaphore::new(buffer),
                     buffer);
    let (tx, rx) = chan::channel(semaphore);
    let tx = Sender::new(tx);
    let rx = Receiver::new(rx);
    (tx, rx)
}
```

More books: https://t.me/booksforits

We can see that the `channel` function is implementing generics. The sender and receiver can send anything if it is consistent, as denoted in the `(Sender<T>, Receiver<T>)` return type. Now that we know the `channel` function can load any type, we can create a message struct outside of the `channel` function with the following code:

```
#[derive(Debug, Clone)]
pub enum Order {
    BUY,
    SELL
}
#[derive(Debug, Clone)]
pub struct Message {
    pub order: Order,
    pub ticker: String,
    pub amount: f32
}
```

Here, we have a struct that has an order, which can either be `BUY` or `SELL`. Our `Message` struct also has a ticker to denote the name of the stock being processed and the amount of stock being processed. Inside our `main` function, we then pass our `Message` struct into the `channel` function and define the stocks that we are going to buy with the following code:

```
let (tx, mut rx) = mpsc::channel::<Message>(1);

let orders = [
    Message { order: Order::BUY,
              amount: 5.5, ticker: "BYND".to_owned()},
    Message { order: Order::BUY,
              amount: 5.5, ticker: "NET".to_owned()},
    Message { order: Order::BUY,
              amount: 5.5, ticker: "PLTR".to_owned()},
];
```

Now that our orders are defined, we loop through the orders, sending and receiving them with the following code:

```
tokio::spawn(async move {
    for order in orders {
        if let Err(e) = tx.send(order.clone()).await {
```

More books: https://t.me/booksforits

```
            println!("send error: {:?}", e);
            return;
        }
        println!("sent: {:?}", order);
    }
});

while let Some(i) = rx.recv().await {
    println!("got: {:?}", i);
}
```

Running our program now gives us the following output:

```
sent: Message { order: "BUY", ticker: "BYND", amount: 5.5 }
sent: Message { order: "BUY", ticker: "NET", amount: 5.5 }
got: Message { order: "BUY", ticker: "BYND", amount: 5.5 }
got: Message { order: "BUY", ticker: "NET", amount: 5.5 }
sent: Message { order: "BUY", ticker: "PLTR", amount: 5.5 }
got: Message { order: "BUY", ticker: "PLTR", amount: 5.5 }
```

We can see here that we are sending and receiving messages across the channel. However, it must be noted before we move on that if we increase the size of the buffer passed into the `channel` function from 1 to 300, we get the following printout:

```
sent: Message { order: "BUY", ticker: "BYND", amount: 5.5 }
sent: Message { order: "BUY", ticker: "NET", amount: 5.5 }
sent: Message { order: "BUY", ticker: "PLTR", amount: 5.5 }
got: Message { order: "BUY", ticker: "BYND", amount: 5.5 }
got: Message { order: "BUY", ticker: "NET", amount: 5.5 }
got: Message { order: "BUY", ticker: "PLTR", amount: 5.5 }
```

This is because the buffer is so large that multiple messages can be sent without having to wait for the channel to be read (drained) until there is a new buffer that is more available.

Working with actors in Tokio

This will be the last time we rewrite our `main.rs` file. However, once we have finished this section, we will have built a basic actor model system. In our system, we will create an actor that keeps track of the orders so that we do not exceed our budget threshold. We then need to build an actor that sends the order, resulting in the following process:

More books: https://t.me/booksforits

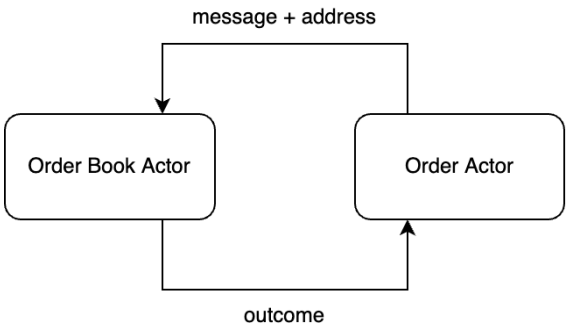


Figure 14.5 – Our stock order interaction for actors

We can see that this time round, we need to send the address of the actor that is making the order with the order message. This is a design choice because, in our system, *order* actors spin up and die quickly after the order has been made. We cannot keep track of the addresses for all the *order* actors in our program in our *order book* actor. Instead, the *order book* actor can get the address from the message to send the response to the *order* actor. First, we must import the following:

```
use tokio::sync::{mpsc, oneshot, mpsc::Sender};
```

Here, we import the `mpsc` channel to send the message to the *order book*. We then import the `oneshot` channel to facilitate the outcome being sent back to the *order*. A `oneshot` channel is a channel that stays open until a message is sent. This is because a `oneshot` channel has a capacity/buffer size of one, so it is bounded to only one message being sent before the channel is closed.

Now that we have imported all that we need, we can move on to defining the message with the following code:

```
#[derive(Debug)]
pub struct Message {
    pub order: Order,
    pub ticker: String,
    pub amount: f32,
    pub respond_to: oneshot::Sender<u32>
}
```

More books: https://t.me/booksforits

Here, we can see that we have attached the `Sender` struct, which enables the receiver to send values to the sender through a channel. With this message, we can build out our *order book* actor with the following code:

```
pub struct OrderBookActor {
    pub receiver: mpsc::Receiver<Message>,
    pub total_invested: f32,
    pub investment_cap: f32
}
```

The state of the *order book* actor will live on throughout the lifetime of the program. The `receiver` field will be used to accept incoming messages from all *order* actors, and a decision on how to process the order will be made on the total invested and the investment cap that we define when we are creating the actor. We now must implement functions that create the *order book* actor, handle incoming messages, and run the actor with the following outline:

```
impl OrderBookActor {
    fn new(receiver: mpsc::Receiver<Message>,
           investment_cap: f32) -> Self {
        . . .
    }
    fn handle_message(&mut self, message: Message) {
        . . .
    }
    async fn run(mut self) {
        . . .
    }
}
```

We can start with the constructor (the `new` function), which takes the following form:

```
fn new(receiver: mpsc::Receiver<Message>, investment_cap:
       f32) -> Self {
    return OrderBookActor {
        receiver,
        total_invested: 0.0,
        investment_cap
    }
}
```

More books: https://t.me/booksforits

The implementation of the constructor is not surprising. We merely pass in a receiver and the investment cap, and automatically set the total invested to `0`. Now that the *order* actor can be constructed, we can handle the messages the *order* actor received with the following code:

```
fn handle_message(&mut self, message: Message) {
    if message.amount + self.total_invested >=
        self.investment_cap {
        println!("rejecting purchase, total invested: {}",
                 self.total_invested);
        let _ = message.respond_to.send(0);
    } else {
        self.total_invested += message.amount;
        println!("processing purchase, total invested: {}",
                 self.total_invested);
        let _ = message.respond_to.send(1);
    }
}
```

Here, we have kept the implementation basic. If the new order brings our invested capital over our threshold, we print out that we have rejected the order and return a `0`. If our new order does not breach the threshold, we then print out that we are processing the order and send a `1`. For our example, the *order* actor is not going to act on the response from the *order book* actor; however, if you were to build in a system where another order is placed (such as a sell order), it is advised that you create another *order* actor based on the outcome.

The only function left for our *order book* actor is the `run` function, which is defined with the following code:

```
async fn run(mut self) {
    println!("actor is running");
    while let Some(msg) = self.receiver.recv().await {
        self.handle_message(msg);
    }
}
```

Here, we merely wait for messages until all the senders in our channel have been killed. If a message is sent to our actor, we process it with our `handle_message` function.

More books: https://t.me/booksforits

Our *order book* actor can now be constructed, run, and receive messages returning a response through a channel with three separate functions. We need to build an *order* actor struct so that we can send messages to our *order book* actor. First, we define our order fields with the following code:

```
struct BuyOrder {
    pub ticker: String,
    pub amount: f32,
    pub order: Order,
    pub sender: Sender<Message>
}
```

Here, we need to define the ticker symbol that we are buying, the amount that we are buying, and the type of order. We then have the `Sender` struct so that we can send messages to the *order book*. For the *order* actor, we only need two functions: the `constructor` function and the `send` function. This is because our *order* actor is the one sending the initial message, so the *order* actor does not need a separate function to wait for messages. Our two functions for our *order* actor have the following outline:

```
impl BuyOrder {
    fn new(amount: f32, ticker: String,
           sender: Sender<Message>) -> Self {
        . . .
    }
    async fn send(self) {
        . . .
    }
}
```

At this point in time, you should be able to code the constructor for the *order* actor yourself. If you stop to have an attempt at this, your code should look like this:

```
fn new(amount: f32, ticker: String,
       sender: Sender<Message>) -> Self {
    return BuyOrder { ticker, amount,
                      order: Order::BUY, sender }
}
```

Here, we merely pass the parameters into the creation of the struct, with the `order` field being automatically set to `BUY`.

More books: https://t.me/booksforits

With the constructor built, the only function left to define is the `send` function, which takes the following form:

```
async fn send(self) {
    let (send, recv) = oneshot::channel();
    let message = Message { order: self.order,
                            amount: self.amount,
                            ticker: self.ticker,
                            respond_to: send};
    let _ = self.sender.send(message).await;
    match recv.await {
        Err(e) => println!("{}", e),
        Ok(outcome) => println!("here is the outcome: {}",
                                outcome)
    }
}
```

Here, we set up a one-shot channel that will close once the response from the *order book* actor has been received. We then package our message with the fields of our *order* actor struct and include the address of the *order* actor in the `respond_to` field of the message. The *order* actor then sends the message and awaits a response, which we merely print out as it is an example.

Our system components are now built. We can now orchestrate our actors to work with each other in our `main` function with the following code:

```
#[tokio::main]
async fn main() {
    let (tx, rx) = mpsc::channel::<Message>(1);
    let tx_one = tx.clone();

    tokio::spawn(async move {
        . . .
    });
    tokio::spawn(async move {
        . . .
    });
    let actor = OrderBookActor::new(rx, 20.0);
    actor.run().await;
}
```

More books: https://t.me/booksforits

Here, we define the channel and clone the sender to the channel once so that we have two senders for the same channel. This is because we have two Tokio threads sending order messages. We then create our *order book* actor and run it. If we cloned another sender but did not use it, our program would hang forever until we shut it down because the *order book* actor would be waiting for all senders for the channel to be killed before stopping.

Inside our threads, we will simply be sending a lot of stocks using loops with the following code:

```
tokio::spawn(async move {
    for _ in 0..5 {
        let buy_actor = BuyOrder::new(5.5,
                                      "BYND".to_owned(),
                                      tx_one.clone());
        buy_actor.send().await;
    }
    drop(tx_one);
});
tokio::spawn(async move {
    for _ in 0..5 {
        let buy_actor = BuyOrder::new(5.5,
                                      "PLTR".to_owned(),
                                      tx.clone());
        buy_actor.send().await;
    }
    drop(tx);
});
```

It must be noted that we drop the `tx_one` and `tx` transmitters after we have sent all of our messages to signal early the task has completed, instead of having to wait for the end of the runtime to close the channel. We have two threads running because we want to properly check if our *order book* is safe to handle concurrent messages. If we run our program, we get the following printout:

```
processing purchase, total invested: 5.5
here is the outcome: 1
processing purchase, total invested: 11
processing purchase, total invested: 16.5
here is the outcome: 1
here is the outcome: 1
rejecting purchase, total invested: 16.5
```

More books: https://t.me/booksforits

```
here is the outcome: 0
rejecting purchase, total invested: 16.5
here is the outcome: 0
. . .
```

Here, we can see messages are being sent and processed to our *order book* actor in an async manner. However, no matter how many times you run the program, you will never exceed your threshold of investment.

And here we have it. Tokio enables us to run async code in the `main` function, and the actor model enables us to build safe async code that is easy to understand without having to rely on locks, handling of threads, passing state over threads, or external infrastructure such as a database. This does not mean that we should run around applying the actor model to everything. However, if you want to write safe async code that has a simple, testable implementation without external infrastructure and fast access to local memory, then the actor model combined with Tokio is very powerful.

Summary

In this chapter, we managed to revisit async programming. Initially, we explored the Tokio framework by looking into what the Tokio framework enabled us to do and then implementing basic async code, which was then implemented by the Tokio framework. We then increased the number of worker threads to demonstrate the point that we get diminishing returns when we simply increase the number of worker threads. We then worked with channels to facilitate sending messages through these channels. This enabled us to send data between different areas of the code even if the code is running in a different thread.

However, while async programming with the Tokio framework is interesting and fun, the basic combination of Tokio and async programming with channels does not directly lead to practical applications by itself. To gain some practical skills with Tokio and async programming, we explored the actor framework. This is where we define structs with their own state, and then communicate between the structs that we now call actors using channels. We then used actors to build a basic system that places orders where the amount of money invested does not exceed a defined threshold, even though we are placing these orders in an async manner. Implementing the actor model in Tokio has enabled us to build safe async systems without having any external infrastructure such as a database. In the next chapter, we take the ability to build custom async systems to the next level by building TCP listeners for our Tokio async application, meaning that we can listen for commands sent to our application from other applications and clients across a network.

More books: https://t.me/booksforits

Further reading

- Tokio documentation: `https://tokio.rs/tokio/tutorial`

Questions

1. How does an actor model prevent data race issues?
2. How can we engineer a two-way communication where an actor can send a message to another actor and get a response back?
3. What happens to our program if we have two actors sending one message each to a third actor but we have cloned three instances of the sender for the MPSC channel facilitating the communication between the actors?
4. What are the advantages of using channels and messages to communicate data between code blocks?

Answers

1. The actor model is where actors send messages to each other. Each actor has a queue for processing the incoming messages for the actor. Because of this, the incoming messages are processed in the order they are received.
2. The actor receiving the initial message has the receiver of the MPSC channel. The actor sending the initial message has the sender for the MPSC channel and creates a one-shot channel. The actor sending the initial message then sends the sender from the one-shot channel in the initial message. The actor sending the initial message then waits for the actor receiving the initial message to process the data and then uses the sender in the initial sender to send back a response.
3. Our program will run as we expect; however, only two of the senders for the MSPC channel will be killed as we only have two actors that are sending messages. This means that one sender will be left over. As one sender is not used up, the channel is not closed, so the receiving actor will continue to keep the program running indefinitely.
4. Channels and messages give us a lot of flexibility. We can send messages across threads. We also do not have to pass data across multiple different areas of code. If a block has a connection to a channel, the code block can receive/send data. We also must note that we can implement different patterns. For instance, we can have a message emitted into a channel and received by multiple subscribers. We can also enforce the direction of traffic or rules such as one-shot. This means we have a lot of control over the flow of the data.

More books: https://t.me/booksforits

More books: https://t.me/booksforits

15
Accepting TCP Traffic with Tokio

In the previous chapter, we managed to get actors running in different threads to send messages to each other. While it is exciting to code the building blocks for async programming, we left that chapter with a not-very-practical application. In this chapter, we will be creating a server with Tokio that listens to **TCP** traffic on a port. If messages are sent, our TCP server will process the incoming data, perform operations through a series of actors and threads, and then return the updated data to the client.

In this chapter, we will cover the following topics:

- Exploring TCP
- Accepting TCP
- Processing bytes
- Passing TCP to an actor
- Keeping track of orders with actors
- Chaining communication between actors
- Responding with TCP
- Sending different commands via the client

By the end of this chapter, you will understand how to use TCP and how to package and unpack data sent via TCP with bytes. With this knowledge, you will be able to create a server using Tokio to listen to incoming messages, process those messages, and then perform units of computation based on the incoming message through a series of threads and actors.

More books: https://t.me/booksforits

Technical requirements

In this chapter, we will be building on the code from *Chapter 14*, *Exploring the Tokio Framework*. This can be found at the following URL: `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter14/working_with_actors`.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter15`.

Exploring TCP

TCP stands for **transmission control protocol**. TCP is one of the most widely used transfer protocols on the internet. TCP is essentially a protocol that transports bytes over a socket from one program or computer to another using an **internet protocol** (**IP**). TCP is used for the world wide web, email, remote administration, and file transfer. The **transport layer security/secure sockets layer** (**TLS/SSL**) protocols are built on top of TCP. This means that HTTP and HTTPS are built on top of TCP.

TCP is a connection-oriented protocol. This is where a connection between the client and server is established before any data is transferred. This is achieved by a three-way handshake:

1. **SYN**: Initially, the client sends a SYN to the server. The SYN is a message with a random number to ensure that the same client is communicating with the server.
2. **SYN-ACK**: The server then responds to the client with the initial sequence number and an additional random number known as the ACK.
3. **ACK**: Finally, the client returns the ACK to the server to establish that the connection has been acknowledged.

Steps 1 and *2* establish and acknowledge the sequence number from the client and the server. *Steps 2* and *3* establish and acknowledge the sequence number from the server to the client:

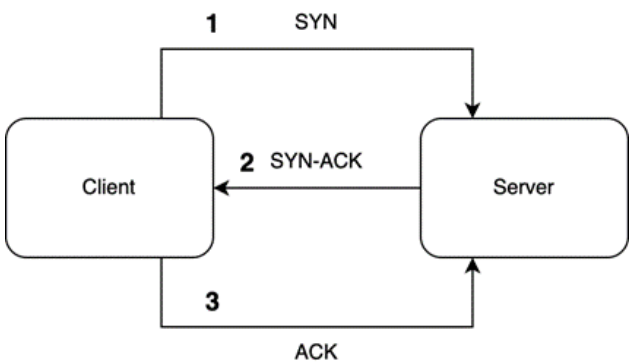


Figure 15.1 – A TCP handshake

More books: https://t.me/booksforits

In this chapter, we will be converting the actor model we built in the previous chapter so that it accepts TCP traffic from outside of our program as commands. First, we need to get our program to accept TCP connections.

Accepting TCP

Before we write any TCP code, we must acknowledge that our code is at risk of becoming bloated due to all the code being in one file. To prevent the `src/main.rs` file from becoming bloated, we must copy all the code apart from the main function from the `src/main.rs` file into a file called `src/actors.rs`. Now, we can wipe our `src/main.rs` file completely and fill it with the following outline:

```
use tokio::net::TcpListener;
use std::{thread, time};

#[tokio::main]
async fn main() {
    let addr = "127.0.0.1:8080".to_string();

    let mut socket = TcpListener::bind(&addr).await.unwrap();
    println!("Listening on: {}", addr);

    while let Ok((mut stream, peer)) =
        socket.accept().await {
        println!("Incoming connection from: {}",
                  peer.to_string());
        tokio::spawn(async move {
            . . .
        });
    }
}
```

Here, we imported a TCP listener to listen to incoming traffic. We also imported structs that enable us to perform a sleep function from the `Tokio` crate and define our `main` runtime function. In our `main` function, we define our address and bind it to a TCP listener. We directly unwrap this because if we fail to bind the address, there is no point in continuing with the program. You could handle the outcome of binding the address by increasing the port number by 1 until you find an open port number, but for this example, we should keep our implementation of the server simple. Then, we have a `while` loop that continues to accept new connections throughout the lifetime of the program,

More books: https://t.me/booksforits

which can be infinite if the program is not interrupted or there is no problem with the socket. Once we get a connection, we spawn a new thread and process the incoming message.

For now, for our incoming message, we will merely sleep for 5 seconds, as seen in the following code:

```
tokio::spawn(async move {
    println!("thread starting {} starting",
               peer.to_string());
    let five_seconds = time::Duration::from_secs(5);
    let begin = time::Instant::now();
    tokio::time::sleep(five_seconds);
    let end = begin.elapsed();
    println!("thread {} finishing {}", peer.to_string(),
              end.as_secs_f32());
});
```

Here, we print when the thread is starting and finish printing the time duration at the end. The duration should be more than the delay. We also make the thread sleep. The printout statements and sleep functionality will enable us to trace what is happening when we send multiple messages from different programs.

Now that we have defined our program, which accepts TCP traffic, we can create a new Rust cargo project in a different directory for the client, which will send messages to the server. In this new project, the `Cargo.toml` file will contain the same dependencies as those in the TCP server. Inside the `main.rs` file, we have the following simple program:

```
use tokio::net::TcpStream;
use tokio::io::AsyncWriteExt;
use std::error::Error;

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    let mut stream =
        TcpStream::connect("127.0.0.1:8080").await?;
    println!("stream starting");
    stream.write_all(b"hello world").await?;
    println!("stream finished");
    Ok(())
}
```

More books: https://t.me/booksforits

This program merely makes a connection with the TCP server and then writes `hello world` in bytes to the TCP stream. Once the bytes have been written, we finish. Now, we can test our server. First, we need to run our server, which will give us the following printout:

```
Listening on: 127.0.0.1:8080
```

The terminal should now be hanging, which means we can run our client in a different terminal three times in a row as quickly as possible to get three requests in before the first thread stops sleeping. This will allow us to see the effect the sleeping threads have on our application. Our client printout should look as follows:

```
    Finished dev [unoptimized + debuginfo] target(s) in
    0.87s
     Running `target/debug/simulation_client`
stream starting
stream finished
    Finished dev [unoptimized + debuginfo] target(s) in
    0.02s
     Running `target/debug/simulation_client`
stream starting
stream finished
    Finished dev [unoptimized + debuginfo] target(s) in
    0.01s
     Running `target/debug/simulation_client`
stream starting
stream finished
```

If we wait for 7 seconds, we can inspect our server terminal, which should have the following printout:

```
Incoming connection from: 127.0.0.1:57716
thread starting 127.0.0.1:57716 starting
Incoming connection from: 127.0.0.1:57717
thread starting 127.0.0.1:57717 starting
Incoming connection from: 127.0.0.1:57718
thread starting 127.0.0.1:57718 starting
thread 127.0.0.1:57716 finishing
thread 127.0.0.1:57717 finishing
thread 127.0.0.1:57718 finishing
```

More books: https://t.me/booksforits

Here, we can see that each process has a port on the local host. As expected with spawning threads to handle the incoming messages, the messages were handled in an async manner. We could handle many more connections if needed.

Now that we have managed to accept bytes through TCP, we will process these bytes in the next section.

Processing bytes

Why do we send bytes through the TCP channel as opposed to the string itself? We send bytes because they have a standardized way of being encoded and decoded. For instance, in this chapter, we are creating a client that is written in Rust. However, the client might be written in JavaScript or Python. Primitive data structures such as string characters can be encoded into bytes and then decoded when received by the TCP server. Because of the UTF-8 standard, we can use these strings anywhere. Our data could be saved in a file by one text editor and loaded by another text editor because they are both using the same encoding.

If we keep exploring the concept of bytes, we will conclude that the only data that a computer can store is bytes. MP3, WAV, JPEG, PNG, and so on are all examples of encoding. If you save any file, you will encode the data into bytes. If we load any file, we will be decoding the data from bytes. Now, let us decode our byte string that was sent over TCP.

In our `main.rs` file in the TCP server project, we first need to import the following:

```
use tokio::io::{BufReader, AsyncBufReadExt, AsyncWriteExt};
```

The `BufReader` struct essentially adds a buffer to any reader, which improves the speed of small frequent reads from the same file or socket. While this greatly helps us as we are expecting multiple small messages to be sent down the same TCP socket, it will not give us any speed improvement if we are trying to read a large stream of data in one go from a file or socket. The other two import traits must be imported to enable the `BufReader` struct to read or write.

Now, we must wipe all code in our thread spawn section of the code to start again as we will be doing a range of different processes. First, inside our thread spawn code, we must print out that we are starting the thread by splitting our stream into a reader and writer, and then creating our buffer reader from the reader and an empty vector to store the processed incoming data:

```
println!("thread starting {} starting", peer.to_string());
let (reader, mut writer) = stream.split();
let mut buf_reader = BufReader::new(reader);
let mut buf = vec![];
```

More books: https://t.me/booksforits

Now that we have everything ready for reading, we can continuously read one line at a time, telling the reader to stop reading once it hits the EOF condition, `'b\n'`, from our stream:

```
loop {
    match buf_reader.read_until(b'\n', &mut buf).await {
        Ok(n) => {
            . . .
        },
        Err(e) => println!("Error receiving message: {}", e)
    }
}
```

We can see that if there is an error, we will print it out. However, the rest of the code that we are concerned with when it comes to processing bytes is in our `Ok` code block.

Inside our `Ok` code block, we initially need to check to see if the stream has closed by checking if zero bytes are received:

```
if n == 0 {
    println!("EOF received");
    break;
}
```

`EOF` stands for *end-of-file*. EOF is the standard way of declaring that we have come to the end of the file or that the data stream has finished. Once we get past the preceding code block, we know that we have some bytes to process. We must convert our incoming bytes into a string using the UTF-8 encoding:

```
let buf_string = String::from_utf8_lossy(&buf);
```

The lossy reference in the preceding code is where nonstandard characters are replaced with a placeholder, so nonstandard characters are lost in translation. This is not going to be an issue for us as we are sending over standard characters. With our string data, we are going to separate values in a message with a `;` delimiter. We will split our string into a vector of strings, replacing all new lines with the following code:

```
let data: Vec<String> = buf_string.split(";")
                                  .map(|x| x.to_string()
                                  .replace("\n", ""))
                                  .collect();
```

More books: https://t.me/booksforits

Now, we can print out the processed message and then clear the buffer so that the line that we are processing does not get caught up in the next processing step:

```
println!(
    "Received message: {:?}",
    data
);
buf.clear();
```

We are now at the end of the loop. Right outside the loop, we must print out that the thread is finished with the following code:

```
println!("thread {} finishing", peer.to_string());
```

With that, we have finished our server for now as we can process bytes. Now, we can go to the `main.rs` file in our client project and write the following byte string:

```
println!("stream starting");
stream.write_all(b"one;two\nthree;four").await?;
println!("stream finished");
```

What outcome do you think we will get from this byte string? If we look at the byte string, we will see that there is a new line, so we are sending two messages in one packet over the TCP connection. Each message has two values due to the `;` delimiter. When we spin up our server and run the client, we will get the following printout:

```
Incoming connection from: 127.0.0.1:59172
thread starting 127.0.0.1:59172 starting
Received message: ["one", "two"]
Received message: ["three", "four"]
EOF received
thread 127.0.0.1:59172 finishing
```

We can see that our server processed two messages in the same thread before we closed the thread. With this, we can see that we have a lot of flexibility with TCP sockets. We are now ready to route TCP traffic to our actors.

More books: https://t.me/booksforits

Passing TCP to an actor

When it comes to routing TCP data to actors, we need to import our actors and channels into the `main.rs` file in our server project with the following code:

```
. . .
use tokio::sync::mpsc;

mod actors;
use actors::{OrderBookActor, BuyOrder, Message};
. . .
```

Now, we have to construct our *order book* actor and run it. However, as you may recall, we merely ran the *order book* actor at the end of the Tokio runtime. However, if we apply this strategy here, we will block the loop from executing, so we can listen to incoming traffic. If we run the *order book* actor after the loop, the *order book* actor will never run as the loop runs indefinitely in a `while` loop and thus blocks the execution of any code following it. In our case, there is a further complication. This complication is that the actor run function enters a `while` loop, which further explains the need to put this entire code in a separate spawned Tokio task. Because of this, we must spawn a thread before the loop:

```
#[tokio::main]
async fn main() {
    let addr = "127.0.0.1:8080".to_string();

    let mut socket =
        TcpListener::bind(&addr).await.unwrap();
    println!("Listening on: {}", addr);

    let (tx, rx) = mpsc::channel::<Message>(1);

    tokio::spawn(async move {
        let order_book_actor = OrderBookActor::new(rx,
                                                   20.0);
        order_book_actor.run().await;
    });
    println!("order book running now");

    while let Ok((mut stream, peer)) =
```

More books: https://t.me/booksforits

```
        socket.accept().await {
        println!("Incoming connection from: {}",
                  peer.to_string());
        let tx_one = tx.clone();
        . . .
```

Note that we directly move the `tx` receiver into the Tokio task without cloning it because `OrderBookActor` is the only actor taking full exclusive ownership of the receiver. Here, we can see that the TCP listener is the same. Then, we create the `mpsc` channel, which we use to create and run our *order book* actor in the first thread that we spawn. Then, we enter the `while` loop to listen for TCP traffic. Note that the clone of the sender of the mpsc channel for the *order book* actor is cloned straight away. This is because we are going to have to clone it again in the reading loop on every next iteration.

Inside the `Ok` block in our loop, we process our byte string, create the new *buy order* actor, and then send the message to the *order book* actor:

```
. . .
let data: Vec<String> = buf_string.split(";")
    .map(|x| x.to_string().replace("\n", "")).collect();
let amount = data[0].parse::<f32>().unwrap();
let order_actor = BuyOrder::new(amount, data[1].clone(),
                                tx_one.clone());
println!("{}: {}", order_actor.ticker, order_actor.amount);
order_actor.send().await;
buf.clear();
```

And this should be it. We can see that the actors that we defined in a standard Tokio runtime without listening to traffic can be plugged into our TCP network application. There is only one thing left to do, and that is to update the message being sent in the client `main.rs` file with the following content:

```
. . .
println!("stream starting");
stream.write_all(b"8.0;BYND;\n9.0;PLTR").await?;
println!("stream finished");
. . .
```

More books: https://t.me/booksforits

Here, we are sending two buy orders (`BYND` and `PLTR`). If we run our server and then run our client, we will get the following printout for the server:

```
Listening on: 127.0.0.1:8080
order book running now
actor is running
Incoming connection from: 127.0.0.1:59769
thread starting 127.0.0.1:59769 starting
BYND: 8
processing purchase, total invested: 8
here is the outcome: 1
PLTR: 9
processing purchase, total invested: 17
here is the outcome: 1
EOF received
thread 127.0.0.1:59769 finishing
```

With this printout, we can see that we run our *order book* actor and listen to TCP traffic before we process incoming traffic. Then, we accept our packet, process the data, and send our data into the actor system. Overall, our application flow takes the following form:

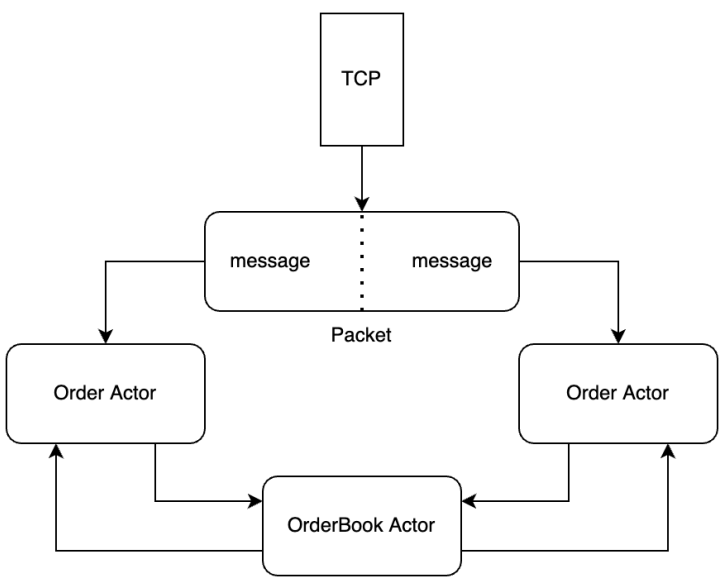


Figure 15.2 – Our TCP application flow

More books: https://t.me/booksforits

With this, we now have a network application that accepts TCP traffic and passes the processed data from the incoming bytes to our actor system. We create *buy order* actors on the fly when we accept a new message via TCP, while our *order book* actor continues to run throughout the lifetime of our program. If we were to add another order book or a different type of actor, we could simply spin off another thread where we construct and run the actor. There is no limit to this, so our system can scale.

Right now, our client does not know what happened. Therefore, our server must reply to our client about what happened. However, before we can do this, we must keep track of our stock orders so that we can return the state of our orders when we need them.

Keeping track of orders with actors

When it comes to keeping track of our orders, we could simply add a HashMap to our order book and add a couple of other messages that can be sent to the *order book* actor. This is one approach. We are getting into territory where there are no clear correct approaches, and people within the community debate on the best approaches to solve problems. In this chapter, we will get used to creating actors and managing multiple actors in Tokio by creating two new actors. One actor will merely keep track of our stock purchases, while the other actor will send messages to the order tracker to get the state of our orders.

First, we need to create a separate file in `src/order_tracker.rs`. In this file, we initially need to import what we need to handle the collection of stocks and the channels that enable the connections between the actors:

```
use tokio::sync::{mpsc, oneshot};
use std::collections::HashMap;
```

Then, we need to create the message struct for the messages sent to our tracker actor:

```
#[derive(Debug, Clone)]
pub enum Order {
    BUY(String, f32),
    GET
}
pub struct TrackerMessage {
    pub command: Order,
    pub respond_to: oneshot::Sender<String>
}
```

Here, we need to pass in a command. This command is needed because the tracker actor can perform multiple actions, such as `BUY` and `GET`. If the command is just a `GET`, then we do not need anything else, which is why the rest of the fields are optional.

More books: https://t.me/booksforits

With this message defined, we can build the most basic actor, which merely sends a `get` message to the tracker actor and returns the state of our orders:

```
pub struct GetTrackerActor {
    pub sender: mpsc::Sender<TrackerMessage>
}
```

Here, we can see that no state is held by `GetTrackerActor`. We could just make this whole actor a function in another actor. However, as stated, we want to get comfortable with managing multiple actors in an async system in this chapter. To enable our `GetTrackerActor` to get data, we must create a `send` function that will send a `GET` command to the tracker actor and return the state of the tracker actor as a string:

```
impl GetTrackerActor {
    pub async fn send(self) -> String {
        println!("GET function firing");
        let (send, recv) = oneshot::channel();
        let message = TrackerMessage {
            command: Order::GET,
            respond_to: send
        };
        let _ = self.sender.send(message).await;
        match recv.await {
            Err(e) => panic!("{}", e),
            Ok(outcome) =>  return outcome
        }
    }
}
```

This method should be familiar to you now. We created a one-shot channel so that the tracker actor could send a message back to the `GetTrackerActor` actor. Then, we sent the `GET` message and waited for a response. You may have also noticed that we are printing out that the `send` function is firing. We will be peppering the code with print statements throughout so that we can track how the `async` code runs in the printout and in what order.

We are now at the stage where to need to create our *order tracker* actor. We need a HashMap and a channel to receive messages, which we can create with the following code:

```
pub struct TrackerActor {
    pub receiver: mpsc::Receiver<TrackerMessage>,
```

More books: https://t.me/booksforits

```
    pub db: HashMap<String, f32>
}
```

This actor is more complex as we need the actor to run to receive messages, handle the messages, and send the state of the order, which takes the following outline:

```
impl TrackerActor {
    pub fn new(receiver: mpsc::Receiver<TrackerMessage>) ->
        Self {
        . . .
    }
    fn send_state(&self, respond_to: oneshot::
                  Sender<String>) {
        . . .
    }
    fn handle_message(&mut self, message: TrackerMessage) {
        . . .
    }
    pub async fn run(mut self) {
        . . .
    }
}
```

If you wish to test your handle on actors, now is a good time to try and implement the preceding functions.

If you have attempted these functions, they should be implemented similarly to what's shown in the following code, which we are going to cover here. First, our constructor takes the following form:

```
pub fn new(receiver: mpsc::Receiver<TrackerMessage>) ->
    Self {
    TrackerActor {
        receiver,
        db: HashMap::new(),
    }
}
```

This constructor should not be a surprise for anyone. We need a HashMap with a string as a key to denote the ticker and a float for the number of stocks we own for that ticker. We also accept a channel receiver to receive messages.

More books: https://t.me/booksforits

The next process we need to define is how to package our data into a string so that we can send it over TCP. We can do this with the following code:

```
fn send_state(&self, respond_to: oneshot::Sender<String>) {
    let mut buffer = Vec::new();
    for key in self.db.keys() {
        let amount = self.db.get(key).unwrap();
        buffer.push(format!("{}:{ };", &key, amount));
    }
    buffer.push("\n".to_string());
    println!("sending state: {}", buffer.join(""));
    respond_to.send(buffer.join(""));
}
```

Here, we create a vector that holds our data. Then, we loop through our HashMap, which is logging our stock holdings. We can see that we separate the ticker from the amount with a `:`, and then we separate the individual stock tickers with counts with a `;`. At this point, our response should be something like `"BYND:8;PLTR:9;\n"`, which means that we have 8 of BYND and 9 of PLTR. Once we have stored the entire state in a vector of strings, we join the vector into one string and then send that string over a channel.

We now have everything we need to handle an incoming message, which can be handled with the following code:

```
fn handle_message(&mut self, message: TrackerMessage) {
    match message.command {
        Order::GET => {
            println!("getting state");
            self.send_state(message.respond_to);
        },
        Order::BUY(ticker, amount) => {
            match self.db.get(&ticker) {
                Some(ticker_amount) => {
                    self.db.insert(ticker, ticker_amount +
                                   amount);
                },
                None => {
                    self.db.insert(ticker, amount);
                }
```

More books: https://t.me/booksforits

```
                }
                println!("db: {:?}", self.db);
            }
        }
    }
```

Here, we match the command being passed via the incoming message. If a `GET` command is passed, we merely return the state with the address to respond to extracted from the incoming message. If a `BUY` command is passed, we extract the parameters for the purchase order from the message and try and get the ticker from the HashMap. If the ticker is not present, we create a new entry. If the ticker is present, we merely increase the count of the ticker that we have bought.

We have now handled our messages and state. There is only one thing left to do and that is to run the actor; this can be achieved with the following code:

```
pub async fn run(mut self) {
    println!("tracker actor is running");
    while let Some(msg) = self.receiver.recv().await {
        self.handle_message(msg);
    }
}
```

With this, our tracker actor is fully working, so it is time to stand back and look at our system and how we envision it to work, as shown in the following diagram:

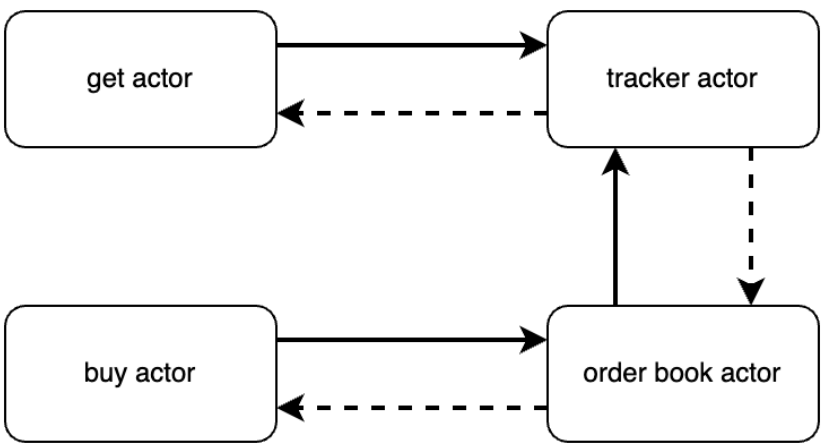


Figure 15.3 – Interactions between actors

More books: https://t.me/booksforits

Here, we can see that there must be an interaction between the *order book* actor and the *tracker actor* when a buy order is executed. Therefore, we need to refactor our *order book* actor to enable chained communication between multiple actors.

Chaining communication between actors

As we can see in *Figure 15.2*, our *order book* actor is running and accepting orders. The *order book* actor then sends a message to the *tracker* actor, updating the state once the `BUY` order is processed. This means that our actor needs to manage two channels. To handle two channels, inside the `src/actors.rs` file, we need to import the tracker message with the following code:

```
use tokio::sync::{mpsc, oneshot, mpsc::Sender};
use crate::order_tracker::TrackerMessage;
```

Now, we must hold two channels, resulting in our `OrderBookActor` struct having the following fields:

```
pub struct OrderBookActor {
    pub receiver: mpsc::Receiver<Message>,
    pub sender: mpsc::Sender<TrackerMessage>,
    pub total_invested: f32,
    pub investment_cap: f32
}
```

Here, the fields are essentially the same, but we are holding onto a sender that sends messages to the tracker. We can see how helpful different messages are. We know exactly where the message is destined to be. With this extra field, we need to slightly change the constructor for `OrderBookActor` with the following code:

```
pub fn new(receiver: mpsc::Receiver<Message>,
           sender: mpsc::Sender<TrackerMessage>,
           investment_cap: f32) -> Self {
    OrderBookActor {
        receiver, sender,
        total_invested: 0.0,
        investment_cap
    }
}
```

More books: https://t.me/booksforits

There is only one other behavior that we must add. Remember that we process our incoming messages in the `handle_message` function. Here, we must send a `TrackerMessage` to the tracker actor with the following code:

```
async fn handle_message(&mut self, message: Message) {
    if message.amount + self.total_invested >=
        self.investment_cap {
        println!("rejecting purchase, total invested: {}",
                 self.total_invested);
        let _ = message.respond_to.send(0);
    }
    else {
        self.total_invested += message.amount;
        println!("processing purchase, total invested: {}",
                 self.total_invested);
        let _ = message.respond_to.send(1);

        let (send, _) = oneshot::channel();
        let tracker_message = TrackerMessage{
            command: "BUY".to_string(),
            ticker: Some(message.ticker),
            amount: Some(message.amount),
            respond_to: send
        };
        let _ = self.sender.send(tracker_message).await;
    }
}
```

As we can see, the logic for deciding whether a buy order is going to be processed is the same, but if the buy order is processed, we merely construct a `TrackerMessage` and send it to the *tracker* actor.

Now that our actors have been built and refactored, our actor system will behave as shown in *Figure 15.2*. We can now implement our new actor system so that we can respond to the TCP traffic with TCP.

More books: https://t.me/booksforits

Responding with TCP

When it comes to responding to TCP, we must implement our actor system in the `src/main.rs` file. First, we need to import our new actors with the following code:

```
. . .
use order_tracker::{TrackerActor, GetTrackerActor,
                    TrackerMessage};
```

Now, we must construct our extra channel in the `main` function with the following code:

```
let addr = "127.0.0.1:8080".to_string();
let socket = TcpListener::bind(&addr).await.unwrap();
println!("Listening on: {}", addr);
let (tx, rx) = mpsc::channel::<Message>(1);
let (tracker_tx, tracker_rx) =
    mpsc::channel::<TrackerMessage>(1);
let tracker_tx_one = tracker_tx.clone();
```

Here, we have a `tracker` channel. With the `tracker` and `main` channels, we can spin up two different threads with the *tracker* actor and *order book* actor with the following code:

```
tokio::spawn( async {
    TrackerActor::new(tracker_rx).run();
});
tokio::spawn(async move {
    let order_book_actor = OrderBookActor::new(
        rx, tracker_tx_one.clone(), 20.0);
    order_book_actor.run().await;
});
```

With this, we now have two of our actors running, awaiting incoming messages. Now, we must manage our incoming TCP traffic and spin up different actors, depending on the command passed in. As a design choice, we are going to have the first string passed in via TCP to be the command that our application has:

```
let buf_string = String::from_utf8_lossy(&buf);
let data: Vec<String> = buf_string.split(";")
    .map(|x| x.to_string().replace("\n", "")).collect();
```

More books: https://t.me/booksforits

```
println!("here is the data {:?}", data);
let command = data[0].clone();
```

Then, we must match our command with the following code:

```
match command.as_str() {
    "BUY" => {
        . . .
    },
    "GET" => {
        . . .
    },
    _ => {
        panic!("{} command not supported", command);
    }
}
buf.clear();
```

For our buy order, we still simply spin up the *buy order* actor and send it to the *order book* actor:

```
println!("buy order command processed");
let amount = data[1].parse::<f32>().unwrap();
let order_actor = BuyOrder::new(amount, data[2].clone(),
                                tx_one.clone());
println!("{}: {}", order_actor.ticker, order_actor.amount);
order_actor.send().await;
```

The main change here is how we manage the incoming data, and this is because we have introduced the `command` parameter. For the `get` command, we create `GetTrackerActor`, which sends a message to the tracker actor. Then, we write the state that we got from the *tracker* actor with the following code:

```
println!("get order command processed");
let get_actor =
    GetTrackerActor{sender: tracker_tx_two.clone()};
let state = get_actor.send().await;
println!("sending back: {:?}", state);
writer.write_all(state.as_bytes()).await.unwrap();
```

More books: https://t.me/booksforits

With this, our server can now accept different commands and track all the buy orders that we have made.

Even though our server is now fully functional, our client will not work. This is because we have not updated our client with the commands. In the next section, we will update our client, which can send multiple different commands.

Sending different commands via the client

Our client is simple, and it is going to stay simple. First, we must ensure our read and write traits are imported as this time, we will be reading a response. Our imports in the `src/main.rs` file should look like this:

```
use tokio::net::TcpStream;
use tokio::io::{BufReader, AsyncBufReadExt, AsyncWriteExt};
use std::error::Error;
```

Then, we must write a series of messages to our connection and then read until we get a new line:

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    let mut stream =
        TcpStream::connect("127.0.0.1:8080").await?;
    let (reader, mut writer) = stream.split();
    println!("stream starting");
    writer.write_all(b"BUY;8.0;BYND;\nBUY;9.0;PLTR\n
                     BUY;9.0;PLTR\nGET\n").await?;
    println!("sent data");

    let mut buf_reader = BufReader::new(reader);
    let mut buf = vec![];
    println!("reading data");
    let _ = buf_reader.read_until(b'\n',
                                  &mut buf).await.unwrap();
    let state_string = String::from_utf8_lossy(&buf);
    println!("{}", state_string);
    Ok(())
}
```

More books: https://t.me/booksforits

With this, we are done. All we must do now is run the server and then run the client. The client has the following printout:

```
stream starting
sent data
reading data
PLTR:9;BYND:8;
```

Here, we get the state of our stock orders after sending our orders. Even though this state is in a single string, we have delimiters so that we can split our data up into something useful. Once our client has run, our server will have the following printout:

```
Incoming connection from: 127.0.0.1:61494
thread starting 127.0.0.1:61494 starting
here is the data ["BUY", "8.0", "BYND", ""]
buy order command processed
BYND: 8
processing purchase, total invested: 8
db: {"BYND": 8.0}
here is the outcome: 1
here is the data ["BUY", "9.0", "PLTR"]
buy order command processed
PLTR: 9
processing purchase, total invested: 17
db: {"PLTR": 9.0, "BYND": 8.0}
here is the outcome: 1
here is the data ["BUY", "9.0", "PLTR"]
buy order command processed
PLTR: 9
rejecting purchase, total invested: 17
here is the outcome: 0
here is the data ["GET"]
get order command processed
GET function firing
getting state
sending state: PLTR:9;BYND:8;

sending back: "PLTR:9;BYND:8;\n"
```

More books: https://t.me/booksforits

```
EOF received
thread 127.0.0.1:61494 finishing
```

This is a long printout, but we can see how our order message got converted into a vector. We can also see how the state of our tracker changes over time, and at the end, we can see how our `get` command is processed with the *get state* actor and *tracker* actor.

Summary

In this chapter, we managed to bring our Tokio async program to the next level by accepting incoming TCP traffic. Then, we processed our TCP traffic, which was packaged in bytes, essentially creating a protocol for processing buy orders for stocks. We must note that we have a lot of flexibility with this approach. We managed to stuff multiple buy orders and then a `get` command into one message. We can become creative with our message structure as there is little constraint in how to package our message and unpack it in the server if the protocols are consistent between the server and the client.

Then, we added more threads and actors to our system to handle multiple commands passed into our server. We finished up this chapter by updating our client and returning the state of our orders. The result is a highly async-safe network application that accepts messages via TCP. This network application is not just for running on our local computers. We can wrap this TCP Tokio network application in Docker and deploy it on a server. You now have the tools to build lower-level network applications to aid your web applications. Considering that our distroless Rust servers are roughly 50 MB in size, these network applications will be an inexpensive aid to whatever problems you are trying to solve.

While having a protocol is useful and gives us more freedom, we will take our protocol processing of TCP traffic to the next level in the next chapter with framing, enabling us to have even more control over how we process and package our messages as they're being sent via TCP.

Further reading

Tokio TCP documentation: `https://docs.rs/tokio/latest/tokio/net/struct.TcpStream.html`.

Questions

1. How do we create an actor that accepts messages and sends messages to other actors?
2. Why do we need to spin our long-running actors up in their own threads?
3. How can we have multiple actors processing the same type of job?

More books: https://t.me/booksforits

Answers

1. We create an actor that has a minimum of two fields. These two fields hold the sender for the channel of the actor we are sending messages to and the receiver of the channel we are receiving messages from. Then, we need a `run` function to enable our actor to run awaiting incoming messages.
2. If we do not create a thread to run our long-running actors, our main runtime will be blocked by this actor running. If only one actor is running after the server is listening, this is OK; however, if there are multiple actors or a loop that is accepting TCP traffic, then we have a problem as the system will essentially be gridlocked and our actor system will not work.
3. We can build an actor that essentially acts like a router. It can keep track of incoming messages and alternate sending messages to multiple different actors who do the same type of job. However, do not do this if multiple actors rely on the internal state.

More books: https://t.me/booksforits

16
Building Protocols on Top of TCP

In the previous chapter, we used the Tokio framework to support an async actor model. Our Tokio framework accepted basic traffic and then sent those messages to actors once the messages were processed. However, our TCP processing was basic. You should not be comfortable building complex systems on this basic TCP process if this book is the only exposure you have had to TCP. In this chapter, we will completely focus on how to package, send, and read data over a TCP connection.

In this chapter, we will cover the following topics:

- Setting up a TCP client and echo server
- Processing bytes over TCP using structs
- Creating frames to separate messages over TCP
- Building an HTTP frame on top of TCP

By the end of this chapter, you will be able to package, send, and read data sent over TCP using a range of different approaches. You will be able to understand how to split your data into frames that can be handled as structs. Finally, you will be able to build an HTTP frame that has a header containing the URL and method, and a body containing data. This will enable you to build whatever data structure you require when sending data over TCP.

Technical requirements

In this chapter, we will be purely focusing on how to process data over a TCP connection. Therefore, we will not be relying on any previous code as we are building our own echo server.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter16`.

More books: https://t.me/booksforits

Setting up our TCP client and server

To explore sending and processing bytes over TCP, we will create a basic echo server and client. We will be dropping any complex logic that we built in the previous chapter because we do not need the distraction of the complex logic when trying to explore the ideas around sending, receiving, and processing bytes.

In a new directory, we should have two cargo projects – one for the server and another for the client. They can take the following file structure:

```
├── client
│   ├── Cargo.toml
│   └── src
│       └── main.rs
└── server
    ├── Cargo.toml
    └── src
        └── main.rs
```

Both projects will be using the same dependency of Tokio, so both projects should have the following dependency defined in their `Cargo.toml` file:

```
[dependencies]
tokio = { version = "1", features = ["full"] }
```

We now need to construct the basic mechanism of an echo server. This is where a message is sent to the server from the client. The server then processes the message sent by the client, re-packages the message, and sends the same message back to the client. We will start by building out our server.

Setting up our TCP server

We can define the server in the `server/src/main.rs` file by initially importing everything we need with the following code:

```
use tokio::net::TcpListener;
use tokio::io::{BufReader, AsyncBufReadExt, AsyncWriteExt};
```

This is so we can listen to incoming TCP traffic, read bytes from that traffic, and write it back to the client sending the message. Then, we need to utilize the Tokio runtime to listen to incoming traffic, spinning up a thread if we get a message. If you completed the previous chapter, this is a good opportunity for you to attempt to do this yourself as we covered the concepts needed to create a TCP server listening for incoming traffic.

More books: https://t.me/booksforits

If you have attempted to write the basics for your TCP server, your code should look like this:

```
#[tokio::main]
async fn main() {
    let addr = "127.0.0.1:8080".to_string();

    let socket = TcpListener::bind(&addr).await.unwrap();
    println!("Listening on: {}", addr);

    while let Ok((mut stream, peer)) =
        socket.accept().await {
        println!("Incoming connection from: {}",
                  peer.to_string());

        tokio::spawn(async move {
            . . .
        });
    }
}
```

Here, we should be familiar with the concept that we create a listener, bind it to an address, and then wait for incoming messages, creating a thread when a message is sent. Inside the thread, we loop through the incoming message until there is a new line with the following code:

```
println!("thread starting {} starting", peer.to_string());
let (reader, mut writer) = stream.split();

let mut buf_reader = BufReader::new(reader);
let mut buf = vec![];

loop {
    match buf_reader.read_until(b'\n', &mut buf).await {
        Ok(n) => {
            if n == 0 {
                println!("EOF received");
                break;
            }
            let buf_string = String::from_utf8_lossy(&buf);
```

More books: https://t.me/booksforits

```
            writer.write_all(buf_string.as_bytes())
                .await.unwrap();
            buf.clear();
        },
        Err(e) => println!("Error receiving message: {}", e)
    }
}
println!("thread {} finishing", peer.to_string());
```

This code should not be a surprise by now. If you are unfamiliar with any of the concepts the preceding code covers, it is advised that you read the previous chapter.

We now have a basic echo server defined, and it is ready to run, which means we can turn our attention to creating our client code in the `client/src/main.rs` file. The same structs and traits will be needed to get the client working. Then, we need to send a standard text message to the TCP server. This is a good time to try and implement the client yourself, and there is nothing that has not been covered multiple times before to build the client.

Setting up our TCP client

If you attempted to build the client yourself, you should have imported the following structs and traits:

```
use tokio::net::TcpStream;
use tokio::io::{BufReader, AsyncBufReadExt, AsyncWriteExt};
use std::error::Error;
```

Then, we must make the TCP connection, send a message, wait for the message to be sent back, and then print it out using the Tokio runtime:

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    let mut stream =
        TcpStream::connect("127.0.0.1:8080").await?;
    let (reader, mut writer) = stream.split();

    println!("stream starting");
    writer.write_all(b"this is a test\n").await?;
    println!("sent data");
```

More books: https://t.me/booksforits

```
    let mut buf_reader = BufReader::new(reader);
    let mut buf = vec![];
    println!("reading data");
    let _ = buf_reader.read_until(b'\n', &mut
        buf).await.unwrap();
    let message = String::from_utf8_lossy(&buf);
    println!("{}", message);
    Ok(())
}
```

We now have a functioning client and server. To test if the client and server are working, we must start the server in one terminal, and then run the client in another terminal by running `cargo run`. The server will have the following printout:

```
stream starting
sent data
reading data
this is a test
```

Our server printout will have the following printout:

```
Listening on: 127.0.0.1:8080
Incoming connection from: 127.0.0.1:60545
thread starting 127.0.0.1:60545 starting
EOF received
thread 127.0.0.1:60545 finishing
```

With that, we have a basic echo server and client working. We can now focus on packaging, unpacking, and processing bytes. In the next section, we will explore the basic method of using structs to standardize processing messages.

Processing bytes using structs

In the previous chapter, we were sending strings to the server. However, the result is that we had to parse individual values into the type that we needed. As string parsing was not well structured, it is not clear to other developers what structure our messages are. We can make our message structure clearer by defining a struct that can be sent over the TCP channel. Sending a struct over a TCP channel can be achieved by converting the message struct into a binary format before sending the struct itself. This is also known as serializing the data.

More books: https://t.me/booksforits

If we are to convert a struct into a binary format, first, we will need to utilize the `serde` and `bincode` crates. With our new crates, both the client and server `Cargo.toml` file should contain the following dependencies:

```
[dependencies]
serde = { version = "1.0.144", features = ["derive"] }
tokio = { version = "1", features = ["full"] }
bincode = "1.3.3"
```

The `serde` crate will be used to serialize the struct, while the `bincode` crate will be used to convert our message struct into a binary format. Now that our dependencies have been defined, we can start creating a message sender client.

Creating a message sender client

We can build out our `client/src/main.rs` file to send structs over TCP. First, we must import what we need:

```
. . .
use serde::{Serialize, Deserialize};
use bincode;
```

With our imports ready, we can define our `Message` struct with the following code:

```
#[derive(Serialize, Deserialize, Debug)]
struct Message {
    pub ticker: String,
    pub amount: f32
}
```

The definition of our `Message` struct takes a similar form to the structs that we use to process the JSON bodies of HTTP requests on our Actix server. However, this time, we will not be using the Actix Web structs and traits to process the struct.

Our `Message` struct can now be used in our `main` function. Remember that inside our `main` function, we have a TCP stream that was created by the following code:

```
let mut stream = TcpStream::connect("127.0.0.1:8080").await?;
let (reader, mut writer) = stream.split();
```

More books: https://t.me/booksforits

Now that we have made a connection, we can create our `Message` struct and convert the `Message` struct into binary format:

```
let message = Message{ticker: String::from("BYND"),
                      amount: 3.2};
let message_bin = bincode::serialize(&message).unwrap();
```

Our `Message` struct is now in binary format. Then, we must send our message over the TCP stream with the following code:

```
println!("stream starting");
writer.write_all(&message_bin).await?;
writer.write_all(b"\n").await?;
println!("sent data");
```

Note that we have sent the message and then a new line. This is because our server is going to read until there is a new line. If we do not send the new line, then the program will hang and never complete.

Now that we have sent our message, we can wait until we see receive the message back again. Then, we must construct the `Message` struct from the binary format and print out the constructed `Message` struct with the following code:

```
let mut buf_reader = BufReader::new(reader);
let mut buf = vec![];
println!("reading data");
let _ = buf_reader.read_until(b'\n',
    &mut buf).await.unwrap();
println!("{:?}", bincode::deserialize::<Message>(&buf));
```

Our client is now ready to send a message to the server.

Processing messages in the server

When it comes to updating our server code, we will aim to unpack the message, print out the message, and then convert the message into binary format to be sent back to the client. At this stage, you should be able to implement the change yourself. This is a good opportunity to revise what we have covered.

If you did attempt to implement the processing of the message on the server in the `server/src/main.rs` file first, you should have imported the following additional requirements:

```
use serde::{Serialize, Deserialize};
use bincode;
```

More books: https://t.me/booksforits

Then, you should have defined the `Message` struct, like so:

```
#[derive(Serialize, Deserialize, Debug)]
struct Message {
    pub ticker: String,
    pub amount: f32
}
```

Now, we only need to process the message, print out the message, and then return the message to the client. We can manage all the processes with the message in the loop inside the thread with the following code:

```
let message =
    bincode::deserialize::<Message>(&buf).unwrap();
println!("{:?}", message);
let message_bin = bincode::serialize(&message).unwrap();
writer.write_all(&message_bin).await.unwrap();
writer.write_all(b"\n").await.unwrap();
buf.clear();
```

Here, we are using the same approach that we used in the client but vice versa – that is, we convert from binary format first and then convert into binary format at the end.

If we run our server and then our client, our server will give us the following printout:

```
Listening on: 127.0.0.1:8080
Incoming connection from: 127.0.0.1:50973
thread starting 127.0.0.1:50973 starting
Message { ticker: "BYND", amount: 3.2 }
EOF received
thread 127.0.0.1:50973 finishing
```

Our client gives us the following printout:

```
stream starting
sent data
reading data
Ok(Message { ticker: "BYND", amount: 3.2 })
```

More books: https://t.me/booksforits

Here, we can see that our `Message` struct can be sent, received, and then sent back again without any compromise. This gives our TCP traffic another level of sophistication as we can have more complex structures for our messages. For instance, one field of our message could be a HashMap, and another field of the message could be a vector of another struct if the struct in the vector has implemented the `serde` traits. We can chop and change the structure of our `Message` struct without having to rewrite our protocol for unpacking and packing a message. Other developers can merely look at our `Message` struct and know what is being sent over the TCP channel. Now that we have improved how we send messages over TCP, we can chunk our stream into frames with framing.

Utilizing framing

So far, we are sending structs over TCP and separating these messages with a new line. Essentially, this is the most basic form of framing. However, there are some drawbacks. We must remember to put in a delimiter such as a new line; otherwise, our program will hang indefinitely. We also run the risk of prematurely splitting the message into two messages by having a delimiter in the data of the message. For instance, when we split our messages up with the new line delimiter, it is not inconceivable to have a chunk of text in a message that has new lines or any special character or byte to denote the need to separate the stream into serializable packages. To prevent such issues, we can use the built-in framing support that Tokio provides.

In this section, we will be rewriting the client and server as the sending and receiving of messages will change. If we try and insert our new approach into existing code of the client, it can easily lead to confusion. Before we write our client and server, we must update the dependencies in the `Cargo.toml` file of both the client and server:

```
[dependencies]
tokio-util = {version = "0.7.4", features = ["full"] }
serde = { version = "1.0.144", features = ["derive"] }
tokio = { version = "1", features = ["full"] }
futures = "0.3.24"
bincode = "1.3.3"
bytes = "1.2.1"
```

Here, we are using a couple more crates. We will cover their needs as we go through the rest of the code in this section. To get to grips with framing, we will start with a simple task, which is rewriting our client so that it supports framing.

More books: https://t.me/booksforits

Rewriting our client so that it supports framing

Remember we are writing our entire client in the `client/src/main.rs` file. First, we must import what we need from Tokio with the following code:

```
use tokio::net::TcpStream;
use tokio_util::codec::{BytesCodec, Decoder};
```

`TcpStream` is for connecting to our server. The `BytesCodec` struct is for shipping raw bytes through connections. We will be using the `BytesCodec` struct to configure the framing. `Decoder` is a trait that decodes bytes that we accept through our connection. However, when it comes to sending data through a connection, we could pass in structs, strings, or anything else that must be converted into bytes. Therefore, we must inspect what is implemented for the `BytesCodec` struct by looking at the source code for `BytesCodec`. The source code can be inspected by looking at the documentation or merely control-clicking or hovering over the `BytesCodec` struct in your editor. When we inspect the source code of the `BytesCodec` struct, we will see the following `Encode` implementations:

```
impl Encoder<Bytes> for BytesCodec {
    . . .
}
impl Encoder<BytesMut> for BytesCodec {
    . . .
}
```

Here, we can only send `Bytes` or `BytesMut` through a connection using a `BytesCodec` struct. We could implement `Encode` for `BytesCodec` for sending other types of data; however, for our use case, this is excessive, and it just makes sense to send `Bytes` over our connection. However, before we write any more code, we might as well inspect the `Bytes` implementation to get an appreciation for how framing works. The implementation of `Encode` for `Bytes` takes the following form:

```
impl Encoder<Bytes> for BytesCodec {
    type Error = io::Error;
    fn encode(&mut self, data: Bytes, buf: &mut BytesMut)
              -> Result<(), io::Error> {
        buf.reserve(data.len());
        buf.put(data);
        Ok(())
    }
}
```

More books: https://t.me/booksforits

Here, we can see that the length of the data being passed is reserved in the buffer. The data is then put into the buffer.

Now that we understand how we are going to encode and decode our messages using framing, we need to import traits from both the `futures` and `bytes` crates to enable us to process our messages:

```
use futures::sink::SinkExt;
use futures::StreamExt;
use bytes::Bytes;
```

The `SinkExt` and `StreamExt` traits essentially enable us to receive messages from the stream asynchronously. The `Bytes` struct will wrap our serialized message to be sent. Then, we must import the traits to enable the serialization of messages and define our message struct:

```
use serde::{Serialize, Deserialize};
use bincode;

use std::error::Error;

#[derive(Serialize, Deserialize, Debug)]
struct Message {
    pub ticker: String,
    pub amount: f32
}
```

We now have everything we need to start working on our runtime. Remember that our main runtime has the following outline:

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    . . .
    Ok(())
}
```

Inside our runtime, we initially make a TCP connection and define the framing with the following code:

```
let stream = TcpStream::connect("127.0.0.1:8080").await?;
let mut framed = BytesCodec::new().framed(stream);
```

More books: https://t.me/booksforits

Then, we define our message, serialize the message, and then wrap the message in `Bytes` with the following code:

```
let message = Message{ticker: String::from("BYND"),
                      amount: 3.2};
let message_bin = bincode::serialize(&message).unwrap();
let sending_message = Bytes::from(message_bin);
```

Then, we can send our message, wait for the message to be sent back, and then deserialize the message to print it out with the following code:

```
framed.send(sending_message).await.unwrap();
let message = framed.next().await.unwrap().unwrap();
let message =
    bincode::deserialize::<Message>(&message).unwrap();
println!("{:?}", message);
```

With all this, our client is built. We can see that we do not have to worry about new lines or any other delimiters. Our code is clean and straightforward when it comes to sending and receiving messages over TCP. Now that our client has been built, we can move on to building our server so that it handles framing.

Rewriting our server so that it supports framing

When it comes to building our server to support framing, there is a lot of overlap with what we have coded in the previous section. At this point, it is a good time to try and build the server yourself. Building the server requires implementing the framing logic that we coded in the previous section into the existing server code.

If you attempted to rewrite the server, first, you should have imported the following structs and traits:

```
use tokio::net::TcpListener;
use tokio_util::codec::{BytesCodec, Decoder};

use futures::StreamExt;
use futures::sink::SinkExt;
use bytes::Bytes;

use serde::{Serialize, Deserialize};
use bincode;
```

More books: https://t.me/booksforits

Note that the `Decoder` trait that we imported allows us to call `.framed` on the bytes codec. There is nothing here that should be new to you. Once we have the necessary imports, we must define the same `Message` struct with the following code:

```
#[derive(Serialize, Deserialize, Debug)]
struct Message {
    pub ticker: String,
    pub amount: f32
}
```

Now, we must define the outline of the server runtime with the following code:

```
#[tokio::main]
async fn main() {
    let addr = "127.0.0.1:8080".to_string();
    let listener = TcpListener::bind(&addr).await.unwrap();
    println!("Listening on: {}", addr);

    loop {
        let (socket, _) = listener.accept().await.unwrap();
        tokio::spawn(async move {
            . . .
        });
    }
}
```

Here, we can see that the listener is looping to accept traffic and spawning threads when messages are received, as in previous server implementations. Inside our thread, we read our framed message with the following code:

```
let mut framed = BytesCodec::new().framed(socket);
let message = framed.next().await.unwrap();
    match message {
        Ok(bytes) => {
            . . .
        },
        Err(err) => println!("Socket closed with error:
                              {:?}", err),
    }
```

More books: https://t.me/booksforits

```
println!("Socket received FIN packet and closed
           connection");
```

As we can see, we do not have a `while` loop anymore. This is because our framing manages the splitting between messages.

Once we have extracted our bytes from our connection, we must implement the same logic that we did in our client, where we process our message, print it out, process it again, and then send it back to the client:

```
let message =
    bincode::deserialize::<Message>(&bytes).unwrap();
println!("{:?}", message);
let message_bin = bincode::serialize(&message).unwrap();
let sending_message = Bytes::from(message_bin);
framed.send(sending_message).await.unwrap();
```

We now have a working client and server that utilizes framing. If we were to start the server and then run the client, the client will give us the following printout:

```
Message { ticker: "BYND", amount: 3.2 }
```

Our server will give us the following printout:

```
Listening on: 127.0.0.1:8080
Message { ticker: "BYND", amount: 3.2 }
Socket received FIN packet and closed connection
```

Our server and client now support framing. We have come a long way. Now, we only have one more concept to explore in this chapter, and this is building an HTTP frame using TCP.

Building an HTTP frame on top of TCP

Before we explored the Tokio framework in this book, we used HTTP to send and receive data to and from servers. The HTTP protocol is essentially built on top of TCP. In this section, while we will create an HTTP frame, we will not mimic the HTTP protocol completely. Instead, to prevent excessive code, we will create a basic HTTP frame to understand the mechanisms utilized when creating an HTTP frame. It also must be stressed that this is for educational purposes. TCP is good for our protocols, but if you want to use HTTP handlers, it is quicker, safer, and less error-prone to use out-of-the-box HTTP handlers such as Hyper. We will cover how to use Hyper HTTP handlers with Tokio in the next chapter.

More books: https://t.me/booksforits

When it comes to an HTTP request, a request generally has a header and a body. When we send over a request, the header will tell us what method is being used and the URL associated with the request. To define our HTTP frame, we need the same structs defining the frame on both the server and client. Therefore, we must have the same code for the `client/src/http_frame.rs` and `server/src/http_frame.rs` files. First, we must import the serialization traits that are needed with the following code:

```
use serde::{Serialize, Deserialize};
```

Then, we must define our HTTP frame with the following code:

```
#[derive(Serialize, Deserialize, Debug)]
pub struct HttpFrame {
    pub header: Header,
    pub body: Body
}
```

As we can see, we have defined a header and body within our `HttpFrame` struct. We define the header and body structs with the following code:

```
#[derive(Serialize, Deserialize, Debug)]
pub struct Header {
    pub method: String,
    pub uri: String,
}
#[derive(Serialize, Deserialize, Debug)]
pub struct Body {
    pub ticker: String,
    pub amount: f32,
}
```

Our basic HTTP frame is now complete, and we can import the HTTP frame into the `main.rs` file of both the client and server with the following code:

```
mod http_frame;
use http_frame::{HttpFrame, Header, Body};
```

We will start by sending our HTTP frame in the `main.rs` file of our client with the following code:

```
let stream = TcpStream::connect("127.0.0.1:8080").await?;
let mut framed = BytesCodec::new().framed(stream);
```

More books: https://t.me/booksforits

```
let message = HttpFrame{
    header: Header{
        method: "POST".to_string(),
        uri: "www.freshcutswags.com/stock/purchase".to_string()
    },
    body: Body{
        ticker: "BYND".to_string(),
        amount: 3.2,
    }
};
let message_bin = bincode::serialize(&message).unwrap();
let sending_message = Bytes::from(message_bin);
framed.send(sending_message).await.unwrap();
```

We can see that our HTTP frame is starting to look like an HTTP request that we would handle when receiving requests in our Actix servers. For the `main.rs` file in our server, there is little change. All we must do is redefine the struct that is being deserialized with the following code:

```
let message = bincode::deserialize::<HttpFrame>(&bytes).
unwrap();
println!("{:?}", message);
let message_bin = bincode::serialize(&message).unwrap();
let sending_message = Bytes::from(message_bin);
framed.send(sending_message).await.unwrap();
```

We now have a basic HTTP frame that we can use to send information. If we were to run our server and then client programs, we would get the following printout for the server:

```
Listening on: 127.0.0.1:8080
HttpFrame { header: Header {
                method: "POST",
                uri: "www.freshcutswags.com/stock/purchase"
            },
            body: Body {
                ticker: "BYND",
                amount: 3.2
            }
        }
```

More books: https://t.me/booksforits

```
Socket received FIN packet and closed connection
```

Our client program will then give us the following printout:

```
HttpFrame { header: Header {
                method: "POST",
                uri: "www.freshcutswags.com/stock/purchase"
            },
            body: Body {
                ticker: "BYND",
                amount: 3.2
            }
        }
```

We can see that there is no corruption in our data. We have now covered all the core essential approaches and methods needed to be versatile in packaging, sending, and reading data over TCP.

Summary

In this chapter, we built a basic TCP client that sends and receives data to an echo server. We started by sending over basic string data and separating the messages with delimiters. Then, we increased the complexity of the data that we sent over a TCP connection by serializing structs. This enabled us to have more complex data structures. This serialization also reduced the handling needed to get the message data in the format that we needed it to be in. For instance, in the previous chapter, we were parsing strings into floats after receiving the message. With structs, nothing is stopping us from having a list of floats as a field, and after the serialization of the message, we would have that field housing a list of floats without any extra lines of code.

The serialization of structs is enough for us to handle most problems, but we explored framing so that we did not have to rely on delimiters to separate the messages that we send over TCP. With framing, we built a basic HTTP frame to visualize what we can do with frames and how HTTP is built on top of TCP. We must remember that implementing an HTTP protocol is more complicated than what we did in this chapter, and it is advised that we utilize established HTTP handlers from crates to handle and process HTTP traffic.

In the next chapter, we will use the established Hyper crate to handle HTTP traffic with the Tokio runtime framework.

More books: https://t.me/booksforits

Further reading

Tokio framing documentation: `https://tokio.rs/tokio/tutorial/framing`.

Questions

1. What is the advantage of framing over using a delimiter?
2. Why did we wrap our serialized message in a `Bytes` struct?
3. How would we be able to send over a string as a frame?

Answers

1. If we use a delimiter such as a new line, the data that we send over TCP might contain a new line in the message. The problem with having a new line in the message means that the message is split before the end of the message is received. Framing gets rid of this issue.
2. We had to wrap our serialized message in a `Bytes` struct because the `Encode` trait is not implemented for any other data type.
3. The simplest way to do this is to implement the `Encode` trait for a string. When implementing the `Encode` trait, we serialize the string and then wrap the string into a `Bytes` struct, reserve the length of the serialized string in the buffer, and then place the serialized string in the buffer.

More books: https://t.me/booksforits

17

Implementing Actors and Async with the Hyper Framework

The actor model has shown us that we can build async code that is safe and easy to maintain. In this chapter, we take the actor model a little further by building a caching mechanism with a background task that keeps running while we accept incoming HTTP requests using the **Hyper** framework. It must be noted that a Hyper framework is a low-level approach to processing HTTP requests. This enables us to build web applications where we have fine grain control over how the HTTP server handles HTTP requests. For instance, if we do not code how to handle **Universal Resource Identifiers** (**URIs**) and methods, the HTTP server built in Hyper will handle all requests, in the same way, no matter the method or URI passed in. Hyper is useful for building custom network applications such as caching mechanisms.

In this chapter, we will cover the following topics:

- Breaking down the actor async project and requirements
- Defining channel messages
- Building a runner actor
- Building a state actor
- Handling HTTP requests using Hyper
- Building an HTTP server using Hyper
- Running our Hyper HTTP server

By the end of this chapter, you will be able to build a low-level HTTP server that runs clean-up processes in the background while accepting HTTP requests. However, it must be noted that in previous chapters, we have built fully functioning applications using frameworks such as Rocket and Actix. It took multiple chapters to build a fully functioning application in Actix. It would not be possible to cover everything needed to build a fully functioning web application in a lower-level framework such

More books: https://t.me/booksforits

as Hyper in one chapter. We are merely using Hyper to set up and receive HTTP requests. However, with what we cover in this chapter, you should be able to build out a fully functioning web application with examples from online Hyper docs as we cover the core concepts of what gets a Hyper HTTP server running. It is advised that if you require a fully functioning web application that supports loads of views and authentication, then it makes sense to go for a higher-level framework such as Actix or Rocket.

Technical requirements

In this chapter, we will be purely focusing on how to build a server using the Hyper framework. Therefore, we will not be relying on any previous code as we are building our own new server.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter17`.

Breaking down our project

We are building a simulation platform where users can log in and interact with fake simulated people via chatbots to see what they say to the fake people. We want to see what the users say to the fake people at the end of the simulation session. The problem is that there are a lot of messages being sent to a fake person in a short amount of time. If we hit the database every time a message is sent, then we will put our database under a lot of strain. Let us say that a user asks a question every 20 seconds; this means we will be hitting the database 6 times per minute as there is a question and an answer per interaction. If we have 800 users running a session at the same time, then we can have up to 4,800 hits a minute. This can put a strain on the database. To reduce the strain on the database, we can build a server in Hyper that caches the chats and periodically sends multiple questions and answers to the database. Before moving forward, this is a chance for you to think back at all the chapters that we have covered and think of a high-level solution to building this caching mechanism.

There are multiple approaches to building the caching server, but to improve our understanding of async programming, we will solve our problem with the actor model. Our approach will have an actor that accepts a chat log and caches it under an ID. We then have another actor that periodically gets message data from the cache actor and sends it to another server. The data flow of our server takes the following form:

More books: https://t.me/booksforits

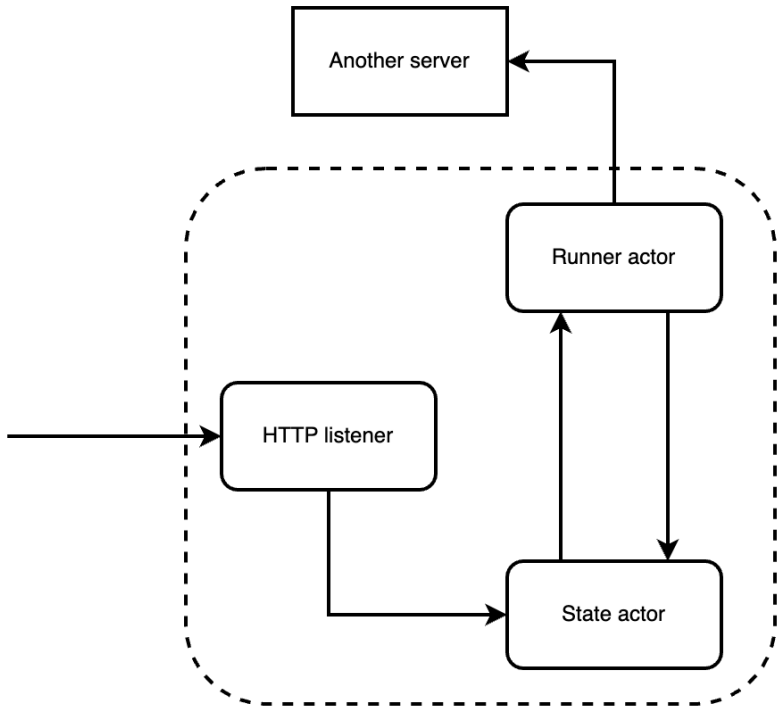


Figure 17.1 – Layers in our app

With this approach, we will need the following file layout for our server:

```
├── Cargo.toml
└── src
    ├── actors
    │   ├── messages.rs
    │   ├── mod.rs
    │   ├── runner.rs
    │   └── state.rs
    └── main.rs
```

More books: https://t.me/booksforits

We will define our actors in the individual files in the `actors` directory. All our code that handles HTTP requests will be defined in the `main.rs` file. We now have all the files that we need. When it comes to our dependencies, we know that we are accepting HTTP requests, sending HTTP requests, serializing data, and running our program in an async manner. With what we are doing in our application, it should not be a surprise that we have the following dependencies in the `Cargo.toml` file:

```
[dependencies]
tokio = { version = "1", features = ["full"] }
hyper = { version = "0.14.20", features = ["full"] }
reqwest = { version = "0.11.12", features = ["json"] }
serde_json = "1.0.86"
serde = { version = "1.0.136", features = ["derive"] }
```

The dependencies should not be a surprise at this stage, apart from the `reqwest` dependency, which we use to send HTTP requests. Now that we have defined the outline of our project, we can move on to building the first part of our system, which is defining the messages for our system.

Defining channel messages

Our runner actor needs to periodically send messages to our state actor and then send a batch of chats to a server. Considering the functionality of our runner actor, we can see that it does not need a state but does need to send messages. Before building our runner actor, we must build the messages that will be sent to actors and servers. In the `src/actors/messages.rs` file, we start by importing what we need with the following code:

```
use serde::Serialize;
use std::env;
```

We will be using the `Serialize` trait to enable us to process the body data from HTTP requests. Now that we have imported what we need, we can define the type of messages that are being sent to actors. If we think about what we need, we will be sending messages to the state actor to either get cached data or insert data. The state actor can return data and return empty data if there are no chats cached. Considering that we have three different types of messages, we have the following enum to define the type of message being sent with the following code:

```
#[derive(Debug, Serialize)]
pub enum MessageType {
    INPUT,
    OUTPUT,
    EMPTY
}
```

More books: https://t.me/booksforits

We then have the struct for our messages that will be sent to and from actors, which takes the following form:

```
#[derive(Debug, Serialize)]
pub struct StateActorMessage {
    pub message_type: MessageType,
    pub chat_id: Option<i32>,
    pub single_data: Option<String>,
    pub block_data: Option<Vec<String>>
}
```

We can see that we must define what type of message is being sent. Everything else is optional. For instance, if the state actor finds that there are no cached chats, then the state actor cannot populate any other fields apart from the message type saying that the message is empty.

Remember that we must send blocks of cached chat messages to a server. We can implement the functionality of sending the data to the server for the `StateActorMessage` struct with the following code. We must note that we have not made the `PostBody` struct yet, but we will do this straight afterward:

```
impl StateActorMessage {
    pub async fn send_to_server(&self) {
        let lib_url = env::var("SERVER_URL").unwrap();
        let joined =
            self.block_data.clone().unwrap().join("$");
        let body = PostBody {
            chat_id: self.chat_id.unwrap(),
            block_data: joined
        };
        let client = reqwest::Client::new();
        let res = client.post(lib_url)
                                .json(&body)
                                .send()
                                .await.unwrap();
        println!("{:?}", res);
    }
}
```

More books: https://t.me/booksforits

We can see that we merely get the server URL from the environment, create one large string with `$` as a delimiter separating the pairs of questions and answers, and then send it to another server. We could make an actor handling the message send data to the server. However, coupling the sending of data to the server with the message struct gives more flexibility as any actor that handles the message can send the data in the message to the server.

When it comes to the `PostBody` struct, it should be no surprise that it takes the following form:

```
#[derive(Debug, Serialize)]
struct PostBody {
    pub chat_id: i32,
    pub block_data: String
}
```

Our messages for our entire application are now defined. We can now move on to building our runner actor.

Building our runner actor

Our runner actor consistently loops, sending messages to the state actor, asking for batched data, and sending the batched data to the server if the batched data is present. This means that our actor will be sending and receiving messages throughout the lifetime of the program. With the behavior of the runner actor in mind, it should not be a shock that we need the following imports in the `src/actors/runner.rs` file:

```
use super::messages::{MessageType, StateActorMessage};
use tokio::sync::mpsc::{Sender, Receiver};

use std::time;
```

We have imported the messages and the modules required to sleep for a period of seconds. We have also used type aliases to define the type of channels our runner actor will be supporting. We can now define the runner actor with the following code:

```
pub struct RunnerActor {
    pub interval: i32,
    pub receiver: Receiver<StateActorMessage>,
    pub sender: Sender<StateActorMessage>,
}
```

More books: https://t.me/booksforits

Here, we are merely defining the number of seconds our actor will wait before making another request for data. We then have a sender and receiver to send and receive messages. With the fields defined, we then need to move on to defining the constructor and run function for our `RunnerActor` struct with the following code:

```
impl RunnerActor {

    pub fn new(receiver: Receiver<StateActorMessage>,
               sender: Sender<StateActorMessage>,
               interval: i32) -> RunnerActor {
        return RunnerActor { interval, receiver, sender }
    }
    pub async fn run(mut self) {
        . . .
    }
}
```

We can see that our constructor function (`new`) is not really needed. In our `new` function, we are merely passing arguments directly into the fields of the `RunnerActor` struct. However, for the effort exerted, it is nice to have. For instance, if we need to add some checks later or send a message to the state actor saying that the `RunnerActor` struct has been constructed, all we need to do is change the behavior in the `new` function, and this behavior will propagate throughout the program where the `new` function is used.

For our `run` function, we run an infinite loop. Inside this loop, we send a message to the state actor. Each iteration of the loop is broken with a sleep function with the following code:

```
pub async fn run(mut self) {
    println!("runner actor is running");
    let seconds = time::Duration::from_secs(self.interval
                                            as u64);

    loop {
        tokio::time::sleep(seconds).await;
        let message = StateActorMessage {
            message_type: MessageType::OUTPUT,
            chat_id: None,
            single_data: None,
            block_data: None
```

More books: https://t.me/booksforits

```
        };
        match self.sender.send(message).await {
            . . .
        };
    }
}
```

Once we have sent the message, we must wait for a response and send the batched data to a server if the response is not empty, which can be done with the following code:

```
match self.sender.send(message).await {
    Ok(_) => {
        let message = self.receiver.recv().await.unwrap();
        match message.message_type {
            MessageType::OUTPUT => {
                message.send_to_server().await;
            },
            _ => {
                println!("state is empty");
            }
        }
    },
    Err(_) => {
        println!("runner is failed to send message");
    }
};
```

Here, we can see that, by now, the logic is not too confusing. With that, we will be able to speed through our state actor and then finally get to creating an HTTP server in Hyper.

Building our state actor

When it comes to our state actor, we must send and receive messages. Our state actor will also have a state where the actor will store the chat logs to be referenced. With the state actor mechanisms in mind, it will not be surprising that we have the following imports in the `src/actors/state.rs` file:

```
use std::collections::{HashMap, VecDeque};
use std::mem;
```

More books: https://t.me/booksforits

```
use tokio::sync::mpsc::{Sender, Receiver};

use super::messages::{MessageType, StateActorMessage};
```

The only difference in the imports is the `mem` module. The `mem` module will enable us to allocate memory. We will cover how we use the `mem` module when we get the message from the state of the actor. We can also see that we have imported `HashMap` and `VecDeque` to handle the state of the actor.

Now that we have imported what we need, we can define our actor struct with the following code:

```
#[derive(Debug)]
pub struct StateActor {
    pub chat_queue: VecDeque<i32>,
    pub chat_logs: HashMap<i32, Vec<String>>,
    pub receiver: Receiver<StateActorMessage>,
    pub sender: Sender<StateActorMessage>,
}
```

The `chat_queue` is where we perform a first-in-first-out queue using the `VecDeque` struct implementation from the standard collections library. We are assuming that the oldest chat logs will generally have more chats than later chat logs, and therefore, we should cache the older chats. The reason we are not merely using a vector as opposed to a queue is that popping off the first element of a queue does not require reallocating all the other elements. If we were to remove the first element of a vector, we would have to reallocate all the other elements in the vector. When it comes to storing the chat logs, we have the chat ID as the key and the question and answers under the chat ID.

Now that we have defined our state actor, we can define the functions needed for our actor with the following code:

```
impl StateActor {

    pub fn new(receiver: Receiver,
               sender: Sender) -> StateActor {
        . . .
    }
    pub fn get_message_data(&mut self,
                            chat_id: i32) -> Vec<String> {
        . . .
    }
    pub fn insert_message(&mut self,
```

More books: https://t.me/booksforits

```
                            chat_id: i32, message_data:
                                String) {
        . . .
    }
    async fn handle_message(&mut self,
                              message: StateActorMessage) {
        . . .
    }
    pub async fn run(mut self) {
        . . .
    }
}
```

Here, we can see that we have a constructor, an extraction of chat data, an insertion of a chat log, the handling of a message based on the type of message being received, and a `run` function to wait for incoming messages sent to the state actor.

We are now ready to shoot through the logic implemented by the functions for the `StateActor` struct. At this stage of the book, you should be able to implement these functions by yourself, which will be good practice.

For our constructor function, we have the following code:

```
pub fn new(receiver: Receiver, sender: Sender) -> StateActor {
    let chat_queue: VecDeque<i32> = VecDeque::new();
    let chat_logs: HashMap<i32, Vec<String>> =
        HashMap::new();
    return StateActor {chat_queue, chat_logs, receiver,
                       sender}
}
```

The constructor is merely creating an empty queue and HashMap and accepts the channels that send and receive messages. For our `get_message` function, we have the following code:

```
pub fn get_message_data(&mut self, chat_id: i32) ->
    Vec<String> {
        self.chat_logs.remove(&chat_id).unwrap()
}
```

More books: https://t.me/booksforits

We can see that we get the chat data based on the chat ID from the chat logs. We then transfer ownership from the HashMap chat logs to the `reference` variable, delete the chat ID key, and then return the data. This enables us to remove and return message data from our state.

For our `insert_message` function, we utilize the following code:

```
pub fn insert_message(&mut self, chat_id: i32,
    message_data: String) {

    match self.chat_logs.get_mut(&chat_id) {
        Some(patient_log) => {
            patient_log.push(message_data);
        },
        None => {
            self.chat_queue.push_back(chat_id);
            self.chat_logs.insert(chat_id,
                                  vec![message_data]);
        }
    }
}
```

Here we can see that we merely insert the new message data into the vector associated with the chat ID. If the chat ID is not present, we attach the chat ID to the queue and create a new vector under the chat ID.

We can now move on to our function that handles the message with the following code:

```
async fn handle_message(&mut self,
    message: StateActorMessage) {
    println!("state actor is receiving a message");

    match message.message_type {
        MessageType::INPUT => {
            self.insert_message(message.chat_id.unwrap(),
                                message.single_data
                                .unwrap());
        },
        MessageType::OUTPUT => {
            . . .
```

More books: https://t.me/booksforits

```
        },
        MessageType::EMPTY => {
            panic!(
              "empty messages should not be sent to the
                   state actor"
            );
        }
    }
    println!("{:?}", self.chat_logs);
    println!("{:?}", self.chat_queue);
}
```

Here, if the message is input, we merely insert the message into our state. If the message is empty, we panic the thread as there should not be any empty messages being sent to the state actor. If we have an output message, this means that we must get the oldest chats located down the bottom of the queue, which takes the following form:

```
MessageType::OUTPUT => {
    match self.chat_queue.pop_front() {
        Some(chat_id) => {
            let data = self.get_message_data(chat_id);
            let message = StateActorMessage {
                message_type: MessageType::OUTPUT,
                chat_id: Some(chat_id),
                single_data: None,
                block_data: Some(data)
            };
            let _ =
                self.sender.send(message).await.unwrap();
        },
        None => {
            let message = StateActorMessage {
                message_type: MessageType::EMPTY,
                chat_id: None,
                single_data: None,
                block_data: None
            };
```

More books: https://t.me/booksforits

```
            let _ =
                self.sender.send(message).await.unwrap();
        }
    }
},
```

Here we can see that if there is nothing in the queue, then we have no chats; thus we return an empty message. If there is a chat ID in the queue, we get the message data and send that data via a message to the runner.

Finally, the `run` function takes the following form:

```
pub async fn run(mut self) {
    println!("state actor is running");
    while let Some(msg) = self.receiver.recv().await {
        self.handle_message(msg).await;
    }
}
```

This `run` function merely waits for incoming messages and processes those incoming messages. We have now defined all our actors and can now move on to building our HTTP server.

Handling HTTP requests using Hyper

When it comes to building our HTTP server, we will implement all the server logic in the `src/main.rs` file. First, we import what we need with the following code:

```
use tokio::sync::{mpsc, mpsc::Sender};
use hyper::{Body, Request, Response, Server};
use hyper::body;
use hyper::service::{make_service_fn, service_fn};
use serde_json;
use serde::Deserialize;
use std::net::SocketAddr;
```

At this stage in the book, these imports should not be confusing to you. Even though we have imported from the `hyper` module, the imports are self-explanatory. We will extract data from a request, create a service to process our request, and create an HTTP server to listen to incoming requests.

More books: https://t.me/booksforits

We will also need to utilize the actors that we have created. Our actors can be imported using the following code:

```
mod actors;
use actors::state::StateActor;
use actors::runner::RunnerActor;
use actors::messages::StateActorMessage;
use actors::messages::MessageType;
```

With all these actors imported, we can work towards accepting requests. Incoming HTTP requests insert new chat logs into our state actor. To achieve inserting the chat log, we will require a chat ID, inputs (a question), and an output (an answer). It would also be good to have a timestamp but to simplify the example, we will use a turn number to denote the time when the chat log was created. To extract all the data that we have just listed from the request, we will need the following struct:

```
#[derive(Deserialize, Debug)]
struct IncomingBody {
      pub chat_id: i32,
      pub timestamp: i32,
      pub input: String,
      pub output: String
}
```

The approach we have taken in processing the body of an incoming request is the same as when we built servers with Rocket and Actix. This is because we are relying on `serde`. The fact that the implementation of processing a body of an incoming request is the same is a testament to the flexibility of implementing traits in Rust.

Seeing as we are accepting incoming requests, we must handle them. We can define the logic of how to handle a request in a function with the following outline:

```
async fn handle(req: Request<Body>, channel_sender:
    Sender<StateActorMessage>) -> Result<Response<Body>,
        &'static str> {
    . . .
}
```

More books: https://t.me/booksforits

In this `handle` function, we are accepting a request with a body and a channel where we can send our messages to our actors. If we were not applying the actor model approach, we could get away with merely accepting requests. We must remember that we handle our requests through a function, and therefore nothing stops us from passing in a database connection or anything else for that matter. Our `handle` function is essentially acting as middleware.

For this example, we are just supporting one view. However, to get a feel for what we can do with Hyper, we might as well print out some basic information about the incoming request with the following code:

```
println!("incoming message from the outside");
let method = req.method().clone();
println!("{}", method);
let uri = req.uri();
println!("{}", uri);
```

If we were to support multiple views, we could implement a `match` statement on the URI of the incoming request, passing the request into another function that housed multiple views for a particular theme, such as authentication or chat logging. Another `match` statement could then pass the request into the correct view function, which processes the request and returns an HTTP response, which the `handle` function would return.

Again, because we are merely supporting one view, we will process the request directly in the `handle` function. We do this by extracting the body data from the request with the following code:

```
let bytes = body::to_bytes(req.into_body()).await.unwrap();
let string_body = String::from_utf8(bytes.to_vec())
    expect("response was not valid utf-8");
let value: IncomingBody = serde_json::from_str(
    &string_body.as_str()).unwrap();
```

We now have all the data that we need to send a chat log to our state actor and return an HTTP response that everything is OK with the following code:

```
let message = StateActorMessage {
    message_type: MessageType::INPUT,
    chat_id: Some(value.chat_id),
    single_data: Some(format!("{}>>{}>>{}>>",
                              value.input,
                              value.output,
                              value.timestamp)),
    block_data: None
```

More books: https://t.me/booksforits

```
};
channel_sender.send(message).await.unwrap();

Ok(Response::new(format!("{:?}", value).into()))
```

With this our `handle` function is complete. Considering what we have done in this section, we can appreciate how low level Hyper is in terms of implementing HTTP. While we have printed out the URI and method, we have not done anything with it. It doesn't matter what method or URI is passed into our server; if the body has the correct data, our server will work. So, let us get our server working in the next section.

Building an HTTP server using Hyper

When it comes to running an HTTP server with Hyper, we will be using Tokio. We have two actors running and two channels to facilitate the communication between actors and requests. First, in the `main` function, we define the address of the server with the following code:

```
#[tokio::main]
async fn main() {
    let addr = SocketAddr::from(([0, 0, 0, 0], 3000));
    . . .
}
```

After the address is defined, we define the two channels for messages with the following code:

```
let (state_tx, state_rx) =
    mpsc::channel::<StateActorMessage>(1);
let (runner_tx, runner_rx) =
    mpsc::channel::<StateActorMessage>(1);
let channel_sender = state_tx.clone();
```

We clone the sender for the state actor because we will pass the sender through the `handle` function with the incoming request. Now that we have our channels, we can spin off two threads to run our actors with the following code:

```
tokio::spawn(async move {
    let state_actor = StateActor::new(state_rx, runner_tx);
    state_actor.run().await;
});
tokio::spawn(async move {
```

More books: https://t.me/booksforits

```
    let lib_runner_actor = RunnerActor::new(runner_rx,
                                            state_tx, 30);
    lib_runner_actor.run().await;
});
```

Here we must be careful to pass the sender of the other actor into the actor we are creating as the actors send messages to each other; they are not sending messages to themselves.

Everything for our caching mechanism is now running. All we need now is to accept incoming HTTP requests, which we can achieve with the following code:

```
let server = Server::bind(&addr).serve(make_service_fn( |_conn|
{
    let channel = channel_sender.clone();
    async {
        Ok::<_, hyper::Error>(service_fn(move |req| {
            let channel = channel.clone();
            async {handle(req, channel).await}
        }))
    }
}));
```

It must be noted that we are returning a future with an async syntax, which is required for the `service_fn` function. Here we can see that we bind our address to the server and then call the `serve` function. Inside this `serve` function, we pass in the `make_service_fn` function, which wraps whatever executable we pass into the `make_service_fn` function in a `MakeServiceFn` struct. The executable we pass into the `make_service_fn` function is a closure that passes in `_conn`, which is an `AddrStream` struct. With the `AddrStream` struct, we can get the address of the peer making the connection to the server. We can also consume the `AddrStream` struct and extract the underlying TCP stream using the `into_inner` method of the `AddrStream` struct. We are not going to play with the `AddrStream` struct in this chapter, as we are keeping it simple by just processing standard HTTP requests.

Inside the closure, we clone the sender again. We need to do this here because we need to clone the channel for every request coming in, as every request will need to send a message to the state actor. We then create a future with the `async` block returning an `Ok` enum, which wraps around the `service_fn` function, where we insert another closure that processes the incoming request. This is where we clone the channel again and return a future of our `handle` function that accepts the incoming request and channel to be processed and an HTTP response to be returned. We can see that it takes more steps to get an HTTP server running compared to other frameworks such as Rocket. However, we also get a lot of fine-grain control.

More books: https://t.me/booksforits

Now that our server block is complete, we can implement a last piece of logic in the `main` function, which prints out the error if there is a problem with the server by implementing the following code:

```
if let Err(e) = server.await {
    eprintln!("server error: {}", e);
}
```

With this, our simple HTTP server should work.

Running our Hyper HTTP server

When we run our server, we first need to export the URL that we are going to send batched chat logs to with the following command:

```
export SERVER_URL="https://httpbin.org/post"
```

This means that we just need to send our HTTP requests to an HTTPBin and get a standard response back. We can then run our server with the following command:

```
cargo run
```

Our server is now running with the following printouts:

```
state actor is running
runner actor is running
```

Our intervals are every 30 seconds for the runner actor to send a message to the state actor. If we just leave our server running, we will get the following printout:

```
state is empty
state actor is receiving a message
{}
[]
state is empty
state actor is receiving a message
{}
[]
state is empty
state actor is receiving a message
{}
[]
```

More books: https://t.me/booksforits

Here we can see that the chat state and queue are empty and that both of our actors are running in their threads! We can then get our Postman application and send the following HTTP request:



Figure 17.2 – HTTP request for our server

Sending the previous request will give the following printout:

```
incoming message from the outside
POST
/test
state actor is receiving a message
{23: ["what is your name>>my name is maxwell>>1>>"]}
[23]
```

Here we can see that the state and queue have been populated. If we press the **Send** button in Postman another 2 times before the initial 30 seconds is up, we get the following printout:

```
incoming message from the outside
POST
/test
state actor is receiving a message
{23: ["what is your name>>my name is maxwell>>1>>",
      "what is your name>>my name is maxwell>>1>>"]}
```

More books: https://t.me/booksforits

```
[23]
incoming message from the outside
POST
/test
state actor is receiving a message
{23: ["what is your name>>my name is maxwell>>1>>",
      "what is your name>>my name is maxwell>>1>>",
      "what is your name>>my name is maxwell>>1>>"]}
[23]
```

We can see that the queue is not increasing as we are sending the same chat ID. We then increase the ID of the chat log and send another request resulting in the following printout:

```
incoming message from the outside
POST
/test
state actor is receiving a message
{24: ["what is your name>>my name is maxwell>>1>>"],
23: ["what is your name>>my name is maxwell>>1>>",
     "what is your name>>my name is maxwell>>1>>",
     "what is your name>>my name is maxwell>>1>>"]}
[23, 24]
```

We can see that the new ID has been inserted into the queue and state. We have managed to make all these requests before the 30 seconds is up, then we can simply wait until we get the following printout:

```
state actor is receiving a message
{23: ["what is your name>>my name is maxwell>>1>>"}
[23]
Response { url: Url { scheme: "https",
cannot_be_a_base: false,
username: "", password: None,
host: Some(Domain("httpbin.org")),
port: None, path: "/post",
. . .
"*", "access-control-allow-credentials": "true"} }
```

More books: https://t.me/booksforits

What has happened here is that the runner has sent a message to the state getting the oldest chat ID and then sent it to the server we defined in our environment variables. We can see that the state has been updated, and the response is OK. Some of the HTTP response has been omitted to avoid printout bloat on the page.

We can conclude that our caching system built with actors on Hyper HTTP is working as we expected! With this, we have come to the end of this chapter, exploring enough about Hyper to get a server up and running with async actors and channels.

Summary

In this chapter, we built a network application that cached chat logs. Our network application also has a background task continuously running to periodically clean up the cache by sending the cached data to a server. We broke down and created our background task runner into an actor system with a queue and then implemented it. This gives you a whole new toolset to solve problems with. Background running actors are not just for running on a server for caching. You can run background runner actors for normal programs too if you build your program on a Tokio runtime.

In the next chapter, we will use Redis persistent storage to manage multiple workers and network applications to transfer messages across multiple network applications.

Further reading

Hyper documentation: `https://hyper.rs/guides`

More books: https://t.me/booksforits

More books: https://t.me/booksforits

18

Queuing Tasks with Redis

Receiving requests, performing an action, and then returning a response to the user can solve a lot of problems in web programming. However, there are times when this simple approach will simply not cut it. For instance, when I was working at MonolithAi, we had a functionality where the user would be able to put in data and parameters and then train a machine learning model on that data at a click of a button. However, trying to train a machine learning model before sending a response to the user would simply take too long. The connection would probably time out. To solve this, we had a **Redis** queue and a pool of workers consuming tasks. The training task would be put into the queue and one of the workers would work on training the model when they got round to it. The HTTP server would accept the request from the user, post the training task to the queue, and respond to the user that the task was posted. When the model was trained, the user would get an update. Another example could be a food ordering application where the food order goes through a series of steps such as confirming the order, processing the order, and then delivering the order.

Considering the MonolithAi example, it is not hard to see why learning how to implement queuing in web programming is not only useful but also gives the developer another solution, increasing the number of problems they can solve.

In this chapter, we will cover the following topics:

- Laying out the queuing project, describing the components and approach needed
- Building an HTTP server
- Building a polling worker
- Getting our application running with Redis
- Defining tasks for workers
- Defining messages for the Redis queue
- Integrating routing in the HTTP server
- Running all servers and workers in Docker

More books: https://t.me/booksforits

By the end of this chapter, you will be able to build a single Rust program that can either be a worker or server depending on the environment variable passed into it. You will also be able to serialize a range of tasks in the form of different structs and insert them into the Redis queue, enabling these structs to be queued and transported across different servers. This will not only give you the skillset to implement queues but also utilize Redis to implement many other solutions, such as multiple servers receiving messages through a broadcast via a Redis pub/sub channel.

Technical requirements

In this chapter, we will be purely focusing on how to build workers using Tokio and Hyper on a Redis queue. Therefore, we will not be relying on any previous code as we are building our own new server.

The code for this chapter can be found at `https://github.com/PacktPublishing/Rust-Web-Programming-2nd-Edition/tree/main/chapter18`.

Breaking down our project

In our system, we have a series of tasks that need to be executed. However, these tasks take a long time to complete. If we were to just have a normal server handling the tasks, the server will end up being choked and multiple users will receive a delayed experience. If the task is too long, then the users' connection might time out.

To avoid degrading users' experience when long tasks are needed, we utilize a queuing system. This is where an HTTP server receives a request from the user. The long task associated with the request is then sent to a first-in-first-out queue to be processed by a pool of workers. Because the task is in the queue, there is nothing more the HTTP server can do apart from respond to the user that the task has been sent and that their request has been processed. Due to the ebbs and flows of traffic, we will not need all our workers and HTTP servers when the traffic is low. However, we will need to create and connect extra HTTP servers and workers when the traffic increases, as seen in the following diagram:

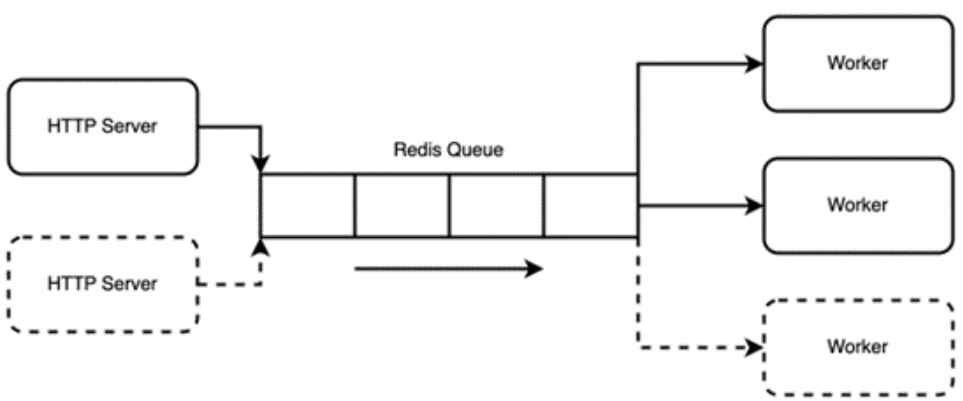


Figure 18.1 – Our approach to processing lengthy tasks

More books: https://t.me/booksforits

Considering the preceding diagram, we will need the following infrastructure:

- **Redis database**: To store the tasks in the queue
- **HTTP server**: To send tasks to the queue to be processed
- **Worker**: To pull/pop/poll/process tasks from the queue

We could build individual applications for the worker and the HTTP server. However, this would increase complexity for no gain. With two separate applications, we would have to maintain two separate Docker images. We would also duplicate a lot of code as the tasks that the HTTP server sends to the Redis queue must be the same tasks that the worker picks up and processes. There could end up being a mismatch between the fields passed from the HTTP server to the worker for a particular task. We can prevent this mismatch by having task structs that have a range of fields for the input and a run function to execute the task with those fields. Serialization traits for these task structs can enable us to pass the fields over the queue and receive them.

When it comes to building an HTTP server and worker, we can build the server so that environment variables are checked once the program is started. If the environment variable states that the application is a worker, the application can then spin up an actor that polls the queue. If the environment variable states that the application is an HTTP server, the application can then run an HTTP server and listen for requests.

For our task queue project, we have the following outline:

```
├── Cargo.toml
├── docker-compose.yml

└── src
    ├── main.rs
    └── tasks
        ├── add.rs
        ├── mod.rs
        ├── multiply.rs
        └── subtract.rs
```

We will define the server entry point in the `src/main.rs` file. We will then define our task structs in the `src/tasks/` directory. In terms of our dependencies in our `Cargo.toml` file, we have the following:

```
[dependencies]
bincode = "1.0"
bytes = "1.2.1"
```

More books: https://t.me/booksforits

```
redis = "0.22.1"
serde_json = "1.0.86"
tokio = { version = "1", features = ["full"] }
hyper = { version = "0.14.20", features = ["full"] }
serde = { version = "1.0.136", features = ["derive"] }
```

None of these dependencies should be new to you apart from the `bytes` and `bincode` crates. We will use `bytes` to convert our struct into HTTP responses and `bincode` to serialize structs into binary to be stored in Redis.

With the approach that we have just laid out in this section, we will be able to build a simple task-processing queue where we can assure that the task definitions between the servers and workers are always in sync. With our approach defined, we can move on to the first part of a task's journey, which is the HTTP server.

Building the HTTP server

For our HTTP server, we need to carry out the following steps:

1. Define a struct that deserializes the HTTP request body.
2. Define a function that handles the incoming request.
3. Define pathways for the program to run based on environment variables.
4. Run a server that listens for incoming requests.

We are not going to section off individual sections for each step as we have covered all of these steps/processes in the previous chapter. Before we carry out all the steps, we must import the following into the `src/main.rs` file:

```
use hyper::{Body, Request, Response, Server};
use hyper::body;
use hyper::service::{make_service_fn, service_fn};
use std::net::SocketAddr;
use std::env;

use serde::{Serialize, Deserialize};
use serde_json;
use bytes::{BufMut, BytesMut};
```

More books: https://t.me/booksforits

You should be familiar with all these imports apart from the `bytes` import, which we will cover when defining the HTTP handle function. First, we will define a trivial struct to serialize the incoming HTTP request bodies with the following code:

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct IncomingBody {
    pub one: String,
    pub two: i32
}
```

This is the same approach to our Actix Web applications. We will be able to annotate our task structs with the `Serialize` and `Deserialize` traits.

Now that we have defined the `IncomingBody` struct, we can define our `handle` function with the following code:

```
async fn handle(req: Request<Body>) ->
    Result<Response<Body>, &'static str> {

    let bytes = body::to_bytes(req.into_body()).await
                                                .unwrap();
    let response_body: IncomingBody =
        serde_json::from_slice(&bytes).unwrap();

    let mut buf = BytesMut::new().writer();
    serde_json::to_writer(&mut buf,
                          &response_body).unwrap();
    Ok(Response::new(Body::from(buf.into_inner().freeze())))
}
```

It must be noted that we are calling the `freeze` function when returning our body. This `freeze` function converts the mutable bytes to immutable, preventing any buffer modifications. Here, we can see that we are accepting a generic body with the request. We can then use `serde` to serialize the body and the `BytesMut` struct (which is essentially just a contiguous slice of memory) to return the body to the user, essentially creating an echo server.

We can now define the `main` function, which is the entry point with the following code:

```
#[tokio::main]
async fn main() {
    let app_type = env::var("APP_TYPE").unwrap();
```

More books: https://t.me/booksforits

```
    match app_type.as_str() {
        "server" => {
            . . .
        },
        "worker" => {
            println!("worker not defined yet");
        }
        _ => {
            panic!("{} app type not supported", app_type);
        }
    }
}
```

Here we can see that the environment variable `"APP_TYPE"` is extracted. Depending on what the app type is, we have different blocks of code being executed. For now, we will just print out a statement that the worker is not defined if the app type is a `"worker"`. We also state that the program is going to panic if the app type is neither a `"server"` nor a `"worker"` type.

In our server block, we defined `addr` and `server` with the following code:

```
let addr = SocketAddr::from(([0, 0, 0, 0], 3000));
let server = Server::bind(&addr).serve(make_service_fn( |_conn|
{
    async {
        Ok::<_, hyper::Error>(service_fn( move |req| {
            async {handle(req).await}
        }))
    }
}));
if let Err(e) = server.await {
    eprintln!("server error: {}", e);
}
```

This is very similar to our server code in the previous chapter.

We then run the server with the following command:

```
APP_TYPE=server cargo run
```

More books: https://t.me/booksforits

We can then send the following request:

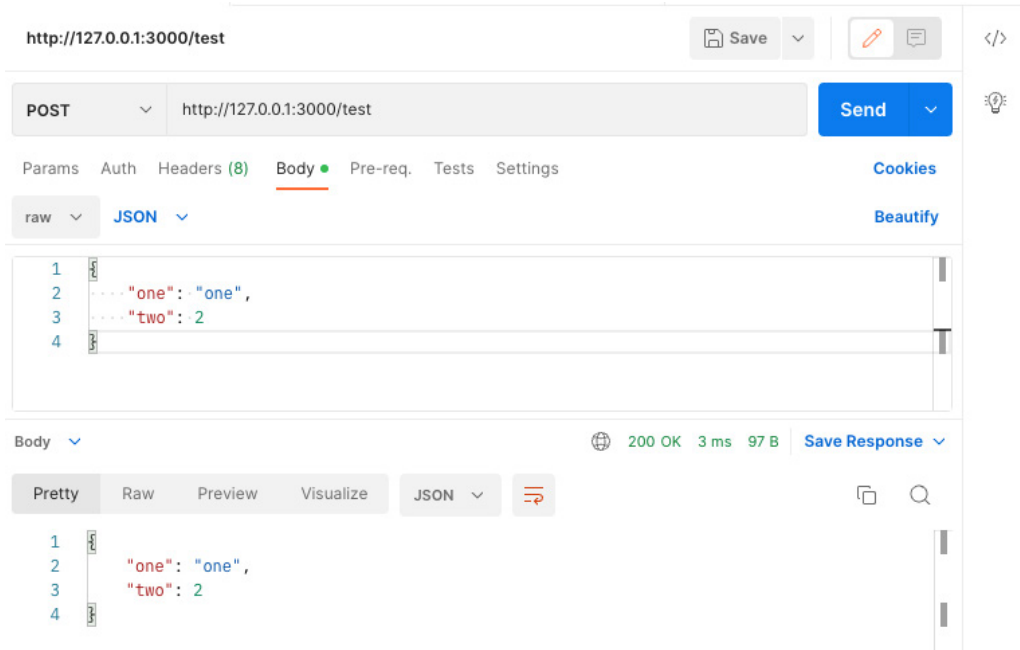


Figure 18.2 – A request to our HTTP server

Here, we can see that our server works and echoes the same body that was sent to the server. We can now move on to building our worker application.

Building the polling worker

Our worker is essentially looping and polling a queue in Redis. If there is a message in the queue, the worker will then execute the task that has been extracted from the queue. For building the polling worker section, the worker will be creating a struct, inserting the struct into the Redis queue, and then extracting that inserted struct from the queue to print out. This is not our desired behavior but this does mean that we can test to see how our queue insertion works quickly. By the end of the chapter, our HTTP servers will be inserting tasks and our workers will be consuming tasks.

We do not want the worker to be polling the Redis queue constantly without any rest. To reduce the polling to a reasonable rate, we will need to make the worker sleep during each loop. Therefore, we must import the following in the `src/main.rs` file to enable us to get our worker sleeping:

```
use std::{thread, time};
```

More books: https://t.me/booksforits

We can now move to the section where the worker is run to define our worker code in the following section in the `main` function:

```
match app_type.as_str() {
    "server" => {
        . . .
    },
    "worker" => {
      // worker code is going to be inserted here
        . . .
    }
    _ => {
        panic!("{} app type not supported", app_type);
    }
}
```

Our worker code takes the following general outline:

```
let client =
    redis::Client::open("redis://127.0.0.1/").unwrap();

loop {
    . . .
}
```

Here, we can see that we define the Redis client and then run the worker on an infinite loop. In this loop, we will be establishing a connection with Redis, polling the queue in Redis, and then removing the connection. We can establish and remove the connection in the loop because the task will take a long time. There is no point in holding onto a Redis connection throughout the duration of a task.

Unfortunately, at the point of writing this book, the Rust Redis crate does not have a simple implementation of queues. However, this should not hold us back. If we know the raw commands needed to get Redis to implement our queue, we can implement our own queues. Redis performs like a SQL database. If you know the commands, you can implement your own logic like in SQL.

Inside our infinite loop, we are going to create a generic struct that has the `Serialize` and `Deserialize` traits implemented, then serialize the struct into binary with the following code:

```
let body = IncomingBody{one: "one".to_owned(), two: 2};
let bytes = bincode::serialize(&body).unwrap();
```

More books: https://t.me/booksforits

Our struct is now a vector of bytes. We will then establish a connection with Redis and push `"some_queue"` with the `"LPUSH"` command to the queue, which inserts the value at the head of the queue, with the following code:

```
let outcome: Option<Vec<u8>>;
{
    let mut con = client.get_connection().unwrap();
    let _ : () = redis::cmd("LPUSH").arg("some_queue")
                                    .arg(bytes.clone())
                                    .query(&mut con)
                                    .unwrap();
    // pop our task from the queue
    outcome = redis::cmd("LPOP").arg("some_queue")
                                .query(&mut con)
                                .unwrap();
}
```

We have `Option<Vec<u8>>` because there may not be anything in the queue. If there is nothing in the queue, then the outcome will be none. Right now, we will never get a none because we are directly inserting tasks into the queue before we extract a task from the queue. However, in periods of low traffic, our workers will be polling queues that could be empty for a while.

Now that we have our outcome, we can process it with the following `match` statement:

```
match outcome {
    Some(data) => {
        . . .
    },
    None => {
        . . .
    }
}
```

If we have some data, we will merely deserialize the binary data and print out the struct with the following code:

```
let deserialized_struct: IncomingBody =
    bincode::deserialize(&data).unwrap();
println!("{:?}", deserialized_struct);
```

More books: https://t.me/booksforits

If there is nothing in the queue, the `outcome` is `None`, and we can just sleep for five seconds before running the loop again with the following code:

```
let five_seconds = time::Duration::from_secs(5);
tokio::time::sleep(five_seconds).await;
```

With this, our worker is ready to be tested. You can always do more when building an async program like this. However, to avoid bloating this chapter, we will stick with our basic application. If you want to further your understanding of Redis, you could investigate building a pub/sub system where one worker continuously polls the queue and the other workers are switched off with an actor listening for a message on a channel. When a main worker gets a new task, the main worker can publish a message to a channel, waking up other workers. If you really want to push yourself, you could investigate Kubernetes controllers and have a main worker spin up and destroy worker pods, depending on the traffic. However, these projects will be beyond the scope of this book.

To get our application working within the scope of one chapter, we must move on to getting our application running with Redis.

Getting our application running with Redis

Running our application with Redis locally will require us to use Redis with Docker, export the `APP_TYPE` environment variable as `"worker"`, and then run our application with Cargo. For our Redis, our `docker-compose.yml` file takes the following form:

```
version: "3.7"

services:
    redis:
      container_name: 'queue-redis'
      image: 'redis'
      ports:
        - '6379:6379'
```

We can then export our `APP_TYPE` environment variable with the following command:

```
export APP_TYPE=worker
```

We can then run our application with the following command:

```
cargo run
```

More books: https://t.me/booksforits

When we run our application, we will get the following printout:

```
IncomingBody { one: "one", two: 2 }
IncomingBody { one: "one", two: 2 }
IncomingBody { one: "one", two: 2 }
IncomingBody { one: "one", two: 2 }
. . .
```

The printout of the `IncomingBody` struct will be infinite because we are running an infinite loop. However, what this shows is that the following mechanism is running and working:

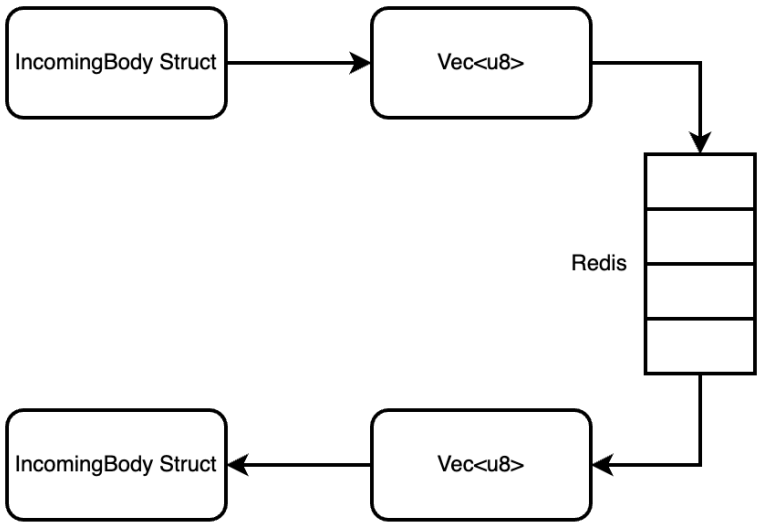


Figure 18.3 – Our process of how we insert and extract data from a Redis queue

Although our worker is working with a Redis queue, it is merely printing out the struct that was put into the Redis queue. In the next section, we build functionality into the structs that we are inserting into the Redis queue so our worker can perform the tasks.

Defining tasks for workers

When it comes to running our tasks, we need fields so we can pass them in as inputs to the task being run. Our tasks also need a `run` function so we can choose when to run tasks as running a task takes a long time. We can define a basic addition task in our `src/tasks/add.rs` file with the following code:

```
use std::{thread, time};
use serde::{Serialize, Deserialize};
```

More books: https://t.me/booksforits

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AddTask {
    pub one: i32,
    pub two: i32
}

impl AddTask {

    pub fn run(self) -> i32 {
        let duration = time::Duration::from_secs(20);
        thread::sleep(duration);
        return self.one + self.two
    }
}
```

None of this code should be a shock. We will implement the `Serialize` and `Deserialize` traits so we can insert the task into the Redis Queue. We can then use a `sleep` function to simulate a long task. Finally, we merely add the two numbers together. For our task in the `src/tasks/multiply.rs` file, the `run` function takes the following form:

```
impl MultiplyTask {

    pub fn run(self) -> i32 {
        let duration = time::Duration::from_secs(20);
        thread::sleep(duration);
        return self.one * self.two
    }
}
```

It should not be a surprise to find out that the `run` function in the `src/tasks/subtract.rs` file has the following structure:

```
impl SubtractTask {

    pub fn run(self) -> i32 {
        let duration = time::Duration::from_secs(20);
```

More books: https://t.me/booksforits

```
        thread::sleep(duration);
        return self.one - self.two
    }
}
```

Now, we want to implement one of our tasks to see whether we can pull a task struct out of a Redis queue and run it. We make the tasks accessible from the module with the following code in the `src/tasks/mod.rs` file:

```
pub mod add;
pub mod multiply;
pub mod subtract;
```

In our `src/main.rs` file, we initially import the tasks with the following code:

```
mod tasks;
use tasks::{
    add::AddTask,
    subtract::SubtractTask,
    multiply::MultiplyTask
};
```

We can now implement one of our tasks in our worker block of code. At the start of this worker block of code, we will swap the `IncomingBody` struct with the `AddTask` struct using the following code:

```
let body = AddTask{one: 1, two: 2};
```

Nothing else needs to change apart from what we do with the `Some` part of the `outcome match` statement, which now takes the following form:

```
let deserialized_struct: AddTask =
    bincode::deserialize(&data).unwrap();
println!("{:?}", deserialized_struct.run());
```

Here, we can see that we deserialized the binary data into an `AddTask` struct, ran the `run` function, and then printed out the outcome. In a real application, we would be inserting the result to a database or sending the result to another server using HTTP. However, in this chapter, we are merely interested in seeing how queuing tasks are executed. We have covered database inserts and HTTP requests many times in the book.

If we run our worker application now, we will get a 15-second delay and then the following printout:

```
3
```

More books: https://t.me/booksforits

If we wait another 15 seconds, we will get another printout that is the same. This shows that our tasks are being pulled from the Redis queue, deserialized, and then ran in the exact same manner that we expect them to as one added to two is three. However, there is a problem here. We can only send and receive the `AddTask` struct. This is not useful as we have two other tasks and we would like to support all of them. Therefore, we must move on to defining messages that can support a range of tasks.

Defining messages for the Redis queue

To support multiple tasks, we must do a two-step approach to packaging our tasks to be inserted into the Redis queue. This means that we will serialize the task struct into `Vec<u8>`, then add this vector of bytes to another struct that has a field denoting what type of task is in the message. We can define this process by first importing the `Serialize` and `Deserialize` traits in the `src/tasks/mod.rs` file with the following code:

```
use serde::{Serialize, Deserialize};
```

We can then define the `enum` task type and message struct with the following code:

```
#[derive(Debug, Clone, Serialize, Deserialize)]

use add::AddTask;
use multiply::MultiplyTask;
use subtract::SubtractTask;

#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum TaskType {
    ADD(AddTask),
    MULTIPLY(MultiplyTask),
    SUBTRACT(SubtractTask)
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TaskMessage {
    pub task: TaskType
}
```

Our message struct is now ready to package a range of tasks to be inserted into the Redis queue. In our `src/main.rs` file, we can import the `TaskType` and `TaskMessage` structs with the following code:

```
mod tasks;
```

More books: https://t.me/booksforits

```
use tasks::{
    add::AddTask,
    TaskType,
    TaskMessage
};
```

We are now ready to rewrite our infinite loop in the worker block of code. We initially create `AddTask`, serialize `AddTask`, and then package this serialized task into the `TaskMessage` with the following code:

```
let body = AddTask{one: 1, two: 2};
let message = TaskMessage{task: TaskType::ADD(body)};
let serialized_message = bincode::serialize(&message).unwrap();
```

We will then establish a Redis connection, then push our serialized message to the Redis queue with the following code:

```
let mut con = client.get_connection().unwrap();
let _ : () = redis::cmd("LPUSH").arg("some_queue")
                                .arg(serialized_message
                                .clone())
                                .query(&mut con).unwrap();
```

We will then pop the task from the Redis queue and drop the connection with the following code:

```
let outcome: Option<Vec<u8>> =
    redis::cmd("RPOP").arg("some_queue").query(&mut con)
    .unwrap();
std::mem::drop(con);
```

We are now moving our `TaskMessage` struct in and out of the Redis queue. We must process `TaskMessage` if there is one. Inside the `match` block of the `Some` statement of `outcome`, we must deserialize the bytes we got from the Redis queue, then match the task type with the following code:

```
let deserialized_message: TaskMessage =
    bincode::deserialize(&data).unwrap();

match deserialized_message.task {
    TaskType::ADD(task) => {
        println!("{:?}", task.run());
```

More books: https://t.me/booksforits

```
    },
    TaskType::MULTIPLY(task) => {
        println!("{:?}", task.run());
    },
    TaskType::SUBTRACT(task) => {
        println!("{:?}", task.run());
    }
}
```

This now enables us to handle individual tasks that we have pulled from the Redis queue and ran.

Our worker now supports all three of our tasks! However, we are currently just creating messages and then directly consuming these messages in the worker. We need to enable the HTTP server to accept a range of different requests to send a range of different tasks to the Redis queue to be consumed by the workers.

Integrating routing in the HTTP server

We are now at the stage of getting our HTTP server to accept incoming requests to create a range of tasks depending on what the URI is. To get our HTTP to support multiple tasks, we essentially must rewrite the `handle` function in the `src/main.rs` file. Before we rewrite the `main` function, we must import what we need with the following code:

```
use hyper::body;
use hyper::http::StatusCode;
```

We are importing these things because we are going to return a `NOT_FOUND` status code if the wrong URI is passed. We also going to be extracting data from the body of the incoming request. Before we refactor our `handle` function, we need to change our `IncomingBody` struct to take in two integers taking the following form:

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct IncomingBody {
    pub one: i32,
    pub two: i32
}
```

Inside our `handle` function, we can define our Redis client, clean our URI by removing trailing slashes, and extract the data from the incoming request with the following code:

```
let client =
    redis::Client::open("redis://127.0.0.1/").unwrap();
```

More books: https://t.me/booksforits

```
let task_type = req.uri().to_string().replace("/", "")"");
let body_bytes =
    body::to_bytes(req.into_body()).await.unwrap();
let body: IncomingBody =
    _json::from_slice(&body_bytes).unwrap();
```

We can see that we can extract the task type from the URI. Right now, we will support `add`, `subtract`, and `multiply`. We now have everything we need from the incoming request; we can construct the appropriate task based on the URI with the following code:

```
let message_type: TaskType;
match task_type.as_str() {
    "add" => {
        let body = AddTask{one: body.one,
                           two: body.two};
        message_type = TaskType::ADD(body);
    },
    "multiply" => {
        let body = MultiplyTask{one: body.one,
                                two: body.two};
        message_type = TaskType::MULTIPLY(body);
    },
    "subtract" => {
        let body = SubtractTask{one: body.one,
                                two: body.two};
        message_type = TaskType::SUBTRACT(body);
    },
    _ => {
        . . .
    }
}
```

We can see that no matter what the task is, we need the task struct to be packed into our `TaskType` enum, which can be serialized into a binary vector for our message to be sent to the Redis queue. For the last part of the `match` statement, which catches all task requests that do not match with "add", "multiply", or "subtract", we merely return a `NOT_FOUND` HTTP response with the following code:

```
let response =
    Response::builder().status(StatusCode::NOT_FOUND)
```

More books: https://t.me/booksforits

```
    .body(Body::from("task not found"));
return Ok(response.unwrap())
```

We now have everything we need to create a generic task message that can be inserted into a Redis queue. With this information, we can create our `TaskMessage` struct and serialize `TaskMessage` after the `match` statement that we have just covered with the following code:

```
let message = TaskMessage{task_type: message_type,
    task: bytes};
let serialized_message =
    bincode::serialize(&message).unwrap();
```

We will then make a Redis connection, push the serialized message to a Redis queue, and then drop the Redis connection with the following code:

```
let mut con = client.get_connection().unwrap();
let _ : () = redis::cmd("LPUSH").arg("some_queue")
                                .arg(serialized_message
                                .clone())
                                .query(&mut con).unwrap();
```

Finally, we return an `Ok` HTTP response stating that the task has been sent with the following code:

```
Ok(Response::new(Body::from("task sent")))
```

Our `handle` function is now complete. All we need to do now is remove the code that inserts an `AddTask` struct to the Redis queue in the worker code block. We are removing the task insertion code from the worker code block because we no longer need the worker to insert tasks. The removal of the insertion code takes the following form:

```
let client =
    redis::Client::open("redis://127.0.0.1/").unwrap();

loop {
    let outcome: Option<Vec<u8>> = {
        let mut con = client.get_connection()
                            .unwrap();
        redis::cmd("RPOP").arg("some_queue")
                          .query(&mut con)
                          .unwrap()
    };
```

More books: https://t.me/booksforits

```
    match outcome {
        . . .
    }
}
```

We are now ready to package these workers and HTTP servers in Docker so we can run our application with as many workers as we want.

Running it all in Docker

We are now at the stage where we can run our entire application in Docker. This enables us to have multiple workers pulling from the same Redis queue. First, we need to define the `Dockerfile` for the build of our worker/server image. We are going to have a distroless build for the Docker build with the following code:

```
FROM rust:1.62.1 as build
ENV PKG_CONFIG_ALLOW_CROSS=1

WORKDIR /app
COPY . .

cargo build --release
FROM gcr.io/distroless/cc-debian10

COPY --from=build /app/target/release/task_queue
/usr/local/bin/task_queue

EXPOSE 3000
ENTRYPOINT ["task_queue"]
```

This distroless build should not be a surprise at this point in the book. We are merely compiling the application and then copying the static binary into the distroless image. Before we run the build in any way, we must ensure that we do not copy over excessive files from the `target` directory into our Docker build with the following code in the `.dockerignore` file:

```
./target
.github
```

More books: https://t.me/booksforits

Our build is now ready. We can define the `docker-compose.yml` with the following outline:

```
version: "3.7"

services:
    server_1:
        . . .
    worker_1:
        . . .
    worker_2:
        . . .
    worker_3:
        . . .
    redis:
      container_name: 'queue-redis'
      image: 'redis'
      ports:
        - '6379:6379'
```

Here, we can see that we have three workers and one server. Our server takes the following form:

```
server_1:
    container_name: server_1
    image: server_1
    build:
      context: .
    environment:
      - 'APP_TYPE=server'
      - 'REDIS_URL=redis://redis:6379'
    depends_on:
        redis:
          condition: service_started
    restart: on-failure
    ports:
    - "3000:3000"
    expose:
      - 3000
```

More books: https://t.me/booksforits

Here, we can see that we can expose the port, point out that the build context is in the current directory, and that our container should start once Redis has started.

A standard worker takes the following form:

```
worker_1:
    container_name: worker_1
    image: worker_1
    build:
      context: .
    environment:
      - 'APP_TYPE=worker'
      - 'REDIS_URL=redis://redis:'
    depends_on:
        redis:
          condition: service_started
    restart: on-failure
```

We can imagine that other workers have the same structure as the preceding worker, which is true. If we want to add another worker, we can have the exact spec as `worker_1` except we just increase the number attached to the image and container name resulting in the new worker being called `worker_2`. You may have noticed that we have added `REDIS_URL` to the environment variables. This is because the workers and servers are having to access the Redis database outside of their container. Passing localhost into the Redis client will result in a failure to connect to Redis as a result. Therefore, we must get rid of all references to the Redis client and replace those references with the following code:

```
let client =
    redis::Client::open(env::var("REDIS_URL").unwrap())
    .unwrap();
```

If we spin up `docker_compose` now and send a range of different HTTP requests to the server, we get the following printout:

```
. . .
queue-redis  | 1:M 30 Oct 2022 18:42:52.334 *
RDB memory usage when created 0.85 Mb
queue-redis  | 1:M 30 Oct 2022 18:42:52.334 *
Done loading RDB, keys loaded: 0, keys expired: 0.
queue-redis  | 1:M 30 Oct 2022 18:42:52.334 *
DB loaded from disk: 0.002 seconds
```

More books: https://t.me/booksforits

```
queue-redis   | 1:M 30 Oct 2022 18:42:52.334 *
Ready to accept connections
worker_1      | empty queue
worker_3      | empty queue
worker_1      | empty queue
worker_3      | empty queue
worker_2      | multiply: 9
worker_3      | multiply: 25
worker_1      | multiply: 8
worker_3      | empty queue
worker_3      | empty queue
worker_2      | multiply: 4
worker_2      | empty queue
. . .
```

It is a big printout, but we can see that Redis spins up and there are multiple workers polling the Redis queue. We can also see that multiple workers are processing multiple tasks at the same time. Examples of how to make the request to the server are depicted here:

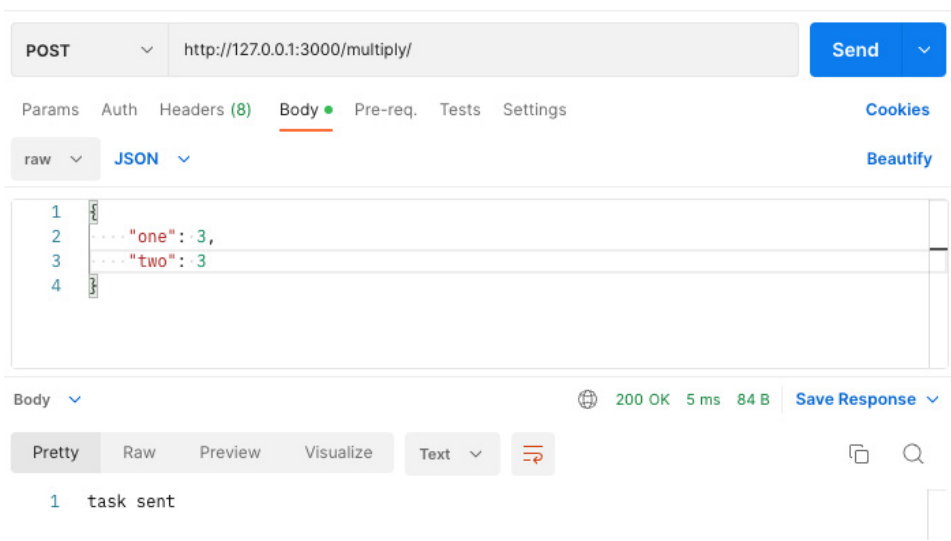


Figure 18.4 – An example of sending a request to our server for multiply

More books: https://t.me/booksforits

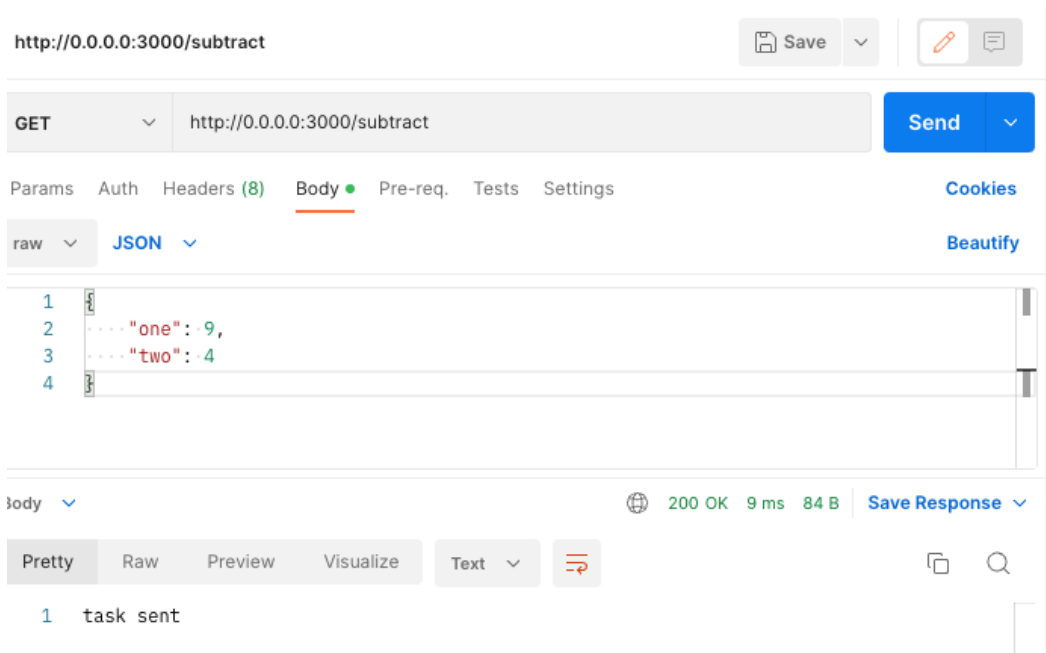


Figure 18.5 – An example of sending a request to our server for subtract

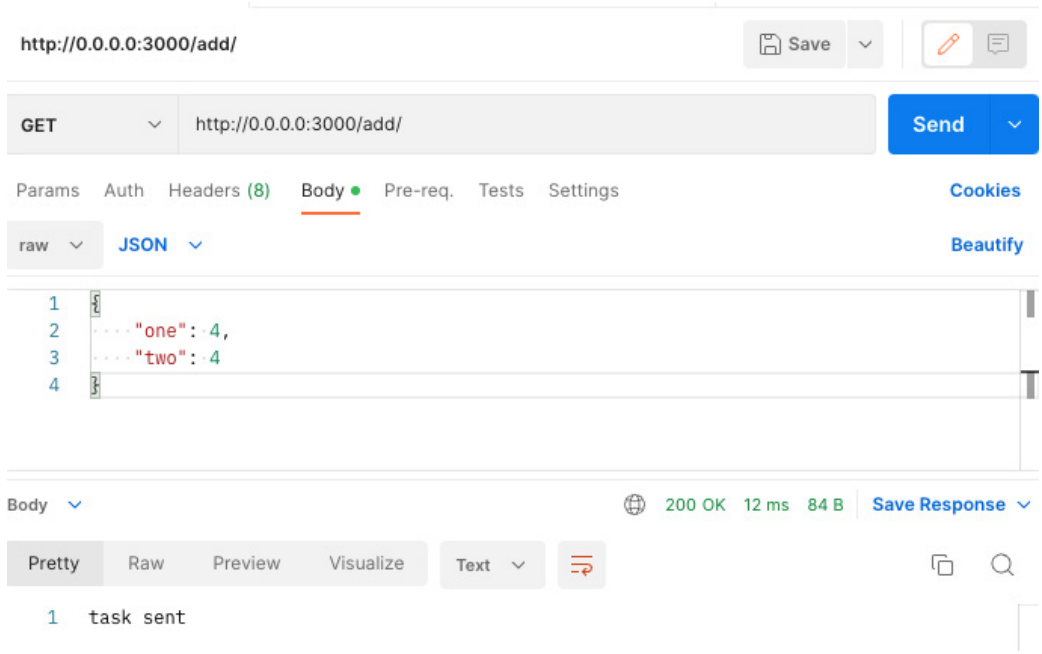


Figure 18.6 – An example of sending a request to our server for add

More books: https://t.me/booksforits

Here we have it! We have a server that accepts requests. Depending on the URI, our server constructs a task, packages it into a message, and then sends it to a Redis queue. We then have multiple workers polling the Redis queue to process the long tasks.

Summary

In this chapter, we built an application that could be run as either a worker or a server. We then built structs that could be serialized and inserted into a Redis queue. This allowed our workers to consume these tasks and then process them in their own time. You now have the power to build systems that process long tasks without having to hold up the HTTP server. The mechanism of serializing Rust structs and inserting them into Redis does not just stop at processing large tasks. We could serialize Rust structs and send them over pub/sub channels in Redis to other Rust servers, essentially creating an actor model approach on a bigger scale. With our distroless images, these Rust servers are only roughly the size of 50 MB, making this concept scalable. We also explored applying raw commands to Redis, which gives you the freedom and confidence to fully embrace what Redis has to offer. A high-level list of all the commands you can do to Redis is given in the *Further reading* section. You will be shocked at what you can do, and I hope you get as excited as me thinking of all the solutions you can achieve with Redis when looking through the available commands.

We have come to the end of the book. I am grateful that you have gotten this far, and I am always happy when readers reach out. Rust is truly a revolutionary programming language. With Rust, we have been able to build and deploy fast tiny servers. We have explored async programming and the actor model. We have built deployment pipelines. Your journey is not over; there is always more to learn. However, I hope that I have exposed you to fundamental concepts in such a way that you can go forward and read further documentation, practice, and someday push the boundaries of web programming.

Further reading

- The Redis documentation on pushing to queues: `https://redis.io/commands/lpush/`
- A concise list of raw Redis commands: `https://www.tutorialspoint.com/redis/redis_lists.htm`
- The Redis Rust crate documentation: `https://docs.rs/redis/latest/redis/`

More books: https://t.me/booksforits

Index

A

Actix Web framework 90
using, to manage views 112-116
Actix Web server
launching 90-92
actor async project
breaking down 582-584
actor model, Tokio framework
for async programming 523-526
actor reference 525
actors 525
communication, chaining between 555, 556
TCP, passing to 547-550
used, for tracking orders 550-555
actors, Tokio framework
working with 529-536
actor system 525
Amazon Resource Name (ARN) 445
American Standard Code for Information Interchange (ASCII) 416
API calls
making, in React 187-192
App component
custom components, constructing 197-199
custom components, managing 197-199
app, connecting to PostgreSQL 230-232
data models, creating 232-235
data, obtaining from database 235-237
application
configuring 243-245
running, on Redis 612, 613
application on AWS
HTTPS, enforcing 437, 438
application programming interface (API) 300
arrays
data, storing 13-16
asynchronous programming 97-100
async syntax 101-107
exploring, with web programming 108-112
attribute 334
authentication requirements
cleaning up 278-283
auth tokens
expiration, configuring 284-287
await syntax 101-107
exploring, with web programming 108-112
AWS
Rust application, deploying 398
Rust application, running 405-408
AWS client
setting up 374-377

More books: https://t.me/booksforits

AWS for EC2
reference link 378
AWS Secure Shell (SSH) key
setting up, for AWS EC2 instance 371-374
AWS Terraform modules
reference link 378

B

Bash
builds, orchestrating with 391, 392
Docker, running in background 223, 224
Bash deployment script
writing 383-387
basic HTML
reading, from files 158, 159
serving 156, 157
basic HTML loaded
serving, from files 159, 160
binary protocol 416
Bugsnag 317
build environment
setting up 370
builds
orchestrating, with Bash 391, 392
bytes
processing 544-546
processing, with structs 567

C

caching 320-324
Cargo
building with 49, 50
documenting with 51-54
interacting with 54-58
software project, managing with 48
used, for shipping crates 50, 51
certificates
obtaining, for URL 438-440
channel messages
defining 584-586
channels, Tokio framework
working with 526-529
clean test pipeline
building 501-506
Clean Web App Repository
layout 484-487
client
rewriting, to support framing 572-574
closures 92-97
cluster 525
code
structuring 59
code on demand 324
cold data 301
commands
sending, via client 559-561
components
inheriting 182-184
concurrency 320
continuous integration (CI) 382
configuring, through GitHub Actions 506-510
crates 51
shipping, with Cargo 50
Cross-Origin Resource Sharing (CORS) 190
CSS
lifting, into React 200-202
serving, with Rust 156
CSS, injecting into HTML 174
base CSS, creating 175-178
CSS, creating for home page 178-180
CSS, serving from Rust 180, 181
JavaScript, serving from Rust 180, 181
tags, adding to HTML 174, 175

More books: https://t.me/booksforits

custom components
constructing, in App component 197-199
creating, in React 192-197
managing, in App component 197-199

D

data
deleting 240-242
extracting from header in requests 148-150
extracting, from views 143-145
mapping, with HashMaps 16-19
storing, in arrays 13-16
storing, in vectors 13-16
database
editing 239, 240
need for 214
to-do item, inserting into 237, 238
database connection pool 245
building 245-248
data serializing 567
deployed application on AWS
URL, attaching to 425, 426
dereference operator 28
desktop application
React application, converting into 203-208
dictionaries 16
Diesel
used, for connecting to PostgreSQL 225-230
distributed denial-of-service (DDoS) attacks 442
distroless tiny server Docker images
building 498-501
DNS record
attributes 433
Docker
database, running 217, 218
entire application, running 621-626
need for 214
ports 219-222
routing 219-222
running in background, with Bash scripts 223, 224
used, for running database 215-217
using, to manage software 387
docker-compose
used, for implementing HTTPS locally 419-425
Docker containers 222
Dockerfile 216
Docker Hub 215
images, deploying 395-398
Docker image files
writing 387, 388
writing, for React frontend application 392-394
Docker images
building 388-390
domain name
registering 431-437
Domain Name System (DNS) 425
DNS records 425
domain registrar 425
zone file 425

E

EC2 build server
building, with Terraform 390
Elastic Compute Cloud (EC2) 415
Elastic Compute Cloud (EC2) instance
AWS SSH key, setting up 371-374
elastic IP addresses
attaching, to server 426-431
Electron 203-207
in applications area 208
React application running 207

More books: https://t.me/booksforits

entire automated testing pipeline
building 360-365
environment
interacting with 71, 72
environment variables
configuration, obtaining 487-491
errors
buffer overrun 21
dangling pointers 20
double frees 21
handling 19, 20
segmentation faults 21
use after frees 20
existing modules
plugging in 463-467
existing test pipeline
plugging in 480, 481
existing views
plugging in 472

F

factories
structs, managing with 65-67
fighting the borrow checker 20
floats
using 10, 12, 13
framing
client, rewriting to support 572-574
server, rewriting to support 574-576
utilizing 571
frontend authentication
adding 287-294
fusing code
initial setup 120-122

G

garbage collection 4, 5
GitHub Actions
continuous integration, configuring through 506-510

H

HashMaps 14
used, for mapping data 16-19
header extraction
simplifying, with traits 150-153
hot data 300
HTML
CSS, injecting into 174
JavaScript, injecting into 163
serving, with Rust 156
HTML file
JavaScript, adding to 160
HTTP/2 protocol
binary protocol 416
compresses headers 416
multiplex streaming 417-419
persistent connections 417
HTTP frame
building, with TCP 576-579
HTTP request
test, creating 353-355
HTTPS 416
enforcing, on application on AWS 437, 438
implementing, locally with docker-compose 419-425
HTTP server
building 606-609
routing, integration 618-621
Hyper framework 581
used, for building HTTP server 596, 597
used, for handling HTTP requests 593-596

More books: https://t.me/booksforits

Hyper HTTP server
running 598-601
Hypertext Transfer Protocol (HTTP) 371

I

images
deploying, on Docker Hub 395-398
immutable borrow 24-27
infrastructure as code (IaC) 377
inner scope 28, 29
integers
using 10-13
internet protocol (IP) 540

J

JavaScript
adding, to HTML file 160
serving, with Rust 156
used, for communicating server 160-162
JavaScript, injecting into HTML 163
API call function, building 169, 170
delete endpoint, adding 163-166
functions for buttons, building 171-173
JavaScript loading function, adding 166
render function, building 167-169
tags, adding 166, 167
JSON
accepting 472-474
extracting from body of request 145-148
returning 472-474
status, returning with 475, 476
JSON files 214
reading 72-75
writing 72-75
JSON Web Token (JWT) 150, 300
JSX 186
JWT unit tests
building 336
building, for web requests 342-344
configuration, building 337
function tests, building 340-342
requirements, defining 338-340

K

Kubernetes 222

L

layered system
mapping 300-305
load balancer
creating, for traffic 442-445
URL, attaching to 450-453
local development database
migrations tool, using to setting up 491
setting up 491-495
login view
JSON, accepting 472-474
JSON, returning 472-474
logout view
raw HTML, returning 474
logs
error 317
informational (info) 316
verbose 316
warning 316

M

macros, for JSON serialization 129-131
custom serialized struct, packaging to be returned to users 141-143
serialization struct, building 131-133

More books: https://t.me/booksforits

serialization structs, integrating into application code 136-141
Serialize trait, implementing 133-136
memory safety 4
message
processing, in server 569-571
message sender client
creating 568, 569
metaprogramming 41
with macros 41-43
middleman attack 417
migration scripts
creating, on database 261-268
running, on database 261-268
MPSC (multiple producers, single consumer channel) 526
multi-factor authentication (MFA) 375
multiple EC2 instances
creating 441
Python deployment script, updating 450
multiple statuses
returning 476, 477
mutable borrow 27, 28

N

Newman 355
used, for automating Postman tests 355-359
NewUser data model
creating 253-257
NGINX 316

O

object-relational mappers (ORMs) 235
OpenSSL, installing in Ubuntu
reference link 422
OpenSSL, installing in Windows
reference link 422
OpenSSL library, installing on macOS
reference link 422
outer scope 28, 29

P

parameters
passing, into views 122-129
polling worker
building 609-612
ports, Docker 219-222
PostgreSQL 252, 311
building 214
connecting to, with Diesel 225-230
Postman 336
Postman tests
automating, with Newman 355-359
variables, managing 495-497
process 97
Python application build script
writing 382, 383
Python deployment script
updating, for multiple EC2 instances 450

Q

queuing system
breaking down 604-606
infrastructure 605

R

raw HTML
returning 474
React
API calls, making 187-192

More books: https://t.me/booksforits

CSS, lifting into 200-202
custom components, creating 192-197
React application
converting, into desktop application 203-208
creating 185, 186
React frontend application
Docker image files, writing for 392-394
record types 433
Redis 311
application, running on 612, 613
Redis queue
messages, defining 616, 617
tasks for workers, defining 613-616
representational state transfer (REST) 300
RESTful services 300
caching 300
code on demand 300
layered system 300
logging 300
statelessness 300
uniform system 300
results
handling 19, 20
Rocket 460
advantages 460
views, registering with 477-479
Rocket traits
implementing 467-472
routing, Docker 219-222
runner actor
building 586-588
Rust
CSS, serving from 180, 181
data types, reviewing 6, 7
JavaScript, serving from 180, 181
strings, using 8-10
used, for serving CSS 156
used, for serving HTML 156
used, for serving JavaScript 156
variables, reviewing 6, 7
Rust application
build script, writing 408-413
deploying, on AWS 398
running, locally 399-404
running, on AWS 405-408
Rust revolution
need for 4-6

S

salting concept 255
schema file
updating 260, 261
security groups
creating, to lock down 445-450
creating, to secure traffic 445-450
server
communicating, with JavaScript 160-162
elastic IP addresses, attaching to 426-431
message, processing 569-571
rewriting, to support framing 574-576
setting up 460-463
server traffic
logging 316-320
Simple Storage Service (S3) bucket 386
software
managing, with Docker 387
software project
managing, with Cargo 48
Stack Overflow solution 109, 110
reference link 109
state actor
building 588-593
statelessness
implementing 309-316
status
returning, with JSON 475, 476

More books: https://t.me/booksforits

str 8
strings
using, in Rust 8-10
string slice 8
structs
building 33-38
managing, with factories 65-67
processing 78-84
used, for processing bytes 567
verifying, with traits 38-41

T

TCP client
setting up 564-567
TCP server
setting up 564-566
Terraform 386
used, for building EC2 build server 390
Terraform build
setting up 377-381
tests, in Postman
creating, for HTTP request 353-355
ordered requests, writing 348-352
writing 345-348
thread 97
three-way handshake
ACK 540
SYN 540
SYN-ACK 540
to-do item
inserting, into database 237, 238
ToDoItem component
creating 193-195
functions 193, 194
to-do item data model
altering 257-260
to-do structs
building 59-65
Tokio crate 111
Tokio framework 515
actor model, for async programming 523-526
actors, working with 529-536
channels, working with 526-529
for async programming 516-520
workers, working with 520-522
traits 75-78
functionality, defining with 67-71
processing 78-84
verifying with 38-41
transmission control protocol (TCP)
accepting 541-543
exploring 540, 541
passing, to actor 547-550
responding with 557, 558
used, for building HTTP frame 576-579
transport layer security/secure sockets layer (TLS/SSL) 540

U

uniform interface
building 305-309
unit tests
building 332-336
Universal Resource Identifiers (URIs) 581
URL
attaching, to deployed application on AWS 425, 426
attaching, to load balancer 450-453
certificates, obtaining 438-440
user authentication 268-274
user data module
creating 252, 253

More books: https://t.me/booksforits

user model
creating 252
migration scripts, creating on database 261-268
migration scripts, running on database 261-268
NewUser data model, creating 253-257
schema file, updating 260, 261
to-do item data model, altering 257-260
User data module, creating 252, 253
user sessions
managing 275-278

V

variable ownership
controlling 20, 21
variables
copying 21, 22
immutable borrowing 24-27
moving 22-24
mutable borrowing 27, 28
running, through lifetimes 30-32
scopes 28-30
vectors
data, storing 13-16
views
data, extracting from 143-145
managing, with Actix Web framework 112-116
parameters, passing into 122-129
registering, with Rocket 477-479
virtual private cloud (VPC) 442

W

web programming
async syntax, exploring with 108-112
await syntax, exploring with 108-112
web requests
tests, building 342-344
workers, Tokio framework
working with 520-522

Y

YAML code 216

More books: https://t.me/booksforits

More books: https://t.me/booksforits



www.packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

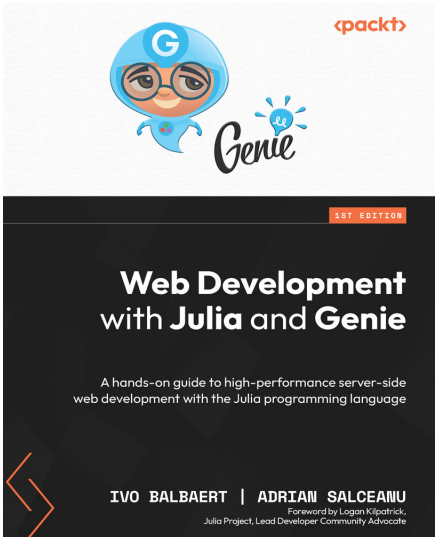
Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

More books: https://t.me/booksforits

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



Web Development with Julia and Genie

Ivo Balbaert , Adrian Salceanu

ISBN: 978-1-80181-113-2

- Understand how to make a web server with HTTP.jl and work with JSON data over the web
- Discover how to build a static website with the Franklin framework
- Explore Julia web development frameworks and work with them
- Uncover the Julia infrastructure for development, testing, package management, and deployment
- Develop an MVC web app with the Genie framework
- Understand how to add a REST API to a web app
- Create an interactive data dashboard with charts and filters
- Test, document, and deploy maintainable web applications using Julia

More books: https://t.me/booksforits



Rust Web Development with Rocket

Karuna Murti

ISBN: 978-1-80056-130-4

- Master the basics of Rust, such as its syntax, packages, and tools
- Get to grips with Rocket's tooling and ecosystem
- Extend your Rocket applications using Rust and third-party libraries
- Create a full-fledged web app with Rocket that handles user content
- Write pattern-matching logic and handle Rust object lifetimes
- Use APIs and async programming to make your apps secure and reliable
- Test your Rocket application and deploy it to production
- Containerize and scale your applications for maximum efficiency

More books: https://t.me/booksforits

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packtpub.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *Rust Web Programming - Second Edition*, we'd love to hear your thoughts! If you purchased the book from Amazon, please click here to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

More books: https://t.me/booksforits

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere? Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



`https://packt.link/free-ebook/9781803234694`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

More books: https://t.me/booksforits